УЧЕБНОЕ ПОСОБИЕ

Программирование дронов и создание автономных беспилотных миссий

Курс «Программные средства БПЛА»

Lua · Python · Bricks · SDK2 и RKNN · ROS 2 и рой · Симулятор



От первой строки кода до автономной миссии: язык, светодиоды и таймеры, события автопилота, полёт по точкам, камера и навигация по маркерам, нейросети на борту Пионера Мини 2, ROS 2 и групповой полёт. Каждый пример запускается в симуляторе и проверяется автоматически.

Корягин Сергей Андреевич
асп. СПбГЭТУ «ЛЭТИ», педагог ЦО «Кудрово»

г. Кудрово, 2026
simulator.sakoryagin.ru
Версия 1.1.119 · сборка 12.11.2026 01:22

Корягин С. А.

Программирование дронов и создание автономных беспилотных миссий: учебное пособие по курсу «Программные средства БПЛА». — Кудрово, 2026. — 432 с.

Аннотация

Пособие вводит в программирование образовательных квадрокоптеров «Геоскан Пионер»: Пионера Базового (Lua и Python, библиотека `pioneer_sdk`) и Пионера Мини 2 (Python на борту, библиотеки `pioneer_sdk2`, `pioneer_rknn`, ROS 2 и `pioneer_swarm`, блоки Pioneer Bricks). Первая часть — язык Lua, светодиодная индикация, таймеры и событийная модель автопилота, автономные миссии по точкам и параметрическим траекториям. Вторая часть — Python: подключение, полёт, камера, визуальные маркеры ArUco, трекинг и групповой полёт Пионера Базового. Третья часть — профессиональная навигация: калибровка камеры, карты маркеров, фильтрация измерений. Четвёртая часть — Пионер Мини 2: программы на борту, нейросети на его нейропроцессоре и блочное программирование Bricks. Пятая часть — ROS 2 и рой на Пионере: пакет `gs_core` (библиотека `rclpy`: сервисы полёта, топики, параметры, узел-миссия) и Swarm Kit (библиотека `pioneer_swarm`: соседи, сообщения, телеметрия, строй дронов). Программы запускаются в веб-симуляторе `simulator.sakoryagin.ru`: в нём есть обе модели дрона и режим «**Обучение**» с пятью треками миссий и автоматической проверкой полёта. API симулятора совпадает с API настоящих дронов, а отличия названы в тексте.

Для кого

Для школьников и студентов, начинающих программировать роботов, и педагогов дополнительного образования. Предварительной подготовки по программированию не требуется; математика — в объёме школьной тригонометрии.

Примеры и проверка

Исходный код всех примеров — в папке `examples/` рядом с книгой; каждый файл подписан в тексте. Примеры для Lua и `pioneer_sdk` автоматически прогоняются в симуляторе: взлетают, проходят точки и садятся. Примеры для Пионера Мини 2 (`examples/python-onboard/`) рассчитаны на борт дрона: их автоматически проверяют на имитации `pioneer_sdk2` и `pioneer_rknn` (по следу команд), примеры ROS 2 — на эмуляции `rclpy` и узла `ros_pioneer_node`, а проекты Bricks сверяют с Python, который сгенерировала сама среда. Примеры роя проверяют на симуляторе с несколькими дронами. Полётные программы Мини 2, ROS 2 и роя на настоящем дроне авторами не запускались. Текст примеров можно копировать прямо из PDF.

Версия 1.1.119 · сборка 12.11.2026 01:22

Содержание

1 Введение: как устроен курс . . . 10

2 Подготовка: где писать и запускать код . . . 13

2.1 Два дрона и два способа запуска . . . 13
2.2 Симулятор: что где находится . . . 14
2.3 Редактор кода . . . 15
2.4 Остальные панели рельса . . . 18
2.5 Режим «Обучение»: книга и симулятор вместе . . . 19
2.6 Первая программа . . . 24
2.7 Настоящие дроны . . . 25

I Программирование на Lua 28

3 Язык Lua . . . 29

3.1 Зачем Lua . . . 29
3.2 Толковый словарь . . . 30
3.3 Переменные и типы . . . 30
3.4 Арифметика и сравнения . . . 33
3.5 Логические операции . . . 34
3.6 Условия . . . 35
3.7 Циклы . . . 37
3.8 Функции . . . 38
3.9 Типичные ошибки и их сообщения . . . 40
3.10 Проверьте себя . . . 41

4 Светодиоды и таймеры . . . 43

4.1 Зачем дрону светодиоды . . . 43
4.2 Толковый словарь . . . 44
4.3 Аппаратная часть: 4 + 25 светодиодов . . . 44
4.4 Модуль Ledbar . . . 46
4.5 Время: почему не sleep . . . 48
4.6 Таймеры . . . 49
4.7 События автопилота . . . 51
4.8 Три способа организовать программу . . . 56
4.9 Примеры комплексных сценариев . . . 57
4.10 Проверьте себя . . . 59
4.11 Практика: проекты подсветки . . . 61

5 Автономные миссии . . . 63

5.1 Что такое миссия . . . 64
5.2 Толковый словарь . . . 64
5.3 Жизненный цикл миссии . . . 64
5.4 Координаты: где точка . . . 66

5.5 Ступени полёта . . . 68
5.6 Полигоны и местность . . . 74
5.7 Измерения: цель, положение, допуск . . . 75
5.8 Окружность: точки или таймер . . . 77
5.9 Архитектура миссии: конечный автомат . . . 80
5.10 Частые ошибки . . . 87
5.11 Проверьте себя . . . 88
5.12 Практика: проекты миссий . . . 90

II Программирование на Python 92

6 Основы Python . . . 93
6.1 Что такое Python и зачем он нужен? . . . 93
6.2 Переменные и типы данных . . . 95
6.3 Арифметические операторы . . . 96
6.4 Логические операции . . . 98
6.5 Ветвление (Условные операторы) . . . 99
6.6 Циклы . . . 100
6.7 Функции . . . 102

7 Pioneer SDK: полёт из Python . . . 105
7.1 Где запускать Python . . . 106
7.2 Подключение и телеметрия . . . 108
7.3 Светодиоды из Python . . . 109
7.4 Паузы, подтверждения и тайм-ауты . . . 111
7.5 Первый полёт: моторы, взлёт, посадка . . . 113
7.6 Полёт по точкам . . . 115
7.7 Полигоны и местность . . . 119
7.8 Измерения: положение, ошибка, допуск . . . 120
7.9 Управление скоростью . . . 121
7.10 Контрольные упражнения с проверкой . . . 123
7.11 Проект главы . . . 126

8 Камера, компьютерное зрение и рой . . . 128
8.1 Камера с нуля . . . 129
8.2 ArUco: словарь и детекция . . . 131
8.3 Геометрия метки: углы, центр, ошибка . . . 133
8.4 Расстояние до метки: solvePnP . . . 136
8.5 ArUco на настоящем дроне . . . 140
8.6 Наведение по метке . . . 148
8.7 Следование за меткой (только на дроне) . . . 151
8.8 Продвинутая работа с камерой (только на дроне) . . . 156
8.9 Датчики в полёте и журнал . . . 157
8.10 Трекинг человека и жестов (только на дроне) . . . 159
8.11 Групповой полет (только на дронах) . . . 160

9 Итоговые проекты: самостоятельные миссии . . . 163
9.1 Метки как карта . . . 164
9.2 Поиск по секторам . . . 164
9.3 Экспедиции по полигонам . . . 164
9.4 Потеря цели и незнакомая карта . . . 165
9.5 Итоговая экспедиция . . . 166
9.6 Проекты для настоящего дрона . . . 167

III Профессиональные навигационные системы 168

10 Теоретические основы визуальной навигации . . . 169
10.1 ArUco и AprilTag . . . 169
10.2 Как детектор находит метку . . . 170
10.3 Коррекция ошибок и расстояние Хэмминга . . . 173
10.4 От пикселей к метрам: модель камеры и PnP . . . 174
10.5 Практика: словари и освещение . . . 176

11 Создание навигационных карт . . . 177
11.1 Калибровка камеры . . . 177
11.2 Генерация и печать маркеров . . . 181
11.3 Форматы карт (JSON, XML, KML) . . . 187
11.4 Визуализация и планирование . . . 194

12 Алгоритмы комплексной навигации . . . 195
12.1 Проблема шума и фильтрация . . . 195
12.2 Системы координат . . . 197
12.3 Точная посадка с PID-регулятором . . . 198

13 Техническая реализация программных модулей . . . 202
13.1 Генератор карт (Marker Map Builder) . . . 202
13.2 Модуль визуализации . . . 205

14 Банк практических заданий . . . 206
14.1 Блок 1: Работа с камерой и ArUco . . . 206
14.2 Блок 2: Телеметрия и безопасность . . . 208
14.3 Блок 3: Миссии и полет по точкам . . . 209
14.4 Блок 4: Групповой полет . . . 210
14.5 Блок 5: Трекинг объектов . . . 210
14.6 Самостоятельные миссии симулятора . . . 211
14.7 Критерии оценки выполнения . . . 212

15 Типичные ошибки и их устранение . . . 213
15.1 Проблемы с камерой и маркерами . . . 213
15.2 Проблемы с подключением и Wi-Fi . . . 214
15.3 Ошибки в коде Python . . . 214
15.4 Особенности симулятора . . . 215
15.5 Аварийные ситуации . . . 216

IV Пионер Мини 2: Python на борту, нейросети и Bricks 217

16 Pioneer SDK 2: программа на борту дрона . . . 218

16.1 Программа переезжает на дрон . . . 219
16.2 Что входит в библиотеку . . . 222
16.3 Что внутри борта . . . 222
16.4 Подключение и жизненный цикл . . . 224
16.5 Взлёт, посадка и порядок команд . . . 227
16.6 Системы координат и единицы . . . 233
16.7 Полёт по точкам . . . 234
16.8 Курс и возврат домой . . . 240
16.9 Глобальные точки по GPS . . . 241
16.10 События: подписка вместо опроса . . . 242
16.11 Полёт со скоростью . . . 247
16.12 Датчики дрона . . . 250
16.13 Навигация: LPS, GPS и оптика . . . 252
16.14 Журнал полёта в CSV . . . 255
16.15 Параметры автопилота . . . 257
16.16 Подсветка . . . 260
16.17 Груз: магнитный захват и механический захват . . . 262
16.18 Пульт RC . . . 264
16.19 Камера и видео в браузере . . . 269
16.20 Наклон камеры: сервопривод . . . 274
16.21 Запись видео и фото: RecorderControl . . . 277
16.22 Компьютерное зрение в контуре управления . . . 279
16.23 Итоговый проект: патрулирование . . . 287
16.24 Справочник команд . . . 290
16.25 Типичные ошибки . . . 294
16.26 Карта примеров: симулятор или дрон . . . 295
16.27 Проверьте себя . . . 296
16.28 Задания к главе . . . 298
16.29 Миссии симулятора по разделам главы . . . 299

17 Pioneer RKNN: нейросети на борту . . . 301

17.1 Нейросеть и нейропроцессор . . . 301
17.2 Симулятор и дрон . . . 302
17.3 Какие модели есть на дроне . . . 303
17.4 Поиск объектов: Yolo . . . 304
17.5 Маски объектов: YoloSeg . . . 307
17.6 Поза человека: YoloPose . . . 308
17.7 Чтение текста: PaddleOCR . . . 309
17.8 Скорость и ресурсы . . . 311
17.9 Проект: найти человека . . . 311
17.10 Своя модель . . . 313
17.11 Задания к главе . . . 314

18 Блоки Pioneer Bricks: программа без кода . . . 316
18.1 Что такое Pioneer Bricks . . . 317
18.2 Проекты и редактор . . . 319
18.3 Блоки по категориям . . . 321
18.4 Во что превращаются блоки . . . 321
18.5 Безопасное завершение . . . 324
18.6 Десять проектов . . . 325
18.7 Особенности и ограничения Bricks 2.3.0 . . . 335
18.8 Блоки и Python: что с чем сравнивать . . . 336

V ROS 2 и рой на Пионере 339

19 ROS 2 на Пионере: пакет gs_core . . . 340
19.1 Что такое ROS 2 . . . 341
19.2 Зачем ROS 2 на Пионере . . . 342
19.3 Узел ros_pioneer_node . . . 342
19.4 Консоль ros2 и первый узел . . . 344
19.5 Сервисы полёта: полётный цикл . . . 345
19.6 Системы координат map и base_link . . . 348
19.7 Отказ сервиса и возврат домой . . . 350
19.8 Топики и телеметрия . . . 351
19.9 События автопилота . . . 354
19.10 Датчики: IMU, дальномер, Ranger . . . 355
19.11 Управление скоростью: cmd_vel . . . 357
19.12 Виртуальный пульт: rc/set_channels . . . 359
19.13 Параметры: автопилота и узла . . . 361
19.14 Система навигации . . . 365
19.15 Полезная нагрузка: светодиоды и захват . . . 366
19.16 Итоговый проект: миссия-узел . . . 369
19.17 Справочник gs_core . . . 372
19.18 Проверьте себя . . . 373
19.19 Задания к главе . . . 375
19.20 Миссии симулятора по разделам главы . . . 375

20 Рой: Swarm Kit и pioneer_swarm . . . 377
20.1 Рой без центра . . . 378
20.2 Библиотека pioneer_swarm . . . 380
20.3 Кто в сети . . . 381
20.4 Сообщения: ящик и опрос . . . 382
20.5 Телеметрия соседей . . . 386
20.6 Ведущий и ведомый . . . 389
20.7 Строй: у каждого своё место . . . 391
20.8 Синхронный взлёт по готовности . . . 393
20.9 Световое шоу . . . 395
20.10 Итоговый проект: поиск и спасение . . . 397

20.11 Проверьте себя . . . 400
20.12 Задания к главе . . . 401
20.13 Миссии симулятора по разделам главы . . . 402

VI Приложения **403**

A Справочник Lua API Пионера . . . **404**
A.1 Команды автопилота: `ap` . . . 404
A.2 События: `Ev` в `callback(event)` . . . 404
A.3 Время и таймеры . . . 405
A.4 Светодиоды: `Ledbar` . . . 405
A.5 Датчики: `Sensors` . . . 406
A.6 Камера: `camera` . . . 406
A.7 Прочее . . . 406
A.8 Идиомы курса . . . 407

B Шпаргалка: Lua, Python и pioneer_sdk2 . . . **408**

C Протокол Plazlink и поля автопилота . . . **411**

D Аппаратура управления . . . **417**
D.1 Настройка пульта . . . 417
D.2 Связь пульта с приемником . . . 419
D.3 Стики и тумблеры . . . 421
D.4 Управление . . . 422
D.5 Пульт в симуляторе . . . 425

E Методика тестирования и безопасность . . . **428**
E.1 Протокол безопасности полетов . . . 428
E.2 Тестирование в разных условиях . . . 429
E.3 Проверка в симуляторе перед полётом . . . 429

F Готовые конфигурации (Templates) . . . **430**
F.1 Файл конфигурации camera_params.yaml . . . 430
F.2 Шаблон скрипта полета (main.py) . . . 430

Список литературы . . . **432**

ГЛАВА 1

Введение: как устроен курс

О чём эта книга

Эта книга учит программировать квадрокоптеры «Геоскан Пионер». Их два, и программируются они по-разному:

	Пионер Базовый	Пионер Мини 2
Языки	Lua (на борту), Python (на компьютере)	Python (на борту), блоки Pioneer Bricks
Библиотеки	`pioneer_sdk`	`pioneer_sdk2`, `pioneer_rknn`, `rclpy` (ROS 2), `pioneer_swarm`
Нейросети	нет	нейропроцессор на борту
Рой дронов	с компьютера (глава 8.11)	Swarm Kit на борту (часть V)
Где в книге	части I–III	части IV и V

Базовый дрон — основа курса: на нём от первой строчки на Lua идёт путь до автономной миссии, в которой дрон сам взлетает, облетает точки, реагирует на нештатные ситуации и садится. Та же программа на Python выходит за пределы симулятора — к камере, маркерам и групповым полётам. Пионер Мини 2 — следующая ступень: программа работает на самом дроне, а рядом есть нейропроцессор. Для него же написаны две заключительные части курса: ROS 2 (пакет `gs_core`), в котором дрон устроен как набор узлов, и Swarm Kit — рой, где дроны находят друг друга и работают вместе без центра. Эти главы лучше читать после основ Python.

Главный принцип курса — **каждый пример запускается**. Программы для Lua и `pioneer_sdk` лежат в папке `examples/` рядом с текстом и автоматически проверяются в симуляторе: взлетают, пролетают свои точки и садятся. Если такой пример не совпадает с тем, что вы видите на экране, — это ошибка книги, а не ваша. Примеры Мини 2 (`examples/python-onboard/`) написаны для борта дрона: их проверяют на имитации библиотек по следу команд (примеры ROS 2 и роя — на эмуляции `rclpy` и `pioneer_swarm` в симуляторе), а какие из них идут и в симуляторе, сказано в самих главах. На настоящем дроне полётные программы Мини 2, ROS 2 и роя авторы не запускали.

Где писать код

Для занятий нужен только браузер: веб-симулятор `https://simulator.sakoryagin.ru` показывает полёт в 3D, умеет Lua, Python и блоки, содержит обе модели дрона и использует те же API, что настоящие Пионеры. Настоящий дрон понадобится ближе к концу

курса — для камеры, компьютерного зрения и бортовых программ. Как запускать программы в симуляторе и на дронах, объясняет глава 2.

Как читать главы

Каждая глава построена одинаково:

1. **Цели** — что вы будете уметь после главы.
2. **Миссии симулятора** — зелёная врезка: какие миссии режима «**Обучение**» закрепляют тему главы (раздел 2.5).
3. **Уроки** — короткая теория с примерами: программы, которые нужно запустить и сверить результат с комментариями в коде.
4. **Задание** — рамка после каждого урока: что сделать и что должно получиться. Проверять результат можно прямо в симуляторе.
5. **Проверьте себя** — вопросы с ответами.
6. **Итоги** — что запомнить.

Рисунок 1 показывает этот цикл по шагам.

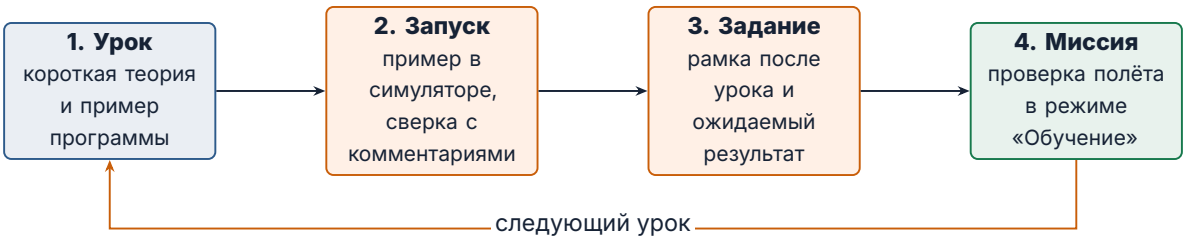


Рис. 1. Рабочий цикл по главе: прочитать урок, запустить пример, выполнить задание, пройти миссию симулятора и перейти к следующему уроку. Главу закрывают «Проверьте себя» и «Итоги».

В тексте встречаются рамки:

- **Информация** — пояснение или ссылка на другой раздел;
- **Внимание** — типичная ошибка или вопрос безопасности;
- **Подсказка** — приём, который экономит время;
- **Правило** — соглашение, которого курс придерживается всегда;
- **Задание** — работа для самостоятельного выполнения;
- **Миссии симулятора** — связь главы с режимом «**Обучение**».

Дорожная карта

1. **Подготовка.** Интерфейс симулятора, режим «**Обучение**», первая программа, настоящие дроны и техника безопасности.
2. **Часть I. Lua** (Пионер Базовый). Язык (переменные, условия, циклы, функции) → светодиоды и таймеры → события автопилота и автономные миссии.
3. **Часть II. Python.** Основы языка → `pioneer_sdk` (Базовый): подключение, светодиоды, взлёт, полёт по точкам → камера, маркеры ArUco, трекинг, рой.
4. **Часть III. Профессиональная навигация.** Теория визуальных маркеров, калибровка камеры, карты, фильтрация и точная посадка (на `pioneer_sdk`).
5. **Часть IV. Пионер Мини 2.** `pioneer_sdk2` (глава 16), нейросети `pioneer_rknn` (глава 17) и блоки Pioneer Bricks (глава 18).
6. **Часть V. ROS 2 и рой на Пионере.** Узлы, топики и сервисы `gs_core` (глава 19): полётный цикл, системы координат `map` и `base_link`, телеметрия, `cmd_vel`, параметры, нагрузка, миссия-узел → Swarm Kit (глава 20): соседи, сообщения, телеметрия соседей, строй, групповой проект.
7. **Приложения.** Справочник Lua API, шпаргалка по языкам и библиотекам (включая ROS 2 и рой), настройка пульта, протокол безопасности.

Lua и Python в курсе идут в одном порядке: сначала язык, потом светодиоды, потом полёт. Поэтому часть про Python можно читать и отдельно, но полезнее — после Lua: событийная модель автопилота, разобранная на Lua, объясняет, почему Python-программа «ждёт точку» именно так. Главы про Мини 2 опираются на Python-основы и на полёт по точкам из `pioneer_sdk`; блоки Bricks можно начать и раньше, не дожидаясь текстового Python. Часть V строится на главе 16: полёт `pioneer_sdk2` объясняет, что делают сервисы `gs_core`, а рой дополняет их общением между дронами.

Планируемые результаты

После курса вы сможете:

- писать читаемый код на Lua и Python и объяснять, что делает каждая строка;
- управлять бортовой периферией — светодиодами и таймерами — без блокировки программы;
- строить программу полёта как конечный автомат: команда → событие → следующая команда;
- предусматривать аварийные ситуации: удар, разряд батареи, потеря связи;
- переносить программу из симулятора на настоящий дрон, учитывая отличия моделей (Lua и `pioneer_sdk` у Базового, `pioneer_sdk2` у Мини 2);
- подключить к полёту компьютерное зрение и нейросети: маркеры ArUco, распознавание людей и текста;
- написать узел ROS 2 для Пионера (сервисы, топики, параметры) и группу дронов, которая общается сообщениями и следует за ведущим в строю.

ГЛАВА 2

Подготовка: где писать и запускать код

Цели главы

- Понять, чем отличаются запуск в симуляторе и на настоящем дроне и чем различаются два дрона курса: Пионер Базовый и Пионер Мини 2.
- Найти в симуляторе редактор, выбор модели и языка, кнопки запуска и терминал.
- Запустить первую программу и увидеть её вывод.
- Запомнить правила безопасности при работе с настоящим Пионером.

2.1 Два дрона и два способа запуска

В курсе два дрона. **Пионер Базовый** программируется на Lua (программа загружается в автопилот) и на Python (программа работает на вашем компьютере). **Пионер Мини 2** Lua не знает: его программы написаны на Python и выполняются на самом дроне, а блоки Pioneer Bricks собирают такую же Python-программу. Для каждого дрона есть два «исполнителя»: симулятор и настоящий аппарат.

	Симулятор в браузере	Настоящий Базовый	Настоящий Мини 2
Что нужно	Любой компьютер с браузером: `https://simulator.sakoryagin.ru`	Дрон, аккумулятор, кабель USB или Wi-Fi, площадка	Дрон, аккумулятор, компьютер в сети Wi-Fi дрона, пульт, площадка
Lua	Язык «**Lua**» (только Базовый)	Pioneer Station: загрузка скрипта на борт, кнопка «**На дрон**» в симуляторе	нет
Python	Язык «**Python**»; библиотека зависит от модели	`pip install pioneer-sdk`, программа на вашем компьютере	`pioneer_sdk2`: программа на борту дрона
Блоки	Blockly (Базовый) или Bricks (Мини 2)	—	Pioneer Bricks на борту
Что видно	Полёт в 3D, терминал, телеметрия	Сам дрон и консоль Pioneer Station	Сам дрон, терминал и окна просмотра на борту
Цена ошибки	Нажать «**Сбросить**»	Сломанный пропеллер или хуже	То же

Где именно выполняется программа, показывает рисунок 2.

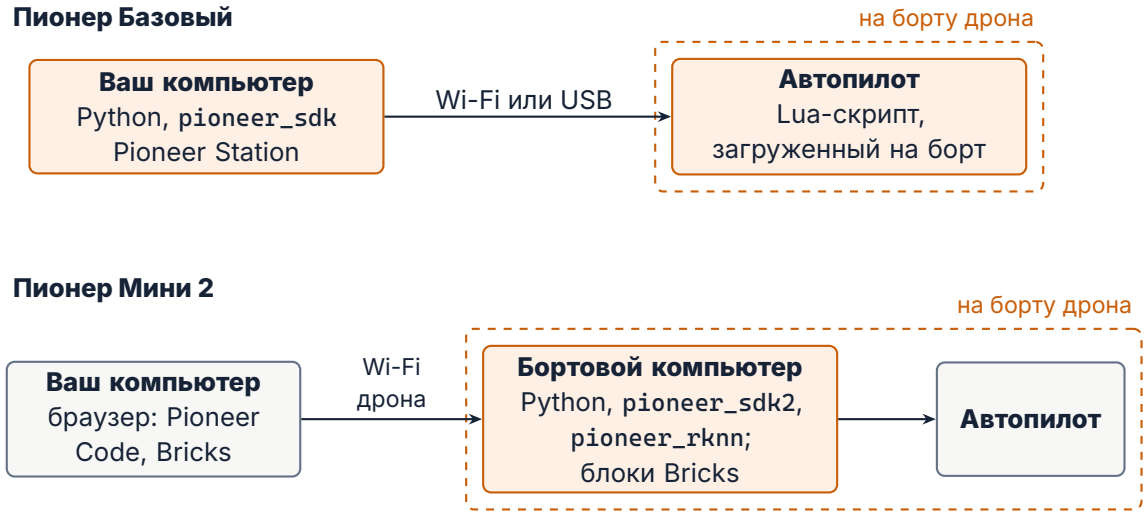


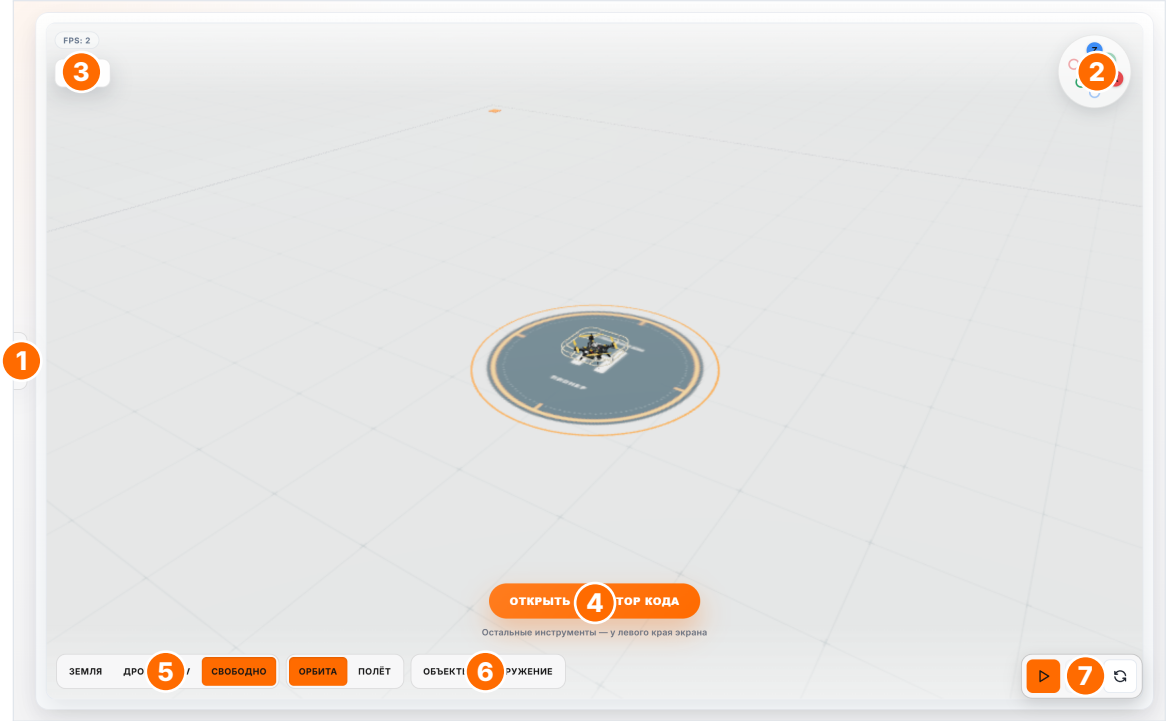
Рис. 2. Где выполняется программа. У Базового Lua работает внутри автопилота, а Python — на вашем компьютере. У Мини 2 Python работает на самом дроне, а компьютер нужен лишь для записи и запуска. Оранжевая заливка — место, где выполняется ваш код. Симулятор воспроизводит те же программные интерфейсы вместо дрона.

Код в обоих случаях один и тот же: симулятор воспроизводит официальный API автопилота (`ap`, `Ev`, `Timer`, `Ledbar`, `Sensors`), библиотеку `pioneer_sdk` для Базового и `pioneer_sdk2` с `pioneer_rknn` для Мини 2. Поэтому в курсе мы сначала отлаживаем программу в симуляторе, а на дрон переносим уже проверенную. Отличия симулятора от железа (упрощённая нейросеть, часть методов `pioneer_sdk2`) названы в соответствующих главах.

> **Сначала симулятор**
>
> Любая новая программа запускается сначала в симуляторе. На настоящем дроне запускается только программа, которая в симуляторе взлетела, выполнила задание и села.

2.2 Симулятор: что где находится

Откройте `https://simulator.sakoryagin.ru`. Регистрация не нужна: симулятор сразу показывает сцену. При первом запуске появится короткая экскурсия по интерфейсу — её можно пройти или пропустить, а позже повторить кнопкой «**Пройти тур ещё раз**» в панели «**Настройки**». После неё экран выглядит так:

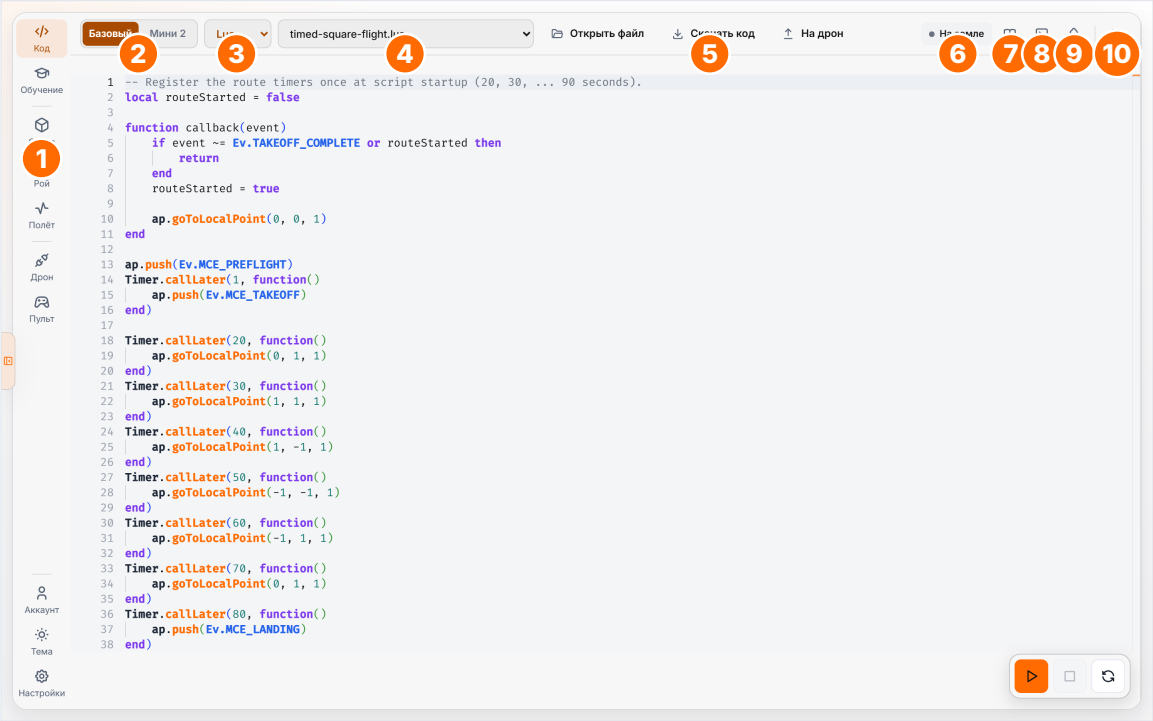


1. Язычок у левого края. За ним прячется «рельс» с панелями: «**Код**», «**Обучение**», «**Сцена**», «**Рой**», «**Полёт**», «**Дрон**», «**Пульт**»; внизу — «**Аккаунт**», «**Тема**» (светлая или тёмная) и «**Настройки**».
2. Компас осей. Ось **X** красная, **Y** зелёная, **Z** синяя и смотрит вверх. Щелчок по оси поворачивает камеру вдоль неё.
3. Запись видео сцены и снимок экрана.
4. «**Открыть редактор кода**» — сюда пишется программа.
5. Камера: «**Земля**» (вид сверху), «**Дрон**» (камера следует за дроном), «**FPV**» (от лица дрона), «**Свободно**» (орбита вокруг точки или свободный полёт камеры). Рядом — «**Орбита**» и «**Полёт**»: способ управления свободной камерой.
6. «**Объекты**» и «**Окружение**»: расстановка домов, маркеров, площадок; время суток и погода.
7. «**Запустить**», «**Стоп**» и «**Сбросить**». Сброс возвращает дрон на стартовую площадку и очищает след полёта.

Камеру вращают левой кнопкой мыши, сдвигают правой, приближают колёсиком.

2.3 Редактор кода

Нажмите «**Открыть редактор кода**» (или «**Код**» на рельсе). Редактор занимает рабочую область, а рельс остаётся слева. Сцена при этом продолжает жить: программа работает и после закрытия редактора.



1. Рельс панелей (тот же, что на сцене); активна панель «**Код**».
2. Модель дрона: «**Базовый**» или «**Мини 2**» (подробнее — ниже, в разделе 2.3.1).
3. Язык программы. У Базового: «**Lua**», «**Python · SDK1**» (библиотека `pioneer_sdk`) и «**Blockly**». У Мини 2: «**Python · SDK2**» (`pioneer_sdk2`) и «**Bricks**». Переключение не стирает код: у каждого языка своя вкладка.
4. Готовые примеры — в том числе программы из этой книги: для Базового папки `04_language`, `05_leds`, `06_missions` и другие, для Мини 2 — программы глав про `pioneer_sdk2`, `pioneer_rknn` и Bricks.
5. «**Открыть файл**» с компьютера, «**Скачать код**» в файл и «**На дрон**»: загрузка Lua-программы на настоящий Пионер Базовый по USB (кнопка видна только при языке Lua; см. раздел 2.7). У Мини 2 на её месте — «**Проект Bricks**».
6. Состояние программы и дрона: «**На земле**», «**Работает**», «**Взлёт**», «**В полёте**», «**Посадка**» (у Мини 2 до запуска — «**Остановлено**»).
7. Справочник API: описание и пример для каждой функции.
8. Терминал: сюда попадает вывод `print` и сообщения автопилота.
9. Уведомления: перед запуском симулятор проверяет программу и объясняет найденные ошибки здесь.
10. Закрыть редактор и вернуться к сцене. Кнопки «**Запустить**», «**Стоп**», «**Сбросить**» — внизу справа, как и на сцене.

2.3.1 Модель дрона: Базовый и Мини 2

В шапке редактора левее списка языков стоит переключатель модели «**Базовый**» / «**Мини 2**» (на снимке выше выбран Базовый). Он решает, какие языки и какая библиотека доступны:

	«Базовый»	«Мини 2»
Языки	«**Lua**», «**Python · SDK1**», «**Blockly**»	«**Python · SDK2**» и блоки (пункт списка называется «**Bricks**»); «**Lua**» недоступен
Python-библиотека	`pioneer_sdk`	`pioneer_sdk2` (импорт чужой библиотеки симулятор остановит с подсказкой)
Справочник и примеры	Lua или `pioneer_sdk`	команды `pioneer_sdk2` и программы глав про Мини 2
Кнопки файла	«**На дрон**» для Lua	«**Проект Bricks**»: скачать блоки как проект, который открывается в Bricks на дроне

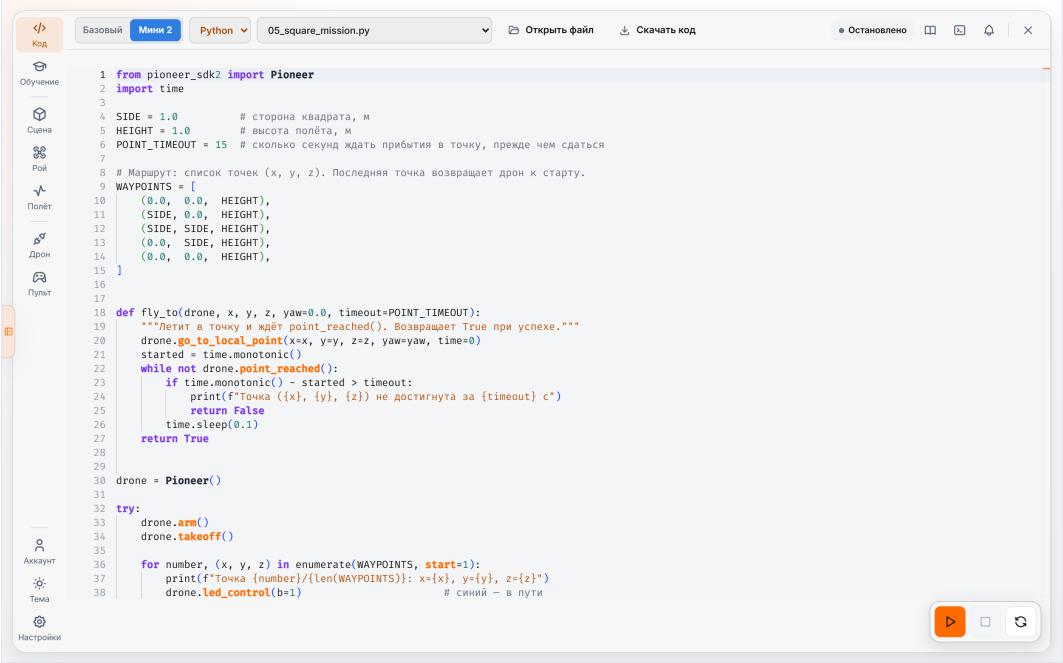


Рис. 3. Редактор с выбранной моделью «**Мини 2**»: язык «**Python · SDK2**», пример из главы про `pioneer_sdk2`. Для блочного режима «**Bricks**» вместо «**На дрон**» показана кнопка «**Проект Bricks**» (глава 18).

Модель можно сменить и кнопкой «**Мини 2**» в панели «**Рой**»: рядом с Базовым на сцене появится второй дрон. Во время работы программы модель не меняют — сначала «**Стоп**». Миссии треков Мини 2 в режиме «**Обучение**» сами выбирают или добавляют нужного дрона.

Пять шагов до запуска

Открыть редактор → выбрать язык → вставить или набрать программу → «**Запустить**» → закрыть редактор и смотреть на сцену. Если дрон ведёт себя не так, как ожидалось, — «**Стоп**», «**Сбросить**», и в терминале читать, что произошло.

2.4 Остальные панели рельса

Остальные панели пригодятся по ходу курса. Режим «**Обучение**» разобран в разделе 2.5.

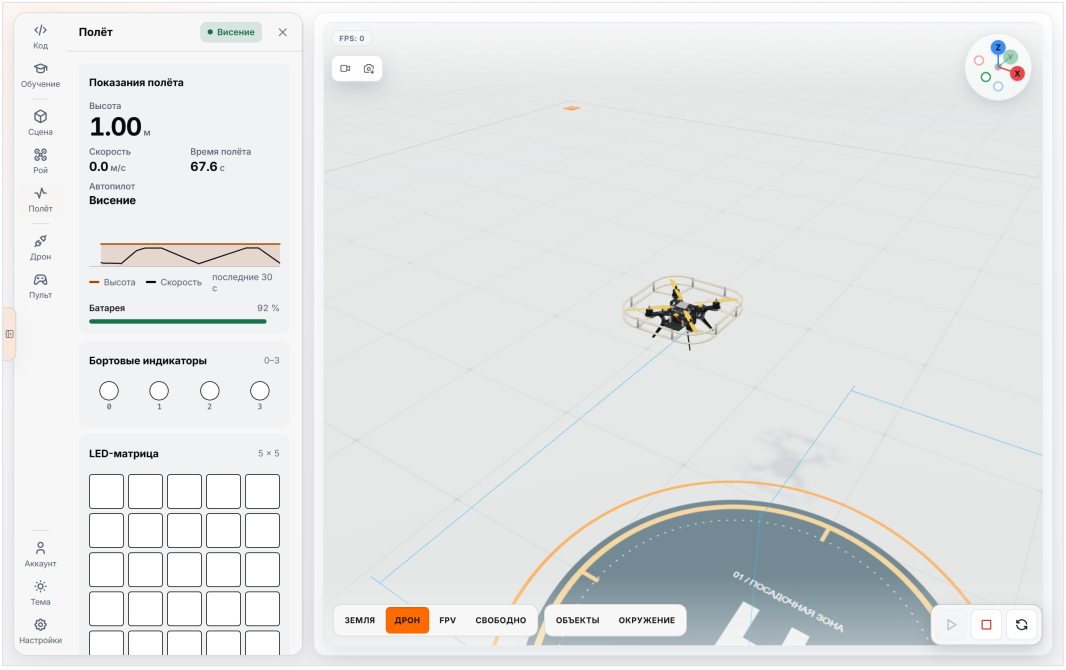


Рис. 4. Панель «**Полёт**»: высота, скорость, состояние автопилота, график последних 30 секунд, заряд батареи, бортовые светодиоды.

- «**Полёт**» — «приборная доска»: высота, скорость, время полёта, состояние автопилота («**Висение**», «**Взлёт**», «**Посадка**»), батарея и индикаторы светодиодной ленты. Пригодится, когда нужно сверить то, что вы видите в сцене, с цифрами.
- «**Сцена**» — каталог объектов и маркеров, генератор полигонов (поле, лес, холмы, город, промышленная зона). Нужна в главах про маршруты и ArUco.
- «**Рой**» — список дронов сцены: кнопки «**Базовый**» и «**Мини 2**» добавляют дрон нужной модели, переключатель «**Модель дрона**» меняет модель выбранного. Раздел «**Внешний Python**»: включите «**Разрешить управление**», скопируйте показанную строку подключения, и программа на вашем компьютере будет летать виртуальным дроном (раздел 7.1).
- «**Дрон**» — связь с настоящим Пионером Базовым по USB (Chrome или Edge): телеметрия и запуск загруженной Lua-программы.
- «**Пульт**» — мастер настройки пульта: дрон в симуляторе повторяет движения стиков (приложение D).
- «**Аккаунт**» — вход и регистрация; нужны, чтобы прогресс обучения сохранялся на сервере и был виден учителю (раздел 2.5).
- «**Настройки**» — скорость симуляции, качество графики, след дрона (цвет, толщина, вид), параметры автопилота, повтор экскурсии и сброс рабочего места.

2.5 Режим «Обучение»: книга и симулятор вместе

В симуляторе есть режим «**Обучение**». Он ведёт по тем же темам, что и книга, но в виде коротких **миссий**: читаете раздел книги, делаете задание в рамке, затем закрепляете его миссией в симуляторе. Это один маршрут, а не два разных курса.

Миссии собраны в пять **треков**. У каждого трека своя карта миссий и свой прогресс:

Трек	Миссий	Дрон и язык
«**Blockly**»	16	Пионер Базовый, блоки
«**Lua**»	93	Пионер Базовый, Lua
«**Python**»	124	Пионер Базовый, Python (`pioneer_sdk`)
«**Мини 2 · SDK2**» (в панели миссий подписан «**Мини 2 · Python**»)	64 в 19 главах	Пионер Мини 2, Python (`pioneer_sdk2`)
«**Мини 2 · Bricks**»	16 в 8 главах	Пионер Мини 2, блоки Pioneer Bricks

Когда вы открываете миссию трека Мини 2, симулятор сам выбирает на сцене дрон «Пионер Мини 2» или добавляет его; миссии остальных треков возвращают Базового. Программы остальных дронов при этом не меняются. Переключатель треков — в верхней части панели миссии:

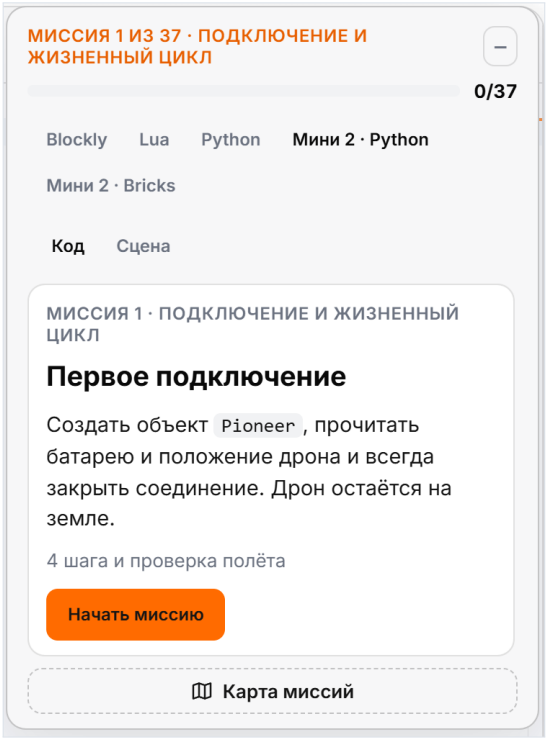
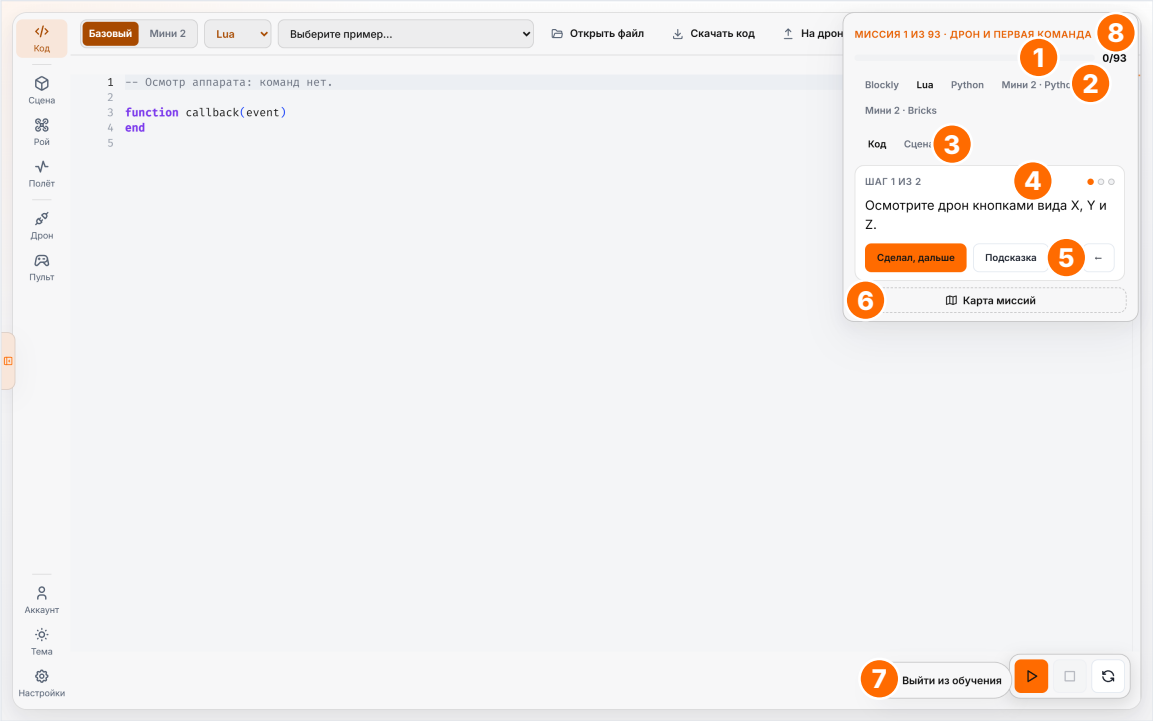


Рис. 5. Панель миссии с выбранным треком «**Мини 2 · Python**»: пять треков, прогресс (0 из 37), вводная карточка первой миссии, кнопки «**Начать миссию**» и «**Карта миссий**».



1. Сколько миссий пройдено из общего числа выбранного трека (на снимке — 0 из 93 в Lua).
2. Трек миссий: «**Blockly**», «**Lua**», «**Python**», «**Мини 2 · Python**» и «**Мини 2 · Bricks**». Активный трек выделен чёрным.
3. Что показывать на экране: «**Код**» (редактор) или «**Сцену**» (чтобы наблюдать за дроном без редактора).
4. Текущий шаг миссии («**Шаг 1 из 2**»); точки справа показывают, сколько шагов осталось. Шаги короткие: один шаг — одно действие.
5. «**Сделал, дальше**» переходит к следующему шагу; «**Подсказка**» раскрывает намёк, а в раскрытой подсказке есть кнопка «**Показать решение**»: используйте её, когда застряли, и обязательно разберите, чем решение отличается от вашего. Кнопка со стрелкой «←» возвращает к предыдущему шагу.
6. «**Карта миссий**»: все миссии выбранного трека по главам.
7. «**Выйти из обучения**» возвращает свободный симулятор. Написанная программа остаётся в редакторе.
8. Кнопка «—» сворачивает панель миссии, чтобы она не закрывала код.

2.5.1 Как устроена миссия

Миссия проходит по порядку: вводная карточка с целью, затем шаги, затем кнопка «**Проверить полёт**». Проверка — не «галочка за старание». Симулятор смотрит на журнал настоящего выполнения: какие команды приняты, какие состояния прошёл автопилот, где дрон был и что напечатала программа. Маршрут засчитывается по порядку фактических прибытий в точки, посадка — по состоянию автопилота и высоте. Поэтому программа, которая только печатает «готово», миссию не пройдёт. Часть миссий Мини 2

(светодиоды, окна камеры, сервопривод) проходится без полёта: проверяется журнал команд и вывод.

Если проверка не прошла, симулятор объясняет причину человеческим языком и показывает, что именно не совпало. Исправьте программу и нажмите «**Проверить полёт**» снова.

2.5.2 Карта миссий и порядок прохождения

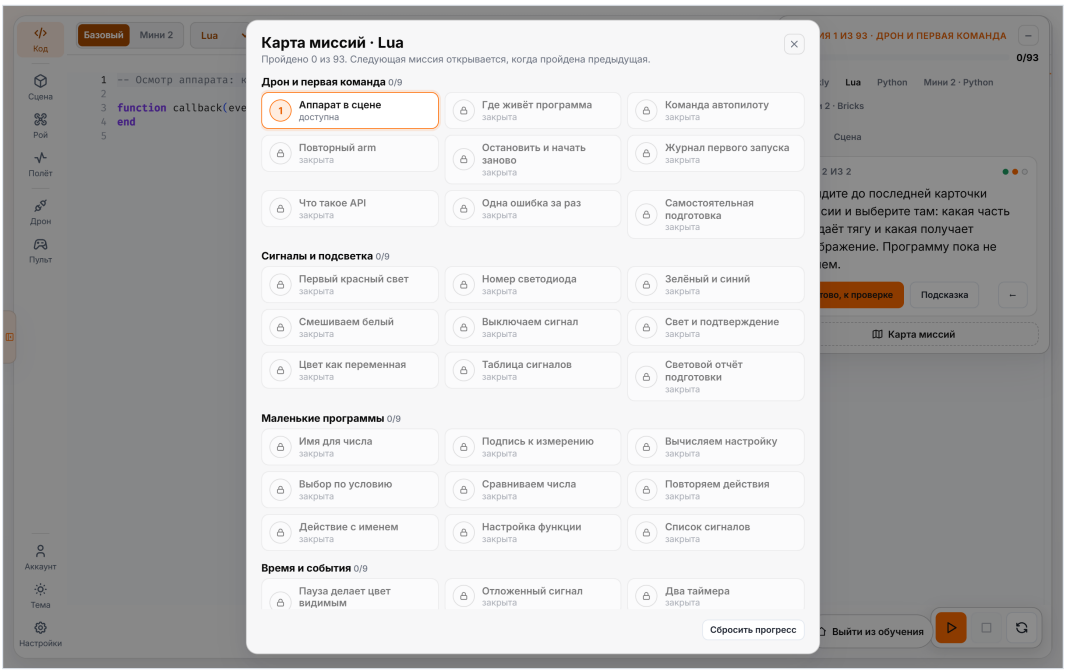


Рис. 6. Карта миссий для трека «**Lua**» (93 миссии, на экране видны первые главы). Следующая миссия открывается только после предыдущей; закрытые отмечены замком.

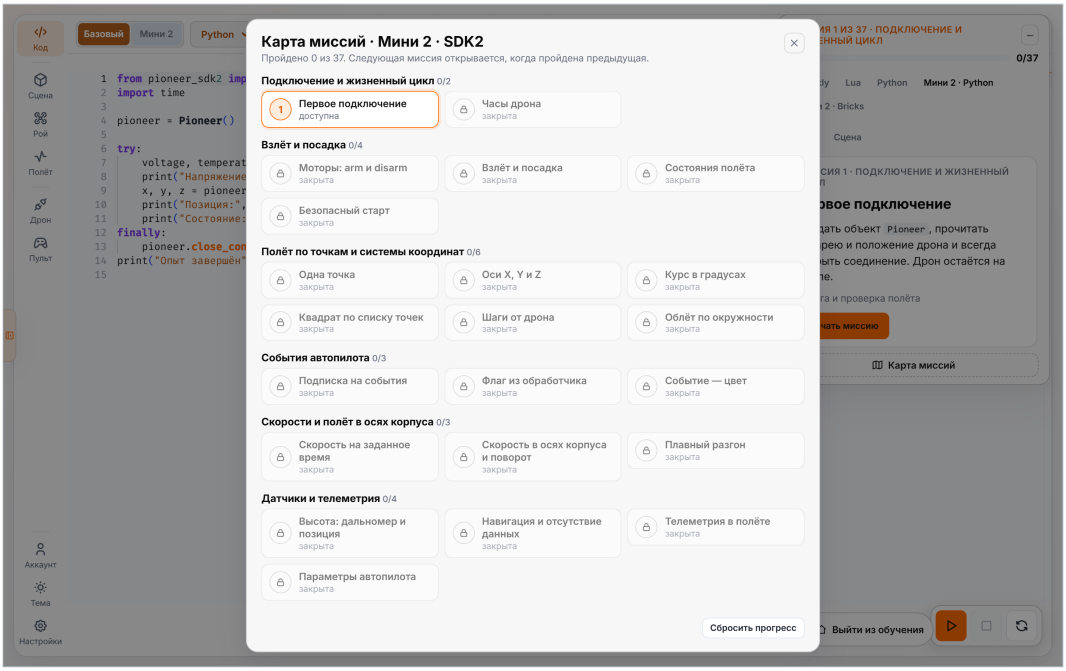


Рис. 7. Карта миссий трека «**Мини 2 · SDK2**»: 37 миссий по главам, от первого подключения до датчиков и параметров автопилота (дальше список прокручивается).

Миссии сгруппированы в главы. На картах Lua и Python это блоки по девять миссий: *Дрон и первая команда*, *Сигналы и подсветка*, *Маленькие программы*, *Время и события* и так далее. Понятие вводится до того, как им пользуются: сначала подготовка двигателей на земле, потом свет, потом переменные и условия, потом время, и только затем полёт. Главы трека «**Мини 2 · SDK2**» следуют порядку главы 16: подключение, взлёт, точки, события, скорости, датчики, подсветка, груз, пульт, камера, сервопривод, метки ArUco, итоговый патруль. Трек «**Мини 2 · Bricks**» — от первых блоков и светодиодов к циклам, датчикам, точкам, камере и метке ArUco; он дополняет проекты главы 18.

2.5.3 Аккаунт, кабинеты ученика и учителя

Панель «**Аккаунт**» необязательна. Без входа прогресс хранится в браузере этого компьютера. Если нужно, чтобы прогресс не терялся и был виден преподавателю, войдите в аккаунт: прогресс по всем пяти трекам хранится на сервере, вместе с ним — ваш код проверок. Сессия действует два часа без обращений к серверу, после этого нужно войти заново. Гость код на сервер не отправляет.

- **Мой кабинет** (ученик): карточка «**Продолжить**» с текущей миссией, прогресс по пяти трекам, список миссий по главам с датой зачёта и комментарием учителя, история попыток со своим кодом, календарь активности и достижения, смена имени и пароля.
- **Кабинет учителя**: таблица класса с прогрессом каждого ученика по пяти трекам и отметкой «застрял на миссии N», группы (классы), создание аккаунтов списком с временными паролями, таблица «ученик × миссия», страница ученика с ручным зачётом и комментарием, экспорт отчёта в CSV и обезличенная сводка о том, какие миссии чаще бросают.

Оценок в баллах в симуляторе нет: их ставит учитель в своём журнале.

2.5.4 Как книга соотносится с миссиями

Вверху каждой главы книги стоит зелёная врезка 🎮 *Миссии симулятора* с номерами миссий по теме главы, а после урока — рамка ✎ *Задание*. Сначала решаем задание в рамке своими руками, затем проходим миссию: в ней те же идеи, но с автоматической проверкой.

Глава книги	Миссии Lua	Миссии Python
Подготовка (гл. 2)	1–9	1–9
Основы языка (Lua: гл. 3; Python: часть II)	19–27	19–27
Светодиоды и таймеры (Lua: гл. 4; Python: гл. 7)	10–18, 28–36	10–18, 28–36
Автономные миссии (Lua: гл. 5; Python: гл. 7)	37–72	37–72
Камера, ArUco и наведение (гл. 8)	—	73–99
Самостоятельные миссии	—	100–108
Расширенные главы: датчики и команды SDK, курс и груз, оптический поток, пульт и Wi-Fi (Python); датчики, таймеры, цвет и GPS, связь (Lua)	73–93	109–124

Камера и ArUco изучаются только на Python: у Lua-автопилота Пионера нет API камеры. Блочный практикум (16 миссий) лучше пройти параллельно с первыми главами: он показывает те же команды в виде блоков.

Для Пионера Мини 2 соответствие такое (миссии SDK2 с 38-й — ROS 2 и рой, часть V книги; снимки экранов ниже сняты, когда в треке было 37 миссий):

Тема (глава 16)	SDK2	Bricks	Где в книге
Подключение, жизненный цикл, взлёт и посадка	1–6	1–2	гл. 16; Bricks — гл. 18
Полёт по точкам и системы координат	7–12	10–12	гл. 16, разд. 16.6; Bricks — гл. 18
События, скорости, полёт в осях корпуса	13–18	—	гл. 16
Датчики и телеметрия	19–22	7–9	гл. 16
Подсветка	23–25	3–6	гл. 16, 18
Груз, захват и виртуальный пульт RC	26–29	—	гл. 16
Камера, окно просмотра, сервопривод, запись	30–34	13–14	гл. 16, 18
Метки ArUco	35–36	15	гл. 16, 18
Итоговый патруль / итоговый проект	37	16	гл. 16, 18
ROS 2: сервисы полёта, топики, параметры и нагрузка, миссия-узел	38–56	—	гл. 19
Рой: общение, телеметрия и строй	57–64	—	гл. 20

Нейросети (глава 17) отдельных миссий не имеют: в симуляторе нейросеть заменена упрощённой имитацией, а приёмы закрепляются миссиями Python про подтверждение результата несколькими кадрами и поиск по секторам.

> **Порядок работы**
>
> Читаем раздел → запускаем примеры в редакторе → решаем задание в рамке → проходим миссию в панели «**Обучение**». Если миссия не проходится, вернитесь к разделу: в нём объяснено, зачем нужен каждый шаг.

2.6 Первая программа

Традиционно первая программа печатает приветствие. На дроне нет экрана, но есть консоль: в симуляторе это терминал редактора, в Pioneer Station — окно «Консоль».

lua/02_sensors/getting_started_hello.lua

```lua
-- Это комментарий, он не исполняется
-- Вывод сообщения в консоль Pioneer Station
print("Системы в норме. Привет, Пионер!")
```

Выберите язык «**Lua**», вставьте программу и нажмите «**Запустить**». Затем откройте терминал (значок **>_** в заголовке редактора):

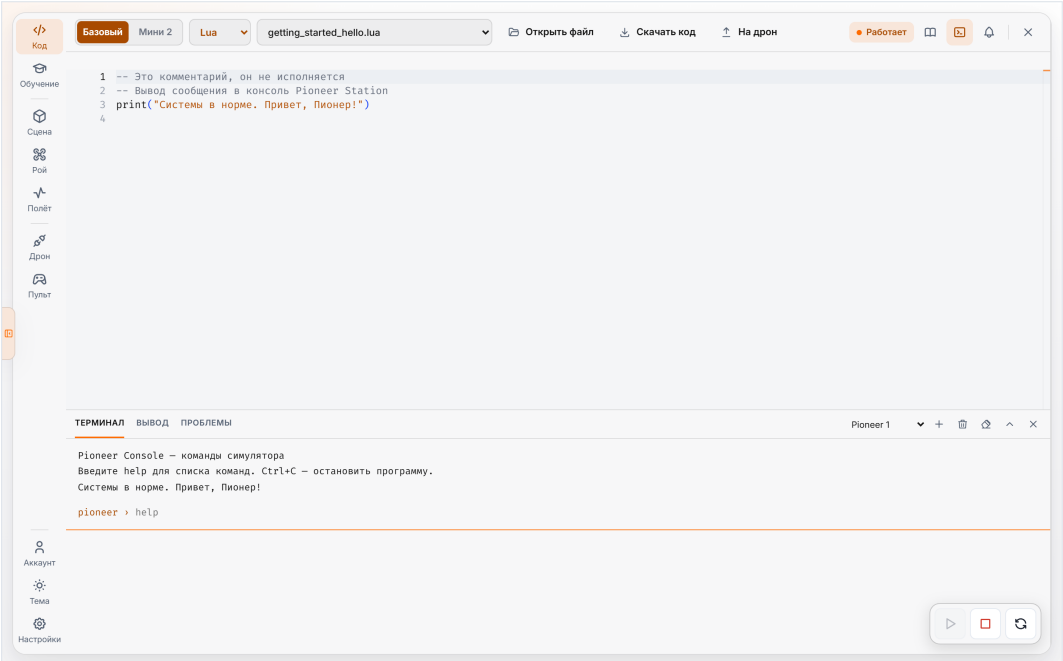


Рис. 8. Терминал показывает всё, что программа напечатала через `print`, и сообщения автопилота.

Что здесь произошло:

- строки, начинающиеся с `--`, — комментарии, Lua их не выполняет;

- `print(…)` печатает текст в консоль. Текст — это **строка**, она записывается в кавычках;
- программа закончилась сразу после печати. Дрон не взлетел, потому что мы не дали ему ни одной команды.

ЗАДАНИЕ 2.1 · Проверка связи

Измените текст в кавычках на своё имя и запустите программу ещё раз. Затем добавьте вторую строку `print(2 + 2)` и убедитесь, что Lua умеет считать.

Что должно получиться: В терминале две строки: ваше сообщение и число `4`. Дрон стоит на площадке.

В симуляторе: миссии Lua: 1–9 в панели «**Обучение**».

Первая программа для Пионера Мини 2

Мини 2 Lua не знает, поэтому его «первая программа» — на Python: в редакторе включите переключатель «**Мини 2**» (раздел 2.3.1), и `print` заработает так же, как выше. Первая программа с подключением к дрону, чтением батареи и светодиодами разобрана в главе 16, а блочный вариант — в главе 18.

Где лежат примеры

Каждая программа в книге подписана именем файла, например `lua/02_sensors/getting_started_hello.lua`. Это путь внутри папки `examples/` курса. Те же файлы доступны в редакторе симулятора через список примеров (для выбранной модели дрона), а текст программы можно скопировать прямо из PDF — отступы и дефисы при копировании сохраняются.

2.7 Настоящие дроны

Всё, что отлажено в симуляторе, переносится на дрон. Но дрон — электромеханическое устройство с вращающимися пропеллерами, поэтому порядок работы с ним строже.

2.7.1 Техника безопасности

Критически важно

Пренебрежение этими правилами приводит к травмам и поломке оборудования. Они не обсуждаются.

- **Механика — только без питания.** Любые монтажные работы, замена деталей, подключение модулей — на обесточенном аппарате.
- **Отладка — без пропеллеров.** При загрузке и проверке скриптов, тестировании моторов и настройке пульта пропеллеры должны быть сняты. Случайный запуск моторов с пропеллерами — неконтролируемый взлёт.

- **Аккумуляторы.** Только исправные LiPo-батареи, зарядка под присмотром. Вздувшийся или повреждённый аккумулятор не используется.
- **Порядок включения.** Сначала пульт, потом питание дрона. Выключение — в обратном порядке.
- **Рабочее место.** На столе нет лишних предметов и проводов; с пропеллерами дрон создаёт сильный поток воздуха.
- **Пульт всегда в руках.** Особенно для программ на борту Мини 2: аварийно перехватить управление у программы можно только с пульта (см. ниже).

2.7.2 Пионер Базовый: Pioneer Station

Основная среда для Lua-программ на настоящем Базовом дроне — **Pioneer Station**: редактор с подсветкой синтаксиса, загрузчик скриптов на борт, консоль для `print` и телеметрии, менеджер прошивок. Дистрибутив бесплатно доступен на сайте поддержки Геоскан.

Порядок подключения:

1. Убедитесь, что пропеллеры сняты (если вы не планируете немедленный вылет на полигоне).
2. Подключите аккумулятор. Прозвучит мелодия инициализации, светодиоды начнут мигать.
3. Соедините дрон с компьютером кабелем micro-USB.
4. Запустите Pioneer Station и выберите COM-порт (обычно определяется сам).
5. Вставьте программу в редактор и нажмите «Загрузить». Скрипт передаётся в память автопилота и запускается; результат — в окне «Консоль».

Загрузить Lua-программу можно и прямо из симулятора: кнопка «**На дрон**» в редакторе (Chrome или Edge) записывает скрипт в дрон по USB, а запускают его из панели «**Дрон**». Программа только загружается и остаётся остановленной, пока вы её не запустите.

Python. Python-программа для Базового работает не на борту, а на вашем компьютере, и общается с дроном по Wi-Fi через библиотеку `pioneer_sdk`. Установка и подключение разобраны в главе 7. Ту же программу можно направить и на симулятор — тогда по Wi-Fi полетит виртуальный дрон на сайте (раздел 7.1).

2.7.3 Пионер Мини 2: программа на борту

У Мини 2 Lua нет, а Pioneer Station для его программ не нужна. На дроне работает Pioneer OS, и программа на Python (`pioneer_sdk2`, `pioneer_rknn`) или блоки Bricks выполняются прямо на нём. Компьютер нужен, чтобы записать программу и запустить её. Подробный порядок и примеры — в главах 16, 17 и 18; здесь только общая схема.

1. Включите дрон и подключите компьютер к его сети Wi-Fi (имя сети начинается с `PMINI2`). Адрес дрона в этой сети — `10.42.0.1`.

2. Включите пульт и переведите тумблер `SWB` в нижнее положение: автопилот проверяет «наличие пилота» и без этого полётную программу не выполнит. Проверка отключается параметром `Copter_flyWithoutRc`, но тогда перехватить управление с пульта будет нельзя; оставляйте её включённой.
3. Откройте в браузере главное меню Pioneer Code: `http://10.42.0.1:9090`. Оттуда — среда разработки (редактор и терминал, порт 9999), Pioneer Bricks (порт 2020), регистр нейросетей и галерея фото и видео.
4. Python: положите программу в рабочую папку и запустите в терминале командой `python3 имя_файла.py` (или скопируйте по `scp` и войдите по SSH). Остановка — `Ctrl` + `C`; в примерах книги это ведёт в блок `finally` и к посадке. Блоки: соберите программу в Bricks и нажмите «**Вжух**».

> **Полётные программы Мини 2 не проверены на дроне**
>
> Примеры глав про Мини 2 проверены на подключение, телеметрию, камеру и окна просмотра; полётные программы авторами на настоящем дроне не запускались. Сначала — симулятор, затем пропеллеры сняты и дрон закреплён или стоит на открытой площадке без людей, пульт в руках.

Версии, на которые опираются главы: Pioneer OS 1.0.0, `pioneer_sdk2` 0.16.0, `pioneer_rknn` 1.10.0, Pioneer Bricks 2.3.0; официальная документация — [1—3].

2.7.4 Изучение чистого Lua

Синтаксис Lua можно тренировать и без дрона: симулятор для этого подходит (терминал показывает `print`), а на компьютере удобны ZeroBrane Studio, Visual Studio Code с расширением Lua или онлайн-интерпретаторы. Код оттуда для дрона всё равно переносится в симулятор или Pioneer Station.

Итоги

- Исполнителей два: симулятор в браузере и настоящий дрон; дронов тоже два: Базовый и Мини 2.
- В симуляторе: редактор → модель и язык → «**Запустить**»; вывод — в терминале, ошибки — в уведомлениях.
- Базовый: Pioneer Station или «**На дрон**» для Lua, `pioneer_sdk` с компьютера для Python.
- Мини 2: Wi-Fi дрона, Pioneer Code на `10.42.0.1`, программа на борту; пульт в руках.

ЧАСТЬ I

Программирование на Lua

ГЛАВА 3

Язык Lua

Цели главы

- Различать типы данных Lua и объявлять переменные — локальные и глобальные.
- Записывать арифметику, сравнения и логические условия.
- Управлять ходом программы: ветвления `if`, циклы `for`, `while`, `repeat`.
- Писать свои функции и понимать, как автопилот вызывает нашу функцию `callback`.
- Узнавать типичные ошибки новичка по сообщению в терминале.

Миссии симулятора к главе

В панели «**Обучение**» выберите язык «**Lua**».

- **1–9 «Дрон и первая команда»**, если вы ещё не прошли их с главой 2: аппарат в сцене, где живёт программа, команда автопилоту, повторный arm, остановить и начать заново, журнал первого запуска, что такое API, одна ошибка за раз, самостоятельная подготовка. Миссия 8 «Одна ошибка за раз» — пара к разделу «Типичные ошибки» этой главы.
- **19–27 «Маленькие программы»**: имя для числа, подпись к измерению, вычисляем настройку, выбор по условию, сравниваем числа, повторяем действия, действие с именем, настройка функции, список сигналов. Это материал уроков главы, применённый к подсветке дрона.

В миссиях 19–27 результат виден по светодиоду: строка `leds:set(0, r, g, b)` зажигает светодиод номер 0, каждая составляющая цвета — число от `0` (выключено) до `1` (максимум). Подробно подсветка разобрана в главе 4, а её миссии 10–18 удобнее пройти вместе с той главой; для миссий 19–27 этой строки достаточно.

3.1 Зачем Lua

Lua — лёгкий встраиваемый язык, созданный в начале 1990-х для расширения программ скриптами. Он компактен, не требует много памяти и быстро работает, поэтому живёт внутри игр, роботов и встраиваемых систем. Автопилот Пионера исполняет Lua прямо на борту: программа загружается в память контроллера и работает без компьютера. Версия языка — Lua 5.3; ту же версию запускает симулятор, поэтому всё, что работает в нём, понятно и бортовому интерпретатору.

В этом курсе Lua нужен, чтобы:

- управлять светодиодами через модуль `Ledbar` (глава 4);
- выполнять действия по расписанию через `Timer`, не останавливая программу;
- отвечать на события автопилота — взлёт завершён, точка достигнута, батарея садится — в функции `callback(event)` (глава 5).

Сейчас запоминать эти имена не нужно: в этой главе мы работаем только с языком.

> **Как работать с главой**
>
> Каждый пример запускайте в симуляторе (язык «**Lua**», кнопка «**Запустить**») и сверяйте вывод на вкладке «**Вывод**» нижней панели редактора с комментариями в коде: справа от `print` написано, что должно появиться. После каждого урока — задание; решайте его, не заглядывая вперёд: оно опирается только на пройденное.

3.2 Толковый словарь

Число вещественное или целое значение: `3.14`, `10`; математика — в библиотеке `math`.

Строка текст в кавычках: `"Lua"`; склейка — оператор `..`, длина — `#s`.

Булево `true` или `false`; результат сравнения и материал для условий.

nil «ничего»: переменная без значения, отсутствующее поле таблицы.

Таблица единственная составная структура Lua: массив `t[1]` и словарь `t.x` одновременно.

Функция именованный блок кода с параметрами и результатом.

Область видимости `local` делает переменную видимой только внутри текущего блока; без `local` переменная глобальная — видна во всём скрипте.

Истинность ложными считаются только `false` и `nil`; ноль и пустая строка — истина.

ipairs / pairs обход таблицы: `ipairs` — по номерам 1..N, `pairs` — по всем ключам в произвольном порядке.

table.unpack раскладывает таблицу в список значений: `table.unpack({1,0,0})` даёт `1, 0, 0`. Глобальной функции `unpack` в Lua 5.3 нет, поэтому в начале программы пишут `local unpack = table.unpack`.

3.3 Переменные и типы

Переменная — это имя, за которым закреплено значение. Тип значения определяется автоматически: число, строка, булево, таблица, функция или nil. Узнать тип можно функцией `type()`.

lua/04_language/variables_intro.lua

```lua
-- локальная числовая переменная
local x = 10
-- глобальная строковая переменная
name = "Pioneer"
-- локальная булева переменная
```

```
local ready = true
-- таблица с позиционными элементами и именованным полем
local t = {1, 2, 3, speed = 3.5}
-- выясняем типы значений
print(type(x), type(name), type(ready), type(t))  -- number  string  boolean
↪  table

-- множественное присваивание: две переменные одной строкой
local a, b = 1, 2
-- обмен значениями без третьей переменной
a, b = b, a
print(a, b)                                       -- 2  1
```

Что происходит.

- `local x = 10` — локальная переменная: живёт в том блоке, где объявлена.
- `name = "Pioneer"` без `local` — глобальная: её видно из любой функции скрипта. Так в курсе объявляют таймеры (глава 4), которые должны жить до конца программы.
- `local a, b = 1, 2` объявляет две переменные одной строкой, а `a, b = b, a` меняет их местами без третьей переменной.
- `print(type(x), ...)` печатает имена типов через табуляцию, а `print(a, b)` после обмена покажет `2 1`.

Строки и подписи

Число и строка — разные типы: `2` участвует в вычислениях, `"2"` — просто текст. Чтобы подписать число в журнале, его склеивают со строкой оператором `..`; число при склейке само превращается в текст. Подпись с единицами измерения («высота, м») избавляет от вопроса «два чего?».

lua/04_language/lua_strings_concat.lua

```
-- число участвует в вычислениях, строка подписывает его в журнале
local height = 2                    -- заданная высота, метры
print("высота, м: " .. height)      -- высота, м: 2

-- склейка .. превращает число в текст; длина строки — #
local name = "Pioneer"
print(#name, name .. "!")           -- 7  Pioneer!
```

Когда число нужно ещё и округлить, пользуются `string.format`: в шаблоне `%.2f` означает «число с двумя знаками после запятой», `%g` — «число без лишних нулей», `%s` — «вставить текст». Эта функция встретится в главе 5, где печатаются координаты.

Форматирование в print

```
print(string.format("x = %.2f м", 1.5))   -- x = 1.50 м
print(string.format("%s: %g", "z", 2.0))  -- z: 2
```

Таблицы

Таблица — и массив, и словарь. Позиционные элементы нумеруются **с единицы**, именованные поля доступны через точку.

lua/04_language/lua_variables_tables.lua

```lua
-- словарь (именованные поля)
local p = {x = 1, y = 2}
-- массив (позиционные элементы)
local a = {10, 20, 30}
-- добавление нового поля
p.z = 3
-- добавление нового элемента массива
a[4] = 40
-- поле словаря, четвёртый элемент и длина массива
print(p.z, a[4], #a)  -- 3  40  4
```

- `#a` — длина массива. После `a[4] = 40` она равна 4.
- Присваивание `t.speed = nil` удаляет поле из таблицы.
- Таблицами в курсе задаются цвета `{1, 0, 0}` и точки маршрута `{x, y, z}`.

> **local по умолчанию**
>
> Объявляйте переменные через `local`. Глобальными оставляйте только то, что должно жить всю программу: таймеры и обязательную функцию `callback` (о ней — в разделе «Функции» этой главы). Переменная, объявленная через `local` в самом начале файла, тоже живёт всю программу и видна всем функциям ниже, — так в миссиях хранят текущее состояние (глава 5).

ЗАДАНИЕ 3.1 · Имена, подписи, таблицы

1. Объявите локальную переменную `x` со значением `42` и напечатайте её.
2. Сохраните высоту `2` в переменной `height` и напечатайте строку «высота, м: 2», не записывая цифру 2 в строке вручную.
3. Создайте таблицу `t` с полем `speed = 3.5` и элементами `1, 2, 3`; напечатайте `t.speed` и `#t`.
4. Добавьте в `t` поле `mode = "auto"`, затем удалите поле `speed` и снова напечатайте `t.speed`.
5. Объявите `local a, b = 1, 2`, поменяйте значения местами одной строкой и напечатайте результат.
6. Напечатайте `type(nil)`, `type("5")` и `type(5)`. Чем отличаются два последних?

Что должно получиться: 1) `42`; 2) `высота, м: 2`, а в коде — склейка `"высота, м: " .. height`; 3) `3.5` и `3`; 4) `nil`; 5) `2` и `1`; 6) `nil`, `string`, `number`: `"5"` — текст, `5` — число.

В симуляторе: миссии Lua: 19–20 в панели «**Обучение**».

3.4 Арифметика и сравнения

Операции: `+ - * / %` (остаток от деления) и `^` (степень). Сравнения: `< <= > >= ==` и `~=` («не равно»). Приоритет как в математике: сначала `^`, затем `* / %`, потом `+ -`; скобки меняют порядок.

lua/04_language/lua_arithmetic_ops.lua

```lua
-- приоритет: умножение выполняется раньше сложения
local a = 2 + 3 * 4
-- остаток от деления (modulo)
local r = 5 % 2
-- возведение 2 в степень 3 (результат всегда вещественный)
local p = 2 ^ 3
print(a, r, p)        -- 14  1  8.0
-- сравнения дают булево значение
local eq = (a == 14)
local gt = (a > 10)
print(eq, gt)         -- true  true
-- "целочисленное" деление с округлением вниз
local q = math.floor(17 / 4)
-- евклидова длина вектора
local len = math.sqrt(3*3 + 4*4)
print(q, len)         -- 4  5.0
```

Что происходит.

- `2 ^ 3` даёт `8.0`, а не `8`: степень и деление `/` всегда возвращают вещественное число. Целые и вещественные числа в Lua 5.3 — два подвида одного типа, в арифметике они смешиваются свободно.
- Оператора `**` в Lua нет — это Python. Степень только через `^`.
- `math.floor(17 / 4)` — целочисленное деление с округлением вниз. В Lua 5.3 есть и оператор `//` (целочисленное деление), но `math.floor` читается понятнее.
- `math.sqrt`, `math.abs`, `math.max`, `math.min`, `math.pi` — то, что понадобится в миссиях. Тригонометрия Lua и автопилот измеряют углы в **радианах**: полный оборот — $2\pi \approx 6.28$ рад, $180^\circ = \pi$. Перевод градусов в радианы: `math.rad(deg)`, обратно — `math.deg(rad)`.

Ограничить значение диапазоном

`v = math.max(0, math.min(1, v))` «зажимает» `v` между 0 и 1. Этот приём пригодится для яркости светодиодов.

ЗАДАНИЕ 3.2 · Считаем заранее, проверяем запуском

Перед каждым запуском запишите ответ на бумаге, затем сравните с выводом.

1. Напечатайте `3 + 4 * 2 - 5` и `(3 + 4) * (2 - 5)`.
2. Напечатайте остаток `17 % 4` и выражение `n % 2 == 0` для `n = 10`.

3. Максимум составляющей цвета — `1`. Сохраните его в `maxRGB` и вычислите половину делением.
4. Вычислите длину вектора `(x, y) = (3, 4)` через `math.sqrt(x*x + y*y)`.
5. Переведите `90` градусов в радианы двумя способами: `deg * math.pi / 180` и `math.rad(deg)`.
6. Вычислите `a + t * (b - a)` (линейная интерполяция) для `a = 0`, `b = 10`, `t = 0.25` и среднее трёх чисел 1, 2, 3.

Что должно получиться: 1) `6` и `-21`; 2) `1` и `true`; 3) `0.5`; 4) `5.0` — корень всегда вещественный; 5) оба способа печатают `1.5707963267949`; 6) `2.5` и `2.0`.

В симуляторе: миссии Lua: 21 в панели «**Обучение**».

3.5 Логические операции

Логика: `and` (И), `or` (ИЛИ), `not` (НЕ). Ложными считаются только `false` и `nil` (истинность): число `0` и пустая строка — истина. Операторы `and` и `or` вычисляют правую часть только при необходимости («короткое замыкание»).

lua/04_language/lua_logic_ops.lua

```lua
local a = 42
-- сравнение возвращает булево значение
local gt = a > 10
-- логическое И и НЕ
local ok = true and not false
-- "тернарный" идиом в Lua через and/or
local sel = gt and 1 or 0
-- композиция условий "в диапазоне"
local within = (a >= 0) and (a <= 100)
print(ok, sel, within)        -- true  1  true

local name = "Pioneer"
-- проверка непустой строки
local nonempty = (name ~= nil) and (name ~= "")
-- исключающее ИЛИ для булевых
local p, q = true, false
local xor = (p and not q) or (q and not p)
print(nonempty, xor)          -- true  true
```

Что происходит.

- `gt and 1 or 0` — идиома «если `gt`, то 1, иначе 0». В Lua нет тернарного оператора, его заменяет эта пара. У неё есть ловушка: если «значение для истины» само `false` или `nil`, результатом станет значение после `or`. Поэтому в середине ставят число, строку или таблицу, но не булево.
- `(a >= 0) and (a <= 100)` — проверка диапазона. Запись `0 <= a <= 100` в Lua не работает.

- `name ~= nil and name ~= ""` — сначала убеждаемся, что значение есть, и только потом сравниваем со строкой. Так защищаются от `nil`.
- Строки сравниваются лексикографически: `"a" < "b"` истинно.

ЗАДАНИЕ 3.3 · Да или нет

1. Для `voltage = 3.4` и `temp = 85` напечатайте «сигнал опасности»: `voltage < 3.5 or temp > 80`.
2. Напечатайте предикат «индекс чётный» для `i = 7`: `i % 2 == 0`.
3. Для `a = 3`, `b = 8` выберите максимум одной строкой через `and/or`.
4. Напечатайте булеву переменную `on = true` до и после `on = not on`.
5. Проверьте закон де Моргана: напечатайте `not (p and q)` и `(not p) or (not q)` для всех четырёх сочетаний `p`, `q`.
6. Напишите условие «`t` не `nil` и у `t` есть поле `value`» и проверьте его для `t = nil` и `t = {value = 5}`.

Что должно получиться: 1) `true`; 2) `false`; 3) `8` (запись `a > b and a or b`); 4) `true`, затем `false`; 5) в каждой из четырёх строк два одинаковых значения, `false false` — только для `p = true, q = true`; 6) для `nil` условие ложно и ошибки нет, для таблицы — истинно.

В симуляторе: миссии Lua: 23 в панели «**Обучение**».

3.6 Условия

Ветвление записывается словами `if` … `then` … `elseif` … `then` … `else` … `end`. Ветка `elseif` (слитно!) может повторяться, `else` — не обязательна, `end` — обязателен всегда.

lua/04_language/lua_conditions_if.lua

```lua
-- классификация числа x: каскад if / elseif / else
local function classify(x)
  if x > 0 then
    return "positive"
  elseif x == 0 then
    return "zero"
  else
    return "negative"
  end
end

print(classify(5))    -- positive
print(classify(0))    -- zero
print(classify(-3))   -- negative
```

Что происходит. Функция `classify` проверяет условия по порядку и возвращает первое подходящее слово. Для `0` первая проверка `x > 0` ложна, вторая `x == 0` истинна — получаем `"zero"`.

Охранное условие

Обращение к полю несуществующей таблицы — ошибка «attempt to index a nil value». Защищаются от неё проверкой перед обращением:

lua/04_language/lua_conditions_check.lua

```lua
-- "охранное" условие: работаем с полем, только если таблица и поле
↪  существуют
local function doubled(t)
  if t and t.value then
    return t.value * 2
  end
  return "нет данных"
end

print(doubled({value = 42}))  -- 84
print(doubled(nil))           -- нет данных
```

Условие `t and t.value` читается так: «`t` существует, и у неё есть поле `value`». Если `t` равно `nil`, до `t.value` дело не доходит — спасает короткое замыкание `and`.

Таблица вместо каскада if

Когда вариантов много и каждому соответствует действие, удобнее таблица: `colors[state]` или даже таблица функций `actions[state]()`. Так в главе 4 устроен светофор — самый маленький конечный автомат.

ЗАДАНИЕ 3.4 · Две ветки, три проверки

1. Флаг `ready = false`. Напечатайте `"красный"`, если флаг ложен, и `"зелёный"`, если истинен. Запустите дважды, поменяв только значение флага.
2. Порог высоты — 2 м. Напечатайте `"выше"`, только если `height` **строго** больше порога, иначе `"не выше"`. Проверьте на 1, 2 и 3: ниже границы, на границе и выше.
3. Тройное ветвление по температуре: `temp < 0` → `"cold"`, от 0 до 20 (не включая 20) → `"cool"`, иначе `"warm"`. Проверьте на -5, 0, 20.
4. Выберите цвет: `{1,0,0}` для `state == "RED"`, `{1,1,0}` для `"YELLOW"`, иначе `{0,1,0}`; напечатайте три составляющие через `table.unpack`.
5. Защитите деление от нуля: если `b ~= 0`, напечатайте `a / b`, иначе — сообщение «делить на ноль нельзя».

Что должно получиться: 1) `красный`, после замены флага — `зелёный`; 2) `не выше`, `не выше`, `выше` — граница 2 к «выше» не относится; 3) `cold`, `cool`, `warm`; 4) для `"YELLOW"` — `1 1 0`; 5) при `b = 0` — сообщение, а не `inf`.

В симуляторе: миссии Lua: 22–23 в панели «**Обучение**».

3.7 Циклы

Три вида повторения: числовой `for` (счётчик от начала до конца включительно, с шагом), `while` (пока условие истинно) и `repeat ... until` (тело выполняется хотя бы один раз). Для таблиц есть `for ... in ipairs(a)` и `for ... in pairs(t)`.

lua/04_language/loops_sum.lua

```lua
-- суммирование 1..10 числовым циклом for
local sum = 0
for i = 1, 10 do
  sum = sum + i
end
print(sum)  -- 55
```

Границы `for i = 1, 10` включительны: `i` принимает значения от 1 до 10. Необязательное третье число — шаг: `for i = 10, 0, -2` считает вниз через одно значение. Светодиоды линейки (29 штук, глава 4) нумеруются с нуля, поэтому их обходят циклом `for i = 0, 28 do`.

lua/04_language/loops_while.lua

```lua
-- while: увеличиваем x, пока он меньше 5
local x = 0
while x < 5 do
  x = x + 1
end
print(x)  -- 5

-- repeat ... until: тело выполняется минимум один раз
repeat
  x = x - 1
until x == 0
print(x)  -- 0
```

lua/04_language/loops_iterators.lua

```lua
-- ipairs: обход массива по порядку, умножаем элементы на 2
local a = {10, 20, 30}
for i, v in ipairs(a) do
  a[i] = v * 2
end
print(a[1], a[2], a[3])  -- 20  40  60

-- pairs: обход словаря, увеличиваем значения на 1
local t = {x = 1, y = 2}
for k, v in pairs(t) do
  t[k] = v + 1
end
print(t.x, t.y)          -- 2  3
```

Что происходит.

- `ipairs` перебирает массив по порядку и останавливается на первом «пропуске» (`nil`); `pairs` перебирает все ключи словаря, порядок не гарантирован.
- `break` прерывает цикл. Оператора `continue` в Lua нет — вместо него условие внутри тела.

Вечный цикл останавливает дрон

Цикл `while true do ... end` без выхода никогда не вернёт управление автопилоту: события перестанут приходить, таймеры — срабатывать. Повторяющиеся действия на дроне делают таймером `Timer.new` (глава 4), а не бесконечным циклом. Программу с `while true do` симулятор не запустит: уведомление «Запуск остановлен» объяснит почему. Другие вечные циклы (например, `while x > 0` с забытым изменением `x`) заранее не распознаются — следите за условием выхода сами.

lua/04_language/lua_platform_infinite_loop.lua

```lua
-- ОПАСНО! Этот код "повесит" выполнение скрипта
while true do
    print("Я завис")
end
```

ЗАДАНИЕ 3.5 · Повторы и списки

1. Напечатайте циклом `for` ровно три индекса: 0, 1, 2. Сначала решите, какими будут начало и конец счётчика.
2. Составьте список `signals` из трёх слов: «ожидание», «готовность», «завершение». Напечатайте его по порядку циклом `ipairs` вместе с номером элемента.
3. Посчитайте сумму чисел от 1 до 100 и найдите максимум в массиве `a = {3, 9, 2, 7}`.
4. Вычислите факториал `n = 6` циклом `while`.
5. Заполните таблицу `idx` значениями от 0 до 28 (как номера светодиодов), добавляя каждый в конец: `idx[#idx + 1] = i`. Напечатайте `#idx`. Что получится, если писать `idx[i] = i`, и почему?
6. Постройте массив из первых 10 чисел Фибоначчи (1, 1, 2, …) и напечатайте его через `table.concat(fib, " ")`.

Что должно получиться: 1) `0`, `1`, `2` в трёх строках (`for i = 0, 2`); 2) `1 ожидание`, `2 готовность`, `3 завершение`; 3) `5050` и `9`; 4) `720`; 5) `29`; при `idx[i] = i` — `28`: элемент с ключом 0 в длину массива не входит, она считается с единицы; 6) `1 1 2 3 5 8 13 21 34 55`.

В симуляторе: миссии Lua: 24, 27 в панели «**Обучение**».

3.8 Функции

Функция — именованный блок кода, который принимает параметры и возвращает результат. Объявляется словом `function`, заканчивается `end`, результат отдаёт `return`.

lua/04_language/lua_functions.lua

```lua
-- функция с параметрами и одним результатом
local function average(a, b)
  return (a + b) / 2
end
print(average(1, 2))  -- 1.5
print(average(1, 3))  -- 2.0

-- функция может вернуть несколько значений сразу
local function describe(x)
  local color = (x > 0) and "green" or "red"
  local word = (x == 0) and "zero" or "nonzero"
  return color, word
end
print(describe(0))    -- red  zero
```

Что происходит.

- `local function average(a, b)` — локальная функция: видна только в этом файле ниже объявления. Вызов `average(1, 2)` подставляет `a = 1`, `b = 2` и возвращает `1.5`.
- Функция может вернуть несколько значений: `return color, word`. Так устроены датчики дрона: `local x, y, z = Sensors.lpsPosition()` возвращает три координаты сразу (все датчики — в приложении A).
- Функцию можно передать как значение. Например, `Timer.callLater(3, blink)` просит вызвать функцию `blink` через 3 секунды (таймеры разобраны в главе 4): сюда передаётся сама функция `blink`, а `Timer.callLater(3, blink())` передал бы результат её вызова. Второе — ошибка, симулятор её ловит.
- Если функция нужна один раз, ей не дают имени: `function() ... end` записывают прямо в месте вызова, например `Timer.callLater(3, function() print("прошло 3 с") end)`. Такая «безымянная» функция встретится во многих примерах следующих глав.

Функция, которую вызываем не мы

Обычно функцию вызывает наша программа. Но есть функция, которую вызывает **автопилот**:

Обработчик событий

```lua
function callback(event)
  -- сюда автопилот передаёт событие:
  -- взлёт завершён, точка достигнута, дрон сел...
end
```

Мы только определяем её — глобально, без `local`, ровно с таким именем. Когда произойдёт событие, автопилот сам вызовет `callback` и передаст номер события. Что с

ним делать, разбирают главы 4 и 5; пока запомните: **в каждой программе, которая командует автопилотом, есть `callback`**, даже пустой, — так написаны все официальные примеры Геоскан. Программам, которые только зажигают светодиоды, он не нужен.

ЗАДАНИЕ 3.6 · Свои функции

1. Напишите функцию `signal()` без параметров, которая печатает `"зелёный"`, и вызовите её один раз.
2. Напишите `report(brightness)`, печатающую `"яркость: "` и переданное значение. Вызовите её с 0.25 и с 0.5.
3. Напишите `square(x)` и `isEven(n)`; напечатайте `square(5)`, `isEven(7)`.
4. Напишите `clamp(v, lo, hi)`, ограничивающую `v` диапазоном `[lo, hi]`; проверьте на `clamp(1.7, 0, 1)` и `clamp(-3, 0, 1)`.
5. Напишите `polar(r, deg)`, возвращающую **два** значения `x, y` точки окружности радиуса `r` под углом `deg` градусов; напечатайте `polar(2, 90)`.

Что должно получиться: 1) одна строка `зелёный`; 2) `яркость: 0.25` и `яркость: 0.5`; 3) `25` и `false`; 4) `1` и `0`; 5) два числа: первое очень близко к нулю (около `1.2e-16`, погрешность вычислений), второе `2.0`.

В симуляторе: миссии Lua: 25–26 в панели «**Обучение**».

3.9 Типичные ошибки и их сообщения

Консоль Pioneer Station показывает исходный текст ошибки Lua и номер строки. Симулятор переводит ту же ошибку в уведомление на русском (колокольчик в заголовке редактора и вкладка «**Проблемы**» нижней панели): «Нет функции Print», «Не хватает end», «Обращение к пустому значению» — тоже с номером строки. Самые частые ошибки:

Сообщение	Причина и лечение
`attempt to call a nil value (global 'Print')`	Регистр букв: `Print`, `PRINT` и `print` — три разных имени. Все функции языка пишутся строчными; с заглавной — только модули `Ledbar`, `Timer`, `Sensors`, `Ev`.
`attempt to index a nil value`	Обращение к полю переменной, в которой `nil`: таблица не создана или опечатка в имени. Проверьте объявление и используйте охранное условие.
`attempt to compare nil with number`	Переменная не получила значения до сравнения — например, опечатка в имени или `local` внутри другого блока.
`'end' expected`	Не хватает `end`: у каждого `if`, `for`, `while`, `function` свой `end`. Выравнивайте код отступами — так пропуск виден.
`unexpected symbol near '*'`	Две звёздочки подряд: оператора `**` в Lua нет. Степень — `^`.

Индексы: с нуля или с единицы

Таблицы Lua нумеруются с **1**: `points[1]` — первая точка. Светодиоды — аппаратный модуль, они нумеруются с **0**: `leds:set(0, ...)` — первый светодиод. Путаница между ними — самая частая ошибка следующей главы.

ЗАДАНИЕ 3.7 · Одна ошибка за раз

Возьмите пример `lua_functions.lua` и сломайте его по очереди тремя способами, каждый раз возвращая исправную версию: замените `print` на `Print`; удалите `end` первой функции; замените `average(1, 2)` на `averge(1, 2)`. Запускайте после каждой поломки и прочитайте уведомление. Затем внесите две ошибки сразу и исправляйте их по одной, начиная с той, о которой сообщили первой.

Что должно получиться: каждое уведомление называет причину и номер строки: для `Print` и `averge` — что такой функции нет, для удалённого `end` — что его не хватает. При двух ошибках сначала сообщается одна — исправив её и запустив снова, вы увидите вторую.

В симуляторе: миссии Lua: 8 в панели «**Обучение**».

3.10 Проверьте себя

1. Что напечатает `print(10 / 2)`?

   (a) `5`
   (b) `5.0`
   (c) `5,0`

   Ответ: (b) — деление всегда даёт вещественное число.

2. Какое из значений ложно в условии `if`?

   (a) `0`
   (b) `""`
   (c) `nil`

   Ответ: (c) — ложны только `false` и `nil`.

3. Чему равно `#{10, 20, 30}`?

   (a) `2`
   (b) `3`
   (c) `30`

   Ответ: (b).

4. Сколько раз выполнится тело `for i = 0, 28 do`?

   (a) 28
   (b) 29
   (c) 27

   Ответ: (b) — обе границы включительно.

5. Какой из вызовов передаёт функцию, а не результат её вызова?

(a) `Timer.callLater(2, blink())`
(b) `Timer.callLater(2, blink)`

Ответ: (b).

6. Что делает `x, y = y, x`?
   (a) Ошибка
   (b) Меняет значения местами
   (c) Присваивает обеим `y`

   Ответ: (b).

Итоги главы

- Переменные объявляются через `local`; глобальные — только для долгоживущего состояния.
- Таблица — массив (с 1) и словарь одновременно; цвета и точки маршрута — таблицы.
- Ложны только `false` и `nil`; `cond and A or B` заменяет тернарный оператор.
- Циклы `for`, `while`, `repeat`; бесконечный цикл на дроне недопустим — вместо него таймер.
- Функции возвращают несколько значений и передаются как аргументы; `callback(event)` вызывает автопилот.

ГЛАВА 4

Светодиоды и таймеры

Цели главы

- Понять, как устроена светодиодная линейка Пионера и почему её адреса — `0..28`.
- Задавать цвет через модуль `Ledbar` и модель RGB.
- Делать анимации на таймерах `Timer.new` и `Timer.callLater`, не останавливая программу.
- Реагировать на события автопилота в `callback(event)`.
- Строить индикацию как конечный автомат: состояние → событие → переход.

Миссии симулятора к главе

В панели «**Обучение**» выберите язык «**Lua**».

- **10–18 «Сигналы и подсветка»**: первый красный свет, номер светодиода, зелёный и синий, смешиваем белый, выключаем сигнал, свет и подтверждение, цвет как переменная, таблица сигналов, световой отчёт подготовки. Закрепляют разделы «Аппаратная часть» и «Модуль Ledbar»; миссия 18 — раздел «События автопилота».
- **28–36 «Время и события»**: пауза делает цвет видимым, отложенный сигнал, два таймера, повторяющийся ритм, получаем уведомление, не реагируем на всё подряд, помним текущий этап, ждём не бесконечно, готовность без угадывания. Закрепляют разделы «Таймеры» и «События автопилота».

Дрон в этих миссиях не взлетает: самая «полётная» команда здесь — подготовка двигателей `ap.push(Ev.MCE_PREFLIGHT)` (в миссиях её называют arm), после которой дрон остаётся на земле.

4.1 Зачем дрону светодиоды

Светодиоды — первый видимый результат программы: дрон стоит на земле, а цвет меняется. Поэтому с них начинают. Но это ещё и рабочий инструмент: по цвету понятно, в каком состоянии программа — готовится, летит, садится, сообщает об ошибке, — даже когда терминала нет рядом. В главе 5 мы будем подсвечивать этапы миссии именно так.

В этой главе дрон не взлетает — кроме последнего примера, «Индикация этапов полёта», который служит мостом к следующей главе. Остальные примеры безопасно запускать и в симуляторе, и на настоящем Пионере со снятыми пропеллерами.

4.2 Толковый словарь

Ledbar модуль управления RGB-светодиодами: `Ledbar.new(N)` создаёт объект, `leds:set(i, r, g, b)` задаёт цвет.

RGB три компоненты цвета — красная, зелёная, синяя — числа от `0.0` (выключено) до `1.0` (максимум).

Индекс светодиода номер в линейке, от `0` до `N−1`.

Таймер «будильник», который вызывает функцию через заданное время (`Timer.callLater`) или периодически (`Timer.new`).

Тик один вызов функции таймера.

Событие сигнал от автопилота — взлёт завершён, точка достигнута, батарея разряжена. Передаётся в `callback(event)` как значение `Ev.*`.

Состояние текущий режим логики программы, например `"RED"` или `"FLIGHT"`; хранится в переменной.

Конечный автомат программа, которая всегда находится ровно в одном состоянии и переходит в другое только по событию.

Блокирующий вызов функция, которая останавливает всю программу, пока не завершится, например `sleep(t)`.

4.3 Аппаратная часть: 4 + 25 светодиодов

На плате Пионера четыре RGB-светодиода — по одному у каждого луча. Модуль светодиодной матрицы добавляет ещё 25. Программа видит их как одну линейку: индексы `0..3` — светодиоды платы, `4..28` — модуль. Всего 29, поэтому в курсе линейка всегда создаётся как `Ledbar.new(29)`: на дроне без модуля лишние индексы просто ни на что не влияют, а в симуляторе отображаются все 29.

Нумерацию индексов показывает рисунок 9.

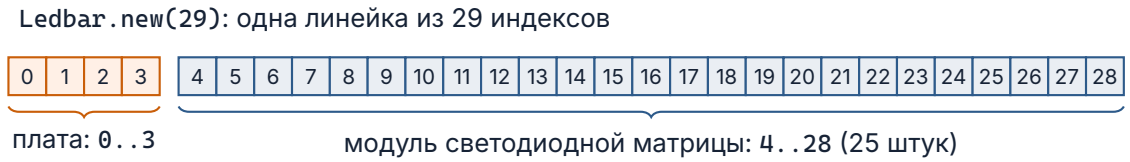
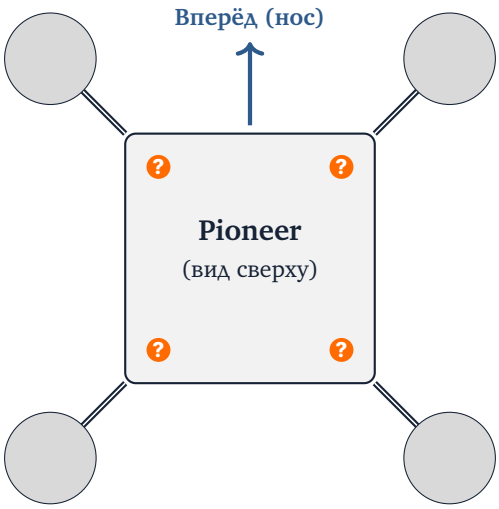


Рис. 9. Нумерация светодиодов в программе. Это нумерация индексов, а не схема расположения: какой индекс у какого угла платы, выясняется опытом.



Четыре светодиода платы получают индексы `0..3`, модуль матрицы — `4..28`. Какой индекс у какого угла, выясняет задание ниже.

Модель RGB

Цвет — три числа от `0.0` до `1.0`: красный, зелёный, синий. Смешивая каналы, получаем любой оттенок:

`{1, 0, 0}`	красный	`{0, 1, 0}`	зелёный
`{0, 0, 1}`	синий	`{1, 1, 0}`	жёлтый
`{1, 0, 1}`	фиолетовый	`{0, 1, 1}`	бирюзовый
`{1, 1, 1}`	белый	`{0, 0, 0}`	выключен

Значения меньше единицы — тусклее: `{0.2, 0, 0}` — тёмно-красный. В Python светодиодами управляет метод `led_control` библиотеки `pioneer_sdk`, и компоненты там — целые числа `0..255` (глава 7); в Lua — только `0..1`. Число больше единицы симулятор не обрезает и не считает ошибкой: `leds:set(0, 255, 0, 0)` даст пересвеченное красное пятно, а не «максимум по шкале 0..255».

ЗАДАНИЕ 4.1 · Цвет в уме

Не запуская программу, назовите цвет для каждой тройки: `{0, 1, 0}`, `{1, 1, 1}`, `{0.5, 0.5, 0.5}`, `{0, 0, 0}`, `{1, 0, 1}`. Какая тройка даёт чистый синий? Как сделать белый вдвое тусклее, не меняя его оттенка?

Что должно получиться: зелёный, белый, белый вдвое тусклее (серый), выключено, фиолетовый; синий — `{0, 0, 1}`; тусклый белый — все три составляющие уменьшить одинаково, например до `0.5`.

В симуляторе: миссии Lua: 12–13 в панели «**Обучение**».

4.4 Модуль Ledbar

Две функции, которых достаточно

- `local leds = Ledbar.new(29)` — создать объект линейки из 29 светодиодов. Вызывается один раз в начале программы.
- `leds:set(i, r, g, b)` — задать цвет светодиода `i` (от `0` до `28`). Обратите внимание на двоеточие: `set` — метод объекта `leds`.

lua/05_leds/leds_basic_green.lua

```lua
local unpack = table.unpack
local ledNumber = 29                    -- размерность буфера
local leds = Ledbar.new(ledNumber)      -- объект управления лентой

local function changeColor(col)
  for i=0, ledNumber-1 do               -- индексы 0..28
    leds:set(i, unpack(col))            -- записываем RGB
  end
end

changeColor({0,1,0})                    -- включаем зелёный
```

Что происходит. Функция `changeColor` принимает таблицу `{r, g, b}`, а `unpack` раскладывает её в три аргумента `leds:set`. Цикл `for i = 0, ledNumber - 1` обходит все 29 индексов. Один вызов `changeColor({0, 1, 0})` — и вся линейка зелёная.

Индекс вне диапазона не вызывает ошибку

`leds:set(29, ...)` на линейке из 29 элементов молча игнорируется: светодиод не загорится, а программа продолжит работать. Если какой-то светодиод «не светится», первым делом проверьте индекс. То же с длиной линейки: на `Ledbar.new(4)` адресуются только светодиоды платы `0..3`, а матрица `4..28` остаётся тёмной.

Исследование: какой индекс где

lua/05_leds/leds_addressing_research.lua

```lua
local leds = Ledbar.new(29)

-- Функция для очистки (выключения всех)
local function clear()
    for i=0, 28 do leds:set(i, 0,0,0) end
end

clear()
-- Включаем индекс 0
leds:set(0, 1, 0, 0)
-- Если загорелся не тот, который вы ожидали — поменяйте 0 на другой индекс
↪  от 0 до 28
```

ЗАДАНИЕ 4.2 · Карта светодиодов

Запустите программу: загорится светодиод с индексом `0`. Посмотрите на дрон (в симуляторе — камерой «**Дрон**»): какой это угол? Замените `0` на `1`, `2`, `3` и заполните таблицу. Если есть модуль матрицы — продолжите до `28`.

Что должно получиться: при каждом запуске горит ровно один светодиод; индексы `0..3` — на корпусе, `4..28` — на матрице (в симуляторе она есть всегда).

В симуляторе: миссии Lua: 10–11 в панели «**Обучение**».

Индекс	Расположение
0	.....
1	.....
2	.....
3	.....
4 …28	.....

Это упражнение учит главному принципу инженера: если документации недостаточно — проведи эксперимент.

ЗАДАНИЕ 4.3 · Сигналы по договорённости

1. Включите на светодиоде 0 чистый красный, а сразу следующей строкой выключите его. Что видно в сцене? Уберите выключение.
2. Заведите переменную `brightness = 0.5` и подставьте её в красную составляющую светодиода 0. После команды напечатайте `"красный включён"`.
3. Три светодиода — три сигнала: 0 — красное «ожидание», 1 — зелёная «готовность», 2 — синее «завершение». После каждой команды печатайте название сигнала.
4. Чётные светодиоды линейки — красные, нечётные — синие (подсказка: `i % 2 == 0`).
5. «Полоса загрузки»: светодиоды от 0 до `k` — зелёные, остальные выключены; `k` задаётся в начале программы.

Что должно получиться: 1) светодиод не загорается вовсе: между командами нет паузы, и видно только последнее состояние; 2) красный вполсилы и строка в журнале — строка подтверждает, что программа дошла до команды, а цвет проверяют по сцене; 3) три разных цвета и три строки по порядку; 4) красные и синие через один по всей линейке; 5) при `k = 9` горят ровно десять светодиодов.

В симуляторе: миссии Lua: 14–17 в панели «**Обучение**».

4.5 Время: почему не sleep

Чтобы цвет менялся, нужно время. Первое, что приходит в голову, — остановить программу на секунду функцией `sleep`:

lua/05_leds/leds_sleep.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)

local function fill(r, g, b)
  for i = 0, ledNumber - 1 do leds:set(i, r, g, b) end
end

fill(1, 0, 0)     -- красный
sleep(1)          -- на борту вся программа стоит секунду: ни событий, ни
↪  таймеров
fill(0, 1, 0)     -- зелёный
sleep(1)
fill(0, 0, 0)     -- выключить

function callback(event)
end
```

Запустите: красный, через секунду зелёный, ещё через секунду линейка гаснет. Работает. Но на борту Пионера, пока `sleep` «спит», программа не делает ничего другого: не принимает события автопилота, не обслуживает другие таймеры, не читает датчики. На земле это терпимо, в полёте — нет: дрон не узнает, что долетел до точки, и не отреагирует на разряд батареи.

Симулятор здесь мягче настоящего дрона: его `sleep` приостанавливает только основной текст программы, а события и таймеры продолжают приходить. Поэтому программа со `sleep` в симуляторе может вести себя «правильно» и сломаться на борту. Не полагайтесь на эту мягкость.

Разницу видно на оси времени (рисунок 10): две паузы по секунде одинаковы по длине, но во втором случае программа в это время свободна.

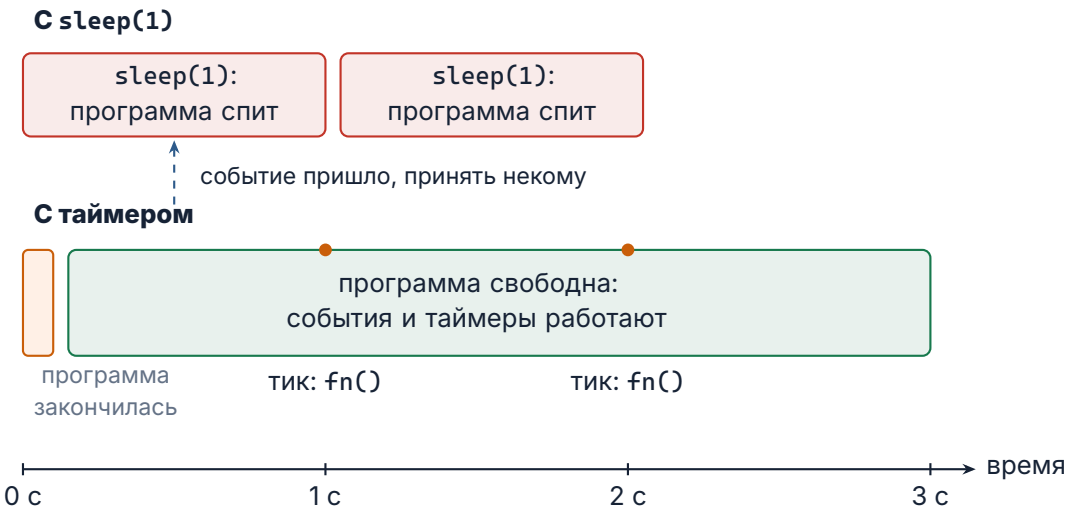


Рис. 10. Две паузы по секунде. Со `sleep` программа всё это время занята и не видит событий. С таймером она сразу свободна, а нужные действия автопилот вызывает сам в нужный момент.

Без sleep

В программах для дрона `sleep` не используется. Пауза — это таймер: «через t секунд сделай вот это», а программа тем временем свободна.

ЗАДАНИЕ 4.4 · Пауза делает цвет видимым

Уберите из `leds_sleep.lua` обе строки `sleep(1)` и запустите. Затем верните их и добавьте между зелёным и выключением жёлтый, тоже на секунду.

Что должно получиться: без пауз линейка сразу выключена: команда цвета не задаёт время показа, и глаз видит только последнее состояние. С паузами — красный, зелёный, жёлтый по секунде, затем темнота; всего около трёх секунд.

В симуляторе: миссии Lua: 28 в панели «**Обучение**».

4.6 Таймеры

API таймеров

- `Timer.callLater(delay, fn)` — вызвать `fn` один раз через `delay` секунд.
- `t = Timer.new(period, fn)` — создать периодический таймер: `fn` вызывается каждые `period` секунд, пока таймер запущен.
- `t:start()` и `t:stop()` — запустить и остановить периодический таймер. Сам по себе `Timer.new` таймер не запускает: без `t:start()` функция не вызовется ни разу.

Время всегда в секундах: `0.5` — полсекунды. Если нужен не «через t секунд», а «в такой-то момент по часам», есть `Timer.callAt` (приложение А).

Отложенное действие

lua/05_leds/leds_timers_delayed.lua

```lua
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
Timer.callLater(3, function ()
  for i=0, ledNumber-1 do leds:set(i, 0, 0, 1) end  -- включить синий
end)
function callback(event)
end
```

Программа заканчивается сразу, но через 3 секунды автопилот вызывает переданную функцию, и линейка загорается синим. Обратите внимание: в `callLater` передаётся *функция*, а не результат её вызова.

Периодическое действие: мигание

lua/05_leds/leds_timers_blink.lua

```lua
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local on = false
function updateBlink()
  local r,g,b = (on and 1 or 0), 0, 0
  for i=0, ledNumber-1 do leds:set(i, r, g, b) end
  on = not on
end
blinkTimer = Timer.new(0.5, updateBlink)  -- 0.5 секунды
blinkTimer:start()

function callback(event)
end
```

Что происходит. Переменная `on` хранит состояние мигалки. Каждые 0.5 с функция `updateBlink` сначала выставляет цвет по текущему состоянию (`on and 1 or 0` — единица, если `on` истинно), а затем меняет состояние на противоположное (`on = not on`) — на следующем тике цвет будет другим. Первый тик линейку гасит, второй зажигает красным, и так по кругу. Таймер `blinkTimer` объявлен глобально — так он живёт до конца программы, и его можно остановить из любой функции.

> **Не больше 16 ожидающих таймеров**
>
> По документации Геоскан, на борту одновременно может ждать не более 16 однократных таймеров (`callLater`): лишний просто не создастся, и действие молча не произойдёт. Симулятор такого лимита не проверяет, поэтому не создавайте десятки `callLater` подряд (например, в цикле) — заведите один периодический таймер или цепочку, где новый вызов ставится из предыдущего.

Один тик — один шаг

Функция таймера должна делать один короткий шаг анимации и возвращаться. Никаких циклов ожидания внутри: следующий шаг таймер вызовет сам.

ЗАДАНИЕ 4.5 · Отложить, повторить, пробежать

1. Через 1 с после запуска включите светодиод 0 зелёным и напечатайте `"зелёный"` — обе строки внутри функции, переданной в `callLater`.
2. Через 2 с после запуска включите зелёный, через 4 с — выключите. Оба `callLater` создайте сразу при старте.
3. Мигание: светлая часть 0.5 с, тёмная 0.5 с. Каков период полного повторения? Затем измените мигалку из примера на синюю с периодом таймера 0.2 с.
4. «Бегущий огонёк»: таймер с периодом 0.1 с зажигает ровно один зелёный светодиод, индекс которого растёт на 1 и после 28 возвращается к 0.

Что должно получиться: 1) через секунду зелёный и строка в журнале одновременно; 2) зелёный горит с 2-й по 4-ю секунду: второй таймер считает время от старта, а не от первого; 3) период — 1 с (свет плюс темнота); синий меняется пять раз в секунду; 4) огонёк обегает всю линейку за 2.9 с, одновременно горит только один светодиод.

В симуляторе: миссии Lua: 29–31 в панели «**Обучение**».

4.7 События автопилота

Таймер — это событие «прошло время». Автопилот умеет сообщать и о других событиях: моторы запущены, взлёт завершён, точка достигнута, дрон сел, батарея разряжена, удар. Для них есть одна функция:

Обработчик событий

```
function callback(event)
  if event == Ev.COPTER_LANDED then
    -- что сделать после посадки
  end
end
```

Мы её только определяем. Вызывает её автопилот — каждый раз, когда что-то произошло, — и передаёт в `event` код события. Коды хранятся в таблице `Ev`; сравнивать нужно именно с `Ev.ИМЯ`, а не с числом. Весь цикл — на рисунке 11: программа отправляет команду, автопилот её выполняет, о результате сообщает вызовом `callback`, а из него уходит следующая команда.

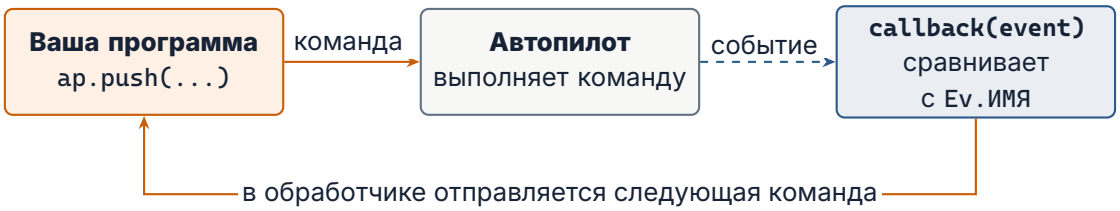


Рис. 11. Цикл «команда — событие»: автопилот выполняет команду, о завершении сообщает вызовом `callback`, а из него уходит следующая команда. Сплошная стрелка — команда, штриховая — событие.

Событие	Когда приходит
`Ev.ENGINES_STARTED`	моторы запущены: автопилот принял подготовку `ap.push(Ev.MCE_PREFLIGHT)`
`Ev.TAKEOFF_COMPLETE`	взлёт завершён, дрон висит на высоте взлёта
`Ev.POINT_REACHED`	достигнута точка, заданная `ap.goToLocalPoint`
`Ev.COPTER_LANDED`	дрон коснулся земли после посадки
`Ev.LOW_VOLTAGE1`, `Ev.LOW_VOLTAGE2`	батарея разряжена: первый порог — предупреждение, второй — пора садиться
`Ev.SHOCK`	удар или столкновение

Подготовка и подтверждение

Самое удобное событие для опытов на земле — `ENGINES_STARTED`. Его вызывает одна команда автопилоту, `ap.push(Ev.MCE_PREFLIGHT)` — подготовка к полёту (в миссиях симулятора её называют arm): моторы запускаются, дрон остаётся на месте. Команда — это просьба, событие — ответ: после `ap.push` подготовка ещё не подтверждена, а когда придёт `ENGINES_STARTED` — уже да. Без взлёта подготовка через несколько секунд истекает, и моторы останавливаются.

lua/05_leds/leds_events_preflight.lua

```lua
-- Готовность по подтверждению, а не по угадыванию
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local phase = "ожидание"                    -- текущий этап программы
local ready = false                         -- подготовка подтверждена?

local function fill(r, g, b)
  for i = 0, ledNumber - 1 do leds:set(i, r, g, b) end
end

fill(1, 0, 0)                               -- красный: ждём подтверждения
print("этап: " .. phase)
ap.push(Ev.MCE_PREFLIGHT)                   -- подготовка (arm): моторы
↪  запускаются, дрон стоит

function callback(event)
```

```
  if event == Ev.ENGINES_STARTED then     -- реагируем только на нужное
  ↪  событие
    ready = true
    phase = "готовность"
    fill(0, 1, 0)                         -- зелёный: подготовка принята
    print("этап: " .. phase)
  end
end

-- тайм-аут: ждём подтверждения не дольше 1.5 с
Timer.callLater(1.5, function()
  if not ready then
    fill(1, 1, 0)                         -- жёлтый: что-то пошло не так
    print("тайм-аут: подготовка не подтверждена")
  end
end)
```

Что происходит.

- Функция `callback` получает *все* события, поэтому первым делом проверяет, какое пришло: зелёный включается только по `ENGINES_STARTED`.
- Переменная `phase` помнит текущий этап: она объявлена в начале файла, а `callback` её меняет. Так одна программа по-разному ведёт себя до и после события.
- Таймер на 1.5 с — **тайм-аут**: если подтверждение не пришло, программа сообщает о проблеме, а не ждёт вечно. Фиксированная пауза сама по себе готовность не доказывает — доказывает флаг `ready`, поставленный событием.
- Повторный `ap.push(Ev.MCE_PREFLIGHT)`, когда подготовка уже идёт, — ошибка: автопилот уже на другом этапе, и симулятор сообщит «Подготовка уже запущена».

ЗАДАНИЕ 4.6 · Ответ на событие

1. Запустите пример. Затем удалите строку `ap.push(Ev.MCE_PREFLIGHT)` и запустите снова.
2. Уменьшите тайм-аут до 1 с и убедитесь, что при отправленной подготовке он не срабатывает.
3. Добавьте третий этап: через 3 с после подтверждения `phase` становится `"завершение"`, линейка — синей, в журнал печатается новый этап.

Что должно получиться: 1) красный, затем почти сразу зелёный и две строки «этап: ожидание», «этап: готовность»; дрон стоит на земле, винты крутятся. Без подготовки линейка остаётся красной, через 1.5 с желтеет, в журнале — строка «тайм-аут…»; 2) тайм-аут молчит; 3) три строки этапов по порядку и синий через 3 с после зелёного.

В симуляторе: миссии Lua: 18, 32–36 в панели «**Обучение**».

Реакция на посадку

lua/05_leds/leds_events_landed.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)

-- пока программа работает, линейка горит белым
for i = 0, ledNumber - 1 do leds:set(i, 1, 1, 1) end

function callback(event)
  if event == Ev.COPTER_LANDED then                          -- дрон сел
    for i = 0, ledNumber - 1 do leds:set(i, 0, 0, 0) end   -- погасить
    ↪ линейку
  end
end
```

Линейка горит белым всё время работы и гаснет, когда автопилот сообщит о посадке. В симуляторе проверить это можно так: добавьте в начало программы взлёт и посадку из главы 5 — или запустите готовый пример `leds_mission_stages.lua` из следующего раздела.

Реакция на разряд батареи

lua/05_leds/leds_events_low_voltage.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local on = false

local function fill(r, g, b)
  for i = 0, ledNumber - 1 do leds:set(i, r, g, b) end
end

-- штатная работа: мигаем зелёным два раза в секунду
blinkTimer = Timer.new(0.5, function()
  on = not on
  fill(0, on and 1 or 0, 0)
end)
blinkTimer:start()

function callback(event)
  if event == Ev.LOW_VOLTAGE2 then                        -- батарея разряжена
    blinkTimer:stop()                                     -- мигание больше не
    ↪ нужно
    Timer.callLater(1, function() fill(1, 0, 0) end)     -- через секунду р
    ↪ овный красный
  end
end
```

Что происходит. В штатном режиме таймер мигает зелёным. Событие `LOW_VOLTAGE2` останавливает таймер и через секунду заливает линейку красным. Пауза — страховка для настоящего Пионера: на борту уже запланированный тик таймера может сработать

ещё раз сразу после `stop()` и перекрасить линейку. В симуляторе `stop()` действует мгновенно, но программа, написанная с запасом, работает одинаково и там, и там.

Разряд батареи в симуляторе наступает сам, только в долгом полёте: событие `LOW_VOLTAGE1` приходит при напряжении около 7.2 В, `LOW_VOLTAGE2` — около 6.9 В. Ждать этого ради проверки индикации не нужно: `callback` — обычная функция, и её можно вызвать самим, например в конце программы:

Проверка без разряда батареи

```
Timer.callLater(2, function()
  callback(Ev.LOW_VOLTAGE2)
end)
```

Короткий вариант того же приёма, без таймера мигания. Останавливать здесь нечего, поэтому секундная пауза не обязательна — это просто отложенная реакция на событие:

lua/05_leds/leds_timers_callback_delayed.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
function callback(event)
  if (event == Ev.LOW_VOLTAGE2) then
    Timer.callLater(1, function ()
      for i=0, ledNumber-1 do leds:set(i, 1, 0, 0) end
    end)
  end
end
```

callback короткий и быстрый

Внутри `callback` — только смена состояния, цвет и команда автопилоту. Никаких `sleep` и длинных циклов: пока обработчик работает, следующие события ждут.

ЗАДАНИЕ 4.7 · Проверка аварийной индикации

Добавьте в `leds_events_low_voltage.lua` последней строкой вызов `callback(Ev.LOW_VOLTAGE2)` через 3 с (таймером, как показано выше). Затем добавьте в `callback` реакцию на `Ev.LOW_VOLTAGE1`: мигание продолжается, но жёлтым, — и проверьте её тем же способом.

Что должно получиться: первые 3 с линейка мигает зелёным, затем мигание прекращается, и через секунду линейка ровно красная. Во втором опыте после вызова с `LOW_VOLTAGE1` мигание становится жёлтым и не останавливается.

4.8 Три способа организовать программу

4.8.1 1. Последовательность с sleep

Действие, пауза, действие. Просто, но блокирует программу: в полёте недопустимо. Годится только для проверки на земле.

4.8.2 2. Таймеры

Каждая анимация — свой таймер, каждый тик — один шаг. Таймеры независимы: мигание навигационных огней и перелив матрицы работают одновременно, а программа остаётся свободной для событий.

4.8.3 3. Конечный автомат

Когда поведение зависит от того, «что уже произошло», одной переменной-флага мало. Конечный автомат — это:

- множество состояний (`"RED"`, `"YELLOW"`, `"GREEN"`);
- события (тик таймера, `Ev.POINT_REACHED`, тумблер пульта);
- правила переходов: «в состоянии A пришло событие X → перейти в B и сделать Y».

Светофор — самый маленький автомат:

lua/05_leds/leds_traffic_light.lua

```lua
local ledNumber = 29
local leds = Ledbar.new(ledNumber)

local function fill(color)
  for i = 0, ledNumber - 1 do leds:set(i, table.unpack(color)) end
end

-- состояния автомата и цвет каждого из них
local colors = {RED = {1, 0, 0}, YELLOW = {1, 1, 0}, GREEN = {0, 1, 0}}
-- таблица переходов: из какого состояния в какое
local nextState = {RED = "YELLOW", YELLOW = "GREEN", GREEN = "RED"}

state = "RED"                                -- текущее состояние (глобальная
↪  переменная)
fill(colors[state])

trafficTimer = Timer.new(2, function()       -- событие "прошло 2 секунды"
  state = nextState[state]                   -- переход в следующее состояние
  fill(colors[state])                        -- действие нового состояния
end)
trafficTimer:start()

function callback(event)
end
```

Что происходит. Таблица `nextState` — это и есть правила переходов, `colors` — действие в каждом состоянии. Таймер приносит событие «прошло 2 секунды», обработчик делает переход и действие. Добавить четвёртое состояние — значит добавить по строке в две таблицы, не трогая логику.

ЗАДАНИЕ 4.8 · Светофор с четырьмя состояниями

Добавьте в светофор состояние `"RED_YELLOW"` (красный с жёлтым; на линейке — оранжевый `{1, 0.5, 0}`) между красным и зелёным, как у настоящего светофора. Печатайте имя состояния при каждом переходе. Сколько строк пришлось изменить?

Что должно получиться: цикл красный → красный с жёлтым → зелёный → жёлтый → красный, по 2 с на состояние, в журнале имена состояний в том же порядке; изменились только строки таблиц `nextState` и `colors` (и одна строка `print`), логика таймера не тронута.

В симуляторе: миссии Lua: 34 в панели «**Обучение**».

4.9 Примеры комплексных сценариев

Перелив цвета и аварийная индикация

Этот скрипт — официальный пример Геоскан, слегка причёсанный:

lua/05_leds/leds_random_color_example.lua

```lua
-- https://learnxinyminutes.com/docs/ru-ru/lua-ru/ ссылка для быстрого
↪  ознакомления с основами языка LUA

-- Скрипт реализует плавный перелив цвета всей линейки и красный цвет при
↪  разряде батареи

-- Упрощение вызова функции распаковки таблиц из модуля table
local unpack = table.unpack
-- Количество светодиодов: 4 на плате + 25 на модуле матрицы
local ledNumber = 29
-- Создание порта управления светодиодами
local leds = Ledbar.new(ledNumber)

-- Функция, изменяющая цвет всех RGB светодиодов линейки
local function changeColor(col)
    for i=0, ledNumber - 1, 1 do
        leds:set(i, unpack(col))
    end
end

-- Функция, реализующая выключение таймера и изменение цвета светодиодов на
↪  красный
local function emergency()
    -- Остановка таймера
    timerRandomLED:stop()
```

```
    -- Изменение цвета светодиодов (через секунду - т.к. после остановки
    ↪  таймера timerRandomLED
    -- его функция выполнится еще раз)
    Timer.callLater(1, function () changeColor({1, 0, 0}) end)
end

-- Функция обработки событий, автоматически вызывается автопилотом
function callback(event)
    -- Вызов функции emergency() при низком напряжении на аккумуляторе
    if (event == Ev.LOW_VOLTAGE2) then
        emergency()
    end
end

color = {0, 0, 0}
step = 0.05
index = 1

-- Создание таймера: каждые 0.05 с он чуть сдвигает одну компоненту цвета
↪  (перелив)
timerRandomLED = Timer.new(0.05, function ()
    color[index] = color[index] + step
    if color[index] > 0.95 or color[index] < 0.05 then
        if index < 3 then
            index = index + 1
        else
            index = 1
            step = -step
        end
    end
    changeColor(color)
end)
-- Запуск созданного таймера
timerRandomLED:start()
```

Таймер каждые 0.05 с чуть сдвигает одну из компонент цвета — получается плавный перелив. Событие `LOW_VOLTAGE2` через функцию `emergency` останавливает таймер и включает красный. Два механизма — таймер и событие — работают вместе, не мешая друг другу.

Индикация этапов полёта

А это — мост к следующей главе. Программа взлетает, летит к точке и садится, а цвет линейки показывает этап:

lua/05_leds/leds_mission_stages.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)

local function fill(r, g, b)
  for i = 0, ledNumber - 1 do leds:set(i, r, g, b) end
end
```

```
fill(1, 1, 1)                                    -- белый: подготовка
ap.push(Ev.MCE_PREFLIGHT)
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end)

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE then
    fill(0, 1, 0)                                -- зелёный: летим к точке
    ap.goToLocalPoint(0, 0.5, 0.8)
  end
  if event == Ev.POINT_REACHED then
    fill(0, 0, 1)                                -- синий: садимся
    ap.push(Ev.MCE_LANDING)
  end
  if event == Ev.COPTER_LANDED then
    fill(0, 0, 0)                                -- сели: погасить
  end
end
```

Белый — подготовка, зелёный — полёт к точке, синий — посадка, выключено — дрон на земле. Команды `ap.push` и `ap.goToLocalPoint` разобраны в главе 5; здесь важно другое: каждая смена цвета стоит рядом с переходом на новый этап, и по дрону видно, где программа. Переменной `state` в этой программе нет: этапы идут строго друг за другом, и каждое событие (`TAKEOFF_COMPLETE`, `POINT_REACHED`, `COPTER_LANDED`) приходит ровно один раз, поэтому само событие однозначно говорит, какой этап начался. Когда одно и то же событие может прийти на разных этапах, понадобится явная переменная состояния — так устроены миссии в разделе 5.9.

ЗАДАНИЕ 4.9 · Свой сценарий подсветки

1. В `leds_random_color_example.lua` замените перелив на плавную пульсацию: яркость красного растёт от 0 до 1 шагами по 0.05 каждые 0.05 с, затем убывает, и так по кругу. Аварийная реакция на `LOW_VOLTAGE2` должна остаться.
2. В `leds_mission_stages.lua` добавьте жёлтый цвет на время взлёта (от команды взлёта до `TAKEOFF_COMPLETE`) и замените точку на `(0, 1, 0.8)`.

Что должно получиться: 1) линейка плавно разгорается и гаснет примерно за 2 с туда и обратно; вызов `callback(Ev.LOW_VOLTAGE2)` останавливает пульсацию и оставляет ровный красный; 2) порядок цветов: белый, жёлтый, зелёный, синий, выключено; дрон пролетает 1 м вдоль оси Y и садится.

4.10 Проверьте себя

1. Сколько светодиодов адресует `Ledbar.new(29)` и какие индексы допустимы?

   (a) 29, индексы 1..29
   (b) 29, индексы 0..28
   (c) 28, индексы 0..28

   Ответ: (b).

2. Как включить зелёный на всей линейке функцией `changeColor`?

(a) `changeColor({0,1,0})`
(b) `changeColor({0,255,0})`
(c) `changeColor(0,1,0)`

Ответ: (a) — таблица с компонентами 0..1.

3. Как включить синий через 3 секунды, не останавливая программу?

(a) `sleep(3); fill(0,0,1)`
(b) `Timer.callLater(3, function() fill(0,0,1) end)`
(c) `Timer.new(3, fill)`

Ответ: (b).

4. Лампа должна переключаться (гореть, гаснуть, снова гореть) каждые полсекунды. Какой период задать? `Timer.new(___, fn)`

(a) 0.5
(b) 500
(c) 50

Ответ: (a) — секунды.

5. Как остановить таймер `t`?

(a) `t:stop()`
(b) `stop(t)`
(c) `t.cancel()`

Ответ: (a).

6. Как называется функция, которую вызывает автопилот при событии?

(a) `onEvent(e)`
(b) `callback(event)`
(c) `handler(ev)`

Ответ: (b).

7. Какое событие соответствует посадке?

(a) `Ev.POINT_REACHED`
(b) `Ev.COPTER_LANDED`
(c) `Ev.LOW_VOLTAGE2`

Ответ: (b).

8. Что хранит переменная `state` в светофоре?

(a) Номер светодиода
(b) Текущее состояние автомата
(c) Период таймера

Ответ: (b).

4.11 Практика: проекты подсветки

Мелкие упражнения остались в уроках; здесь — проекты на всю главу. Близкие по смыслу решения (мигалка, стробоскоп, SOS, двоичный счётчик на четыре разряда и другие) лежат в `examples/lua-tasks/05_leds/` — загляните туда только после своей попытки. Каждый проект сдаётся по критериям: всё, что перечислено в «Что должно получиться», должно быть видно в сцене или в журнале.

ЗАДАНИЕ 4.10 · Полицейская мигалка и стробоскоп

Режим 1: чётные светодиоды красные, нечётные синие; каждые 0.3 с цвета меняются местами. Режим 2: белые вспышки — 0.05 с включено, 0.2 с выключено (два состояния с разной длительностью — это цепочка `callLater` из обработчика, а не один периодический таймер). Режимы переключаются сами каждые 5 с.

Что должно получиться: в сцене попеременно 5 с мигалки (соседние светодиоды всегда разного цвета) и 5 с белых вспышек, заметно более коротких, чем паузы; в журнале при смене режима — его название.

ЗАДАНИЕ 4.11 · Сигнал SOS

Три короткие вспышки (0.2 с), три длинные (0.6 с), три короткие; пауза между вспышками 0.2 с, между буквами 0.6 с, между повторами сигнала 2 с. Длительности хранятся в таблице, автомат помнит индекс текущей вспышки.

Что должно получиться: по сцене читаются три точки, три тире, три точки с отчётливыми паузами между буквами; один сигнал без паузы повтора длится 5.4 с и повторяется; длительности меняются правкой одной таблицы.

ЗАДАНИЕ 4.12 · Двоичный счётчик секунд

Раз в секунду на светодиодах `0..5` отображается число секунд с запуска в двоичном виде: младший разряд — светодиод 0, единица — зелёный, ноль — выключено. Одновременно печатайте число в журнал.

Что должно получиться: на 5-й секунде горят светодиоды 0 и 2 (`101`), на 6-й — 1 и 2; после 63 счётчик возвращается к нулю; число в журнале совпадает с картинкой.

ЗАДАНИЕ 4.13 · Авария

Линейка в штатном режиме переливается (как в `leds_random_color_example.lua`). По `Ev.LOW_VOLTAGE2` все таймеры останавливаются, линейка горит ровным красным; по `Ev.SHOCK` — мигает красным с периодом 0.2 с. Для проверки вызовите `callback` сами по таймеру: через 3 с с `Ev.SHOCK`, в другом запуске — с `Ev.LOW_VOLTAGE2`.

Что должно получиться: первые 3 с — перелив; после «удара» — только красное мигание, перелив не возвращается; после «разряда» — ровный красный без мигания. В журнале — строка с названием аварии.

В симуляторе: миссии Lua: 33 в панели «**Обучение**».

Итоги главы

- Линейка — 29 светодиодов с индексами `0..28`; цвет — три числа `0..1`.
- `Ledbar.new(29)` один раз, `leds:set(i, r, g, b)` сколько угодно; обход — `for i = 0, 28`.
- Паузы — только таймерами: `callLater` для одного действия, `Timer.new` для повторяющихся.
- События автопилота приходят в `callback(event)`; сравнивать с `Ev.ИМЯ`.
- Сложное поведение — конечный автомат: переменная состояния плюс таблица переходов.

ГЛАВА 5

Автономные миссии

Цели главы

- Знать жизненный цикл миссии: предстарт → взлёт → точки → посадка.
- Отдавать команды автопилоту (`ap.push`, `ap.goToLocalPoint`) и дожидаться их выполнения по событиям.
- Понимать систему координат сцены и задавать точки в метрах.
- Летать по фигурам: прямая, многоугольник, окружность — точками и таймером.
- Строить миссию как конечный автомат с аварийными реакциями и запуском с пульта.
- Проверять результат измерением: читать положение, сравнивать с целью с допуском, называть причину остановки.

Миссии симулятора к главе

В панели «**Обучение**» выберите язык «**Lua**».

- **37–45 «Высота и координаты»**: первая высота, три оси, знак направления, где начало, одна точка маршрута, высота над поверхностью, расстояние до цели, допуск прибытия, курс корпуса. Разделы «Жизненный цикл миссии» и «Координаты».
- **46–54 «Маршрут по этапам»**: подтверждаем точку, две точки, угол маршрута, замыкаем квадрат, список точек, возвращение домой, первая посадка, полный короткий полёт, повторяемость маршрута. Раздел «Ступени полёта».
- **55–63 «Полигоны и местность»**: один seed — одна карта, свободная стартовая зона, лесной коридор, холмы и высота, обход здания, промышленный участок, гоночный полигон, новый seed, маршрут через разные зоны. Раздел «Полигоны и местность».
- **64–72 «Измерения и устойчивость»**: читаем положение, как часто измерять, нет данных, достигли высоты, ошибка положения, не дёргаемся у границы, ограничиваем действие, причина остановки, отчёт полёта. Раздел «Измерения: цель, положение, допуск».

Окружности, конечные автоматы и запуск с пульта (вторая половина главы) идут дальше миссий: для них в книге есть свои задания и проекты.

5.1 Что такое миссия

Автономная миссия — заранее запрограммированная последовательность действий, которую дрон выполняет без участия пилота. Главное, что нужно понять в этой главе: автопилот Пионера — **не исполнитель команд по порядку**, а машина событий. Программа *отдаёт команду* («взлетай») и заканчивается. Когда автопилот выполнит команду, он *сообщит об этом событием* (`TAKEOFF_COMPLETE`) — и только тогда программа отдаёт следующую.

Поэтому программа миссии — это не список команд, а набор ответов на вопрос «что делать, когда произошло X». Ответы живут в `callback(event)`, и мы уже умеем его писать.

5.2 Толковый словарь

Автопилот (`ap`) модуль команд полёта: предстарт, взлёт, посадка, полёт к точке, курс.

Команда (`ap.push(Ev.MCE_*)`) просьба автопилоту сменить режим. Выполняется не мгновенно.

Событие (`Ev.*` в `callback`) сообщение автопилота о том, что команда выполнена или что-то произошло.

Локальная система координат оси X, Y на плоскости и Z вверх, начало — начало координат сцены (дрон по умолчанию стоит именно там), единицы — метры.

Телеметрия измеренные данные о дроне: `Sensors.lpsPosition()` — где он на самом деле, в отличие от цели, куда его послали.

Допуск насколько измерение может отличаться от цели, чтобы считать её достигнутой, например 0.15 м.

Курс (`yaw`) угол поворота носа дрона вокруг вертикальной оси, в радианах.

Счётчик шагов переменная с номером текущей точки маршрута: по ней программа понимает, какую точку только что достигла.

Failsafe аварийная реакция: что делать при ударе, разряде батареи, потере связи.

RC-каналы (`Sensors.rc`) значения стиков и тумблеров пульта, от -1 до 1.

5.3 Жизненный цикл миссии

Этапы любой миссии

1. **Инициализация** — переменные, таймеры, светодиоды. Дрон на земле.
2. **Предстарт** — `ap.push(Ev.MCE_PREFLIGHT)`: проверка датчиков, запуск моторов. Ответ — `Ev.ENGINES_STARTED`.
3. **Взлёт** — `ap.push(Ev.MCE_TAKEOFF)`: подъём на высоту взлёта (параметр автопилота `Flight_com_takeoffAlt`; в симуляторе дрон поднимается не меньше чем на 1 м над стартовой поверхностью). Подъём занимает время: отправленная команда ещё не значит, что взлёт закончен. Ответ — `Ev.TAKEOFF_COMPLETE`.

4. **Маршрут** — `ap.goToLocalPoint(x, y, z)` столько раз, сколько точек. Ответ на каждую — `Ev.POINT_REACHED`.
5. **Возврат** — ещё одна точка над стартом. Отдельной команды нет: координаты дома запоминают до подготовки, `local homeX, homeY = Sensors.lpsPosition()`, и в конце летят в `(homeX, homeY, z)`. Если дрон стартует из начала сцены, это просто `(0, 0, z)`.
6. **Посадка** — `ap.push(Ev.MCE_LANDING)`: снижение и касание. Ответ — `Ev.COPTER_LANDED`, моторы выключаются сами.

Команды и события парами

Команда	Что делает автопилот	Событие по завершении
`ap.push(Ev.MCE_PREFLIGHT)`	проверяет датчики, запускает моторы	`Ev.ENGINES_STARTED`
`ap.push(Ev.MCE_TAKEOFF)`	взлетает на высоту взлёта	`Ev.TAKEOFF_COMPLETE`
`ap.goToLocalPoint(x, y, z)`	летит в точку, метры	`Ev.POINT_REACHED`
`ap.updateYaw(rad)`	поворачивает нос, радианы	`---`
`ap.push(Ev.MCE_LANDING)`	снижается и садится	`Ev.COPTER_LANDED`
`ap.push(Ev.ENGINES_DISARM)`	немедленно выключает моторы	`---`

Те же пары во времени показаны на рисунке 12.

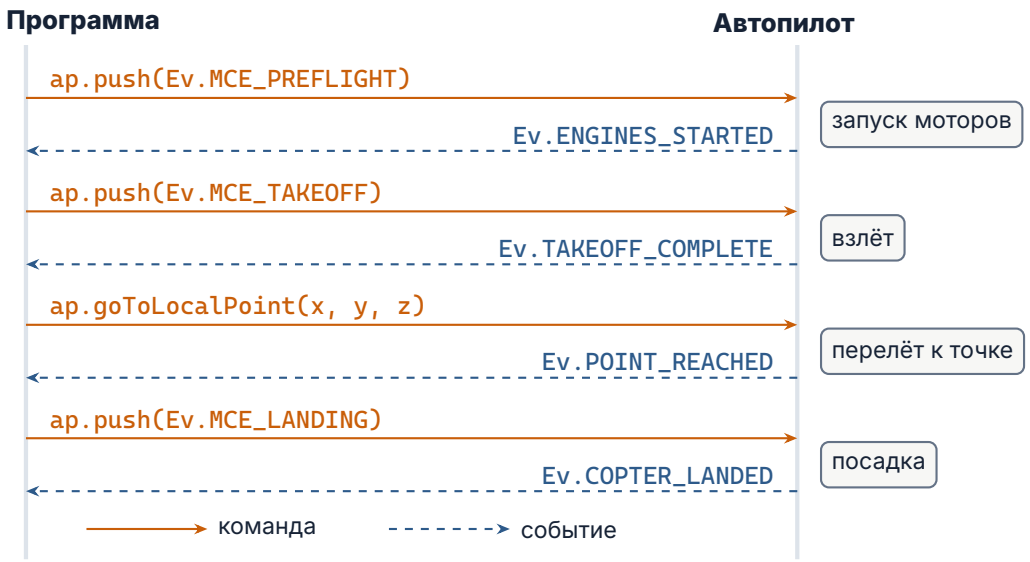


Рис. 12. Команды и события идут парами: каждая команда миссии завершается своим событием, и только после него отдаётся следующая.

> **ENGINES_DISARM в воздухе — это падение**
>
> Команда выключает моторы мгновенно, где бы дрон ни был. Её место — после `COPTER_LANDED` или в аварийной реакции на удар, когда дрон уже неуправляем. Для штатного завершения — только `MCE_LANDING`.

Две формы ожидания

Между предстартом и взлётом официальные примеры Геоскан выдерживают паузу таймером: `Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end)`. Можно и строже — взлетать по событию `ENGINES_STARTED`. Оба варианта работают; строгий надёжнее, потому что пауза сама готовность не доказывает (глава 4). Только не затягивайте: без взлёта подготовка через несколько секунд истекает (в симуляторе — через 8 с), и моторы останавливаются. В примерах книги чаще первый вариант, потому что он короче. Но для точек маршрута ожидание по времени не годится: сколько секунд займёт перелёт, заранее неизвестно, поэтому следующая точка — только по `POINT_REACHED`.

> **callback обязателен**
>
> Даже если программе не нужны события, объявите пустую `function callback(event) end`: так написаны все официальные примеры Геоскан, и без него события автопилота некому принять: на борту скрипт без `callback` может остановиться после первой команды миссии. Симулятор отсутствие `callback` отдельно не проверяет и программу выполнит, поэтому привычку придётся держать самим. Он предупредит лишь в одном случае: если команды миссии идут подряд без пауз — без таймеров и без `callback`.

ЗАДАНИЕ 5.1 · Первая высота

Напишите программу: подготовка; взлёт — по событию `ENGINES_STARTED`, а не по таймеру; по `TAKEOFF_COMPLETE` напечатайте `"Опыт завершён"` и отправьте посадку. Перед каждой командой печатайте её название.

Что должно получиться: дрон поднимается примерно на метр, зависает и садится; в журнале порядок: «подготовка», «взлёт», затем, только после подъёма, «Опыт завершён», «посадка». Строка «Опыт завершён» не появляется сразу после команды взлёта.

В симуляторе: миссии Lua: 37 в панели «**Обучение**».

5.4 Координаты: где точка

Точки задаются в **локальной системе координат** в **метрах**:

- оси **X** и **Y** лежат на плоскости земли, ось **Z** направлена вверх. На старте нос дрона смотрит вдоль положительной оси **Y** — поэтому в учебных миссиях «вперёд» значит

«по Y», а ось X идёт поперёк, вправо от носа. Положительные направления осей показывает компас камеры «**Земля**»: прежде чем лететь, найдите их там;

- начало координат $(0, 0, 0)$ — фиксированная точка сцены, а не место старта и не место последнего взлёта. Дрон по умолчанию стоит именно в начале, но если его перенести или дронов несколько, старт и начало различаются: `ap.goToLocalPoint(0, 0, 1)` ведёт в точку над началом сцены, а не над местом, откуда дрон взлетел;
- где дрон на самом деле, сообщает `Sensors.lpsPosition()` — три числа `x, y, z` в той же системе. На настоящем Пионере координаты даёт система позиционирования (ультразвуковая LPS или оптическая); в симуляторе — сцена.

Оси и курс на схеме — на рисунке 13, а как они выглядят в симуляторе — на следующем рисунке.

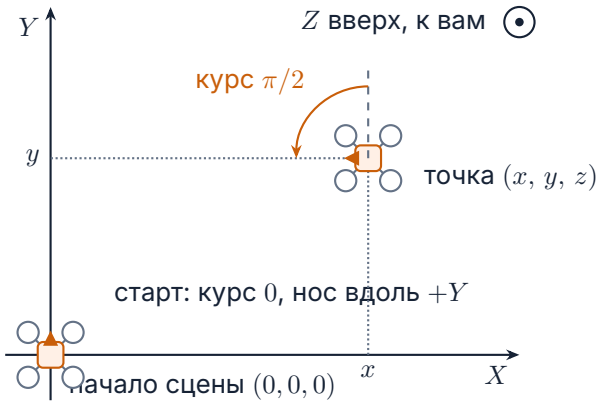


Рис. 13. Локальная система координат, вид сверху: X вправо, Y вперёд (вдоль носа на старте), Z вверх. Курс растёт против часовой стрелки: $\pi/2$ — нос смотрит влево.

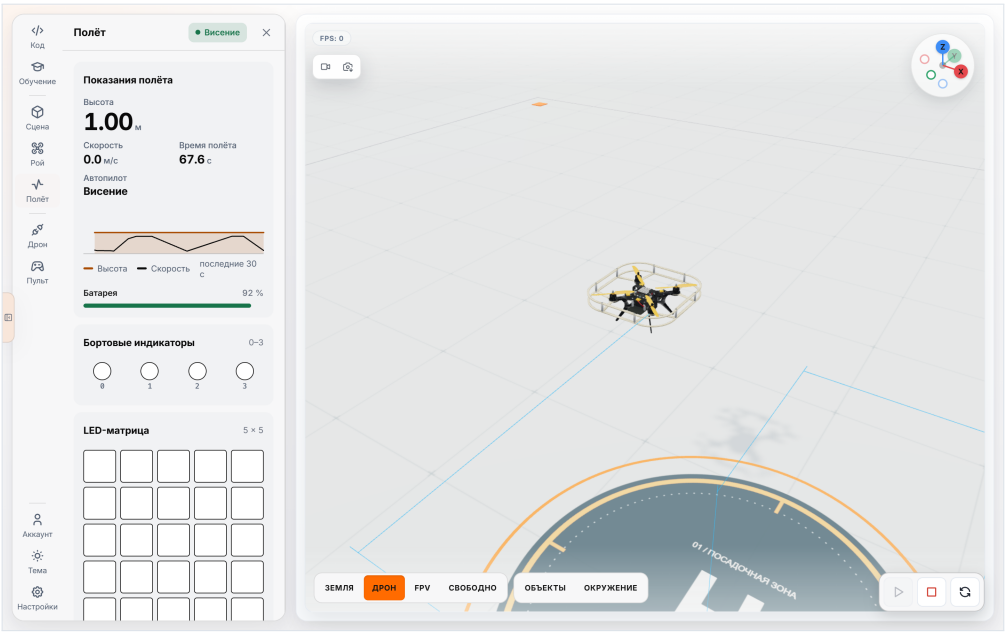


Рис. 14. Камера «**Земля**» — вид сверху, оси как на компасе в правом верхнем углу. `Ctrl`+щелчок по земле или объекту показывает координаты точки, которые можно подставить в `ap.goToLocalPoint`.

Z — это высота цели, а не поверхности

`Ctrl`+щелчок показывает высоту поверхности, по которой щёлкнули. Лететь нужно выше: над крышей высотой 13 м задавайте $z = 14$. Над ровной площадкой удобная учебная высота — от 0.7 до 1.5 м.

Высота над поверхностью

Z отсчитывается от плоскости сцены, а не от того, что под дроном. Над холмом высотой 0.5 м дрон на $z = 1$ висит всего в полуметре от склона. Расстояние до поверхности прямо под дроном измеряет дальномер: `Sensors.range()` возвращает его в метрах (`Sensors.tof()` — то же в миллиметрах). Одна и та же Z над ровной землёй и над холмом даёт разный просвет.

Курс

Курс (`yaw`) — поворот носа вокруг вертикальной оси. Автопилот принимает его в **радианах**: $\pi/2 \approx 1.57$ — четверть оборота влево, курс 0 — нос вдоль +Y, как на старте. Переводить градусы удобно функцией `math.rad(deg)`. Команда `ap.updateYaw(rad)` меняет курс, не меняя точку: корпус поворачивается, а дрон остаётся на месте. Её вызывают перед `goToLocalPoint`, чтобы дрон летел носом вперёд, или чтобы повернуть камеру к объекту.

ЗАДАНИЕ 5.2 · Разведка площадки

1. Включите камеру «**Земля**», найдите на компасе положительные направления X и Y и `Ctrl`+щёлкните по четырём точкам вокруг старта. Запишите их координаты — они понадобятся для маршрута в следующем разделе.
2. Не готовя двигатели, напечатайте положение дрона строкой `"Старт: "` и тремя числами из `Sensors.lpsPosition()`. Затем перенесите дрон в другое место сцены и запустите снова.
3. Прикиньте без запуска: на каком расстоянии друг от друга точки $(2, 0, 1)$ и $(-2, 0, 1)$? Где окажется дрон после `ap.updateYaw(math.pi)`?

Что должно получиться: 1) четыре записанные тройки чисел, у точек на ровной земле Z около нуля; 2) для дрона по умолчанию — «Старт:» и три числа около нуля, после переноса — другие X и Y: начало сцены осталось на месте; 3) 4 м, они по разные стороны от начала; после поворота на π дрон там же, где был, только нос смотрит в обратную сторону.

В симуляторе: миссии Lua: 39–40, 45 в панели «**Обучение**».

5.5 Ступени полёта

Каждая программа ниже — на одну ступень сложнее предыдущей. Запускайте каждую, смотрите камерой «**Дрон**» и читайте журнал на вкладке «**Вывод**».

Ступень 1. Только моторы

lua/06_missions/missions_engines_arm.lua

```lua
ap.push(Ev.MCE_PREFLIGHT)                  -- подготовка (arm): моторы
↪  запускаются, дрон стоит

function callback(event)                    -- обработчик событий
  if event == Ev.ENGINES_STARTED then       -- автопилот подтвердил подготовку
    print("Моторы запущены")
  end
end
```

Одна команда — подготовка: автопилот запускает моторы, дрон остаётся на земле, а подтверждение приходит событием `ENGINES_STARTED`. Так проверяют моторы на настоящем дроне со снятыми пропеллерами. В справочнике есть и команда `ap.push(Ev.ENGINES_ARM)` (приложение А); в симуляторе она делает то же, что `MCE_PREFLIGHT`, и повторять её после подготовки не нужно — автопилот уже на другом этапе и отклонит её. Без взлёта подготовка через 8 с истекает, и моторы останавливаются.

Ступень 2. Взлёт и посадка

lua/06_missions/mission_takeoff_landing.lua

```lua
ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function()              -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                  -- инициировать взлёт
end)

function callback(event)                    -- обработчик системных событий
  if event == Ev.TAKEOFF_COMPLETE then      -- взлёт завершён
    ap.push(Ev.MCE_LANDING)                 -- выполнить посадку
  end
  if event == Ev.COPTER_LANDED then         -- посадка завершена
    ap.push(Ev.ENGINES_DISARM)              -- выключить двигатели
  end
end
```

Что происходит. Предстарт → через секунду взлёт → по событию `TAKEOFF_COMPLETE` посадка → по `COPTER_LANDED` команда `ENGINES_DISARM`. После посадки автопилот и сам выключает моторы, так что эта строка — страховка; зато она показывает единственное безопасное место для `ENGINES_DISARM` в штатной миссии. Команда посадки только *начинает* снижение; закончилась посадка, когда пришло `COPTER_LANDED`. Четыре строки в `callback` — это вся миссия.

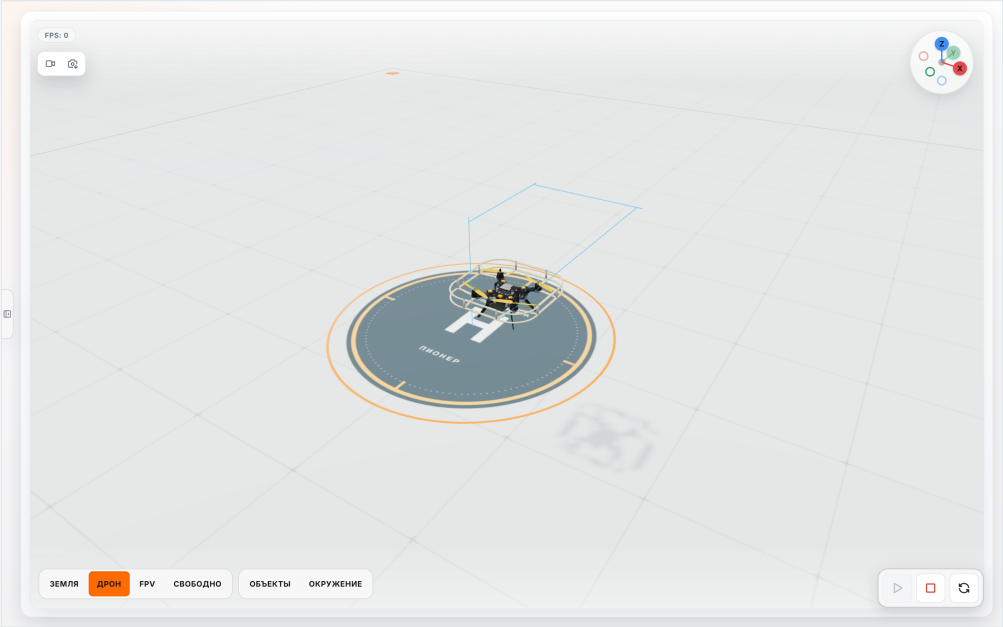


Рис. 15. Камера «**Дрон**» следует за дроном; голубая линия — след полёта.

Ступень 3. Полёт по прямой

lua/06_missions/missions_straight_flight.lua

```lua
ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function()              -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                  -- взлёт
end)

local unpack = table.unpack                -- сокращение доступа к распаковке
local target = {1.0, 0.0, 0.8}             -- целевая точка (x,y,z)

function callback(event)                    -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then      -- после взлёта
    ap.goToLocalPoint(unpack(target))       -- команда полёта к точке
  end
  if event == Ev.POINT_REACHED then         -- точка достигнута
    ap.push(Ev.MCE_LANDING)                 -- выполнить посадку
  end
end
```

После взлёта — точка $(1.0, 0.0, 0.8)$: метр по оси X (поперёк носа) на высоте 0.8 м. Точку отправляют только после `TAKEOFF_COMPLETE`: команда движения во время взлёта — ошибка «Маршрут слишком рано». Посадка — по `POINT_REACHED`, то есть когда дрон действительно там.

> **Посадка раньше точки**
>
> Частая ошибка — отправить точку и в том же обработчике, через таймер на пару секунд, — посадку. По умолчанию дрон летит со скоростью около 0.4 м/с (параметр `Copter_pos_vMax`): за две секунды он не долетит и метра, и посадка отменит

перелёт. Следующая команда — по следующему событию.

ЗАДАНИЕ 5.3 · Одна точка: оси и знаки

Меняйте в `missions_straight_flight.lua` только цель и каждый раз предсказывайте результат до запуска.

1. Цель $(0, 0, 2)$: меняется только Z.
2. Цель $(1, 1, 1)$. Затем $(-1, 1, 1)$: изменился только знак X.
3. Цель $(2, 0, 1)$; перед посадкой напечатайте `"Прибыл"` и положение из `Sensors.lpsPosition()`.

Что должно получиться: 1) дрон поднимается над стартом до 2 м и садится, не смещаясь по земле; 2) вторая точка зеркальна первой относительно оси Y — дрон уходит в другую сторону по X; 3) «Прибыл» появляется только после остановки у цели, а напечатанный X близок к 2, но не равен ему точно (например, 1.87 или 2.04).

В симуляторе: миссии Lua: 38–39, 41, 44 в панели «**Обучение**».

Ступень 4. Несколько точек и счётчик шагов

Каждая точка заканчивается одним и тем же событием `POINT_REACHED`. Чтобы знать, *какая* точка достигнута, нужен счётчик:

lua/06_missions/missions_points_flight.lua

```lua
local unpack = table.unpack                -- сокращение для распаковки
local points = {                           -- набор точек (маршрут)
  {0.0, 0.5, 0.8}, {0.5, 0.5, 0.8}, {0.5, 0.0, 0.8}, {0.0, 0.0, 0.8}
}
local curr = 1                             -- номер следующей точки маршрута

local function nextPoint()                 -- функция перехода к следующей
↪  точке
  if curr <= #points then                  -- если есть непройденные точки
    ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
    curr = curr + 1                        -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                -- иначе посадка
  end
end

ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function()              -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)                  -- взлёт
end)

function callback(event)                    -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end -- старт маршрута
  if event == Ev.POINT_REACHED then nextPoint() end    -- шаг по достижению
  ↪  точки
```

```
end
```

Что происходит. Маршрут — таблица точек, `curr` — номер следующей. Функция `nextPoint` отправляет точку с номером `curr` и увеличивает счётчик; когда точки кончились — садится. Её вызывают два события: `TAKEOFF_COMPLETE` (первая точка) и `POINT_REACHED` (все остальные). Координаты отделены от логики полёта: чтобы изменить маршрут, правят только таблицу. Это основной шаблон маршрута в курсе (рисунок 16); так же устроены официальные примеры Геоскан. Отправить вторую точку сразу вслед за первой нельзя: новая цель заменит прежнюю, и первую дрон не посетит.

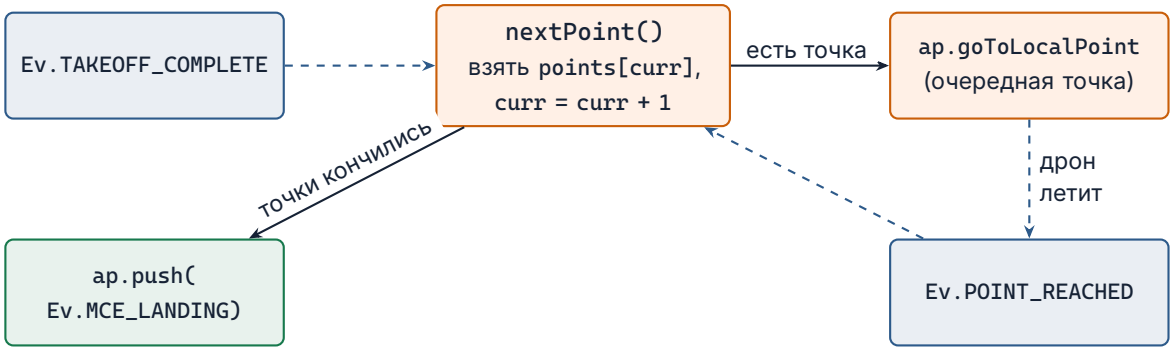


Рис. 16. Маршрут по таблице точек. Одна функция `nextPoint` вызывается по двум событиям: после взлёта (первая точка) и после каждой достигнутой точки (все остальные); когда точки кончились, она отправляет посадку.

Ступень 5. Фигуры из точек

Треугольник — три вершины на окружности радиуса r под углами $0°$, $120°$, $240°$: $x = r\cos a$, $y = r\sin a$.

lua/06_missions/missions_triangle_flight.lua

```lua
local unpack = table.unpack                 -- распаковка таблицы
local r = 0.6                               -- радиус описанной окружности
local z = 0.9                               -- высота полёта
local angles = {0, 120, 240}                -- углы вершин (градусы)
local points = {}                           -- массив координат вершин
for i=1, #angles do                         -- вычисление координат
  local a = angles[i] * math.pi / 180       -- радианы
  points[i] = {r*math.cos(a), r*math.sin(a), z} -- (x,y,z) вершины
end
local curr = 1                              -- текущая вершина
local function nextPoint()                  -- переход к следующей
  if curr <= #points then                   -- если есть вершины
    ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
    curr = curr + 1                         -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                 -- завершение — посадка
  end
end
```

```
ap.push(Ev.MCE_PREFLIGHT)                    -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)                      -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end     -- первый шаг
  if event == Ev.POINT_REACHED then nextPoint() end        -- шаг по событию
end
```

Квадрат со стороной a и центром в начале координат — вершины $(\pm a/2, \pm a/2, z)$:

lua/06_missions/missions_square_flight.lua

```
local unpack = table.unpack                  -- распаковка таблиц
local a = 1.0                                -- длина стороны квадрата
local z = 0.9                                -- высота полёта
local h = a/2                                -- половина стороны (координаты ±h)
local points = {                             -- вершины против часовой стрелки
↪  (вид сверху)
  {-h, -h, z}, {h, -h, z}, {h,  h, z}, {-h,  h, z}
}
local curr = 1                               -- индекс текущей вершины
local function nextPoint()                   -- переход к следующей вершине
  if curr <= #points then                    -- если есть вершины
    ap.goToLocalPoint(unpack(points[curr])) -- полёт к вершине
    curr = curr + 1                          -- увеличить индекс
  else
    ap.push(Ev.MCE_LANDING)                  -- завершение — посадка
  end
end
ap.push(Ev.MCE_PREFLIGHT)                    -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)                      -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then nextPoint() end     -- первый шаг
  if event == Ev.POINT_REACHED then nextPoint() end        -- шаг по
  ↪  достижению
end
```

Время перелёта

У `ap.goToLocalPoint` есть четвёртый аргумент — время перелёта в секундах: `ap.goToLocalPoint(1, 0, 0.8, 6)` долетит за 6 с. Без него скорость берётся из параметров автопилота.

ЗАДАНИЕ 5.4 · Маршрут по этапам

1. Угол: через вершину $(0, 2, 1)$ в $(2, 2, 1)$. Вершину нужно посетить, а не срезать.
2. Квадрат $(2, 0, 1) \to (2, 2, 1) \to (0, 2, 1) \to (0, 0, 1)$ таблицей точек в `missions_points_flight.lua`.
3. Возвращение домой: до подготовки запомните `local homeX, homeY = Sensors.lpsPosition()`, посетите $(0, 2, 1)$ и вернитесь в `(homeX, homeY, 1)`, затем сядьте. Проверьте, перенеся дрон в другое место сцены.

4. Свой маршрут из четырёх точек задания «Разведка площадки» с возвратом домой. Запустите его три раза подряд, не меняя код.

Что должно получиться: 1) след на сцене — прямой угол с вершиной в $(0, 2)$; 2) замкнутый квадрат, посадка в начале сцены; 3) дрон садится там, откуда взлетел, даже после переноса, а не в начале сцены; 4) три одинаковых следа: каждый запуск начинается со сброса сцены, поэтому из одинаковых условий маршрут повторяется.

В симуляторе: миссии Lua: 46–54 в панели «**Обучение**».

5.6 Полигоны и местность

Пустая площадка прощает всё. На полигоне маршрут проверяется местностью: деревьями, холмами, зданиями, мачтами, воротами. Готовые полигоны — в панели «**Сцена**», раздел «Готовые полигоны»: учебные лес, холмы, город, промышленная зона, экспедиция (смешанная местность), гоночная трасса и другие. В миссиях 55–63 нужная учебная сцена создаётся сама; её подпись — «Учебная сцена · урок N · seed S».

- **Seed** — число, из которого генератор получает расстановку объектов. Тот же seed — та же карта, поэтому трудный маршрут можно повторять и отлаживать. Другой seed — другая расстановка, и старые координаты обхода могут стать неверными: сначала осмотрите карту.
- **Размер дрона.** Координаты цели — это центр дрона, а винты выступают в стороны. Коридор между деревьями должен быть шире корпуса с запасом, стартовая зона — свободна вокруг винтов.
- **Высота до препятствия.** Высоту поднимают *до* того, как маршрут пересечёт холм или мачту: отдельной точкой над стартом, а не на ходу. Высоту препятствия узнают `Ctrl`+щелчком по нему.
- **Обход.** Если прямая к цели пересекает здание, путь строят из нескольких точек с запасом от фасада. Ворота проходят тремя точками — подход, центр проёма, выход, — иначе дрон зацепит раму, срезая угол.
- **Участки.** Длинный маршрут через разные зоны делят на участки, у каждого — своя высота и свои ограничения; так его легче проверять по частям.

ЗАДАНИЕ 5.5 · Маршрут по местности

Откройте полигон «Учебный: лес» и проложите маршрут на 8 м вперёд по Y через свободный коридор: подъём над стартом, точка в конце коридора, посадка. Затем на полигоне «Учебный: холмы» пройдите 12 м вперёд по Y, подняв высоту до начала пересечения холмов. Высоты выбирайте по `Ctrl`+щелчку.

Что должно получиться: в лесу след идёт по коридору, не задевая крон, событие `SHOCK` не приходит; на холмах дрон набирает высоту над стартом, потом летит горизонтально, и дальномер `Sensors.range()` в самой высокой точке холма показывает просвет больше нуля.

В симуляторе: миссии Lua: 55–63 в панели «**Обучение**».

5.7 Измерения: цель, положение, допуск

Команда сообщает, куда дрон *должен* прилететь; **телеметрия** — где он *на самом деле*. Это разные числа, и проверять миссию нужно по второму.

lua/06_missions/missions_telemetry.lua

```lua
-- Телеметрия: цель, измерение, ошибка, допуск и причина остановки
local target = {2, 0, 1}                  -- цель x, y, z, м
local tolerance = 0.15                    -- допуск прибытия, м
local timeout = 20                        -- предел ожидания перелёта, с
local arrived = false

local function report(label)              -- строка журнала: подпись и
↪  измерение
  local x, y, z = Sensors.lpsPosition()
  print(string.format("%s: %.2f, %.2f, %.2f", label, x, y, z))
end

-- раз в 0.5 с: ошибка по X = цель минус измерение
measureTimer = Timer.new(0.5, function()
  local x = Sensors.lpsPosition()         -- первое из трёх чисел — x
  print(string.format("Ошибка X: %.2f", target[1] - x))
end)

report("Старт")
ap.push(Ev.MCE_PREFLIGHT)
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end)

-- тайм-аут перелёта: цель не достигнута вовремя — называем причину и садимся
local function onTimeout()
  if not arrived then
    measureTimer:stop()
    print("тайм-аут: цель не достигнута за " .. timeout .. " с")
    ap.push(Ev.MCE_LANDING)
  end
end

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE then
    report("После взлёта")
    print(string.format("Цель: %g, %g, %g", target[1], target[2], target[3]))
    ap.goToLocalPoint(target[1], target[2], target[3])
    measureTimer:start()
    Timer.callLater(timeout, onTimeout)   -- отсчёт с начала перелёта
  elseif event == Ev.POINT_REACHED and not arrived then
    arrived = true
    measureTimer:stop()
    report("Прибытие")
    local x = Sensors.lpsPosition()
    if math.abs(target[1] - x) <= tolerance then  -- сравниваем с допуском, а
    ↪  не на равенство
      print("Цель достигнута")
    else
      print("Остановка вне допуска")
    end
    ap.push(Ev.MCE_LANDING)
```

```
  elseif event == Ev.COPTER_LANDED then
    report("После посадки")
  end
end
```

Что происходит.

- `report` печатает *измеренное* положение с подписью: до подготовки, после взлёта, по прибытии, после посадки. Цель печатается отдельной строкой — в журнале видно и намерение, и результат. Такой журнал — отчёт полёта: по нему находят первый этап, где поведение отклонилось.
- Таймер раз в 0.5 с печатает **ошибку** — цель минус измерение. Знак ошибки показывает, в какую сторону ещё лететь; к концу перелёта она уменьшается к нулю. Период опроса выбирают по задаче: слишком редкий пропустит быстрые изменения, слишком частый засорит журнал.
- Прибытие проверяют с **допуском**: `math.abs(target[1] - x) <= tolerance`. Измерение и управление неточны, поэтому `x == 2` почти никогда не выполняется.
- **Тайм-аут** ограничивает ожидание и называет причину остановки: «тайм-аут: цель не достигнута…» — не то же самое, что штатное «Цель достигнута».

Ещё три приёма из миссий 66–70:

- **Нет данных.** Отсутствующее измерение — `nil`, а не ноль. Сначала `if value == nil then print("Нет данных") return end`, и только потом сравнение с числом, иначе — ошибка «Сравнение с пустым значением». Координаты в симуляторе есть всегда, но так пишут любую обработку датчика.
- **Зона спокойствия.** Возле границы шум измерений переключает решение туда и обратно. Ошибку внутри `[-tolerance, tolerance]` считают нулём: решение «лететь дальше» принимают, только когда ошибка явно больше допуска.
- **Ограничение.** Рассчитанная величина — скорость, шаг, яркость — должна оставаться в допустимом диапазоне: `math.max(-limit, math.min(limit, v))`. Большая ошибка не означает неограниченную скорость.

ЗАДАНИЕ 5.6 · Отчёт по измерениям

1. Запустите пример и выпишите из журнала строки «Цель» и «Прибытие». Совпадают ли числа?
2. Замените период опроса 0.5 с на 0.2 с, потом на 2 с. Сколько строк «Ошибка X» получается за перелёт?
3. Уменьшите `timeout` до 3 с. Какая причина остановки появится в журнале?
4. Напишите функцию `zone(e, tol)`: 0 — если `e` внутри допуска, 1 — больше допуска, -1 — меньше минус допуска. Проверьте на ошибках -0.04, 0.03, -0.02, 0.2, -0.2 при допуске 0.1.

Что должно получиться: 1) не совпадают точно: прибытие отличается от цели на

сантиметры, но в пределах 0.15 м, поэтому печатается «Цель достигнута»; 2) при 0.2 с строк примерно в 10 раз больше, чем при 2 с, а ошибка в них убывает к нулю; 3) «тайм-аут: цель не достигнута за 3 с», и дрон садится, не долетев; 4) `0 0 0 1 -1`.

В симуляторе: миссии Lua: 64–72 в панели «**Обучение**».

5.8 Окружность: точки или таймер

Окружность радиуса r в плоскости XY задаётся параметрически:

$$x(\theta) = r\cos\theta, \qquad y(\theta) = r\sin\theta, \qquad z = \text{const},$$

где θ — угол в радианах. Если удобнее думать в градусах, переводите: $\theta = \text{deg}\cdot\pi/180$ или `math.rad(deg)`.

Есть два способа пролететь окружность.

Способ 1. Много точек

Разбить окружность на 12 точек через 30° и пройти их счётчиком шагов, как любой маршрут. Дрон будет притормаживать в каждой вершине — получится двенадцатиугольник. Так устроен пример с индикацией в разделе 5.9.

Способ 2. Таймер двигает цель

Каждые 0.1 с сдвигать угол на несколько градусов и отправлять новую точку. Дрон не успевает остановиться — цель всё время впереди — и летит плавно:

lua/06_missions/missions_circle_timer.lua

```lua
local r = 1.0                              -- радиус, м
local height = 1.0                         -- высота, м
local angle = 0                            -- текущий угол, градусы
local state = "CIRCLE"                     -- CIRCLE -> RETURN -> LANDING

-- дрон действительно в точке (x, y, z)? допуск 0.2 м
local function near(x, y, z)
  local px, py, pz = Sensors.lpsPosition()
  return math.sqrt((px - x)^2 + (py - y)^2 + (pz - z)^2) < 0.2
end

circleTimer = Timer.new(0.1, function()    -- тик каждые 0.1 с
  if state ~= "CIRCLE" then return end     -- лишний тик после stop() ничего
  ↪  не делает
  if angle >= 360 then                     -- круг пройден
    circleTimer:stop()
    state = "RETURN"
    ap.goToLocalPoint(0, 0, height)        -- возвращаемся в центр
    return
  end
```

```
  local rad = math.rad(angle)              -- градусы -> радианы
  ap.goToLocalPoint(r * math.cos(rad), r * math.sin(rad), height)
  angle = angle + 5                        -- шаг 5 град.: полный круг за 72
  ↪  тика
end)

ap.push(Ev.MCE_PREFLIGHT)
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end)

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE then
    circleTimer:start()
  end
  -- POINT_REACHED последней точки круга мог прийти уже после перехода в
  ↪  RETURN,
  -- поэтому садимся, только если дрон действительно над центром
  if event == Ev.POINT_REACHED and state == "RETURN" and near(0, 0, height)
  ↪  then
    state = "LANDING"
    ap.push(Ev.MCE_LANDING)                -- центр достигнут — садимся
  end
end
```

Что происходит.

- Таймер стартует только после `TAKEOFF_COMPLETE`: отправлять точки до взлёта нельзя.
- Пока дрон кружит, события `POINT_REACHED` могут приходить снова и снова: каждая новая цель заново ждёт прибытия, и событие приходит, как только дрон оказывается ближе 0.15 м к текущей цели. Такие события игнорируются: переменная `state` равна `"CIRCLE"`. Здесь впервые одно и то же событие означает разное в зависимости от этапа, поэтому программа хранит состояние и проверяет его вместе с событием: `event == Ev.POINT_REACHED and state == "RETURN"`. Подробно этот приём разобран в разделе 5.9.
- После 360° таймер останавливает сам себя, переводит автомат в `"RETURN"` и отправляет центр. Следующий `POINT_REACHED` в состоянии `"RETURN"` означает «центр достигнут», и только тогда посадка.
- Функция `near` сверяет с этим событием реальное положение дрона (`Sensors.lpsPosition()`, допуск 0.2 м). Событие о последней точке круга могло прийти чуть позже перехода в `"RETURN"`; без проверки программа приняла бы его за «центр» и посадила дрон на окружности. Правило: если событие могло опоздать, проверяй не только событие, но и то, что оно описывает.
- Первая строка таймера, `if state ~= "CIRCLE" then return end`, — защита: на борту Пионера тик, уже запланированный в момент `stop()`, может сработать ещё раз (глава 4), и без проверки он отправил бы лишнюю точку. В симуляторе `stop()` действует сразу, но проверка делает программу надёжной на обоих.

Курс по касательной

Чтобы дрон летел «носом вперёд», нос должен смотреть по касательной к окружности. Касательная к точке под углом θ (при обходе против часовой стрелки) повёрнута относительно оси X на $\theta + 90°$. Курс же отсчитывается от оси Y: при курсе 0 нос смотрит вдоль +Y, то есть уже повёрнут на $90°$ от X. Поэтому нужный курс — просто $\psi = \theta$. В том же тике, что и точка, обновляется курс:

lua/06_missions/missions_circle_yaw_timer.lua

```lua
local r = 0.3                               -- радиус окружности, м
local height = 0.7                          -- высота полёта, м
local step = 3                              -- шаг угла за тик, градусы
local angle = 0                             -- текущий угол точки, градусы
local state = "CIRCLE"                      -- CIRCLE -> RETURN -> LANDING

-- дрон действительно в точке (x, y, z)? допуск 0.2 м
local function near(x, y, z)
  local px, py, pz = Sensors.lpsPosition()
  return math.sqrt((px - x)^2 + (py - y)^2 + (pz - z)^2) < 0.2
end

circleTimer = Timer.new(0.1, function()     -- каждые 0.1 с: новая точка и
↪  новый курс
  if state ~= "CIRCLE" then return end      -- лишний тик после stop() не
  ↪  должен затереть возврат в центр
  angle = (angle + step) % 360
  local rad = angle * math.pi / 180         -- градусы -> радианы
  ap.updateYaw(rad)                         -- курс 0 = нос вдоль +Y, курс =
  ↪  угол точки: нос по касательной
  ap.goToLocalPoint(r * math.cos(rad), r * math.sin(rad), height)
end)

ap.push(Ev.MCE_PREFLIGHT)                   -- предстарт
Timer.callLater(2, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт через 2 с

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE then
    circleTimer:start()                     -- начинаем кружить
    Timer.callLater(12, function()          -- один круг: 360 / 3 шага * 0.1 с
    ↪  = 12 с
      circleTimer:stop()
      state = "RETURN"
      ap.goToLocalPoint(0, 0, height)       -- возвращаемся в центр
    end)
  end
  -- садимся только над центром: запоздалый POINT_REACHED точки круга не в
  ↪  счёт
  if event == Ev.POINT_REACHED and state == "RETURN" and near(0, 0, height)
  ↪  then
    state = "LANDING"
    ap.push(Ev.MCE_LANDING)
  end
end
```

Здесь круг ограничен временем: $360°/3° \cdot 0.1$ с $= 12$ с, после чего таймер останавли-

вается, дрон возвращается в центр и садится по событию. Длительность круга — как раз «пауза по смыслу»: сам автопилот о конце круга не сообщает. Та же проверка `state ~= "CIRCLE"` в начале таймера не даёт запоздалому тику затереть точку возврата в центр.

ЗАДАНИЕ 5.7 · Восьмёрка

Две окружности с центрами $(\pm r, 0)$, проходимые в противоположных направлениях. Подсказка: два состояния `"LEFT"` и `"RIGHT"`, шаг угла разного знака, переход — когда угол прошёл $360°$.

Что должно получиться: след на сцене — две касающиеся окружности с общей точкой в начале сцены; на второй дрон движется в обратную сторону; после второго круга — возврат в центр и посадка, `POINT_REACHED` во время кружения не сбивает автомат.

5.9 Архитектура миссии: конечный автомат

Три принципа

1. **События, а не время.** Следующая команда — по событию о завершении предыдущей. Таймер — только для пауз, которые нужны по смыслу.
2. **Ничего не блокировать.** Ни `sleep`, ни циклов ожидания: `callback` должен возвращаться мгновенно.
3. **Failsafe.** Удар (`Ev.SHOCK`) — выключить моторы; разряд (`Ev.LOW_VOLTAGE2`) — прекратить маршрут и сесть.

Когда у миссии несколько этапов, одной переменной-счётчика мало: одно и то же событие `POINT_REACHED` в состоянии «выхожу на окружность» и в состоянии «возвращаюсь в центр» означает разное. Поэтому программа хранит **состояние** и проверяет его вместе с событием. В светофоре (глава 4) событие было одно — тик таймера, — и правила переходов умещались в таблицу `nextState[state]`. В миссии событий несколько, и переход зависит от пары «состояние + событие»; поэтому правила записываются условиями вида `event == Ev.X and state == "Y"`.

Окружность по точкам с индикацией

lua/06_missions/missions_lesson12_circle_points.lua

```lua
-- Полёт по окружности из 12 точек с индикацией этапов
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local unpack = table.unpack

local function changeColor(color)
  for i = 0, ledNumber - 1 do leds:set(i, unpack(color)) end
end
```

```
local colors = {
  red = {1, 0, 0}, white = {1, 1, 1}, green = {0, 1, 0},
  blue = {0, 0, 1}, off = {0, 0, 0}
}

-- точки окружности: 12 штук через 30 град.
local r = 0.5
local height = 0.8
local points = {}
for i = 1, 12 do
  local a = math.rad(i * 30)
  points[i] = {r * math.cos(a), r * math.sin(a), height}
end

local curr_point = 1
local state = "PREPARE"                    -- PREPARE -> FLIGHT -> LANDING;
↪  при ударе EMERGENCY

local function nextPoint()
  if curr_point <= #points then
    ap.goToLocalPoint(unpack(points[curr_point]))
    curr_point = curr_point + 1
  else
    state = "LANDING"
    changeColor(colors.blue)               -- синий: садимся
    ap.push(Ev.MCE_LANDING)
  end
end

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE and state == "PREPARE" then
    state = "FLIGHT"
    changeColor(colors.green)              -- зелёный: маршрут
    nextPoint()
  end
  if event == Ev.POINT_REACHED and state == "FLIGHT" then
    nextPoint()
  end
  if event == Ev.COPTER_LANDED and state == "LANDING" then
    changeColor(colors.off)                -- сели: погасить
  end
  if event == Ev.SHOCK then
    state = "EMERGENCY"                    -- маршрут больше не продолжаем
    changeColor(colors.red)                -- авария: красный и выключить
    ↪  моторы
    ap.push(Ev.ENGINES_DISARM)
  end
end

changeColor(colors.red)                    -- красный: программа загружена
Timer.callLater(2, function()
  changeColor(colors.white)                -- белый: предстарт
  ap.push(Ev.MCE_PREFLIGHT)
end)
Timer.callLater(4, function() ap.push(Ev.MCE_TAKEOFF) end)
```

Что происходит. Состояния: PREPARE → FLIGHT → LANDING. Цвет линейки показывает этап: красный — программа загружена, белый — предстарт, зелёный — маршрут, синий — посадка, выключено — на земле. POINT_REACHED обрабатывается только в состоянии FLIGHT, поэтому случайное событие на посадке ничего не сломает; так же TAKEOFF_COMPLETE принимается только в PREPARE, а COPTER_LANDED гасит линейку только после штатной посадки. SHOCK в любом состоянии — красный, ENGINES_DISARM и переход в EMERGENCY: из него маршрут уже не продолжается, и красный цвет остаётся.

Непрерывная окружность с курсом

lua/06_missions/missions_lesson14_circle.lua

```lua
-- Непрерывный полёт по окружности: точка и курс обновляются таймером
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local unpack = table.unpack

local function changeColor(color)
  for i = 0, ledNumber - 1 do leds:set(i, unpack(color)) end
end

local colors = {
  red = {1, 0, 0}, white = {1, 1, 1}, yellow = {1, 1, 0},
  purple = {1, 0, 1}, blue = {0, 0, 1}, off = {0, 0, 0}
}

local r = 0.5                                -- радиус, м
local height = 0.8                           -- высота, м
local step = 3                               -- градусов за тик
local angle = 0
local state = "PREPARE"                      -- PREPARE -> TO_CIRCLE -> CIRCLE
↪  -> RETURN -> LANDING; при ударе EMERGENCY

local function near(x, y, z)                 -- дрон действительно в точке?
↪  допуск 0.2 м
  local px, py, pz = Sensors.lpsPosition()
  return math.sqrt((px - x)^2 + (py - y)^2 + (pz - z)^2) < 0.2
end

circleTimer = Timer.new(0.1, function()
  if state ~= "CIRCLE" then return end       -- лишний тик после stop() не
  ↪  должен затереть возврат в центр
  angle = (angle + step) % 360
  local rad = math.rad(angle)
  ap.updateYaw(rad)                          -- курс 0 = нос вдоль +Y, курс =
  ↪  угол точки: нос по касательной
  ap.goToLocalPoint(r * math.cos(rad), r * math.sin(rad), height)
end)

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE and state == "PREPARE" then
    state = "TO_CIRCLE"
    changeColor(colors.yellow)               -- жёлтый: выходим на окружность
    ap.goToLocalPoint(r, 0, height)
```

```
  elseif event == Ev.POINT_REACHED and state == "TO_CIRCLE" then
    state = "CIRCLE"
    changeColor(colors.purple)              -- фиолетовый: кружим
    circleTimer:start()
    Timer.callLater(12, function()          -- один круг: 360 / 3 шага * 0.1 с
      if state ~= "CIRCLE" then return end -- после аварии круг уже прерван
      circleTimer:stop()
      state = "RETURN"
      changeColor(colors.blue)              -- синий: возвращаемся в центр
      ap.goToLocalPoint(0, 0, height)
    end)
  elseif event == Ev.POINT_REACHED and state == "RETURN" and near(0, 0,
  ↪  height) then
    state = "LANDING"
    ap.push(Ev.MCE_LANDING)
  elseif event == Ev.COPTER_LANDED and state == "LANDING" then
    changeColor(colors.off)
  elseif event == Ev.SHOCK then
    state = "EMERGENCY"                     -- никакие таймеры и события
    ↪  больше не ведут маршрут
    circleTimer:stop()
    changeColor(colors.red)                 -- авария: красный и выключить
    ↪  моторы
    ap.push(Ev.ENGINES_DISARM)
  end
end

changeColor(colors.white)                   -- белый: предстарт
ap.push(Ev.MCE_PREFLIGHT)
Timer.callLater(2, function() ap.push(Ev.MCE_TAKEOFF) end)
```

Что происходит. Пять состояний: `PREPARE` → `TO_CIRCLE` (жёлтый, выход в точку $(r, 0)$) → `CIRCLE` (фиолетовый, таймер двигает цель и курс 12 с) → `RETURN` (синий, точка над центром) → `LANDING`. Цепочка `elseif` в `callback` — это и есть таблица переходов: каждая строка — «событие и состояние → действие». `SHOCK` переводит автомат в `EMERGENCY`; таймер окружности и отложенный конец круга (`Timer.callLater(12, ...)`) первым делом проверяют, что автомат всё ещё в `CIRCLE`, поэтому после аварии они не отправят точку и не перекрасят линейку. Посадка в `RETURN` — только когда `near(0, 0, height)` подтверждает, что дрон над центром (раздел 5.8). Схема переходов — на рисунке 17.

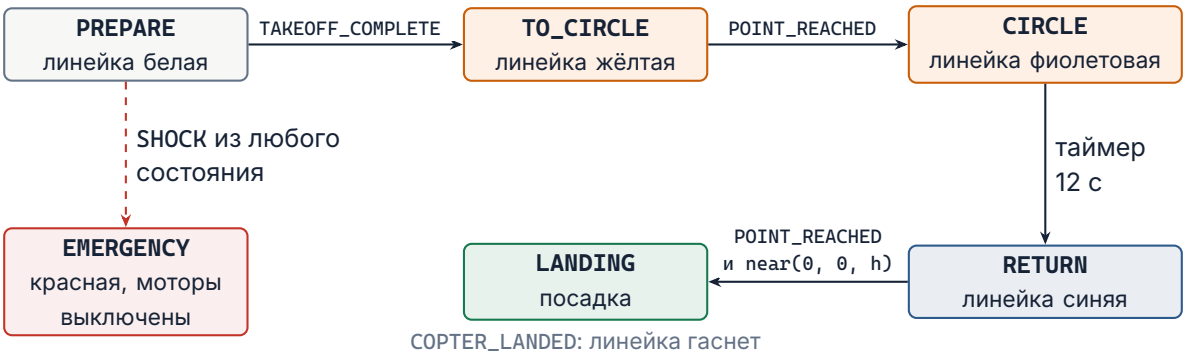


Рис. 17. Автомат миссии «окружность»: пять состояний и события, которые переводят автомат из одного в другое. Удар (SHOCK) переводит в EMERGENCY из любого состояния.

Аварийные реакции

lua/06_missions/missions_failsafe.lua

```lua
-- Патруль между двумя точками с аварийными реакциями
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local points = {{1, 0, 1}, {-1, 0, 1}}
local curr = 1
local state = "TAKEOFF"                    -- TAKEOFF -> PATROL -> LANDING;
↪  при ударе EMERGENCY
local lowBattery = false                   -- разряд пришёл, пока дрон
↪  взлетал

local function fill(r, g, b)
  for i = 0, ledNumber - 1 do leds:set(i, r, g, b) end
end

local function nextPoint()
  ap.goToLocalPoint(table.unpack(points[curr]))
  curr = curr % #points + 1                -- 1 -> 2 -> 1 -> ...
end

local function land()
  state = "LANDING"
  fill(1, 0, 0)
  ap.push(Ev.MCE_LANDING)                  -- садимся там, где летим
end

function callback(event)
  if event == Ev.SHOCK then                -- удар в любом состоянии:
  ↪  немедленно выключить моторы
    state = "EMERGENCY"
    fill(1, 0, 0)
    ap.push(Ev.ENGINES_DISARM)
  elseif event == Ev.TAKEOFF_COMPLETE and state == "TAKEOFF" then
    if lowBattery then
      land()                               -- батарея села ещё на взлёте:
      ↪  маршрут не начинаем
    else
```

```
      state = "PATROL"
      fill(0, 1, 0)                          -- зелёный: патрулируем
      nextPoint()
    end
  elseif event == Ev.POINT_REACHED and state == "PATROL" then
    nextPoint()
  elseif event == Ev.LOW_VOLTAGE2 then
    if state == "PATROL" then
      land()                                 -- батарея садится: прекращаем
      ↪  маршрут
    elseif state == "TAKEOFF" then
      lowBattery = true                      -- посадку во время взлёта
      ↪  автопилот не примет: сядем после него
      fill(1, 0, 0)
    end                                      -- в LANDING и EMERGENCY
    ↪  повторное событие ничего не меняет
  end
end

ap.push(Ev.MCE_PREFLIGHT)
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end)
```

Что происходит. Дрон патрулирует между двумя точками бесконечно. Состояния: `TAKEOFF` → `PATROL` → `LANDING`, при ударе — `EMERGENCY`. Одно событие получает разную реакцию в зависимости от состояния:

- `SHOCK` — в любом состоянии сразу выключение моторов: аварию не откладывают.
- `LOW_VOLTAGE2` в `PATROL` — посадка на месте. Во время взлёта автопилот команду посадки не примет, поэтому в `TAKEOFF` программа только запоминает разряд (`lowBattery`) и садится сразу после `TAKEOFF_COMPLETE`, не начиная маршрут. В `LANDING` и `EMERGENCY` повторное событие (напряжение может «прыгать» около порога) ничего не меняет.
- `POINT_REACHED` ведёт маршрут только в `PATROL`.

В симуляторе аварийные события приходят только «по-настоящему»: `SHOCK` — при столкновении (например, с деревом или стеной), `LOW_VOLTAGE1` и `LOW_VOLTAGE2` — при разряде батареи в долгом полёте. Кнопки, которая вызывает их вручную, нет. Чтобы проверить реакцию, вызовите обработчик сами, как в главе 4: `Timer.callLater(12, function() callback(Ev.LOW_VOLTAGE2) end)`.

Запуск с пульта

На настоящем дроне миссию удобно запускать тумблером: программа загружена, дрон стоит, пилот отходит и переводит тумблер. `Sensors.rc()` возвращает восемь каналов (значения от −1 до 1), тумблер SwA — восьмой:

lua/06_missions/missions_rc_initiation.lua

```lua
local unpack = table.unpack
local points = {{0, 2, 1}, {0, 1, 1}, {0, 2, 1}, {0, 1, 1}} -- маршрут
local curr_point = 1
local started = false                        -- миссия уже запущена?

local function nextPoint()
  if curr_point <= #points then
    ap.goToLocalPoint(unpack(points[curr_point]))
    curr_point = curr_point + 1
  else
    ap.push(Ev.MCE_LANDING)                  -- точки кончились — посадка
  end
end

function callback(event)
  if event == Ev.TAKEOFF_COMPLETE then Timer.callLater(1, nextPoint) end
  if event == Ev.POINT_REACHED then Timer.callLater(1, nextPoint) end
end

-- опрашиваем тумблер SwA (8-й канал) раз в секунду, пока миссия не запущена
startTimer = Timer.new(1, function()
  local rc_chans = table.pack(Sensors.rc()) -- 8 значений каналов в таблицу
  if rc_chans[8] > 0 and not started then   -- тумблер SwA переключён:
  ↪  значение канала больше нуля
    started = true
    startTimer:stop()                        -- больше не опрашиваем
    ap.push(Ev.MCE_PREFLIGHT)
    Timer.callLater(2, function() ap.push(Ev.MCE_TAKEOFF) end)
  end
end)
startTimer:start()
```

Что происходит. Таймер раз в секунду читает каналы: `table.pack(Sensors.rc())` собирает восемь значений в таблицу, а `rc_chans[8]` — восьмой из них. Как только он положителен — флаг `started`, таймер останавливается (иначе он отдавал бы предстарт каждую секунду), предстарт, через 2 с взлёт. Дальше маршрут ведёт уже знакомый счётчик шагов; `Timer.callLater(1, nextPoint)` — секундное зависание в каждой точке: следующая команда по-прежнему отдаётся только после события, просто с паузой. Настройка пульта и соответствие тумблеров каналам — в приложении D. В симуляторе роль пульта играет геймпад, подключённый к компьютеру и настроенный в панели «**Пульт**»; без него все каналы равны нулю, и программа будет ждать тумблер бесконечно. Нет геймпада — проверьте остальную программу, временно заменив условие запуска на `true`.

Светодиоды для отладки

В примерах этого раздела с индикацией цвет меняется вместе с состоянием. Это дешёвый и надёжный журнал: на полигоне без терминала по цвету видно, на каком этапе программа зависла. Приучите себя подсвечивать состояния с самого начала.

ЗАДАНИЕ 5.8 · Автомат с запасом прочности

1. В `missions_failsafe.lua` добавьте вызов `callback(Ev.LOW_VOLTAGE2)` через 12 с после запуска. Затем перенесите его на 1.5 с — пока дрон ещё взлетает.
2. Добавьте реакцию на `Ev.LOW_VOLTAGE1`: маршрут продолжается, но линейка становится жёлтой, а в журнал печатается `"низкий заряд, этап: "` и текущее состояние.
3. Добавьте в патруль счётчик кругов: после трёх полных проходов туда и обратно — штатная посадка, в журнале `"остановка: маршрут пройден"`. Для аварийной посадки печатайте `"остановка: разряд"`.

Что должно получиться: 1) при 12 с дрон прекращает патруль и садится на месте, линейка красная; при 1.5 с он сначала завершает взлёт, маршрут не начинает и сразу садится; 2) жёлтая линейка и строка с состоянием `PATROL`, патруль не прерывается; 3) ровно шесть точек патруля, посадка и строка с причиной остановки — по журналу видно, штатной она была или аварийной.

В симуляторе: миссии Lua: 71 в панели «**Обучение**».

5.10 Частые ошибки

Перед запуском симулятор проверяет программу и объясняет найденное в уведомлениях (колокольчик в заголовке редактора), а во время полёта останавливает её, если команды противоречат друг другу. Вот что он чаще всего находит:

- **Команды подряд без ожидания.** `ap.push(Ev.MCE_PREFLIGHT)` и сразу `ap.push(Ev.MCE_TAKEOFF)`. Взлёт — после `ENGINES_STARTED` или через таймер, точка — после `TAKEOFF_COMPLETE`, посадка — после `POINT_REACHED`.
- **Посадка раньше точки.** Таймер короче перелёта — дрон сядет, не долетев. Садиться по событию.
- **Точка раньше взлёта.** `ap.goToLocalPoint` до завершения взлёта автопилот не выполнит, а симулятор сообщит «Маршрут слишком рано». Точка — по `TAKEOFF_COMPLETE`.
- **Повторная подготовка.** `ap.push(Ev.MCE_PREFLIGHT)` второй раз, когда подготовка уже идёт (например, в ответ на `ENGINES_STARTED`), — ошибка «Подготовка уже запущена»: автопилот уже на другом этапе.
- **Нет `callback`.** На борту такая программа останавливается после первой команды. Симулятор её выполнит и отдельно об этом не предупредит — проверяйте сами.
- **Все точки по одному событию.** Без счётчика шагов вторая точка затирает первую. Шаблон — `nextPoint` с `curr`.
- **Точка «в земле».** `Ctrl`+щелчок показывает высоту поверхности; z цели должен быть выше.
- **Градусы вместо радиан.** `ap.updateYaw(90)` — это 90 радиан, а не $90°$: нос встанет совсем не туда. Нужно `math.rad(90)`.

- **Таймер не остановлен.** Таймер окружности продолжает слать точки после команды посадки — дрон не сядет. `t:stop()` перед сменой этапа.
- **Функция вызвана вместо передачи.** `Timer.callLater(1, nextPoint())` выполняет `nextPoint` сразу и передаёт таймеру её результат, `nil`; симулятор предупредит ещё до запуска, а во время работы остановит программу ошибкой «ожидал функцию». Нужно `Timer.callLater(1, nextPoint)`.

Где смотреть

Вкладка «**Вывод**» нижней панели редактора показывает `print` и сообщения автопилота о каждом переходе: предстарт, взлёт, точка достигнута; `print` к тому же всплывает подписью над дроном в сцене. Уведомления об ошибках — под колокольчиком и на вкладке «**Проблемы**». Панель «**Полёт**» показывает текущее состояние автопилота и координаты. Если дрон «завис» — сначала туда.

ЗАДАНИЕ 5.9 · Найдите ошибку по поведению

Внесите в `missions_points_flight.lua` по одной ошибке за запуск и запишите, что сообщает симулятор и что видно в сцене: (1) вызов `nextPoint()` сразу после `ap.push(Ev.MCE_TAKEOFF)`, а не в `callback`; (2) `ap.push(Ev.MCE_PREFLIGHT)` ещё раз в ответ на `ENGINES_STARTED`; (3) посадка через `Timer.callLater(2, ...)` после первой точки вместо `POINT_REACHED`. Каждый раз возвращайте исправную версию.

Что должно получиться: (1) уведомление «Маршрут слишком рано»: точка, отправленная до `TAKEOFF_COMPLETE`, не выполняется, и маршрут начинается не с первой точки или не начинается вовсе; (2) уведомление о повторной подготовке: команда не подходит текущему состоянию; (3) дрон садится, не долетев до первой точки, хотя ошибок нет — такую ошибку видно только по сцене и журналу.

В симуляторе: миссии Lua: 4, 52 в панели «**Обучение**».

5.11 Проверьте себя

1. Как начать миссию? `ap.push(___)`

   (a) `Ev.MCE_PREFLIGHT`

   (b) `Ev.START`

   (c) `Ev.MCE_TAKEOFF`

   Ответ: (a) — предстарт всегда первый.

2. Какое событие сообщает, что взлёт завершён?

   (a) `Ev.ENGINES_STARTED`

   (b) `Ev.TAKEOFF_COMPLETE`

   (c) `Ev.POINT_REACHED`

   Ответ: (b).

3. Как задать курс 90°?

(a) `ap.updateYaw(90)`

(b) `ap.updateYaw(math.rad(90))`

(c) `ap.updateYaw(90 / 180)`

Ответ: (b) — радианы.

4. Когда отправлять вторую точку маршрута?

(a) Через 4 с после первой

(b) По `POINT_REACHED` первой

(c) Сразу вслед за первой

Ответ: (b).

5. Как безопасно выполнить действие через 2 секунды?

(a) `sleep(2); action()`

(b) `Timer.callLater(2, action)`

(c) `Timer.new(2, action)`

Ответ: (b).

6. Как организовать непрерывное обновление точки окружности?

(a) `while true do ap.goToLocalPoint(...) end`

(b) `Timer.new(0.1, function() ap.goToLocalPoint(...) end)`

(c) `repeat ... until false`

Ответ: (b).

7. Что делать по `Ev.SHOCK`?

(a) Продолжить маршрут

(b) `ap.push(Ev.MCE_LANDING)`

(c) Остановить таймеры и `ap.push(Ev.ENGINES_DISARM)`

Ответ: (c).

8. Дрон прилетел в точку $(2, 0, 1)$, `Sensors.lpsPosition()` вернул `x = 1.93`. Как правильно проверить прибытие?

(a) `x == 2`

(b) `math.abs(2 - x) <= 0.15`

(c) `x > 2`

Ответ: (b) — измерение неточно, сравнивают с допуском.

9. Где в `rc_chans = table.pack(Sensors.rc())` тумблер SwA?

(a) `rc_chans[1]`

(b) `rc_chans[8]`

(c) `rc_chans["SwA"]`

Ответ: (b).

5.12 Практика: проекты миссий

Решения и дополнительные идеи (спираль, лоитер, лепесток, радиус с пульта, ограничение скорости поворота, повтор после посадки) — в `examples/lua-tasks/07_advanced_missions/`. Во всех проектах: предстарт и взлёт как в примерах, светодиодная индикация этапов, реакции на `SHOCK` и `LOW_VOLTAGE2`, а в журнале — строки с подписанными этапами и причиной остановки. Проект принят, если всё перечисленное в «Что должно получиться» видно в сцене или в журнале.

ЗАДАНИЕ 5.10 · Многоугольник из N точек

Сгенерируйте циклом N вершин окружности радиуса 0.5 м под углами $360^\circ \cdot i/N$, пройдите их счётчиком шагов, вернитесь в центр и сядьте. Проверьте для $N = 3, 6, 12$, меняя только одно число.

Что должно получиться: след — правильный треугольник, шестиугольник, двенадцатиугольник с центром в начале сцены; в журнале по строке на каждую достигнутую вершину с её номером; посадка в центре; при вызове `callback(Ev.LOW_VOLTAGE2)` посреди маршрута дрон садится на месте, линейка красная.

ЗАДАНИЕ 5.11 · Два квадрата

Пролетите квадрат со стороной 1 м дважды подряд и только потом садитесь. Подсказка: счётчик кругов рядом со счётчиком шагов.

Что должно получиться: на сцене — один квадрат, пройденный дважды (след ложится сам на себя); в журнале номера кругов 1 и 2 и восемь достигнутых вершин; посадка только после восьмой.

ЗАДАНИЕ 5.12 · Восьмёрка с курсом

Две окружности с противоположным направлением обхода и общей точкой в центре; нос всё время по касательной; завершение — в центре и посадка. Окружности ведёт таймер, переходы — конечный автомат с подсветкой состояний.

Что должно получиться: след — восьмёрка; камера «**Дрон**» показывает, что нос смотрит по направлению движения на обеих петлях; цвета линейки меняются при каждом переходе; посадка в центре, подтверждённая `near`.

ЗАДАНИЕ 5.13 · Старт и останов тумблером

Тумблер SwA (`rc_chans[8]`) вверх — предстарт, взлёт, окружность; тумблер вниз — прекратить, вернуться в точку над стартом, сесть. Проверьте с геймпадом, настроенным в панели «**Пульт**».

Что должно получиться: пока тумблер внизу, дрон стоит; после переключения вверх взлетает и кружит; после переключения вниз в любой момент окружности возвращается и садится; повторное переключение во время посадки ничего не ло-

мает; в журнале — причина остановки «тумблер».

ЗАДАНИЕ 5.14 · Отчёт о точности

Пройдите 8 точек окружности. По каждому `POINT_REACHED` печатайте номер точки, время с предыдущего события (`time()` возвращает секунды) и ошибку положения — расстояние от измеренного положения до цели. В конце напечатайте среднюю и наибольшую ошибку.

Что должно получиться: восемь строк отчёта и итоговая строка; все ошибки меньше 0.15 м — допуска, с которым автопилот засчитывает прибытие; интервалы между точками близки друг к другу. По отчёту видно, на какой точке ошибка была наибольшей.

В симуляторе: миссии Lua: 72 в панели «**Обучение**».

Итоги главы

- Миссия — цепочка «команда → событие → команда»; цепочку ведёт `callback`.
- Предстарт, взлёт, точки, посадка; счётчик шагов для маршрута; таймер для плавных траекторий.
- Координаты в метрах от начала сцены, Z вверх; на старте нос смотрит вдоль +Y; курс в радианах.
- Сложная миссия — конечный автомат: состояние проверяется вместе с событием.
- Failsafe обязателен: `SHOCK` — моторы выключить, `LOW_VOLTAGE2` — сесть.
- Результат проверяют измерением: `Sensors.lpsPosition()`, сравнение с допуском, тайм-аут и причина остановки в журнале.

ЧАСТЬ II

Программирование на Python

ГЛАВА 6

Основы Python

Цели главы

- Понять, чем Python отличается от Lua, и запустить первую программу.
- Работать с переменными, типами, арифметикой и логикой.
- Управлять ходом программы: ветвления `if/elif/else`, циклы `for` и `while`.
- Писать функции — строительный материал для программ полёта в следующей главе.

Миссии симулятора к главе

В панели «**Обучение**» выберите язык «**Python**».

- **1–9 «Дрон и первая команда»**: аппарат в сцене, где живёт программа, команда автопилоту, повторный arm, остановить и начать заново, журнал первого запуска, что такое API, одна ошибка за раз, самостоятельная подготовка. Здесь впервые встречаются `print` и единственная команда дрону `pioneer.arm()`.
- **19–27 «Маленькие программы»**: имя для числа, подпись к измерению, вычисляем настройку, выбор по условию, сравниваем числа, повторяем действия, действие с именем, настройка функции, список сигналов. Это материал уроков этой главы, применённый к подсветке дрона.

В миссиях 19–27 подсветку включает команда `pioneer.led_control(номер, r, g, b)`; подробно она разобрана в разделе 7.3, а миссии 10–18 про подсветку удобнее пройти вместе со следующей главой.

Как работать с главой

Примеры запускайте в симуляторе, выбрав язык «**Python**»: вывод `print` появится в терминале, как и для Lua. Структура главы повторяет главу про Lua, поэтому сравнивайте конструкции двух языков — это лучший способ запомнить оба. Ответы к задачам для самостоятельного решения и к заданиям уроков лежат в папке `examples/python-answers`: сначала решите сами, потом сверьтесь с файлом.

6.1 Что такое Python и зачем он нужен?

Python — это высокоуровневый язык программирования, который был создан Гвидо ван Россумом и впервые выпущен в 1991 году. Назван он не в честь змеи, а в честь популярного британского комедийного шоу «Летающий цирк Монти Пайтона».

Почему Python?

- **Простота:** Синтаксис Python похож на обычный английский язык.
- **Универсальность:** Используется в веб-разработке, науке, AI и робототехнике.
- **Мощь:** Огромное количество библиотек (OpenCV, NumPy, TensorFlow) для решения сложных задач.

Для программирования квадрокоптеров Pioneer мы выбрали Python, так как он позволяет легко взаимодействовать с оборудованием и обрабатывать видеопоток с камеры.

Сравнение Python и Lua

Мы уже знакомы с языком Lua, который используется для написания скриптов автопилота. Давайте посмотрим, чем Python отличается от Lua:

Характеристика	Lua	Python
Назначение	Встраиваемый скриптовый язык (для игр, контроллеров)	Универсальный язык общего назначения
Синтаксис	Использует `end` для блоков кода	Использует **отступы** для блоков кода
Индексация	С 1 (первый элемент — 1)	С 0 (первый элемент — 0)
Библиотеки	Ограниченный набор	Огромная экосистема (PyPI)
Скорость	Очень быстрый	Медленнее, но мощнее

Таблица 1. Сравнение Lua и Python

> **Важное отличие**
>
> Главное, что нужно запомнить: в Python блоки кода (циклы, функции, условия) выделяются **отступами** (обычно 4 пробела), а не ключевыми словами `do ... end`.

Первая программа

Традиционно изучение любого языка начинается с программы, которая выводит приветствие на экран.

python/01_python_basics/01_hello_world.py

```python
# Первая программа на Python
print("Hello, World!")
```

Здесь функция `print()` выводит текст, заключенный в кавычки, в консоль.

Первая команда дрону

В миссиях 1–9 рядом с `print` стоит ещё одна строка — `pioneer.arm()`: просьба автопилоту запустить моторы. Дрон при этом остаётся на земле. Как подключаться к дрону и что делает каждая команда — в главе 7; пока достаточно понимать, что строки программы выполняются сверху вниз, а опечатка в имени (`pioneer.amr()`) останавливает программу на этой строке с ошибкой `AttributeError`.

ЗАДАНИЕ 6.1 · Привет, мир!

Напишите программу из двух строк: первая выводит ваше имя и фамилию, вторая — фразу `Готов к полёту`. Затем намеренно допустите опечатку во второй строке (`pirnt`), запустите программу и найдите в терминале номер строки с ошибкой.

Что должно получиться: сначала в терминале две строки в заданном порядке; с опечаткой — первая строка и сообщение `NameError` с номером второй строки.

В симуляторе: миссии Python: 6–9 в панели «**Обучение**».

6.2 Переменные и типы данных

Переменная — это именованная область памяти, в которой хранится значение. В Python не нужно заранее объявлять тип переменной, он определяется автоматически при присваивании.

Основные типы данных

- `int` (integer) — целые числа (например, `10`, `-5`).
- `float` — числа с плавающей точкой (например, `3.14`, `1.5`).
- `str` (string) — строки текста (например, `"Pioneer"`).
- `bool` (boolean) — логические значения (`True`, `False`).

python/01_python_basics/02_variables.py

```
# Переменные и типы данных
name = "Pioneer"  # Строка (str)
altitude = 1.5    # Число с плавающей точкой (float)
battery = 100     # Целое число (int)
is_flying = False # Логический тип (bool)

print("Дрон:", name)
print("Высота:", altitude)
print("Тип переменной altitude:", type(altitude))
```

Функция `type()` позволяет узнать тип переменной. Имя переменной выбирают по смыслу: `altitude = 1.5` читается как «высота», а просто `a = 1.5` — нет. Число и строка различаются: `1.5` участвует в вычислениях, `"1.5 м"` — только текст. Поэтому единицы измерения не вписывают в число, а добавляют подписью при выводе: `print("Высота:", altitude, "м")`.

Задачи для самостоятельного решения

1. Создайте переменную `speed` и присвойте ей значение 5. Выведите её на экран.
2. Создайте переменную `height` и присвойте ей значение 1.5. Выведите её.
3. Создайте переменную `name` со значением `"Pioneer"`. Выведите её.
4. Создайте переменную `score` = 10. Затем измените её значение на 20. Выведите `score`.
5. Создайте переменную `x` = 100. Затем присвойте ей строку `"Сто"`. Выведите тип переменной до и после изменения.
6. Даны `a = 5` и `b = 7`. Создайте переменную `c`, равную их сумме. Выведите `c`.
7. Даны `part1 = "Geo"` и `part2 = "skan"`. Создайте `full`, сложив их.
8. Создайте строку `s = "Fly"`. Выведите её 3 раза подряд (используйте умножение).
9. Какого типа переменная `pi = 3.1415`? Выведите результат функции `type(pi)`.
10. Присвойте переменным `x`, `y`, `z` значения 1, 2, 3 в одной строке.
11. Даны `a = 10`, `b = 20`. Поменяйте их значения местами без использования третьей переменной.
12. Создайте переменную `is_flying` со значением `True`. Выведите её.
13. Дано целое число `n = 5`. Преобразуйте его в дробное и сохраните в `f`.
14. Дана строка `s_num = "123"`. Преобразуйте её в число `num` и прибавьте 1.
15. Даны `mass = 10` и `accel = 9.8`. Вычислите `force = mass * accel`.

ЗАДАНИЕ 6.2 · Личная карточка

Создайте переменные для вашего имени, возраста и любимого предмета и выведите их одной строкой. Затем создайте переменную `height` со значением 1.5 и выведите её с подписью «Высота:» и единицами «м».

Что должно получиться: две строки, например `Иван 14 информатика` и `Высота: 1.5 м`. Ответы к задачам урока и к этому заданию — в файле `examples/python-answers/02_variables_answers.py`.

В симуляторе: миссии Python: 19–20 в панели «**Обучение**».

6.3 Арифметические операторы

Python поддерживает все стандартные математические операции.

- `+` Сложение
- `-` Вычитание
- `*` Умножение
- `/` Деление (всегда возвращает `float`)
- `//` Целочисленное деление (отбрасывает дробную часть)
- `%` Остаток от деления
- `**` Возведение в степень

python/01_python_basics/03_arithmetic.py

```python
# Арифметические операции
x = 10
y = 3

print("Сложение:", x + y)
print("Деление:", x / y)
print("Целочисленное деление:", x // y) # 3
print("Остаток от деления:", x % y)     # 1
print("Возведение в степень:", x ** y)  # 1000
```

Обратите внимание на приоритет операций: умножение и деление выполняются раньше сложения и вычитания, как в математике. Скобки `()` используются для изменения порядка действий. Прежде чем запускать программу с выражением, посчитайте ожидаемый ответ в уме или на бумаге: совпадение с выводом — самая простая проверка, что формула записана верно.

Задачи для самостоятельного решения

1. Сложите числа 15 и 38. Выведите результат.
2. Вычтите из 100 число 45. Выведите результат.
3. Умножьте 7 на 8.
4. Разделите 50 на 4. Какой тип данных получился?
5. Сколько раз число 4 помещается в 50 целиком? Используйте `//`.
6. Чему равен остаток от деления 20 на 3? Используйте `%`.
7. Возведите 2 в степень 10 (2 в 10-й степени).
8. Вычислите `2 + 2 * 2`. Затем `(2 + 2) * 2`. Сравните результаты.
9. Сторона квадрата `a = 5`. Найдите периметр `P = 4 * a`.
10. Даны стороны `a = 10`, `b = 20`. Найдите площадь `S = a * b`.
11. Сколько секунд в одном часе? (60 секунд * 60 минут).
12. Даны числа 10, 20, 30. Найдите их среднее арифметическое.
13. Дано число 12345. Найдите его последнюю цифру, используя оператор `% 10`.
14. Дано четное число `n = 10`. Получите следующее четное число (+2).
15. Умножьте -5 на -5. Проверьте знак результата.

ЗАДАНИЕ 6.3 · Калькулятор

Задайте стороны прямоугольного участка переменными `a = 5` и `b = 8` (метры). Выведите площадь и периметр с подписями и единицами. Сначала посчитайте ответ вручную.

Что должно получиться: `Площадь: 40 кв. м` и `Периметр: 26 м`. Решение — в конце файла `examples/python-answers/03_arithmetic_answers.py`.

В симуляторе: миссии Python: 21 в панели «**Обучение**».

6.4 Логические операции

Логические операторы позволяют комбинировать условия. Результатом всегда является `True` (Истина) или `False` (Ложь).

Операторы сравнения

- `==` Равно
- `!=` Не равно
- `>` Больше
- `<` Меньше
- `>=` Больше или равно
- `<=` Меньше или равно

Логические связки

- `and` (И) — истина, только если оба условия истинны.
- `or` (ИЛИ) — истина, если хотя бы одно условие истинно.
- `not` (НЕ) — инвертирует значение (делает истину ложью и наоборот).

python/01_python_basics/04_logic.py

```python
# Логические операции
voltage = 3.8
is_connected = True

# Проверка условий
is_low_battery = voltage < 3.5
ready_to_fly = is_connected and not is_low_battery

print("Низкий заряд?", is_low_battery)
print("Готов к полету?", ready_to_fly)
```

Сравнение с порогом проверяют на трёх значениях: ниже порога, ровно на пороге и выше. Ошибка «`<` вместо `<=`» видна только на самом пороге. Числа напряжения в учебных примерах (3.5, 3.8 В) — напряжение одного литиевого элемента, чтобы упражнения были короче; у двухэлементной батареи Пионера пороги другие: 7.2 В и 6.9 В (раздел 7.2).

Задачи для самостоятельного решения

1. Проверьте, больше ли 10 числа 5.
2. Проверьте, меньше или равно 5 числу 5.
3. Равны ли числа 100 и 100?
4. Правда ли, что 10 не равно 20?
5. Проверьте условие: (5 > 0) и (5 < 10).
6. Проверьте условие: (5 > 0) или (5 > 10).

7. Инвертируйте значение `True` с помощью `not`.
8. Дано `x = 15`. Проверьте, лежит ли `x` между 10 и 20 (строго).
9. Дано `n = 4`. Проверьте, делится ли оно на 2 без остатка (`n % 2 == 0`).
10. Равны ли строки ”test” и ”Test”?
11. Дано `battery = 4.0` и `connected = True`. Разрешен ли полет? (нужно и то, и другое).
12. Дано `obstacle = False`. Есть ли опасность? (Если препятствие есть ИЛИ высота < 0.5).
13. Проверьте сложное условие: (2 + 2 == 4) и (не (3 > 5)).
14. Равны ли число 5 и строка ”5”?
15. Чему равно `bool(1)`? А `bool(0)`?

ЗАДАНИЕ 6.4 · Проверка безопасности

Полёт разрешён, если скорость ветра `wind_speed` меньше 5 м/с. Проверьте выражение на значениях 4, 5 и 7 и выведите для каждого значение и результат сравнения.

Что должно получиться: `4 True`, `5 False`, `7 False`. Решение — в конце файла `examples/python-answers/04_logic_answers.py`.

В симуляторе: миссии Python: 23 в панели «**Обучение**».

6.5 Ветвление (Условные операторы)

Условные операторы `if`, `elif` и `else` позволяют программе принимать решения в зависимости от условий.

python/01_python_basics/05_branching.py

```python
# Ветвление: Робот на перекрестке
direction = "forward"

if direction == "left":
    print("Поворачиваю налево")
elif direction == "right":
    print("Поворачиваю направо")
elif direction == "forward":
    print("Еду прямо")
else:
    print("Останавливаюсь, путь неясен")
```

Обратите внимание на отступы (4 пробела) — они обязательны в Python и обозначают блоки кода. У каждой ветки должен быть понятный результат: если ветка `else` ничего не печатает, по выводу не отличить «условие ложно» от «программа сюда не дошла».

Задачи для самостоятельного решения

1. Дано число `x = 10`. Если оно больше 0, выведите ”Положительное”.

2. Дано `n = 7`. Если делится на 2 без остатка — "Четное", иначе — "Нечетное".
3. Дано `age = 15`. Если `age >= 18`, выведите "Взрослый", иначе "Ребенок".
4. Дано `password = "1234"`. Если пароль совпадает с "secret", пишем "Доступ разрешен", иначе "Отказ".
5. Даны `a = 10`, `b = 20`. Выведите большее из них.
6. Дано `x = 0`. Используя `elif`, выведите "Плюс", "Минус" или "Ноль".
7. Дано `color = "red"`. Если "green" — "Ехать", "yellow" — "Ждать", "red" — "Стоять".
8. `voltage = 3.6`. Если < 3.5 — "Посадка!", иначе "Полет нормальный".
9. Делится ли число 15 на 3 и на 5 одновременно?
10. Дано `name = "Alex"`. Если длина имени > 5 букв, напишите "Длинное", иначе "Короткое".
11. Дана строка `s = ""`. Если она пустая (длина 0), напишите "Пусто".
12. `weight = 80`. < 50 — "Легкий", 50-100 — "Средний", > 100 — "Тяжелый".
13. `armed = True`, `ready = False`. Если оба `True` — "Взлет", иначе "Ждать".
14. `login = "user"`. Если не равен "admin", вывести "Ошибка".
15. `a=5, b=2, c=8`. Найдите самое большое число из трех через `if`.

ЗАДАНИЕ 6.5 · Светофор

В переменной `color` записан цвет светофора: `"red"`, `"yellow"` или `"green"`. Выведите действие: `Стоп`, `Приготовиться` или `Ехать`, а для любого другого значения — `Неизвестный цвет`. Проверьте все четыре ветки, меняя значение переменной.

Что должно получиться: при `color = "yellow"` — `Приготовиться`, при `"blue"` — `Неизвестный цвет`. Решение — в конце файла `examples/python-answers/05_branching_answers.py`.

В симуляторе: миссии Python: 22 в панели «**Обучение**».

6.6 Циклы

Циклы позволяют выполнять один и тот же блок кода многократно.

Цикл for

Используется, когда известно количество повторений. Функция `range(N)` создает последовательность от 0 до N-1.

Цикл while

Выполняется, пока условие истинно. Будьте осторожны: если условие никогда не станет ложным, получится «бесконечный цикл».

python/01_python_basics/06_loops.py

```python
# Циклы
import time

# Цикл for: мигаем светодиодом 5 раз (имитация)
print("Начинаю мигать...")
for i in range(5):
    print(f"Мигание №{i+1}: ВКЛ")
    time.sleep(0.5)
    print(f"Мигание №{i+1}: ВЫКЛ")
    time.sleep(0.5)

# Цикл while: летим пока есть заряд
battery = 10
while battery > 0:
    print(f"Летим... Заряд: {battery}%")
    battery -= 2
print("Посадка!")
```

- `import time` подключает стандартный модуль `time`; `time.sleep(0.5)` — пауза на полсекунды. В отличие от Lua на борту, Python-программа работает на компьютере (в симуляторе — в браузере) и автопилот не блокирует, поэтому `sleep` здесь допустим (глава 7).
- `f"Мигание №{i+1}"` — f-строка: выражение в фигурных скобках вычисляется и подставляется в текст. `battery -= 2` — то же, что `battery = battery - 2`.
- `break` — немедленно прерывает цикл.
- `continue` — переходит к следующей итерации.

Список и цикл по списку

Несколько значений в определённом порядке хранят в **списке** — значения через запятую в квадратных скобках. Цикл `for` перебирает элементы списка по очереди, поэтому одно и то же действие применяется ко всем:

Список сигналов

```python
signals = ["ожидание", "готовность", "завершение"]
for signal in signals:
    print(signal)
print("Всего:", len(signals), "первый:", signals[0])
```

Нумерация элементов в Python начинается с нуля: `signals[0]` — первый элемент, `signals[2]` — третий. Функция `len()` возвращает длину списка. В главе 7 списком будут вершины маршрута.

Задачи для самостоятельного решения

1. Выведите числа от 0 до 4 используя `range(5)`.
2. Выведите слово ”Привет” 3 раза.

3. Выведите числа от 1 до 5 (используйте `range(1, 6)`).
4. Выведите четные числа от 0 до 8 (`range(0, 9, 2)`).
5. Найдите сумму чисел от 1 до 100.
6. Выведите числа от 5 до 1 (`range(5, 0, -1)`).
7. Выведите таблицу умножения на 5 (от 1*5 до 10*5).
8. Дана строка ”Дрон”. Выведите каждую букву с новой строки.
9. Используя `while`, выводите `x`, пока `x < 3`. Начальное `x = 0`.
10. Ищите число 7 в диапазоне от 0 до 10. Если нашли — используйте `break`.
11. Выведите числа от 0 до 4, пропустив 2 (используйте `continue`).
12. Выведите квадраты чисел от 1 до 5.
13. Посчитайте длину строки ”Hello” с помощью цикла (без `len()`).
14. Выведите 5 строк, в каждой по одной звездочке `*`.
15. Выведите лесенку из звезд (1, 2, 3...).

ЗАДАНИЕ 6.6 · Обратный отсчёт

С помощью `range` с отрицательным шагом выведите числа от 10 до 1, каждое с новой строки, а затем — `Поехали!`. Цикл должен выполниться ровно 10 раз.

Что должно получиться: 11 строк: `10`, `9`, …, `1`, `Поехали!`.

В симуляторе: миссии Python: 24 в панели «**Обучение**».

ЗАДАНИЕ 6.7 · Список сигналов

Создайте список из трёх названий сигналов: «ожидание», «готовность», «завершение». Выведите каждый элемент циклом `for`, а затем количество элементов через `len()`. Добавьте в список четвёртый сигнал и убедитесь, что цикл менять не пришлось.

Что должно получиться: три (после правки — четыре) строки с названиями и строка `Всего сигналов: 3` (затем `4`). Решения обоих заданий — в конце файла `examples/python-answers/06_loops_answers.py`.

В симуляторе: миссии Python: 27 в панели «**Обучение**».

6.7 Функции

Функция — это именованный блок кода, который можно вызывать многократно. Функции делают код чище и понятнее.

Ключевое слово `def` объявляет функцию. `return` возвращает результат.

python/01_python_basics/07_functions.py

```python
# Функции
def calculate_area(length):
    """Вычисляет площадь квадрата."""
```

```
    return length * length

side = 5
area = calculate_area(side)
print(f"Площадь квадрата со стороной {side} равна {area}")

def check_battery(voltage):
    if voltage < 3.5:
        return "Critical"
    elif voltage < 3.8:
        return "Low"
    else:
        return "Normal"

print("Статус батареи 3.6В:", check_battery(3.6))
```

Область видимости

Переменные, созданные внутри функции, не видны снаружи. Это называется локальной областью видимости.

Параметр — имя в скобках после имени функции: при вызове `calculate_area(5)` в нём оказывается значение 5. Одна функция с параметром заменяет несколько почти одинаковых кусков кода. Функция без `return` тоже полезна — она просто выполняет действия, например печатает или включает подсветку; результат её вызова — `None`.

Задачи для самостоятельного решения

1. Напишите функцию `say_hello()`, которая печатает ”Привет!”. Вызовите её.
2. Напишите функцию `greet(name)`, которая печатает ”Привет, <name>!”.
3. Напишите функцию `add(a, b)`, которая возвращает сумму двух чисел.
4. Напишите функцию `square(x)`, которая возвращает x в квадрате.
5. Напишите функцию `my_max(a, b)`, которая возвращает большее из двух чисел.
6. Напишите функцию `is_even(n)`, которая возвращает `True`, если число четное.
7. Напишите функцию `perimeter(a, b)`, возвращающую периметр прямоугольника.
8. Напишите функцию `m_to_cm(meters)`, которая переводит метры в сантиметры.
9. Напишите функцию `check_battery(voltage)`. Если < 3.5 -> ”Low”, иначе ”OK”.
10. Напишите функцию `my_abs(x)`. Если x < 0, возвращает -x, иначе x.
11. Напишите функцию `avg(a, b)`, возвращающую среднее арифметическое.
12. Напишите функцию `log_error(msg)`, которая печатает ”ERROR: <msg>”.
13. Напишите функцию `repeat(s, n)`, возвращающую строку s, повторенную n раз.
14. Напишите функцию `flight_time(cap, cons)`, считающую время полета (емкость / расход).
15. Напишите функцию `full_name(first, last)`, возвращающую ”Фамилия Имя”.

ЗАДАНИЕ 6.8 · Конвертер

Напишите функцию `celsius_to_fahrenheit(c)`, которая возвращает температуру по Фаренгейту по формуле $F = C \times 1.8 + 32$. Вызовите её в цикле для 0, 100 и -40 и выведите пары «Цельсий — Фаренгейт».

Что должно получиться: `0 32.0`, `100 212.0`, `-40 -40.0`. Решение — в конце файла `examples/python-answers/07_functions_answers.py`.

В симуляторе: миссии Python: 25–26 в панели «**Обучение**».

ГЛАВА 7

Pioneer SDK: полёт из Python

Цели главы

- Запускать Python-программу в симуляторе, на настоящем дроне и с компьютера на симулятор.
- Подключаться к дрону, читать телеметрию и управлять подсветкой через `led_control`.
- Выполнять взлёт и посадку безопасно: пауза после `arm()`, ожидание после `land()`.
- Летать по точкам и фигурам, дожидаясь каждой точки через `point_reached()`.
- Ждать подтверждений с ограничением по времени, читать измерения и сравнивать их с целью с допуском.

Миссии симулятора к главе

Язык «**Python**», панель «**Обучение**». Порядок миссий совпадает с порядком уроков главы.

- **10–18 «Сигналы и подсветка»**: первый красный свет, номер светодиода, зелёный и синий, смешиваем белый, выключаем сигнал, свет и подтверждение, цвет как переменная, таблица сигналов, световой отчёт подготовки.
- **28–36 «Время и события»**: пауза делает цвет видимым, отложенный сигнал, два таймера, повторяющийся ритм, получаем уведомление, не реагируем на всё подряд, помним текущий этап, ждём не бесконечно, готовность без угадывания.
- **37–45 «Высота и координаты»**: первая высота, три оси, знак направления, где начало, одна точка маршрута, высота над поверхностью, расстояние до цели, допуск прибытия, курс корпуса.
- **46–54 «Маршрут по этапам»**: подтверждаем точку, две точки, угол маршрута, замыкаем квадрат, список точек, возвращение домой, первая посадка, полный короткий полёт, повторяемость маршрута.
- **55–63 «Полигоны и местность»**: один seed — одна карта, свободная стартовая зона, лесной коридор, холмы и высота, обход здания, промышленный участок, гоночный полигон, новый seed, маршрут через разные зоны.
- **64–72 «Измерения и устойчивость»**: читаем положение, как часто измерять, нет данных, достигли высоты, ошибка положения, не дёргаемся у границы, ограничиваем действие, причина остановки, отчёт полёта.

7.1 Где запускать Python

Для управления Пионером из Python есть библиотека `pioneer_sdk`: команды взлёта, посадки, полёта в точку, светодиоды, телеметрия, камера. У Python-программы, как и у Lua, два исполнителя.

В симуляторе

Выберите в редакторе язык «**Python**». Библиотека уже «установлена»: `from pioneer_sdk import Pioneer` работает, а дрон на сцене выполняет команды. Вывод `print` — в терминале, как и для Lua. Программа выполняется в браузере (интерпретатор Pyodide), поэтому адрес в `Pioneer(ip=...)` здесь ни на что не влияет: команды получает выбранный на сцене дрон.

> **Чего нет у настоящего pioneer_sdk**
>
> Справочник редактора (кнопка с книгой) описывает и команды, которые есть только в симуляторе: `Camera(view="bottom")` — нижняя камера (глава 8), `get_optical_flow()` — датчик оптического потока, `Vehicle` — машины и поезда сцены. `get_yaw()` и `cargo_grab()`/`cargo_release()` есть лишь в версии pioneer_sdk 0.6.1 с GitFlic и работают только при `Pioneer(simulator=True)`. В программах для настоящего дрона этих команд не используйте.

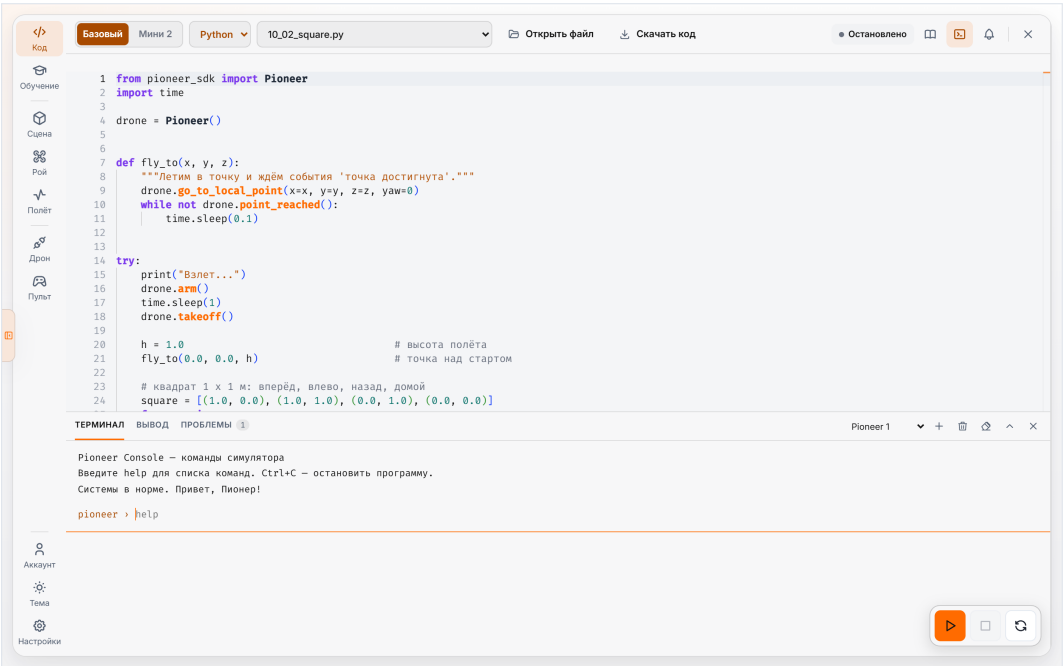


Рис. 18. Редактор в режиме Python. Переключение языка не теряет Lua-программу: у каждого языка своя вкладка.

На настоящем дроне

Python-программа работает на вашем компьютере и общается с дроном по Wi-Fi:

Терминал

```
pip install pioneer-sdk
```

Дрон создаёт сеть Wi-Fi (имя вида `Pioneer_...`); подключитесь к ней — и `Pioneer()` найдёт дрон по адресу по умолчанию.

С компьютера — на симулятор

Третий вариант: программа на вашем компьютере, а летает дрон в симуляторе. На сайте у дрона включается «**Разрешить внешние команды**», а в программе вместо адреса дрона указывается адрес сайта:

Подключение к симулятору

```
from pioneer_sdk import Pioneer
pioneer = Pioneer(ip='simulator.sakoryagin.ru',
                  mavlink_port=8001)
```

Точный фрагмент подключения (у каждого дрона сцены свой порт) показывает панель подключения дрона; там же — фрагмент для камеры. Дальше программа ничем не отличается от программы для настоящего дрона. Так удобно отлаживать то, что в браузере недоступно: сторонние библиотеки, запись файлов, полноценный OpenCV.

> **Lua ждёт событий, Python — опрашивает**
>
> В Lua следующую команду отдают из `callback` по событию. В Python программа сама периодически спрашивает автопилот: `while not pioneer.point_reached(): time.sleep(0.1)`. Смысл тот же — не отдавать следующую команду, пока предыдущая не выполнена, — меняется только форма.

ЗАДАНИЕ 7.1 · Первая связь

В симуляторе напишите программу из трёх строк: импорт `Pioneer`, создание объекта `pioneer = Pioneer()` и вывод `pioneer.get_autopilot_state()`. Затем добавьте перед выводом `pioneer.arm()` и сравните результат. Дрон не должен взлететь.

Что должно получиться: сначала в терминале `DISARMED`; после добавления `arm()` — `ARMED`, винты вращаются, дрон стоит на площадке.

В симуляторе: миссии Python: 3–5 в панели «**Обучение**».

7.2 Подключение и телеметрия

python/02_pioneer_sdk/08_drone_connect.py

```python
from pioneer_sdk import Pioneer
import time

# Подключение к дрону
drone = Pioneer()
print("Дрон подключен!")

try:
    # Запрашиваем состояние батареи несколько раз
    for _ in range(3):
        voltage = drone.get_battery_status()
        print(f"Напряжение: {voltage} В")
        time.sleep(1)
finally:
    # Всегда закрываем соединение
    drone.close_connection()
    print("Соединение закрыто")
```

- `Pioneer()` — объект связи с дроном. Создаётся один раз.
- `get_battery_status()` — напряжение батареи в вольтах. Если данных ещё нет, команда возвращает `None`: отсутствующее измерение — не ноль, его проверяют до сравнения с порогом (`if voltage is not None:`). Виртуальный дрон измерения отдаёт всегда, поэтому ветку «нет данных» в симуляторе проверяют подстановкой: `voltage = None` перед проверкой.
- `try: ... finally: ...` — конструкция Python: сначала выполняется блок `try`, а блок `finally` выполняется *всегда* после него, даже если в `try` произошла ошибка. Поэтому `close_connection()` в `finally` закрывает связь даже при ошибке или `Ctrl+C`; иначе порт останется занят.

Напряжение батареи

Батарея Пионера в симуляторе двухэлементная: полностью заряженная показывает около 8.4 В, при 7.2 В автопилот предупреждает о низком заряде (пора возвращаться), при 6.9 В начинает аварийную посадку. Пороги в своих программах берите из этих чисел, а не из напряжения одного элемента (4.2 В).

ЗАДАНИЕ 7.2 · Диагностика

Подключитесь к дрону, прочитайте напряжение и сохраните порог в переменной `LOW = 7.2`. Если данных нет, напечатайте «Нет данных»; если напряжение ниже порога — «Low Battery!», иначе — «Батарея в норме» и само напряжение с подписью «В». Соединение закройте в `finally`.

Что должно получиться: в симуляторе со свежей батареей — `Батарея в норме 8.4 В` (число может немного отличаться); при `LOW = 9` — `Low Battery!`.

В симуляторе: миссии Python: 64, 66, 109 в панели «**Обучение**».

7.3 Светодиоды из Python

Первый видимый результат без взлёта — как и в Lua, подсветка. Метод один:

led_control

`pioneer.led_control(led_id=255, r=0, g=255, b=0)` — цвет светодиода `led_id`: номера от `0` до `3` выбирают один из четырёх светодиодов, `255` означает «все сразу». Компоненты — целые числа от `0` до `255` (в Lua было `0..1`). Настоящая библиотека значения вне диапазона отвергает ошибкой `ValueError`. Аргументы можно писать и без имён, по порядку: `pioneer.led_control(0, 255, 0, 0)` — светодиод 0 красный.

`(255, 0, 0)` красный	`(0, 255, 0)` зелёный	`(0, 0, 255)` синий
`(255, 255, 0)` жёлтый	`(255, 255, 255)` белый	`(0, 0, 0)` выключено

Команда цвета не задаёт, сколько цвет будет гореть: следующая команда сразу его заменит. Чтобы цвет было видно, между командами ставят паузу `time.sleep(секунды)`. И ещё одно: строка `print("зелёный")` в журнале подтверждает только то, что программа дошла до этой строки. Правильность цвета проверяют на сцене.

Пример связывает цвет с показаниями дальномера — датчика расстояния до поверхности под дроном:

python/03_flight_control/LED_hight_change.py

```python
from pioneer_sdk import Pioneer
import time

# Определяем временной интервал обновления данных
delta_time = 0.1

# Инициализируем переменные для цвета светодиодов (RGB)
r = 0
g = 0
b = 0

# Определяем пороговые значения высоты для смены цвета
height_1 = 0.25
height_2 = 0.5
height_3 = 0.75

#  <------- height_1 ------- height_2 ------- height_3 ------->
#   красный          зеленый           синий           белый

if __name__ == "__main__":                    # Проверяем, что скрипт
↪  запущен как основной файл
    pioneer_mini = Pioneer(logger=False)      # Создаем объект управления
    ↪  дроном, отключая логирование
    curr_time = time.time()                   # Фиксируем текущее время

    try:
```

```
        while True:                                                  # Запускаем
        ↪  бесконечный цикл
            if time.time() - curr_time > delta_time:                 # Проверяем,
            ↪  прошло ли заданное время (delta_time)
                tof_data = pioneer_mini.get_dist_sensor_data()  # Получаем
                ↪  данные с датчика расстояния
                print(tof_data)                                      # Выводим
                ↪  данные в консоль для мониторинга

                if tof_data is not None:                             # Проверяем,
                ↪  что данные успешно получены с датчика

                    # Определяем цвет светодиодов в зависимости от высоты
                    if tof_data <= height_1:
                        r = 255  # Красный
                        g = 0
                        b = 0
                    elif height_1 < tof_data <= height_2:
                        r = 0
                        g = 255  # Зеленый
                        b = 0
                    elif height_2 < tof_data <= height_3:
                        r = 0
                        g = 0
                        b = 255  # Синий
                    elif tof_data >= height_3:
                        r = 255
                        g = 255
                        b = 255  # Белый

                    # Управляем светодиодами дрона
                    pioneer_mini.led_control(led_id=255, r=r, g=g, b=b)

                    # Обновляем время последнего обновления
                    curr_time = time.time()
            time.sleep(0.01)                                         # Короткая
            ↪  пауза: без неё цикл грузит процессор на 100%

    except KeyboardInterrupt:
        print("Остановка программы. Выключаем светодиоды...")
        pioneer_mini.led_control(led_id=255, r=0, g=0, b=0)  # Выключаем
        ↪  светодиоды

    finally:
        pioneer_mini.close_connection()                          # Закрываем
        ↪  соединение с дроном
```

Что происходит. Бесконечный цикл `while True` раз в 0.1 с читает дальномер `get_dist_sensor_data()` и выбирает цвет по порогам высоты. В Python такой цикл допустим: программа работает на компьютере, автопилот дрона он не блокирует. Выход — `Ctrl+C` (в симуляторе — кнопка остановки): блок `except KeyboardInterrupt` гасит подсветку, `finally` закрывает соединение. На настоящем дроне поднимите его рукой и смотрите, как меняется цвет; в симуляторе для этого нужно взлететь — добавьте в начало программы `arm()`, `takeoff()` и точку на высоте 1 м (следующие разделы).

ЗАДАНИЕ 7.3 · Первая индикация

Подключитесь, включите все светодиоды зелёным на 2 с, затем светодиод 0 — красным на 1 с, затем выключите все и корректно закройте соединение. Яркость держите в одной переменной `BRIGHT = 255` и подставляйте её во все команды цвета: поменяли число в одном месте — изменилась яркость всей программы.

Что должно получиться: на сцене: две секунды зелёные все четыре светодиода, секунду красный только один, затем подсветка гаснет; программа завершается без ошибок.

В симуляторе: миссии Python: 10–16 в панели «**Обучение**».

ЗАДАНИЕ 7.4 · Светофор батареи

Раз в секунду читайте напряжение и показывайте его цветом: выше 8.0 В — зелёный, от 7.2 до 8.0 — жёлтый, ниже 7.2 — красный, нет данных — подсветка выключена. Пороги храните в переменных, выбор цвета оформите функцией `battery_color(voltage)`, которая возвращает тройку `(r, g, b)`.

Что должно получиться: со свежей батареей симулятора дрон светится зелёным; если временно поменять порог 8.0 на 9.0 — жёлтым.

В симуляторе: миссии Python: 17–18, 110 в панели «**Обучение**».

7.4 Паузы, подтверждения и тайм-ауты

В Lua следующее действие запускает событие: таймер сработал, моторы запустились, точка достигнута. В Python программа идёт сверху вниз, и время в ней задают по-другому.

- **Пауза.** `time.sleep(1)` останавливает программу на секунду: всё, что ниже, выполнится позже. Паузы идут одна за другой и складываются — две паузы по секунде дают действие на второй секунде от старта. Мигание — это цикл из света, паузы, темноты и паузы.
- **Подтверждение.** Команда — это просьба. Выполнена ли она, сообщает состояние автопилота `get_autopilot_state()`: `"DISARMED"` — моторы выключены, `"ARMED"` — моторы запущены, дрон на земле, `"TAKEOFF"` — идёт взлёт, `"MISSION"` — дрон в воздухе и готов к командам полёта, `"LANDING"` — посадка. Есть и короткий способ: `arm()` возвращает `True`, если команда принята.
- **Тайм-аут.** Ждать подтверждения бесконечно нельзя: если оно не придёт, программа зависнет. Ожидание ограничивают сроком: `deadline = time.monotonic() + 3` и цикл, пока не наступило состояние *и* не истёк срок. После цикла проверяют, чем он закончился.

python/03_flight_control/10_05_wait_state.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()


def wait_state(state, limit):
    """Ждать состояния автопилота не дольше limit секунд.

    Возвращает True, если состояние наступило, и False по тайм-ауту.
    """
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


try:
    drone.led_control(led_id=255, r=255, g=0, b=0)       # красный: ещё не
    ↪ готов
    drone.arm()
    if not wait_state("ARMED", 3):
        print("тайм-аут: подготовка не подтверждена")
    else:
        drone.led_control(led_id=255, r=0, g=255, b=0)   # зелёный: моторы
        ↪ запущены
        drone.takeoff()
        if wait_state("MISSION", 15):
            print("Взлёт завершён")
            time.sleep(2)                                # немного повисим
        else:
            print("тайм-аут: взлёт не завершился")
        drone.land()
        if wait_state("DISARMED", 15):                   # сел и сам выключил
        ↪ моторы
            print("Посадка подтверждена")
finally:
    drone.led_control(led_id=255, r=0, g=0, b=0)
    drone.disarm()
    drone.close_connection()
```

Что происходит. Функция `wait_state` опрашивает автопилот каждые 0.05 с и возвращает `True`, как только состояние наступило, или `False`, когда срок истёк. Подсветка показывает этап: красный — ещё не готов, зелёный — подготовка подтверждена. Взлёт отправляется только в успешной ветке, а в ветке тайм-аута программа сообщает о проблеме и ничего не делает в воздухе. `time.monotonic()` — часы, которые нельзя перевести назад; для измерения промежутков они надёжнее `time.time()`.

Пауза не доказывает готовность

`time.sleep(1)` после `arm()` — это надежда, что за секунду моторы запустятся. Подтверждение — состояние `"ARMED"`. Пауза нужна, чтобы человек успел увидеть результат; решение «можно дальше» принимают по подтверждению и с тайм-аутом.

ЗАДАНИЕ 7.5 · Мигание и готовность

Сделайте три полных синих мигания всеми светодиодами: полсекунды свет, полсекунды темнота. Затем отправьте `arm()` и подождите состояния `"ARMED"` не дольше 1.5 с: при успехе включите зелёный и напечатайте «готовность», иначе — «тайм-аут». Проверьте вторую ветку: уберите `arm()` из программы.

Что должно получиться: три синих вспышки за 3 с, затем зелёный и строка `готовность`; без `arm()` — через 1.5 с строка `тайм-аут`, подсветка выключена, моторы не запущены.

В симуляторе: миссии Python: 28–36 в панели «**Обучение**».

7.5 Первый полёт: моторы, взлёт, посадка

Моторы

python/02_pioneer_sdk/09_01_arm_disarm.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()
try:
    print("Запуск моторов...")
    drone.arm()
    time.sleep(3) # Моторы вращаются 3 секунды
finally:
    print("Остановка моторов")
    drone.disarm()
```

`arm()` запускает моторы на холостых оборотах, `disarm()` выключает. Блок `try ... finally` гарантирует выключение моторов даже при ошибке.

Взлёт и посадка

python/02_pioneer_sdk/09_02_takeoff_land.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()
try:
```

```
    drone.arm()
    time.sleep(1)          # даём автопилоту перейти в режим готовности
    print("Взлет!")
    drone.takeoff()
    time.sleep(5)          # взлёт занимает пару секунд, остальное — висим
    print("Посадка...")
    drone.land()
    time.sleep(5)          # ждём, пока дрон действительно сядет
finally:
    drone.disarm()         # гарантированная остановка моторов
    drone.close_connection()
```

Что происходит.

- `arm()` → секунда паузы: автопилот переходит в режим готовности. Без паузы команда взлёта может быть отвергнута. Надёжнее не пауза, а ожидание состояния `"ARMED"` с тайм-аутом (раздел 7.4).
- `takeoff()` — подъём на высоту взлёта (около метра). Команда возвращается сразу, взлёт занимает пару секунд; остаток из пяти секунд дрон висит. Конец взлёта подтверждает состояние `"MISSION"`: только после него дрон принимает точки маршрута.
- `land()` — посадка. Она тоже занимает время, и `disarm()` в `finally` выполнится *сразу* после `land()`, если ничего не ждать. Поэтому после посадки — ещё `time.sleep(5)`: выключать моторы можно только на земле.

> **disarm в воздухе**
>
> `disarm()` выключает моторы немедленно. Вызывайте его только когда дрон сел: после `land()` подождите несколько секунд или дождитесь состояния `"DISARMED"` — после касания земли автопилот сам выключает моторы.

ЗАДАНИЕ 7.6 · Первая высота

Перепишите программу взлёта и посадки без угадывания времени: после `arm()` ждите `"ARMED"` (не дольше 3 с), после `takeoff()` — `"MISSION"` (не дольше 15 с), повисите 3 с, после `land()` ждите `"DISARMED"`. На каждом этапе печатайте его название. Функцию `wait_state` возьмите из раздела 7.4.

Что должно получиться: дрон поднимается примерно на 1 м, висит, садится на место старта; в терминале строки этапов идут в порядке «подготовка», «взлёт», «посадка», «сели», а строки «тайм-аут» нет.

В симуляторе: миссии Python: 37, 52 в панели «**Обучение**».

7.6 Полёт по точкам

Система координат

Та же, что в Lua (раздел 5): X вперёд, Y влево, Z вверх, метры от начала координат сцены. Знак выбирает сторону: `x=-1` — метр назад. Команда:

go_to_local_point

`pioneer.go_to_local_point(x=1.0, y=0.0, z=1.0, yaw=0)` — лететь в точку; `yaw` — курс в радианах. В симуляторе курс можно опустить, но указывайте его всегда: программа будет одинаково вести себя и на дроне. Автопилот подтверждает прибытие: `pioneer.point_reached()` вернёт `True` — один раз на каждую точку.

Две вещи, на которых чаще всего ошибаются:

- **Z — не расстояние до земли.** `z` отсчитывается от начала координат сцены. Над холмом или крышей поверхность ближе: сколько осталось до неё, показывает дальномер `get_dist_sensor_data()`.
- **Прибытие — с допуском.** Дрон не останавливается точно в заданных числах: точка засчитывается, когда он оказался ближе нескольких сантиметров к цели (в симуляторе — около 0.15 м). Поэтому измеренная координата после прибытия — `1.93`, а не `2.0`.

Вперёд и назад

python/03_flight_control/10_01_forward_back.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()


def fly_to(x, y, z):
    """Летим в точку и ждём, пока автопилот сообщит, что она достигнута."""
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    while not drone.point_reached():
        time.sleep(0.1)


try:
    print("Арминг и взлет...")
    drone.arm()
    time.sleep(1)            # даём автопилоту перейти в режим готовности
    drone.takeoff()

    h = 1.0                  # высота полёта
    fly_to(0.0, 0.0, h)      # сначала занимаем точку над стартом

    print("Вперед!")
    fly_to(1.0, 0.0, h)      # 1 метр вперёд по оси X
```

```
    print("Домой!")
    fly_to(0.0, 0.0, h)       # обратно в точку старта

    print("Посадка...")
    drone.land()
    time.sleep(5)             # ждём, пока дрон сядет
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. Вспомогательная функция `fly_to` отправляет точку и ждёт `point_reached()`, опрашивая автопилот каждые 0.1 с. Это Python-эквивалент события `POINT_REACHED`: следующая точка — только после подтверждения предыдущей. Если отправить две точки подряд без ожидания, вторая просто заменит первую. Первая точка — над стартом: так программа дожидается конца взлёта.

> **Не угадывать время**
>
> `time.sleep(4)` после `go_to_local_point` — плохой способ ждать: перелёт может занять и 2, и 6 секунд. Ждите `point_reached()`. В собственных программах добавляйте к ожиданию и тайм-аут, как в `wait_state`: если точка недостижима, программа не зависнет.

Квадрат

python/03_flight_control/10_02_square.py

```
from pioneer_sdk import Pioneer
import time

drone = Pioneer()


def fly_to(x, y, z):
    """Летим в точку и ждём события 'точка достигнута'."""
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    while not drone.point_reached():
        time.sleep(0.1)


try:
    print("Взлет...")
    drone.arm()
    time.sleep(1)
    drone.takeoff()

    h = 1.0                                  # высота полёта
    fly_to(0.0, 0.0, h)                      # точка над стартом

    # квадрат 1 x 1 м: вперёд, влево, назад, домой
```

```
    square = [(1.0, 0.0), (1.0, 1.0), (0.0, 1.0), (0.0, 0.0)]
    for x, y in square:
        print(f"Летим в ({x}, {y})")
        fly_to(x, y, h)

    drone.land()
    time.sleep(5)
finally:
    drone.disarm()
    drone.close_connection()
```

Точки — **список** пар `(x, y)`. Список в Python — упорядоченный набор значений в квадратных скобках, аналог таблицы-массива Lua, но нумерация с нуля: `square[0]` — первая вершина. Пара в круглых скобках — кортеж, неизменяемый набор значений; запись `for x, y in square` на каждом шаге раскладывает очередную пару в две переменные, как `table.unpack` в Lua. Добавить вершину — одна строка. Координаты маршрута отделены от логики полёта: один и тот же цикл пролетит любой список.

Домой — это сохранённая точка

«Вернуться домой» не обязательно значит лететь в $(0, 0)$: дрон мог стартовать не в начале координат. Положение старта запоминают до взлёта и возвращаются в него:

Сохранить дом

```
home = drone.get_local_position_lps()   # [x, y, z] или None
...
if home is not None:
    fly_to(home[0], home[1], 1.0)
```

Окружность

python/03_flight_control/10_03_circle_math.py

```
from pioneer_sdk import Pioneer
import time
import math

drone = Pioneer()


def fly_to(x, y, z):
    """Летим в точку и ждём события 'точка достигнута'."""
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    while not drone.point_reached():
        time.sleep(0.1)


try:
    print("Взлет...")
```

```
    drone.arm()
    time.sleep(1)
    drone.takeoff()

    h = 1.0             # высота полёта
    radius = 0.5        # радиус окружности, м
    points_count = 12   # чем больше точек, тем ровнее круг

    # окружность с центром в точке старта: угол от 0 град. до 360 град. с
    ↪ шагом 30 град.
    for i in range(points_count + 1):
        angle_deg = 360 / points_count * i
        angle_rad = math.radians(angle_deg)   # sin и cos принимают радианы
        x = radius * math.cos(angle_rad)
        y = radius * math.sin(angle_rad)
        print(f"Точка {i}: x={x:.2f}, y={y:.2f}")
        fly_to(x, y, h)

    drone.land()
    time.sleep(5)
finally:
    drone.disarm()
    drone.close_connection()
```

Двенадцать точек через $30°$: $x = R\cos\alpha$, $y = R\sin\alpha$, угол переводится в радианы функцией `math.radians`. Последняя, тринадцатая точка совпадает с первой — круг замыкается.

ЗАДАНИЕ 7.7 · Угол и квадрат

Задайте маршрут списком точек `[(2, 0), (2, 2), (0, 2), (0, 0)]` на высоте 1 м и пролетите его функцией `fly_to`. Перед взлётом сохраните положение старта, а после последней вершины вернитесь в него и сядьте. В каждой вершине печатайте её номер.

Что должно получиться: дрон по очереди посещает все четыре вершины квадрата 2×2 м (а не срезает угол по диагонали), возвращается к месту старта и садится; в терминале номера вершин 0, 1, 2, 3.

В симуляторе: миссии Python: 46–51 в панели «**Обучение**».

ЗАДАНИЕ 7.8 · Восьмёрка

Соедините две окружности радиусом 0.5 м с центрами $(0.5, 0)$ и $(-0.5, 0)$ в одну траекторию: точки первой окружности обходите против часовой стрелки, второй — по часовой. Начните и закончите в точке $(0, 0)$, генерацию точек одной окружности оформите функцией `circle_points(cx, cy, r, n, direction)`.

Что должно получиться: след дрона на сцене — восьмёрка, обе петли одного размера и касаются в точке старта; дрон садится там, откуда взлетел.

В симуляторе: миссии Python: 53–54 в панели «**Обучение**».

7.7 Полигоны и местность

На ровном поле годится любой маршрут. В миссиях 55–63 сцену заполняет генератор: лес, холмы, городские дома, промышленный участок с мачтами, гоночная трасса с воротами. Расстановку он строит по числу **seed**: при том же seed карта повторяется до сантиметра, поэтому трудный маршрут можно отлаживать запуск за запуском. Новый seed — новая карта, и прежние координаты обхода могут стать неверными.

Правила маршрута среди препятствий:

- **Сначала осмотреться.** Перед программой облетите сцену камерой редактора и выпишите координаты препятствий и свободных коридоров.
- **Габарит, а не центр.** Точка маршрута — это центр дрона, а винты на десятки сантиметров дальше. Между линией полёта и кроной, мачтой или фасадом оставляйте боковой запас.
- **Высота — до препятствия.** Над холмами сначала поднимаются над стартом на безопасную высоту и только потом летят вперёд: при подъёме «по пути» дрон врежется в склон.
- **Обход — из простых отрезков.** Если прямая к цели пересекает здание, маршрут строят из точек сбоку от него, с запасом от фасада.
- **Ворота — три точки.** Чтобы пройти через середину проёма, ставят подводящую точку перед воротами, точку в центре и выходную — за ними, все на одной линии.

Маршрут по участкам

```
route = [
    (0, 0, 4),     # подъём над стартом до безопасной высоты
    (0, 12, 4),    # над холмами на той же высоте
    (0, 12, 1.5),  # снижение уже над целью
]
for x, y, z in route:
    fly_to(x, y, z)
```

ЗАДАНИЕ 7.9 · Обход здания

В городской сцене миссии 59 здание стоит в точке $(0, 6)$ и перекрывает прямую от старта к финишу $(0, 12)$. Составьте маршрут на высоте 3 м из точек, которые обходят здание справа вдоль фасада с запасом не меньше метра, и пролетите его списком через `fly_to`.

Что должно получиться: дрон доходит до $(0, 12, 3)$, ни разу не коснувшись здания; при повторном запуске с тем же seed траектория та же.

В симуляторе: миссии Python: 59 в панели «**Обучение**».

7.8 Измерения: положение, ошибка, допуск

Команда сообщает, куда дрон *должен* лететь; где он *есть*, сообщает телеметрия. `get_local_position_lps()` возвращает измеренные `[x, y, z]` в метрах или `None`, если данных нет. Из измерения и цели получается **ошибка** — «цель минус измерение»: её знак показывает, в какую сторону ещё лететь, величина — сколько.

python/03_flight_control/10_06_measure_error.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()

TARGET_X = 1.0      # цель по оси X, м
TOLERANCE = 0.15    # допуск прибытия, м


def wait_state(state, limit):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


def read_x():
    """Измеренная координата X или None, если данных нет."""
    position = drone.get_local_position_lps()
    if position is None:        # нет измерения — это не ноль
        return None
    return position[0]


try:
    drone.arm()
    wait_state("ARMED", 3)
    drone.takeoff()
    wait_state("MISSION", 15)

    drone.go_to_local_point(x=TARGET_X, y=0, z=1, yaw=0)
    reason = "тайм-аут"
    deadline = time.monotonic() + 20
    while time.monotonic() < deadline:
        x = read_x()
        if x is None:
            print("Нет данных")
        else:
            error = TARGET_X - x            # цель минус измерение
            print(f"x = {x:.2f} м, осталось {error:.2f} м")
            if abs(error) < TOLERANCE:      # сравниваем с допуском, а не на
            ↪  равенство
                reason = "цель в допуске"
                break
        time.sleep(0.5)
```

```
    print("Причина остановки:", reason)

    drone.land()
    wait_state("DISARMED", 15)
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. Раз в полсекунды программа читает положение и печатает, сколько осталось до цели. Отсутствие данных обрабатывается отдельной веткой — считать его нулём нельзя: «ноль метров до цели» выглядело бы как прибытие. Виртуальный дрон положение отдаёт всегда, а дальномер над самой землёй показывает `0`, а не `None`; на настоящем дроне `None` встречается, поэтому ветка остаётся. Прибытие проверяется с допуском `abs(error) < TOLERANCE`, а не равенством `x == 1.0`, которое почти никогда не выполнится. В конце программа сообщает *причину* остановки: «цель в допуске» или «тайм-аут» — по журналу видно, чем закончился этап.

Ещё три приёма понадобятся в главе про камеру:

- **Частота опроса.** Слишком редкий опрос пропускает быстрые изменения, слишком частый засоряет журнал. Для журнала хватает 2–5 раз в секунду, для регулятора нужно 10–20.
- **Мёртвая зона.** Около границы измерение «дрожит», и решение переключается туда-обратно. Внутри небольшой зоны вокруг цели действие не требуется: `0 if abs(error) <= 0.1 else ...`.
- **Ограничение.** Вычисленная величина должна оставаться в допустимых пределах: `max(-limit, min(limit, value))` обрезает `value` до диапазона от `-limit` до `limit`. Большая ошибка не означает неограниченную скорость.

ЗАДАНИЕ 7.10 · Отчёт полёта

Взлетите и долетите до точки $(2, 0, 1)$. Пока дрон летит, раз в 0.5 с печатайте «x=… ошибка=…», а функцией `clamp(value, limit)` — скорость, которую выдал бы регулятор `0.5 * error`, ограниченную 0.4 м/с. После прибытия напечатайте измеренный X, «В допуске» или «Вне допуска» (допуск 0.15 м) и причину остановки этапа.

Что должно получиться: ошибка в журнале уменьшается от 2 к нулю, расчётная скорость сначала равна 0.4 и уменьшается только в последнем метре; после прибытия X близок к 2, но не равен ему ровно, и печатается `В допуске`.

В симуляторе: миссии Python: 64–72 в панели «**Обучение**».

7.9 Управление скоростью

Точка отвечает на вопрос «куда лететь». Иногда нужно другое — «с какой скоростью лететь прямо сейчас»: держать маркер в центре кадра, плавно снижаться по дальномеру,

следовать за человеком. Для этого есть команда скорости:

set_manual_speed

`pioneer.set_manual_speed(vx=0.5, vy=0, vz=0, yaw_rate=0)` — скорость в м/с по осям X, Y, Z той же системы координат, что у `go_to_local_point`, и скорость поворота `yaw_rate` в рад/с. Команда действует коротко (в симуляторе — около 0.5 с), поэтому её повторяют в цикле каждые 0.05–0.1 с. Перестали присылать — дрон зависает на месте. Вариант `set_manual_speed_body_fixed` задаёт те же скорости относительно носа дрона.

python/03_flight_control/10_04_manual_speed.py

```python
from pioneer_sdk import Pioneer
import time

drone = Pioneer()


def move(vx, vy, vz, seconds):
    """Лететь с заданной скоростью (м/с) seconds секунд, затем зависнуть.

    Команда скорости действует коротко, поэтому её повторяют в цикле
    каждые 0.05 с — так же устроен официальный пример set_manual_speed.py.
    """
    start = time.time()
    while time.time() - start < seconds:
        drone.set_manual_speed(vx=vx, vy=vy, vz=vz, yaw_rate=0)
        time.sleep(0.05)
    drone.set_manual_speed(vx=0, vy=0, vz=0, yaw_rate=0)   # остановиться


try:
    drone.arm()
    time.sleep(1)                # даём автопилоту перейти в режим готовности
    drone.takeoff()
    time.sleep(5)                # взлёт и зависание

    move(0.5, 0, 0, 2)           # 2 с по оси X со скоростью 0.5 м/с
    move(0, 0.5, 0, 2)           # 2 с по оси Y
    move(0, 0, 0.3, 1)           # 1 с вверх
    time.sleep(1)

    drone.land()
    time.sleep(5)                # ждём, пока дрон сядет
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. Функция `move` двадцать раз в секунду повторяет одну и ту же скорость, пока не пройдёт заданное время, и затем присылает нулевую скорость. Здесь ждать по времени правильно: длительность движения задаём мы сами, автопилоту «прибывать» некуда. На этой команде построены задания глав про камеру и навигацию: скорость пропорциональна ошибке (P-регулятор) и пересчитывается на каждом кадре.

ЗАДАНИЕ 7.11 · Короткий импульс

Взлетите на высоту 1 м над стартом, запомните измеренный X. Затем функцией `move` дайте импульс 0.3 м/с по оси X длительностью 1 с, явно остановитесь нулевой скоростью, подождите 1 с и напечатайте, на сколько сместился дрон. Повторите с длительностью 2 с.

Что должно получиться: после первого импульса смещение порядка 0.3 м, после второго — порядка 0.6 м (точные числа зависят от разгона); после нулевой команды дрон висит на месте, а не продолжает лететь.

В симуляторе: миссии Python: 70 в панели «**Обучение**».

7.10 Контрольные упражнения с проверкой

Для тренировки точности есть три упражнения с модулем `grader.py`: он записывает траекторию и сравнивает положение дрона с целью в контрольных точках (допуск 0.5 м по горизонтали, 0.2 м по высоте, 10° по курсу).

python/exercises/ex1_forward.py

```python
from pioneer_sdk import Pioneer
from grader import FlightGrader
import time

"""
Упражнение 1 – «Полёт вперёд»: модуль должен обеспечить ровный прямолинейный
↪  проход заданной дистанции без возврата.
"""

def exercise_01():
    drone = Pioneer()
    grader = FlightGrader(drone)

    try:
        print("[System] Упражнение 1: Полет вперед")
        print("Ожидание LPS...")
        # Проверка готовности LPS (упрощенно)

        grader.start_mission()

        # 1. Взлет
        print("Взлет...")
        drone.arm()
        time.sleep(1) # Даем автопилоту перейти в режим готовности
        drone.takeoff()
        time.sleep(5)

        # 2. Установка цели (x=1.0, y=0, z=1.0)
        target_x = 1.0
        grader.set_target(target_x, 0.0, 1.0, 0.0)

        # 3. Полет вперед
        print(f"Полет к точке x={target_x}...")
```

```
        drone.go_to_local_point(x=target_x, y=0.0, z=1.0, yaw=0.0)

        # Ожидание прибытия (около 5 с)
        time.sleep(5)

        # 4. Проверка позиции
        grader.check_checkpoint("Точка Вперед")

        # 5. Посадка (Конец упражнения, без возврата)
        print("Посадка...")
        drone.land()
        time.sleep(5)  # ждём, пока дрон сядет: disarm() в finally выключает
        ↪  моторы сразу

    finally:
        grader.finish_mission()
        drone.disarm()
        drone.close_connection()

if __name__ == '__main__':
    exercise_01()
```

python/exercises/ex2_forward_turn_return.py

```
from pioneer_sdk import Pioneer
from grader import FlightGrader
import time
import math

"""
Упражнение 2 – «Полёт вперёд и назад»: модуль должен выполнить точный проход
↪  вперёд на заданное расстояние, немедленный разворот на 180 град. и
↪  возвращение на исходную точку без смещения по бокам.
"""

def exercise_02():
    drone = Pioneer()
    grader = FlightGrader(drone)

    try:
        print("[System] Упражнение 2: Вперед и Назад (с разворотом)")
        grader.start_mission()

        # 1. Взлет
        drone.arm()
        time.sleep(1) # Даем автопилоту перейти в режим готовности
        drone.takeoff()
        time.sleep(5)

        # 2. Полет вперед (x=1.0)
        dist = 1.0
        print(f"Этап 1: Вперед к x={dist}...")
        grader.set_target(dist, 0.0, 1.0, 0.0)
        drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=0.0)
        time.sleep(5)
        grader.check_checkpoint("Точка Вперед")
```

```
        # 3. Разворот на 180 (Немедленный)
        print("Этап 2: Немедленный разворот на 180...")
        # Примечание: go_to_local_point с новым yaw. Позиция остается
        ↪ прежней.
        # Используем радианы для yaw, так как это стандарт.
        yaw_180 = math.radians(180)

        grader.set_target(dist, 0.0, 1.0, 180.0) # Ожидаем yaw=180 в точке
        ↪ x=1
        drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=yaw_180)
        time.sleep(3)
        grader.check_checkpoint("Разворот 180")

        # 4. Возврат на старт (x=0)
        # Мы сохраняем yaw=180, то есть летим "носом" к точке старта.
        print("Этап 3: Возврат на старт...")
        grader.set_target(0.0, 0.0, 1.0, 180.0)
        drone.go_to_local_point(x=0.0, y=0.0, z=1.0, yaw=yaw_180)
        time.sleep(5)
        grader.check_checkpoint("Точка Дома (Задом)")

        drone.land()
        time.sleep(5)  # ждём, пока дрон сядет: disarm() в finally выключает
        ↪ моторы сразу

    finally:
        grader.finish_mission()
        drone.disarm()
        drone.close_connection()

if __name__ == '__main__':
    exercise_02()
```

Третье упражнение, `ex3_forward_turn_return.py`, отличается только строгим контролем курса на всех этапах. Файл `grader.py` должен лежать рядом с программой.

Почему здесь ждут по времени

В упражнениях после `go_to_local_point` стоит `time.sleep(5)`, а не ожидание `point_reached()`: `grader` проверяет, где дрон оказался к заданному моменту, — это часть задания на точность. В собственных программах действует правило «Не угадывать время» выше. После `land()` упражнения тоже ждут несколько секунд, чтобы `disarm()` в `finally` не выключил моторы в воздухе.

ЗАДАНИЕ 7.12 · Точно в цель

Запустите `ex1_forward.py` и `ex2_forward_turn_return.py` и прочитайте отчёт `grader`. Затем измените первое упражнение: цель $(1.5, 0.5, 1.2)$ с курсом $90°$. Обратите внимание на единицы: `grader.set_target` принимает курс в градусах (`90`), а `go_to_local_point` — в радианах (`math.pi / 2`). Запустите изменённую программу три раза подряд из одинаковых условий (сброс сцены перед каждым запуском).

Что должно получиться: в отчёте каждой контрольной точки — `PASSED`, в конце — `УСПЕХ: Все проверки пройдены!`; все три запуска дают одинаковый результат.

В симуляторе: миссии Python: 54 в панели «**Обучение**».

7.11 Проект главы

Задания проверяются в симуляторе: результат должен быть виден на сцене и в журнале.

ЗАДАНИЕ 7.13 · Калькулятор полёта

Задайте в переменных время полёта (минуты) и скорость (м/с). Функцией `distance(minutes, speed)` вычислите путь и выведите его в метрах и километрах с подписями. Проверьте на 5 мин и 2 м/с.

Что должно получиться: `600 м` и `0.6 км`.

ЗАДАНИЕ 7.14 · Умная батарея

Функция `check_battery(voltage)` возвращает «Отличный заряд» выше 8.0 В, «Средний» от 7.2 до 8.0 В и «Посадка немедленно!» ниже 7.2 В или при `None`. Вызовите её перед взлётом: при «Посадка немедленно!» программа не взлетает и показывает красный свет.

Что должно получиться: со свежей батареей симулятора дрон взлетает; если временно передать в функцию 7.0, дрон остаётся на земле с красной подсветкой.

В симуляторе: миссии Python: 110 в панели «**Обучение**».

ЗАДАНИЕ 7.15 · Квадрат с индикацией

Квадрат со стороной 1 м на высоте 1.5 м. Перед полётом к каждой вершине — свой цвет подсветки (список цветов той же длины, что список вершин), после посадки — подсветка выключена. Каждое ожидание — `point_reached()` с тайм-аутом, а не `time.sleep`; в конце программа печатает отчёт: сколько вершин подтверждено и причину остановки.

Что должно получиться: на сцене дрон проходит четыре вершины, цвет меняется перед каждым перелётом, после посадки подсветка погашена; в отчёте `4 из 4` и «маршрут завершён».

В симуляторе: миссии Python: 53 в панели «**Обучение**».

Итоги

- `Pioneer()` один раз; `close_connection()` и `disarm()` — в `finally`.
- `arm()` → `"ARMED"` → `takeoff()` → `"MISSION"` → точки через `go_to_local_point` и `point_reached()` → `land()` → `"DISARMED"` → `disarm()`.

- Любое ожидание — с тайм-аутом; пауза показывает результат человеку, но не доказывает готовность.
- Цвета в Python — `0..255`, курс — радианы, координаты — метры от начала сцены.
- Измерение может отсутствовать (`None`); прибытие проверяют с допуском, команды ограничивают пределом.
- Точка — `go_to_local_point`; скорость — `set_manual_speed`, повторяемая в цикле.

ГЛАВА 8

Камера, компьютерное зрение и рой

Цели главы

- Получать кадры с камеры дрона, понимать размеры кадра и его оси.
- Находить на кадре маркеры ArUco, выбирать нужный по ID и вычислять его центр и ошибку относительно центра кадра.
- Наводить дрон на маркер обратной связью и сажать его на маркер.
- На настоящем дроне — оценивать положение по маркеру, калибровать камеру, распознавать человека и управлять несколькими дронами.

Миссии симулятора к главе

Язык «**Python**», панель «**Обучение**». Уроки главы идут в том же порядке.

- **73–81 «Камера с нуля»**: подключаем камеру, ширина и высота кадра, право и низ на изображении, центр кадра, поворот и изображение, высота и масштаб, объект вне кадра, поток кадров, кадр недоступен.
- **82–90 «ArUco шаг за шагом»**: что закодировано в метке, словарь маркеров, первая детекция, ничего не найдено, выбираем нужный ID, четыре угла, центр маркера, ошибка относительно центра, стабильное распознавание.
- **91–99 «Наведение по метке»**: измеряем без движения, определяем направление, короткое исправление, двигаемся медленнее у цели, не дрожим в центре, две оси наведения, потеряли метку, снижаемся только над целью, посадка по ArUco.
- **100–108 «Самостоятельные миссии»** — итоговые проекты раздела 9.

Что из главы работает в симуляторе

Симулятор снимает кадры виртуальной камерой (всегда 640×360) и подменяет библиотеку `cv2` упрощённой моделью. Маркеры она находит не по пикселям, а по геометрии: метка считается найденной, если она целиком в кадре, обращена к камере, не закрыта и занимает не меньше 10 пикселей по стороне. Поэтому логику распознавания и наведения отлаживают в браузере, а качество настоящего OpenCV на шумных фотографиях, освещение и печать меток проверяют только на дроне.

Работает в симуляторе	Только компьютер или настоящий дрон
`Camera()` — передняя камера (вертикальный угол обзора 80°), `Camera(view="bottom")` — нижняя (72°); `get_cv_frame()`, `frame.shape`	RTSP-поток `VideoStream` для плеера VLC; шумы, блики и смаз настоящей оптики
`cv2.imshow`, `waitKey`, рисование (`circle`, `line`, `rectangle`, `putText`), `cvtColor`, `resize`, `inRange`, контуры и моменты	`cv2.imwrite` (в симуляторе возвращает `True`, но файла не создаёт), `imread`, `absdiff`, `threshold`, `GaussianBlur`, `undistort`
`cv2.aruco`: словари, `ArucoDetector`, `detectMarkers`, `drawDetectedMarkers`	калибровка камеры (`findChessboardCorners`, `calibrateCamera`): камера симулятора идеальная
`cv2.solvePnP`, `estimatePoseSingleMarkers`, `drawFrameAxes`; `cv2.FileStorage` отдаёт матрицу идеальной нижней камеры (без дисторсии)	`mediapipe` (импорт завершится ошибкой)
`set_manual_speed` для наведения по кадрам	рой из нескольких настоящих дронов по Wi-Fi (раздел о групповом полёте)

⚠ Программы с пометкой «только на дроне»

Примеры с потоками, калибровкой, распознаванием человека и роем написаны для компьютера, подключённого к настоящему Пионеру. В браузере они либо завершатся ошибкой (импорт `mediapipe`, функции, которых в учебной модели нет), либо будут «работать» без результата: `imwrite` не создаст файл. Это ограничение среды, а не ошибка в программе. Программу с компьютера можно направить и на дрон в симуляторе (раздел 7.1): тогда работает настоящий OpenCV, но камеру симулятора нужно подключать фрагментом из панели подключения.

8.1 Камера с нуля

Камера отдаёт **кадры** — картинки из выбранного направления. Прежде чем что-то распознавать, нужно убедиться, что кадр приходит, и понять, как в нём устроены координаты.

python/camera_examples/13_01_first_frame.py

```python
from pioneer_sdk import Pioneer, Camera
import time
import cv2

drone = Pioneer()
# view="bottom" - расширение симулятора: нижняя камера, которая смотрит в
↪  пол.
# В pioneer_sdk для настоящего дрона такого аргумента нет: там пишут
↪  Camera().
camera = Camera(view="bottom")
```

```
def wait_state(state, limit):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


def go(x, y, z):
    """Лететь в точку, дождаться прибытия и дать корпусу успокоиться."""
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    deadline = time.monotonic() + 30
    while not drone.point_reached() and time.monotonic() < deadline:
        time.sleep(0.05)
    time.sleep(1.5)


try:
    drone.arm()
    wait_state("ARMED", 3)
    drone.takeoff()
    wait_state("MISSION", 15)
    go(0, 0, 3)                          # с высоты 3 м в кадре видны рисунки
    ↪ на полу

    for i in range(3):                   # три свежих кадра, а не один старый
        frame = camera.get_cv_frame()
        if frame is None:                # кадр не пришёл: дальше его не
        ↪ передаём
            print("Нет кадра")
        else:
            h, w = frame.shape[:2]       # сначала высота (строки), потом
            ↪ ширина
            center = (w // 2, h // 2)    # (x, y): x вправо, y вниз
            print(f"Кадр {i}: {w}x{h}, центр {center}")
            cv2.circle(frame, center, 4, (0, 0, 255), -1)   # красная точка,
            ↪ цвет BGR
            cv2.imshow("Нижняя камера", frame)
            cv2.waitKey(1)               # на компьютере без waitKey окно не
            ↪ обновится
        time.sleep(0.15)                 # пауза: следующий кадр будет новым

    drone.land()
    wait_state("DISARMED", 15)
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит.

- **Подключение.** `Camera(view="bottom")` в симуляторе выбирает нижнюю камеру: она смотрит в пол, и с высоты 3 м в кадр попадают рисунки и метки на земле. Аргумент `view` — расширение симулятора, у `Camera` настоящей библиотеки его нет: на

дроне пишут `Camera()`. Окно «Нижняя камера» на сцене (кнопка с камерой рядом с компасом) показывает то же, что получает программа.

- **Кадр может не прийти.** `get_cv_frame()` возвращает `None`, если кадра нет. Пустой кадр нельзя ни показывать, ни передавать распознаванию, ни тем более использовать для команд движения — поэтому проверка `frame is None` стоит сразу после чтения.
- **Размер.** Кадр — массив пикселей; `frame.shape` — это (высота, ширина, число цветов): сначала строки, потом столбцы. В симуляторе кадр всегда 640×360, на дроне размер зависит от камеры, поэтому его не записывают в программу числом, а берут из `shape`.
- **Оси.** Начало координат кадра — левый верхний угол; x растёт вправо, y — *вниз*. Центр кадра — `(w // 2, h // 2)`, в симуляторе $(320, 180)$. Точка задаётся парой `(x, y)`, то есть сначала по ширине — обратный порядок по сравнению с `shape`.
- **Цвет.** OpenCV хранит цвет в порядке BGR: `(0, 0, 255)` — красный.
- **Поток.** Решение принимается по свежему кадру, поэтому кадр читают внутри цикла. Пауза 0.15 с между кадрами гарантирует, что следующий будет новым: камера отдаёт не больше 30 кадров в секунду.
- **Окно.** В симуляторе `cv2.imshow` показывает кадр сразу; на компьютере окно обновляется только при вызове `cv2.waitKey(1)`, поэтому он стоит после `imshow` всегда.

Как движется картинка

При курсе 0 у нижней камеры симулятора правая сторона кадра смотрит вдоль $+X$ сцены, верхняя — вдоль $+Y$. Если дрон сместился на 30 см по $+X$, неподвижный рисунок на полу уезжает в кадре *влево*. Поворот корпуса вращает картинку, хотя ничего на полу не сдвинулось, а с большей высоты тот же рисунок занимает меньше пикселей. И если метки нет в кадре, это значит только, что камера её сейчас не видит, — на полигоне она осталась.

ЗАДАНИЕ 8.1 · Камера с нуля

Взлетите на высоту 3 м над стартом нижней камерой и снимите кадр. Сместитесь на 0.3 м по $+X$ и снимите второй кадр, затем поднимитесь до 4 м и снимите третий. Покажите кадры в трёх окнах с красной точкой в центре каждого, напечатайте размеры кадров. Добавьте проверку `None` и проверьте её, присвоив перед ней `frame = None`.

Что должно получиться: в терминале `640x360` и центр `(320, 180)`; на втором кадре рисунки на полу сдвинуты влево относительно первого, на третьем — меньше; с `frame = None` программа печатает «Нет кадра» и не падает.

В симуляторе: миссии Python: 73–81 в панели «**Обучение**».

8.2 ArUco: словарь и детекция

Для роботов существуют специальные визуальные метки (маркеры), которые легко распознать на изображении. Самые популярные — **ArUco** и **AprilTag**.

Что такое ArUco-маркер?

Это черно-белый квадрат с уникальным узором внутри. В узор закодирован **ID** — номер рисунка. ID только называет метку: ни расстояния, ни координаты из числа ID не получить. Что означает метка с данным номером («цель», «северо-запад», «посадочная площадка»), задаёт договорённость — таблица в программе.

- **Словарь (Dictionary):** набор допустимых рисунков, `DICT_<сетка>_<сколько номеров>`. С Пионером используют маркеры 6×6: учебные метки симулятора и карты — `DICT_6X6_250` (250 маркеров), часть старых примеров этой главы — `DICT_6X6_50`. Детектор находит только метки своего словаря: рисунок другого словаря виден, но не распознаётся.
- **AprilTag:** Другой популярный формат, поддерживаемый той же библиотекой (например, `DICT_APRILTAG_36h11`).

> **⚠ Словарь в симуляторе — точно DICT_6X6_250**
>
> В настоящем OpenCV первые 50 маркеров `DICT_6X6_250` совпадают с `DICT_6X6_50`, поэтому на дроне метки 0–49 находят оба словаря. Симулятор сравнивает словари строго: детектор с `DICT_6X6_50` или `DICT_4X4_50` не найдёт на сцене ничего. Пишите `DICT_6X6_250`.

Детекция шаг за шагом

Для работы с камерой и маркерами мы используем библиотеку **OpenCV** (`cv2`). Детектор создаётся один раз, а кадров обрабатывает сколько угодно:

Детектор

```
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(dictionary)
corners, ids, rejected = detector.detectMarkers(frame)
```

`detectMarkers` ищет на кадре квадраты, сверяет их узор со словарём и возвращает три значения:

- `ids` — номера найденных маркеров: массив строк, в каждой одно число, поэтому номер берут как `int(ids[i][0])`. **Если меток нет, `ids` равен `None`**, а не пустому списку. Это штатный исход: кадр есть, меток в нём нет. Его нужно отличать от отсутствия кадра (`frame is None`) из прошлого урока.
- `corners` — углы маркеров: `corners[i][0]` — четыре точки (x, y) маркера с номером `ids[i]`, по часовой стрелке начиная с верхнего левого угла самой метки. Это *пиксели кадра*, а не метры сцены.
- `rejected` — отвергнутые кандидаты (квадраты, не подошедшие к словарю); для работы обычно не нужны.

В кадр может попасть несколько меток, и порядок в списке не определён. Нужную метку выбирают *по значению ID*, а углы берут по тому же индексу:

Выбрать метку 12

```
if ids is not None:
    for i, value in enumerate(ids):     # i – номер, value – [ID]
        if int(value[0]) == 12:
            points = corners[i][0]
```

Функция `enumerate` выдаёт на каждом шаге цикла пару: номер элемента и сам элемент.

ЗАДАНИЕ 8.2 · Охота на маркеры

Взлетите на 3 м нижней камерой симулятора, создайте детектор `DICT_6X6_250` и для одного кадра напечатайте «Вижу маркер X!» для каждого найденного ID. Затем создайте второй детектор с `DICT_4X4_50`, обработайте тот же кадр и напечатайте его список. Отлетите на $(0, 5, 3)$ и убедитесь, что программа сообщает «меток нет», не пытаясь читать ID.

Что должно получиться: над стартом — строки `Вижу маркер 10!`, `Вижу маркер 11!`, `Вижу маркер 12!` (порядок может быть любым), у чужого словаря — пустой список; в точке $(0, 5, 3)$ — `меток нет` без ошибки.

В симуляторе: миссии Python: 82–86 в панели «**Обучение**».

8.3 Геометрия метки: углы, центр, ошибка

По четырём углам считают **центр** метки — среднее координат: $c_x = (x_1 + x_2 + x_3 + x_4)/4$, c_y — так же. Сравнивая центр метки с центром кадра, получают **ошибку** в пикселях:

$$e_x = c_x - \frac{w}{2}, \qquad e_y = c_y - \frac{h}{2}.$$

Знак показывает сторону: $e_x > 0$ — метка правее центра, $e_y > 0$ — *ниже* (ось y кадра направлена вниз). Величина — насколько далеко. Это ещё не команда дрону, а только измерение. Порядок углов, центр и знаки ошибки показаны на рисунке 19.

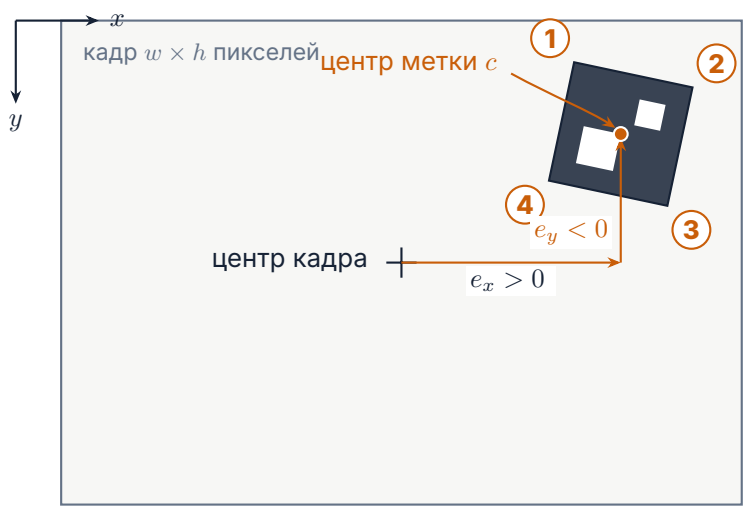


Рис. 19. Углы метки пронумерованы так, как их возвращает детектор: по часовой стрелке от левого верхнего. Центр метки c — среднее четырёх углов; ошибка — его смещение от центра кадра. Ось y кадра направлена вниз, поэтому «выше центра» даёт $e_y < 0$.

Одного кадра мало, чтобы поверить в находку: на шумном изображении детектор может ошибиться. Метку **подтверждают** несколькими кадрами подряд: счётчик растёт при каждом обнаружении и обнуляется при потере. Три обнаружения «с перерывом» — не подтверждение.

python/aruco_examples/11_01_find_marker.py

```python
from pioneer_sdk import Pioneer, Camera
import time
import cv2

TARGET_ID = 12          # нужная метка: номер рисунка, а не расстояние
CONFIRM = 3             # столько кадров подряд нужно, чтобы поверить в находку

drone = Pioneer()
camera = Camera(view="bottom")    # расширение симулятора; на дроне -
↪  Camera()

# Словарь детектора должен совпадать со словарём, которым напечатаны метки.
# Учебные метки симулятора - DICT_6X6_250.
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(dictionary)


def wait_state(state, limit):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


def go(x, y, z):
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    deadline = time.monotonic() + 30
```

```
    while not drone.point_reached() and time.monotonic() < deadline:
        time.sleep(0.05)
    time.sleep(1.5)


def read_target(target_id):
    """Кадр и четыре угла нужной метки; (кадр, None), если метки нет."""
    frame = camera.get_cv_frame()
    if frame is None:
        return None, None
    corners, ids, _ = detector.detectMarkers(frame)
    if ids is None:                          # меток нет — штатный исход, не
    ↪  ошибка
        return frame, None
    for i, value in enumerate(ids):          # ищем по ID, а не по месту в
    ↪  списке
        if int(value[0]) == target_id:
            return frame, corners[i][0]
    return frame, None


try:
    drone.arm()
    wait_state("ARMED", 3)
    drone.takeoff()
    wait_state("MISSION", 15)
    go(0, 0, 3)

    count = 0
    for step in range(10):
        frame, points = read_target(TARGET_ID)
        if points is None:
            count = 0                        # потеря сбрасывает счёт подряд
            print("Метка", TARGET_ID, "не видна")
        else:
            count += 1
            h, w = frame.shape[:2]
            cx = sum(float(p[0]) for p in points) / 4   # центр — среднее
            ↪  углов
            cy = sum(float(p[1]) for p in points) / 4
            ex, ey = cx - w / 2, cy - h / 2             # ошибка в пикселях
            print(f"Центр ({cx:.1f}, {cy:.1f}), ошибка ({ex:.1f}, {ey:.1f}),
            ↪  подряд {count}")
            for k in range(4):                          # обводим метку по
            ↪  углам
                a, b = points[k], points[(k + 1) % 4]
                cv2.line(frame, (int(a[0]), int(a[1])), (int(b[0]),
                ↪  int(b[1])), (0, 255, 0), 2)
            cv2.circle(frame, (round(cx), round(cy)), 4, (0, 0, 255), -1)
            cv2.imshow("ArUco", frame)
            cv2.waitKey(1)
            if count >= CONFIRM:
                print("Подтверждено: метка", TARGET_ID)
                break
        time.sleep(0.15)

    drone.land()
```

```
    wait_state("DISARMED", 15)
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. Функция `read_target` объединяет всё из этого урока: читает кадр, проверяет `None`, запускает детектор, обрабатывает пустой результат и выбирает метку по ID. Основной цикл считает центр и ошибку, обводит метку линиями по углам (`(k + 1) % 4` после последнего угла возвращает к первому), рисует центр и ведёт счётчик подряд идущих обнаружений. Дрон в этой программе только висит: измерение отделено от движения.

ЗАДАНИЕ 8.3 · Ошибка относительно центра

Измените пример: печатайте ошибку только знаками — «метка правее/левее центра», «выше/ниже центра». Затем поставьте дрон в $(0.3, 0, 3)$ и в $(0.9, 0, 3)$ и сравните знак e_x. Подтверждение сделайте строже: пять кадров подряд, а на третьем шаге цикла отлетите на $(0, 5, 3)$ и вернитесь.

Что должно получиться: над стартом метка 12 правее и выше центра ($e_x > 0$, $e_y < 0$); из $(0.9, 0, 3)$ знак e_x меняется на минус; после отлёта счётчик обнуляется, и «Подтверждено» печатается только после пяти новых обнаружений подряд.

В симуляторе: миссии Python: 87–90 в панели «**Обучение**».

8.4 Расстояние до метки: solvePnP

Центр метки и ошибка в пикселях говорят, *в какую сторону* лететь, но не *насколько далеко* метка в метрах (рисунок 20). Метры получаются из размера метки и параметров камеры. Если известны сторона метки S, фокусное расстояние камеры f (в пикселях) и видимая сторона метки s (в пикселях), то по модели камеры-«обскуры» $s \approx f\,S/d$, откуда $d \approx f\,S/s$: метка вдвое дальше — вдвое меньше на кадре. Функция `cv2.solvePnP` решает ту же задачу строго для всех четырёх углов и находит не только расстояние, но и поворот.

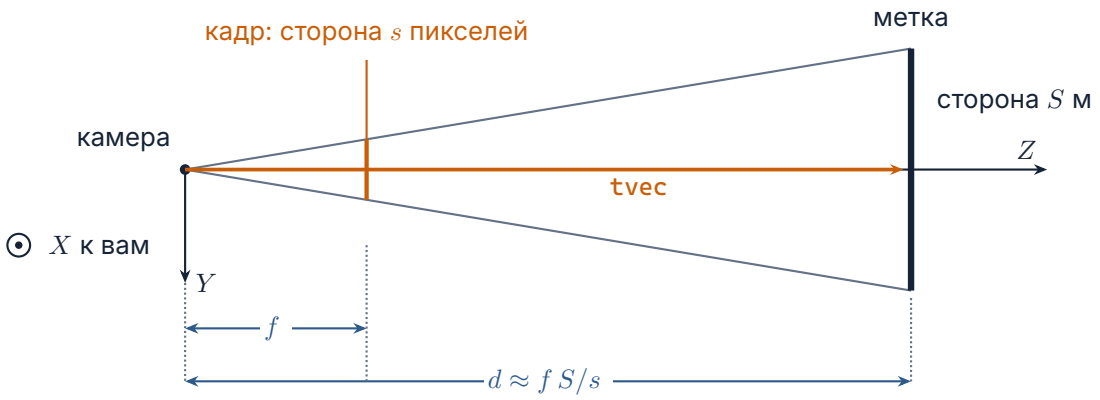


Рис. 20. Модель камеры-«обскуры», вид сбоку. Система камеры: Z вперёд, Y вниз, X вправо по кадру (к наблюдателю). Метка вдвое дальше — вдвое меньше на кадре. `tvec` — положение центра метки в этой системе; его длина — расстояние до метки.

Ей нужны три вещи: координаты четырёх углов метки в её собственной системе (центр — начало, порядок такой же, как у `corners`), те же четыре угла на кадре и параметры камеры. Параметры — это **матрица камеры** K (фокусные расстояния и центр кадра в пикселях) и коэффициенты искажений линзы; их получают калибровкой (глава 11) и хранят в файле `data.yml`. Результат — `tvec`: положение центра метки *в системе камеры* (X вправо по кадру, Y вниз, Z вперёд из объектива, метры) и `rvec`: поворот метки. Длина вектора `tvec` — расстояние до метки.

python/aruco_examples/11_02_marker_distance.py

```python
from pioneer_sdk import Pioneer, Camera
import time
import cv2
import numpy as np

TARGET_ID = 12
MARKER = 0.84          # сторона ЧЁРНОГО квадрата метки, м. Метка симулятора:
↪  лист 1.05 м, чёрный квадрат ~0.84 м.
                       # На настоящем дроне измерьте линейкой рисунок без
                        ↪  белого поля.
half = MARKER / 2

# Углы метки в её системе координат: центр - начало, X вправо, Y вверх, Z от
↪  метки к наблюдателю.
# Порядок - как у углов из детектора: левый верхний, правый верхний, правый
↪  нижний, левый нижний.
OBJECT_POINTS = np.array([(-half, half, 0), (half, half, 0), (half, -half,
↪  0), (-half, -half, 0)], dtype=np.float32)

# Параметры камеры: матрица K и коэффициенты искажений. В симуляторе
↪  FileStorage отдаёт идеальную
# нижнюю камеру (кадр 640x360, угол обзора по вертикали 72 градуса, искажений
↪  нет).
# На настоящем дроне здесь лежит файл data.yml, полученный калибровкой.
storage = cv2.FileStorage("data.yml", cv2.FILE_STORAGE_READ)
K = storage.getNode("mtx").mat()
dist = storage.getNode("dist").mat()
storage.release()
```

```
drone = Pioneer()
camera = Camera(view="bottom")    # расширение симулятора; на дроне –
↪  Camera()
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(dictionary)


def wait_state(state, limit):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


def go(x, y, z):
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    deadline = time.monotonic() + 30
    while not drone.point_reached() and time.monotonic() < deadline:
        time.sleep(0.05)
    time.sleep(1.5)                # корпус успокаивается, кадр снимается
    ↪  без смаза


def marker_pose(target_id):
    """Вектор tvec метки в системе камеры (метры) или None: нет кадра, нет
    ↪  метки, поза не найдена."""
    frame = camera.get_cv_frame()
    if frame is None:
        return None
    corners, ids, _ = detector.detectMarkers(frame)
    if ids is None:
        return None
    for i, value in enumerate(ids):
        if int(value[0]) == target_id:
            ok, rvec, tvec = cv2.solvePnP(OBJECT_POINTS, corners[i][0], K,
            ↪  dist,
                                          flags=cv2.SOLVEPNP_IPPE_SQUARE)
            return tvec.ravel() if ok else None
    return None


try:
    drone.arm()
    wait_state("ARMED", 3)
    drone.takeoff()
    wait_state("MISSION", 15)

    for z in (2.0, 3.0, 4.0):
        go(0.6, 0.3, z)            # над меткой 12 учебной сцены
        tvec = marker_pose(TARGET_ID)
        if tvec is None:
            print(f"z={z}: метка {TARGET_ID} не видна")
            continue
        distance = float(np.linalg.norm(tvec))
```

```
            print(f"z={z}: tvec=({tvec[0]:.2f}, {tvec[1]:.2f}, {tvec[2]:.2f}) м,
            ↪  до метки {distance:.2f} м")

    drone.land()
    wait_state("DISARMED", 15)
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. Матрица камеры читается из `data.yml`: в симуляторе `cv2.FileStorage` отдаёт параметры идеальной нижней камеры (кадр 640×360, угол обзора по вертикали $72°$, значит $f = 180/\tan 36° \approx 248$ пикселей, искажений нет), на настоящем дроне — результат вашей калибровки. Функция `marker_pose` повторяет уже знакомую цепочку «кадр → проверка `None` → детектор → метка по ID» и в конце вызывает `solvePnP` с флагом `SOLVEPNP_IPPE_SQUARE`, рассчитанным именно на квадратную метку. Дрон поднимается над меткой на 2, 3 и 4 м и печатает `tvec` и расстояние: камера смотрит вниз, поэтому `tvec[2]` близко к высоте полёта, а `tvec[0]` и `tvec[1]` около нуля.

Три источника ошибки

- **Размер метки.** В коде стоит сторона *чёрного квадрата*, а не листа с белым полем. Ошибка в размере даёт пропорциональную ошибку расстояния: вместо 15 см записали 10 см — расстояние занижено в 1,5 раза.
- **Порядок углов.** Объектные точки должны идти в том же порядке, что `corners`: левый верхний, правый верхний, правый нижний, левый нижний. При другом порядке расстояние сохранится, но поворот метки окажется неверным.
- **Камера.** Матрица из `data.yml` относится к конкретной камере и разрешению. Для передней камеры симулятора (угол обзора $80°$) готовая матрица не подходит: $f = 180/\tan 40° \approx 214$ пикселей; на настоящем дроне нужна своя калибровка.

ЗАДАНИЕ 8.4 · Высота по метке

Симулятор. Запустите `11_02_marker_distance.py` в сцене миссии Python № 82–90 (метка 12 лежит около $(0.6, 0.3)$). Сравните напечатанное расстояние с высотой по `get_local_position_lps()`. Затем замените `MARKER = 0.84` на `0.42` и запустите ещё раз.

Что должно получиться: расстояние до метки близко к высоте полёта (расхождение — в пределах десятков сантиметров: камера установлена на корпусе, а не в точке, где измеряется z); с вдвое меньшим размером метки расстояние, которое печатает программа, вдвое меньше. Объясните, почему.

В симуляторе: миссии Python: 78, 88 в панели «**Обучение**».

8.5 ArUco на настоящем дроне

Дальше — программы для компьютера, подключённого к Пионеру. В них два отличия от учебных: камера одна, `Camera()` без `view`, а распознавание идёт по настоящим пикселям, поэтому маркер печатают на бумаге или показывают на экране телефона.

python/aruco_examples/11_aruco_basics_example1_detect.py

```
import cv2
import cv2.aruco as aruco
import numpy as np
from pioneer_sdk import Camera
import threading
import time

class ArucoDetectorThread:
    def __init__(self):
        self.camera = Camera()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        # 1. Настройка словаря ArUco (6x6, 50 маркеров - стандарт для
        ↪  Pioneer)
        self.aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        # Обеспечение совместимости разных версий OpenCV
        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):
            # Новый API (OpenCV 4.7+)
            detector = aruco.ArucoDetector(self.aruco_dict, self.parameters)
            return detector.detectMarkers(image)
        else:
            # Старый API
            return aruco.detectMarkers(image, self.aruco_dict,
            ↪  parameters=self.parameters)

    def capture_thread(self):
        while self.running:
            frame = self.camera.get_cv_frame()
            if frame is not None:
                with self.frame_lock:
                    self.latest_frame = frame.copy()
            time.sleep(0.01)

    def processing_thread(self):
        while self.running:
            frame_to_process = None
            with self.frame_lock:
                if self.latest_frame is not None:
                    frame_to_process = self.latest_frame.copy()
```

```
            if frame_to_process is None:
                time.sleep(0.01)
                continue

            # Переводим в оттенки серого (для ускорения)
            gray = cv2.cvtColor(frame_to_process, cv2.COLOR_BGR2GRAY)

            # 3. Детекция маркеров
            corners, ids, rejected = self.detect_markers_compat(gray)

            # Если маркеры найдены
            if ids is not None:
                # Рисуем рамки вокруг маркеров
                aruco.drawDetectedMarkers(frame_to_process, corners, ids)
                # Выводим ID найденных маркеров в консоль (можно
                ↪  закомментировать, если спамит)
                # print(f"Вижу маркеры: {ids.flatten()}")

            with self.frame_lock:
                self.processed_frame = frame_to_process

    def run(self):
        print("Запуск детекции маркеров... Нажмите ESC для выхода.")

        t_capture = threading.Thread(target=self.capture_thread)
        t_process = threading.Thread(target=self.processing_thread)

        t_capture.daemon = True
        t_process.daemon = True

        t_capture.start()
        t_process.start()

        try:
            while True:
                display_frame = None
                with self.frame_lock:
                    if self.processed_frame is not None:
                        display_frame = self.processed_frame.copy()
                    elif self.latest_frame is not None:
                        display_frame = self.latest_frame.copy()

                if display_frame is not None:
                    cv2.imshow('Pioneer Camera', display_frame)

                if cv2.waitKey(1) == 27:
                    self.running = False
                    break

                time.sleep(0.03)

        finally:
            self.running = False
            t_capture.join(timeout=1.0)
            t_process.join(timeout=1.0)
            cv2.destroyAllWindows()
```

```
if __name__ == "__main__":
    detector = ArucoDetectorThread()
    detector.run()
```

Разбор кода

1. `camera.get_cv_frame()` получает изображение с камеры дрона.
2. `cv2.cvtColor` переводит картинку в оттенки серого (так алгоритм работает быстрее и надёжнее).
3. `detect_markers_compat` вызывает детектор в форме, которую поддерживает установленная версия OpenCV: `ArucoDetector` (OpenCV 4.7 и новее) или старую функцию `aruco.detectMarkers`.
4. Результат — те же `corners` и `ids`, что в учебном примере; `drawDetectedMarkers` обводит найденные метки.

Сама программа оформлена как **класс** (`class ArucoDetectorThread`): данные хранятся в полях `self.…`, а работа разложена по методам. Методы `capture_thread` и `processing_thread` запускаются в отдельных **потоках** (`threading.Thread`) и работают одновременно с показом кадра; общий кадр защищён замком `frame_lock`, чтобы потоки не читали и не писали его в один момент. Потоки подробнее — в разделе про групповой полёт этой главы. Пример использует словарь `DICT_6X6_50`: печатайте маркеры с номерами 0–49.

Определение координат маркера в потоках

Тот же расчёт, оформленный как программа с потоками: один поток читает кадры, другой ищет метку и считает `solvePnP`, третий показывает результат с осями метки (`cv2.drawFrameAxes`). Матрица камеры читается из `data.yml`, размер метки — 5 см (измерьте и поправьте для своей).

Определение координат

```
import cv2
import numpy as np
from pioneer_sdk import Camera
import cv2.aruco as aruco
import threading
import time

def load_coefficients(path):
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()
    cv_file.release()
    return camera_matrix, dist_coeffs

class PoseEstimatorThread:
    def __init__(self, camera_matrix, dist_coeffs):
```

```
        self.camera = Camera()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        self.camera_matrix = camera_matrix
        self.dist_coeffs = dist_coeffs

        # Dictionary of aruco-markers
        self.aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        # Parameters for marker detection
        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

        self.size_of_marker = 0.05  # side length in meters

        # Углы маркера в его системе координат (центр - начало, Z - от
        ↪  маркера к камере) в том же
        # порядке, в каком их возвращает детектор: левый верхний, правый
        ↪  верхний, правый нижний, левый нижний.
        self.points_of_marker = np.array([
            (-self.size_of_marker / 2, self.size_of_marker / 2, 0),
            (self.size_of_marker / 2, self.size_of_marker / 2, 0),
            (self.size_of_marker / 2, -self.size_of_marker / 2, 0),
            (-self.size_of_marker / 2, -self.size_of_marker / 2, 0),
        ], dtype=np.float32)

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):
            # Новый API (OpenCV 4.7+)
            detector = aruco.ArucoDetector(self.aruco_dict, self.parameters)
            return detector.detectMarkers(image)
        else:
            # Старый API
            return aruco.detectMarkers(image, self.aruco_dict,
            ↪  parameters=self.parameters)

    def capture_thread(self):
        while self.running:
            frame = self.camera.get_cv_frame()
            if frame is not None:
                with self.frame_lock:
                    self.latest_frame = frame.copy()
            time.sleep(0.01)

    def processing_thread(self):
        while self.running:
            frame_to_process = None
            with self.frame_lock:
                if self.latest_frame is not None:
                    frame_to_process = self.latest_frame.copy()

            if frame_to_process is None:
```

```
                time.sleep(0.01)
                continue

            # Detect markers
            corners, ids, rejected =
            ↪  self.detect_markers_compat(frame_to_process)

            if ids is not None and len(ids) > 0:
                # Calculate pose for first detected marker
                success, rvecs, tvecs = cv2.solvePnP(
                    self.points_of_marker, corners[0], self.camera_matrix,
                    ↪  self.dist_coeffs
                )

                if success:
                    # Draw axes
                    cv2.drawFrameAxes(frame_to_process, self.camera_matrix,
                    ↪  self.dist_coeffs, rvecs, tvecs, 0.1)
                    # print(f"Marker Pose: {tvecs.T}") # Optional logging

            with self.frame_lock:
                self.processed_frame = frame_to_process

    def run(self):
        print("Запуск оценки позы... Нажмите ESC для выхода.")

        t_capture = threading.Thread(target=self.capture_thread)
        t_process = threading.Thread(target=self.processing_thread)

        t_capture.daemon = True
        t_process.daemon = True

        t_capture.start()
        t_process.start()

        try:
            while True:
                display_frame = None
                with self.frame_lock:
                    if self.processed_frame is not None:
                        display_frame = self.processed_frame.copy()
                    elif self.latest_frame is not None:
                        display_frame = self.latest_frame.copy()

                if display_frame is not None:
                    cv2.imshow("Pose Estimation", display_frame)

                if cv2.waitKey(1) == 27:
                    self.running = False
                    break

                time.sleep(0.03)

        finally:
            self.running = False
            t_capture.join(timeout=1.0)
            t_process.join(timeout=1.0)
```

```
            cv2.destroyAllWindows()

if __name__ == "__main__":
    # Ensure data.yml exists or provide correct path
    try:
        camera_matrix, dist_coeffs = load_coefficients("data.yml")
        estimator = PoseEstimatorThread(camera_matrix, dist_coeffs)
        estimator.run()
    except Exception as e:
        print(f"Ошибка загрузки калибровки или запуска: {e}")
        print("Убедитесь, что файл 'data.yml' с параметрами калибровки
        ↪  существует.")
```

Посадка по маркеру (только на дроне)

В этом примере дрон взлетает, занимает точку $(0, 0, 1)$ над стартом и висит, пока камера ищет маркеры. Как только в кадре появляется маркер с ID 42, дрон садится. Логика полёта работает в третьем потоке и только читает список `detected_ids`, который заполняет поток обработки. Это посадка «увидел — сел» без наведения: дрон сядет там, где висел. Как сначала навестись на метку, а потом сесть точно на неё, — в следующем разделе.

Полет по маркеру

```
import cv2
import cv2.aruco as aruco
from pioneer_sdk import Pioneer, Camera
import threading
import time

class DroneFlightThread:
    def __init__(self):
        self.camera = Camera()
        self.drone = Pioneer()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()
        self.detected_ids = []

        # ArUco setup
        self.aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):
            detector = aruco.ArucoDetector(self.aruco_dict, self.parameters)
            return detector.detectMarkers(image)
        else:
```

```
            return aruco.detectMarkers(image, self.aruco_dict,
            ↪  parameters=self.parameters)

    def capture_thread(self):
        while self.running:
            frame = self.camera.get_cv_frame()
            if frame is not None:
                with self.frame_lock:
                    self.latest_frame = frame.copy()
            time.sleep(0.01)

    def processing_thread(self):
        while self.running:
            frame_to_process = None
            with self.frame_lock:
                if self.latest_frame is not None:
                    frame_to_process = self.latest_frame.copy()

            if frame_to_process is None:
                time.sleep(0.01)
                continue

            gray = cv2.cvtColor(frame_to_process, cv2.COLOR_BGR2GRAY)
            corners, ids, rejected = self.detect_markers_compat(gray)

            current_ids = []
            if ids is not None:
                aruco.drawDetectedMarkers(frame_to_process, corners, ids)
                current_ids = ids.flatten().tolist()

            # Update shared state
            self.detected_ids = current_ids

            with self.frame_lock:
                self.processed_frame = frame_to_process

    def flight_logic(self):
        print("Арминг и взлет...")
        self.drone.arm()
        time.sleep(1) # Даем автопилоту перейти в режим готовности
        self.drone.takeoff()
        time.sleep(2)

        print("Летим в точку наблюдения...")
        self.drone.go_to_local_point(x=0, y=0, z=1, yaw=0)

        landing_marker_id = 42

        while self.running:
            if landing_marker_id in self.detected_ids:
                print(f"Посадочный маркер ({landing_marker_id}) найден!
                ↪  Сажусь...")
                self.drone.land()
                self.running = False # Stop other threads
                break

            time.sleep(0.1)
```

```
    def run(self):
        print("Запуск полетной миссии... Нажмите ESC для выхода.")

        t_capture = threading.Thread(target=self.capture_thread)
        t_process = threading.Thread(target=self.processing_thread)
        t_flight = threading.Thread(target=self.flight_logic)

        t_capture.daemon = True
        t_process.daemon = True
        t_flight.daemon = True # Flight logic runs in background

        t_capture.start()
        t_process.start()
        t_flight.start()

        try:
            while self.running:
                display_frame = None
                with self.frame_lock:
                    if self.processed_frame is not None:
                        display_frame = self.processed_frame.copy()
                    elif self.latest_frame is not None:
                        display_frame = self.latest_frame.copy()

                if display_frame is not None:
                    cv2.imshow('Pioneer Camera', display_frame)

                if cv2.waitKey(1) == 27:
                    print("Принудительная посадка...")
                    self.drone.land()
                    self.running = False
                    break

                time.sleep(0.03)

        finally:
            self.running = False
            # Wait a bit for threads to notice
            time.sleep(0.5)
            cv2.destroyAllWindows()
            self.drone.close_connection()

if __name__ == "__main__":
    mission = DroneFlightThread()
    mission.run()
```

ЗАДАНИЕ 8.5 · Маркер на бумаге

Только на дроне. Распечатайте три маркера словаря `DICT_6X6_50` с разными номерами (или откройте их на экране телефона). Измените первый пример так, чтобы он печатал «Вижу маркер X!» один раз при появлении метки в кадре, а не на каждом кадре, и «Потерял маркер X» при её исчезновении.

Что должно получиться: при показе маркера в терминале одна строка о появлении, при уходе из кадра — одна строка о потере; маркер из другого словаря программа не замечает.

8.6 Наведение по метке

Ошибка из прошлого урока говорит, насколько метка смещена от центра кадра. **Обратная связь** превращает её в движение: измерили — немного сдвинулись — измерили снова. Собирают такой регулятор по шагам, проверяя каждый.

1. **Измерить без движения.** Несколько кадров подряд печатаем ошибку, дрон висит. Даже у неподвижного дрона числа немного «дрожат» — это шум измерения.
2. **Знак поправки.** Ошибку кадра нужно связать с осью сцены. У нижней камеры симулятора при курсе 0 право кадра — это $+X$ сцены, верх кадра — $+Y$. Метка правее центра ($e_x > 0$) лежит по $+X$ от дрона, поэтому $v_x > 0$. Ось y кадра направлена вниз: метка ниже центра ($e_y > 0$) лежит по $-Y$, поэтому у v_y знак противоположный. Знак проверяют коротким ограниченным движением: ошибка после него должна уменьшиться.
3. **Короткая команда.** Двигаются импульсом: несколько повторов `set_manual_speed` по 0.1 с, затем явная нулевая скорость и новый кадр. Долгое движение по старой ошибке уводит цель из кадра.
4. **Коэффициент и предел.** Скорость пропорциональна ошибке: $v_x = k \cdot e_x / 640$, где деление на ширину кадра делает ошибку безразмерной, а k переводит её в м/с. Чем ближе метка к центру, тем медленнее дрон — это P-регулятор. Скорость ограничивают: `max(-0.3, min(0.3, v))`.
5. **Допуск.** Внутри мёртвой зоны в несколько пикселей вокруг центра команда нулевая: иначе дрон бесконечно исправляет шум. «По центру» засчитывается, когда метка в допуске несколько кадров подряд.
6. **Потеря метки.** Если метки нет, старая ошибка больше ничего не значит: скорость обнуляют сразу, а ожидание возвращения метки ограничивают числом кадров.
7. **Снижение и посадка.** Центрирование и снижение — отдельные этапы: снижаются только после подтверждённого центра, на тех же X и Y, затем снова центрируются и садятся.

python/aruco_examples/12_01_land_on_marker.py

```python
from pioneer_sdk import Pioneer, Camera
import time
import cv2

TARGET_ID = 12
K = 2.0              # коэффициент: нормированная ошибка -> скорость, м/с
LIMIT = 0.3          # предел скорости, м/с
DEAD_ZONE = 8        # допуск центрирования, пиксели
STABLE = 3           # столько кадров подряд в допуске = «по центру»
```

```
drone = Pioneer()
camera = Camera(view="bottom")    # расширение симулятора; на дроне –
↪  Camera()
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(dictionary)


def wait_state(state, limit):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if drone.get_autopilot_state() == state:
            return True
        time.sleep(0.05)
    return False


def go(x, y, z):
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0)
    deadline = time.monotonic() + 30
    while not drone.point_reached() and time.monotonic() < deadline:
        time.sleep(0.05)
    time.sleep(1.5)


def observation(target_id):
    """Ошибка (ex, ey) центра метки от центра кадра в пикселях или None."""
    frame = camera.get_cv_frame()
    if frame is None:
        return None
    corners, ids, _ = detector.detectMarkers(frame)
    if ids is None:
        return None
    for i, value in enumerate(ids):
        if int(value[0]) == target_id:
            points = corners[i][0]
            h, w = frame.shape[:2]
            cx = sum(float(p[0]) for p in points) / 4
            cy = sum(float(p[1]) for p in points) / 4
            return cx - w / 2, cy - h / 2
    return None


def clamp(value, limit):
    return max(-limit, min(limit, value))


def align(target_id, max_frames=120):
    """Навести дрон на метку. True – метка в центре STABLE кадров подряд."""
    stable = 0
    lost = 0
    for _ in range(max_frames):
        error = observation(target_id)
        if error is None:
            drone.set_manual_speed(vx=0, vy=0, vz=0, yaw_rate=0)   # старую
            ↪  ошибку не используем
            stable = 0
            lost += 1
```

```
            if lost >= 10:                                          # поиск не
            ↪ бесконечен
                print("Метка потеряна")
                return False
        else:
            lost = 0
            ex, ey = error
            if abs(ex) <= DEAD_ZONE and abs(ey) <= DEAD_ZONE:
                drone.set_manual_speed(vx=0, vy=0, vz=0, yaw_rate=0)
                stable += 1
                if stable >= STABLE:
                    return True
            else:
                stable = 0
                # Нижняя камера, курс 0: вправо по кадру = +X сцены, вверх по
                ↪ кадру = +Y.
                # Пиксельная ось y растёт вниз, поэтому у vy знак минус.
                vx = clamp(K * ex / 640, LIMIT)
                vy = clamp(-K * ey / 360, LIMIT)
                drone.set_manual_speed(vx=vx, vy=vy, vz=0, yaw_rate=0)
        time.sleep(0.1)
    drone.set_manual_speed(vx=0, vy=0, vz=0, yaw_rate=0)
    return False


try:
    drone.arm()
    wait_state("ARMED", 3)
    drone.takeoff()
    wait_state("MISSION", 15)
    go(0, 0, 3)

    if not align(TARGET_ID):
        print("Цель не наведена: сажусь на месте")
    else:
        print("По центру на 3 м, снижаюсь")
        position = drone.get_local_position_lps()
        if position is not None:
            go(position[0], position[1], 1.5)       # снижение - отдельный
            ↪ этап
        if align(TARGET_ID):
            print("По центру на 1.5 м, посадка на метку")
        else:
            print("Центр на 1.5 м не подтверждён: сажусь на месте")
    drone.land()
    wait_state("DISARMED", 20)
    position = drone.get_local_position_lps()
    if position is not None:
        print(f"Сел в точке ({position[0]:.2f}, {position[1]:.2f})")
finally:
    drone.disarm()
    drone.close_connection()
```

Что происходит. `observation` возвращает ошибку метки или `None`. `align` — регулятор по двум осям: при потере метки он останавливает дрон и через 10 кадров без метки сдаётся, в допуске — держит нулевую скорость и считает кадры подряд, иначе — зада-

ёт ограниченную скорость, пропорциональную ошибке. Основная программа наводится на высоте 3 м, снижается до 1.5 м над измеренными X и Y, наводится ещё раз и только после этого сажает дрон. Если какой-то этап не удался, программа говорит об этом и сажает дрон на месте — посадку «по цели» при этом успешной не объявляет.

> **Знаки и курс**
>
> Связь «право кадра $= +X$, верх $= +Y$» верна для нижней камеры при курсе 0. После поворота корпуса оси кадра поворачиваются вместе с ним, и знаки поправок нужно пересчитать (или задавать скорость командой `set_manual_speed_body_fixed` в осях дрона). У камеры настоящего дрона своё направление, поэтому знак каждой оси проверяют коротким опытом заново.

ЗАДАНИЕ 8.6 · Знак поправки

Возьмите из примера `observation`. Над стартом на высоте 3 м измерьте ошибку метки 12, затем одну секунду двигайтесь по оси X со скоростью 0.2 м/с (повторы по 0.1 с), явно остановитесь и измерьте снова. Знак скорости выберите так, чтобы ошибка уменьшилась, и проверьте обратный знак отдельным запуском.

Что должно получиться: при $v_x = +0.2$ значение e_x уменьшается (метка 12 стоит правее центра), при $v_x = -0.2$ — растёт; после нулевой команды дрон висит.

В симуляторе: миссии Python: 91–93 в панели «**Обучение**».

ЗАДАНИЕ 8.7 · Посадка по ArUco

Запустите пример, затем поменяйте его параметры по одному: коэффициент `K` 2 → 6, допуск `DEAD_ZONE` 8 → 0, знак у `vy`. Для каждого варианта запишите, что происходит. Верните исходные значения и проверьте ветку потери: укажите `TARGET_ID = 249`, которого на карте нет.

Что должно получиться: с исходными параметрами дрон центрируется над меткой 12, снижается, садится на неё и печатает точку посадки около $(0.6, 0.3)$; с большим `K` он проскакивает центр, без допуска дольше «дрожит» около него, с неверным знаком метка уходит из кадра; с ID 249 программа печатает «Метка потеряна» и сажает дрон на месте.

В симуляторе: миссии Python: 94–99 в панели «**Обучение**».

8.7 Следование за меткой (только на дроне)

Посадка выше использует ошибку в пикселях. Если метка стоит впереди, а не внизу, а нужно держать её в центре кадра и на заданном расстоянии, добавляют расстояние из `tvec` (раздел 8.4). Сначала — следование одним поворотом, без расстояния:

1. Найти центр метки на кадре (среднее арифметическое углов).
2. Найти центр кадра: половины ширины и высоты из `frame.shape`.
3. Если $X_{marker} > X_{center}$, метка справа → поворачиваем вправо; если меньше — влево.

4. Скорость поворота пропорциональна ошибке e_x и ограничена, как **align** из прошлого урока.

python/aruco_examples/12_follow_aruco.py

```
import cv2
import cv2.aruco as aruco
import numpy as np
from pioneer_sdk import Pioneer, Camera
import threading
import time
from collections import deque

def load_coefficients(path):
    try:
        cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
        camera_matrix = cv_file.getNode("mtx").mat()
        dist_coeffs = cv_file.getNode("dist").mat()
        cv_file.release()
        return camera_matrix, dist_coeffs
    except Exception:
        print("Warning: Calibration file not found, using dummy values.")
        return np.eye(3), np.zeros((5, 1))

class FollowArucoThread:
    def __init__(self, camera_matrix, dist_coeffs):
        self.camera = Camera()
        self.drone = Pioneer()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        self.camera_matrix = camera_matrix
        self.dist_coeffs = dist_coeffs

        # Flight control variables
        self.latest_coordinates = None # [x, y, z] relative to camera
        self.x_center = None

        # ArUco setup
        self.aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)

        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

        self.size_of_marker = 0.1 # m
        # Углы маркера в его системе координат (центр - начало, Z - от
        ↪  маркера к камере) в том же
        # порядке, в каком их возвращает детектор: левый верхний, правый
        ↪  верхний, правый нижний, левый нижний.
        self.points_of_marker = np.array([
            (-self.size_of_marker / 2, self.size_of_marker / 2, 0),
            (self.size_of_marker / 2, self.size_of_marker / 2, 0),
            (self.size_of_marker / 2, -self.size_of_marker / 2, 0),
```

```
            (-self.size_of_marker / 2, -self.size_of_marker / 2, 0),
        ], dtype=np.float32)

        # FPS calculation
        self.frame_times = deque(maxlen=30)
        self.last_fps_log_time = time.time()

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):
            detector = aruco.ArucoDetector(self.aruco_dict, self.parameters)
            return detector.detectMarkers(image)
        else:
            return aruco.detectMarkers(image, self.aruco_dict,
            ↪  parameters=self.parameters)

    def capture_thread(self):
        while self.running:
            frame = self.camera.get_cv_frame()
            if frame is not None:
                with self.frame_lock:
                    self.latest_frame = frame.copy()
            time.sleep(0.01)

    def processing_thread(self):
        while self.running:
            frame_to_process = None
            with self.frame_lock:
                if self.latest_frame is not None:
                    frame_to_process = self.latest_frame.copy()

            if frame_to_process is None:
                time.sleep(0.01)
                continue

            try:
                corners, ids, _ =
                ↪  self.detect_markers_compat(frame_to_process)

                if ids is not None and len(corners) > 0:
                    # Calculate center for visualization
                    x_c = int(sum([p[0] for p in corners[0][0]]) / 4)
                    y_c = int(sum([p[1] for p in corners[0][0]]) / 4)

                    # Draw visual feedback
                    dot_size = 5
                    frame_to_process[y_c - dot_size:y_c + dot_size, x_c -
                    ↪  dot_size:x_c + dot_size] = [0, 0, 255]
                    aruco.drawDetectedMarkers(frame_to_process, corners)

                    # Solve PnP
                    success, rvecs, tvecs = cv2.solvePnP(
                        self.points_of_marker, corners[0],
                        ↪  self.camera_matrix, self.dist_coeffs
                    )

                    if success:
```

```
                    self.latest_coordinates = [tvecs.item(0),
                    ↪  tvecs.item(1), tvecs.item(2)]
                    self.x_center = x_c
                else:
                    self.latest_coordinates = None
                    self.x_center = None
            else:
                self.latest_coordinates = None
                self.x_center = None

            # FPS Logging
            now = time.time()
            self.frame_times.append(now)
            if now - self.last_fps_log_time > 2.0 and
            ↪  len(self.frame_times) > 1:
                fps = len(self.frame_times) / (self.frame_times[-1] -
                ↪  self.frame_times[0])
                # print(f"[Video Thread] FPS: {fps:.2f}")
                self.last_fps_log_time = now

            with self.frame_lock:
                self.processed_frame = frame_to_process

        except cv2.error as e:
            print(f"OpenCV error: {e}")

def control_loop(self):
    """Main control loop for following the marker"""
    # Note: This logic assumes we want to control the drone based on
    ↪  vision.
    # Implementation depends on specific requirements (e.g., keep marker
    ↪  in center).

    # Simple P-controller constants
    k_p_yaw = 0.1
    k_p_fwd = 0.2
    target_dist = 1.0

    while self.running:
        if self.latest_coordinates is not None:
            x, y, z = self.latest_coordinates
            dist = z # Z in camera frame is distance forward

            # Simple logic:
            # 1. Rotate to keep x near 0 (center of image horizontally)
            # 2. Move forward/back to keep z near target_dist

            yaw_cmd = 0 # Calculate based on x
            vx_cmd = 0  # Calculate based on z

            # Placeholder for actual control logic
            # self.drone.set_manual_speed(vx=vx_cmd, vy=0, vz=0,
            ↪  yaw_rate=yaw_cmd)
            pass

        time.sleep(0.05)
```

```
    def run(self):
        print("Запуск следования за маркером... Нажмите ESC для выхода.")

        t_capture = threading.Thread(target=self.capture_thread)
        t_process = threading.Thread(target=self.processing_thread)
        t_control = threading.Thread(target=self.control_loop)

        t_capture.daemon = True
        t_process.daemon = True
        t_control.daemon = True

        t_capture.start()
        t_process.start()
        t_control.start()

        try:
            while self.running:
                display_frame = None
                with self.frame_lock:
                    if self.processed_frame is not None:
                        display_frame = self.processed_frame.copy()
                    elif self.latest_frame is not None:
                        display_frame = self.latest_frame.copy()

                if display_frame is not None:
                    cv2.imshow('Follow ArUco', display_frame)

                if cv2.waitKey(1) == 27:
                    self.running = False
                    break

                time.sleep(0.03)

        finally:
            self.running = False
            time.sleep(0.5)
            cv2.destroyAllWindows()
            self.drone.close_connection()

if __name__ == "__main__":
    camera_matrix, dist_coeffs = load_coefficients("data.yml")
    follower = FollowArucoThread(camera_matrix, dist_coeffs)
    follower.run()
```

Этот пример — заготовка следования для настоящего дрона. Поток обработки ищет маркер (словарь `DICT_4X4_50`: маркер печатают из этого словаря; симулятор такие метки не найдёт, его метки нарисованы словарём `DICT_6X6_250`), рисует точку в его центре и через `cv2.solvePnP` вычисляет смещение маркера относительно камеры. Метод `control_loop` пока ничего не отправляет дрону: дрон не взлетает, а команды движения закомментированы. Дописать регулятор по пунктам списка выше — ваша задача; устроен он так же, как `align` из прошлого урока, только ошибка e_x управляет скоростью поворота `yaw_rate`.

ЗАДАНИЕ 8.8 · Посадка на базу

Только на дроне. Дрон висит на высоте 1 м. Если он видит маркер ID 1, он 1 с летит вперёд со скоростью 0.2 м/с; если ID 2 — назад; если ID 0 — садится. Каждое решение принимается по метке, подтверждённой тремя кадрами подряд; без меток скорость нулевая.

Что должно получиться: показ маркера 1 сдвигает дрон вперёд примерно на 20 см, маркера 2 — назад, маркер 0 сажает его; короткое мелькание маркера в кадре дрон не двигает.

8.8 Продвинутая работа с камерой (только на дроне)

Мы уже использовали камеру для поиска маркеров. Теперь разберемся, как работать с видеопотоком более гибко. Всё в этом разделе требует компьютера, подключённого к настоящему Пионеру.

Класс VideoStream

Для получения «сырого» видеопотока (без автоматического декодирования в OpenCV формат, что иногда нужно для оптимизации) используется класс `VideoStream`.

python/camera_examples/camera_stream.py

```python
from pioneer_sdk import VideoStream

if __name__ == "__main__":
    stream = VideoStream()      # Создаем объект потока видео

    while True:                 # Бесконечный цикл, ожидающий команд
        cmd = input()           # Ожидаем ввода команды от пользователя в
        ↪ консоль

        if cmd == "start":      # Если введена команда 'start'
            stream.start()      # Запускаем видеопоток

        elif cmd == "stop":     # Если введена команда 'stop'
            stream.stop()       # Останавливаем видеопоток
```

Этот код запускает передачу видео по Wi-Fi. Вы можете открыть этот поток в плеере VLC (адрес обычно `rtsp://192.168.4.1:8554/front`). В симуляторе `VideoStream` — только обёртка над той же камерой: поток наружу браузер не отдаёт, открыть его в VLC не получится.

Захват кадров и фотографирование

Чтобы сделать фото, нужно получить текущий кадр и сохранить его в файл с помощью `cv2.imwrite(имя_файла, кадр)`. В симуляторе `imwrite` возвращает `True`, но файла не создаёт: файловая система Python в браузере живёт в памяти вкладки.

Калибровка камеры

Для точных измерений (например, расстояния до маркера через `solvePnP`, раздел 8.4) нужно знать оптические параметры камеры: матрицу камеры и коэффициенты дисторсии. Камера симулятора идеальная, её матрицу программе отдаёт `cv2.FileStorage`; калибровка нужна только настоящей камере, и подробно она разобрана в главе 11.

Процесс калибровки:

1. Распечатать шахматную доску (Chessboard).
2. Сделать серию снимков доски под разными углами.
3. Запустить скрипт калибровки (`camera_examples/camera_calibration.py`; его разбор — в главе 11). Он вычислит параметры и сохранит их в файл `data.yml`: матрица камеры — узел `mtx`, коэффициенты дисторсии — узел `dist`. Этот же файл читают примеры с оценкой позы из предыдущих разделов.

> **Зачем это нужно?**
>
> Без калибровки прямые линии на изображении могут казаться изогнутыми (эффект «рыбьего глаза»), что приведет к ошибкам в навигации.

ЗАДАНИЕ 8.9 · Фотограф

Только на дроне. Напишите программу, которая показывает видео с дрона в окне `cv2.imshow`. При нажатии клавиши `S` текущий кадр сохраняется в файл `photo_N.jpg`, где N — порядковый номер фото, начиная с 1; клавиша `Q` завершает программу и закрывает окно. Пустой кадр (`None`) не сохранять.

Что должно получиться: после трёх нажатий `S` рядом с программой лежат `photo_1.jpg`–`photo_3.jpg`, в терминале — три строки «Сохранено: photo_N.jpg»; после `Q` окно закрывается без ошибки.

8.9 Датчики в полёте и журнал

Положение и ошибку мы уже читали (раздел 7.8). Здесь — остальные датчики Пионера и запись данных полёта.

Основные датчики

- **Барометр:** Измеряет высоту по давлению воздуха.
- **IMU (Гироскоп + Акселерометр):** Измеряет углы наклона и ускорения.
- **Дальномер (ToF):** Лазерный датчик расстояния (смотрит вниз), измеряет дистанцию до поверхности под дроном — пола, крыши, холма. Команда `get_dist_sensor_data()`, метры или `None`.

- **Система оптического потока (OptiFlow):** Камера, смотрящая вниз, для удержания позиции. В симуляторе её показания читает `get_optical_flow()` — расширение симулятора: тройка (поток по X, поток по Y в рад/с в осях дрона, качество 0–255); скорость над поверхностью примерно равна потоку, умноженному на высоту.

Из этих датчиков программе на Python в симуляторе доступны положение LPS (`get_local_position_lps()`), курс (`get_yaw()`, только в версии SDK с GitFlic), дальномер, напряжение батареи и оптический поток; углов крена и тангажа и высоты по барометру в симуляторе нет.

Чтение данных дальномера

Дальномер полезен для точного удержания высоты при посадке или полете над препятствиями.

python/get_telemetry/GET_dist_sensor_data.py

```python
from pioneer_sdk import Pioneer
import time

# Создаем объект mini класса Pioneer
mini = Pioneer()

# Бесконечный цикл для постоянного опроса дальномера
while True:
    # Получаем данные с дальномера (в метрах)
    distance = mini.get_dist_sensor_data()

    # Проверяем, получены ли данные (не None)
    if distance is not None:
        # Выводим расстояние в консоль
        print(f'Расстояние: {distance:.2f} м')

    # Задержка вывода сообщений в консоль
    time.sleep(2)
```

Программа опрашивает дальномер бесконечно и печатает только настоящие измерения: `None` пропускается (виртуальный дальномер `None` не возвращает: над самой землёй он показывает 0). Остановить её — `Ctrl+C` или кнопкой остановки симулятора. Координаты относительно точки старта читает `get_local_position_lps()`; над ровным полом её z и показание дальномера почти совпадают, над холмом — нет.

ЗАДАНИЕ 8.10 · Мягкая посадка

Взлетите на 2 м. Используя дальномер, снижайтесь командой `set_manual_speed` с отрицательной `vz` (раздел 7.9): 0.5 м/с, пока до поверхности больше 0.5 м, затем 0.1 м/с. Когда до поверхности меньше 0.15 м, отправьте нулевую скорость и `land()`. Каждые 0.5 с печатайте показание дальномера; при `None` скорость нулевая.

Что должно получиться: в журнале показание дальномера сначала падает быстро, после 0.5 м — заметно медленнее; дрон касается площадки без удара, состояние

становится `DISARMED`.

В симуляторе: миссии Python: 65, 67 в панели «**Обучение**».

ЗАДАНИЕ 8.11 · Черный ящик

Только на компьютере (настоящий дрон или программа с компьютера на симулятор, раздел 7.1): в браузере файл программе недоступен. Во время полёта по квадрату каждые 0.5 с записывайте в файл `flight_log.txt` строку «время;x;y;z;напряжение», а при `None` — слово `нет` вместо числа. Файл открывайте конструкцией `with open(...) as f:`, чтобы он закрылся и при ошибке.

Что должно получиться: после посадки в файле по строке на каждые полсекунды полёта, время растёт, координаты повторяют вершины квадрата; файл открывается в любом текстовом редакторе.

В симуляторе: миссии Python: 72 в панели «**Обучение**».

8.10 Трекинг человека и жестов (только на дроне)

Используя библиотеку **MediaPipe** от Google, мы можем распознавать позу человека (скелет) в реальном времени даже на обычном процессоре. Напомним: `mediapipe` в браузерном симуляторе недоступна, поэтому примеры этой главы запускаются только на компьютере с реальным дроном.

Как это работает?

1. Получаем кадр с камеры. 2. MediaPipe находит ключевые точки (суставы): плечи, локти, запястья, нос и т.д. 3. Анализируя взаимное расположение точек, мы можем определить жест.

Пример: Управление жестами

В примере `human-tracking.py` реализована сложная логика управления. Давайте разберем основные идеи.

Определение жеста «Руки вверх»: Если запястья (y_{wrist}) выше носа (y_{nose}), значит человек поднял руки.

Логика жеста

```
if (left_wrist.y < nose.y) and (right_wrist.y < nose.y):
    print("Жест: Взлет!")
    drone.arm()          # сначала моторы, как в главе про полёт
    time.sleep(1)
    drone.takeoff()
```

Пропорциональное управление (P-регулятор)

Чтобы дрон поворачивался за человеком, мы вычисляем ошибку: насколько центр человека смещен от центра кадра.

$$Error = X_{person} - X_{center}$$

Управляющее воздействие (скорость поворота) пропорционально ошибке:

$$YawRate = K_p \times Error$$

Где K_p — коэффициент чувствительности.

ЗАДАНИЕ 8.12 · Джедай

Только на дроне. Напишите упрощённую программу жестов: дрон стоит на земле и не взлетает, камера включена, MediaPipe ищет позу на каждом кадре.

1. Поднята левая рука (запястье выше носа) — один светодиод красный.
2. Поднята правая — другой светодиод зелёный (номера светодиодов подберите опытом).
3. Руки скрещены (расстояние между запястьями меньше 0.1 в нормированных координатах MediaPipe) — все светодиоды мигают по очереди красным, зелёным и синим.

Жест засчитывается, если держится пять кадров подряд; команду подсветки отправляйте только при смене жеста.

Что должно получиться: поднятая рука зажигает свой светодиод примерно через четверть секунды; опущенные руки гасят подсветку; случайное движение в одном кадре цвет не меняет.

8.11 Групповой полет (только на дронах)

Управление роем (Swarm) — сложная, но захватывающая задача. Главное здесь — синхронизация и единая система координат.

Архитектура роя

Мы будем использовать централизованную архитектуру:

- **Один компьютер (Наземная станция)** управляет всеми дронами.
- Каждый дрон имеет свой уникальный IP-адрес.
- Программа создает отдельный поток (`Thread`) для каждого дрона.

Что с роем в симуляторе

В симуляторе каждый новый объект `Pioneer` с другим именем или адресом привязывается к следующему свободному дрону сцены, поэтому логику роя можно попробовать на сцене с несколькими дронами. Однако примеры ниже рассчитаны на настоящие IP-адреса и потоки `threading`, а в браузере потоки выполняются по очереди и общий момент команд не гарантируют; в симуляторе рой не проверялся, и результат надёжно получается только на настоящих дронах.

Пример: Синхронный взлет

В файле `group_flight_examples/swarm_OPT.py` показан пример, где несколько дронов выполняют задачи параллельно.

Ключевые моменты: 1. Создаем список IP-адресов. 2. Для каждого IP создаем объект `Pioneer`. Адрес обязательно передается по имени — `Pioneer(ip=ip)`: первый позиционный параметр конструктора называется `name`, поэтому запись `Pioneer(ip)` задаст дрону имя, а подключение пойдет на адрес по умолчанию. 3. Используем `threading.Thread` для запуска функций управления одновременно.

Псевдокод роя

```
drones = [Pioneer(ip=ip1), Pioneer(ip=ip2)]

def mission(drone):
    drone.arm()
    time.sleep(1)   # пауза между arm и takeoff обязательна
    drone.takeoff()
    time.sleep(5)
    drone.land()

# Запуск потоков
threads = []
for d in drones:
    t = threading.Thread(target=mission, args=(d,))
    t.start()
    threads.append(t)

# Ожидание завершения
for t in threads:
    t.join()
```

Безопасность роя

При групповых полетах риск столкновения возрастает многократно! 1. Всегда проверяйте координаты старта. 2. Расстояние между дронами должно быть не менее 1.5 метров. 3. Держите палец на кнопке аварийного отключения (или имейте об-

щий скрипт `disarm_all`).

ЗАДАНИЕ 8.13 · Танец

Только на настоящих дронах (на сцене симулятора — по желанию, см. врезку выше). Напишите программу для двух дронов с разными IP: дрон А взлетает; дрон Б ждёт 5 секунд, затем взлетает; дрон А садится; дрон Б садится. Каждым дроном управляет своя функция в своём потоке, а моменты событий согласуйте через `threading.Event` (А ставит флаг «взлетел», Б ждёт его). Расстояние между точками старта — не меньше 1.5 м.

Что должно получиться: дроны взлетают с интервалом около 5 с и садятся в порядке А, Б; если один дрон не подключился, программа сообщает об этом и не поднимает второй.

Итоги части о зрении и рое

- Кадр может не прийти (`None`), метки в кадре может не быть (`ids is None`) — это разные случаи, и оба проверяют до любых вычислений.
- ID метки называет рисунок, а не место: смысл ID задаёт таблица в программе, положение метки — только кадр.
- Ошибка метки относительно центра кадра — измерение в пикселях; из неё скорость делает регулятор с коэффициентом, пределом, мёртвой зоной и обработкой потери метки.
- Расстояние в метрах даёт `solvePnP` по размеру метки и параметрам камеры; размер и порядок углов должны совпадать с реальными.
- В симуляторе работают камера, ArUco, наведение и `solvePnP` с идеальной камерой; калибровка, `imwrite`, MediaPipe и настоящий рой — только на компьютере и дроне.

Контрольные вопросы

1. Чем отличается `frame is None` от `ids is None` и что делает программа в каждом случае?
2. Метка правее и ниже центра кадра. Какие знаки у e_x и e_y и почему у нижней камеры симулятора v_y берут с противоположным знаком к e_y?
3. Почему нельзя двигаться по ошибке, измеренной несколько кадров назад?
4. Что изменится в расстоянии из `tvec`, если в коде размер метки указан вдвое меньше настоящего?
5. Зачем в групповом полёте проверять подключение всех дронов до взлёта первого?

ГЛАВА 9

Итоговые проекты: самостоятельные миссии

Цели главы

- Соединить маршрут, камеру, распознавание, наведение и посадку в одной программе.
- Для каждого этапа заранее определить наблюдаемый результат и условие перехода к следующему.
- Проверять не только успех, но и ограниченные неуспешные исходы: цели нет, цель потеряна, карта другая.

Миссии симулятора к главе

Язык «**Python**», панель «**Обучение**», **100–108 «Самостоятельные миссии»**: карта из нескольких меток, поиск по секторам, лесная экспедиция, осмотр крыш, контрольные точки арены, инспекция промышленной зоны, неожиданная потеря цели, незнакомая карта, итоговая экспедиция. Новых команд в них нет: всё нужное — в главах 7 и 8.

Как оформлять проект

Программа собирается из функций прошлых глав: `wait_state`, `go` (точка с ожиданием и тайм-аутом), `read_target` (кадр и углы метки), `observation` (ошибка метки), `align` (наведение). У каждого этапа — своё сообщение в журнале и своё условие перехода; любой поиск и любое ожидание ограничены; при неудаче программа сообщает причину, обнуляет скорость и сажает дрон. Номер метки в коде не превращают в координаты: где метка, говорит только кадр.

Общие критерии оценки

1. **Результат на сцене:** дрон делает то, что требует проект, и садится; при неудаче — безопасно садится с понятным сообщением.
2. **Журнал:** по выводу программы восстанавливается порядок этапов, прочитанные ID и причина остановки.
3. **Ограниченность:** нет бесконечных ожиданий, движения по старому кадру и посадки «по цели» без подтверждения.
4. **Читаемость:** код разбит на функции, параметры (ID, высоты, допуски) вынесены в переменные.

9.1 Метки как карта

Несколько меток превращают полигон в карту с контрольными местами. Назначение метки хранят в словаре Python: `names = {10: "северо-запад", 11: "юго-восток", 12: "цель"}`, где ключ — ID, значение — смысл. Проверка `marker_id in names` отвечает, есть ли такой ID в таблице.

ЗАДАНИЕ 9.1 · Карта из нескольких меток

С высоты 3 м над стартом прочитайте все видимые метки и для каждой напечатайте «ID: назначение» по таблице `names`. Метки, которых в таблице нет, печатайте как «ID: неизвестна». После отчёта сядьте.

Что должно получиться: в журнале три строки: `10:северо-запад`, `11:юго-восток`, `12:цель` (в любом порядке), напечатаны только реально прочитанные ID; дрон садится на старте.

В симуляторе: миссии Python: 100 в панели «**Обучение**».

9.2 Поиск по секторам

Большую область делят на участки, которые камера видит целиком: с высоты 3 м нижняя камера видит около ± 2 м по Y, поэтому сектора ставят через 4 м. Список секторов конечен, каждый посещается один раз, после находки поиск прекращается.

ЗАДАНИЕ 9.2 · Поиск по секторам

Найдите метку 12, проверяя сектора $Y = 0, 4, 8$ при $X = 0$ на высоте 3 м. В каждом секторе печатайте его номер; после находки — «Найдена: 12» и прекратите поиск. Если метки нет ни в одном секторе, напечатайте «Поиск истёк». Вернитесь на старт и сядьте.

Что должно получиться: в сцене миссии проверяются два пустых сектора и сектор с целью, после находки дрон сразу летит домой; с несуществующим ID посещаются все три сектора и печатается `Поиск истёк`.

В симуляторе: миссии Python: 101 в панели «**Обучение**».

9.3 Экспедиции по полигонам

Маршрут доставляет к месту, камера подтверждает цель: это две разные проверки. Высоту и путь выбирают по местности (раздел 7.7), а кадр снимают уже после прибытия и паузы, когда корпус успокоился.

ЗАДАНИЕ 9.3 · Лесная экспедиция и крыши

В лесу пролетите центральный коридор ($X = 0$) до точки осмотра $(0, 8, 3)$, прочитайте ID 12, вернитесь и сядьте. В спальном районе наберите над стартом высоту 9 м, осмотрите крыши домов в $(9, -7)$ и $(-9, -7)$, напечатайте прочитанные ID и вернитесь на старт по той же высоте.

Что должно получиться: в лесу дрон не касается деревьев и печатает `Прочитан: 12`; над домами печатаются ID 102 и 101, высота везде выше крыш с запасом.

В симуляторе: миссии Python: 102–103 в панели «**Обучение**».

ЗАДАНИЕ 9.4 · Контрольные точки арены

На Геоскан Арене подлетите на высоте 6 м к карте ArUco в $(-9, 10.5)$, снизьтесь до 4 м и осмотрите её из двух точек: над $(-9, 10.5)$ и со сдвигом на 0.5 м по $+X$. В каждой точке напечатайте координаты прибытия и прочитанные ID, затем вернитесь на старт через высоту 6 м и сядьте.

Что должно получиться: из обеих точек в журнале ID 10 и 11 рядом с координатами точки; во втором кадре метки смещены влево относительно первого; посадка на старте.

В симуляторе: миссии Python: 104 в панели «**Обучение**».

ЗАДАНИЕ 9.5 · Инспекция промышленной зоны

Задайте участки списком пар `[(8, 12), (16, 13)]` — координата Y участка на линии $X = 0$ и ID, который должен там быть. На каждом участке ищите именно его ID; если цель участка не найдена, прервите инспекцию с сообщением и сядьте.

Что должно получиться: в журнале по порядку `Прочитан: 12` и `Прочитан: 13`; если поменять ID в списке местами, программа останавливается на первом участке с сообщением «Цель участка не найдена».

В симуляторе: миссии Python: 105 в панели «**Обучение**».

9.4 Потеря цели и незнакомая карта

Надёжная программа знает, что делать, когда всё пошло не так: при потере цели — обнулить скорость и вести ограниченное восстановление; на новой карте — не полагаться на запомненные координаты.

ЗАДАНИЕ 9.6 · Неожиданная потеря цели

Найдите метку 12 над стартом, отлетите на $(0, 5, 3)$, где её не видно, и восстановите цель не больше чем за три попытки: на каждой обнулите скорость и снимите кадр, на второй вернитесь к старту. При обнаружении напечатайте «Восстановлена» и прекратите поиск, иначе — «Поиск истёк». Сядьте.

Что должно получиться: журнал: «Потеряна», затем «Восстановлена» после возврата; если убрать возврат — три строки «Потеряна» и `Поиск истёк`.

В симуляторе: миссии Python: 106 в панели «**Обучение**».

ЗАДАНИЕ 9.7 · Незнакомая карта

Запустите программу посадки на метку (`12_01_land_on_marker.py`) без изменений на смешанной карте миссии 107, где метка 12 стоит в другом месте. Координаты метки в программу не записывайте. Напечатайте точку посадки и сравните её с $(0.6, 0.3)$.

Что должно получиться: дрон садится на метку 12 там, где она стоит на этой карте; напечатанная точка посадки заметно отличается от $(0.6, 0.3)$.

В симуляторе: миссии Python: 107 в панели «**Обучение**».

9.5 Итоговая экспедиция

ЗАДАНИЕ 9.8 · Итоговая экспедиция

Соберите полную миссию с параметром `target_id`: подготовка и взлёт с подтверждением, поиск метки по секторам $Y = 0, 1, -1$, намеренная потеря (отлёт на $(0, 5, 3)$), восстановление через точки $Y = 4$ и 0, наведение `align`, посадка на метку и отчёт. Запустите дважды: с `target_id = 12` и с `target_id = 249`, которого на карте нет.

Критерии проверки в симуляторе:

1. С ID 12: в журнале по порядку «Цель миссии: 12», находка, «Потеряна», «Восстановлена», центр подтверждён, дрон сел на метку.
2. С ID 249: поиск обходит три сектора, печатает «Поиск истёк», дрон возвращается на старт и садится без ложного сообщения об успехе.
3. Во время потери цели нет ненулевых команд скорости; все ожидания ограничены по времени или числу попыток.
4. Каждый переход между этапами объяснён в журнале свежим наблюдением, которое его разрешило.

Что должно получиться: оба запуска завершаются посадкой, отчёт первого — успех с восстановлением, второго — тайм-аут поиска.

В симуляторе: миссии Python: 108 в панели «**Обучение**».

9.6 Проекты для настоящего дрона

ЗАДАНИЕ 9.9 · Автономный инспектор

Только на дроне. Облетите четыре «стеллажа» — маркеры ID 0, 1, 2, 3 по углам комнаты. У каждого найдите маркер поворотом на месте (ограниченный поиск), наведитесь на него и зафиксируйте в журнале; затем вернитесь на старт и сядьте.

Что должно получиться: в журнале четыре подтверждённых ID в порядке обхода; если маркер не найден за отведённое время, программа пропускает стеллаж с сообщением и продолжает маршрут.

ЗАДАНИЕ 9.10 · Следуй за мной

Только на дроне. Удерживайте человека в центре кадра на безопасном расстоянии: поворот `yaw_rate` пропорционален смещению центра человека (MediaPipe, раздел 8.10); если плечи занимают больше половины ширины кадра — отлёт назад, если меньше четверти — вперёд. Скорости ограничены, при потере человека дольше 3 с — зависание и посадка.

Что должно получиться: дрон поворачивается вслед за человеком и держит дистанцию, не приближаясь ближе 1.5 м; когда человек уходит из кадра, дрон останавливается и садится.

ЗАДАНИЕ 9.11 · Гонка по воротам

Только на дроне (или на гоночном полигоне симулятора без камеры). Пролетите через ворота, отмеченные ArUco-маркерами, как можно быстрее: для каждых ворот — подводящая точка, центр проёма, выходная точка. Усложнение: ворота стоят на разной высоте.

Что должно получиться: дрон проходит каждые ворота через середину проёма, не задевая рамку; время прохождения печатается в конце.

В симуляторе: миссии Python: 61 в панели «**Обучение**».

Итоги

- Итоговая программа собирается из проверенных функций: ожидание состояния, перелёт с тайм-аутом, чтение метки, наведение.
- У каждого этапа есть наблюдаемый результат и условие перехода; любой поиск и ожидание ограничены, любая неудача заканчивается безопасной посадкой с причиной в журнале.
- ID метки — справочный ключ, а не координата: положение цели берут из свежего кадра, поэтому программа переносится на другую карту.

ЧАСТЬ III

Профессиональные навигационные системы

ГЛАВА 10

Теоретические основы визуальной навигации

В части о зрении вы находили метки ArUco, считали центр метки и ошибку относительно центра кадра, садились на метку. Эта глава объясняет, *как* устроены эти шаги внутри детектора и что нужно добавить, чтобы перейти от пикселей к метрам. Прочитав её, вы сможете объяснить, почему детектор не находит метку, оценить расстояние до метки по её размеру на кадре и выбрать словарь маркеров.

Где выполнять задания главы

- **Симулятор.** `Camera(view="bottom")`, детектор `cv2.aruco`, `solvePnP` с идеальной камерой, наведение `set_manual_speed`, процедурные полигоны с метками. Распознавание моделируется по геометрии сцены: детектор возвращает углы тех меток, которые целиком видны камере. Внутренних шагов OpenCV (порог, контуры, декодирование) в браузере нет, а `cv2.imwrite` ничего не сохраняет.
- **Компьютер с OpenCV** (`pip install opencv-contrib-python`). Разбор конвейера детектора, генерация маркеров, фильтры. Дрон не нужен.
- **Настоящий дрон или веб-камера.** Калибровка, расстояние в метрах по реальной метке, печать и монтаж, освещение, блики.

Каждое задание помечено, где его выполнять.

Миссии симулятора

Перед этой главой пройдите в панели «**Обучение**» миссии Python № 73–81 (камера) и № 82–90 (ArUco шаг за шагом). Ими проверяются словарь, детекция, отсутствие метки, четыре угла, центр и устойчивое распознавание — всё, на что опирается эта глава.

10.1 ArUco и AprilTag

В отличие от GPS, который работает только на улице и ошибается на метры, визуальные метки позволяют дрону определять положение с точностью до сантиметров, в том числе в помещении.

ArUco [4] и **AprilTag** [5] — самые популярные семейства фидуциальных (опорных) маркеров: чёрно-белые квадраты с закодированным номером, по которым алгоритм находит и номер, и ориентацию метки. В OpenCV оба семейства распознаёт один модуль `cv2.aruco` [6]: AprilTag в нём — просто ещё один словарь. Опыт со «своим» и «чужим»

словарём вы уже проделали в уроках 82–83 и в задании «Охота на маркеры» (раздел 8.2); здесь разбирается, что происходит *внутри* детектора.

10.2 Как детектор находит метку

Обнаружение состоит из четырёх этапов. Понимание этого конвейера (pipeline) нужно для диагностики, когда дрон «не видит» метку.

1. Предварительная обработка

Камера передает цветное изображение (в OpenCV — в порядке каналов BGR). Алгоритм сначала переводит его в оттенки серого (Grayscale), так как цвет не несет информации для черно-белых маркеров.

Затем применяется **адаптивная бинаризация** (Adaptive Thresholding). В отличие от простого порога (где все пиксели ярче 127 становятся белыми, а темнее — черными), адаптивный метод вычисляет порог для каждой небольшой области изображения. Это позволяет находить маркеры даже при неравномерном освещении (например, когда одна часть маркера в тени).

Программа ниже показывает весь конвейер на одной картинке. Без аргументов она сама рисует маркер ID 12 на листе, правая часть которого в тени, и сравнивает общий и адаптивный порог; с именем файла — обрабатывает вашу фотографию. Запускайте её на компьютере: в симуляторе функций порога и контуров OpenCV нет.

Конвейер детектора по шагам (OpenCV, компьютер)

```
"""Конвейер детектора ArUco по шагам: серый кадр, порог, контуры, детекция.

Запускается на компьютере с полной библиотекой OpenCV:
    pip install opencv-contrib-python
    python binarization_example.py            # тестовая картинка с тенью
    python binarization_example.py photo.jpg  # своя фотография маркера
В симуляторе cv2 — учебная модель: adaptiveThreshold и findContours там нет.
"""
import sys

import cv2
import numpy as np


def make_test_image():
    """Маркер ID 12 (DICT_6X6_250) на белом листе, освещённом
    ↪  неравномерно."""
    dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
    marker = cv2.aruco.generateImageMarker(dictionary, 12, 200)
    sheet = np.full((400, 600), 255, np.uint8)
    sheet[100:300, 200:400] = marker
    # Слева светло, справа тень: яркость падает с 100% до 25%.
    shade = np.linspace(1.0, 0.25, sheet.shape[1])
    sheet = (sheet * shade[np.newaxis, :]).astype(np.uint8)
```

```
    return cv2.cvtColor(sheet, cv2.COLOR_GRAY2BGR)


def count_quads(binary, min_area=400):
    """Сколько контуров похожи на маркер: 4 вершины, выпуклый, не мелкий."""
    # OpenCV 3 возвращает три значения, OpenCV 4 — два; контуры всегда
    ↪ предпоследние.
    contours = cv2.findContours(binary, cv2.RETR_LIST,
    ↪ cv2.CHAIN_APPROX_SIMPLE)[-2]
    quads = 0
    for contour in contours:
        if cv2.contourArea(contour) < min_area:
            continue
        approx = cv2.approxPolyDP(contour, 0.03 * cv2.arcLength(contour,
        ↪ True), True)
        if len(approx) == 4 and cv2.isContourConvex(approx):
            quads += 1
    return quads


def detect_ids(gray):
    dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
    if hasattr(cv2.aruco, "ArucoDetector"):
        detector = cv2.aruco.ArucoDetector(dictionary,
        ↪ cv2.aruco.DetectorParameters())
        corners, ids, _ = detector.detectMarkers(gray)
    else:
        corners, ids, _ = cv2.aruco.detectMarkers(gray, dictionary)
    return [] if ids is None else [int(value[0]) for value in ids]


def main():
    if len(sys.argv) > 1:
        frame = cv2.imread(sys.argv[1])
        if frame is None:
            print("Не удалось прочитать файл:", sys.argv[1])
            return
    else:
        frame = make_test_image()

    # 1. Цвет маркеру не нужен: переходим к оттенкам серого.
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 2а. Один порог на весь кадр: в тени белые клетки становятся «чёрными».
    _, global_binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)

    # 2б. Адаптивный порог: (src, maxValue, method, type, blockSize, C).
    # blockSize — размер окрестности (нечётный), C — поправка к среднему
    ↪ окрестности.
    adaptive_binary = cv2.adaptiveThreshold(
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV,
        ↪ 21, 7)

    # 3. Кандидаты в маркеры — выпуклые четырёхугольники.
    print("Четырёхугольников, общий порог:", count_quads(global_binary))
    print("Четырёхугольников, адаптивный порог:",
    ↪ count_quads(adaptive_binary))
```

```
    # 4. Полный детектор: выпрямление перспективы и чтение битов по словарю.
    print("Найдены ID:", detect_ids(gray))

    cv2.imshow("gray", gray)
    cv2.imshow("global threshold", global_binary)
    cv2.imshow("adaptive threshold", adaptive_binary)
    cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__ == "__main__":
    main()
```

2. Поиск контуров

Алгоритм ищет границы связных областей на бинарном изображении (после `THRESH_BINARY_INV` чёрная рамка маркера становится белой областью). Из всех найденных контуров отбираются только те, которые:

- Являются замкнутыми.
- Аппроксимируются четырехугольником (имеют 4 вершины).
- Являются выпуклыми (Convex).
- Имеют достаточный размер (площадь).

Именно это делает функция `count_quads` в примере выше.

3. Исправление перспективы

Найденный четырехугольник может быть искажен перспективой (выглядеть как трапеция или ромб), если камера смотрит на него под углом. Алгоритм вычисляет матрицу гомографии, чтобы «выпрямить» изображение и получить идеальный квадрат (Canonical Marker Image).

4. Декодирование

Выпрямленный квадрат разбивается на сетку: биты данных плюс чёрная рамка шириной в одну клетку (у маркера 6×6 это 8×8 клеток). Каждая клетка анализируется: черная — 0, белая — 1. Полученная матрица битов сверяется со словарем (Dictionary) во всех четырёх поворотах — так детектор узнаёт и номер маркера, и то, какой из углов «первый». Поэтому `corners` всегда перечисляет углы в одном порядке: по часовой стрелке, начиная с верхнего левого угла самого маркера.

ЗАДАНИЕ 10.1 · Тень и порог

Компьютер с OpenCV. Запустите `binarization_example.py` без аргументов. Сравните окна `global threshold` и `adaptive threshold` и числа четырёхугольников. Затем сделайте тень глубже: в `make_test_image` замените `0.25` на `0.1`, — и снова сравните.

Что должно получиться: при общем пороге белые клетки маркера в тени становятся «чёрными» и четырёхугольников меньше, адаптивный порог сохраняет рамку и клетки; детектор печатает `Найдены ID: [12]`. Объясните, почему от глубины тени зависит только общий порог.

10.3 Коррекция ошибок и расстояние Хэмминга

Маркеры ArUco спроектированы так, чтобы быть устойчивыми к ошибкам считывания. Даже если несколько пикселей (битов) будут распознаны неверно (из-за блика или грязи), алгоритм все равно сможет узнать маркер.

Это достигается за счет избыточности информации. Каждый словарь имеет параметр **Hamming Distance** (Расстояние Хэмминга) — минимальное количество битов, на которое отличаются любые два маркера в словаре (с учётом их поворотов).

- Если словарь имеет расстояние Хэмминга $d = 4$, система может обнаружить до $d-1 = 3$ ошибок и исправить до $\lfloor (d-1)/2 \rfloor = 1$ ошибки.
- Чем больше бит в маркере (например, 6×6 против 4×4), тем дальше друг от друга можно разнести маркеры словаря и тем ниже риск ложных срабатываний. Чем больше маркеров в словаре при той же сетке, тем меньше это расстояние: `DICT_6X6_50` надёжнее, чем `DICT_6X6_1000`.

ЗАДАНИЕ 10.2 · Расстояние Хэмминга своими руками

Компьютер или симулятор, только Python. Запишите два маркера 4×4 списками строк: `A = ["1010", "0110", "1100", "0011"]` и `B = ["1011", "0100", "1100", "1011"]`. Напишите функцию `hamming(a, b)` (число несовпадающих битов) и функцию `rotate(m)` (поворот матрицы на 90° по часовой стрелке). Напечатайте расстояние от `A` до `B` и до трёх поворотов `B`.

Что должно получиться: без поворота — 3; для поворотов на 90°, 180° и 270° — 7, 11 и 9. Минимум (3) и есть расстояние между этими маркерами с учётом поворотов: такой словарь обнаружит 2 ошибки и исправит одну.

Какой словарь выбрать

При выборе словаря нужно искать баланс между количеством доступных ID и размером маркера (читаемостью).

Словарь	Размер сетки	Кол-во ID	Применение
DICT_4X4_50	4×4 бита	50	Дальние дистанции, низкое разрешение. Меньше деталей — легче распознать.
DICT_5X5_1000	5×5 бит	1000	Компромисс: много номеров при средней надёжности.
DICT_6X6_250	6×6 бит	250	Высокая надежность, используется в картах Пионера и в симуляторе; его первые 50 маркеров — это `DICT_6X6_50`.
DICT_APRILTAG_36h11	6×6 бит	587	Промышленный стандарт, очень высокая устойчивость к ложным срабатываниям (минимальное расстояние Хэмминга 11).

Таблица 2. Характеристики словарей маркеров

10.4 От пикселей к метрам: модель камеры и PnP

Модель камеры-«обскуры». Точка с координатами (X, Y, Z) в системе камеры (Z — вперёд из объектива) попадает на кадр в пиксель

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y,$$

где f_x, f_y — фокусные расстояния *в пикселях*, (c_x, c_y) — центр кадра. Эти числа и коэффициенты дисторсии образуют внутренние параметры камеры (**Camera Matrix** и **Distortion Coefficients**), их получают калибровкой [7]. Из формулы следует главное: метка стороной S на расстоянии d занимает на кадре примерно $s \approx f\,S/d$ пикселей, то есть размер на кадре обратно пропорционален расстоянию. Фокусное расстояние связано с углом обзора по вертикали φ и высотой кадра h: $f = (h/2)/\tan(\varphi/2)$. Для нижней камеры симулятора ($h = 360$, $\varphi = 72°$) это $f \approx 248$ пикселей.

Задача PnP (Perspective-n-Point) обратная: известны

1. координаты четырёх углов метки в её собственной системе (3D-точки), например $(-S/2, S/2, 0)$, $(S/2, S/2, 0)$, $(S/2, -S/2, 0)$, $(-S/2, -S/2, 0)$: S — сторона метки, начало координат в её центре, порядок такой же, как у углов из детектора;
2. координаты этих же углов на кадре (2D-точки, пиксели);
3. внутренние параметры камеры.

Функция `cv2.solvePnP` находит признак успеха и два вектора: `rvec` — вращение метки относительно камеры (формат Родригеса) и `tvec` — смещение (x, y, z) центра метки в системе камеры.

Важно!

Вектор `tvec` показывает координаты *метки в системе камеры*. Чтобы получить координаты *камеры (дрона) в системе метки*, преобразование инвертируют:

$$R = Rodrigues(rvec), \qquad Position_{drone} = -R^T \cdot tvec.$$

Что есть в симуляторе

В симуляторе `solvePnP` работает: он решает задачу по углам метки из детектора и параметрам идеальной камеры (`cv2.FileStorage` отдаёт матрицу нижней камеры без дисторсии). Поэтому расстояние до метки там считается как на настоящей камере, но у настоящей камеры матрица иная, а метка печатается и измеряется вами. Метры положения дрона в обоих случаях лучше брать из телеметрии (`get_local_position_lps`), а метку использовать как цель или поправку.

ЗАДАНИЕ 10.3 · Масштаб метки

Симулятор. Зависните над меткой 12 (в $(0.6, 0.3)$) на высотах 2, 3 и 4 м (сцена миссий Python № 82–90). На каждой высоте по свежему кадру найдите среднюю длину четырёх сторон метки по её углам (`math.dist` между соседними углами) и напечатайте z, s и $s \cdot z$. Затем сравните s с $f\,S/z$, где $f \approx 248$ пикселей, а S — сторона чёрного квадрата метки (около 0.84 м).

Что должно получиться: s уменьшается с высотой (порядка 100, 70 и 50 пикселей), а $s \cdot z$ почти одинаково на всех трёх высотах (расхождение в пределах нескольких процентов); s совпадает с $f\,S/z$ до нескольких процентов. Отсюда по размеру метки на кадре оценивают расстояние до неё.

В симуляторе: миссии Python: 78, 87–88 в панели «**Обучение**».

ЗАДАНИЕ 10.4 · Расстояние по tvec

Настоящий дрон или веб-камера. Откалибруйте камеру (глава 11), распечатайте метку со стороной 10 см и запустите `11_02_marker_distance.py`, поправив размер метки и убрав взлёт. Поставьте метку на 0,5, 1,0 и 1,5 м от камеры (рулеткой) и запишите третью компоненту `tvec`.

Что должно получиться: `tvec[2]` отличается от измеренного расстояния не боль-

ше чем на 5%. Если ошибка растёт с расстоянием пропорционально — в коде указан неверный размер метки; если хаотична — повторите калибровку.

10.5 Практика: словари и освещение

ЗАДАНИЕ 10.5 · Анализ словарей

Компьютер и принтер, камера дрона или веб-камера. Сгенерируйте метку ID 1 из словарей 4×4, 5×5 и 6×6 одного физического размера (5 см) скриптом `19_map_creation_example2_generation.py`, распечатайте их и отходите с камерой, пока метки распознаются.

Что должно получиться: таблица «словарь — предельное расстояние»; первым пропадает 6×6: при том же размере его клетки мельче и раньше сливаются в пиксели камеры.

ЗАДАНИЕ 10.6 · Освещение и блики

Настоящая камера. Запустите детектор на живом видео и постепенно затемняйте комнату, затем посветите фонариком прямо на метку. Посмотрите окно с чёрно-белым изображением (`binarization_example.py`).

Что должно получиться: при темноте метка пропадает из-за шума и смаза, при блике — из-за белого пятна на месте чёрных клеток; в окне бинаризации видно, какие клетки «перевернулись».

Контрольные вопросы

1. Почему для навигации используют ArUco, а не QR-коды? (Подсказка: сравните сложность определения ориентации и устойчивость к наклону.)
2. Что такое адаптивная бинаризация и когда простой порог ошибается?
3. Как детектор узнаёт, что наклонённая трапеция — это квадрат, и какой из углов у метки первый?
4. В какой системе координат задан вектор `tvec` и как получить положение дрона относительно метки?
5. Метка вдвое дальше от камеры. Как изменится её размер на кадре и значение `tvec[2]`?
6. Почему положение дрона в метрах в симуляторе берут из телеметрии, а метку используют как цель?

ГЛАВА 11

Создание навигационных карт

Карта маркеров связывает номер метки с её местом на полигоне. В миссиях симулятора вы уже пользовались такой договорённостью: в миссии Python № 100 метка 10 означала «северо-запад», 11 — «юго-восток», 12 — «цель», а в миссиях № 104–105 маршрут строился по известным координатам меток. Эта глава учит готовить такую карту для настоящего полигона: откалибровать камеру, напечатать и наклеить маркеры, записать их положения в файл. Калибровка, печать и монтаж выполняются на настоящем оборудовании; в симуляторе проверяют только готовую карту, «обещание» «в этой точке видна такая метка».

11.1 Калибровка камеры

Перед тем как создавать карту, необходимо откалибровать камеру. Камера дрона — это не идеальный оптический прибор. Линзы (особенно широкоугольные «рыбий глаз») вносят искажения: прямые линии на краях кадра могут выглядеть изогнутыми. Это называется **дисторсией** (Distortion).

Если не учесть эти искажения, координаты маркеров будут вычислены с ошибкой в десятки сантиметров.

> **Только настоящая камера**
>
> Калибруют камеру дрона или веб-камеру компьютера с полной библиотекой OpenCV [8]. Камера симулятора — идеальная модель без дисторсии: калибровать её не нужно, а её матрицу ($f \approx 248$ пикселей, центр $(320, 180)$) программе отдаёт `cv2.FileStorage`. Функций калибровки (`findChessboardCorners`, `calibrateCamera`) в браузерном `cv2` нет.

11.1.1 Процесс калибровки

Для калибровки используется специальный шаблон — шахматная доска (Chessboard). Метод восходит к работе Чжана [7]: по нескольким снимкам плоского шаблона под разными углами вычисляются параметры камеры.

1. Распечатайте доску и наклейте её на жесткий лист. Скрипт ниже ищет 6×9 *внутренних углов* — это доска из 7×10 клеток. Для другой доски поменяйте константу `CHECKERBOARD`.
2. Запустите скрипт калибровки.

3. Покажите доску камере под разными углами и в разных частях кадра (20-30 кадров, клавиша `1` сохраняет кадр, `Esc` — завершает съёмку).
4. Скрипт вычислит:
   - **Camera Matrix**: Фокусное расстояние (f_x, f_y) и оптический центр (c_x, c_y).
   - **Distortion Coefficients**: Коэффициенты радиальной (k_1, k_2, k_3) и тангенциальной (p_1, p_2) дисторсии.
   - **Ошибку репроекции** — насколько в среднем (в пикселях) расходятся найденные углы доски и углы, пересчитанные по полученной модели.

Скрипт ниже, как и `camera_calibration.py` из раздела про камеру, сохраняет параметры в `data.yml` (узлы `mtx` и `dist`); примеры навигации загружают их оттуда при запуске. Шаблон `camera_params.yaml` из приложения показывает тот же набор чисел в формате, который принимают сторонние инструменты.

Скрипт калибровки камеры

```python
#!/usr/bin/python3

"""
Калибровка камеры по шахматной доске: матрица камеры и коэффициенты
дисторсии сохраняются в data.yml (узлы mtx и dist).
Нужна полная библиотека OpenCV (pip install opencv-contrib-python) и
↪  настоящая
камера — дрона или компьютера. Камера симулятора — идеальная модель без
дисторсии, калибровать её не нужно.
Подробнее: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
"""

import cv2
import numpy as np
import glob
import time
from pioneer_sdk import Camera


def get_images_from_folder(folder_name, file_type="*.jpg"):
    """Загрузить снимки доски из папки `folder_name`."""
    images_list = glob.glob(folder_name + "/" + file_type)
    images = []
    for fname in images_list:
        img = cv2.imread(fname)
        images.append(img)
    return images


def get_images_from_drone_camera(camera, save=False):
    """Снимать кадры с камеры дрона: клавиша 1 — сохранить кадр, Esc или q —
    ↪  закончить."""
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        frame = camera.get_cv_frame()
        if frame is not None:
            cv2.imshow("frames", frame)
```

```
            key = cv2.waitKey(1)
            if key == 27 or key == ord("q"):
                cv2.destroyAllWindows()
                return images
            if key == ord("1"):
                images.append(frame)
                print("Frame #" + str(len(images)))
                if save:
                    cv2.imwrite("frame_" + str(len(images)) + ".jpg", frame)
                time.sleep(1)


def get_images_from_computer_camera(capture, save=False):
    """То же с веб-камерой компьютера (cv2.VideoCapture)."""
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        ret, frame = capture.read()
        cv2.imshow("frames", frame)

        key = cv2.waitKey(1)
        if key == 27 or key == ord("q"):
            cv2.destroyAllWindows()
            return images
        if key == ord("1"):
            images.append(frame)
            print("Frame #" + str(len(images)))
            if save:
                cv2.imwrite("frame_" + str(len(images)) + ".jpg", frame)
            time.sleep(1)


def calibrate(images):
    # Число ВНУТРЕННИХ углов доски: 6 x 9 углов - это доска 7 x 10 клеток
    CHECKERBOARD = (6, 9)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
    ↪  0.001)

    objpoints = []  # 3D-координаты углов доски (в клетках) для каждого
    ↪  снимка
    imgpoints = []  # те же углы на изображении, пиксели

    # Углы доски в её собственной системе: (0,0,0), (1,0,0), ... - доска
    ↪  плоская, z = 0
    objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
    objp[0, :, :2] = np.mgrid[0 : CHECKERBOARD[0], 0 :
    ↪  CHECKERBOARD[1]].T.reshape(-1, 2)

    gray = None
    for img in images:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # ret = True, если найдены все углы доски
        ret, corners = cv2.findChessboardCorners(
            gray,
            CHECKERBOARD,
            cv2.CALIB_CB_ADAPTIVE_THRESH
```

```
            + cv2.CALIB_CB_FAST_CHECK
            + cv2.CALIB_CB_NORMALIZE_IMAGE,
        )

        if ret == True:
            objpoints.append(objp)
            # Уточняем положение углов до долей пикселя
            corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
            ↪  criteria)

            imgpoints.append(corners2)

            # Показываем найденные углы
            img = cv2.drawChessboardCorners(img, CHECKERBOARD, corners2, ret)

        cv2.imshow("img", img)
        cv2.waitKey(0)

    cv2.destroyAllWindows()

    if len(objpoints) < 10:
        raise RuntimeError("Доска найдена на %d снимках, нужно хотя бы 10" %
        ↪  len(objpoints))

    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
        objpoints, imgpoints, gray.shape[::-1], None, None
    )
    # ret — средняя ошибка репроекции в пикселях: хорошая калибровка — меньше
    ↪  1
    print("Снимков с доской:", len(objpoints))
    print("Ошибка репроекции, пиксели: %.3f" % ret)

    return mtx, dist


def save_coefficients(mtx, dist, path):
    """Сохранить матрицу камеры и коэффициенты дисторсии в файл."""
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_WRITE)
    cv_file.write("mtx", mtx)
    cv_file.write("dist", dist)
    cv_file.release()


def load_coefficients(path):
    """Прочитать матрицу камеры и коэффициенты дисторсии из файла."""
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)

    # .mat() превращает узел файла в матрицу питру
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()

    cv_file.release()
    return camera_matrix, dist_coeffs


def main():
    """
```

```
    В этом примере изображения берутся с камеры дрона.
    Однако вы можете изменить источник изображений, заменив
    функцию `get_images_from_drone_camera` на другую, например:

    ```
    # Загрузить изображения из папки
    images = get_images_from_folder('images')
    # ...
    ```

    ```
    # Получить изображения с веб-камеры компьютера
    capture = cv2.VideoCapture(0)
    images = get_images_from_computer_camera(capture)
    capture.release()
    # ...
    ```
    """

    # Снимки с камеры дрона
    camera = Camera()
    images = get_images_from_drone_camera(camera)

    mtx, dist = calibrate(images)

    save_coefficients(mtx, dist, "data.yml")

if __name__ == "__main__":
    main()
```

ЗАДАНИЕ 11.1 · Калибровка

Настоящий дрон или веб-камера. Проведите процедуру калибровки камеры Pioneer.

1. Сделайте 20 снимков шахматной доски.
2. Запустите скрипт калибровки.
3. Сравните изображение до и после устранения дисторсии (функция `cv2.undistort`). Прямые ли линии косяков дверей на краях кадра?

Что должно получиться: файл `data.yml` с узлами `mtx` и `dist`; скрипт печатает ошибку репроекции меньше 1 пикселя; на исправленном кадре прямые линии у краёв кадра прямые.

11.2 Генерация и печать маркеров

Используйте скрипт `19_map_creation_example2_generation.py` для создания изображений. Он запускается на компьютере и складывает PNG-файлы в папку `markers`.

Генерация маркеров ArUco/AprilTag

```python
"""Генерация PNG-маркеров ArUco и AprilTag для печати.

Запуск на компьютере: pip install opencv-contrib-python; python
↪ 19_map_creation_example2_generation.py
Файлы появятся в папке markers рядом с местом запуска.
"""
import json
import os

import cv2
import cv2.aruco as aruco

output_dir = "markers"
os.makedirs(output_dir, exist_ok=True)

# Словари, из которых генерируем маркеры
dictionaries = {
    "DICT_4X4_50": aruco.DICT_4X4_50,
    "DICT_5X5_100": aruco.DICT_5X5_100,
    "DICT_6X6_250": aruco.DICT_6X6_250,
    "DICT_7X7_1000": aruco.DICT_7X7_1000,
    "DICT_APRILTAG_36h11": aruco.DICT_APRILTAG_36h11
}

def add_visual_aids(image, size_pixels):
    """Белое поле вокруг маркера и оси X (красная) и Y (зелёная) - для
    ↪ плаката."""
    color_img = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)

    # Белое поле (quiet zone) шириной 10% стороны
    border_size = int(size_pixels * 0.1)
    color_img = cv2.copyMakeBorder(color_img, border_size, border_size,
    ↪ border_size, border_size,
                                   cv2.BORDER_CONSTANT, value=[255, 255,
                                   ↪ 255])

    # Начало осей - левый верхний угол самого маркера
    origin_x = border_size
    origin_y = border_size

    cv2.arrowedLine(color_img, (origin_x, origin_y), (origin_x + 100,
    ↪ origin_y), (0, 0, 255), 2)
    cv2.putText(color_img, "X", (origin_x + 110, origin_y + 10),
    ↪ cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

    cv2.arrowedLine(color_img, (origin_x, origin_y), (origin_x, origin_y +
    ↪ 100), (0, 255, 0), 2)
    cv2.putText(color_img, "Y", (origin_x + 10, origin_y + 110),
    ↪ cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    return color_img

def generate_marker(dict_name, marker_id, size_pixels=200,
↪ visual_aids=False):
    """Один маркер: PNG со стороной size_pixels пикселей."""
```

```
    try:
        aruco_dict = aruco.getPredefinedDictionary(dictionaries[dict_name])

        marker_image = aruco.generateImageMarker(aruco_dict, marker_id,
        ↪  size_pixels)

        if visual_aids:
            final_image = add_visual_aids(marker_image, size_pixels)
            suffix = "_viz"
        else:
            final_image = marker_image
            suffix = ""

        filename = f"{output_dir}/{dict_name}_id{marker_id}{suffix}.png"
        cv2.imwrite(filename, final_image)
        print(f"Создан: {filename}")
    except Exception as e:
        print(f"Ошибка: маркер {dict_name} ID {marker_id}: {e}")

def generate_json_metadata(dict_name, count, size_pixels):
    # Размер сетки по имени словаря: DICT_4X4_50 -> 4
    bits = 0
    if "4X4" in dict_name: bits = 4
    elif "5X5" in dict_name: bits = 5
    elif "6X6" in dict_name: bits = 6
    elif "7X7" in dict_name: bits = 7
    elif "APRILTAG" in dict_name: bits = "variable"

    data = {
        "dictionary": dict_name,
        "bits": bits,
        "count": count,
        "size_pixels": size_pixels,
        "description": "Marker set for Geoskan Pioneer"
    }
    filename = f"{output_dir}/{dict_name}_metadata.json"
    with open(filename, 'w') as f:
        json.dump(data, f, indent=4)
    print(f"Описание набора: {filename}")

def generate_batch(dict_name, start_id, count, size_pixels=200,
↪  visual_aids=False):
    """Маркеры с номерами start_id .. start_id + count - 1."""
    print(f"--- {dict_name}: {count} маркеров ---")
    for i in range(start_id, start_id + count):
        generate_marker(dict_name, i, size_pixels, visual_aids)

    generate_json_metadata(dict_name, count, size_pixels)

if __name__ == "__main__":
    # 1. Наборы по словарям
    generate_batch("DICT_4X4_50", 0, 5)
    generate_batch("DICT_5X5_100", 0, 2)
    generate_batch("DICT_6X6_250", 0, 5)  # словарь карт Пионера и симулятора
    generate_batch("DICT_7X7_1000", 0, 2)
    generate_batch("DICT_APRILTAG_36h11", 0, 5)
```

```
    # 2. Маркеры с осями — для объяснения ориентации
    generate_marker("DICT_6X6_250", 0, 300, visual_aids=True)
    generate_marker("DICT_APRILTAG_36h11", 0, 300, visual_aids=True)

    # 3. Большие наборы (раскомментируйте при необходимости)
    # generate_batch("DICT_6X6_250", 0, 50)
    # generate_batch("DICT_APRILTAG_36h11", 0, 50)

    print(f"Готово: маркеры в папке '{output_dir}'.")
```

11.2.1 Векторные маркеры

Для печати в типографии или создания масштабируемых карт рекомендуется использовать векторные форматы (PDF/SVG). Следующий скрипт генерирует маркеры с использованием векторной графики. Одна клетка маркера здесь — один квадрат: для этого маркер сначала строится размером «пиксель на клетку» (у 4×4 это 6 пикселей с рамкой, у 6×6 и AprilTag 36h11 — 8).

Генерация векторных маркеров

```
"""Векторные (PDF) маркеры и лист-карта маркеров для печати без потери
↪  чёткости.

Запуск на компьютере: pip install opencv-contrib-python matplotlib
Файлы появятся в папке markers/vector рядом с местом запуска.
"""
import os

import cv2.aruco as aruco
import matplotlib.pyplot as plt
import matplotlib.patches as patches

output_dir = os.path.join("markers", "vector")
os.makedirs(output_dir, exist_ok=True)


def get_marker_bits(dictionary, marker_id, side_pixels):
    """
    Маркер, в котором один пиксель — одна клетка. side_pixels — сетка данных
    плюс чёрная рамка шириной в клетку с каждой стороны:
    ArUco 4x4 -> 6, ArUco 6x6 -> 8, AprilTag 36h11 (6x6 бит данных) -> 8.
    Если задать другое число, OpenCV растянет картинку и клетки станут
    ↪  неровными.
    """
    return aruco.generateImageMarker(dictionary, marker_id, side_pixels)

def draw_vector_marker(ax, img, x_offset=0, y_offset=0, size=1.0):
    """Рисует маркер квадратами matplotlib — получается векторная графика."""
    rows, cols = img.shape
    pixel_size = size / rows

    # Белый фон
    rect = patches.Rectangle((x_offset, y_offset), size, size, linewidth=0,
    ↪  facecolor='white')
```

```
    ax.add_patch(rect)

    # Чёрные клетки; строка 0 картинки — верх, а у matplotlib ось Y идёт
    ↪  вверх
    for r in range(rows):
        for c in range(cols):
            if img[r, c] == 0:
                rect_x = x_offset + c * pixel_size
                rect_y = y_offset + (rows - 1 - r) * pixel_size

                rect = patches.Rectangle((rect_x, rect_y), pixel_size,
                ↪  pixel_size, linewidth=0, facecolor='black')
                ax.add_patch(rect)

def save_single_marker(dict_name, marker_id, bit_size, filename):
    dictionary = aruco.getPredefinedDictionary(getattr(aruco, dict_name))
    img = get_marker_bits(dictionary, marker_id, bit_size)

    fig, ax = plt.subplots(figsize=(4, 4))
    draw_vector_marker(ax, img, size=10)

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)
    ax.axis('off')
    ax.set_aspect('equal')

    plt.savefig(filename, format='pdf', bbox_inches='tight', pad_inches=0)
    plt.close()
    print(f"Сохранён маркер: {filename}")

def save_grid_map(dict_name, rows, cols, bit_size, filename):
    dictionary = aruco.getPredefinedDictionary(getattr(aruco, dict_name))

    # Лист А4: 8.27 x 11.69 дюйма
    fig, ax = plt.subplots(figsize=(8.27, 11.69))

    margin_x = 1
    margin_y = 1
    cell_width = (8.27 - 2 * margin_x) / cols
    cell_height = (11.69 - 2 * margin_y) / rows
    marker_size = min(cell_width, cell_height) * 0.8

    # Заполняем лист с левого верхнего угла
    start_x = margin_x
    start_y = 11.69 - margin_y - cell_height

    for i in range(rows):
        for j in range(cols):
            marker_id = i * cols + j
            img = get_marker_bits(dictionary, marker_id, bit_size)

            x = start_x + j * cell_width + (cell_width - marker_size) / 2
            y = start_y - i * cell_height + (cell_height - marker_size) / 2

            draw_vector_marker(ax, img, x_offset=x, y_offset=y,
            ↪  size=marker_size)
```

```
            # Подпись с номером
            ax.text(x + marker_size/2, y - 0.2, f"ID: {marker_id}",
            ↪  ha='center', va='top', fontsize=8)

    ax.set_xlim(0, 8.27)
    ax.set_ylim(0, 11.69)
    ax.axis('off')
    ax.set_aspect('equal')

    plt.savefig(filename, format='pdf', bbox_inches='tight')
    plt.close()
    print(f"Сохранён лист: {filename}")

if __name__ == "__main__":
    # 1. Отдельные маркеры
    save_single_marker("DICT_4X4_50", 0, 6,
    ↪  f"{output_dir}/aruco_4x4_id0.pdf")
    save_single_marker("DICT_6X6_250", 12, 8,
    ↪  f"{output_dir}/aruco_6x6_id12.pdf")
    save_single_marker("DICT_APRILTAG_36h11", 0, 8,
    ↪  f"{output_dir}/apriltag_id0.pdf")

    # 2. Листы-карты: 4 x 4 = 16 маркеров
    save_grid_map("DICT_4X4_50", 4, 4, 6, f"{output_dir}/aruco_grid_4x4.pdf")
    save_grid_map("DICT_APRILTAG_36h11", 4, 4, 8,
    ↪  f"{output_dir}/apriltag_grid_4x4.pdf")
```

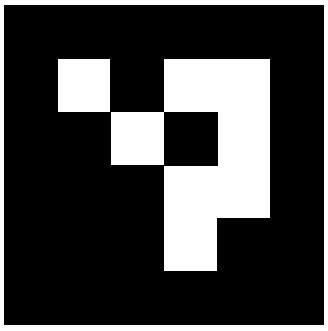


Рис. 21. ArUco 4x4

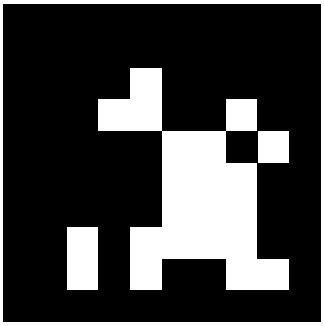


Рис. 22. AprilTag 36h11

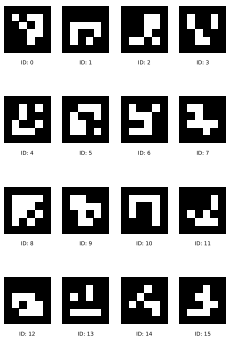


Рис. 23. Карта 4x4

Важно!

При печати в настройках принтера выберите «Реальный размер» (Actual Size) или «Масштаб 100%». Иначе размер маркера изменится, и навигация будет врать.

ЗАДАНИЕ 11.2 · Свой набор меток

Компьютер, принтер, камера. Сгенерируйте маркеры ID 10, 11 и 12 словаря `DICT_6X6_250` (как в миссиях симулятора), напечатайте их со стороной чёрного

квадрата 15 см и проверьте линейкой. Покажите их камере, запустив детектор из главы об ArUco со словарём `DICT_6X6_250`.

Что должно получиться: сторона каждого напечатанного маркера 150 ± 1 мм; детектор печатает все три номера. Если маркер не распознаётся, проверьте, тем ли словарём он сгенерирован.

11.2.2 Требования к монтажу

1. **Жесткость:** Маркер должен быть идеально плоским. Изгиб бумаги даже на 5 мм приведет к ошибке определения угла (Yaw/Pitch/Roll) на 10-15 градусов. Наклейте маркер на картон, пластик или фанеру.
2. **Quiet Zone:** Вокруг черной рамки маркера обязательно должно быть белое поле шириной не менее одного бита. Если наклеить черный маркер на черный пол, он не распознается.
3. **Матовое покрытие:** Глянцевая бумага или скотч создают блики от ламп. В месте блика камера видит белое пятно вместо черного пикселя. Используйте матовую ламинацию или матовый скотч.
4. **Освещение:** Избегайте размещения маркеров в глубокой тени или напротив ярких окон (контражур).

11.3 Форматы карт (JSON, XML, KML)

Карта маркеров — это файл, описывающий положение каждого маркера. Наш инструмент `19_map_creation_example3_builder.py` строит сетку 10×10 маркеров словаря `DICT_6X6_250` и сохраняет её в форматах JSON, XML, KML, YAML, а также схемой в PDF и картинкой PNG для печати. Координаты в нём — в локальной системе Пионера: X вперёд от старта, Y влево, Z вверх, метры.

Инструмент создания карт

```
"""Конструктор карты маркеров: сетка маркеров и экспорт в JSON, XML, KML,
↪  YAML, PDF и PNG.

Запуск на компьютере: pip install opencv-contrib-python matplotlib numpy
↪  pyyaml
Файлы появятся в папке maps рядом с местом запуска.
Координаты — локальная система Пионера: X вперёд от старта, Y влево, Z вверх,
↪  метры.
"""
import json
import math
import os

import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
```

```
class Marker:
    def __init__(self, marker_id, size, x, y, z, yaw=0,
    ↪  dictionary="DICT_6X6_250"):
        self.id = marker_id
        self.size = size  # сторона чёрного квадрата, м
        self.x = x        # локальные координаты центра, м
        self.y = y
        self.z = z        # высота (0 – на полу)
        self.yaw = yaw    # поворот маркера, градусы
        self.dictionary = dictionary

    def to_dict(self):
        return {
            "id": self.id,
            "size": self.size,
            "position": {"x": self.x, "y": self.y, "z": self.z},
            "orientation": {"yaw": self.yaw},
            "dictionary": self.dictionary
        }


class MarkerMap:
    def __init__(self, name="MyMap", origin_gps=(0, 0)):
        self.name = name
        self.origin_lat = origin_gps[0]  # широта точки старта, градусы
        self.origin_lon = origin_gps[1]  # долгота точки старта, градусы
        self.markers = []

    def add_marker(self, marker):
        self.markers.append(marker)

    def generate_grid(self, rows, cols, step_x, step_y, start_id=0,
    ↪  marker_size=0.2, dictionary="DICT_6X6_250"):
        """Сетка rows x cols маркеров на полу; первый – в начале
        ↪  координат."""
        for i in range(rows):
            for j in range(cols):
                self.add_marker(Marker(
                    marker_id=start_id + i * cols + j,
                    size=marker_size,
                    x=i * step_x,
                    y=j * step_y,
                    z=0,
                    yaw=0,
                    dictionary=dictionary
                ))
        print(f"Сетка {rows}x{cols}: {len(self.markers)} маркеров.")

    def save_json(self, filename):
        data = {
            "map_name": self.name,
            "origin": {"lat": self.origin_lat, "lon": self.origin_lon},
            "markers": [m.to_dict() for m in self.markers]
        }
        with open(filename, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=4, ensure_ascii=False)
```

```
        print(f"JSON: {filename}")

    def save_yaml(self, filename):
        """YAML со списком маркеров и их поз: такой формат читают инструменты
        ↪ на ROS."""
        import yaml

        markers_data = []
        for m in self.markers:
            markers_data.append({
                "id": int(m.id),
                "size": float(m.size),
                "frame_id": "map",
                "pose": {
                    "position": {"x": float(m.x), "y": float(m.y), "z":
                    ↪ float(m.z)},
                    # Поворот вокруг оси Z в виде кватерниона
                    "orientation": {
                        "x": 0.0,
                        "y": 0.0,
                        "z": math.sin(math.radians(m.yaw) / 2),
                        "w": math.cos(math.radians(m.yaw) / 2)
                    }
                }
            })
        with open(filename, "w", encoding="utf-8") as f:
            yaml.dump({"markers": markers_data}, f, default_flow_style=False)
        print(f"YAML: {filename}")

    def save_png_grid(self, filename, resolution_px=4096):
        """Карта одной картинкой: маркеры в масштабе, для печати на
        ↪ баннере."""
        import cv2
        import cv2.aruco as aruco

        xs = [m.x for m in self.markers]
        ys = [m.y for m in self.markers]
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)

        padding_m = 1.0  # белое поле вокруг карты, м
        width_m = (max_y - min_y) + 2 * padding_m    # по горизонтали
        ↪ картинки идёт Y
        height_m = (max_x - min_x) + 2 * padding_m   # по вертикали - X
        scale = resolution_px / max(width_m, height_m)  # пикселей на метр

        img_h = int(height_m * scale)
        img_w = int(width_m * scale)
        image = np.full((img_h, img_w), 255, dtype=np.uint8)
        print(f"PNG: {img_w}x{img_h} пикселей, {scale:.1f} пикс/м")

        for m in self.markers:
            name = m.dictionary if hasattr(aruco, m.dictionary) else
            ↪ "DICT_6X6_250"
            adict = aruco.getPredefinedDictionary(getattr(aruco, name))
            side = int(m.size * scale)
            marker_img = aruco.generateImageMarker(adict, m.id, side)
```

```
            # Вид сверху, X – вверх по листу, Y – влево: у картинки строки
            ↪ идут вниз,
            # столбцы – вправо, поэтому обе координаты отражаем.
            center_col = int((max_y - m.y + padding_m) * scale)
            center_row = int((max_x - m.x + padding_m) * scale)
            top, left = center_row - side // 2, center_col - side // 2
            if 0 <= top <= img_h - side and 0 <= left <= img_w - side:
                image[top:top + side, left:left + side] = marker_img
            else:
                print(f"Маркер {m.id} не помещается на лист")

        cv2.imwrite(filename, image)
        print(f"PNG: {filename}")

    def save_pdf(self, filename, trajectory=None):
        """Схема карты на листе А4: квадраты маркеров, номера, ориентация,
        ↪ маршрут."""
        fig, ax = plt.subplots(figsize=(11.69, 8.27))  # А4, альбомный лист
        ↪ (дюймы)

        for m in self.markers:
            half = m.size / 2
            ax.add_patch(patches.Rectangle(
                (m.x - half, m.y - half), m.size, m.size,
                linewidth=1, edgecolor="black", facecolor="none"))
            ax.text(m.x, m.y, str(m.id), ha="center", va="center",
            ↪ fontsize=8, fontweight="bold")
            # Стрелка показывает поворот маркера
            arrow_len = m.size * 0.4
            dx = arrow_len * math.cos(math.radians(m.yaw))
            dy = arrow_len * math.sin(math.radians(m.yaw))
            ax.arrow(m.x, m.y, dx, dy, head_width=m.size * 0.1,
            ↪ head_length=m.size * 0.1,
                     fc="blue", ec="blue", alpha=0.5)

        if trajectory:
            traj_x = [p[0] for p in trajectory]
            traj_y = [p[1] for p in trajectory]
            ax.plot(traj_x, traj_y, "r--", label="маршрут", linewidth=2,
            ↪ alpha=0.7)
            ax.scatter(traj_x[0], traj_y[0], c="green", marker="o",
            ↪ label="старт")
            ax.scatter(traj_x[-1], traj_y[-1], c="red", marker="x",
            ↪ label="финиш")
            ax.legend()

        if self.markers:
            ax.set_xlim(min(m.x for m in self.markers) - 1, max(m.x for m in
            ↪ self.markers) + 1)
            ax.set_ylim(min(m.y for m in self.markers) - 1, max(m.y for m in
            ↪ self.markers) + 1)
        ax.set_aspect("equal")
        ax.set_xlabel("X, м")
        ax.set_ylabel("Y, м")
        ax.set_title(f"Карта маркеров: {self.name}")
        ax.grid(True, linestyle=":", alpha=0.6)
```

```
        plt.figtext(0.02, 0.02, f"Маркеров: {len(self.markers)}\nСтарт:
        ↪  {self.origin_lat}, {self.origin_lon}", fontsize=10)

        plt.savefig(filename, format="pdf")
        plt.close()
        print(f"PDF: {filename}")

    def local_to_gps(self, x, y):
        """
        Приближённый перевод метров в градусы для небольших участков.
        Считаем, что на старте дрон смотрит на север: X - на север, Y (влево)
        ↪  - на запад.
        Для геодезической точности нужна библиотека pyproj и проекция UTM.
        """
        lat_deg_per_meter = 1 / 111132.928
        lon_deg_per_meter = 1 / (111132.928 *
        ↪  math.cos(math.radians(self.origin_lat)))
        lat = self.origin_lat + x * lat_deg_per_meter
        lon = self.origin_lon - y * lon_deg_per_meter  # влево = на запад =
        ↪  долгота меньше
        return lat, lon

    def save_kml(self, filename):
        """KML для Google Earth и QGIS: каждый маркер - метка на карте."""
        kml_header = """<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <name>{name}</name>
"""
        kml_footer = """  </Document>
</kml>"""

        kml_body = ""
        for m in self.markers:
            m_lat, m_lon = self.local_to_gps(m.x, m.y)
            # В KML порядок координат: долгота, широта, высота
            kml_body += f"""
    <Placemark>
      <name>Marker {m.id}</name>
      <description>Size: {m.size} m, Yaw: {m.yaw}</description>
      <Point>
        <coordinates>{m_lon:.7f},{m_lat:.7f},{m.z}</coordinates>
      </Point>
    </Placemark>"""

        with open(filename, "w", encoding="utf-8") as f:
            f.write(kml_header.format(name=self.name) + kml_body +
            ↪  kml_footer)
        print(f"KML: {filename}")

    def save_xml(self, filename):
        """Простой XML-формат; разбор - через xml.etree.ElementTree."""
        xml_content = f"""<?xml version="1.0" encoding="UTF-8"?>
<map>
    <name>{self.name}</name>
    <origin>
        <lat>{self.origin_lat}</lat>
```

```
        <lon>{self.origin_lon}</lon>
    </origin>
    <markers>
"""
        for m in self.markers:
            xml_content += f"""        <marker id="{m.id}">
            <size>{m.size}</size>
            <position>
                <x>{m.x}</x>
                <y>{m.y}</y>
                <z>{m.z}</z>
            </position>
            <orientation>
                <yaw>{m.yaw}</yaw>
            </orientation>
            <dictionary>{m.dictionary}</dictionary>
        </marker>
"""
        xml_content += """    </markers>
</map>"""

        with open(filename, "w", encoding="utf-8") as f:
            f.write(xml_content)
        print(f"XML: {filename}")


if __name__ == "__main__":
    out = "maps"
    os.makedirs(out, exist_ok=True)

    # Полигон 4.5 x 4.5 м: сетка 10 x 10 маркеров через 0.5 м, сторона
    ↪ маркера 0.2 м.
    # 100 номеров (0..99) помещаются в DICT_6X6_250 — словарь карт Пионера и
    ↪ симулятора.
    # Для сетки больше 250 маркеров нужен словарь крупнее, например
    ↪ DICT_6X6_1000.
    my_map = MarkerMap("Field_10x10", origin_gps=(59.9343, 30.3351))
    my_map.generate_grid(rows=10, cols=10, step_x=0.5, step_y=0.5,
    ↪ start_id=0,
                         marker_size=0.2, dictionary="DICT_6X6_250")

    # Маршрут для схемы: квадрат над картой на высоте 1 м
    trajectory = [(0.5, 0.5, 1.0), (4.0, 0.5, 1.0), (4.0, 4.0, 1.0), (0.5,
    ↪ 4.0, 1.0), (0.5, 0.5, 1.0)]

    my_map.save_json(os.path.join(out, "field_map.json"))
    my_map.save_xml(os.path.join(out, "field_map.xml"))
    my_map.save_kml(os.path.join(out, "field_map.kml"))
    my_map.save_pdf(os.path.join(out, "field_map.pdf"),
    ↪ trajectory=trajectory)
    my_map.save_yaml(os.path.join(out, "field_map.yaml"))
    my_map.save_png_grid(os.path.join(out, "field_map_4096.png"),
    ↪ resolution_px=4096)
```

11.3.1 JSON (JavaScript Object Notation)

Основной формат для роботов. Легко читается программами на Python/C++.

Пример карты JSON

```
{
  "map_name": "Warehouse_Zone_A",
  "origin": {"lat": 59.9, "lon": 30.3},
  "markers": [
    {
      "id": 10,
      "size": 0.2,
      "position": {"x": 5.0, "y": 0.0, "z": 1.5},
      "orientation": {"yaw": 90.0}
    }
  ]
}
```

11.3.2 XML (eXtensible Markup Language)

Более строгий формат, часто используется в промышленных системах.

Пример карты XML

```
<map>
    <name>Warehouse_Zone_A</name>
    <marker id="10">
        <size>0.2</size>
        <position>
            <x>5.0</x> <y>0.0</y> <z>1.5</z>
        </position>
    </marker>
</map>
```

11.3.3 KML (Keyhole Markup Language)

Формат для отображения на географических картах (Google Earth, QGIS). Позволяет визуализировать схему расположения маркеров поверх спутниковых снимков. Это полезно для планирования полетов на улице. Для пересчёта метров в градусы нужно знать, куда смотрел дрон на старте; инструмент считает, что на север (X — на север, Y — на запад).

ЗАДАНИЕ 11.3 · Сверка карты с местностью

Симулятор. Карта — это обещание: «в этой точке видна такая метка». Проверьте его на встроенной Геоскан Арене (сцена миссии Python № 104). Запишите карту строкой JSON прямо в программе (у программы в браузере нет доступа к файлам компьютера) и разберите её через `json.loads`: `{"points": [{"x": -9, "y": 10.5, "ids": [10, 11]}]}`. Для каждой точки

поднимитесь на $Z = 6$, долетите до неё, снизьтесь до $Z = 4$, прочитайте ID в свежем кадре и сравните множество прочитанных ID с ожидаемым. Затем вернитесь на старт через $Z = 6$ и сядьте.

Что должно получиться: для верной карты программа печатает «совпадает»; если заменить в карте 11 на 12, она печатает «расхождение: нет 12» и всё равно возвращается и садится.

В симуляторе: миссии Python: 100, 104 в панели «**Обучение**».

11.4 Визуализация и планирование

Для больших полигонов рекомендуется сначала создать план в CAD-системе или нарисовать схему на миллиметровке, определить координаты установки маркеров, и только потом заносить их в файл карты. Схему в PDF строит метод `save_pdf` инструмента выше; он же наносит запланированный маршрут.

ЗАДАНИЕ 11.4 · Создание полигона

Настоящий полигон. Создайте карту для полета «восьмеркой».

1. Нарисуйте траекторию на полу мелом.
2. Разместите 2 маркера в центрах петель восьмерки.
3. Измерьте их координаты рулеткой относительно точки старта (X — вперёд, Y — влево).
4. Создайте JSON файл карты и схему `save_pdf` с траекторией.

Что должно получиться: JSON-файл с двумя маркерами и PDF-схема, на которой оба маркера лежат в центрах петель; координаты в файле совпадают с рулеткой с точностью до 2–3 см.

Контрольные вопросы

1. Что такое дисторсия и почему она мешает навигации?
2. Почему нельзя ламинировать маркеры глянцевой пленкой?
3. Зачем нужно белое поле вокруг маркера?
4. В чем преимущество формата KML перед JSON?
5. Почему номер метки сам по себе ничего не говорит о её месте (вспомните миссии № 100 и № 104)?

ГЛАВА 12

Алгоритмы комплексной навигации

Миссии симулятора

Глава продолжает миссии Python № 64–72 (измерения и устойчивость) и № 91–99 (наведение по метке): ограничение скорости, мёртвая зона, потеря метки и снижение только над целью — это и есть практическая часть регулятора, о котором идёт речь ниже.

12.1 Проблема шума и фильтрация

Данные, получаемые от системы компьютерного зрения, всегда содержат шум. Координаты дрона «дрожат» даже если он висит неподвижно. Это происходит из-за дискретности матрицы камеры, изменений освещения и вибраций корпуса.

Если передавать эти «сырые» координаты напрямую в полетный контроллер, дрон будет дергаться, пытаясь отработать каждое ложное смещение.

12.1.1 Фильтр Калмана (Kalman Filter)

Это стандарт де-факто в робототехнике [9]. Фильтр Калмана не просто усредняет значения, он предсказывает, где дрон *должен* оказаться в следующий момент времени, и корректирует это предсказание на основе новых измерений.

Алгоритм работает циклически в два этапа:

1. **Prediction (Предсказание):** На основе физической модели (скорость, ускорение) мы оцениваем новую позицию. При этом растет неопределенность (мы не уверены в прогнозе).
2. **Update (Коррекция):** Мы получаем измерение от камеры. Сравниваем его с прогнозом. Чем точнее датчик (меньше шум измерения R), тем больше мы верим ему. Чем точнее наша модель (меньше шум процесса Q), тем больше верим прогнозу.

Пример 1D фильтра Калмана

```python
# Метод класса KalmanFilter1D; весь класс и опыт с ним —
# в файле examples/python/advanced_navigation/kalman_filter.py
def update(self, measurement):
```

```
    # 1. Предсказание: модель «величина не меняется», неопределённость растёт
    ↪  на Q
    x_pred = self.x
    P_pred = self.P + self.Q

    # 2. Коррекция: коэффициент Калмана K решает, насколько верить измерению
    K = P_pred / (P_pred + self.R)
    self.x = x_pred + K * (measurement - x_pred)
    self.P = (1 - K) * P_pred
    return self.x
```

Выше приведен только ключевой метод `update()` с простейшей моделью «величина не меняется». Полный запускаемый скрипт с классом `KalmanFilter1D`, экспоненциальным сглаживанием для сравнения и модельным опытом находится в файле `advanced_navigation/kalman_filter.py` — именно его нужно запускать в заданиях ниже. Он работает на компьютере (нужен `numpy`, для графика — `matplotlib`): дрон висит на высоте 1,5 м, датчик шумит, а на 50-м шаге дрон «толкнули» на 2,0 м. Программа печатает разброс оценок до скачка и через сколько шагов каждый фильтр «догнал» новую высоту.

ЗАДАНИЕ 12.1 · Сравнение фильтров

Компьютер. Запустите скрипт `kalman_filter.py`.

1. Измените `MEASUREMENT_NOISE` (шум измерения R) с 0.2 на 0.01, затем на 5.0. Как меняются разброс оценки и время реакции?
2. Сравните фильтр Калмана и экспоненциальное сглаживание при резком скачке высоты. Кто быстрее реагирует? Подберите `ALPHA` так, чтобы разброс сглаживания стал как у Калмана.

Что должно получиться: с параметрами по умолчанию разброс оценок около 0,03 м против 0,18 м у измерений, реакция на скачок — 23 шага у Калмана и 18 у сглаживания. При малом R фильтр почти повторяет измерения (быстро, но шумно), при большом — сглаживает сильнее и реагирует медленнее. Вывод: фильтр со статической моделью всегда меняет плавность на запаздывание.

ЗАДАНИЕ 12.2 · Сглаживаем ошибку наведения

Симулятор. В сцене миссии Python № 95 (дрон над меткой 12 в точке $(0.6, 0.3, 3)$) в 20 свежих кадрах вычисляйте ошибку e_x и пропускайте её через экспоненциальное сглаживание с $\alpha = 0.3$. Печатайте сырую и сглаженную ошибку. Затем отлетите к $Y = 5$ (метка пропадает, как в миссии № 97) и вернитесь.

Что должно получиться: пока метка видна, сглаженная ошибка меняется плавнее сырой; при потере метки программа *сбрасывает* фильтр (`value = None`) и обнуляет скорость, а после возвращения начинает сглаживание заново, а не со старого значения.

В симуляторе: миссии Python: 95–97 в панели «**Обучение**».

12.2 Системы координат

Одна из самых частых ошибок новичков — путаница в системах координат.

- **Body Frame (Связанная система):** оси привязаны к корпусу. В робототехнике чаще всего X смотрит вперёд (нос дрона), Y — влево, Z — вверх. Если дрон поворачивается, оси поворачиваются вместе с ним. Внимание: `pioneer_sdk` в командах `..._body_fixed` понимает связанные оси иначе — *x вправо, y вперёд*, z вверх.
- **World Frame / NED (North-East-Down):** Ось X смотрит на Север, Y — на Восток, Z — Вниз (к земле). Эта система неподвижна. Локальная система Пионера, в которой мы задавали точки (раздел 5), тоже неподвижна, но устроена иначе: X вперёд от старта, Y влево, Z *вверх*. Если дрон на старте смотрит на север, при переходе между ними меняются знаки Y и Z.
- **Camera Frame (система камеры OpenCV):** X вправо по кадру, Y вниз по кадру, Z — вперёд из объектива. В ней выражены `tvec` и `rvec`. Для нижней камеры ось Z камеры смотрит вниз, и связь с осями дрона задаётся тем, как камера закреплена на корпусе.
- **Marker Frame:** Ось Z обычно смотрит перпендикулярно плоскости маркера (вверх от него).

Для навигации нам нужно переводить положение маркера, измеренное камерой, в глобальные координаты (World Frame). Сначала вектор из системы камеры поворачивают в систему корпуса (R_{cam} — как закреплена камера), затем — в мировую (R_{drone} — текущая ориентация дрона):

$$Pos_{world} = Pos_{drone} + R_{drone} \cdot R_{cam} \cdot Pos_{marker_camera}$$

ЗАДАНИЕ 12.3 · Оси кадра и оси дрона

Симулятор. Повторите опыт миссий Python № 75 и № 92 для обеих осей: зависните над меткой 12 на $Z = 3$, запишите ошибку (e_x, e_y), затем одну секунду лететь со скоростью 0,2 м/с сначала по $+X$, потом (после возврата) по $+Y$ (`set_manual_speed`, команда каждые 0,1 с, затем явная остановка) и снова измерьте ошибку.

Что должно получиться: таблица «движение дрона — изменение e_x, e_y» для нижней камеры симулятора; по ней выбраны знаки поправок, и они совпадают с контроллером `align` миссии № 96 (v_x со знаком $+e_x$, v_y со знаком $-e_y$).

В симуляторе: миссии Python: 75, 92, 96 в панели «**Обучение**».

ЗАДАНИЕ 12.4 · Мировые и связанные оси

Симулятор. Взлетите на $Z = 1.5$. При курсе 0 запомните положение, две секунды подавайте `set_manual_speed(0.3, 0, 0, 0)` (каждые 0,1 с), остановитесь и напечатайте смещение по X и Y (`get_local_position_lps`). Повторите то же с `set_manual_speed_body_fixed(0, 0.3, 0, 0)`. Затем поверните дрон на курс $\pi/2$ (`go_to_local_point(x, y, z, yaw=math.pi/2)`) и проделайте оба опыта ещё раз.

Что должно получиться: таблица из четырёх смещений длиной около 0,6 м: у

`set_manual_speed` направление смещения при обоих курсах одно и то же (мировая ось X), у `set_manual_speed_body_fixed` оно поворачивается на 90° вместе с курсом — дрон летит туда, куда смотрит нос.

В симуляторе: миссии Python: 45 в панели «**Обучение**».

12.3 Точная посадка с PID-регулятором

Для автоматической посадки на зарядную станцию используется PID-регулятор.

- **P (Proportional):** Чем дальше мы от центра, тем сильнее наклоняемся к нему.
- **I (Integral):** Если нас сносит ветром, мы накапливаем ошибку и наклоняемся сильнее, чтобы компенсировать снос.
- **D (Derivative):** Если мы летим к центру слишком быстро, мы «тормозим» заранее, чтобы не пролететь мимо.

Команда скорости на каждом шаге:

$$v = K_p e + K_i \sum e\,\Delta t + K_d \frac{e - e_{\text{пред}}}{\Delta t},$$

после чего v обязательно ограничивают сверху (миссия № 70), а внутри мёртвой зоны обнуляют (миссия № 95).

Программа ниже — П-регулятор из миссий № 94–99, собранный в одну посадку: на каждой ступени высоты (3, 2 и 1,5 м) она добивается трёх кадров подряд с меткой в допуске 8 пикселей и только потом снижается дальше; при потере метки скорость обнуляется. Запускайте её в редакторе симулятора в сцене миссии № 99.

Посадка на метку ступенями (симулятор)

```python
"""Посадка на ArUco-метку в симуляторе: наведение по пикселям и снижение
↪  ступенями.

Где запускать: редактор симулятора, язык Python, сцена с меткой 12 под
↪  стартом
(её готовит миссия Python 99 «Посадка по ArUco» в панели «Обучение»).
Camera(view="bottom") - расширение симулятора; на настоящем дроне камеру
выбирают под его оборудование, а знаки поправок проверяют заново.
"""
import time

import cv2
from pioneer_sdk import Pioneer, Camera

TARGET_ID = 12
KP = 2.0                  # П-коэффициент: доля кадра -> м/с
V_MAX = 0.3               # предел скорости, м/с
TOL_PX = 8                # допуск центрирования, пиксели
STABLE_FRAMES = 3         # столько кадров подряд метка должна быть в допуске
LEVELS = [3.0, 2.0, 1.5]  # ступени снижения, м
```

```
pioneer = Pioneer()
camera = Camera(view="bottom")
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(dictionary)


def wait_for(test, limit=120):
    deadline = time.monotonic() + limit
    while time.monotonic() < deadline:
        if test():
            return
        time.sleep(0.05)
    raise RuntimeError("тайм-аут ожидания")


def go(x, y, z):
    pioneer.go_to_local_point(x, y, z, yaw=None)
    wait_for(pioneer.point_reached)
    time.sleep(1.5)  # корпус успокаивается, следующий кадр снят уже на месте


def clamp(value, limit):
    return max(-limit, min(limit, value))


def observe(target_id):
    """Свежий кадр -> ошибка (ex, ey) центра метки от центра кадра и размеры
    ↪ кадра."""
    frame = camera.get_cv_frame()
    if frame is None:
        return None
    corners, ids, _ = detector.detectMarkers(frame)
    if ids is None:
        return None
    for i, value in enumerate(ids):
        if int(value[0]) == target_id:
            points = corners[i][0]
            h, w = frame.shape[:2]
            cx = sum(float(p[0]) for p in points) / 4
            cy = sum(float(p[1]) for p in points) / 4
            return cx - w / 2, cy - h / 2, w, h
    return None


def align(target_id, attempts=120):
    """П-регулятор по двум осям; True, когда метка STABLE_FRAMES кадров
    ↪ подряд в допуске."""
    stable = 0
    for _ in range(attempts):
        seen = observe(target_id)
        if seen is None:
            pioneer.set_manual_speed(0, 0, 0, 0)   # по устаревшей ошибке не
            ↪ летим
            stable = 0
        else:
            ex, ey, w, h = seen
```

```
            if abs(ex) <= TOL_PX and abs(ey) <= TOL_PX:
                pioneer.set_manual_speed(0, 0, 0, 0)
                stable += 1
                if stable >= STABLE_FRAMES:
                    return True
            else:
                stable = 0
                # Знаки — для нижней камеры симулятора (миссии 92 и 96):
                # метка правее центра -> летим по +X, ниже центра -> по -Y.
                vx = clamp(KP * ex / w, V_MAX)
                vy = clamp(-KP * ey / h, V_MAX)
                pioneer.set_manual_speed(vx, vy, 0, 0)
        time.sleep(0.1)
    pioneer.set_manual_speed(0, 0, 0, 0)
    return False


pioneer.arm()
pioneer.takeoff()
wait_for(lambda: pioneer.get_autopilot_state() == "MISSION")
try:
    go(0, 0, LEVELS[0])
    for level in LEVELS:
        x, y, _ = pioneer.get_local_position_lps()
        go(x, y, level)                    # снижаемся, сохраняя измеренные
        ↪ X и Y
        if not align(TARGET_ID):
            raise RuntimeError("Метка не наведена на высоте %.1f м" % level)
        print("По центру на высоте", level)
    pioneer.land()
    wait_for(lambda: pioneer.get_autopilot_state() == "DISARMED")
    lx, ly, _ = pioneer.get_local_position_lps()
    print("Посадка в точке:", round(lx, 2), round(ly, 2))
except RuntimeError as error:
    pioneer.set_manual_speed(0, 0, 0, 0)
    pioneer.land()
    print("Прервано:", error)
```

⚠ При переносе на настоящий дрон

`Camera(view="bottom")` — расширение симулятора. На дроне камеру выбирают под его оборудование, а знаки и коэффициенты проверяют заново коротким импульсом (как в миссии № 92): от того, как закреплена камера, зависит, в какую сторону лететь при $e_x > 0$.

ЗАДАНИЕ 12.5 · Посадка в круг

Симулятор. Возьмите `staged_landing_sim.py` в сцене миссии Python № 99.

1. Запустите как есть и запишите точку посадки.
2. Добавьте D-составляющую: храните прошлую ошибку и прибавляйте $K_d(e - e_{\text{пред}})$ с $K_d = 0.5$ (ошибку берите в долях кадра, как K_p). Сравните число кадров до «По центру» на высоте 3 м.

3. Уменьшите допуск `TOL_PX` до 4 пикселей. Что происходит со временем наведения?

Что должно получиться: дрон садится не дальше 0,15 м от метки (в этой сцене она лежит около точки $(0.6; 0.3)$ — сравните с напечатанной точкой посадки); с D-составляющей наведение заканчивается за меньшее или равное число кадров без раскачки; при слишком узком допуске растёт время, а если метку не удаётся удержать в допуске за 120 попыток, программа печатает «Прервано» и садится.

В симуляторе: миссии Python: 94–99 в панели «**Обучение**».

Посадку на незнакомой карте (миссия № 107) вы уже выполняли в главе 9: программа, которая берёт положение метки только из кадров, садится на неё где бы та ни стояла, а программа с запомненными координатами — нет.

Контрольные вопросы

1. Зачем нужен фильтр Калмана, если можно просто взять среднее арифметическое за 10 кадров?
2. Что произойдет с PID-регулятором, если перепутать знак у коэффициента P?
3. В какой системе координат (Body, World или камеры) камера выдает вектор `tvec`?
4. Почему при потере метки нельзя продолжать полёт со старой скоростью?

Итоги

- Данные зрения шумят: сырое измерение сглаживают (экспоненциальное сглаживание, фильтр Калмана), а при потере цели фильтр сбрасывают.
- Перед расчётом поправки выясняют систему координат: оси кадра, оси корпуса и оси сцены различаются, а `set_manual_speed` и `set_manual_speed_body_fixed` понимают оси по-разному.
- Регулятор: P — пропорционально ошибке, I — накопленная ошибка, D — скорость её изменения; результат всегда ограничивают, а внутри мёртвой зоны обнуляют.
- Точная посадка идёт ступенями: центрирование, подтверждение несколькими кадрами, снижение, снова центрирование.

ГЛАВА 13

Техническая реализация программных модулей

В этой главе мы разберем исходный код инструментов, которые используются для создания карт и навигации. Всё здесь — обычный Python на компьютере: дрон и симулятор понадобятся только в последнем задании, чтобы получить журнал настоящего полёта.

13.1 Генератор карт (Marker Map Builder)

Скрипт `19_map_creation_example3_builder.py` (глава «Создание навигационных карт») позволяет создавать цифровые карты полигонов и экспортировать их в форматы JSON, XML, KML, YAML, PDF и PNG.

13.1.1 Структура класса Marker

Каждый маркер описывается классом `Marker`, который хранит:

- `id`: Уникальный номер маркера.
- `size`: Физический размер стороны в метрах.
- `x, y, z`: Локальные координаты относительно начала координат карты.
- `yaw`: Угол поворота (ориентация), градусы.
- `dictionary`: Тип словаря (например, DICT_6X6_250).

Класс Marker

```
class Marker:
    def __init__(self, marker_id, size, x, y, z, yaw=0,
    ↪  dictionary="DICT_6X6_250"):
        self.id = marker_id
        self.size = size  # in meters
        self.x = x        # local x (meters)
        self.y = y        # local y (meters)
        self.z = z        # local z (meters) - height
        self.yaw = yaw    # orientation (degrees)
        self.dictionary = dictionary
```

ЗАДАНИЕ 13.1 · Свой парсер

Компьютер. Напишите функцию `load_json(filename)`, которая считывает файл карты (формат метода `save_json`) и возвращает список объектов класса `Marker`. Сохраните сетку 10×10 из конструктора, загрузите её обратно и сравните.

Что должно получиться: после загрузки 100 маркеров; у каждого `id`, `size`, `x`, `y`, `z`, `yaw` и `dictionary` совпадают с исходными (проверка `assert` в цикле проходит без ошибок).

13.1.2 Экспорт в XML

Метод `save_xml` генерирует структуру, понятную большинству промышленных систем. Мы используем f-строки для формирования XML-тегов.

Экспорт в XML

```
def save_xml(self, filename):
    xml_content = f"""<?xml version="1.0" encoding="UTF-8"?>
<map>
    <name>{self.name}</name>
    <origin>
        <lat>{self.origin_lat}</lat>
        <lon>{self.origin_lon}</lon>
    </origin>
    <markers>
"""
    for m in self.markers:
        xml_content += f"""        <marker id="{m.id}">
            <size>{m.size}</size>
            <position>
                <x>{m.x}</x>
                <y>{m.y}</y>
                <z>{m.z}</z>
            </position>
            <orientation>
                <yaw>{m.yaw}</yaw>
            </orientation>
            <dictionary>{m.dictionary}</dictionary>
        </marker>
"""
    xml_content += """    </markers>
</map>"""
    with open(filename, 'w', encoding='utf-8') as f:
        f.write(xml_content)
```

У f-строк есть слабое место: они не экранируют специальные символы. Если в названии карты встретится `&` или `<`, получится файл, который ни одна программа не прочитает. Надёжнее строить XML модулем `xml.etree.ElementTree` — он сам экранирует текст.

ЗАДАНИЕ 13.2 · XML без сюрпризов

Компьютер. Назовите карту `"Склад & цех"` и сохраните её методом `save_xml`. Попробуйте прочитать файл: `xml.etree.ElementTree.parse(filename)`. Затем напишите `save_xml_etree(self, filename)`, которая строит то же дерево через `ET.Element` и `ET.SubElement` и сохраняет `ET.ElementTree(root).write(filename, encoding="utf-8", xml_declaration=True)`.

Что должно получиться: файл из f-строк не читается (`ParseError`); файл из `ElementTree` читается, а `root.find("name").text` возвращает исходное название `Склад & цех`.

13.1.3 Экспорт в KML для Google Earth

Для отображения маркеров на глобусе Google Earth используется формат KML. Главная сложность здесь — перевод локальных метров (x, y) в глобальные градусы широты и долготы. Её решает метод `local_to_gps` конструктора.

Мы используем приближенную формулу (для малых расстояний):

- 1 градус широты ≈ 111132 метра.
- 1 градус долготы $\approx 111132 \cdot \cos(\text{широта})$ метров.

Кроме того, нужно знать, куда смотрел дрон на старте. Конструктор считает, что на север: тогда X Пионера — на север (широта растёт), а Y, направленный влево, — на запад (долгота *уменьшается*). В самом файле KML координаты записываются в порядке «долгота, широта, высота».

Точность GPS

Данная формула имеет погрешность. Для профессиональной геодезии следует использовать библиотеку `pyproj` и проекцию UTM.

ЗАДАНИЕ 13.3 · Проверка пересчёта

Компьютер. Для карты с началом в точке $(59.9343;\ 30.3351)$ вызовите `local_to_gps(100, 0)` и `local_to_gps(0, 100)`. Затем откройте KML-файл в Google Earth или QGIS и линейкой программы измерьте расстояние между маркерами 0 и 9 сетки 10×10.

Что должно получиться: $(59.93520;\ 30.33510)$ и $(59.93430;\ 30.33330)$ с точностью до пятого знака; расстояние между маркерами 0 и 9 — 4,5 м с ошибкой не больше нескольких сантиметров.

13.2 Модуль визуализации

Для проверки правильности карты полезно отрисовать её на 2D-плоскости перед полетом. Библиотека `matplotlib` позволяет построить схему расположения маркеров. Функция ниже написана как метод класса `MarkerMap` (отсюда параметр `self`).

Визуализация карты

```
import matplotlib.pyplot as plt

def plot_map(self):
    x_coords = [m.x for m in self.markers]
    y_coords = [m.y for m in self.markers]
    ids = [m.id for m in self.markers]

    plt.figure(figsize=(10, 10))
    plt.scatter(x_coords, y_coords, c='red', marker='s', s=100)

    for i, txt in enumerate(ids):
        plt.annotate(f"ID {txt}", (x_coords[i], y_coords[i]), xytext=(5, 5),
        ↪  textcoords='offset points')

    plt.grid(True)
    plt.axis('equal')
    plt.title(f"Map: {self.name}")
    plt.xlabel("X (meters)")
    plt.ylabel("Y (meters)")
    plt.show()
```

ЗАДАНИЕ 13.4 · Трек полёта поверх карты

Симулятор, затем компьютер. В симуляторе выполните секторный поиск метки 12 (миссия Python № 101) и после каждой команды печатайте строку `track;x;y;z` из `get_local_position_lps()`, а при каждом чтении кадра — `seen;id1,id2` со списком прочитанных ID. Скопируйте журнал из терминала в файл `track.txt`. На компьютере постройте карту с маркером 12 и нанесите на неё трек: строки `track` — линией, места, где была прочитана метка 12, — крестиками.

Что должно получиться: рисунок, на котором линия трека проходит через три сектора с шагом 4 м по Y, а крестики стоят в том секторе, где метка лежит на карте; после находки трек идёт обратно к старту.

В симуляторе: миссии Python: 101 в панели «**Обучение**».

ГЛАВА 14

Банк практических заданий

Задания для закрепления материала сверх уроков и миссий. Они не обязательны: выбирайте по теме и уровню, а базовые упражнения по маршрутам, телеметрии и итоговым миссиям уже есть в главах 7, 8 и 9.

- **Базовый (Б):** проверка понимания основных команд.
- **Продвинутый (П):** требует написания алгоритма.
- **Комплексный (К):** мини-проект из нескольких тем.

У каждого задания указано, где его выполнять: **симулятор** (редактор Python на сайте, нижняя камера `Camera(view="bottom")`), **компьютер** (Python с полной библиотекой OpenCV) или **дрон** (настоящий Пионер или веб-камера). Строка «Опора» называет пример из папки `examples/python`, с которого удобно начать. Миссии из строки «В симуляторе» открываются в панели «**Обучение**»; сцену, которую готовит миссия, можно использовать и для своей программы.

> **Чего нет в симуляторе**
>
> Браузерный `cv2` — учебная модель: есть кадр камеры, детектор ArUco, `solvePnP`, рисование, `imshow`, цветовые маски (`inRange`), контуры и моменты. Нет `imwrite` (возвращает `True`, но ничего не сохраняет), `absdiff`, `threshold`, размытия, распознавания лиц и MediaPipe. В `pioneer_sdk` симулятора нет углов крена и тангажа и высоты по барометру: есть положение LPS, курс (`get_yaw()`, только версия SDK с GitFlic), дальномер, батарея и оптический поток. Задания, которым это нужно, помечены «дрон» или «компьютер».

14.1 Блок 1: Работа с камерой и ArUco

ЗАДАНИЕ 14.1 · (Б) Фотограф

Дрон. Напишите скрипт, который делает фото каждые 5 секунд и сохраняет их в папку, в названии которой — дата и время запуска (`time.strftime("%Y-%m-%d_%H-%M-%S")`).

Что должно получиться: за минуту работы в новой папке 12 файлов JPG с возрастающими номерами; ни один файл не пустой (`cv2.imread` читает каждый).
Опора: `camera_examples/take_photo.py`, `camera_examples/get_frames_from_camera.py`.

ЗАДАНИЕ 14.2 · (Б) Детектор движения

Дрон или веб-камера, компьютер. Сравнивайте два последовательных серых кадра (`cv2.absdiff`, затем `cv2.threshold`). Если количество изменившихся пикселей превышает порог — выводите сообщение «Движение обнаружено!».

Что должно получиться: на неподвижной сцене сообщений нет 10 секунд подряд; взмах рукой перед камерой даёт сообщение в течение секунды. Порог подобран так, что мерцание ламп его не превышает.
Опора: `camera_examples/camera_stream.py`.

ЗАДАНИЕ 14.3 · (П) Умный светофор

Дрон; логику маски можно отладить и в симуляторе на цветных предметах сцены. Распечатайте красный и зеленый круги. Если дрон видит зеленый цвет — он взлетает на 1 метр. Если красный — садится. Цвет ищите маской в пространстве HSV (`cv2.inRange`) и считайте долю пикселей маски.

Что должно получиться: зелёный круг перед камерой — взлёт, красный — посадка; решение принимается по трём кадрам подряд, поэтому случайный кадр с цветной одеждой не поднимает дрон.
Опора: `camera_examples/camera_stream.py`, `mission_examples/PiZero_test.py`.

ЗАДАНИЕ 14.4 · (П) Дистанция

Дрон. Дрон висит напротив маркера. Если маркер приближается (человек подходит с маркером в руках) — дрон отлетает назад, сохраняя дистанцию 1 метр. Расстояние берите из `tvec` (нужна калибровка камеры).

Что должно получиться: при медленном (до 0,3 м/с) приближении маркера расстояние держится в пределах 1 ± 0.2 м; если маркер пропал из кадра, дрон останавливается, а не продолжает движение.
Опора: `aruco_examples/12_follow_aruco.py`, `aruco_examples/11_aruco_basics_example2_coordinates.py`.

ЗАДАНИЕ 14.5 · (К) Инвентаризация

Симулятор, затем дрон. Облетите заданный список точек осмотра, в каждой прочитайте ID в свежем кадре и составьте отчёт «ID 12 найден в точке (0; 8) в 12:00:05» (время — `time.strftime("%H:%M:%S")`). В симуляторе используйте промышленный коридор миссии № 105 (метки 12 и 13) или карту арены миссии № 104; на дроне разложите в комнате 5 маркеров с ID от 0 до 4 и допишите запись отчёта в файл.

Что должно получиться: в отчёте каждый ID встречается один раз и привязан к точке, в которой он действительно прочитан; ID, которых нет на месте, отчёт называет «не найден». Порядок строк совпадает с порядком облёта.

В симуляторе: миссии Python: 104–105 в панели «**Обучение**».

14.2 Блок 2: Телеметрия и безопасность

ЗАДАНИЕ 14.6 · (П) Высотомер

Симулятор. Взлетите на $Z = 9$ над свободной улицей спального района (сцена миссии № 103) и пролетите над крышей дома в точке $(9; -7)$. Печатайте раз в 0,2 с высоту LPS (z) и дальномер. В каких местах они отличаются? На дроне сравните LPS-высоту и дальномер над столом.

Что должно получиться: над улицей значения почти равны, над крышей дальномер показывает меньше на высоту дома (больше 5 м), а z не меняется. Вывод: дальномер измеряет расстояние до поверхности под дроном, а не высоту над стартом.

В симуляторе: миссии Python: 42, 103 в панели «**Обучение**».
Опора: `get_telemetry/GET_dist_sensor_data.py`.

ЗАДАНИЕ 14.7 · (П) Компас

Симулятор; на дроне — только с версией SDK, где есть ***`get_yaw()`***. Напишите программу, которая разворачивает дрон по курсу 0°, 90°, 180° и 270° (в радианах: `math.radians`), не меняя положения, и после каждого поворота печатает `get_yaw()`. Курс отсчитывается в локальной системе, а не от севера.

Что должно получиться: четыре строки, в которых измеренный курс отличается от заданного меньше чем на 0,05 рад (с учётом того, что $3\pi/2$ и $-\pi/2$ — один и тот же курс); положение по x, y за время разворотов меняется не больше чем на 0,1 м.

В симуляторе: миссии Python: 45 в панели «**Обучение**».

ЗАДАНИЕ 14.8 · (К) Виртуальная стена

Симулятор, затем дрон. Задайте границы зоны полета (X и Y от -2 до 2 м). Двигайте дрон командой `set_manual_speed` со скоростью 0,3 м/с наружу; если телеметрия показывает выход за границы, дрон должен остановиться и вернуться в центр.

Что должно получиться: дрон останавливается не дальше 0,2 м за границей и возвращается в $(0; 0)$; программа печатает причину остановки («граница X»), а скорость ограничена 0,3 м/с при любой ошибке положения.

В симуляторе: миссии Python: 68–71 в панели «**Обучение**».
Опора: `mission_examples/set_manual_speed.py`.

14.3 Блок 3: Миссии и полет по точкам

ЗАДАНИЕ 14.9 · (Б) Лестница

Симулятор или дрон. Пролетите вперед на 1 метр, набрав 0.5 метра высоты. Повторите 3 раза (как по ступенькам). Точки вычисляйте в цикле, а не перечисляйте вручную.

Что должно получиться: точки $(1; 0; 1.5)$, $(2; 0; 2.0)$, $(3; 0; 2.5)$ при старте с высоты 1 м; финальная высота 2,5 м по LPS с ошибкой меньше 0,1 м.

В симуляторе: миссии Python: 50 в панели «**Обучение**».
Опора: `03_flight_control/10_01_forward_back.py`.

ЗАДАНИЕ 14.10 · (П) Спираль

Симулятор или дрон. Напишите генератор точек для полета по спирали (расширяющийся круг с набором высоты): радиус растёт от 0,5 до 1,5 м, высота — от 1 до 2 м за два витка, 12 точек на виток.

Что должно получиться: 24 точки; расстояние от каждой точки до оси $(0; 0)$ растёт, высота растёт монотонно; дрон проходит их по порядку и садится.
Опора: `03_flight_control/10_03_circle_math.py`.

ЗАДАНИЕ 14.11 · (П) Патруль

Симулятор. Дрон летает между двумя точками $(0; 0; 3)$ и $(0; 8; 3)$, пока в кадре нижней камеры не появится метка 12, но не больше пяти проходов. На дроне вместо метки можно взять препятствие по дальномеру.

Что должно получиться: после первого кадра с меткой 12 патруль прекращается и дрон садится; если метки нет, программа печатает «Патруль завершён» после пяти проходов. Бесконечного цикла нет.

В симуляторе: миссии Python: 35, 102 в панели «**Обучение**».

ЗАДАНИЕ 14.12 · (К) Сканирование поля

Симулятор. Реализуйте полет «змейкой» над зоной 8×8 м на высоте 3 м так, чтобы кадры нижней камеры перекрывали всю зону (с $Z = 3$ камера видит по Y около ± 2 м). В каждой точке снимайте кадр и печатайте все прочитанные ID. На дроне с той же логикой получают фотоплан.

Что должно получиться: маршрут из параллельных проходов с шагом не больше 4 м; итоговый список ID без повторов; программа печатает число точек съёмки и время облёта.

В симуляторе: миссии Python: 101 в панели «**Обучение**».
Опора: `mission_examples/mission_with_frames.py`.

14.4 Блок 4: Групповой полет

ЗАДАНИЕ 14.13 · (Б) Синхронный старт

Настоящие дроны; в симуляторе — на сцене с двумя дронами (см. врезку в разделе 8.11). Два дрона взлетают одновременно по команде с ПК, висят 5 секунд и садятся. В симуляторе добавьте второй дрон в список дронов и возьмите его адрес из настроек подключения.

Что должно получиться: разница между командами взлёта двух дронов меньше 0,5 с; оба дрона садятся, и программа закрывает оба соединения.
Опора: `group_flight_examples/swarm_OPT.py`.

ЗАДАНИЕ 14.14 · (П) Карусель

Настоящие дроны; в симуляторе — на сцене с двумя дронами (см. врезку в разделе 8.11). Два дрона висят напротив друг друга и меняются местами, пролетая по дуге (чтобы не столкнуться).

Что должно получиться: в течение всего манёвра расстояние между дронами по телеметрии не меньше 1 м; каждый дрон заканчивает в стартовой точке другого.
Опора: `group_flight_examples/swarm_circle_IRNAV.py`.

ЗАДАНИЕ 14.15 · (П) Ведущий-ведомый

Настоящие дроны; в симуляторе — на сцене с двумя дронами (см. врезку в разделе 8.11). Первый дрон летит по программе (квадрат). Второй дрон получает телеметрию первого и повторяет его движения со смещением (на 1 метр сзади).

Что должно получиться: после каждой вершины квадрата ведомый находится в 1 м позади ведущего с ошибкой меньше 0,3 м; если телеметрия ведущего не пришла (`None`), ведомый висит на месте.
Опора: `group_flight_examples/swarm_OPT.py`.

ЗАДАНИЕ 14.16 · (К) Шоу дронов

Настоящие дроны или сцена с четырьмя дронами. Составьте хореографию для 4 дронов под музыку (используйте мигание светодиодами в такт).

Что должно получиться: сценарий записан таблицей «время — точка — цвет» для каждого дрона; ни в один момент два дрона не ближе 1 м друг к другу; смена цвета попадает в такт с ошибкой меньше 0,2 с.
Опора: `group_flight_examples/swarm_OPT.py`, глава о групповом полёте.

14.5 Блок 5: Трекинг объектов

Распознавание людей и лиц в симуляторе не моделируется (MediaPipe в браузере недоступен): задания блока выполняются с настоящим дроном или с веб-камерой компью-

тера.

ЗАДАНИЕ 14.17 · (Б) Приветствие

Дрон. Если дрон видит лицо человека — он мигает зеленым и делает «кивок» (короткое движение вперёд-назад).

Что должно получиться: кивок выполняется один раз на появление лица, а не на каждом кадре; без лица в кадре дрон неподвижен.
Опора: `PioneerHumanTracking/human-tracking.py`.

ЗАДАНИЕ 14.18 · (П) Следи за цветом

Дрон. Возьмите яркий оранжевый мяч. Дрон должен поворачиваться (yaw) так, чтобы мяч всегда был в центре кадра.

Что должно получиться: при медленном перемещении мяча центр пятна остаётся в пределах 10% ширины кадра от центра; если мяч пропал, вращение прекращается.
Опора: `camera_examples/camera_stream.py`.

ЗАДАНИЕ 14.19 · (П) Держи дистанцию

Дрон. Дрон видит лицо. Если лицо занимает слишком много места в кадре (человек близко) — отлетаем. Слишком мало (человек далеко) — подлетаем.

Что должно получиться: в установившемся режиме высота рамки лица держится в заданном диапазоне; скорость ограничена (не больше 0,3 м/с) и обнуляется в мёртвой зоне.
Опора: `PioneerHumanTracking/human-tracking.py`.

ЗАДАНИЕ 14.20 · (К) Оператор

Дрон. Дрон летит боком параллельно идущему человеку, снимая его профиль (требует сложной координации движений).

Что должно получиться: человек всё время в кадре, а боковая скорость дрона следует за его шагом; при потере человека дрон останавливается и висит.
Опора: `PioneerHumanTracking/human-tracking.py`.

14.6 Самостоятельные миссии симулятора

Девять заключительных миссий панели «**Обучение**» (№ 100–108: карта из меток, поиск по секторам, экспедиции, потеря цели, итоговая экспедиция) разобраны в главе 9 со всеми критериями проверки, поэтому здесь не повторяются.

14.7 Критерии оценки выполнения

Задание считается выполненным, если:

1. Код запускается без синтаксических ошибок.
2. Выполнено условие из строки «Что должно получиться»; в симуляторе — на двух разных seed полигона или после сброса сцены, на дроне — в реальных условиях не менее 3 раз подряд.
3. Предусмотрены неуспешные исходы: метка не найдена, кадр не получен (`None`), точка не достигнута за отведённое время. Программа сообщает причину и сажает дрон, а не висит бесконечно.
4. Код содержит комментарии, объясняющие логику работы.
5. Соблюдены меры безопасности (дрон не вылетает за пределы зоны, экстренная посадка работает).

ГЛАВА 15

Типичные ошибки и их устранение

В процессе работы с дроном часто возникают ситуации, когда «код правильный, но ничего не работает». В этой главе собраны самые частые проблемы.

15.1 Проблемы с камерой и маркерами

15.1.1 Дрон не видит маркер

- **Причина 1: Плохое освещение.** Камере темно, выдержка увеличивается, появляется «смаз» (Motion Blur) при движении.
- **Решение:** Включите дополнительный свет. Маркер должен быть освещен равномерно.
- **Причина 2: Блики.** Маркер заламинирован глянцевой пленкой или лежит под лампой.
- **Решение:** Переместите маркер или используйте матовый скотч.
- **Причина 3: Неверный размер.** В коде указан размер 0.2 метра, а напечатан маркер 18 см.
- **Решение:** Измерьте черную рамку линейкой и исправьте константу в коде.
- **Причина 4: Quiet Zone.** Маркер вырезан по черному контуру без белого поля.
- **Решение:** Наклейте его на белый лист бумаги.

15.1.2 Дрон летит не туда

- **Причина 1: Перепутаны оси.** Вы думаете, что X — это вправо, а это вперед.
- **Решение:** Проверьте ориентацию (Body Frame). Ось X дрона смотрит вперед — туда же, куда нос и передняя камера; Y — влево, Z — вверх. Не путайте с системой самой камеры в OpenCV: там вперед «из объектива» смотрит ось Z (вектор `tvec`). И с командами `..._body_fixed` в `pioneer_sdk`: в них x — вправо, y — вперёд.
- **Причина 1a: Перепутан знак поправки.** Дрон отлетает от метки вместо того, чтобы приближаться к ней.
- **Решение:** Поменяйте знак коэффициента по этой оси и проверьте опытом: после короткого импульса скорости ошибка должна уменьшиться (миссия Python № 92).
- **Причина 2: Ошибка калибровки.** Камера «рыбий глаз» сильно искажает картинку по краям.
- **Решение:** Проведите калибровку камеры заново.
- **Причина 3: Yaw (Рысканье).** Дрон повернулся на 90 градусов, но код продолжает отправлять команды в старой системе координат.

- **Решение:** Используйте глобальную систему координат (World Frame) или учитывайте текущий угол Yaw.

ЗАДАНИЕ 15.1 · Неверный знак

Симулятор. В контроллере `align` из `staged_landing_sim.py` поменяйте знак поправки по Y (`vy = clamp(KP * ey / h, V_MAX)`) и запустите программу над меткой 12. Опишите, что видно в журнале, и верните знак.

Что должно получиться: с неверным знаком $|e_y|$ растёт от кадра к кадру, пока метка не уходит из кадра; контроллер обнуляет скорость, за 120 попыток не достигает центра, и программа печатает «Прервано» и садится, а не улетает дальше.

В симуляторе: миссии Python: 92, 96 в панели «**Обучение**».

15.2 Проблемы с подключением и Wi-Fi

15.2.1 Connection Refused / Timeout

- **Причина 1:** Компьютер подключился к другой Wi-Fi сети (с интернетом).
- **Решение:** Проверьте, что вы подключены к точке доступа Pioneer_XXXX.
- **Причина 2:** Брандмауэр Windows блокирует порты `pioneer_sdk`: 8001 (команды MAVLink, UDP) и 8888 (видео с камеры).
- **Решение:** Разрешите Python доступ в частные сети.

15.3 Ошибки в коде Python

15.3.1 IndentationError

Python очень чувствителен к отступам. Не смешивайте табы и пробелы. Используйте 4 пробела для отступа.

15.3.2 OpenCV Error: Assertion failed

Обычно означает, что функция получила пустое изображение (None).

- Проверьте, что `camera.get_cv_frame()` не вернул None.
- Проверьте, что файл изображения существует (если читаете с диска).

15.3.3 ValueError: The truth value of an array ... is ambiguous

Детектор возвращает `ids` массивом numpy или `None`. Запись `if ids:` для массива из нескольких элементов падает с этой ошибкой. Проверяйте явно: `if ids is not None:`, а номер берите как `int(ids[i][0])`.

ЗАДАНИЕ 15.2 · Найдите три ошибки

Симулятор. Программа должна напечатать номера всех меток в кадре, но падает или печатает не то:

```
frame = camera.get_cv_frame()
corners, ids, _ = detector.detectMarkers(frame)
if ids:
    for i in range(len(ids) + 1):
        print("ID:", ids[i])
```

Исправьте её и проверьте над меткой 12 и вдали от меток (у $Y = 5$, как в миссии Python № 85).

Что должно получиться: над меткой печатается `ID: 12` (число, а не `[12]`), вдали от меток — «меток нет», и программа не падает; кадр `None` тоже обработан до вызова детектора.

В симуляторе: миссии Python: 81, 84–85 в панели «**Обучение**».

15.4 Особенности симулятора

Программа, которая работает в симуляторе, почти без изменений летит на настоящем дроне — но есть отличия, о которых стоит помнить в обе стороны.

- **`Camera(view="bottom")`** — расширение симулятора. Настоящий `pioneer_sdk` не знает параметра `view`: на дроне уберите его и подключайте камеру, которая установлена на аппарате.
- **Модель OpenCV.** В браузере `cv2` умеет получать и показывать кадр, рисовать и искать ArUco. `solvePnP`, цветовые маски (`inRange`) и контуры в нём есть, но `cv2.imwrite` ничего не сохраняет, а функций вроде `threshold`, `absdiff`, `adaptiveThreshold`, `undistort` или калибровки нет — обращение к ним даёт `AttributeError`. Такие программы запускайте на компьютере.
- **Словарь.** Симулятор сравнивает словарь детектора со словарём меток по имени. Метки миссий нарисованы из `DICT_6X6_250`; детектор с другим словарём (даже `DICT_6X6_50`) вернёт `ids = None`.
- **Метка «не видна».** Модель находит метку, только если она целиком в поле зрения камеры и ничем не закрыта. Метка у края кадра детектором не возвращается — это нормальный исход (миссии № 79 и 85).
- **Устаревший кадр.** Перед решением берите новый кадр: после `go_to_local_point` дождитесь `point_reached()` и дайте корпусу успокоиться (около 1,5 с), иначе кадр снят ещё в движении.
- **Нет данных.** Виртуальный дрон всегда отдаёт положение и батарею, а дальномер над землёй показывает `0`, а не `None`. На настоящем дроне `None` встречается, поэтому проверку оставляют.
- **`set_manual_speed`** действует около 0,5 с: команду повторяют в цикле, а в конце явно отправляют нулевую скорость.

15.5 Аварийные ситуации

15.5.1 Дрон не реагирует на Land

Если программная посадка не срабатывает:

1. Не паникуйте.
2. Остановите программу и выключите моторы: тумблер взвода на пульте (раздел D) или кнопка Disarm в наземной станции (если есть).
3. Только если моторы не выключаются: аккуратно поймайте дрон снизу за аккумулятор (берегите пальцы от пропеллеров!) и отключите аккумулятор. Не рассчитывайте, что моторы остановятся сами, если перевернуть дрон, — проверьте, есть ли такая защита в прошивке вашего аппарата.

> **Внимание!**
>
> Никогда не пытайтесь схватить дрон сверху или сбоку — пропеллеры вращаются со скоростью свыше 10 000 об/мин и могут нанести серьезные травмы.

ЧАСТЬ IV

Пионер Мини 2: Python на борту, нейросети и Bricks

ГЛАВА 16

Pioneer SDK 2: программа на борту дрона

Цели главы

После главы вы сможете:

- объяснить, чем программа на борту Пионера Мини 2 отличается от программы на компьютере, и запустить Python-программу с библиотекой `pioneer_sdk2`;
- безопасно провести дрон через весь цикл «моторы → взлёт → задание → посадка» и объяснить, почему SDK 2 отвергает команды не по порядку;
- лететь по точкам (локальным и от дрона) и по скоростям, задавать курс и возвращаться домой, а о событиях автопилота узнавать подпиской;
- читать датчики, навигацию и параметры автопилота, управлять подсветкой, захватом и виртуальным пультом;
- показывать видео в браузере, наклонять камеру, писать видео и использовать кадры в контуре управления;
- найти любой метод в справочнике и отличить то, что есть у Мини 2, от того, что есть только у Базового.

Нужно знать: основы Python (главы части II) и полёт по точкам с `pioneer_sdk` из главы 7.

Миссии симулятора к главе

В режиме «**Обучение**» есть трек ««**Мини 2 · SDK2**»»: его первые 37 миссий (13 глав трека) с проверкой полёта идут в порядке этой главы (таблица соответствия — в конце главы); миссии 38–64 относятся к части V (ROS 2 и рой). Подготовкой могут служить миссии Python 28–36 «Время и события» и 46–54 «Маршрут по этапам». Программы главы запускаются на модели Мини 2: на панели «**Рой**» нажмите «**Мини 2**». Что в симуляторе упрощено, сказано в тексте и в справочнике.

На какую версию библиотеки опирается глава

Основа главы — сборник примеров к `pioneer_sdk2` 0.16.0, проверенный на Пионере Мини 2 (Pioneer OS 1.0.0, Python 3.12). Там, где сборник молчит, подробности взяты из исходников `pioneer_sdk2` 0.11.0 (единственная версия на PyPI) и из справочника симулятора; такие места помечены «**по 0.11.0**»: на 0.16.0 они не проверены и могут отличаться. Версию на своём дроне узнайте командой `pip3 list | grep pioneer`.

> Полётные программы сборника на дроне авторами *не запускались*: проверены подключение, телеметрия, камера и окна просмотра. Поэтому любую полётную программу запускайте сначала в симуляторе, затем на открытой площадке без людей.

16.1 Программа переезжает на дрон

В главе 7 Python-программа работала на компьютере и отдавала команды по Wi-Fi. У Пионера Мини 2 и у модулей с микрокомпьютером Radxa Zero 3W есть собственный компьютер с операционной системой Pioneer OS, и программа может работать прямо на нём:

- полёт не зависит от Wi-Fi: пропала связь с ноутбуком — программа продолжает работать;
- кадры камеры приходят без сетевой задержки, а рядом есть нейропроцессор (глава 17);
- ноутбук нужен только чтобы записать программу и запустить её.

Для бортовых программ есть библиотека `pioneer_sdk2`. В образ Pioneer OS для Мини 2 и Radxa Zero 3W она уже встроена; на модуле Raspberry Pi Zero её ставят командой `pip install pioneer-sdk2` (нужен Python 3.12). По описанию пакета она работает и на Windows 10/11, Linux и macOS с процессором Apple Silicon: на компьютере `Pioneer` подключается к автопилоту по последовательному порту или по TCP, а камера берёт кадры по RTSP.

Рабочее место: Pioneer Code

На борту Мини 2 работает веб-платформа **Pioneer Code**. Откройте в браузере `http://10.42.0.1:9090` (адрес дрона в режиме точки доступа; если дрон подключён к вашей сети — его адрес в ней). Разделы живут на своих портах:

Раздел	Порт	Назначение
Главное меню	9090	переход к разделам, версии компонентов
Среда разработки	9999	редактор кода (как VS Code) с терминалом; рабочая папка `workspace`
Модели ИИ	7777	регистр нейросетей для `pioneer_rknn` (глава 17)
Галерея медиа	5000	фото и видео, снятые камерой (`RecorderControl`)
Pioneer Bricks	2020	программирование блоками (глава 18)

Удобнее всего писать программы в среде разработки: слева дерево файлов, справа редактор, снизу терминал. Скопируйте программы главы в `workspace`, откройте нужную и запустите в терминале командой `python3 имя.py`. Второй способ — с компьютера, подключённого к Wi-Fi дрона (имя сети и пароль указаны в инструкции к вашему дрону):

Терминал компьютера

```
scp sdk2_01_connect.py <пользователь>@10.42.0.1:~/
ssh <пользователь>@10.42.0.1
python3 sdk2_01_connect.py
```

Остановить программу — `Ctrl` + `C`.

Безопасность прежде всего

- Автопилот проверяет «наличие пилота»: включите пульт и переведите тумблер `SWB` в нижнее положение. Проверку можно отключить параметром `Copter_flyWithoutRc = 1` (например, в Pioneer Station), но тогда аварийно перехватить управление с пульта нельзя.
- Первые запуски полётных программ проводите в сетке или на открытой площадке без людей.
- Каждая полётная программа главы завершается блоком `finally`, который сажает дрон. Не удаляйте его, когда переделываете примеры.
- Держите руку на `Ctrl` + `C`: прерывание тоже ведёт в `finally` и к посадке.

Пионер Мини 2 в симуляторе

На панели «**Рой**» нажмите «**Мини 2**» — рядом с Базовым появится второй дрон. Пока он выбран, редактор работает только на Python: Lua и Blockly у Мини 2 нет, справочник и автодополнение показывают команды `pioneer_sdk2`, а в списке примеров — программы этой главы. Импорт `pioneer_sdk` для Мини 2 (и `pioneer_sdk2` для Базового) симулятор остановит с подсказкой.

Чем симулятор отличается от дрона

Программа выполняется в браузере, а не на борту. Порядок команд, события, `interval`, цвет 0..1 и сообщения `logger` повторяют настоящую библиотеку; отличия такие:

- `ImageViewer` открывает окно симулятора, а не страницу `10.42.0.1:8889`; аргументы `serial` и `tcp` конструктора не используются;
- у Мини 2 два светодиода — передние индикаторы (0 — левый, 1 — правый); захват `grab_close()` подхватывает груз под дроном, как магнит Базового;
- навигация всегда LPS: GPS, глобальные точки и магнитный захват `cargo_*` не работают (методы пишут пояснение и возвращают `False` или `None`); `reboot_board()` запрещена, как на настоящем Мини 2;
- `get_param` и `set_param` знают те же 232 параметра, что автопилот Мини 2 (см. раздел «Параметры»); неизвестные имена возвращают `None` и `False`, а значения живут до конца программы;
- `RecorderControl` проверяет аргументы и печатает «сделал бы», но файлов не создаёт;
- вызов `land()` на земле сразу возвращается с предупреждением, а на дроне про-

> грамма зависнет.
>
> В справочнике в конце главы у каждого метода есть столбец «Симулятор».

Чем SDK 2 отличается от SDK

Большая часть команд называется так же. Отличия, на которых чаще всего ошибаются:

	pioneer_sdk (Базовый)	**pioneer_sdk2 (Мини 2)**
Где работает программа	на компьютере, по Wi-Fi	на борту (или на компьютере по serial/TCP)
Языки дрона	Lua, Python, Blockly	Python и блоки Pioneer Bricks; Lua и Blockly нет
Импорт	`from pioneer_sdk import Pioneer`	`from pioneer_sdk2 import Pioneer`
`arm`, `takeoff`, `land`	возвращаются сразу, ждать — `time.sleep`	ждут события автопилота: моторы запущены, взлёт завершён, дрон сел
Порядок команд	не проверяется	проверяется: взлёт без `arm` — ошибка
Курс `yaw` в точке	радианы, обязателен	можно не указывать; единицы — см. врезку в разделе «Системы координат»
Команда скорости	повторять каждые 0.05 с	одна команда держит скорость `interval` секунд
Цвет подсветки	0..255	**0..1**
Светодиоды	четыре и матрица 5×5	число зависит от платы (в симуляторе — два)
События	только опрос `point_reached()`	опрос и подписка `subscribe`
Камера	`Camera()`, окно `cv2.imshow`	`Camera(CameraType.MAIN)`, видео в браузере через `ImageViewer`
Груз	магнит `cargo_*`	механический захват `grab_*`
Только у Мини 2	—	`ServoCamera`, `RecorderControl`, модуль Ranger, `pioneer_rknn`

ЗАДАНИЕ 16.1 · Мини 2 на сцене

Добавьте в симуляторе Пионер Мини 2, выберите его и запустите программу из одной строки `from pioneer_sdk import Pioneer`. Затем исправьте импорт на `pioneer_sdk2` и напечатайте `Pioneer().get_battery_status()`.

Что должно получиться: первый запуск симулятор останавливает с подсказкой про `pioneer_sdk2`; второй печатает пару (напряжение в вольтах, температура).

16.2 Что входит в библиотеку

Всё, что вы используете, импортируется из пакета `pioneer_sdk2`:

Имя	Что это
`Pioneer`	главный класс: подключение к автопилоту и все команды полёта, датчиков, груза, параметров, пульта
`Event`	перечисление событий автопилота для `subscribe`
`Camera, CameraType`	захват кадров камеры; тип камеры (`MAIN` — передняя, `OPT` — нижняя)
`ImageViewer`	показ кадров видеопотоком в браузере (только драйвер камеры gstreamer, то есть бортовые платы)
`ServoCamera, ServoPriority`	сервопривод наклона камеры Мини 2 и приоритет его команд
`RecorderControl`	запись видео и фото в галерею дрона (Мини 2)

Три перечисления лежат глубже и импортируются из подмодуля: `from pioneer_sdk2.piosdk2 import FlyState, NavSystem, NavStatus`. `FlyState` (`ON_LAND`, `ARMED`, `IN_SKY`) описывает этап полёта, `NavSystem` (`GPS`, `LPS`, `OPT`) — вид навигации, `NavStatus` (`NO_DATA`, `CANNOT`, `LOW`, `OK`) — доверие к оценке позиции. Значения перечислений сравнивают по имени (`state.name == "IN_SKY"`): так программа работает и на дроне, и в симуляторе.

> **Набор классов зависит от платы**
>
> Настройки платы лежат в файле `board_config.json` внутри пакета, и по ним библиотека решает, что у вас есть: `Camera` и `CameraType` — где есть камера; `ImageViewer` — при драйвере gstreamer (Мини 2, Radxa Zero 3W); `ServoCamera`, `RecorderControl` и `grab_*` — Мини 2; `cargo_*` — Базовый Пионер; `get_ranger_data` — плата с модулем Ranger. Вызов метода, которого плата не поддерживает, не падает: метод печатает пояснение и возвращает `False` или `None`. Проверяйте результат.

16.3 Что внутри борта

Цифры этого раздела сняты с настоящих бортов (Мини 2 и Базовый Пионер с модулем Radxa Zero 3W, образ Pioneer OS на Ubuntu 24.04, версии: `pioneer_sdk2` 0.16.0, `pioneer_dds` 0.4.0, `pioneer_swarm` 0.1.3, `plazlink-server-rs` 0.2.1, Pioneer Bricks 2.3.0).

Цепочка связи. Автопилот отдаёт данные по UART на скорости 115200 (у Мини 2 порт `/dev/ttyS3`, у Базового с Radxa — `/dev/ttyS4`). Их принимает служба `plazlink-server-rs` (в системе она называется `plazlink-core`) и открывает TCP-порт `20556`. Библиотека `pioneer_sdk2` при вызове `Pioneer()` подключается именно к

`127.0.0.1:20556`, поэтому `tcp="127.0.0.1:20556"` в конструкторе — не случайное число. Через эту же службу работают Pioneer Bricks, служба роя `swarm-server` и программы с ROS 2: автопилот один, а клиентов к нему подключается несколько.

Звено	Что делает
Автопилот	управляет моторами, хранит 232 параметра, отдаёт поля телеметрии
UART 115200	проводная связь автопилота с бортовым компьютером
`plazlink-server-rs` (TCP 20556)	протокол Plazlink: даёт нескольким программам доступ к одному автопилоту
`pioneer_sdk2`	удобный Python-интерфейс: `arm`, `takeoff`, `get_*`
`swarm-server`, Bricks, ROS 2	публикуют данные и принимают команды по сети

Настройки платы. Файл `board_config.json` внутри пакета `pioneer_sdk2` решает, какие модули доступны программе. У двух бортов он разный:

	Мини 2	Базовый с Radxa Zero 3W
Перезагрузка платы `reboot_board()`	запрещена	разрешена
Команда `go_to_local_point`	версия 2: курс и точка одной командой	версия 1: сначала `set_yaw`, потом точка; у `go_to_local_point_body_fixed` смещение берётся в осях сцены от текущей позиции
Ranger (`get_ranger_data`)	есть	нет: `None` и сообщение «Модуль Ranger не поддерживается»
Захват (`grab_*`)	есть	нет: `False` и сообщение «Модуль захвата не поддерживается»
Магнитный захват (`cargo_*`)	нет	есть
Сервопривод камеры	есть, $-80 \dots 30°$	нет
Камеры	основная 1080×720 при 30 к/с для программы (запись до 3840×2160), нижняя 1280×800 при 60 к/с	только основная 1080×720 при 30 к/с, запись выключена
Навигация по умолчанию	оптическая (`Flight_com_navSystem` = 2)	оптическая

Симулятор повторяет эти различия: у Базового с модулем Radxa Zero 3W (язык «Python · SDK2») он отключает Ranger и `grab_*`, включает `reboot_board()` и ведёт перелёт по версии 1; у Мини 2 перезагрузка запрещена, а Ranger и захват доступны.

Службы и порты Мини 2. Кроме разделов Pioneer Code из таблицы выше, на борту работают: `plazlink-core` (TCP 20556), потоковый сервер камеры (RTSP 8554, WebRTC 8889, управление 9997), gRPC-служба борта (50051), сервис записей (5000) и служба роя `swarm-server`.

> **Что такое DDS и при чём тут рой**
>
> **DDS** (Data Distribution Service) — стандарт обмена данными «издатель — подписчик» без центрального сервера; на нём построен ROS 2. Данные ходят по *топикам* (имя и тип сообщения), участники сами находят друг друга в сети, а команды идут *сервисами* (запрос — ответ). На борту работает реализация Fast DDS 2.14, а библиотека `pioneer_dds` даёт Python-обёртки `Publisher`, `Subscriber` и `Service`. Служба `swarm-server` читает данные через `pioneer_sdk2` и публикует их в DDS по описанию типа дрона: позицию, скорость и курс (`navigation/local/position`, `velocity`, `yaw`, по 10 Гц), IMU, высоту и батарею (`sensors/imu`, `altitude`, `battery_state`), а также события полёта (`flight/callback_event`). Команды `flight/arm`, `disarm`, `takeoff`, `land`, `go_to_local_point`, `go_to_global_point`, `set_yaw`, `payload/grab` и `payload/led/set` служба принимает как сервисы и выполняет через SDK. Каждый борт регулярно объявляет себя сообщением `Heartbeat` из пакета `swarm_msgs`: по нему рой узнаёт, кто в сети. Подробности — в части «ROS 2 и рой на Пионере».

16.4 Подключение и жизненный цикл

Конструктор

```
Pioneer(serial=None, tcp="127.0.0.1:20556",
        baudrate=57600, wait_callback=True,
        safety_command=True, logger=True)
```

Аргумент	Смысл
`serial`	имя последовательного порта (например, `COM3`); если задан, подключение идёт по нему
`tcp`	адрес и порт автопилота `"хост:порт"`; по умолчанию `127.0.0.1:20556` — адрес на борту (без порта берётся 5500, по 0.11.0)
`baudrate`	скорость последовательного порта, по умолчанию 57600
`wait_callback`	`True` — `arm`, `takeoff`, `land` ждут события автопилота
`safety_command`	`True` — включён контроль порядка команд (см. ниже)
`logger`	`True` — библиотека печатает ход команд: `-> ARM`, `Моторы запущены`

Используйте что-то одно: `serial` или `tcp`. Если TCP-соединение открыть не удалось, библиотека печатает «Не получилось подключиться по TCP» и завершает программу. На модуле Raspberry Pi Zero адрес автопилота нужно указать: `Pioneer(tcp="10.42.0.1:20556")`, если вы подключаетесь к модулю, или адрес из

свойств системы, если модуль сам подключается к вашей точке доступа. На Мини 2 и Radxa Zero 3W адрес по умолчанию уже верный.

python-onboard/pioneer_sdk2/sdk2_01_connect.py

```python
from pioneer_sdk2 import Pioneer
import time

# На борту дрона адрес по умолчанию верный: TCP 127.0.0.1:20556.
pioneer = Pioneer()

try:
    # (напряжение, температура)
    battery = pioneer.get_battery_status()
    if battery is not None:
        voltage, temperature = battery
        print(f"Батарея: {voltage:.2f} В, {temperature} °C")

    # (x, y, z), метры
    position = pioneer.get_local_position_lps()
    print("Позиция:", position)

    # Цвет в SDK 2 — доли от 0 до 1, а не числа 0..255.
    # Все светодиоды — зелёные
    pioneer.led_control(r=0, g=1, b=0)
    time.sleep(2)
    # нулевой — оранжевый
    pioneer.led_control(led_id=0, r=1, g=0.5, b=0)
    time.sleep(2)
    pioneer.led_control()                    # Все погасить
finally:
    pioneer.close_connection()
```

Что происходит. `Pioneer()` подключается к автопилоту и сразу запускает фоновый поток слежения за событиями. `get_battery_status()` возвращает пару (напряжение в вольтах, температура), `get_local_position_lps()` — тройку координат в метрах; если данных нет, команды возвращают `None`, поэтому результат проверяют перед распаковкой. Цвет подсветки — доли от 0 до 1 (подробнее — в разделе «Подсветка»): `g=1` — зелёный на полную яркость, `g=0.5` — на половину. `led_control()` без аргументов гасит все светодиоды.

ЗАДАНИЕ 16.2 · Телеметрия Мини 2

Раз в секунду пять раз печатайте напряжение батареи и координаты `get_local_position_lps()` с проверкой на `None`. На время опроса включите левый светодиод зелёным (`g=1`), правый — оранжевым на половине яркости, в конце погасите оба.

Что должно получиться: пять строк телеметрии без ошибок распаковки; оба передних индикатора светятся во время опроса и гаснут в конце.

В симуляторе: миссии Мини 2 SDK2: 1 в панели «**Обучение**».

Закрыть, заглушить, перезагрузить

Методы соединения

`connect()` — установить соединение. Конструктор вызывает его сам, звать вручную обычно не нужно.
`close_connection()` — закрыть соединение и остановить фоновый поток. Ставьте в `finally`.
`set_logger(value=True)` — включить или выключить печать хода команд уже после создания объекта.
`reboot_board()` — перезагрузить плату автопилота.

Когда объект `Pioneer` удаляется, библиотека закрывает соединение сама, но явный вызов в `finally` надёжнее. После закрытия методы чтения возвращают `None`, а команды — `False` или ничего: исключение они обычно не бросают. Чтобы работать дальше, создайте новый объект.

`set_logger(False)` заглушает сообщения библиотеки (`-> ARM`, `Взлёт завершён` и подобные); свои `print` не затрагивает. Выключайте журнал, когда программа печатает быстрый поток телеметрии.

Перезагрузка платы

`reboot_board()` работает, только если конфигурация платы её разрешает: тогда она посылает команду перезагрузки, *закрывает соединение*, ждёт 5 секунд и возвращает `True`. Если не разрешает — печатает «Перезагрузка платы ... запрещена» и возвращает `False`. На Пионере Мини 2 перезагрузка через SDK запрещена: метод всегда возвращает `False` (в симуляторе так же). Звать её в воздухе нельзя. (По 0.11.0.)

Время: uptime, flight_time, time

Метод	Что возвращает
`uptime()`	целое число секунд с момента идентификации в системе навигации (проще — сколько работает автопилот); `None` при ошибке
`flight_time()`	время с начала полёта, секунды; `None` при ошибке
`time()`	время автопилота, секунды от начала GPS-эпохи; `None` при ошибке

Для замера интервалов в программе подходят `time.time()` из Python или `flight_time()` (он считает только полёт). `time()` нужен, чтобы сопоставить моменты с журналом автопилота или с другим дроном: он берётся у самого автопилота.

ЗАДАНИЕ 16.3 · Журнал полёта

Напечатайте `uptime()` до `arm()`, затем после взлёта и после посадки печатайте `flight_time()`. После взлёта выключите журнал через `set_logger(False)` и убе-

дитесь, что сообщения библиотеки пропали, а ваши `print` остались. (Взлёт и посадку см. в следующем разделе.)

Что должно получиться: `flight_time()` до взлёта — около нуля или `None`, после взлёта растёт; после `set_logger(False)` строки «-> Посадка» и «Посадка завершена» не печатаются.

В симуляторе: миссии Мини 2 SDK2: 2 в панели «**Обучение**».

16.5 Взлёт, посадка и порядок команд

Цель раздела: научиться запускать моторы, взлетать и садиться так, чтобы дрон оказывался на земле при любом исходе, и понять правило, по которому SDK 2 принимает или отвергает команды.

python-onboard/pioneer_sdk2/sdk2_02_takeoff_land.py

```
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()
in_air = False

try:
    # arm() ждёт запуска моторов; не дождался за 5 с — False.
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")

    in_air = True
    pioneer.takeoff()  # возвращается, когда взлёт завершён
    print("В воздухе")
    time.sleep(3)            # просто висим

    pioneer.land()           # возвращается, когда дрон на земле
    in_air = False
    print("Сели")
finally:
    if in_air:  # программу прервали в воздухе — сначала сесть
        pioneer.land()
    pioneer.disarm()
    pioneer.close_connection()
```

Что происходит. В SDK 2 команды взлёта и посадки *блокирующие*: `arm()` возвращается, когда автопилот сообщил о запуске моторов, `takeoff()` — когда взлёт завершён, `land()` — когда дрон стоит на земле. Пауз `time.sleep` после них больше не нужно: программа сама ждёт ровно столько, сколько длится действие. Полётные команды (`go_to_*`, `set_manual_speed*`, `set_yaw`, `rtl`), наоборот, *не блокируют*: они отправляют команду и сразу возвращаются (раздел о полёте по точкам). Разницу показывает рисунок 24.

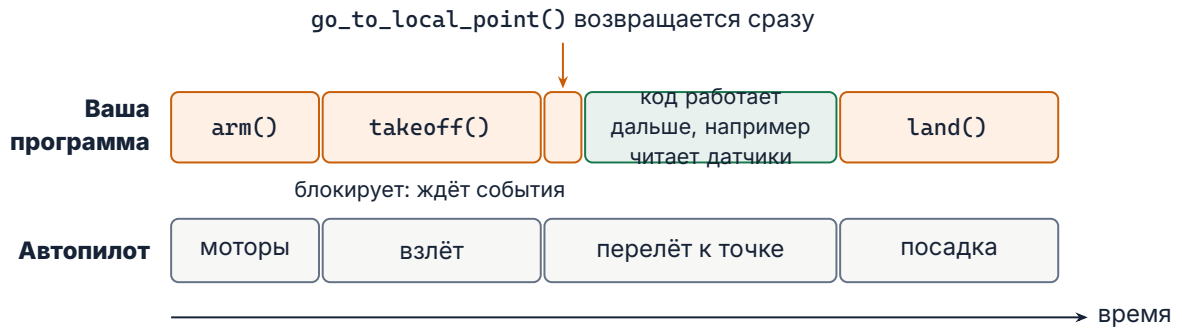


Рис. 24. Блокирующие команды (`arm`, `takeoff`, `land`) ждут события автопилота. Полётные команды (`go_to_*`, `set_manual_speed*`, `rtl`) только отправляют команду, и программа продолжает работу, пока дрон летит.

arm, disarm, takeoff, land

`arm(timeout=5, retries=0)` — запустить моторы. Ждёт события `ENGINES_STARTED` до `timeout` секунд; не дождался — выключает моторы и пробует ещё `retries` раз; если все попытки неудачны, возвращает `False`.
`takeoff()` — взлететь; ждёт `TAKEOFF_COMPLETE`.
`land()` — сесть; ждёт `COPTER_LANDED`.
`disarm()` — выключить моторы немедленно (в воздухе дрон упадёт).

Программа проверяет результат `arm()` и завершается, не пытаясь взлетать. Если моторы не запускаются, проверьте навигацию (статус `NavStatus.OK`, раздел «Навигация»), затем увеличьте `timeout` или задайте `retries=2`. `takeoff()` поднимает дрон на высоту из параметра автопилота `Copter_com_takeoffAlt`.

finally: сначала посадка

Если программу прервали в полёте (`Ctrl` + `C` или ошибка), `disarm()` в `finally` выключил бы моторы в воздухе. Переменная `in_air` помнит, что дрон летит, и тогда `finally` сначала сажает его. Звать `land()` на земле нельзя: команда будет вечно ждать события «дрон сел», которое уже не придёт (в симуляторе — предупреждение, на дроне программа зависнет).

Не отключайте wait_callback без нужды

При `wait_callback=False` `takeoff()` и `land()` возвращаются сразу, не дожидаясь результата, а контроль порядка команд отключается. По исходникам 0.11.0, в этом режиме цикл ожидания внутри `arm()` не завершается, поэтому `arm()` использовать нельзя. Оставляйте `wait_callback=True`.

Порядок команд — конечный автомат

SDK 2 сам ведёт состояние полёта — это знакомый по главе 5.9 конечный автомат с тремя состояниями (рисунок 25):

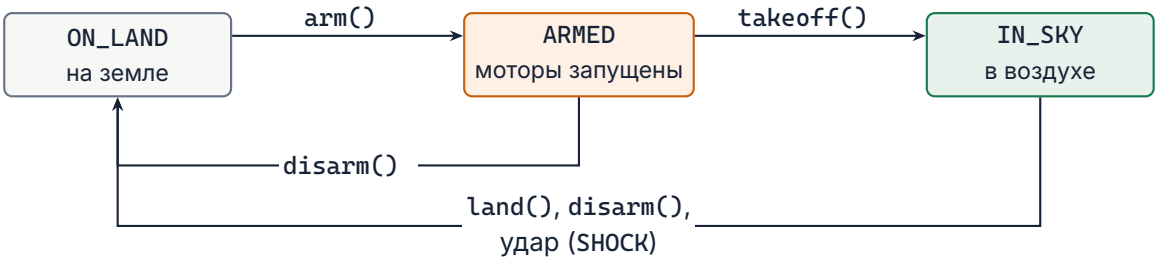


Рис. 25. Состояния полёта в SDK 2 и команды, которые переводят из одного в другое. Прямые стрелки — штатный путь вперёд, нижние — возврат на землю.

В каждом состоянии разрешена своя команда: на земле — `arm()`, с запущенными моторами — `takeoff()`, в воздухе — полётные команды. Остальные SDK проверяет так:

Состояние	`arm()`	`takeoff()`	полётные команды
ON_LAND	выполняется	RuntimeError	игнорируется, `False`
ARMED	игнорируется, `False`	выполняется	RuntimeError
IN_SKY	RuntimeError	игнорируется, `False`	выполняются

Правило за таблицей одно: команду *прошлого* этапа SDK молча пропускает (взлетать второй раз незачем), а попытку *перескочить* этап — например, `takeoff()` без `arm()` — останавливает исключением `RuntimeError`. `land()` и `disarm()` не проверяются: они должны работать всегда. Контроль включён, пока у `Pioneer` включены оба флага `wait_callback` и `safety_command` (так по умолчанию). Состояние ведётся только внутри одной программы: если одновременно запущено несколько полётных программ, часть событий каждая из них может не увидеть.

`get_fly_state()` возвращает текущее состояние как `FlyState`. SDK меняет его по событиям автопилота: `ENGINES_STARTED` — `ARMED`; `TAKEOFF_COMPLETE` — `IN_SKY`; `COPTER_LANDED`, `SHOCK` и выключение моторов — `ON_LAND`.

Две оговорки к таблице

В 0.11.0 `set_manual_speed` и `set_manual_speed_body_fixed` на земле только печатают предупреждение, но команду всё равно отправляют. Кроме того, две «лишние» клетки (повторный `arm()` и повторный `takeoff()` в воздухе) в сборнике Геоскана описаны наоборот; в хорошей программе таких вызовов нет, поэтому на практике это не важно.

После удара SDK считает, что дрон на земле

Событие `SHOCK` переводит автомат в `ON_LAND`, даже если дрон ещё в воздухе. После этого полётные команды SDK игнорирует (`False`), пока вы не запустите моторы заново, поэтому, обрабатывая удар, принимайте решение сами: `land()` не проверяется и работает всегда.

ЗАДАНИЕ 16.4 · Без пауз

Перенесите программу взлёта и посадки из главы 7 на SDK 2: уберите все `time.sleep` после `arm()`, `takeoff()` и `land()`, проверьте результат `arm()`. Затем вызовите `takeoff()` без `arm()` и объясните по таблице, что произошло.

Что должно получиться: первая программа взлетает, висит заданное время и садится; вторая останавливается исключением `RuntimeError` ещё на земле.

В симуляторе: миссии Мини 2 SDK2: 3–5 в панели «**Обучение**».

ЗАДАНИЕ 16.5 · Журнал состояний

Печатайте `get_fly_state().name` до `arm()`, после `arm()`, после `takeoff()` и после `land()`. Затем вызовите `arm()` второй раз сразу после первого и посмотрите, что вернётся.

Что должно получиться: состояния идут `ON_LAND`, `ARMED`, `IN_SKY`, `ON_LAND`; второй `arm()` возвращает `False` и пишет в журнал, что команда проигнорирована (прошлый этап).

В симуляторе: миссии Мини 2 SDK2: 5 в панели «**Обучение**».

Безопасный запуск двигателей

python-onboard/pioneer_sdk2/03_arm_disarm_safe.py

```python
from pioneer_sdk2 import Pioneer
import time

drone = Pioneer()

try:
    state = drone.get_fly_state().name                # .name — строка:
    ↪  "ON_LAND", "ARMED", "IN_SKY"
    print("Текущее состояние:", state)

    if state != "ON_LAND":
        raise RuntimeError("Двигатели можно запускать только на земле")

    # Обратный отсчёт: три красные вспышки, затем зелёный
    for _ in range(3):
        drone.led_control(r=1)
        time.sleep(0.5)
        drone.led_control()
        time.sleep(0.5)

    if not drone.arm(timeout=5, retries=1):           # ждём событие
    ↪  ENGINES_STARTED
        raise RuntimeError("Двигатели не запустились")

    drone.led_control(g=1)
    print("Состояние после arm():", drone.get_fly_state().name)   # ожидаем
    ↪  ARMED
```

```
    time.sleep(3)

finally:
    if drone.get_fly_state().name == "ARMED":          # выключаем, только
    ↪  если включены
        drone.disarm()
    drone.led_control()
    drone.close_connection()
```

Что происходит. Состояние проверяется *до* действия: `get_fly_state().name` — строка `"ON_LAND"`, `"ARMED"` или `"IN_SKY"`, и если дрон не на земле, программа останавливается с понятной ошибкой. Три красные вспышки предупреждают людей рядом о запуске двигателей. `arm(timeout=5, retries=1)` блокирует программу до события `ENGINES_STARTED` и возвращает `True` или `False`. В `finally` двигатели выключаются, *только если* они включены, то есть по состоянию, а не «на всякий случай».

> **Почему не запускаются двигатели**
>
> Если автопилот не видит пульт (тумблер `SWB`), аккумулятор разряжен (событие `LOW_CHARGE`) или дрон стоит неровно, моторы не запустятся и `arm()` вернёт `False`.

Шаблон полётной программы

Любая полётная программа устроена одинаково: запустить, взлететь, выполнить задание, сесть — и при любом исходе привести дрон в безопасное состояние. Вынесем это в шаблон:

python-onboard/pioneer_sdk2/04_takeoff_hover_land.py

```
"""Шаблон безопасной полётной программы.

Скопируйте этот файл и вписывайте своё полётное задание в функцию mission().
"""
from pioneer_sdk2 import Pioneer
import time


def mission(drone):
    """Полётное задание: здесь дрон уже в воздухе."""
    drone.go_to_local_point(x=0, y=0, z=1.0, yaw=0, time=3)   # зависаем на
    ↪  высоте 1 м
    while not drone.point_reached():
        time.sleep(0.1)
    print("Зависли на высоте:", drone.get_dist_sensor_data())
    time.sleep(5)


def safe_shutdown(drone):
    """Приводит дрон в безопасное состояние из любого положения."""
    state = drone.get_fly_state().name
    if state == "IN_SKY":
```

```
        print("Посадка...")
        drone.land()
    elif state == "ARMED":
        drone.disarm()


drone = Pioneer()

try:
    if not drone.arm():
        raise RuntimeError("arm() не выполнен")
    if not drone.takeoff():                    # блокирует программу до события
    ↪ TAKEOFF_COMPLETE
        raise RuntimeError("takeoff() не выполнен")
    mission(drone)
    drone.land()                               # после посадки двигатели
    ↪ выключатся сами

except KeyboardInterrupt:                      # Ctrl+C в терминале
    print("Прервано пользователем")
except Exception as error:                     # любая другая ошибка в задании
    print("Ошибка:", error)
finally:
    safe_shutdown(drone)
    drone.close_connection()
```

Что происходит. Менять нужно только функцию `mission(drone)`: она вызывается, когда дрон уже в воздухе. `safe_shutdown(drone)` смотрит на состояние: в воздухе — посадка, на земле с включёнными двигателями — выключение. Результаты `arm()` и `takeoff()` проверяются: при неудаче выбрасывается исключение, и программа сразу попадает в `finally`, а не пытается лететь дальше. Три ветки обработки: `KeyboardInterrupt` (`Ctrl` + `C`), любое другое исключение и обязательный `finally`. После взлёта на высоту из `Copter_com_takeoffAlt` команда `go_to_local_point` выводит дрон на нужную высоту. Устройство шаблона — на рисунке 26.

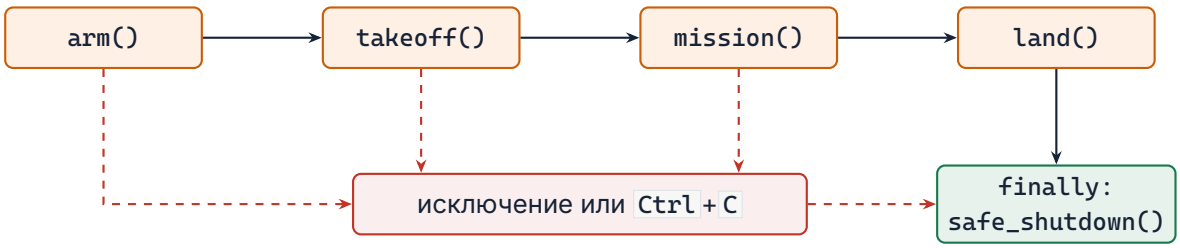


Рис. 26. Каркас полётной программы. Штатный путь — верхняя строка; при любой ошибке управление идёт в `finally`, и дрон приводится в безопасное состояние: в воздухе садится, на земле выключает моторы.

ЗАДАНИЕ 16.6 · Поворот и ошибка в задании

Добавьте в `mission()` поворот `set_yaw(90)` (перед ним хотя бы раз вызовите `go_to_local_point`). Затем вставьте в `mission()` строку `1 / 0` и убедитесь,

что дрон садится. Наконец, добавьте проверку навигации перед взлётом: если `get_nav_status_lps()` не `OK`, не взлетать.

Что должно получиться: в первом случае дрон поворачивается; во втором — ошибка печатается и дрон садится через `safe_shutdown`; в третьем при плохом статусе моторы не запускаются.

В симуляторе: миссии Мини 2 SDK2: 6 в панели «**Обучение**».

16.6 Системы координат и единицы

Цель раздела: понимать, в каких осях и единицах задаются точки, скорости и курс, — от этого зависит каждая полётная команда.

Команды точек работают в двух системах координат. *Локальная* система неподвижна: начало $(0, 0)$ — точка старта, ось Z направлена вверх. *Связанная* (body fixed) система движется вместе с дроном: Y_b — вперёд (куда смотрит камера), X_b — вправо (рисунок 27).

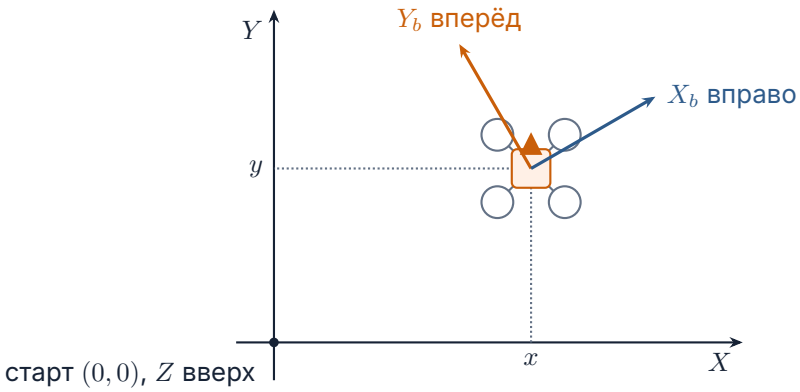


Рис. 27. Локальная система (оси X, Y, начало в точке старта, Z вверх) неподвижна. Связанная система (X_b, Y_b) движется вместе с дроном и повёрнута на его курс: Y_b смотрит вперёд, X_b — вправо.

`go_to_local_point(x, y, z, yaw)` — абсолютная точка в неподвижной системе; `go_to_local_point_body_fixed(...)` — смещение в осях корпуса от текущего положения. Единицы и знаки:

Величина	Единицы и смысл
`x, y, z`	метры; Z вверх
`yaw` в `go_to_*`	**единицы нужно проверить** (врезка ниже); в программах главы — градусы
`set_yaw(yaw)`	градусы
`vx, vy, vz`	м/с; в `*_body_fixed`: $v_y > 0$ — вперёд, $v_x > 0$ — вправо
`yaw_rate`	рад/с; положительное значение — поворот влево (против часовой)
`led_control(r, g, b)`	яркость каналов от 0 до 1; `led_id=255` — все светодиоды

Единицы курса: проверьте на своём дроне

В симуляторе, в его справочнике, в исходниках 0.11.0 и в программах этой главы курс `yaw` в `go_to_local_point` и `go_to_local_point_body_fixed` — в *градусах*; в `set_yaw` он тоже в градусах. Но в сборнике примеров Геоскана (его примеры 06 и 07) утверждается другое: в `go_to_*` курс в радианах (там курс считают через `math.atan2` без перевода и переводят углы через `math.radians`), а градусы — только в `set_yaw`. Полёты сборника на дроне не запускались, и расхождение нельзя снять без дрона.

Проверьте единицы сами: дважды вызовите `go_to_local_point(0, 0, 1, yaw=90)` и затем `yaw=1.57`, после каждого печатайте `get_local_yaw_lps()`. Если первый вызов даёт курс около 90, а второй — около 1.6, то курс в градусах. Если первый поворачивает дрон на странный угол (90 рад $\approx 117^\circ$ после приведения к диапазону ± 180), а второй даёт около 90 — в радианах. Знак поворота тоже сверьте глазами: в симуляторе положительный курс отсчитывается от оси X к оси Y (против часовой стрелки, вид сверху), а по описанию Pioneer Code положительный `set_yaw` — по часовой.

Пока не проверили, надёжнее вообще не задавать `yaw` в `go_to_*` (оставьте `None`) и поворачивать командой `set_yaw(градусы)`. Переносите программы из главы 7 внимательно: `yaw=math.pi` там означает поворот на 180°, а здесь (если курс в градусах) тот же поворот — `yaw=180`.

16.7 Полёт по точкам

Цель раздела: отправлять дрона в точки, дожидаться прибытия и описывать маршрут данными.

python-onboard/pioneer_sdk2/sdk2_03_square.py

```
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()
in_air = False


def fly_to(x, y, z, yaw):
    """Лететь в точку (метры, курс в градусах) и ждать."""
    pioneer.go_to_local_point(x=x, y=y, z=z, yaw=yaw)
    while not pioneer.point_reached():
        time.sleep(0.1)


try:
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    in_air = True
    pioneer.takeoff()

    h = 1.0
```

```
    # Вершины и курс: нос смотрит туда, куда летит дрон.
    square = [(1, 0, 0), (1, 1, 90), (0, 1, 180), (0, 0, -90)]
    for x, y, yaw in square:
        print(f"Летим в ({x}, {y}), курс {yaw}°")
        fly_to(x, y, h, yaw)

    fly_to(0, 0, h, 0)       # развернуться носом вперёд
    pioneer.land()
    in_air = False
finally:
    if in_air:
        pioneer.land()
    pioneer.disarm()
    pioneer.close_connection()
```

Что происходит. `go_to_local_point` в SDK 2 *не* блокирующая: команда отправляет точку и сразу возвращается. Дожидаться прибытия по-прежнему нужно через `point_reached()`, как в главе 7. Курс в примере — в градусах: 0 — нос вдоль оси Y (так в симуляторе), положительный поворот ведёт нос влево, к $-X$; у каждой вершины свой курс, и дрон летит по квадрату носом вперёд. Отдельной точки над стартом нет: `takeoff()` возвращается уже после взлёта.

Команды точек

`go_to_local_point(x, y, z, yaw=None, time=0)` — лететь в точку локальной системы координат (метры). `yaw` — курс (единицы — по врезке выше); если не указан, курс не меняется. `time` — желаемое время перелёта в секундах: 0 оставляет выбор скорости автопилоту, положительное значение делает перелёт предсказуемым по времени.
`go_to_local_point_body_fixed(x, y, z, yaw=None, time=0)` — то же, но координаты — смещение относительно текущего положения дрона.
`point_reached()` — `True` один раз после прибытия, затем снова `False`.
`point_deceleration()` — `True` один раз, когда дрон начал тормозить перед точкой.

Обе команды точек возвращают `False`, если SDK проигнорировал их по таблице состояний (дрон ещё на земле); в нормальном полёте возвращаемого значения нет.

Смещение «от дрона»: в какую сторону оси

У `go_to_local_point_body_fixed` ось x направлена вправо от носа дрона, y — вперёд, z — вверх (так в сборнике Геоскана и в симуляторе), а `yaw` — поворот от текущего курса. Поэтому «лететь вперёд на 1 м» — это `y=1`, а не `x=1`.

Торможение перед точкой. `point_deceleration()` срабатывает раньше, чем `point_reached()`: автопилот сообщает, что дрон подлетает и начал сбрасывать скорость. Это удобное место, чтобы заранее подготовить следующий шаг — включить подсветку или навести камеру. Пример `sdk2_09_safe_start.py` (раздел «Навигация») использует оба признака.

Опрашивайте каждую точку

SDK запоминает события, даже если вы их не опрашиваете, а `point_reached()` сбрасывает флаг, только когда вернул `True`. Если вы отправили две точки подряд и ни разу не вызвали `point_reached()` для первой, опрос второй может сразу вернуть `True`. Правило: после каждой точки — свой цикл ожидания.

ЗАДАНИЕ 16.7 · Вперёд и назад

Перенесите программу «Вперёд и назад» из главы 7 на SDK 2: ожидание точки через `point_reached()` с тайм-аутом. На обратном пути развернитесь носом к старту (`set_yaw(180)` или `yaw=180` в точке — по результату проверки единиц).

Что должно получиться: дрон пролетает 1 м по $+X$, разворачивается на $180°$, возвращается и садится на старте.

В симуляторе: миссии Мини 2 SDK2: 7–9 в панели «**Обучение**».

ЗАДАНИЕ 16.8 · Время перелёта

Отправьте дрона в точку $(2, 0, 1)$ с `time=0`, замерьте время до `point_reached()` часами `time.time()`; потом то же самое с `time=8`. Заодно запомните, через сколько секунд после команды срабатывает `point_deceleration()`.

Что должно получиться: при `time=0` перелёт идёт с обычной скоростью и занимает заметно меньше 8 с; при `time=8` время перелёта близко к 8 с (точность зависит от дрона). Торможение срабатывает перед самым прибытием.

Миссия как данные: список точек

python-onboard/pioneer_sdk2/05_square_mission.py

```
from pioneer_sdk2 import Pioneer
import time

SIDE = 1.0          # сторона квадрата, м
HEIGHT = 1.0        # высота полёта, м
POINT_TIMEOUT = 15  # сколько секунд ждать прибытия в точку, прежде чем
↪  сдаться

# Маршрут: список точек (x, y, z). Последняя точка возвращает дрон к старту.
WAYPOINTS = [
    (0.0,  0.0,  HEIGHT),
    (SIDE, 0.0,  HEIGHT),
    (SIDE, SIDE, HEIGHT),
    (0.0,  SIDE, HEIGHT),
    (0.0,  0.0,  HEIGHT),
]


def fly_to(drone, x, y, z, yaw=0.0, timeout=POINT_TIMEOUT):
```

```
    """Летит в точку и ждёт point_reached(). Возвращает True при успехе."""
    drone.go_to_local_point(x=x, y=y, z=z, yaw=yaw, time=0)
    started = time.monotonic()
    while not drone.point_reached():
        if time.monotonic() - started > timeout:
            print(f"Точка ({x}, {y}, {z}) не достигнута за {timeout} с")
            return False
        time.sleep(0.1)
    return True


drone = Pioneer()

try:
    drone.arm()
    drone.takeoff()

    for number, (x, y, z) in enumerate(WAYPOINTS, start=1):
        print(f"Точка {number}/{len(WAYPOINTS)}: x={x}, y={y}, z={z}")
        drone.led_control(b=1)                          # синий — в пути
        if not fly_to(drone, x, y, z):
            break                                       # не долетели —
            ↪ прекращаем миссию
        drone.led_control(g=1)                          # зелёный — точка
        ↪ достигнута
        time.sleep(1)

    drone.land()

except KeyboardInterrupt:
    print("Прервано, посадка")
finally:
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.led_control()
    drone.close_connection()
```

Что происходит. Маршрут — это *данные*: список `WAYPOINTS` отделён от логики полёта, и чтобы облететь треугольник или «змейку», достаточно поменять список. `fly_to()` отправляет неблокирующую команду и опрашивает `point_reached()` каждые 0.1 с. `time.monotonic()` — часы, которые не скачут при смене системного времени, ими удобно мерить интервалы; если точка не достигнута за `POINT_TIMEOUT`, функция возвращает `False`, и миссия прекращается. Светодиоды показывают фазу: синий — в пути, зелёный — на точке.

ЗАДАНИЕ 16.9 · Змейка

Сгенерируйте «змейку» для обследования прямоугольника 2×1 м с шагом 0.5 м функцией, а не вручную. Добавьте к точке четвёртый элемент — курс — и разверните дрон носом по направлению движения. Выведите длину маршрута и фактическое время полёта.

Что должно получиться: дрон проходит змейку строка за строкой; в журнале — длина маршрута и время полёта.

В симуляторе: миссии Мини 2 SDK2: 10 в панели «**Обучение**».

Относительные перемещения

python-onboard/pioneer_sdk2/06_body_fixed_moves.py

```python
from pioneer_sdk2 import Pioneer
import time


def wait_point(drone, timeout=10):
    started = time.monotonic()
    while not drone.point_reached() and time.monotonic() - started < timeout:
        time.sleep(0.1)


def step(drone, forward=0.0, right=0.0, up=0.0, turn_deg=0.0):
    """Смещение относительно корпуса дрона.

    В связанной системе координат ось Y смотрит вперёд (по камере),
    ось X — вправо, ось Z — вверх. Поворот yaw задаётся в градусах.
    """
    drone.go_to_local_point_body_fixed(x=right, y=forward, z=up,
                                       yaw=turn_deg, time=0)
    wait_point(drone)


drone = Pioneer()

try:
    drone.arm()
    drone.takeoff()
    step(drone, up=0.5)                  # +0.5 м к текущей высоте

    # «Лесенка»: шаг вперёд — поворот на 90° — снова шаг вперёд...
    # Так как смещения считаются от корпуса, дрон обходит квадрат,
    # не зная ни одной абсолютной координаты.
    for _ in range(4):
        step(drone, forward=0.8)
        step(drone, turn_deg=90)

    drone.land()
finally:
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.close_connection()
```

Что происходит. `go_to_local_point_body_fixed()` задаёт *смещение* относительно текущего положения и курса: «вперёд» — всегда туда, куда смотрит камера. Функция `step()` переименовывает оси в понятные слова: `forward` → `y`, `right` → `x`. Четыре пары

«вперёд + поворот» замыкают квадрат, хотя программа не знает ни одной абсолютной координаты.

ЗАДАНИЕ 16.10 · Шестиугольник и спираль

Нарисуйте в воздухе правильный шестиугольник (угол поворота $360°/6$). Напишите функцию `spiral(n)`: каждый шаг вперёд на 0.2 м длиннее предыдущего. Сравните: какой из примеров, 05 или 06, точнее возвращается в точку старта? Почему ошибки в относительном режиме накапливаются?

Что должно получиться: шестиугольник замыкается; спираль раскручивается; относительный маршрут заканчивается дальше от старта, потому что ошибка каждого шага прибавляется к следующему.

В симуляторе: миссии Мини 2 SDK2: 11 в панели «**Обучение**».

Облёт по окружности: камера смотрит в центр

python-onboard/pioneer_sdk2/07_circle_face_center.py

```python
from pioneer_sdk2 import Pioneer
import math
import time

CENTER_X, CENTER_Y = 0.0, 0.0   # центр окружности (локальные координаты), м
RADIUS = 0.8                    # радиус, м
HEIGHT = 1.2                    # высота, м
POINTS = 12                     # на сколько отрезков делим окружность


def circle_point(index):
    """Возвращает (x, y, yaw) для i-й точки; yaw (градусы) направлен в центр
    ↪ окружности."""
    angle = 2 * math.pi * index / POINTS            # положение на
    ↪ окружности, рад
    x = CENTER_X + RADIUS * math.cos(angle)
    y = CENTER_Y + RADIUS * math.sin(angle)
    yaw = math.degrees(math.atan2(CENTER_Y - y, CENTER_X - x))   # «из точки
    ↪ в центр», градусы
    return x, y, yaw


def wait_point(drone, timeout=10):
    started = time.monotonic()
    while not drone.point_reached() and time.monotonic() - started < timeout:
        time.sleep(0.05)


drone = Pioneer()

try:
    drone.arm()
    drone.takeoff()
```

```
    x, y, yaw = circle_point(0)
    drone.go_to_local_point(x=x, y=y, z=HEIGHT, yaw=yaw, time=0)   # выходим
    ↪ на окружность
    wait_point(drone)

    for i in range(1, POINTS + 1):                    # +1 — замыкаем круг
        x, y, yaw = circle_point(i)
        print(f"{i:2d}: x={x:+.2f} y={y:+.2f} yaw={yaw:+6.1f}°")
        drone.go_to_local_point(x=x, y=y, z=HEIGHT, yaw=yaw, time=0)
        wait_point(drone)

    drone.go_to_local_point(x=CENTER_X, y=CENTER_Y, z=HEIGHT, yaw=0, time=0)
    wait_point(drone)
    drone.land()
finally:
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.close_connection()
```

Что происходит. Точка на окружности радиуса R с центром (x_c, y_c) задаётся углом $\varphi_i = 2\pi i/N$:

$$x = x_c + R\cos\varphi, \qquad y = y_c + R\sin\varphi.$$

Чтобы нос смотрел в центр, курс должен совпадать с направлением вектора из точки в центр: $\psi = \operatorname{atan2}(y_c - y,\ x_c - x)$. Функция `math.atan2(dy, dx)`, в отличие от `atan(dy/dx)`, учитывает знаки и работает во всех четырёх четвертях. Цикл идёт до `POINTS + 1`, чтобы последняя точка совпала с первой. Чем больше `POINTS`, тем ближе многоугольник к окружности, но тем чаще дрон тормозит в точках. Курс в примере переведён в градусы (`math.degrees`), как и во всей главе (см. врезку про единицы).

ЗАДАНИЕ 16.11 · Спираль и восьмёрка

Сделайте спираль: радиус уменьшается на 0.05 м на каждой точке, а высота растёт. Нарисуйте «восьмёрку» (лемнискату): $x = R\sin\varphi$, $y = R\sin\varphi\cos\varphi$.

Что должно получиться: спираль сходится к центру и поднимается; восьмёрка пересекает центр дважды за круг.

В симуляторе: миссии Мини 2 SDK2: 12 в панели «**Обучение**».

16.8 Курс и возврат домой

set_yaw и rtl

`set_yaw(yaw)` — повернуться на месте на курс `yaw` (градусы). Возвращает `bool`. Действует только в воздухе, как полётная команда; перед ней хотя бы раз вызовите `go_to_local_point`.
`rtl()` — вернуться домой (Return To Launch). Команда не блокирующая: возвращает `True` сразу после отправки.

`rtl()` не проверяет состояние и не ждёт окончания полёта. Дрон сам возвращается в точку взлёта; признак, что всё закончилось — состояние `ON_LAND`. Чтобы дождаться посадки, не вызывайте `land()` (он ждёт события, которое придёт ещё не скоро), а опрашивайте состояние:

Ждём возвращения

```
pioneer.rtl()
started = time.monotonic()
while pioneer.get_fly_state().name != "ON_LAND":
    if time.monotonic() - started > 30:
        break                       # слишком долго: решайте сами
    time.sleep(0.2)
print("Дома")
```

Состояние `ON_LAND` SDK ставит по событию `COPTER_LANDED`, так что цикл закончится, когда автопилот сообщит о посадке. Тайм-аут нужен всегда: если цикл ждёт дольше ожидаемого, вызовите `land()` сами.

ЗАДАНИЕ 16.12 · Домой по сигналу

Взлетите, отлетите в точку $(1.5, 0, 1)$, затем вызовите `rtl()` и дождитесь посадки циклом по `get_fly_state()` с тайм-аутом 30 с. Напечатайте время возвращения.

Что должно получиться: дрон возвращается к точке взлёта и садится, программа печатает «Дома» и время; после выхода из цикла `get_fly_state()` равен `ON_LAND`.

16.9 Глобальные точки по GPS

Локальные координаты отсчитываются в системе локальной навигации (LPS). Если на дроне установлен модуль GPS/ГНСС, летать можно и по географическим координатам:

Глобальные точки (по 0.11.0)

`go_to_global_point(latitude, longitude, altitude, yaw=0)` — лететь в точку: широта и долгота в градусах, высота над уровнем моря в метрах, курс.
`go_to_global_point_relative(`
`latitude_offset, longitude_offset,`
`altitude_offset, yaw=0)` — лететь в точку, заданную смещением от текущего положения.

Обе команды работают только когда навигация дрона — GPS: если `get_nav_system()` вернул не `GPS`, библиотека печатает «Команда может быть выполнена только когда установлена система навигации GPS» и возвращает `False`. На земле команда игнорируется (`False`), в состоянии `ARMED` — `RuntimeError`, как у любой полётной команды. Результат `True` означает «команда отправлена», а не «дрон прилетел»: прибытие определяется через `point_reached()`.

Пример: сместиться на север (по 0.11.0)

```
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()
if pioneer.get_nav_system().name != "GPS":
    raise SystemExit("Навигация не GPS")

lat, lon, alt = pioneer.get_global_position_gps()
pioneer.arm()
pioneer.takeoff()
pioneer.go_to_global_point(lat + 0.0001, lon, alt + 5)
while not pioneer.point_reached():
    time.sleep(0.1)
pioneer.land()
```

Особенности глобальных точек в 0.11.0

- `yaw=0` означает «курс не задавать»: библиотека выставляет флаг курса только при `yaw != 0`. Нулевой курс этой командой не задать; для него используйте `set_yaw(0)` после прибытия.
- В исходнике `go_to_global_point_relative` стоит пометка `CHECK`: смещения широты и долготы переводятся в целые числа иначе, чем абсолютные координаты, а единицы высоты в исходнике и в README описаны по-разному (градусы или метры). Проверьте единицы на безопасной высоте малым смещением.
- `0.0001` градуса — порядка 11 м по широте; большие смещения проверяйте на открытом месте.

В симуляторе навигация локальная (`NavSystem.LPS`), поэтому обе команды печатают пояснение и возвращают `False`, а программа не падает.

ЗАДАНИЕ 16.13 · Защита от неверной навигации

Напишите функцию `go_global(lat, lon, alt)`, которая сначала проверяет `get_nav_system()` и `get_nav_status_gps()` и только при навигации GPS и статусе `OK` отдаёт команду, иначе печатает причину отказа и возвращает `False`. В симуляторе вызовите её для любой точки.

Что должно получиться: в симуляторе функция печатает, что навигация не GPS, и возвращает `False`, не вызывая `go_to_global_point`; на дроне с исправным GPS — отправляет точку.

16.10 События: подписка вместо опроса

Цель раздела: реагировать на сообщения автопилота (низкий заряд, удар, прибытие), не опрашивая их в цикле.

Опрос `point_reached()` знает только про одну точку. Автопилот сообщает и о другом:

батарея садится, сильный удар, моторы запущены. В SDK 2 на события можно подписаться — как на `callback` в Lua:

subscribe и unsubscribe

`pioneer.subscribe(функция, Event.LOW_VOLTAGE1)` — вызывать функцию при этом событии; без второго аргумента — при любом событии. Функция получает событие одним аргументом; `event.name` — его имя.
`pioneer.unsubscribe(функция, событие)` — отписаться: передайте те же функцию и событие, что при подписке. Если событие в `subscribe` не указывали, то и здесь не указывайте (по умолчанию — `Event.ALL`).

Одну функцию можно подписать на несколько событий, и на одно событие — несколько функций. События лежат в перечислении `Event`:

Событие	Когда приходит
`ALL`	не событие: подписка «на всё»
`ENGINES_STARTED`	моторы запущены
`TAKEOFF_COMPLETE`	взлёт завершён
`POINT_DECELERATION`	дрон начал тормозить перед точкой
`POINT_REACHED`	точка достигнута
`COPTER_LANDED`	дрон сел
`LOW_VOLTAGE1`	батарея садится, но вернуться домой хватит
`LOW_VOLTAGE2`	батарея почти пуста, автопилот начал аварийную посадку
`LOW_CHARGE`	критически низкий заряд: запуск моторов невозможен
`SHOCK`	столкновение или сильный удар, моторы выключаются
`DISARM`	моторы выключены (есть в 0.16)

Событие DISARM и числовые коды

Событие `DISARM` есть в `pioneer_sdk2` 0.16 (проверено на бортах Мини 2 и Базового с Radxa Zero 3W); в сборнике Геоскана и в исходниках 0.11.0 его нет. Если вашей версии оно неизвестно, `Event.DISARM` завершится ошибкой `AttributeError`; проверяйте так: `hasattr(Event, "DISARM")`.
Числовые значения событий (`event.value`) сняты с настоящего борта: `ALL` 0, `NULL` 1, `COPTER_LANDED` 3, `LOW_VOLTAGE1` 4, `LOW_VOLTAGE2` 5, `POINT_REACHED` 6, `POINT_DECELERATION` 7, `TAKEOFF_COMPLETE` 8, `ENGINES_STARTED` 9, `SHOCK` 10, `LOW_CHARGE` 11, `DISARM` 12. Сравнивайте события по имени, а не по числу: числа принадлежат протоколу автопилота и могут измениться в новой версии.

python-onboard/pioneer_sdk2/sdk2_04_events.py

```
from pioneer_sdk2 import Pioneer, Event
import time
```

```
pioneer = Pioneer()
in_air = False
low_battery = False


def on_event(event):
    """Вызывается SDK при каждом событии автопилота."""
    global low_battery
    print("Событие:", event.name)
    if event == Event.LOW_VOLTAGE1:
        # только запомнить: решает основной цикл
        low_battery = True


# без второго аргумента — все события
pioneer.subscribe(on_event)


def fly_to(x, y, z, can_abort=True):
    """Лететь в точку; False — если прервано из-за батареи."""
    pioneer.go_to_local_point(x=x, y=y, z=z, yaw=0)
    while not pioneer.point_reached():
        if can_abort and low_battery:
            return False
        time.sleep(0.1)
    return True


try:
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    in_air = True
    pioneer.takeoff()

    patrol = [(1, 0), (1, 1), (0, 1), (0, 0)]
    state = "PATROL"
    laps = 0
    while state == "PATROL":
        for x, y in patrol:
            if not fly_to(x, y, 1.0):
                state = "RETURN"
                break
        laps += 1
        if laps == 3:
            state = "RETURN"

    if low_battery:
        print("Батарея садится — возвращаемся")
        pioneer.led_control(r=1, g=0, b=0)
    fly_to(0, 0, 1.0, can_abort=False)   # домой, не прерываясь
    pioneer.land()
    in_air = False
finally:
    if in_air:
        pioneer.land()
    pioneer.disarm()
    pioneer.close_connection()
```

Что происходит. Это патруль с аварийным возвратом — тот же автомат, что в Lua-миссии из главы 5.9, только на Python. Функция `on_event` печатает каждое событие, а при `LOW_VOLTAGE1` поднимает флаг `low_battery`. Решение принимает основной цикл: `fly_to` замечает флаг, прерывает ожидание точки, и автомат переходит из `PATROL` в `RETURN`. Домой дрон летит уже без проверки флага (`can_abort=False`): иначе низкий заряд прервал бы и возвращение.

Ждать событие, а не опрашивать

python-onboard/pioneer_sdk2/09_events.py

```python
from pioneer_sdk2 import Pioneer, Event
import threading
import time

# Цвет светодиодов (r, g, b) для каждого события автопилота
EVENT_COLORS = {
    Event.ENGINES_STARTED:    (1, 0.4, 0),   # жёлтый — двигатели запущены
    Event.TAKEOFF_COMPLETE:   (0, 1, 0),     # зелёный — взлёт выполнен
    Event.POINT_DECELERATION: (0, 0, 1),     # синий — торможение перед
    ↪ точкой
    Event.POINT_REACHED:      (0, 1, 1),     # голубой — точка достигнута
    Event.COPTER_LANDED:      (1, 0, 1),     # фиолетовый — посадка
    Event.LOW_VOLTAGE1:       (1, 0, 0),     # красный — пора возвращаться
}

point_reached = threading.Event()            # «флажок», которого можно
↪ дождаться из основного потока


def on_event(event):
    """Вызывается библиотекой в отдельном потоке при каждом событии."""
    print(f"[{time.strftime('%H:%M:%S')}] событие: {event}")
    r, g, b = EVENT_COLORS.get(event, (1, 1, 1))
    drone.led_control(r=r, g=g, b=b)
    if event == Event.POINT_REACHED:
        point_reached.set()                  # будим основной поток


def go(x, y, z):
    point_reached.clear()                    # сбрасываем флажок ДО отправки
    ↪ команды
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0, time=0)
    if not point_reached.wait(timeout=15):   # ждём без опроса в цикле
        print("Не дождались POINT_REACHED")


drone = Pioneer()
for event in EVENT_COLORS:
    drone.subscribe(on_event, event)

try:
    drone.arm()
    drone.takeoff()
    go(0, 0, 1.0)
```

```
    go(1, 0, 1.0)
    go(0, 0, 1.0)
    drone.land()
    time.sleep(2)                                # успеваем увидеть событие
    ↪ COPTER_LANDED
finally:
    for event in EVENT_COLORS:
        drone.unsubscribe(on_event, event)
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.led_control()
    drone.close_connection()
```

Что происходит. Словарь `EVENT_COLORS` связывает событие с цветом; подписка в цикле по ключам словаря избавляет от повторяющихся строк. `on_event(event)` вызывается библиотекой в параллельном потоке. `threading.Event` — потокобезопасный флажок: обработчик делает `set()`, основной поток ждёт `wait(timeout)`. Порядок «`clear()` → команда → `wait()`» важен: если сбросить флажок после команды, можно пропустить быстро пришедшее событие. В `finally` подписки снимаются через `unsubscribe()`.

> **Обработчик события — короткий**
>
> SDK вызывает обработчик из своего служебного потока, который принимает данные от автопилота. Пока обработчик работает, следующие события ждут. Поэтому в обработчике только запоминают факт (флаг, счётчик), а летают — в основном цикле. Блокирующую команду вроде `land()` из обработчика не вызывайте: она ждёт события, которое этот же поток и должен доставить.

ЗАДАНИЕ 16.14 · Удар в патруле

Добавьте в патруль обработку события `SHOCK`: обработчик только поднимает флаг, а основной цикл, заметив его, включает красную подсветку и сажает дрон. Каждое событие печатайте по имени `event.name`.

Что должно получиться: в обычном полёте в журнале `ENGINES_STARTED`, `TAKEOFF_COMPLETE`, `POINT_REACHED`…; при ударе — `SHOCK`, красный свет и посадка из основного цикла.

ЗАДАНИЕ 16.15 · Низкий заряд

По событию `LOW_VOLTAGE1` включайте мигание красным и вызывайте `rtl()`. Посчитайте время между `POINT_DECELERATION` и `POINT_REACHED` для разных расстояний перелёта.

Что должно получиться: в журнале видна пара событий с интервалом, растущим с расстоянием; при низком заряде дрон мигает красным и возвращается домой.

ЗАДАНИЕ 16.16 · Подсчёт точек

Подпишите функцию на `Event.POINT_REACHED` и считайте прибытия в счётчик; пролетите три точки подряд, дожидаясь прибытия в каждую по значению счётчика, а не через `point_reached()`. В конце отпишитесь через `unsubscribe` тем же способом, каким подписывались.

Что должно получиться: счётчик растёт на единицу за каждую точку и равен 3 к концу маршрута; после `unsubscribe` счётчик больше не меняется.

В симуляторе: миссии Мини 2 SDK2: 13–15 в панели «**Обучение**».

16.11 Полёт со скоростью

Цель раздела: управлять дроном скоростью, а не координатой, — как нужно регуляторам и слежению.

python-onboard/pioneer_sdk2/sdk2_05_manual_speed.py

```
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()
in_air = False


def move(vx, vy, vz, seconds):
    """Лететь с заданной скоростью (м/с) seconds секунд.

    В SDK 2 одна команда держит скорость interval секунд,
    после этого дрон сам останавливается и зависает.
    """
    pioneer.set_manual_speed(vx=vx, vy=vy, vz=vz, yaw_rate=0,
                             interval=seconds)
    time.sleep(seconds)


try:
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    in_air = True
    pioneer.takeoff()

    move(0.5, 0, 0, 2)       # 2 с по оси X со скоростью 0.5 м/с
    move(0, 0.5, 0, 2)       # 2 с по оси Y
    move(0, 0, 0.3, 1)       # 1 с вверх
    # Поворот на месте: yaw_rate в рад/с (не больше 1.82).
    pioneer.set_manual_speed(vx=0, vy=0, vz=0, yaw_rate=0.5,
                             interval=2)
    time.sleep(3)

    pioneer.land()
    in_air = False
finally:
```

```
    if in_air:
        pioneer.land()
    pioneer.disarm()
    pioneer.close_connection()
```

Что происходит. В SDK 2 у `set_manual_speed` есть аргумент `interval`: одна команда держит скорость столько секунд, после чего дрон сам останавливается. Повторять команду каждые 0.05 с, как в главе 7.9, не нужно. В регуляторе, который пересчитывает скорость на каждом кадре, `interval` делают коротким (0.3–0.5 с): если программа зависнет, дрон через полсекунды остановится сам.

Команды скорости

`set_manual_speed(vx, vy, vz, yaw_rate, interval=1.0)` — лететь со скоростью в осях локальной системы координат (м/с); `yaw_rate` в рад/с, положительное значение — влево.
`set_manual_speed_body_fixed(vx, vy, vz, yaw_rate, interval=1.0)` — то же, но скорости заданы в осях самого дрона: v_x — вправо, v_y — вперёд.

Команда скорости не ждёт и ничего не возвращает. `yaw_rate` не больше 1.82 рад/с по модулю: большее значение SDK отбрасывает без сообщения (по 0.11.0; симулятор ведёт себя так же и пишет пояснение). Скорости по 0.11.0 округляются до целых сантиметров в секунду, поэтому значения меньше 0.01 м/с превращаются в ноль. Остановить дрон раньше `interval` можно командой с нулевыми скоростями.

Повторять команду скорости

python-onboard/pioneer_sdk2/08_manual_speed.py

```
from pioneer_sdk2 import Pioneer
import time

SEND_PERIOD = 0.2       # команду скорости нужно повторять, пока движение
↪ должно продолжаться
COMMAND_INTERVAL = 0.3  # сколько секунд действует одна команда (чуть больше
↪ периода отправки)


def move(drone, vx=0.0, vy=0.0, vz=0.0, yaw_rate=0.0, duration=1.0):
    """Держит заданную скорость (в осях корпуса) в течение duration
    ↪ секунд."""
    end = time.monotonic() + duration
    while time.monotonic() < end:
        drone.set_manual_speed_body_fixed(vx, vy, vz, yaw_rate,
        ↪ COMMAND_INTERVAL)
        time.sleep(SEND_PERIOD)
    drone.set_manual_speed_body_fixed(0, 0, 0, 0, COMMAND_INTERVAL)   # явная
    ↪ остановка
    time.sleep(0.5)
```

```
drone = Pioneer()

try:
    drone.arm()
    drone.takeoff()

    move(drone, vz=0.3, duration=2)             # подъём со скоростью 0.3 м/с
    move(drone, vy=0.3, duration=2)             # вперёд (ось Y корпуса)
    move(drone, yaw_rate=0.8, duration=4)       # разворот на месте, рад/с
    move(drone, vy=0.3, yaw_rate=0.5, duration=6)   # дуга: вперёд + поворот
    ↪ одновременно
    move(drone, vz=-0.3, duration=2)            # снижение

    drone.land()
finally:
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.close_connection()
```

Что происходит. Команда повторяется каждые `SEND_PERIOD` = 0.2 с, а `COMMAND_INTERVAL` выбран чуть больше (0.3 с): между повторами нет «провалов», а если программа зависнет, дрон остановится через 0.3 с сам. В конце движения отправляется нулевая скорость — остановка происходит сразу, а не после истечения интервала. Скорости можно комбинировать: `vy` и `yaw_rate` вместе дают полёт по дуге радиуса $R = v/\omega$.

ЗАДАНИЕ 16.17 · Скорость без цикла

Одной командой `set_manual_speed(..., interval=2)` пролетите вперёд со скоростью 0.3 м/с, затем поверните на месте с `yaw_rate=0.5` в течение 3 с. Сравните измеренные положение и курс `get_local_yaw_lps()` до и после.

Что должно получиться: смещение около 0.6 м, после чего дрон сам зависает; курс меняется примерно на 86° (0.5 рад/с $\times$ 3 с).

В симуляторе: миссии Мини 2 SDK2: 16–17 в панели «**Обучение**».

ЗАДАНИЕ 16.18 · Плавный старт

Сделайте «плавный старт»: скорость линейно растёт от 0 до 0.4 м/с за 2 с. Оцените радиус дуги по формуле $R = v/\omega$ и проверьте на практике. Реализуйте полёт по квадрату только командами скорости и сравните точность с примером 05.

Что должно получиться: дрон разгоняется без рывка; радиус дуги близок к v/ω; квадрат из скоростей замыкается хуже, чем квадрат из точек.

В симуляторе: миссии Мини 2 SDK2: 18 в панели «**Обучение**».

ЗАДАНИЕ 16.19 · Слишком быстрый поворот

Подайте `yaw_rate=2.0` на 2 с, затем `yaw_rate=1.8` на 2 с, измеряя курс через `get_local_yaw_lps()`. Объясните различие.

Что должно получиться: при `yaw_rate=2.0` (больше 1.82) дрон не поворачивается, и ошибки нет — SDK отбросил команду; при 1.8 курс меняется на $206°$ за 2 с (1.8 рад/с $\times$ 2 с), но `get_local_yaw_lps()` приводит курс к диапазону ± 180 и покажет изменение около $-154°$.

16.12 Датчики дрона

Цель раздела: читать датчики и не путать «нет данных» с нулём.

Все методы чтения датчиков устроены одинаково: они ничего не ждут и ничего не отправляют дрону, а возвращают последнее полученное от автопилота значение (кортеж или число). Если данных нет (датчика нет на плате, нет связи, автопилот ещё не прислал значение), метод возвращает `None`, поэтому результат всегда проверяют перед распаковкой. Взлетать для чтения датчиков не нужно.

Метод	Что возвращает
`get_accel()`	ускорение (a_x, a_y, a_z) по осям дрона
`get_gyro()`	угловая скорость (g_x, g_y, g_z), рад/с
`get_mag()`	показания магнитометра (m_x, m_y, m_z)
`get_orientation()`	крен, тангаж, рыскание $(roll, pitch, yaw)$, градусы
`get_altitude()`	высота по барометру, метры
`get_dist_sensor_data()`	дальность дальномера вниз, метры
`get_optical_data()`	оптический поток: $(flow_x, flow_y,)$; поток — числа датчика, дальность — метры
`get_ranger_data()`	пять расстояний модуля Ranger, метры: справа, слева, спереди, сзади, по вертикали (в симуляторе — вниз)
`get_motors_rpm()`	список из четырёх чисел — обороты моторов

python-onboard/pioneer_sdk2/sdk2_08_sensors.py

```python
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()


def show(name, value):
    """Напечатать показание; None - датчика нет или данных ещё нет."""
    if value is None:
        print(f"{name}: нет данных")
    else:
        print(f"{name}: {value}")
```

```
try:
    # Датчики читаются на земле, взлетать не нужно.
    for i in range(3):
        print(f"--- замер {i + 1} ---")
        show("ускорение (ax, ay, az)", pioneer.get_accel())
        show("гироскоп, рад/с", pioneer.get_gyro())
        show("магнитометр", pioneer.get_mag())
        show("крен, тангаж, курс, град", pioneer.get_orientation())
        show("высота, м", pioneer.get_altitude())
        show("дальномер вниз, м", pioneer.get_dist_sensor_data())
        show("оптический поток (fx, fy, дальность)",
        ↪  pioneer.get_optical_data())
        show("позиция LPS, м", pioneer.get_local_position_lps())
        show("скорость LPS, м/с", pioneer.get_local_velocity_lps())

        # Ranger возвращает пять None, если модуля нет.
        right, left, front, back, below = pioneer.get_ranger_data()
        show("Ranger: справа, слева, спереди, сзади, снизу",
             None if right is None else (right, left, front, back, below))

        # Обороты четырёх моторов; на земле с выключенными моторами — нули.
        show("обороты моторов", pioneer.get_motors_rpm())
        time.sleep(1)
finally:
    pioneer.close_connection()
```

Что происходит. Программа три раза подряд печатает все показания. Функция `show` отличает «нет данных» (`None`) от нуля. `get_ranger_data()` устроен иначе: когда модуля Ranger нет, он возвращает не `None`, а пять значений `None` подряд, поэтому распаковка проходит всегда, а проверять нужно первое значение. Если плата не поддерживает Ranger, библиотека дополнительно печатает «Модуль Ranger не поддерживается» (по 0.11.0; в симуляторе Ranger есть всегда, дальность ограничена 4 м).

Какой датчик что даёт.

- **IMU** — акселерометр, гироскоп, магнитометр. По акселерометру и гироскопу видно, как дрон движется и вращается; по магнитометру — куда повёрнут нос относительно поля Земли. По описанию пакета магнитометр работает «только для GPS»: на дроне с локальной навигацией он может вернуть `None`. В симуляторе он моделирует поле Земли по курсу.
- **Ориентация** — углы, посчитанные автопилотом: `get_orientation()` даёт крен, тангаж и рыскание сразу, `get_local_yaw_lps()` — только курс.
- **Высота** — три способа: `get_altitude()` (барометр), `get_dist_sensor_data()` (расстояние до поверхности под дроном) и `get_local_position_lps()[2]` (координата z в локальной системе). Значения у них не обязаны совпадать: над препятствием дальномер покажет меньше.

- **Оптический поток** — нижняя камера и дальномер определяют, как «плывёт» пол под дроном; так дрон без Локуса и GPS удерживает позицию. Дальность `get_optical_data()[2]` та же, что у `get_dist_sensor_data()`.
- **Ranger** — модуль из пяти дальномеров, смотрящих в пяти направлениях.
- **Обороты моторов** — четыре числа; если не удалось прочитать хотя бы одно, в списке будут четыре `None`.

> **Единицы измерения**
>
> SDK не указывает единицы у ускорения и магнитометра: по 0.11.0 это число из поля автопилота, поделённое на 1000. Гироскоп — рад/с, ориентация — градусы, высота и дальности — метры. Чтобы понять единицы ускорения, положите дрон ровно и посмотрите, чему равна ось, направленная вверх.

ЗАДАНИЕ 16.20 · Какие у ускорения единицы

Оставьте неподвижный дрон на земле и напечатайте `get_accel()` 10 раз. Определите, по какой оси значение заметно больше остальных, и по числу догадайтесь, в каких единицах ускорение.

Что должно получиться: все десять замеров близки друг к другу; одна из осей заметно отличается от остальных. На настоящем дроне она покажет силу тяжести, в симуляторе значения модельные.

ЗАДАНИЕ 16.21 · Тревога по Ranger

Если Ranger нет (первое значение `None`), завершите программу с сообщением до взлёта. Иначе взлетите на 1 м и летите вперёд со скоростью 0.2 м/с короткими командами (`interval=0.5`), пока `get_ranger_data()` не покажет впереди меньше 0.5 м; тогда остановитесь (нулевая скорость) и сядьте.

Что должно получиться: перед препятствием дрон останавливается до столкновения и садится; без Ranger печатается сообщение и взлёта нет.

В симуляторе: миссии Мини 2 SDK2: 19 в панели «**Обучение**».

16.13 Навигация: LPS, GPS и оптика

Цель раздела: знать, чьей оценке положения доверяет дрон, и взлетать только при годной навигации.

Автопилот определяет положение одной из трёх систем навигации; какая используется — решает параметр автопилота, а библиотека его читает:

Система навигации

`get_nav_system(update=False)` — возвращает `NavSystem`: `GPS` (спутники ГНСС), `LPS` (локальная система «Локус» — ультразвук и ИК-маяки) или `OPT` (оптическая

навигация по камере и дальномеру). Значение запоминается; `update=True` читает заново.

Для каждой системы в библиотеке свои методы чтения; они работают, только когда эта система установлена и выбрана:

Метод	Что возвращает
`get_nav_status_lps()`	`NavStatus` локальной навигации
`get_local_position_lps()`	позиция (x, y, z), метры
`get_local_velocity_lps()`	скорость (v_x, v_y, v_z), м/с
`get_local_yaw_lps()`	курс, градусы, от -180 до 180
`get_nav_status_gps()`	`NavStatus` GPS
`get_global_position_gps()`	$(,,)$: градусы, градусы, метры
`get_global_velocity_gps()`	(v, v, v), м/с
`get_satellites_count()`	$(GPS,)$

Если модуль навигации не установлен, метод печатает «Модуль навигации не установлен» и возвращает `None`; в `get_global_*` и `get_satellites_count` значение `None` означает и «нет модуля», и «модуль не отвечает».

Статус навигации. `NavStatus` показывает, можно ли верить позиции: `NO_DATA` — модуль ничего не прислал, `CANNOT` — позицию оценить невозможно, `LOW` — доверие малое, `OK` — оценка успешна. Взлетать надо при `OK`: если `arm()` не удаётся, первое, что стоит проверить, — статус навигации. Для OPT отдельного метода статуса нет.

python-onboard/pioneer_sdk2/sdk2_09_safe_start.py

```python
from pioneer_sdk2 import Pioneer
import time

pioneer = Pioneer()
in_air = False

try:
    # Какая навигация и насколько ей можно верить.
    print("Навигация:", pioneer.get_nav_system().name)
    status = pioneer.get_nav_status_lps()
    print("Статус LPS:", None if status is None else status.name)
    if status is None or status.name != "OK":
        raise SystemExit("Навигация не готова, взлёт отменён")

    print("Состояние:", pioneer.get_fly_state().name)      # ON_LAND
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    print("Состояние:", pioneer.get_fly_state().name)      # ARMED

    in_air = True
    pioneer.takeoff()
```

```
    print("Состояние:", pioneer.get_fly_state().name)      # IN_SKY

    pioneer.go_to_local_point(x=1, y=0, z=1, yaw=0)
    slowing = False
    while not pioneer.point_reached():
        if not slowing and pioneer.point_deceleration():
            slowing = True
            print("Начали тормозить перед точкой")
        time.sleep(0.1)
    print(f"Прилетели, полёт идёт {pioneer.flight_time()} с")

    pioneer.land()
    in_air = False
    print("Состояние:", pioneer.get_fly_state().name)      # ON_LAND
finally:
    if in_air:
        pioneer.land()
    pioneer.disarm()
    pioneer.close_connection()
```

Что происходит. Программа проверяет систему навигации и её статус, и только потом запускает моторы. Состояние `get_fly_state()` печатается на каждом этапе. Перелёт в точку отслеживает оба события: сначала `point_deceleration()` (дрон начал тормозить), затем `point_reached()`. После прибытия `flight_time()` показывает, сколько секунд идёт полёт. Нет надёжной навигации — `SystemExit` до взлёта: в `finally` тогда сработает только `disarm()` и `close_connection()`.

⚠ Скорость по GPS идёт в других осях

`get_global_velocity_gps()` возвращает скорость по сторонам света (север, восток, вертикаль), а не по осям локальной системы, как `get_local_velocity_lps()`. Не подставляйте одно вместо другого. Знак вертикальной составляющей — по названию поля автопилота, это скорость «вниз»; проверьте его лёгким подъёмом, прежде чем использовать в расчётах.

ЗАДАНИЕ 16.22 · Готов ли дрон взлетать

Напишите функцию `nav_ok(pioneer)`: она печатает имя системы навигации и статус нужной системы (LPS — `get_nav_status_lps`, GPS — `get_nav_status_gps`; для OPT напечатайте, что статус неизвестен) и возвращает `True`, только если имя статуса `OK`. Используйте её перед `arm()`.

Что должно получиться: в симуляторе печатается `LPS` и `OK`, функция возвращает `True`, дрон взлетает; при статусе, отличном от `OK`, функция возвращает `False` и взлёта нет.

В симуляторе: миссии Мини 2 SDK2: 6 в панели «**Обучение**».

ЗАДАНИЕ 16.23 · Спутники и позиция

На дроне с модулем GPS (на улице) напечатайте раз в секунду 10 раз: число спутников, статус и широту с долготой с шестью знаками. Дождитесь, когда статус станет `OK`. В симуляторе GPS нет — убедитесь, что методы возвращают `None` и программа не падает.

Что должно получиться: на улице число спутников растёт, статус доходит до `OK`, координаты перестают «плавать»; в симуляторе все строки — «нет данных», без ошибок и исключений.

В симуляторе: миссии Мини 2 SDK2: 20 в панели «**Обучение**».

16.14 Журнал полёта в CSV

Показания датчиков удобно записывать в файл: по журналу видно, что происходило в полёте. Запись ведёт отдельный фоновый поток параллельно с миссией.

python-onboard/pioneer_sdk2/10_telemetry_logger.py

```
from pioneer_sdk2 import Pioneer
import csv
import threading
import time


class TelemetryLogger(threading.Thread):
    """Фоновый поток: раз в period секунд пишет строку телеметрии в
    ↪ CSV-файл."""

    def __init__(self, drone, filename, period=0.2):
        super().__init__(daemon=True)           # daemon — поток не помешает
        ↪ завершению программы
        self.drone = drone
        self.filename = filename
        self.period = period
        self._stop_event = threading.Event()

    def run(self):
        with open(self.filename, "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(["t", "state", "x", "y", "z", "range", "roll",
            ↪ "pitch", "yaw"])
            t0 = time.monotonic()
            while not self._stop_event.is_set():
                position = self.drone.get_local_position_lps() or [None] * 3
                orientation = self.drone.get_orientation() or [None] * 3
                writer.writerow([
                    round(time.monotonic() - t0, 2),
                    self.drone.get_fly_state().name,
                    *position,
                    self.drone.get_dist_sensor_data(),
                    *orientation,
                ])
```

```
                self._stop_event.wait(self.period)       # пауза, которую
                ↪ можно прервать stop()

    def stop(self):
        self._stop_event.set()
        self.join()


drone = Pioneer()
drone.set_logger(False)
logger = TelemetryLogger(drone,
↪ f"flight_{time.strftime('%Y%m%d_%H%M%S')}.csv")
logger.start()

try:
    drone.arm()
    drone.takeoff()
    for x, y in [(0, 0), (1, 0), (1, 1), (0, 0)]:
        drone.go_to_local_point(x=x, y=y, z=1.0, yaw=0, time=0)
        while not drone.point_reached():
            time.sleep(0.1)
    drone.land()
    time.sleep(3)                                  # записываем и момент
    ↪ посадки
finally:
    logger.stop()
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.close_connection()
    print("Лог сохранён:", logger.filename)
```

Что происходит. Класс `TelemetryLogger` наследует `threading.Thread`, логика записи — в `run()`. `daemon=True` не даёт потоку удерживать программу после завершения основного кода. Остановка идёт через `threading.Event`: `wait(period)` работает как пауза, которую можно прервать, поэтому после `stop()` поток завершается сразу. Имя `_stop_event` выбрано не случайно: у `threading.Thread` уже есть внутренний метод `_stop`, и его переопределение ломает `join()`. Запись `value or [None] * 3` подставляет пустые значения, если метод вернул `None`. Имя файла содержит дату и время, поэтому журналы разных полётов не перезаписывают друг друга. В симуляторе файл создаётся внутри Python-среды браузера и на компьютер не скачивается: для проверки логики смотрите печать программы.

Для анализа скопируйте CSV на компьютер и постройте графики (программа запускается на компьютере, нужен `matplotlib`):

python-onboard/pioneer_sdk2/10b_plot_log.py

```
"""Строит траекторию полёта по CSV-логу из примера 10 (запускается на
↪ компьютере).

python 10b_plot_log.py flight_20261014_120000.csv
"""
```

```python
import csv
import sys
import matplotlib.pyplot as plt

rows = []
with open(sys.argv[1], encoding="utf-8") as file:
    for row in csv.DictReader(file):
        if row["x"] not in ("", "None"):           # пропускаем строки без
        ↪ координат
            rows.append({k: float(row[k]) for k in ("t", "x", "y", "z")})

t = [r["t"] for r in rows]
fig, (top, side) = plt.subplots(1, 2, figsize=(11, 4.5))

top.plot([r["x"] for r in rows], [r["y"] for r in rows], marker=".")
top.set_title("Вид сверху")
top.set_xlabel("x, м")
top.set_ylabel("y, м")
top.axis("equal")
top.grid(True)

side.plot(t, [r["z"] for r in rows])
side.set_title("Высота от времени")
side.set_xlabel("t, с")
side.set_ylabel("z, м")
side.grid(True)

plt.tight_layout()
plt.savefig(sys.argv[1].replace(".csv", ".png"), dpi=150)
plt.show()
```

ЗАДАНИЕ 16.24 · Журнал с событиями

Добавьте в журнал колонку `event` с событиями автопилота (подписка из раздела про события) и колонку с текущей целевой точкой. Определите по журналу максимальную скорость перелёта (разность координат по времени).

Что должно получиться: в CSV видны строки с событиями `POINT_REACHED` и `TAKEOFF_COMPLETE`; скорость перелёта получается порядка десятых долей м/с.

В симуляторе: миссии Мини 2 SDK2: 21 в панели «**Обучение**».

16.15 Параметры автопилота

Цель раздела: читать и менять настройки автопилота так, чтобы после программы они гарантированно вернулись к исходным.

У автопилота есть настройки — параметры: имя и число. Через SDK их можно читать и менять:

get_param и set_param

`get_param(name, update=False)` — значение параметра `name` (число) или `None`, если такого параметра нет или не удалось прочитать. При первом обращении библиотека загружает список параметров автопилота и печатает «-> Загрузка словаря автопилота ...»; без `update=True` возвращается значение, запомненное при включении, и оно может устареть. `update=True` читает значение из автопилота заново. `set_param(name, value)` — записать значение (оно хранится как вещественное число). Возвращает `True`, если параметр с таким именем есть и команда отправлена, иначе `False`.

Так, `get_nav_system()` узнаёт навигацию из параметра `Flight_com_navSystem`, а высоту взлёта задаёт параметр `Copter_com_takeoffAlt`. Описание остальных параметров смотрите в документации автопилота вашей платы.

Безопасная смена параметра

```
name = "Flight_com_takeoffAlt"
old = pioneer.get_param(name, update=True)
print("было:", old)
if old is not None:
    pioneer.set_param(name, 0.7)       # ваше значение
    print("стало:", pioneer.get_param(name, update=True))
    # ... работа ...
    pioneer.set_param(name, old)       # вернуть как было
```

⚠ Параметры меняют поведение дрона

Опечатка в имени параметра безвредна (`set_param` вернёт `False`), а вот верное имя с неверным значением изменит поведение дрона в полёте. Перед записью читайте старое значение, запоминайте его и возвращайте в конце программы. По описанию Pioneer Code параметры автопилота *сохраняются после перезагрузки*, поэтому ошибочное значение (например, источника сигнала RC) может сделать дрон неуправляемым с пульта.

ℹ Параметры в симуляторе

Симулятор знает те же 232 параметра, что настоящий автопилот Мини 2 (имена и значения сняты с борта). Неизвестное имя даёт `None` и `False`, как на дроне. Значения живут до конца программы и полёт не меняют (кроме набора для пульта, см. раздел «Пульт RC»); настройки автопилота в панели симулятора `set_param` не трогает. Два значения симулятор намеренно держит своими: `Flight_com_navSystem` равен 1 (LPS, менять нельзя), а у борта 2 (оптика, OPT); `Flight_com_takeoffAlt` равен 0.4 м, а у борта 0.5 м, потому что симулятор взлетает на 0.4 м. Имя высоты взлёта — именно `Flight_com_takeoffAlt`; параметра `Copter_com_takeoffAlt` на борту нет. Значения для пульта на борту по умолчанию такие: `Copter_man_rcMode0/1/2` = 1/2/4, `Copter_flyWithoutRc` = 1, `SensorMux_rc` = 0.

Контекстный менеджер для параметров. Возврат старого значения удобно спрятать в конструкцию `with`:

python-onboard/pioneer_sdk2/11_params_context.py

```python
from pioneer_sdk2 import Pioneer
from contextlib import contextmanager


@contextmanager
def temporary_params(drone, **new_values):
    """Временно меняет параметры автопилота и гарантированно возвращает
    ↪ старые.

    with temporary_params(drone, Copter_flyWithoutRc=1.0):
        ...  # здесь параметр изменён
    # здесь параметр снова исходный
    """
    old_values = {}
    try:
        for name, value in new_values.items():
            old_values[name] = drone.get_param(name, update=True)   # читаем
            ↪ из автопилота, не из кэша
            if not drone.set_param(name, value):
                raise RuntimeError(f"Не удалось установить {name}={value}")
            print(f"{name}: {old_values[name]} -> {value}")
        yield old_values
    finally:
        for name, value in old_values.items():                      # откат в
        ↪ любом случае
            if value is not None:
                drone.set_param(name, value)
                print(f"{name}: восстановлено {value}")


drone = Pioneer()

try:
    print("Высота взлёта сейчас:", drone.get_param("Flight_com_takeoffAlt"))

    with temporary_params(drone, Flight_com_takeoffAlt=0.7):
        print("Внутри блока:", drone.get_param("Flight_com_takeoffAlt",
        ↪ update=True))
        # здесь можно, например, выполнить arm()/takeoff() с изменённой
        ↪ высотой взлёта

    print("После блока :", drone.get_param("Flight_com_takeoffAlt",
    ↪ update=True))
finally:
    drone.close_connection()
```

Код до `yield` выполняется при входе в блок `with`, код в `finally` — при выходе, *даже если внутри блока была ошибка*. Значения читаются с `update=True`, результат `set_param` проверяется. Программа временно ставит высоту взлёта 0.7 м и возвращает прежнюю.

ЗАДАНИЕ 16.25 · Резервная копия параметров

Напишите функцию `with_param(pioneer, name, value, action)`, которая читает старое значение параметра, записывает новое, выполняет `action()` и в `finally` возвращает старое значение. Проверьте её на неверном имени параметра: если параметра нет, действие выполнять не нужно. Для проверки на дроне пишите в параметр то же значение, что прочитали.

Что должно получиться: для верного имени параметр после программы равен исходному значению, даже если `action()` упал; для неверного имени `set_param` печатает «Установка параметра … не удалась», а ваша функция не выполняет действие и сообщает об ошибке.

В симуляторе: миссии Мини 2 SDK2: 22 в панели «**Обучение**».

16.16 Подсветка

Цель раздела: показывать состояние программы цветом и не путать шкалу цвета SDK 2 со шкалой первого SDK.

led_control

`led_control(led_id=255, r=0, g=0, b=0)` — цвет светодиода. `r`, `g`, `b` — яркость от 0 до 1 (значения вне диапазона обрезаются). `led_id` от 0 до 255, 255 — все светодиоды; число вне диапазона 0–255 даёт `ValueError`. Возвращает `True`, если команда отправлена. Вызов без аргументов гасит всё.

Цвет в SDK 2

Цвет — доли от 0 до 1, а не числа 0..255, как в `pioneer_sdk`. Передали `r=255` — получили полную яркость красного без ошибки: значение просто обрезается до 1. Ошибки не будет, но ожидаемого цвета тоже.

Сколько светодиодов у вашего Мини 2, зависит от платы и версии. В симуляторе их два: 0 — левый, 1 — правый передний индикатор. Примеры сборника Геоскана рассчитаны на четыре. По исходникам 0.11.0 на настоящей плате Мини 2 подсветка может быть общей: библиотека записывает цвет в общий модуль светодиодов, а `led_id` не использует, и любой номер красит всё сразу. Если вам нужны разные цвета у разных светодиодов, проверьте это на своём дроне.

python-onboard/pioneer_sdk2/02_led_effects.py

```
from pioneer_sdk2 import Pioneer
import colorsys                    # стандартная библиотека: перевод цвета
↪  из HSV в RGB
import time

LED_COUNT = 4                      # светодиоды нумеруются с 0; 255 — все
↪  сразу
```

```
ALL_LEDS = 255


def rainbow(drone, duration=6.0, step=0.1):
    """Плавно прокручивает «радугу» по всем светодиодам."""
    start = time.time()
    while time.time() - start < duration:
        phase = (time.time() - start) / duration          # 0..1 — доля
        ↪  пройденного времени
        for led in range(LED_COUNT):
            hue = (phase + led / LED_COUNT) % 1.0          # каждый светодиод
            ↪  сдвинут по оттенку
            r, g, b = colorsys.hsv_to_rgb(hue, 1.0, 1.0)   # яркость каналов
            ↪  в диапазоне 0..1
            drone.led_control(led, r=r, g=g, b=b)
        time.sleep(step)


def running_light(drone, r, g, b, laps=3, delay=0.15):
    """«Бегущий огонь»: в каждый момент горит один светодиод."""
    for _ in range(laps):
        for led in range(LED_COUNT):
            drone.led_control(ALL_LEDS)                    # без цвета =
            ↪  выключить все
            drone.led_control(led, r=r, g=g, b=b)
            time.sleep(delay)


def pulse(drone, r, g, b, times=3, steps=10, delay=0.04):
    """Мягкое «дыхание» — яркость растёт и падает."""
    for _ in range(times):
        for k in list(range(steps + 1)) + list(range(steps, -1, -1)):
            level = k / steps
            drone.led_control(ALL_LEDS, r=r * level, g=g * level, b=b *
            ↪  level)
            time.sleep(delay)


drone = Pioneer()
drone.set_logger(False)             # отключаем вывод ответа на каждую
↪  команду — их будут сотни

try:
    rainbow(drone)
    running_light(drone, r=0, g=0, b=1)
    pulse(drone, r=1, g=0.4, b=0)   # жёлтый = красный + немного зелёного
finally:
    drone.led_control(ALL_LEDS)     # гасим светодиоды
    drone.close_connection()
```

Что происходит. Три эффекта. `rainbow()`: модуль `colorsys` переводит оттенок (от 0 до 1) в RGB, а сдвиг `led / LED_COUNT` даёт соседним светодиодам разные цвета, и «радуга» бежит по кругу. `running_light()`: каждый шаг гасит всё и зажигает следующий светодиод. `pulse()`: яркость каждого канала умножается на уровень `level`, так меняется яркость без смены оттенка. `set_logger(False)` отключает печать ответа на каждую

команду — в анимации их сотни. В примере `LED_COUNT = 4`; на своём дроне уточните число светодиодов (в симуляторе адресуются светодиоды 0 и 1).

Цвета смешиваются аддитивно: `r=1, g=0.4` даёт жёлтый, `r=1, b=1` — фиолетовый, все каналы по 1 — белый. Такие сочетания удобно использовать, чтобы показывать состояние программы в полёте.

ЗАДАНИЕ 16.26 · Светофор и двоичное число

Сделайте «светофор»: красный 3 с, жёлтый 1 с, зелёный 3 с, мигающий зелёный. Напишите функцию `show_number(n)`, показывающую число от 0 до 15 в двоичном виде на четырёх светодиодах (в симуляторе — от 0 до 3 на двух). Свяжите цвет с высотой по дальномеру: ниже 0.5 м — зелёный, выше 1.5 м — красный.

Что должно получиться: светофор переключается по расписанию; `show_number(5)` зажигает 1-й и 3-й светодиоды; цвет меняется вместе с высотой.

В симуляторе: миссии Мини 2 SDK2: 23–25 в панели «**Обучение**».

16.17 Груз: магнитный захват и механический захват

Цель раздела: брать и сбрасывать груз методами, которые есть именно у вашего дрона.

В зависимости от дрона груз берёт магнит или механический захват. SDK 2 управляет обоими.

Магнитный захват груза (Базовый Пионер)

`cargo_grab()` — включить магнит. `cargo_release()` — выключить магнит. `cargo_set(grab)` — то же с булевым аргументом: `True` включает, `False` выключает. Все три возвращают `True`, если успешно; если плата не поддерживает магнитный захват, печатают сообщение и возвращают `False`.

Механический захват (Пионер Мини 2)

`grab_close(movement_time=0, velocity=20)` — закрыть захват.
`grab_open(movement_time=0, velocity=20)` — открыть захват.
`grab_stop()` — остановить движение захвата.
`movement_time` — сколько секунд двигать захват; 0 — двигать до упора. `velocity` — скорость в процентах, от 0 до 100. Все три метода возвращают `bool`.

python-onboard/pioneer_sdk2/17_delivery_grab.py

```
"""Доставка груза захватом «Пионер Мини 2»: забрать в точке A и отпустить в
↪  точке B."""
from pioneer_sdk2 import Pioneer
import time
```

```
POINT_A = (0.0, 0.0)      # где лежит груз (дрон стартует рядом)
POINT_B = (1.5, 0.5)      # куда доставить
CRUISE_Z = 1.0            # высота перелёта, м
DROP_Z = 0.4              # высота сброса, м


def go(drone, x, y, z):
    drone.go_to_local_point(x=x, y=y, z=z, yaw=0, time=0)
    while not drone.point_reached():
        time.sleep(0.1)


drone = Pioneer()

try:
    # 1. На земле: открываем захват, ждём, пока груз вложат, и закрываем
    drone.grab_open()                          # movement_time=0 — до упора
    drone.led_control(r=1, g=0.4)              # жёлтый: «вложите груз»
    print("Вложите груз, через 5 секунд захват закроется")
    time.sleep(5)
    drone.grab_close(movement_time=0, velocity=60)
    time.sleep(2)
    drone.led_control(g=1)

    # 2. Перелёт
    drone.arm()
    drone.takeoff()
    go(drone, *POINT_A, CRUISE_Z)
    go(drone, *POINT_B, CRUISE_Z)
    go(drone, *POINT_B, DROP_Z)                # снижаемся над точкой B

    # 3. Сброс
    drone.led_control(b=1)
    drone.grab_open(movement_time=2, velocity=100)
    time.sleep(2.5)
    print("Груз доставлен")

    # 4. Возврат
    go(drone, *POINT_B, CRUISE_Z)
    go(drone, *POINT_A, CRUISE_Z)
    drone.land()
finally:
    drone.grab_stop()
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.led_control()
    drone.close_connection()
```

Что происходит. Сценарий из этапов «погрузка — перелёт — сброс — возврат». На земле захват открывается (`movement_time=0` — до упора), светодиод жёлтый просит вложить груз, через 5 секунд захват закрывается со скоростью 60%. Затем дрон взлетает, летит в точку A и в точку B на высоте перелёта и снижается над точкой B: груз сбрасывают с малой высоты, а не с большой. Каждый этап подсвечен своим цветом — по светодиодам видно, на каком шаге программа. В `finally` вызывается `grab_stop()`, чтобы привод не продолжал движение.

Важные детали захвата

- При `movement_time > 0` метод ждёт столько секунд и только потом возвращается; при `movement_time = 0` возвращается сразу, а привод продолжает двигаться до упора. Поэтому после `grab_close()` без времени сделайте паузу, а при необходимости остановите привод `grab_stop()`. (В симуляторе захват срабатывает мгновенно, а `movement_time` не ждёт.)
- Скорость `velocity` вне диапазона 0–100 исключения в 0.11.0 не вызывает: библиотека печатает сообщение и возвращает `False`. Симулятор в этом случае печатает сообщение и бросает `ValueError`. Держите скорость в диапазоне.
- Скорость по умолчанию зависит от версии: в 0.16 (справочник симулятора) она 20, в 0.11.0 — 100. Чтобы поведение не зависело от версии, всегда указывайте `velocity` явно (в примере: 60 при захвате, 100 при сбросе).
- Если плата захват не поддерживает, методы печатают «Модуль захвата не поддерживается» и возвращают `False`, а программа продолжает работу: проверяйте результат.
- В симуляторе `grab_close()` подхватывает груз под дроном, как магнит Базового, а `cargo_*` у Мини 2 возвращают `False`: магнита у этой модели нет.

ЗАДАНИЕ 16.27 · Доставка груза

Измените пример 17: дрон берёт груз, стоящий под точкой старта, перелетает в $(1.5, 1)$ на высоту 1 м и отпускает груз, затем возвращается домой через `rtl()` (с ожиданием посадки, как в разделе про возврат). Проверяйте результат каждого `grab_*`; если `grab_close()` вернул `False`, не взлетайте.

Что должно получиться: в симуляторе груз перемещается в новую точку; при неисправном захвате программа печатает причину и не взлетает.

В симуляторе: миссии Мини 2 SDK2: 26–27 в панели «**Обучение**».

ЗАДАНИЕ 16.28 · Захват по времени

Закройте захват на 0.5 с, потом на 1 с с `velocity` 50, замерьте `time.time()` вокруг каждого вызова. Затем вызовите `grab_close()` без времени и сразу `grab_stop()`.

Что должно получиться: на дроне время вызова первых двух близко к 0.5 и 1 с, а вызов без времени возвращается практически мгновенно, и привод останавливается по `grab_stop()`; в симуляторе все три вызова возвращаются сразу.

16.18 Пульт RC

Цель раздела: управлять дроном «как с пульта» из программы (например, с геймпада) и читать тумблер.

Помимо программного управления, дроном можно управлять «как с пульта»: SDK умеет отправлять в автопилот значения каналов и читать канал пульта.

Пульт: подготовка и ограничения

Чтобы автопилот слушал каналы от программы, нужно выставить параметры: `Copter_man_rcMode0=6.0`, `Copter_man_rcMode1=3.0`, `Copter_man_rcMode2=3.0`, `Copter_flyWithoutRc=1.0`, `SensorMux_rc=2.0`. Запомните прежние значения и возвращайте их в конце программы. Эти параметры меняют поведение автопилота, поэтому проверяйте программу на привязанном дроне, готовом к перехвату.

Методы RC (по 0.11.0)

`send_rc_channels(channel_1=0, channel_2=0, channel_3=0, channel_4=0, channel_5=1, channel_6=0, channel_7=1, channel_8=0)` — отправить значения восьми каналов; каждое число от -1 до 1. Возвращает `bool`. Каналы нужно отправлять *постоянно*: автопилоту нужен непрерывный сигнал.
`get_rc_channel()` — значение одного канала пульта (целое число) или `None`. По описанию Pioneer Code это 8-й канал (тумблер SWA).
`set_rc_trigger(callback)` — вызывать функцию `callback(value)`, когда значение этого канала меняется.
`rc_sdk1_to_sdk2(...)` — перевод значений каналов из SDK 1 в SDK 2 (см. предупреждение ниже).

Каналы. Первые четыре канала — стики, каждый от -1 до 1; каналы 5–8 — тумблеры пульта.

Канал	Орган	Значения
`channel_1`	правый стик, крен	-1 влево, 0 центр, $+1$ вправо
`channel_2`	правый стик, тангаж	-1 вперёд, 0 центр, $+1$ назад
`channel_3`	левый стик, газ	$+1$ вверх, 0 центр, -1 вниз
`channel_4`	левый стик, рыскание	$+1$ налево, 0 центр, -1 направо
`channel_5`	тумблер SWC	0 / 1 / 2 (вверх / центр / вниз)
`channel_6`	тумблер SWD	0 / 2
`channel_7`	тумблер SWB	0 / 1 / 2
`channel_8`	тумблер SWA	0 / 2

По умолчанию `channel_5` и `channel_7` равны 1, остальные — 0. Значения стиков SDK умножает на 32760, тумблеров — на 32767 и округляет.

Плавные стики. Резкий рывок стика — резкий манёвр. Пример отправляет каналы каждые 50 мс даже когда стики в нейтрали и плавно ведёт стик по трапециевидному профилю: разгон, удержание, торможение.



python-onboard/pioneer_sdk2/12_rc_ramp.py

```python
from pioneer_sdk2 import Pioneer
import time

RC_PARAMETERS = {                       # без этих значений автопилот
↪  проигнорирует «виртуальный пульт»
    "Copter_man_rcMode0": 6.0,
    "Copter_man_rcMode1": 3.0,
    "Copter_man_rcMode2": 3.0,
    "Copter_flyWithoutRc": 1.0,
    "SensorMux_rc": 2.0,
}
PERIOD = 0.05                           # каналы шлём 20 раз в секунду —
↪  автопилоту нужен непрерывный сигнал


def send_sticks(drone, roll=0.0, pitch=0.0, throttle=0.0, yaw=0.0):
    """Стики в диапазоне -1..1; тумблеры в «полётном» положении."""
    drone.send_rc_channels(channel_1=roll, channel_2=pitch,
    ↪  channel_3=throttle, channel_4=yaw,
                           channel_5=1, channel_6=0, channel_7=1,
                           ↪  channel_8=0)


def ramp(drone, axis, target, duration):
    """Плавно двигает один стик от 0 до target, держит и плавно возвращает в
    ↪  0.

    Резкий рывок стика = резкий манёвр. Линейное нарастание делает полёт
    ↪  мягким.
    """
    steps = max(1, int(duration / 3 / PERIOD))
    profile = ([target * i / steps for i in range(steps)] +       # разгон
               [target] * steps +                                 # удержание
               [target * (steps - i) / steps for i in range(steps + 1)])  #
               ↪  торможение
    for value in profile:
        send_sticks(drone, **{axis: value})
        time.sleep(PERIOD)


drone = Pioneer()
old = {name: drone.get_param(name, update=True) for name in RC_PARAMETERS}

try:
    for name, value in RC_PARAMETERS.items():
        drone.set_param(name, value)

    drone.arm()
```

```
    drone.takeoff()
    ramp(drone, "pitch", -0.3, duration=3)    # вперёд (pitch: -1 — вперёд)
    ramp(drone, "yaw", 0.4, duration=3)       # поворот налево
    ramp(drone, "roll", 0.3, duration=3)      # вправо
    for _ in range(20):                       # секунда нейтрали перед
    ↪  посадкой
        send_sticks(drone)
        time.sleep(PERIOD)
    drone.land()
finally:
    send_sticks(drone)
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    for name, value in old.items():           # возвращаем параметры
    ↪  автопилота
        if value is not None:
            drone.set_param(name, value)
    drone.close_connection()
```

Что происходит. Исходные параметры запоминаются до изменения и возвращаются в `finally`. Дрон взлетает командами SDK, а дальше летит стиками. Функция `ramp()` строит профиль из трёх участков одинаковой длительности. Запись `**{axis: value}` распаковывает словарь в именованные аргументы: одна функция управляет любым стиком по его имени. Знаки: `pitch` -1 — вперёд, положительный `yaw` — поворот налево. Частота 20 раз в секунду взята из сборника; сколько нужно именно вашему автопилоту, в описании не сказано — подберите на дроне.

> **RC в симуляторе**
>
> Физический пульт к симулятору не подключён. Автопилот симулятора слушает `send_rc_channels`, только если выставлен набор параметров из таблицы выше и дрон в воздухе: тогда стики превращаются в команды скорости (тангаж -1 — вперёд со скоростью 1 м/с, газ даёт вертикальную скорость до 0.5 м/с). Канал 8 читается обратно: `get_rc_channel()` возвращает то, что программа отправила в `channel_8`, а `set_rc_trigger` срабатывает при его изменении.

Чтение канала пульта. `get_rc_channel()` вернёт значение сразу. `set_rc_trigger(callback)` запускает слежение в фоновом потоке: поток проверяет канал примерно десять раз в секунду и вызывает `callback(value)`, когда значение изменилось (в том числе первый раз, когда оно прочитано). Функция выполняется в служебном потоке — правило прежнее: она короткая, ставит флаг, а действие делает основной цикл. Снять триггер в SDK нельзя — он живёт до закрытия соединения (по 0.11.0), поэтому не регистрируйте триггер в цикле.

> **rc_sdk1_to_sdk2 в 0.11.0 считает с ошибкой; в 0.16 не проверялось**
>
> Задумано, видимо, перевести числа 1000–2000 из SDK 1 в $-1\ldots1$ по формуле $(x - 1500)/500$. Но в исходниках 0.11.0 скобок нет (`x - 1500 / 500`), по-

этому из каждого значения вычитается 3, а не пересчитывается шкала; кроме того, третий и четвёртый каналы считаются от уже изменённых первых двух. Например, `rc_sdk1_to_sdk2(1500, 1500, 1500, 1500, 2000)` вернёт `(1497.0, 1497.0, 1494.0, 1494.0, 1)`, а не нули. Симулятор повторяет эту формулу как есть. Не используйте функцию без проверки; пересчитывайте шкалу сами: `(x - 1500) / 500`. Пятый канал она переводит правильно: 2000 даёт 1, любое другое число — 0.

Режимы программы по тумблеру. Пока программа идёт, тумблер можно использовать как переключатель режимов:

python-onboard/pioneer_sdk2/13_rc_trigger_modes.py

```python
from pioneer_sdk2 import Pioneer
import time

# Тумблер SWA (8-й канал) переключает «режим» программы:
#   0 — дежурный (мигаем синим), 2 — активный (горим зелёным, печатаем
↪  телеметрию)
mode = "standby"


def on_switch(channel_8):
    """Вызывается в отдельном потоке только при ИЗМЕНЕНИИ положения
    ↪  тумблера."""
    global mode
    mode = "active" if channel_8 == 2 else "standby"
    print("Тумблер SWA =", channel_8, "-> режим", mode)


drone = Pioneer()
drone.set_logger(False)
drone.set_rc_trigger(on_switch)                # регистрируем обработчик
↪  тумблера
print("Начальное значение 8 канала:", drone.get_rc_channel())

try:
    blink = False
    while True:                                # основной цикл не блокируется
    ↪  обработчиком
        if mode == "active":
            drone.led_control(g=1)
            print("высота:", drone.get_dist_sensor_data())
        else:
            blink = not blink
            drone.led_control(b=1 if blink else 0)
        time.sleep(0.5)
except KeyboardInterrupt:
    pass
finally:
    drone.led_control()
    drone.close_connection()
```

Что происходит. `set_rc_trigger(func)` регистрирует функцию, которая вызывается в отдельном потоке *при изменении* положения тумблера SWA и получает значение 8-го канала. Обработчик только меняет глобальную переменную `mode`, а основной цикл решает, что делать в каждом режиме: «событие меняет состояние — цикл действует по состоянию». `get_rc_channel()` читает текущее значение напрямую.

ЗАДАНИЕ 16.29 · Тумблер как кнопка

Добавьте третий режим, который запускает миссию из примера 05 по тумблеру. Сделайте «аварийную кнопку»: при переключении тумблера в полёте — немедленная посадка (флаг в обработчике, `land()` в основном цикле). Тумблер «нажимайте» программой: `send_rc_channels(channel_8=положение)`.

Что должно получиться: тумблер переключает режимы без остановки основного цикла; при аварийном переключении дрон садится из основного цикла, а не из обработчика.

В симуляторе: миссии Мини 2 SDK2: 29 в панели «**Обучение**».

ЗАДАНИЕ 16.30 · Пересчёт шкалы SDK 1

Напишите свою функцию `rc1_to_rc2(ch1, ch2, ch3, ch4, ch5)`, которая переводит каналы 1000–2000 в $-1 \dots 1$ по формуле $(x - 1500)/500$ и пятый канал в 1 при 2000, иначе 0. Проверьте на значениях 1000, 1500, 2000.

Что должно получиться: 1000 даёт -1, 1500 даёт 0, 2000 даёт 1; для пятого канала 2000 даёт 1, остальные — 0.

ЗАДАНИЕ 16.31 · Пульт по сценарию

Опишите управление списком шагов «(канал, значение, секунды)»: например, тангаж -0.3 на 2 с, поворот 0.4 на 1 с, нейтраль на 1 с. Цикл отправляет `send_rc_channels` каждые 0.05 с по текущему шагу, в конце отправляет нули. Параметры RC верните в `finally`.

Что должно получиться: дрон по очереди летит вперёд, поворачивает и зависает; после выхода из цикла отправлены нулевые значения, параметры автопилота прежние.

В симуляторе: миссии Мини 2 SDK2: 28 в панели «**Обучение**».

16.19 Камера и видео в браузере

Цель раздела: получить кадр с камеры дрона, показать его в браузере и нарисовать на нём телеметрию.

На дроне нет экрана, поэтому `cv2.imshow` из главы 8 здесь не работает. Кадры берут из `Camera`, а смотрят в браузере — через `ImageViewer`:

python-onboard/pioneer_sdk2/sdk2_06_camera_stream.py

```python
import cv2
from pioneer_sdk2 import Camera, CameraType, ImageViewer

camera = Camera(camera_type=CameraType.MAIN)   # передняя камера
viewer = ImageViewer()  # видеопоток для браузера
print("Смотрите в браузере: http://10.42.0.1:8889/stream")

count = 0
try:
    while True:
        frame = camera.get_cv_frame()  # numpy-массив, цвета BGR
        count += 1
        h, w = frame.shape[:2]
        cv2.putText(frame, f"{w}x{h} #{count}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
        viewer.imshow(name="stream", frame=frame)
except KeyboardInterrupt:  # Ctrl+C в терминале
    print("Остановлено")
finally:
    camera.stop()
    viewer.close()
```

Что происходит. `Camera(camera_type=CameraType.MAIN)` открывает переднюю камеру; у Мини 2 есть ещё `CameraType.OPT` — нижняя камера оптического потока. `get_cv_frame()` возвращает кадр — массив numpy с цветами в порядке BGR, тот же формат, что в главе про камеру. `viewer.imshow(name="stream", frame=frame)` отправляет кадр в видеопоток с именем `stream`; на компьютере, подключённом к Wi-Fi дрона, он открывается в браузере по адресу `http://10.42.0.1:8889/stream/` (видео идёт по WebRTC; если дрон подключён к вашей сети — подставьте его адрес). Имя потока — часть адреса, так что программа может показывать несколько потоков сразу. Путь кадра — на рисунке 28.

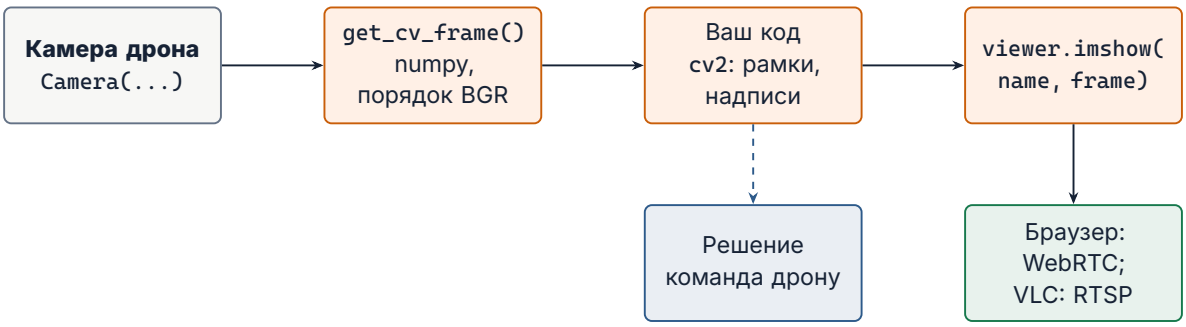


Рис. 28. Путь кадра на борту Мини 2. Экрана у дрона нет, поэтому кадр показывает браузер; тот же кадр можно использовать для решения (раздел о компьютерном зрении).

Тот же поток доступен по RTSP, например, для VLC: `rtsp://10.42.0.1:8554/stream`; адрес вида `rtsp://...:8889` не работает.

Останавливайте и поток, и камеру

В конце программы нужно вызвать и `viewer.close()`, и `camera.stop()`. Если забыть `camera.stop()`, поток приёма кадров продолжит работать, и программа после основного цикла не завершится — такая ошибка была даже в примере из документации Геоскана.

Окно пишет «stream not found»

Это значит, что программа сейчас ничего не публикует: она ещё не запустилась, уже завершилась или имя потока в адресе не совпадает с первым аргументом `imshow`. Страница сама переподключается каждые несколько секунд — достаточно запустить программу.

Camera

`Camera(camera_type=CameraType.MAIN)` — открыть камеру. Типы камер (`CameraType.MAIN`, `CameraType.OPT`) формируются из настроек платы: на плате без нижней камеры `OPT` не будет.
`camera.get_cv_frame(timeout=5.0)` — следующий кадр (numpy, BGR); нет кадра за `timeout` секунд — обычно `TimeoutError`, но на всякий случай проверяйте и `None`.
`camera.stop()` — остановить захват. Ставьте в `finally`.

Кадры копятся в очереди камеры, поэтому цикл с `get_cv_frame()` идёт со скоростью камеры, а если ваша обработка медленнее, вы начинаете отставать от реальности. После резкого изменения сцены (например, наклона камеры) первые кадры могут оказаться старыми: пропустите несколько. На платах с драйвером gstreamer можно открыть несколько камер сразу (переднюю и нижнюю).

ImageViewer

`ImageViewer()` — создать показ.
`viewer.imshow(name, frame, fps=30)` — отправить кадр в поток `name`; поток создаётся при первом кадре, и размер этого кадра задаёт размер потока, так что все следующие кадры того же имени должны быть того же размера. `viewer.close()` — закрыть все потоки. Ставьте в `finally`.

Камера на компьютере: драйвер rtsp

Библиотека выбирает драйвер камеры по настройкам платы. На бортовых платах (Мини 2, Radxa Zero 3W) кадры идут из общей памяти по драйверу `gstreamer`, и `ImageViewer` доступен. На платах с драйвером `rtsp` (так настроена конфигурация, которая поставляется с пакетом для Raspberry Pi Zero) `Camera(camera_type, camera_ip="10.42.0.1:8554")` читает RTSP-поток по адресу вида `rtsp://10.42.0.1:8554/<имя камеры>`; адрес обязательно с портом, иначе `ValueError`. `ImageViewer` в этом режиме нет: показывать кадры придётся самому, например `cv2.imshow` на компьютере.

Минимальная трансляция. Самый короткий вариант — 30 секунд берём кадры и отправляем в окно:

python-onboard/pioneer_sdk2/14a_camera_stream.py

```
from pioneer_sdk2 import ImageViewer, Camera # импортируем классы из
↪  библиотеки pioneer_sdk2
import time                                  # библиотека time содержит
↪  функции для работы со временем

video_drone_1 = ImageViewer()                # создаем экземпляр класса
↪  ImageViewer
camera_drone_1 = Camera()                    # создаем экземпляр класса
↪  Camera

my_time = time.time()                        # объявляем переменную my_time,
↪  присваиваем текущее время (в секундах с 1970 года)
cycle_time = 30                              # объявляем переменную
↪  cycle_time, присваиваем желаемую длительность цикла

while time.time() - my_time < cycle_time:        # запускаем цикл, код будет
↪  повторяться, пока условие верно
    frame = camera_drone_1.get_cv_frame()        # получаем кадр и
    ↪  присваиваем данные переменной frame
    video_drone_1.imshow("video", frame, fps=30) # запускаем трансляцию,
    ↪  содержит аргументы:
                                                 # video - название
                                                 ↪  трансляции
                                                 # frame - переменная с ранее
                                                 ↪  полученным кадром
                                                 # fps=30 - количество кадров
                                                 ↪  в секунду при передаче
                                                 ↪  видео

                                                 # трансляция выполняется по
                                                 ↪  адресу:
                                                 ↪  10.42.0.1:8889/video
                                                 # ip   - по умолчанию
                                                 ↪  10.42.0.1 или
                                                 ↪  скопировать из раздела
                                                 ↪  хот-спот на компьютере
                                                 # 8889 - порт трансляции
                                                 # name - название трансляции

                                                 # при подключения
                                                 ↪  квадрокоптера к вашей
                                                 ↪  сети:
                                                 # трансляция выполняется по
                                                 ↪  адресу: ваш
                                                 ↪  ip:8889/video

video_drone_1.close()                            # останавливаем видеопоток
camera_drone_1.stop()                            # останавливаем камеру: без
↪  этого поток захвата кадров

                                                 # продолжит работать и
                                                 ↪  программа не завершится
```

Рисуем поверх кадра (OSD). Кадр — обычный массив numpy, поэтому на нём можно рисовать средствами OpenCV: телеметрия, перекрестие, подписи.

python-onboard/pioneer_sdk2/14_camera_osd.py

```
from pioneer_sdk2 import Pioneer, Camera, ImageViewer
import cv2
import time

drone = Pioneer()
camera = Camera()                  # основная камера (CameraType.MAIN по
↪ умолчанию)
viewer = ImageViewer()             # поток будет доступен по адресу
↪ http://10.42.0.1:8889/osd/
print("Окно просмотра: http://10.42.0.1:8889/osd/")
drone.set_logger(False)

fps = 0.0
last = time.monotonic()


def draw_osd(frame, lines):
    """Рисует полупрозрачную подложку и строки телеметрии в левом верхнем
    ↪ углу."""
    overlay = frame.copy()
    cv2.rectangle(overlay, (5, 5), (330, 20 + 25 * len(lines)), (0, 0, 0),
    ↪ -1)
    cv2.addWeighted(overlay, 0.5, frame, 0.5, 0, dst=frame)          # 50%
    ↪ прозрачности
    for i, text in enumerate(lines):
        cv2.putText(frame, text, (12, 30 + 25 * i), cv2.FONT_HERSHEY_SIMPLEX,
                    0.6, (255, 255, 255), 1, cv2.LINE_AA)
    # перекрестие в центре кадра
    h, w = frame.shape[:2]
    cv2.line(frame, (w // 2 - 15, h // 2), (w // 2 + 15, h // 2), (0, 255,
    ↪ 0), 1)
    cv2.line(frame, (w // 2, h // 2 - 15), (w // 2, h // 2 + 15), (0, 255,
    ↪ 0), 1)


try:
    while True:
        try:
            frame = camera.get_cv_frame(timeout=1.0)
        except TimeoutError:                          # кадр не пришёл вовремя
        ↪ — просто ждём следующий
            continue
        if frame is None:
            continue

        now = time.monotonic()
        fps = 0.9 * fps + 0.1 / max(now - last, 1e-3)   # сглаженный FPS
        last = now

        rng = drone.get_dist_sensor_data()
        draw_osd(frame, [
            f"STATE  {drone.get_fly_state().name}",
```

```
            f"RANGE  {rng:.2f} m" if rng is not None else "RANGE  --",
            f"BATT   {drone.get_battery_status()}",
            f"FPS    {fps:4.1f}",
        ])
        viewer.imshow("osd", frame, fps=30)   # трансляция:
        ↪ 10.42.0.1:8889/osd
except KeyboardInterrupt:
    pass
finally:
    viewer.close()
    camera.stop()
    drone.close_connection()
```

Что происходит. Если кадр не пришёл вовремя, `get_cv_frame` может выбросить `TimeoutError` или вернуть `None`, поэтому пример обрабатывает оба случая. Полупрозрачная подложка: прямоугольник рисуется на копии кадра и смешивается с оригиналом через `cv2.addWeighted`. Частота кадров сглажена экспоненциальным средним: $\text{fps} \leftarrow 0.9\,\text{fps} + 0.1/\Delta t$. В `finally` закрываются и трансляция, и камера.

ЗАДАНИЕ 16.32 · Две камеры в одном окне

Откройте обе камеры Мини 2 (`CameraType.MAIN` и `CameraType.OPT`) и транслируйте их в два потока `front` и `down`. На каждом кадре подпишите, откуда он (`cv2.putText`). Закройте всё в `finally`.

Что должно получиться: по адресам `.../front/` и `.../down/` (в симуляторе — два окна) идёт видео с двух камер, подписи не путаются; после `Ctrl+C` программа останавливается без трассировки.

В симуляторе: миссии Мини 2 SDK2: 30–32 в панели «**Обучение**».

Кадры не приходят

Если `get_cv_frame` всегда выбрасывает `TimeoutError`, возможно, завис сервис захвата видео (в журнале ядра `dmesg` — `not active buffer, skip frame`). Перезапустите его на дроне: `sudo systemctl restart media-server`.

16.20 Наклон камеры: сервопривод

Цель раздела: направлять камеру вниз или вперёд программой и получать корректные кадры после поворота.

Передняя камера Мини 2 стоит на сервоприводе и наклоняется вверх-вниз. Им управляет класс `ServoCamera`:

ServoCamera

`servo = ServoCamera()` — подключиться к сервоприводу (SDK проверяет, что он есть на борту; на плате без него — `ValueError`).
`servo.set_angle(angle, priority=ServoPriority.LOW)` — наклонить камеру: `angle` в градусах от -80 (почти вниз) до 30 (вверх), 0 — прямо вперёд. Угол вне диапазона — `ValueError`. Возвращает `True`, если команда отправлена; `False`, если сервопривод не отвечает. `priority` — приоритет команды: `ServoPriority.HIGH`, `MEDIUM` или `LOW`.

python-onboard/pioneer_sdk2/sdk2_07_servo_camera.py

```python
import cv2
import time
from pioneer_sdk2 import Pioneer, Camera, CameraType, ImageViewer
from pioneer_sdk2 import ServoCamera

pioneer = Pioneer()
camera = Camera(camera_type=CameraType.MAIN)
servo = ServoCamera()              # сервопривод наклона камеры
viewer = ImageViewer()
in_air = False


def show(seconds, angle):
    """Показывать кадры seconds секунд с подписью угла камеры."""
    start = time.time()
    while time.time() - start < seconds:
        frame = camera.get_cv_frame()
        cv2.putText(frame, f"angle {angle}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
        viewer.imshow(name="stream", frame=frame)


try:
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    in_air = True
    pioneer.takeoff()

    # 0 - камера смотрит вперёд, -80 - почти вниз, 30 - вверх.
    for angle in (0, -45, -80, 0):
        print("Наклон камеры:", angle)
        servo.set_angle(angle)
        show(2, angle)

    pioneer.land()
    in_air = False
finally:
    if in_air:
        pioneer.land()
    pioneer.disarm()
    camera.stop()
    viewer.close()
    pioneer.close_connection()
```

Что происходит. Дрон взлетает и по очереди наклоняет камеру: вперёд, на 45° вниз, почти отвесно вниз и обратно. После каждой команды программа две секунды показывает кадры с подписью угла — видно, как в кадр входит площадка под дроном. `set_angle` возвращается сразу, а сервопривод поворачивается за доли секунды, поэтому первые кадры после команды ещё «доезжают». Наклон меняет и кадры `Camera(CameraType.MAIN)`: камеру, смотрящую вниз, можно использовать, чтобы искать метки на полу, не сажая дрон.

SDK только передаёт приоритет вместе с углом службе сервопривода на борту; как она разрешает спор между командами разных приоритетов, описание SDK не говорит. Если наклоном управляет одна программа, оставляйте `LOW` по умолчанию. Если служба сервопривода недоступна (программа запущена не на дроне), `set_angle` не бросает исключение, а возвращает `False`.

Серия снимков под разными углами

python-onboard/pioneer_sdk2/15_servo_panorama.py

```python
from pioneer_sdk2 import Camera, ServoCamera
import cv2
import time

ANGLES = [30, 10, -10, -30, -50, -80]    # допустимый диапазон сервопривода:
↪ -80..30 градусов
THUMB_WIDTH = 320                        # ширина миниатюры в итоговом
↪ «контактном листе»

camera = Camera()
servo = ServoCamera()


def fresh_frame(timeout=5.0, skip=5):
    """Возвращает свежий кадр: первые несколько кадров из очереди могут быть
    ↪ «старыми»."""
    frame = None
    for _ in range(skip):
        frame = camera.get_cv_frame(timeout=timeout)
    return frame


shots = []
try:
    for angle in ANGLES:
        if not servo.set_angle(angle):
            print("Сервопривод не принял угол", angle)
            continue
        time.sleep(1.5)                                  # ждём, пока камера
        ↪ повернётся и успокоится
        frame = fresh_frame()
        if frame is None:
            continue
        cv2.imwrite(f"tilt_{angle:+03d}.jpg", frame)
        h, w = frame.shape[:2]
```

```
        thumb = cv2.resize(frame, (THUMB_WIDTH, int(h * THUMB_WIDTH / w)))
        cv2.putText(thumb, f"{angle:+d} deg", (8, 25),
        ↪  cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
        shots.append(thumb)
        print("Снимок под углом", angle)

    if shots:
        sheet = cv2.vconcat(shots)                          # все миниатюры
        ↪  одна под другой
        cv2.imwrite("tilt_sheet.jpg", sheet)
        print("Сохранён tilt_sheet.jpg")
finally:
    servo.set_angle(0)
    camera.stop()
```

Что происходит. Кадры копятся в очереди, поэтому сразу после поворота можно получить «старый» снимок — `fresh_frame()` пропускает несколько кадров. Миниатюры масштабируются с сохранением пропорций и склеиваются `cv2.vconcat` в один «контактный лист»; формат `f"{angle:+03d}"` даёт имена вида `tilt_+30.jpg`, `tilt_-50.jpg`. В `finally` камера возвращается в горизонтальное положение. В симуляторе `cv2.imwrite` — заглушка: программа проходит до конца, но файлы не сохраняются.

ЗАДАНИЕ 16.33 · Метки на полу

Взлетите, наклоните камеру Мини 2 на $-80°$ и показывайте поток через `ImageViewer` с подписью текущего угла. Затем верните камеру в $0°$ и сядьте. Если кадра нет 5 с, обработайте `TimeoutError` и сядьте.

Что должно получиться: в окне камеры виден пол под дроном и подпись `angle -80`; после возврата — снова вид вперёд; программа завершается посадкой.

В симуляторе: миссии Мини 2 SDK2: 33 в панели «**Обучение**».

ЗАДАНИЕ 16.34 · Плавный наклон

Наклоняйте камеру от $0°$ до $-80°$ шагом $10°$ каждые 0.5 с и обратно. Попробуйте угол -90 и обработайте исключение.

Что должно получиться: камера плавно опускается и поднимается; угол -90 вызывает `ValueError`, который программа ловит и печатает допустимый диапазон.

В симуляторе: миссии Мини 2 SDK2: 33 в панели «**Обучение**».

16.21 Запись видео и фото: RecorderControl

Цель раздела: записывать видео полёта и снимать фото в заданных точках, не расходуя на это кадры вашей программы.

Класс `RecorderControl` (Мини 2) пишет видео и снимает фото в галерею дрона. Запись идёт параллельно с полётом и не требует кадров в вашей программе.

RecorderControl (по сборнику Геоскана, 0.16)

`recorder = RecorderControl()` — подключиться к службе записи.
`recorder.get_camera_recording_configs(camera_type)` — список доступных режимов записи камеры (ширина, высота, fps).
`recorder.start_recording(camera_type, config, output_dir=None)` — начать запись видео; по умолчанию файлы кладутся в `/mnt/media/videos/`.
`recorder.stop_recording(camera_type)` — закончить запись.
`recorder.take_photo(file_name, camera_type, config, output_dir=None)` — сохранить снимок; расширение имени — `.jpg`, `.jpeg` или `.png`. Пустое имя или неверное расширение — `ValueError`, ошибка камеры — `RuntimeError`.

python-onboard/pioneer_sdk2/16_recorder_mission.py

```python
from pioneer_sdk2 import Pioneer, RecorderControl, CameraType
import time

PHOTO_POINTS = [(1, 0), (1, 1), (0, 1)]       # В этих точках делаем фото
HEIGHT = 1.2

drone = Pioneer()
recorder = RecorderControl()

configs = recorder.get_camera_recording_configs(CameraType.MAIN)   #
↪  доступные режимы: ширина, высота, fps
print("Режимы записи основной камеры:")
for i, config in enumerate(configs):
    print(f"  [{i}] {config}")
config = configs[0] if configs else None      # None — библиотека выберет
↪  первый доступный режим


def wait_point():
    while not drone.point_reached():
        time.sleep(0.1)


recording = False
try:
    recorder.start_recording(CameraType.MAIN, config)   # запись идёт
    ↪  параллельно с полётом
    recording = True
    drone.arm()
    drone.takeoff()

    for n, (x, y) in enumerate(PHOTO_POINTS, start=1):
        drone.go_to_local_point(x=x, y=y, z=HEIGHT, yaw=0, time=0)
        wait_point()
        time.sleep(1)                          # гасим раскачку перед снимком
        name = f"point_{n}_{time.strftime('%H%M%S')}.jpg"
        recorder.take_photo(name, CameraType.MAIN, config)
        print("Фото:", name)

    drone.go_to_local_point(x=0, y=0, z=HEIGHT, yaw=0, time=0)
    wait_point()
```

```
    drone.land()
finally:
    if recording:
        recorder.stop_recording(CameraType.MAIN)   # Видео попадёт в
        ↪ /mnt/media/videos/
    if drone.get_fly_state().name == "IN_SKY":
        drone.land()
    drone.close_connection()
```

Что происходит. Программа печатает режимы записи основной камеры и берёт первый; `config=None` означает, что библиотека выберет первый доступный. Видео пишется весь полёт, а в трёх вершинах после секундной паузы (чтобы погасить раскачку) делается фото. Флаг `recording` гарантирует, что `stop_recording()` вызывается, только если запись началась. Готовые файлы лежат в галерее медиа: `http://10.42.0.1:5000`, папки `photos` и `videos`.

RecorderControl в симуляторе

Симулятор проверяет аргументы, ведёт состояние записи (повторный старт — `RuntimeError`, `stop_recording()` без записи — `False`) и печатает «сделал бы», но файлов не создаёт. Режимы основной камеры там: 1280×720 и 640×360 при 30 кадрах в секунду.

ЗАДАНИЕ 16.35 · Облёт с видео

Поставьте в центр площадки предмет, облетите его по окружности (пример 07) и запишите видео облёта через `RecorderControl`. Фото сделайте в каждой четвёртой точке с подписью времени в имени файла.

Что должно получиться: в галерее дрона появилось видео всего облёта и фото с уникальными именами (в симуляторе — соответствующие строки журнала); запись останавливается и при ошибке (в `finally`).

В симуляторе: миссии Мини 2 SDK2: 34 в панели «**Обучение**».

16.22 Компьютерное зрение в контуре управления

Цель раздела: превратить изображение в команду дрону — найти цель на кадре, посчитать ошибку и подать её на регулятор.

До сих пор изображение с камеры только показывалось. В этом разделе оно *влияет на полёт*. Все три примера устроены по одной схеме замкнутого контура: камера → распознавание → ошибка → регулятор → команда скорости → движение дрона → новый кадр.

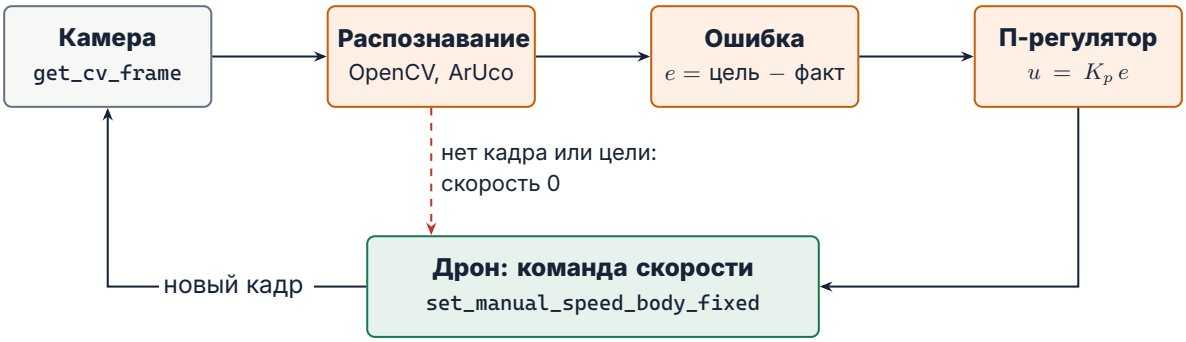


Рис. 29. Замкнутый контур компьютерного зрения: изображение становится ошибкой, ошибка — командой скорости, команда меняет положение дрона, и приходит новый кадр. Штриховая стрелка — безопасная ветка: нет цели — нулевая скорость.

Безопасность контура

Если кадра или цели нет, надо отправлять нулевую скорость: дрон зависнет, а не продолжит выполнять последнюю команду. Команды скорости короткие (`interval=0.3`), коэффициенты начинайте с малых значений: слишком большой коэффициент K_p раскачивает дрон — он «проскакивает» цель и колеблется; уменьшите K_p вдвое.

Поворот к цветному объекту

python-onboard/pioneer_sdk2/18_color_tracking_yaw.py

```python
from pioneer_sdk2 import Pioneer, Camera, ImageViewer
import cv2
import numpy as np
import time

# Диапазон цвета в HSV (здесь — насыщенный красный; у красного два диапазона оттенка: 0..10 и 170..180)
RED_RANGES = [((0, 120, 80), (10, 255, 255)), ((170, 120, 80), (180, 255, 255))]
MIN_AREA = 800          # меньшие пятна считаем шумом, пикс²
YAW_KP = 0.004          # коэффициент П-регулятора: рад/с на пиксель ошибки
MAX_YAW_RATE = 0.6      # ограничение скорости поворота, рад/с
DEAD_ZONE = 20          # пикселей: внутри этой зоны цель считаем «по центру»


def find_color_blob(frame):
    """Возвращает (cx, cy, area) самого крупного пятна заданного цвета или None."""
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = np.zeros(hsv.shape[:2], dtype=np.uint8)
    for low, high in RED_RANGES:
        mask |= cv2.inRange(hsv, np.array(low), np.array(high))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))  # убираем «соль»
```

```
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
    ↪  cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        return None
    biggest = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(biggest)
    if area < MIN_AREA:
        return None
    m = cv2.moments(biggest)
    return int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"]), area


def yaw_command(cx, frame_width):
    """П-регулятор: чем дальше цель от центра, тем быстрее поворот."""
    error = frame_width / 2 - cx                # >0 — цель слева от центра
    if abs(error) < DEAD_ZONE:
        return 0.0
    return float(np.clip(YAW_KP * error, -MAX_YAW_RATE, MAX_YAW_RATE))


drone = Pioneer()
camera = Camera()
viewer = ImageViewer()
print("Окно просмотра: http://10.42.0.1:8889/color/")
drone.set_logger(False)

try:
    drone.arm()
    drone.takeoff()
    drone.go_to_local_point(x=0, y=0, z=1.2, yaw=0, time=0)
    while not drone.point_reached():
        time.sleep(0.1)

    while True:
        try:
            frame = camera.get_cv_frame(timeout=0.5)
        except TimeoutError:
            frame = None
        if frame is None:
            drone.set_manual_speed_body_fixed(0, 0, 0, 0, 0.3)   # нет кадра
            ↪  — зависаем
            continue

        blob = find_color_blob(frame)
        yaw_rate = 0.0
        if blob is not None:
            cx, cy, area = blob
            yaw_rate = yaw_command(cx, frame.shape[1])
            cv2.circle(frame, (cx, cy), 10, (0, 255, 0), 2)
            cv2.putText(frame, f"yaw_rate={yaw_rate:+.2f}", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
        drone.set_manual_speed_body_fixed(0, 0, 0, yaw_rate, 0.3)
        viewer.imshow("color", frame)   # трансляция: 10.42.0.1:8889/color
except KeyboardInterrupt:
    pass
finally:
    if drone.get_fly_state().name == "IN_SKY":
```

```
            drone.set_manual_speed_body_fixed(0, 0, 0, 0, 0.3)
            drone.land()
    viewer.close()
    camera.stop()
    drone.close_connection()
```

Что происходит. Цвет ищется в пространстве HSV: оттенок (H), насыщенность (S) и яркость (V) описывают цвет так, что порог меньше зависит от освещения. У красного два диапазона оттенка на краях шкалы (0–10 и 170–180), их маски объединяются операцией `|=`. Морфологическое открытие `MORPH_OPEN` удаляет одиночные шумовые пиксели. Центр масс пятна считается через моменты: $c_x = m_{10}/m_{00}$, $c_y = m_{01}/m_{00}$. П-регулятор: $\omega = K_p\,(w/2 - c_x)$, ограниченный `MAX_YAW_RATE`; мёртвая зона `DEAD_ZONE` убирает дрожание около центра. Знак: цель левее центра — ошибка положительна — `yaw_rate > 0`, то есть поворот влево. Если кадра нет, отправляется нулевая скорость.

ЗАДАНИЕ 16.36 · Другой цвет, высота и дистанция

Подберите диапазон HSV для зелёного или синего предмета (подсказка: напишите программу-«пипетку», печатающую HSV центра кадра). Добавьте управление по высоте: держите объект по центру и по вертикали с помощью `vz`. Добавьте дистанцию по площади пятна: `vy` пропорционально $(S_{\text{цель}} - S)$.

Что должно получиться: дрон висит и плавно поворачивается вслед за предметом выбранного цвета; при потере предмета останавливается; поток `.../color/` показывает найденный центр.

ArUco-маркеры: распознавание и расстояние

python-onboard/pioneer_sdk2/19_aruco_distance.py

```
"""Поиск ArUco-маркеров и оценка расстояния до них.

Нужен файл data.yml с параметрами камеры — его создаёт скрипт калибровки
(camera_calibration.py из репозитория примеров Pioneer-SDK2).
"""
from pioneer_sdk2 import Camera, ImageViewer
import cv2
import numpy as np

MARKER_SIZE = 0.10      # длина стороны напечатанного маркера, м — измерьте
↪ линейкой!

# Углы маркера в его собственной системе координат (центр маркера — начало
↪ отсчёта)
half = MARKER_SIZE / 2
MARKER_POINTS = np.array([(-half,  half, 0), (half,  half, 0),
                          (half, -half, 0), (-half, -half, 0)],
                          ↪ dtype=np.float32)
```

```
def load_calibration(path="data.yml"):
    storage = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
    matrix = storage.getNode("mtx").mat()
    dist = storage.getNode("dist").mat()
    storage.release()
    if matrix is None:
        raise FileNotFoundError("Нет файла калибровки " + path)
    return matrix, dist


camera_matrix, dist_coeffs = load_calibration()
detector = cv2.aruco.ArucoDetector(cv2.aruco.getPredefinedDictionary(
↪  cv2.aruco.DICT_4X4_50),
                                   cv2.aruco.DetectorParameters())
camera = Camera()
viewer = ImageViewer()
print("Окно просмотра: http://10.42.0.1:8889/aruco/")

try:
    while True:
        try:
            frame = camera.get_cv_frame(timeout=1.0)
        except TimeoutError:
            continue
        if frame is None:
            continue

        corners, ids, _ = detector.detectMarkers(frame)
        if ids is not None:
            cv2.aruco.drawDetectedMarkers(frame, corners, ids)
            for marker_corners, marker_id in zip(corners, ids.flatten()):
                ok, rvec, tvec = cv2.solvePnP(MARKER_POINTS,
                ↪  marker_corners[0],
                                              camera_matrix, dist_coeffs,
                                              flags=cv2.SOLVEPNP_IPPE_SQUARE)
                if not ok:
                    continue
                x, y, z = tvec.flatten()               # смещение маркера
                ↪  относительно камеры, м
                distance = float(np.linalg.norm(tvec))
                cv2.drawFrameAxes(frame, camera_matrix, dist_coeffs, rvec,
                ↪  tvec, MARKER_SIZE / 2)
                text = f"id={marker_id} d={distance:.2f}m x={x:+.2f}
                ↪  y={y:+.2f}"
                px, py = marker_corners[0][0].astype(int)
                cv2.putText(frame, text, (px, py - 10),
                ↪  cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)
                print(text)
        viewer.imshow("aruco", frame)   # трансляция: 10.42.0.1:8889/aruco
except KeyboardInterrupt:
    pass
finally:
    viewer.close()
    camera.stop()
```

Что происходит. Для расчёта нужна калибровка камеры: матрица камеры `mtx` и коэффициенты искажений `dist` хранятся в файле `data.yml`; его создаёт программа калибровки по шахматной доске `camera_calibration.py` из репозитория примеров Pioneer-SDK2 (15 снимков доски 6×9 внутренних углов с разных ракурсов). `ArucoDetector.detectMarkers()` возвращает углы, номера и отвергнутые кандидаты. `solvePnP` решает обратную задачу: зная реальные 3D-координаты углов маркера (`MARKER_POINTS`) и их положение на снимке, находит поворот `rvec` и смещение `tvec` маркера относительно камеры; флаг `SOLVEPNP_IPPE_SQUARE` предназначен для плоских квадратов. Расстояние — длина вектора $d = \|\mathbf{t}\| = \sqrt{t_x^2 + t_y^2 + t_z^2}$. `drawFrameAxes` рисует оси маркера — визуальная проверка, правильно ли решена задача.

> **⚠ Размер маркера**
>
> Ошибка в `MARKER_SIZE` пропорционально искажает все расстояния: если указать 0.10 м для маркера со стороной 0.05 м, дистанции окажутся вдвое больше. Измеряйте чёрный квадрат маркера линейкой.

Удержание дистанции до маркера

python-onboard/pioneer_sdk2/20_aruco_follow.py

```python
"""Удержание дистанции до ArUco-маркера и поворот к нему (П-регулятор).

Захват кадров и распознавание идут в отдельном потоке, чтобы медленная
обработка изображения не задерживала отправку команд скорости.
"""
from pioneer_sdk2 import Pioneer, Camera, ImageViewer
import cv2
import numpy as np
import threading
import time

MARKER_SIZE = 0.10        # сторона маркера, м
TARGET_DISTANCE = 1.2     # желаемая дистанция до маркера, м
KP_DIST = 0.6             # м/с на метр ошибки дистанции
KP_YAW = 1.5              # рад/с на радиан углового отклонения
MAX_V, MAX_YAW = 0.4, 0.6
LOST_TIMEOUT = 0.5        # через сколько секунд без маркера останавливаться

half = MARKER_SIZE / 2
MARKER_POINTS = np.array([(-half, half, 0), (half, half, 0),
                          (half, -half, 0), (-half, -half, 0)],
                          ↪  dtype=np.float32)


class MarkerTracker(threading.Thread):
    def __init__(self, matrix, dist):
        super().__init__(daemon=True)
        self.matrix, self.dist = matrix, dist
        self.detector = cv2.aruco.ArucoDetector(
            cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50),
            ↪  cv2.aruco.DetectorParameters())
```

```
        self.camera, self.viewer = Camera(), ImageViewer()
        print("Окно просмотра: http://10.42.0.1:8889/follow/")
        self.lock = threading.Lock()
        self.tvec, self.seen_at = None, 0.0
        self.running = True

    def run(self):
        while self.running:
            try:
                frame = self.camera.get_cv_frame(timeout=0.5)
            except TimeoutError:
                continue
            if frame is None:
                continue
            corners, ids, _ = self.detector.detectMarkers(frame)
            if ids is not None:
                ok, rvec, tvec = cv2.solvePnP(MARKER_POINTS, corners[0][0],
                ↪  self.matrix, self.dist,
                                              flags=cv2.SOLVEPNP_IPPE_SQUARE)
                if ok:
                    with self.lock:                          # данные читает
                    ↪  другой поток — защищаем
                        self.tvec, self.seen_at = tvec.flatten(),
                        ↪  time.monotonic()
                    cv2.drawFrameAxes(frame, self.matrix, self.dist, rvec,
                    ↪  tvec, half)
            self.viewer.imshow("follow", frame)   # трансляция:
            ↪  10.42.0.1:8889/follow
    def latest(self):
        """Последнее положение маркера или None, если он давно не виден."""
        with self.lock:
            if self.tvec is None or time.monotonic() - self.seen_at >
            ↪  LOST_TIMEOUT:
                return None
            return self.tvec.copy()

    def stop(self):
        self.running = False
        self.join()
        self.viewer.close()
        self.camera.stop()


def control(tvec):
    """По положению маркера (x вправо, z вперёд — оси камеры) вычисляет vy и
    ↪  yaw_rate."""
    x, _, z = tvec
    distance_error = z - TARGET_DISTANCE           # >0 — маркер дальше, чем
    ↪  нужно
    angle_error = np.arctan2(x, z)                 # >0 — маркер правее оси
    ↪  камеры
    vy = np.clip(KP_DIST * distance_error, -MAX_V, MAX_V)
    yaw_rate = np.clip(-KP_YAW * angle_error, -MAX_YAW, MAX_YAW)   # минус:
    ↪  положительный yaw_rate — влево
    return float(vy), float(yaw_rate)
```

```
storage = cv2.FileStorage("data.yml", cv2.FILE_STORAGE_READ)
matrix, dist = storage.getNode("mtx").mat(), storage.getNode("dist").mat()
storage.release()

tracker = MarkerTracker(matrix, dist)
tracker.start()
drone = Pioneer()
drone.set_logger(False)

try:
    drone.arm()
    drone.takeoff()
    drone.go_to_local_point(x=0, y=0, z=1.3, yaw=0, time=0)
    while not drone.point_reached():
        time.sleep(0.1)

    while True:
        tvec = tracker.latest()
        vy, yaw_rate = control(tvec) if tvec is not None else (0.0, 0.0)
        drone.set_manual_speed_body_fixed(0, vy, 0, yaw_rate, 0.3)
        time.sleep(0.1)
except KeyboardInterrupt:
    pass
finally:
    if drone.get_fly_state().name == "IN_SKY":
        drone.set_manual_speed_body_fixed(0, 0, 0, 0, 0.3)
        drone.land()
    tracker.stop()
    drone.close_connection()
```

Что происходит. Два потока: `MarkerTracker` непрерывно читает камеру и решает PnP, основной поток раз в 0.1 с отправляет команды скорости — медленная обработка кадра не задерживает управление. `threading.Lock` защищает общие данные (`tvec`, `seen_at`) от одновременного чтения и записи. Потеря цели: `latest()` возвращает `None`, если маркер не виден дольше `LOST_TIMEOUT`, и тогда скорость нулевая. В осях камеры z — расстояние вперёд, x — вправо, регулятор:

$$v_y = K_d\,(z - d^*), \qquad \omega = -K_\psi \operatorname{atan2}(x, z),$$

обе величины ограничены через `np.clip`. Минус перед вторым коэффициентом: положительный `yaw_rate` поворачивает влево.

ЗАДАНИЕ 16.37 · Улучшить слежение

Добавьте управление высотой по t_y, чтобы маркер оставался в центре кадра по вертикали. Добавьте интегральную составляющую (ПИ-регулятор) и сравните точность удержания дистанции. Сделайте посадку на маркер: снижайтесь, пока маркер виден под камерой (камера $-80°$), затем `land()`.

Что должно получиться: дрон держится около 1.2 м от маркера; при отходе маркера следует за ним; при потере маркера останавливается; посадка на маркер заканчивается `land()` над ним.

В симуляторе: миссии Мини 2 SDK2: 35–36 в панели «**Обучение**».

Что запускается в симуляторе

В симуляторе есть `cv2`, `cv2.aruco` (словарь `DICT_6X6_250` на сцене с метками на полу) и `solvePnP`; на них построены миссии ArUco (поиск метки и её положение в кадре). Но примеры 19 и 20 написаны под словарь 4×4, свой размер маркера и файл калибровки `data.yml`, которого в симуляторе нет, поэтому надёжно запускать их можно только на дроне. Пример 18 можно пробовать в симуляторе, если на сцене есть яркий красный предмет и камера Мини 2 отдаёт кадры.

16.23 Итоговый проект: патрулирование

Цель раздела: собрать из приёмов главы законченную программу — маршрут как данные, события, камера, подсветка и безопасное завершение при любом исходе.

Объединим в одной программе миссию по точкам, события автопилота, камеру, сервопривод, светодиоды и аварийное завершение. Программа оформлена как класс.

python-onboard/pioneer_sdk2/21_patrol_project.py

```
"""Итоговый проект: патрулирование периметра с фотофиксацией и защитой по
↪  батарее.

Что объединено: миссия по точкам, события автопилота, камера, сервопривод,
светодиоды, журнал полёта и корректное аварийное завершение.
"""
from pioneer_sdk2 import Pioneer, Camera, ServoCamera, Event
import cv2
import os
import threading
import time

PERIMETER = [(0, 0), (2, 0), (2, 2), (0, 2)]   # вершины патрулируемого
↪  периметра, м
HEIGHT = 1.5
LAPS = 2
CAMERA_ANGLE = -30                             # смотрим немного вниз
PHOTO_DIR = time.strftime("patrol_%Y%m%d_%H%M%S")


class Patrol:
    def __init__(self):
        self.drone = Pioneer()
        self.drone.set_logger(False)
        self.camera = Camera()
        self.servo = ServoCamera()
        self.abort = threading.Event()         # выставляется из обработчика
        ↪  событий
        self.point = threading.Event()
```

```
        self.reason = None                      # событие, из-за которого
        ↪ прервана миссия
        for event in (Event.POINT_REACHED, Event.LOW_VOLTAGE1,
        ↪ Event.LOW_VOLTAGE2, Event.SHOCK):
            self.drone.subscribe(self.on_event, event)

    # ---------- события ----------
    def on_event(self, event):
        if event == Event.POINT_REACHED:
            self.point.set()
        elif event in (Event.LOW_VOLTAGE1, Event.LOW_VOLTAGE2, Event.SHOCK):
            self.reason = event
            self.abort.set()                    # основной поток увидит флаг и
            ↪ прекратит миссию
            self.point.set()                    # и проснётся, если ждал точку

    # ---------- действия ----------
    def fly_to(self, x, y, timeout=20):
        self.point.clear()
        self.drone.go_to_local_point(x=x, y=y, z=HEIGHT, yaw=0, time=0)
        if not self.point.wait(timeout):
            raise TimeoutError(f"Точка ({x}, {y}) не достигнута")
        if self.abort.is_set():
            raise RuntimeError(f"Миссия прервана: {self.reason}")

    def snapshot(self, tag):
        try:
            frame = self.camera.get_cv_frame(timeout=3.0)
        except TimeoutError:
            frame = None
        if frame is None:
            print("Нет кадра для", tag)
            return
        stamp = time.strftime("%H:%M:%S")
        cv2.putText(frame, f"{tag} {stamp}", (10, 30),
        ↪ cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
        path = os.path.join(PHOTO_DIR, f"{tag}.jpg")
        cv2.imwrite(path, frame)
        print("Снимок:", path)

    # ---------- сценарий ----------
    def run(self):
        os.makedirs(PHOTO_DIR, exist_ok=True)
        self.servo.set_angle(CAMERA_ANGLE)
        self.drone.arm()
        self.drone.takeoff()
        for lap in range(1, LAPS + 1):
            for n, (x, y) in enumerate(PERIMETER, start=1):
                self.drone.led_control(b=1)
                self.fly_to(x, y)
                self.drone.led_control(g=1)
                time.sleep(1)
                self.snapshot(f"lap{lap}_p{n}")
        self.fly_to(*PERIMETER[0])
        self.drone.land()

    def shutdown(self):
```

```
        if self.abort.is_set():
            print("АВАРИЙНОЕ ЗАВЕРШЕНИЕ:", self.reason)
            self.drone.led_control(r=1)
        if self.drone.get_fly_state().name == "IN_SKY":
            if self.reason == Event.LOW_VOLTAGE1:
                self.drone.rtl()                # заряда хватит — возвращаемся
                ↪ к точке взлёта
                time.sleep(8)
            self.drone.land()
        self.servo.set_angle(0)
        self.camera.stop()
        self.drone.close_connection()


patrol = Patrol()
try:
    patrol.run()
except Exception as error:
    print("Ошибка:", error)
finally:
    patrol.shutdown()
```

Что происходит. Класс `Patrol` хранит всё состояние (дрон, камера, сервопривод, флаги) и делит логику на методы: события, действия (`fly_to`, `snapshot`), сценарий (`run`) и завершение (`shutdown`). Два флага `threading.Event`: `point` — точка достигнута, `abort` — опасное событие; обработчик опасного события выставляет оба, поэтому основной поток просыпается и сразу видит причину. Реакция на разные события разная: при `LOW_VOLTAGE1` заряда хватает на возврат — вызывается `rtl()`; при `LOW_VOLTAGE2` автопилот уже сам начал экстренную посадку; при `SHOCK` миссия прерывается. Фото подписываются номером точки и временем и складываются в отдельную папку для каждого запуска (`patrol_ГГГГММДД_ЧЧММСС`).

Обратите внимание на `shutdown()`: он вызывается из `finally` при любом исходе и сам решает, что делать, по состоянию `get_fly_state()`. Это и есть шаблон безопасной программы из начала главы, развёрнутый до целого проекта. Пауза `time.sleep(8)` после `rtl()` — грубое ожидание возврата; в своих программах заменяйте её циклом по `get_fly_state()` (раздел «Курс и возврат домой»).

> **Что запускается в симуляторе**
>
> Проект использует `Camera`, `ServoCamera`, события и `cv2.imwrite`. В симуляторе Мини 2 всё это есть, и по проекту сделана итоговая миссия трека, но `cv2.imwrite` там заглушка: папка и снимки не создаются. Реальные фото проверяйте на дроне.

ЗАДАНИЕ 16.38 · Развитие проекта

Добавьте журнал полёта (пример 10) и видеозапись (пример 16). Сравнивайте снимок каждой точки с предыдущим кругом (`cv2.absdiff`) и сообщайте об изменениях — «охранная» функция. Подключите нейросеть из главы 17: ищите людей на

снимках и при обнаружении зависайте и мигайте.

Что должно получиться: два круга по периметру, по снимку в каждой вершине в папке запуска, посадка в начальной точке; при низком заряде — красный свет, возврат и посадка; новые функции работают, не нарушая завершения через `finally`.

В симуляторе: миссии Мини 2 SDK2: 37 в панели «**Обучение**».

16.24 Справочник команд

Справочник собран по сборнику примеров Геоскана (0.16.0), исходникам и описанию `pioneer_sdk2` 0.11.0 и справочнику симулятора. В столбце «Симулятор» — как метод ведёт себя на модели Мини 2 в симуляторе. Везде, где метод возвращает `None`, это значит «нет данных или ошибка». Если не сказано иного, `pioneer` — объект `Pioneer`.

Соединение и время

Метод	Что делает	Симулятор
`Pioneer(serial=None, tcp="127.0.0.1:20556", baudrate=57600, wait_callback=True, safety_command=True, logger=True)`	подключиться к автопилоту (serial *или* tcp)	да; serial и tcp не используются
`connect()`	установить соединение; конструктор вызывает сам	автоматически
`close_connection()`	закрыть соединение	да
`set_logger(value=True)`	включить или выключить журнал библиотеки	да
`reboot_board()`	перезагрузить плату, закрыть соединение, ждать 5 с; `bool` (на Мини 2 всегда `False`)	как на Мини 2: запрещена, `False`
`time()`	время автопилота, с (от начала GPS-эпохи); `float` или `None`	да
`uptime()`	секунд с момента идентификации в навигации; `int` или `None`	да
`flight_time()`	секунд с начала полёта; `float` или `None`	да

Полёт

Метод	Что делает	Симулятор
`arm(timeout=5, retries=0)`	запустить моторы и ждать `ENGINES_STARTED`; `bool`	да
`disarm()`	выключить моторы немедленно; `bool`	да
`takeoff()`	взлететь и ждать `TAKEOFF_COMPLETE`; `bool`	да

Метод	Что делает	Симулятор
`land()`	сесть и ждать `COPTER_LANDED`; `bool`	да; на земле возвращается сразу
`go_to_local_point(x, y, z, yaw=None, time=0)`	лететь в точку, м; единицы курса — см. врезку в разделе о координатах; не ждёт	да, курс в градусах
`go_to_local_point_body_fixed(x, y, z, yaw=None, time=0)`	лететь на смещение от текущего положения	да
`go_to_global_point(latitude, longitude, altitude, yaw=0)`	лететь в точку GPS: градусы, градусы, м над уровнем моря; только навигация GPS	пояснение и `False`
`go_to_global_point_relative(latitude_offset, longitude_offset, altitude_offset, yaw=0)`	лететь в точку GPS по смещению от текущего положения	пояснение и `False`
`point_reached()`	`True` один раз после прибытия	да
`point_deceleration()`	`True` один раз, когда дрон тормозит перед точкой	да
`set_manual_speed(vx, vy, vz, yaw_rate, interval=1.0)`	скорость, м/с и рад/с, на `interval` секунд; `yaw_rate` не больше 1.82	да
`set_manual_speed_body_fixed(vx, vy, vz, yaw_rate, interval=1.0)`	то же в осях дрона	да
`set_yaw(yaw)`	повернуться на курс в градусах; `bool`	да
`rtl()`	вернуться домой, не ждёт; `bool`	да
`get_fly_state()`	`FlyState`: `ON_LAND`, `ARMED` или `IN_SKY`	да

События, подсветка, груз

Метод	Что делает	Симулятор
`subscribe(callback, event=Event.ALL)`	вызывать `callback(event)` при событии	да
`unsubscribe(callback, event=Event.ALL)`	отписаться; аргументы те же, что при подписке	да
`led_control(led_id=255, r=0, g=0, b=0)`	цвет светодиодов, 0..1; `bool`	да, два светодиода
`grab_close(movement_time=0, velocity=20)`	закрыть захват Мини 2; скорость 0..100 % (по умолчанию 20 в 0.16, 100 в 0.11.0)	да; ждать не нужно
`grab_open(movement_time=0, velocity=20)`	открыть захват	да
`grab_stop()`	остановить захват	да
`cargo_set(grab)`	магнитный захват Базового Пионера: `True` включить, `False` выключить	`False`: магнита нет
`cargo_grab()`	включить магнит	`False`
`cargo_release()`	выключить магнит	`False`

Датчики

Метод	Что делает	Симулятор
`get_battery_status()`	(напряжение В, температура) или `None`	да
`get_accel()`	ускорение (x, y, z)	да
`get_gyro()`	угловая скорость (x, y, z), рад/с	да
`get_mag()`	магнитометр (x, y, z)	да, модель
`get_orientation()`	(крен, тангаж, рыскание), градусы	да
`get_altitude()`	высота по барометру, м	да
`get_dist_sensor_data()`	дальномер вниз, м	да
`get_optical_data()`	(поток x, поток y, дальность м)	да
`get_ranger_data()`	(право, лево, вперёд, назад, по вертикали), м; пять `None`, если модуля нет	да, до 4 м
`get_motors_rpm()`	список из четырёх чисел, обороты	да

Навигация и параметры

Метод	Что делает	Симулятор
`get_nav_system(update=False)`	`NavSystem`: `GPS`, `LPS` или `OPT`	да (всегда LPS)
`get_nav_status_lps()`	`NavStatus` локальной навигации	да (`OK`)
`get_local_position_lps()`	(x, y, z), м	да
`get_local_velocity_lps()`	(vx, vy, vz), м/с	да
`get_local_yaw_lps()`	курс, градусы, $-180 \ldots 180$	да
`get_nav_status_gps()`	`NavStatus` GPS	`None` и сообщение
`get_global_position_gps()`	(широта, долгота, высота)	`None` и сообщение
`get_global_velocity_gps()`	(север, восток, вертикаль), м/с	`None` и сообщение
`get_satellites_count()`	(спутники GPS, спутники ГЛОНАСС)	`None` и сообщение
`get_param(name, update=False)`	параметр автопилота или `None`	все 232 параметра Мини 2
`set_param(name, value)`	записать параметр; `bool`	только для известных

Пульт RC

Метод	Что делает	Симулятор
`send_rc_channels(channel_1=0, channel_2=0, channel_3=0, channel_4=0, channel_5=1, channel_6=0, channel_7=1, channel_8=0)`	отправить значения каналов, $-1 \dots 1$; отправлять постоянно	да; управляет, если выставлены параметры RC и дрон в воздухе
`get_rc_channel()`	значение канала пульта, целое, или `None`	возвращает `channel_8` из последней отправки
`set_rc_trigger(callback)`	вызывать `callback(value)` при смене значения канала	да
`rc_sdk1_to_sdk2(channel_1=0, channel_2=0, channel_3=0, channel_4=0, channel_5=2000)`	перевод каналов SDK 1 в SDK 2; в 0.11.0 считает с ошибкой	да, с той же ошибкой

Камера и сервопривод

Класс и метод	Что делает	Симулятор
`Camera(camera_type= CameraType.MAIN)`	открыть камеру: `MAIN` (передняя), `OPT` (нижняя)	да
`camera.get_cv_frame(timeout=5.0)`	кадр numpy BGR; `TimeoutError` при отсутствии кадра	да
`camera.stop()`	остановить захват	да
`ImageViewer()`	создать показ кадров	да, окно
`viewer.imshow(name, frame, fps=30)`	отправить кадр в поток `name`	да
`viewer.close()`	закрыть все потоки	да
`ServoCamera()`	подключиться к сервоприводу камеры	да
`servo.set_angle(angle, priority=ServoPriority.LOW)`	наклон камеры, $-80 \dots 30°$; вне диапазона `ValueError`	да
`RecorderControl()`	служба записи видео и фото (Мини 2)	да, без файлов
`recorder.get_camera_recording_configs(camera_type)`	режимы записи камеры (ширина, высота, fps)	да
`recorder.start_recording(camera_type, config, output_dir)`	начать запись видео (по умолчанию `/mnt/media/videos/`)	да, без файла
`recorder.stop_recording(camera_type)`	закончить запись	да
`recorder.take_photo(file_name, camera_type, config, output_dir)`	сохранить фото (`.jpg`, `.jpeg`, `.png`)	да, без файла

Перечисления

Имя	Значения
`Event`	`ALL`, `COPTER_LANDED`, `LOW_VOLTAGE1`, `LOW_VOLTAGE2`, `POINT_REACHED`, `POINT_DECELERATION`, `TAKEOFF_COMPLETE`, `ENGINES_STARTED`, `SHOCK`, `LOW_CHARGE`, `DISARM` (числовые коды — в разделе «События»)
`FlyState`	`ON_LAND`, `ARMED`, `IN_SKY`
`NavSystem`	`GPS`, `LPS`, `OPT`
`NavStatus`	`NO_DATA`, `CANNOT`, `LOW`, `OK`
`CameraType`	`MAIN`, `OPT` (набор зависит от платы)
`ServoPriority`	`HIGH`, `MEDIUM`, `LOW`

ЗАДАНИЕ 16.39 · Справочник в деле

Не глядя в примеры, по одной таблице справочника напишите программу: **`arm`** с проверкой, **`takeoff`**, точка $(0, 1, 1)$ с курсом $90°$, **`rtl()`**. Печатайте **`get_local_yaw_lps()`** после каждого этапа.

Что должно получиться: дрон долетает до точки носом вдоль $+Y$ (курс около 90), затем сам возвращается в точку взлёта и садится.

16.25 Типичные ошибки

Что случилось	Почему и что делать
`arm()` вернул `False`	нет пульта (`SWB` не внизу), разряжен аккумулятор, дрон стоит неровно, плохая навигация; проверьте статус и увеличьте **`timeout`**
`takeoff()` без **`arm()`** бросил `RuntimeError`	пропущен этап: сначала **`arm()`** и проверка его результата
`RuntimeError` у **`go_to_*`**	команда отправлена в состоянии `ARMED`: сначала **`takeoff()`**
Программа зависла на **`land()`**	дрон уже на земле или посадка не завершилась; **`land()`** ждёт события без тайм-аута. Зовите его только в воздухе
Дрон после `SHOCK` не выполняет команды	SDK считает, что дрон сел (`ON_LAND`); решайте сами: **`land()`** или новый **`arm()`**
Программа «пролетает» все точки сразу	после **`go_to_*`** нет ожидания **`point_reached()`**
Дрон повернулся не на тот угол	не те единицы курса в **`go_to_*`** (градусы или радианы): проверьте их на дроне, поворачивайте **`set_yaw`**
Дрон не повернулся при `yaw_rate=2.0`	значение больше 1.82 SDK отбросил молча

Что случилось	Почему и что делать
Дрон останавливается при `set_manual_speed`	истёк `interval`: повторяйте команду чаще, чем длится интервал
Цвет слишком яркий или не тот	в SDK 2 цвет 0..1, не 0..255
`point_reached()` сразу `True`	остался флаг от предыдущей точки; опрашивайте каждую точку
`get_*` вернул `None`, дальше `TypeError` при распаковке	нет датчика или данных; проверяйте результат перед распаковкой
Метод GPS или груза ничего не делает, пишет «не поддерживается»	нужной аппаратуры нет на борту: `cargo_*` — Базовый Пионер, `grab_*` и `ServoCamera` — Мини 2
`get_param` вернул `None`	нет такого параметра (имена см. приложение «Протокол Plazlink» и справочник Pioneer Code); например, высота взлёта называется `Flight_com_takeoffAlt`
RC-каналы игнорируются	не выставлены параметры `SensorMux_rc` и `Copter_man_rcMode*`, дрон не в воздухе или каналы отправляются с перерывами
Нет потока `ImageViewer`	драйвер камеры не gstreamer или неверное имя потока в адресе (`stream not found`)
Программа после камеры не завершается	не вызван `camera.stop()`
Кадр «запаздывает» после поворота камеры	в очереди старые кадры: пропустите несколько
`get_cv_frame` всегда `TimeoutError`	завис сервис видео: `sudo systemctl restart media-server`
Дрон раскачивается при слежении	слишком большой коэффициент K_p — уменьшите вдвое

16.26 Карта примеров: симулятор или дрон

Файлы лежат в каталоге `examples/python-onboard/pioneer_sdk2/` и в симуляторе в списке примеров Мини 2. Столбец «Симулятор» показывает, что вы увидите при запуске в симуляторе. Примеры проходят автоматическую проверку на модели библиотеки; в самом симуляторе запускайте их сами.

Файл	Тема	Симулятор
`sdk2_01_connect`	подключение, телеметрия, LED	да
`sdk2_02_takeoff_land`	взлёт, посадка, finally	да
`sdk2_03_square`	квадрат по точкам	да
`sdk2_04_events`	подписка на события	да
`sdk2_05_manual_speed`	скорость	да
`sdk2_06_camera_stream`	камера, ImageViewer	да, окно
`sdk2_07_servo_camera`	наклон камеры	да

Файл	Тема	Симулятор
`sdk2_08_sensors`	все датчики	да
`sdk2_09_safe_start`	навигация, состояние	да
`02_led_effects`	эффекты LED	да
`03_arm_disarm_safe`	безопасный запуск	да
`04_takeoff_hover_land`	шаблон программы	да
`05_square_mission`	миссия-список точек	да
`06_body_fixed_moves`	смещения от дрона	да
`07_circle_face_center`	окружность, курс в центр	да
`08_manual_speed`	повтор команды скорости	да
`09_events`	ожидание событий	да
`10_telemetry_logger`	журнал в CSV	да, файл не скачать
`10b_plot_log`	график журнала	на компьютере
`11_params_context`	параметры автопилота	да (параметр известен)
`12_rc_ramp`, `13_rc_trigger_modes`	виртуальный пульт	да, по правилам из раздела «Пульт RC»
`14a_camera_stream`, `14_camera_osd`	видео, OSD	да, окно
`15_servo_panorama`	серия снимков	да, файлы не сохраняются
`16_recorder_mission`	RecorderControl	да, файлы не создаются
`17_delivery_grab`	груз захватом	да
`18_color_tracking_yaw`	слежение за цветом	возможно, нужен красный предмет
`19_aruco_distance`, `20_aruco_follow`	ArUco	нет: нужны `data.yml`, словарь 4×4
`21_patrol_project`	проект-патруль	да, снимки не сохраняются

16.27 Проверьте себя

1. Что произойдёт при вызове `takeoff()` сразу после `Pioneer()`, без `arm()`?

   (a) вернётся `False`

   (b) будет `RuntimeError`

   (c) дрон взлетит

   Ответ: (b) — пропущен этап автомата; команду *прошлого* этапа SDK молча пропускает, а перескок этапа останавливает исключением.

2. Какие из команд ждут завершения действия?

   (a) `arm`, `takeoff`, `land`

   (b) `go_to_local_point`, `set_manual_speed`

   (c) все команды SDK 2

   Ответ: (a). Точки и скорости неблокирующие: прибытие ждут через `point_reached()`.

3. Как правильно дождаться прибытия в точку?

(a) `time.sleep(10)`

(b) цикл `point_reached()` с тайм-аутом

(c) вообще не ждать

Ответ: (b) — пауза не знает, прилетел ли дрон, а без ожидания следующая команда отменит текущую точку.

4. Программу прервали `Ctrl` + `C`, когда дрон был в воздухе. Что должно сделать `finally`?

(a) сразу `disarm()`

(b) `land()`, а затем `disarm()` и закрытие соединения

(c) ничего

Ответ: (b) — `disarm()` в воздухе выключит моторы, и дрон упадёт.

5. Что сделает `led_control(r=255)`?

(a) полный красный: значение обрежется до 1

(b) ошибка

(c) светодиод выключится

Ответ: (a) — ошибки нет, цвет в SDK 2 — доли от 0 до 1 (в первом SDK максимумом было 255). Поэтому оттенки, подобранные в шкале 0..255 (скажем, оранжевый 255, 128, 0), превращаются в «чистые» цвета: средние значения тоже обрезаются до 1.

6. Что будет, если программа зависла после `set_manual_speed(0, 0.3, 0, 0, 0.5)`?

(a) дрон летит вперёд, пока не сядет батарея

(b) дрон остановится через 0.5 с

(c) дрон сразу упадёт

Ответ: (b) — одна команда держит скорость `interval` секунд.

7. Что сделает SDK при `yaw_rate=2.5`?

(a) повернёт на максимум

(b) отбросит команду без ошибки

(c) бросит исключение

Ответ: (b) — предел 1.82 рад/с; вместо поворота дрон просто стоит.

8. `get_local_position_lps()` вернул `None`. Как поступить?

(a) распаковать `x, y, z = ...`

(b) проверить на `None` и обработать

(c) считать координаты нулями

Ответ: (b) — «нет данных» не равно нулю; распаковка `None` даёт `TypeError`.

9. Можно ли звать `land()` из обработчика `subscribe`?

(a) можно

(b) нельзя: обработчик ставит флаг, а садится основной цикл

(c) можно, если в `finally`

Ответ: (b) — обработчик работает в служебном потоке, который должен доставить событие посадки.

10. Что нужно сделать перед записью параметра автопилота?

(a) ничего

(b) прочитать и запомнить старое значение, а в конце вернуть

(c) перезагрузить плату

Ответ: (b) — параметры сохраняются после перезагрузки; вернуть их удобно в `finally` или через `with`.

11. Что должно быть в `finally` программы с камерой и `ImageViewer`?

(a) `viewer.close()` и `camera.stop()`

(b) только `viewer.close()`

(c) ничего

Ответ: (a) — иначе поток приёма кадров продолжит работать, и программа не завершится.

16.28 Задания к главе

ЗАДАНИЕ 16.40 · Телеметрия до Ctrl+C

Подключитесь к дрону и раз в секунду печатайте напряжение батареи и высоту по дальномеру, пока не нажата `Ctrl` + `C`; соединение закройте в `finally`.

Что должно получиться: строки идут раз в секунду, после `Ctrl+C` программа печатает «Соединение закрыто» без трассировки ошибки.

В симуляторе: миссии Мини 2 SDK2: 1, 19 в панели «**Обучение**».

ЗАДАНИЕ 16.41 · Квадрат с камерой

Мини-проект. Дрон облетает квадрат, а программа показывает в браузере видео с камеры с подписью: номер вершины, к которой летит дрон, и напряжение батареи. Ожидание точки и показ кадров идут в одном цикле.

Что должно получиться: видео не замирает во время перелётов, подпись меняется при отправке каждой вершины; после четвёртой вершины дрон садится.

В симуляторе: миссии Мини 2 SDK2: 10, 32 в панели «**Обучение**».

ЗАДАНИЕ 16.42 · Осторожный разведчик

Мини-проект. Дрон проверяет навигацию (`OK`) и батарею, взлетает и летит вдоль стены короткими шагами скорости, пока `get_ranger_data()` показывает впереди больше 0.7 м. В обработчике `LOW_VOLTAGE1` поднимается флаг; при флаге или препятствии дрон разворачивается `set_yaw` на $180°$, возвращается `rtl()` и ждёт посадки по `get_fly_state()`. Подпись на видео: расстояние до препятствия и напряжение.

Что должно получиться: при чистой трассе дрон летит до конца отрезка и возвращается; при препятствии или флаге батареи — сразу разворот и возврат, программа завершается посадкой и закрывает камеру, `ImageViewer` и соединение.

16.29 Миссии симулятора по разделам главы

Трек ««**Мини 2 · SDK2**»» в режиме «**Обучение**» начинается с 37 миссий в 13 группах (дальше идут миссии 38–64 по ROS 2 и рою, часть V). Миссии сами выбирают или создают дрон Мини 2, открывают редактор Python с шаблоном `pioneer_sdk2` и проверяют полёт.

№	Группа миссий	Раздел главы
1–2	Подключение и жизненный цикл	подключение, время
3–6	Взлёт и посадка	взлёт, посадка и порядок команд; навигация перед стартом
7–12	Полёт по точкам и системы координат	координаты и единицы, полёт по точкам, курс
13–15	События автопилота	события
16–18	Скорости и полёт в осях корпуса	полёт со скоростью
19–22	Датчики и телеметрия	датчики, навигация, журнал, параметры
23–25	Подсветка	подсветка
26–27	Груз и захват	груз
28–29	Виртуальный пульт RC	пульт RC
30–32	Камера и ImageViewer	камера и видео
33–34	Сервопривод камеры и запись	наклон камеры, RecorderControl
35–36	Метки ArUco	компьютерное зрение (ArUco)
37	Итоговый патруль	итоговый проект

В трек не вошли полёт по GPS, возврат `rtl()`, слежение за цветом и нейросети: в симуляторе их нельзя проверить по полёту.

Итоги

- `pioneer_sdk2` работает на борту дрона: программу копируют на дрон или пишут в среде Pioneer Code и запускают `python3`.
- `arm()`, `takeoff()`, `land()` ждут окончания действия; `go_to_local_point` и команды скорости — нет, ждите `point_reached()`.
- SDK 2 ведёт автомат ON_LAND → ARMED → IN_SKY: команда прошлого этапа игнорируется, пропуск этапа — `RuntimeError`. Состояние читает `get_fly_state()`.
- Каркас программы: проверить результат `arm()`, взлететь, выполнить задание, сесть; `finally` сажает дрон, если он в воздухе, и закрывает соединение.
- Цвет — 0..1; скорость держится `interval` секунд; `yaw_rate` — рад/с, не больше 1.82; единицы курса в `go_to_*` проверьте на дроне (градусы в симуляторе), `set_yaw` — градусы.
- Точки бывают локальные (с вариантом «от дрона») и глобальные по GPS; `rtl()` не ждёт посадки — опрашивайте `get_fly_state()` с тайм-аутом.

- События — через `subscribe`; обработчик только ставит флаг, действует основной цикл.
- Датчики возвращают `None`, когда данных нет; навигацию проверяйте через `NavStatus` до `arm()`.
- Параметры автопилота: прочитать, записать, вернуть старое значение.
- Груз: `cargo_*` — магнит Базового, `grab_*` — захват Мини 2; скорость захвата задавайте явно.
- Пульт RC: нужен набор параметров, каналы шлют постоянно; `rc_sdk1_to_sdk2` на 0.11.0 считает неверно.
- Видео: `Camera` → `ImageViewer` → браузер, в `finally` — `viewer.close()` и `camera.stop()`; запись и фото — `RecorderControl`; наклон камеры — `ServoCamera().set_angle(angle)`, от -80 до $30°$.
- В симуляторе Мини 2 добавляется на панели «**Рой**»; GPS, магнитный захват, запись файлов и большинство параметров там недоступны.

ГЛАВА 17

Pioneer RKNN: нейросети на борту

Цели главы

После главы вы сможете:

- объяснить, что нейросеть-детектор — функция «кадр → объекты», и зачем дрону нейропроцессор;
- выбрать класс `pioneer_rknn` под задачу: `Yolo`, `YoloSeg`, `YoloPose` или `PaddleOCR`;
- подготовить вход модели, разобрать ответ `run()` и пересчитать рамки в координаты кадра;
- настроить пороги уверенности и подтверждать результат несколькими кадрами;
- собрать миссию «найти человека» и назвать, чем симулятор отличается от дрона.

Миссии симулятора к главе

Отдельных миссий к главе нет. Приёмы — подтверждение результата несколькими кадрами, ограниченный поиск, остановка при потере цели — отрабатываются в миссиях Python 90 «Стабильное распознавание», 97 «Потеряли метку» и 101 «Поиск по секторам». Программы самой главы запускаются в редакторе Python на дроне «Пионер Мини 2» (панель «**Рой**»); на другом дроне `pioneer_rknn` остановится с подсказкой.

17.1 Нейросеть и нейропроцессор

В главе 8 дрон находил на кадре маркеры: у маркера строгая форма, и найти его можно правилами. Человека, машину или дорожный знак правилами не описать — их находит **нейросеть**. Для программы это просто функция: на входе кадр, на выходе список найденных объектов с рамками и уверенностью. Как распознавать, нейросеть «выучила» заранее на тысячах размеченных картинок; результат обучения хранится в файле модели.

Нейросети требуют много вычислений, и обычный процессор дрона справляется с ними медленно. В процессоре Rockchip RK3576 у Пионера Мини 2 есть отдельный блок — **нейропроцессор** (NPU) с двумя ядрами, который считает саму сеть в разы быстрее. Модель для него хранится в особом формате — файле `.rknn`. Библиотека `pioneer_rknn` (версия 1.10.0) прячет работу с NPU за несколькими классами. Как и `pioneer_sdk2`, она встроена в образ Pioneer OS и работает на борту: `pioneer_sdk2` даёт кадры и полёт, `pioneer_rknn` — распознавание.

Класс	Задача	Архитектуры	Что возвращает `run()`
`Yolo`	объекты: рамка, класс, уверенность	`yolov8`, `yolov11`, `yolov26`	`(boxes, classes, scores)` или `None`
`YoloSeg`	то же и маска каждого объекта	`yolov8-seg`, `yolov11-seg`, `yolov26-seg`	`(boxes, classes, scores, masks)` или `None`
`YoloPose`	люди и 17 точек скелета	`yolov8-pose`	список людей: `score`, `xmin…ymax`, `keypoint`
`PaddleOCR`	строки текста и их чтение	PP-OCRv5 (детектор и распознаватель)	`(четырёхугольники, тексты)` или `(None, None)`
`ModelContainer`	загрузка модели и запуск на NPU; основа остальных классов	любая (`custom`)	список «сырых» выходов NPU
`ModelRegistry`	список моделей на дроне, загрузка и удаление	—	—

ЗАДАНИЕ 17.1 · Конвейер на бумаге

Нарисуйте схему конвейера «камера → подготовка → нейросеть → решение → команда» для задачи «зажечь зелёный, когда в кадре бутылка» и подпишите на каждой стрелке, что передаётся: кадр BGR, массив $1 \times 640 \times 640 \times 3$, рамки и классы, `True`/`False`, цвет.

Что должно получиться: пять блоков, на стрелках — форматы данных; решение принимается по классу 39 («бутылка» в COCO) и уверенности выше порога.

17.2 Симулятор и дрон

Нейросеть в браузере не запустить: файл `.rknn` считает только NPU. Поэтому в симуляторе `pioneer_rknn` — та же библиотека с теми же классами, формами входа и ответа, но вместо нейросети работает «зрение» симулятора: он знает, где в сцене стоят люди, машины и площадки, проецирует их в кадр камеры дрона и отдаёт результат в формате настоящей библиотеки. Программа, которая работает в симуляторе, на дроне запустится без правок; обратное верно не всегда.

	Дрон	Симулятор
Где есть библиотека	Пионер Мини 2; Пионер Базовый с модулем Radxa Zero	только дрон «Пионер Мини 2»
Что видит нейросеть	пиксели кадра: любые объекты 80 классов COCO	объекты сцены в поле зрения камеры; пиксели массива не важны, но полностью чёрный вход не даёт ничего
Классы `Yolo`	все 80 классов COCO	человек (0), машина (2), автобус (5), поезд (6), грузовик (7); деревьев, зданий и грузов в COCO нет
Маски и поза	маски и скелеты по пикселям	маска — контур объекта, скелет — по модели человека в сцене
Текст `PaddleOCR`	любой печатный текст	надписи площадок: «Н», «ПОСАДКА», «ЗАРЯДКА», номер и «СТАРТ / N»; камера должна смотреть на площадку сверху, с высоты не более 2 м
Уверенность	реальная, с ошибками модели	растёт с размером объекта в кадре; объект виден не меньше чем на 45 % и не мельче 14 пикселей; за преградой его нет
Регистр моделей	пять предустановленных моделей, свои добавляются	те же пять имён; `upload_model` лишь записывает строку в регистр, а `model_path` даёт `FileNotFoundError`
Размер кадра	задаёт камера дрона (в примерах Геоскана 1080×720)	640×360
Скорость	десятки миллисекунд на кадр (раздел 17.8)	скорость браузера: число кадров в секунду на дрон не переносится

Размер кадра на дроне и в симуляторе разный, поэтому масштаб рамок всегда считайте по `frame.shape`, а не по константам. Результаты симулятора — учебные: «уверенность 0.9» в нём значит «объект хорошо виден», а не «нейросеть узнала».

17.3 Какие модели есть на дроне

Модели хранятся в **регистре** — службе на борту дрона (программный адрес `http://127.0.0.1:7777/model`). У регистра есть веб-интерфейс: главное меню Pioneer Code `http://10.42.0.1:9090` → «Модели ИИ». На Мини 2 предустановлено пять моделей: `yolov8n`, `yolov8n-seg`, `yolov8n-pose`, `PP-OCRv5_mobile_det` и `eslav_PP-OCRv5_mobile_rec` (последняя читает и кириллицу, и латиницу).

Любой класс-обёртка открывает модель одним из двух способов: по имени из регистра, `Yolo(model_name="yolov8n")`, — так регистр сам знает архитектуру и версию; или по пути к файлу, `Yolo(model_path="/home/pioneermini/best.rknn")`. Нужен хотя бы один из аргументов, иначе `ValueError`. Если имени нет в регистре —

`FileNotFoundError`; если архитектура не подходит классу (например, модель позы открывают классом `Yolo`) — снова `ValueError`. Необязательный аргумент `npu_core` выбирает ядро NPU: `[0]`, `[1]`, `[0, 1]` или `None` (выбрать автоматически). Номер несуществующего ядра не вызывает исключения: библиотека печатает сообщение и завершает программу.

python-onboard/pioneer_rknn/rknn_01_models.py

```python
from pioneer_rknn import ModelRegistry

# Регистр моделей — служба на борту дрона с файлами .rknn.
registry = ModelRegistry()

models = registry.list_model()
if models is None:
    print("Регистр моделей не отвечает")
else:
    print("Модели на дроне:")
    print(models)
    print(registry.get_model_info("yolov8n"))
```

Что происходит. `list_model()` возвращает описание моделей регистра, `get_model_info(имя)` — сведения об одной; при ошибке обе возвращают `None`. Сведения о модели — словарь с полями `name`, `version`, `arch` и `filename` (путь к файлу `.rknn` на дроне); у `yolov8n` это архитектура `yolov8` и версия 1.0.0.

Запустите эту программу первой: имена моделей в примерах ниже должны совпадать с теми, что она напечатает.

ЗАДАНИЕ 17.2 · Модели на борту

Допишите `rknn_01_models.py`: для имён `yolov8n`, `yolov8n-seg`, `yolov8n-pose` и вымышленного `yolov9x` печатайте архитектуру из `get_model_info`, а при `None` — «нет сведений».

Что должно получиться: для первых трёх — `yolov8`, `yolov8-seg`, `yolov8-pose`, для `yolov9x` — «нет сведений»; программа не падает.

17.4 Поиск объектов: Yolo

YOLO (*You Only Look Once*) — семейство нейросетей, которые за один проход по кадру находят все объекты. Модель `yolov8n` обучена на наборе COCO и различает 80 классов: человек (класс 0), машина (2), собака (16), бутылка (39) и другие. Буква `n` (*nano*) — самый маленький и быстрый вариант. Названий классов в библиотеке нет, только номера: словарь «номер → название» держат в программе.

python-onboard/pioneer_rknn/rknn_02_detect.py

```python
import cv2
import numpy as np
from pioneer_rknn import Yolo
from pioneer_sdk2 import Camera, CameraType, ImageViewer

SIZE = 640  # вход модели 640 x 640
# часть классов COCO
NAMES = {0: "person", 2: "car", 16: "dog", 39: "bottle"}

# модель из регистра на дроне
model = Yolo(model_name="yolov8n")
camera = Camera(camera_type=CameraType.MAIN)
viewer = ImageViewer()


def prepare(frame):
    """Кадр камеры -> вход нейросети: 640x640 и пакет из одного
    ↪ изображения."""
    image = cv2.resize(frame, (SIZE, SIZE))
    # ось «пакет»: (640, 640, 3) -> (1, 640, 640, 3); цвета остаются BGR, как
    ↪ у кадра
    return np.expand_dims(image, 0)


def detections(result):
    """Ответ Yolo -> список (рамка, класс, уверенность).

    Рамка — (x1, y1, x2, y2) в пикселях входа 640x640.
    Если объектов нет, модель возвращает None.
    """
    if result is None or result[0] is None:
        return []
    boxes, classes, scores = result
    return list(zip(boxes, classes, scores))


try:
    while True:
        frame = camera.get_cv_frame()
        h, w = frame.shape[:2]
        # масштаб обратно к кадру камеры
        kx, ky = w / SIZE, h / SIZE

        result = model.run([prepare(frame)])
        for (x1, y1, x2, y2), cls, score in detections(result):
            p1 = (int(x1 * kx), int(y1 * ky))
            p2 = (int(x2 * kx), int(y2 * ky))
            name = NAMES.get(int(cls), int(cls))
            label = f"{name} {score:.2f}"
            cv2.rectangle(frame, p1, p2, (0, 255, 0), 2)
            corner = (p1[0], max(p1[1] - 8, 20))
            cv2.putText(frame, label, corner, cv2.FONT_HERSHEY_SIMPLEX,
                        0.6, (0, 255, 0), 2)

        viewer.imshow(name="stream", frame=frame)
except KeyboardInterrupt:
```

```
    print("Остановлено")
finally:
    model.release()  # освободить нейропроцессор
    camera.stop()
    viewer.close()
```

Что происходит. Цикл — это конвейер из четырёх шагов:

1. **Кадр.** `camera.get_cv_frame()` — изображение BGR.
2. **Подготовка.** Нейросеть принимает вход строго заданной формы. `prepare` сжимает кадр до 640×640 и добавляет измерение «пакет»: `np.expand_dims` превращает массив формы `(640, 640, 3)` в `(1, 640, 640, 3)` — пакет из одной картинки. Цвета остаются в порядке BGR: так подают кадр и примеры Геоскана, и код, который генерирует Bricks.
3. **Нейросеть.** `model.run([...])` считает модель на NPU и отбирает найденное (*постобработка*). Список в аргументе — список входов; у YOLO вход один.
4. **Результат.** Рамки пересчитываются из координат входа 640×640 в координаты кадра (`kx`, `ky`) и рисуются; кадр уходит в браузер, как в главе 16.

Массив формы `(640, 640, 3)`, без оси «пакет», — самая частая ошибка входа: модель ждёт именно `(1, 640, 640, 3)`. Конвейер целиком, с формами данных между шагами, — на рисунке 30.

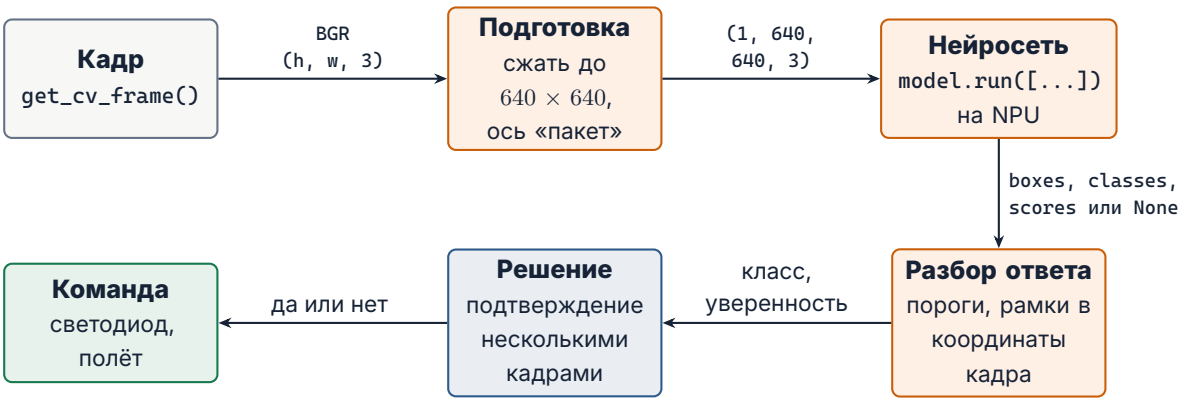


Рис. 30. Конвейер распознавания на борту: кадр превращается в вход нейросети, ответ разбирается, а решение принимается по уверенности и нескольким кадрам подряд. Под стрелками — форма передаваемых данных.

Формат ответа — в одной функции

`Yolo.run` возвращает кортеж (рамки, классы, уверенности): рамка — `(x1, y1, x2, y2)` в пикселях входа 640×640, а не кадра; если объектов нет — `None`, поэтому ответ нельзя распаковывать, не проверив. Весь разбор собран в функции `detections`, остальная программа от формата не зависит. Параметры `img_width` и `img_height` конструктора — размер *входа модели*, а не кадра камеры.

Пороги

Нейросеть не говорит «да» или «нет» — она оценивает уверенность от 0 до 1. Атрибут `object_thresh` (у `Yolo` и `YoloSeg` по умолчанию 0.25, у `YoloPose` — 0.4) отбрасывает всё, в чём модель уверена меньше. Один и тот же объект модель часто находит несколькими перекрывающимися рамками; **NMS** (*non-maximum suppression*) оставляет из них самую уверенную, а `nms_thresh` (0.45; у позы 0.5) задаёт, насколько рамки одного класса должны перекрываться (мера перекрытия — IoU, отношение площади пересечения к площади объединения), чтобы считаться одним объектом. Повысили `object_thresh` — меньше ложных находок, но и настоящие объекты теряются чаще. Понизили `nms_thresh` — рамки сливаются агрессивнее, и близко стоящие объекты могут «слиться»; повысили — остаются дубли. Пороги можно менять у готового объекта, перезагружать модель не нужно.

> **Освобождайте нейропроцессор**
>
> `model.release()` возвращает ресурсы NPU. Вызывайте её в `finally`, как `close_connection()`: тогда модель освобождается и после `Ctrl` + `C`, и после ошибки.

ЗАДАНИЕ 17.3 · Фильтр и порог

Выводите на кадр только людей и бутылки (классы 0 и 39), а `object_thresh` поднимите с 0.25 до 0.5. Затем измерьте, сколько кадров в секунду обрабатывает программа, и выведите число на кадр.

Что должно получиться: рамки есть только у людей и бутылок, ложных рамок меньше, чем при 0.25; число кадров в секунду видно на кадре (на дроне — порядка десяти, в симуляторе — другое).

17.5 Маски объектов: YoloSeg

`YoloSeg` расширяет `Yolo`: кроме рамки, класса и уверенности возвращает **маску** каждого объекта — массив 640×640, в котором 1 означает пиксель объекта, а 0 — всё остальное (маска уже обрезана по рамке). Модель — `yolov8n-seg`, параметры конструктора те же, что у `Yolo`, вход готовится так же. Маска точнее рамки: по ней считают площадь объекта, его центр масс, вырезают фон.

Доля кадра, занятая людьми

```
boxes, classes, scores, masks = model.run(
    [prepare(frame)])
people = [m for m, c in zip(masks, classes)
          if int(c) == 0]
if people:
    union = np.maximum.reduce(people)
    print("Доля:", union.mean())  # 0..1
```

Сегментация считается дольше детекции: сеть на NPU почти такая же, но постобработка масок на процессоре тяжелее. Для готовых блоков Bricks (глава 18) `YoloSeg` не предусмотрен.

17.6 Поза человека: YoloPose

`YoloPose` находит людей и для каждого — 17 ключевых точек скелета. В главе 8.10 это делала библиотека MediaPipe на компьютере; здесь то же самое считается на борту.

`run` возвращает список найденных людей. У каждого есть `score` (уверенность), рамка `xmin`, `ymin`, `xmax`, `ymax` (в координатах входа 640×640) и `keypoint` — массив 17×3: для каждой точки x, y и уверенность точки. Номера точек — по стандарту COCO:

0	нос	5, 6	плечи (лев., прав.)	11, 12	бёдра
1, 2	глаза	7, 8	локти	13, 14	колени
3, 4	уши	9, 10	запястья	15, 16	лодыжки

python-onboard/pioneer_rknn/rknn_03_pose_signal.py

```python
import cv2
import numpy as np
from pioneer_rknn import YoloPose
from pioneer_sdk2 import Pioneer, Camera, CameraType

SIZE = 640
# Номера точек скелета (порядок COCO): плечи и запястья.
L_SHOULDER, R_SHOULDER, L_WRIST, R_WRIST = 5, 6, 9, 10

pioneer = Pioneer()
# имя — как в списке регистра
model = YoloPose(model_name="yolov8n-pose")
camera = Camera(camera_type=CameraType.MAIN)


def prepare(frame):
    # кадр BGR -> (1, 640, 640, 3)
    return np.expand_dims(cv2.resize(frame, (SIZE, SIZE)), 0)


def hands_up(person):
    """Обе кисти выше плеч? Ось y на картинке — вниз."""
    k = person.keypoint  # массив 17 x 3: x, y, уверенность
    left = k[L_WRIST][1] < k[L_SHOULDER][1]
    right = k[R_WRIST][1] < k[R_SHOULDER][1]
    return left and right


signal = None
try:
    while True:
        frame = camera.get_cv_frame()
```

```
        people = model.run([prepare(frame)]) or []
        now = any(hands_up(p) for p in people if p.score > 0.5)
        if now != signal:  # менять цвет только при смене жеста
            signal = now
            if signal:
                print("Руки подняты")
                pioneer.led_control(r=0, g=1, b=0)
            else:
                pioneer.led_control(r=1, g=0, b=0)
except KeyboardInterrupt:
    print("Остановлено")
finally:
    pioneer.led_control()
    model.release()
    camera.stop()
    pioneer.close_connection()
```

Что происходит. Жест «руки вверх» — обе кисти выше плеч. Ось y на картинке направлена *вниз*, поэтому «выше» значит «меньше по y». Подсветка меняется только при смене жеста (`now != signal`): незачем двадцать раз в секунду отправлять автопилоту одну и ту же команду. Дрон при этом не взлетает — жест только включает зелёный или красный цвет, так программу безопасно проверить, держа дрон в руках. Точки, закрытые от камеры, модель всё равно выдаёт, но с малой уверенностью (третий элемент точки): для надёжного жеста их стоит отбрасывать.

ЗАДАНИЕ 17.4 · Одна рука

Добавьте в `rknn_03_pose_signal.py` второй жест — поднята только одна рука — и показывайте его синим цветом. Жест засчитывайте, если он держится три кадра подряд, а точки с уверенностью ниже 0.3 не используйте.

Что должно получиться: обе руки вверх — зелёный, одна — синий, руки опущены — красный; цвет не мигает при коротких ошибках модели.

17.7 Чтение текста: PaddleOCR

Распознавание текста (OCR) — две нейросети подряд. Первая (*детектор*, `PP-OCRv5_mobile_det`) находит на кадре строки текста и обводит их четырёхугольниками, вторая (*распознаватель*, `eslav_PP-OCRv5_mobile_rec`) читает буквы внутри каждого.

python-onboard/pioneer_rknn/rknn_04_ocr.py

```
from pioneer_rknn import PaddleOCR
from pioneer_sdk2 import Camera, CameraType

# Две модели: одна находит строки текста, другая читает буквы.
ocr = PaddleOCR(det_model_name="PP-OCRv5_mobile_det",
                rec_model_name="eslav_PP-OCRv5_mobile_rec")
```

```
camera = Camera(camera_type=CameraType.MAIN)

last = ""
try:
    while True:
        frame = camera.get_cv_frame()
        # рамки строк и варианты текста
        boxes, results = ocr.run(frame)
        words = []
        # без текста библиотека возвращает (None, None)
        for variants in results or []:
            # первый вариант — самый вероятный
            text, score = variants[0]
            if score > 0.6:
                words.append(text)
        line = " ".join(words)
        if line and line != last:  # печатать только новое
            print("Вижу текст:", line)
            last = line
except KeyboardInterrupt:
    print("Остановлено")
finally:
    camera.stop()
```

Что происходит. В отличие от `Yolo`, `ocr.run` получает кадр целиком, без подготовки, и возвращает два списка: четырёхугольники строк и варианты прочтения для каждой строки — пары (текст, уверенность). Если текста нет, вернётся `(None, None)`, поэтому в программе стоит `results or []`. Программа берёт первый вариант — самый вероятный — и отбрасывает неуверенные: сам `PaddleOCR` уже отбрасывает строки ниже порога `drop_score` (0.5), а программа добавляет более строгий порог 0.6. Названия моделей должны совпадать с именами в регистре (`rknn_01_models.py`).

Распознавание ошибается на похожих буквах разных алфавитов: на дроне в одном русском слове буква «С» была прочитана как латинская (пример — в главе 18). Поэтому прочитанное сравнивают по смыслу, а не посимвольно. Кроме того, `cv2.putText` не умеет рисовать кириллицу: подписи на кадре делайте латиницей, иначе вместо букв будут вопросительные знаки.

ЗАДАНИЕ 17.5 · Слово-команда

Программа на основе `rknn_04_ocr.py` при уверенном прочтении слова «ПОСАДКА» два раза подряд включает красный свет и печатает «Команда: посадка»; другие слова только печатает. В симуляторе наклоните камеру вниз (`ServoCamera`) и держите дрон над посадочной площадкой не выше 2 м; на дроне покажите камере лист с надписью.

Что должно получиться: при показе надписи — одна строка «Команда: посадка» и красный свет; случайный текст на фоне команду не вызывает.

17.8 Скорость и ресурсы

На Мини 2 один вызов `Yolo.run()` вместе с постобработкой занимает порядка 60–70 мс, а программа детекции в видеопотоке выдаёт 12–15 кадров в секунду. Самое неожиданное: узким местом оказывается не NPU. Он считает саму сеть около 12 мс, а остальное время — подготовка входа и постобработка (порог, NMS), то есть обычный процессор. Из этого следуют два практических вывода.

- Первый `run()` после загрузки модели заметно медленнее следующих: выделяется память и «прогревается» NPU. Измеряя скорость, первые вызовы отбрасывайте.
- Отдать одной небольшой модели оба ядра почти бесполезно; ускоряет другое — несколько моделей параллельно на разных ядрах (`npu_core=[0]` и `npu_core=[1]`), например по одной на каждую камеру. Это упражнение для продвинутых, в главе его нет.

17.9 Проект: найти человека

Соберём всё вместе. Дрон взлетает, медленно поворачивается на месте и смотрит в камеру; увидев человека, останавливается, включает зелёный свет и садится. Не нашёл за 30 секунд — красный свет и посадка.

python-onboard/pioneer_rknn/rknn_05_search_person.py

```
import cv2
import numpy as np
import time
from pioneer_rknn import Yolo
from pioneer_sdk2 import Pioneer, Camera, CameraType

SIZE = 640
PERSON = 0                  # класс "человек" в COCO
CONFIRM = 3  # столько кадров подряд, чтобы поверить
SEARCH_TIME = 30            # секунд на поиск

pioneer = Pioneer()
model = Yolo(model_name="yolov8n")
camera = Camera(camera_type=CameraType.MAIN)
in_air = False


def prepare(frame):
    # кадр BGR -> (1, 640, 640, 3)
    return np.expand_dims(cv2.resize(frame, (SIZE, SIZE)), 0)


def person_visible():
    result = model.run([prepare(camera.get_cv_frame())])
    if result is None or result[0] is None:
        return False
    boxes, classes, scores = result
    for cls, score in zip(classes, scores):
```

```
        if int(cls) == PERSON and score > 0.5:
            return True
    return False


try:
    if not pioneer.arm():
        raise SystemExit("Моторы не запустились")
    in_air = True
    pioneer.takeoff()

    state = "SEARCH"
    seen = 0
    start = time.time()
    while state == "SEARCH":
        # Медленный поворот на месте. interval короткий: если
        # программа зависнет, дрон через 0.5 с остановится сам.
        pioneer.set_manual_speed(vx=0, vy=0, vz=0,
                                 yaw_rate=0.3, interval=0.5)
        seen = seen + 1 if person_visible() else 0
        if seen >= CONFIRM:
            state = "FOUND"
        elif time.time() - start > SEARCH_TIME:
            state = "NOT_FOUND"

    pioneer.set_manual_speed(vx=0, vy=0, vz=0,
                             yaw_rate=0, interval=0.5)
    if state == "FOUND":
        print("Человек найден")
        pioneer.led_control(r=0, g=1, b=0)
    else:
        print("Никого не нашли")
        pioneer.led_control(r=1, g=0, b=0)
    time.sleep(3)

    pioneer.land()
    in_air = False
finally:
    if in_air:
        pioneer.land()
    pioneer.disarm()
    pioneer.led_control()
    model.release()
    camera.stop()
    pioneer.close_connection()
```

Что происходит. Это конечный автомат из трёх состояний: `SEARCH` → `FOUND` или `NOT_FOUND`, затем посадка. Три приёма делают его надёжным:

- **Подтверждение.** Нейросеть иногда ошибается на одном кадре. Человек засчитывается, только если его видно `CONFIRM` кадров подряд: счётчик `seen` растёт, пока человек виден, и обнуляется, как только пропал.
- **Короткий `interval`.** Поворот задаётся на каждом кадре с `interval=0.5`. Если программа зависнет, через полсекунды дрон перестанет вращаться сам. Положительный `yaw_rate` — поворот влево, не больше 1.82 рад/с (глава 16).

- **Таймаут.** Поиск не бесконечен: через `SEARCH_TIME` секунд автомат уходит в `NOT_FOUND`.

> **⚠ Первый запуск — без людей рядом**
>
> Полётные программы сборника Геоскана на дроне не запускались: сначала проверьте программу в симуляторе, затем на земле (закомментируйте взлёт) и с человеком на фото, а не в двух метрах от винтов. Дрон, который «ищет человека», не должен лететь к нему.

ЗАДАНИЕ 17.6 · Найти человека

В симуляторе поставьте человека вне поля зрения камеры, чтобы дрон нашёл его поворотом; на дроне — сначала на земле, с человеком на фото. Запустите `rknn_05_search_person.py`, затем уменьшите `SEARCH_TIME` до 10 с и проверьте ветку «никого не нашли».

Что должно получиться: с человеком в кадре — «Человек найден» и зелёный свет после нескольких кадров подряд; без людей через 10 с — «Никого не нашли», красный свет.

17.10 Своя модель

Модель из регистра знает 80 классов COCO. Чтобы дрон находил что-то своё — дорожные знаки, посадочные метки, детали на конвейере, — модель дообучают:

1. собирают и размечают фотографии нужных объектов;
2. обучают YOLOv8 или YOLO11 на компьютере или в облаке (библиотека Ultralytics);
3. экспортируют модель в ONNX с `opset_version=19` — этого требует регистр;
4. загружают в регистр: веб-интерфейс «Модели ИИ» или `ModelRegistry().upload_model(name, version, filepath, arch)`, где `version=""` означает «предыдущая плюс один», а `arch` — архитектура, например `"yolov8"`, или `"custom"` для своей сети. В `.rknn` ONNX переводит сам регистр. Готовый файл `.rknn` можно загрузить через веб-интерфейс или открыть напрямую: `Yolo(model_path="/home/pioneermini/best.rknn")`.

У своей модели свои классы: номера в ответе `run` — это номера из вашего набора данных, а не COCO. Обёртки `Yolo` разбирают выходы сети в том виде, в каком их отдаёт `yolov8n`, поэтому экспорт YOLO сверяйте с примерами репозитория https://github.com/airockchip/rknn_model_zoo. Сеть другой архитектуры открывают через `ModelContainer`: его наследник сам готовит вход и разбирает «сырые» выходы.

В симуляторе свою модель запустить нельзя: `upload_model` лишь записывает строку в регистр, а `model_path` приводит к `FileNotFoundError`. План дообучения и загрузки отрабатывается на бумаге и на дроне.

ЗАДАНИЕ 17.7 · План своей модели

Составьте план дообучения модели для поиска посадочной площадки: какие фотографии нужны и сколько, как их разметить, какой архитектурой обучать, в какой формат экспортировать и как загрузить на дрон. Укажите, какой номер класса будет у площадки в ответе `run`.

Что должно получиться: пять пунктов по шагам раздела; номер класса — из вашего набора данных (обычно 0), а не из COCO.

17.11 Задания к главе

ЗАДАНИЕ 17.8 · Счётчик людей

Посчитайте людей в кадре и показывайте число цветом подсветки: никого — выключена, один — зелёный, больше — синий. Цвет меняйте только при смене числа. В симуляторе расставьте людей перед камерой дрона.

Что должно получиться: подсветка соответствует числу людей в кадре и не мигает, пока число не меняется.

ЗАДАНИЕ 17.9 · Следование за человеком

Мини-проект. Дрон поворачивается так, чтобы человек был в центре кадра: `yaw_rate` пропорционален смещению центра рамки от центра кадра (П-регулятор из главы 8.10); человек левее центра — положительный `yaw_rate`. Короткий `interval`, ограничение $|yaw_rate| \le 1.82$ и посадка, если человек потерян дольше 5 секунд.

Что должно получиться: дрон поворачивается вслед за человеком и не раскачивается возле центра; через 5 с без человека в кадре садится.

Контрольные вопросы

1. Чем нейросеть-детектор отличается от поиска маркера по правилам? Что она получает и что возвращает?
2. Какую часть работы одного вызова `run()` делает NPU, а какую — процессор? Почему быстрее не станет, если отдать модели оба ядра?
3. Какой формы вход у `Yolo` и в каких координатах приходят рамки? Как пересчитать рамку в кадр 1080×720?
4. Как повышение `object_thresh` и снижение `nms_thresh` влияют на число найденных объектов?
5. Почему решение «человек найден» принимают по нескольким кадрам подряд?
6. Что вернёт `ocr.run` на кадре без текста? Что вернёт `Yolo.run` на кадре без объектов?
7. Что из результатов `pioneer_rknn` в симуляторе нельзя переносить на дрон без

> проверки?

Итоги

- Нейросеть — функция «кадр → объекты»; NPU считает саму сеть быстро, подготовка входа и постобработка идут на процессоре. Модель для NPU — файл `.rknn`.
- `Yolo` — объекты, `YoloSeg` — объекты с масками, `YoloPose` — скелет из 17 точек, `PaddleOCR` — текст; модели берутся из регистра по имени или из файла.
- Вход Yolo готовят так: 640×640, пакет из одного кадра, цвета BGR; рамки приходят в координатах 640×640 и пересчитываются в координаты кадра.
- `object_thresh` отсекает неуверенное, `nms_thresh` убирает дубли; отсутствие результата — `None` или `(None, None)`, его проверяют до разбора.
- Разбор ответа держите в одной функции; результат одного кадра подтверждайте несколькими. `model.release()` — в `finally`.
- В симуляторе вместо нейросети работает «зрение» сцены: людей, машин и надписи площадок видно, остальное нет; свою модель не загрузить, скорость не показательна.

ГЛАВА 18

Блоки Pioneer Bricks: программа без кода

Цели главы

После главы вы сможете:

- объяснить, что такое Pioneer Bricks и где выполняется блочная программа;
- собирать программы для Пионера Мини 2 из блоков: светодиоды, датчики, полёт, камера, нейросети;
- читать код, который Bricks генерирует из блоков, и связывать блоки с вызовами `pioneer_sdk2` и `pioneer_rknn`;
- обходить ограничения Bricks 2.3.0;
- отличать, что из блочной программы работает в симуляторе, а что только на дроне.

Блоки в симуляторе

Когда на панели «**Рой**» выбран Пионер Мини 2, переключатель языка редактора показывает «**Python**» и «**Bricks**»: блоки Bricks заменяют у Мини 2 обычный Blockly. В редакторе есть набор блоков по образцу Bricks 2.3.0, примеры по темам (среди них десять проектов `u01`–`u10` этой главы), открытие и сохранение проектов `.xml`. Все десять проектов запускаются в симуляторе; в проектах 7–9 вместо нейросети работает «зрение» симулятора (раздел «Симулятор и дрон» главы 17). Отличия от дрона:

- восемь блоков зрения (поворот и отражение кадра, размытие, Кэнни, цветовая и бинарная маски, контуры, форма контура) отключены с пометкой «Только на дроне» и в код не попадают;
- ошибки Bricks 2.3.0 из раздела 18.7 симулятор не повторяет: программа только из блоков светодиодов и вложенный блок распознавания текста в нём работают, а на дроне — нет;
- скорость кадров и распознавания в симуляторе другая, чем на борту.

Трек обучения ««**Мини 2 · Bricks**»» описан в конце главы.

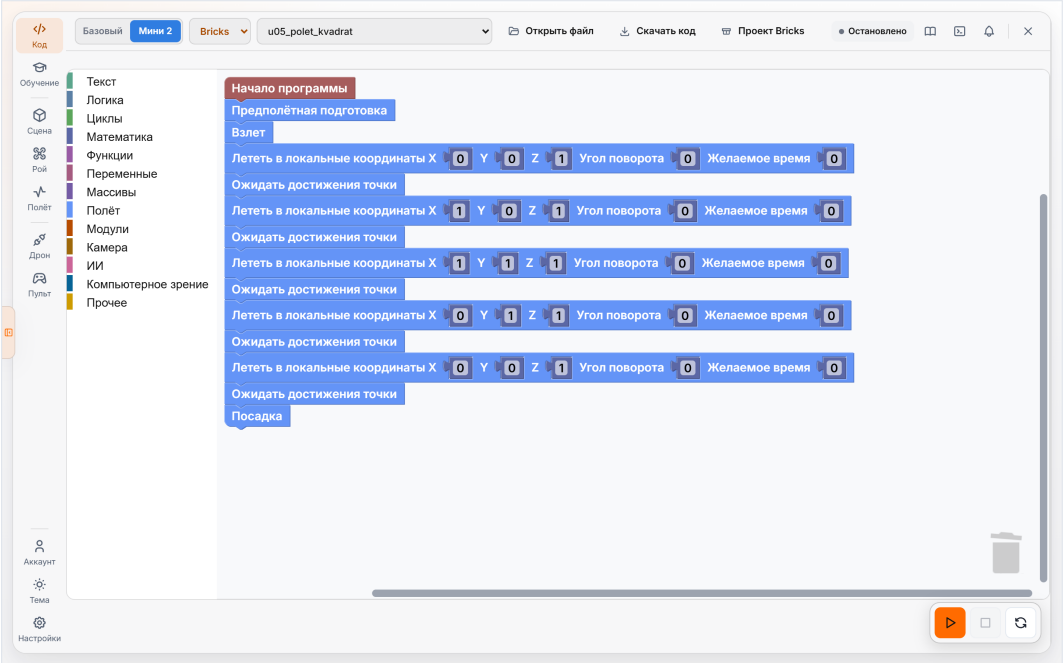


Рис. 31. Редактор Bricks в симуляторе: выбрана модель «**Мини 2**», язык «**Bricks**», открыт проект `u05_polet_kvadrat` (полёт по квадрату). Слева — категории блоков, справа вверху — кнопка «**Проект Bricks**».

18.1 Что такое Pioneer Bricks

Предыдущие главы были про текстовые программы на Python, которые работают на самом дроне. Но программировать можно и без текста. **Pioneer Bricks** — официальная среда визуального программирования Геоскана, построенная на Google Blockly. Программа собирается из цветных блоков, как пазл: блок «Взлёт» нельзя вставить туда, где нужно число, а блок с числом — туда, где нужна команда. Синтаксических ошибок, которые мучают новичка в тексте (забытая скобка, неверный отступ), в блоках просто не бывает.

Главное для нас: Bricks не изобретает свой язык. При сохранении он *генерирует программу на Python*, которая использует те же библиотеки `pioneer_sdk2` и `pioneer_rknn`, что и главы 16 и 17. Поэтому Bricks — удобная ступень перед текстом: любой блок можно «раскрыть» и увидеть, какой вызов за ним стоит. Глава написана по версии **Bricks 2.3.0** (Pioneer OS 1.0.0, `pioneer_sdk2` 0.16.0, `pioneer_rknn` 1.10.0). Что именно проверено на дроне: проект 5 (полёт) не запускался; проекты 7, 9 и 10 запускались на Мини 2 как сгенерированный код; остальные проверены по сгенерированному коду.

Где это запускается. Bricks работает на борту Мини 2, как и среда Pioneer Code. Подключитесь к Wi-Fi дрона и откройте в браузере главное меню Pioneer Code (`http://10.42.0.1:9090`), в нём пункт «Pioneer Bricks», либо сразу `http://10.42.0.1:2020`. Программа выполняется на дроне, а не в браузере. Путь от блоков до полёта — на рисунке 32.

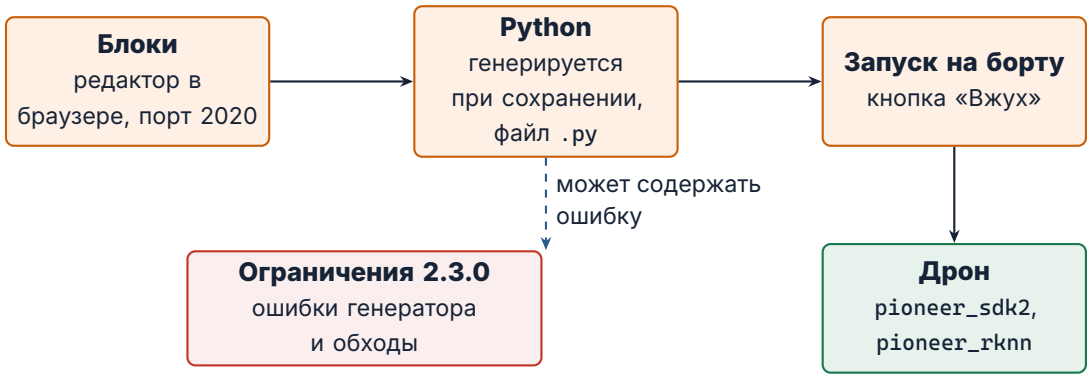


Рис. 32. Путь блочной программы. Блоки превращаются в обычную программу на Python, и она выполняется на дроне, а не в браузере. Если сгенерированный код ошибочен, программа не запустится: типичные случаи и обходы собраны в разделе 18.7.

Кнопка «Вжух» запускает программу на дроне

Кнопка «**Вжух**» в правом верхнем углу редактора *выполняет* программу на дроне. Если в ней есть блоки из категории «Полёт» (*Предполётная подготовка*, *Взлёт*, *Лететь в…*), дрон запустит двигатели и взлетит. Перед нажатием проверьте программу, площадку и людей вокруг; кнопка «**Стоп**» рядом прерывает выполнение. Правила безопасности те же, что в главе 16: пульт в руках, свободная площадка, аварийная остановка под рукой.

Из десяти проектов главы полётный только пятый, он помечен отдельно. Остальные выполняются при выключенных двигателях: дрон стоит на земле. Пока блоки категории «Полёт» не нужны, не добавляйте их в программу.

18.2 Проекты и редактор

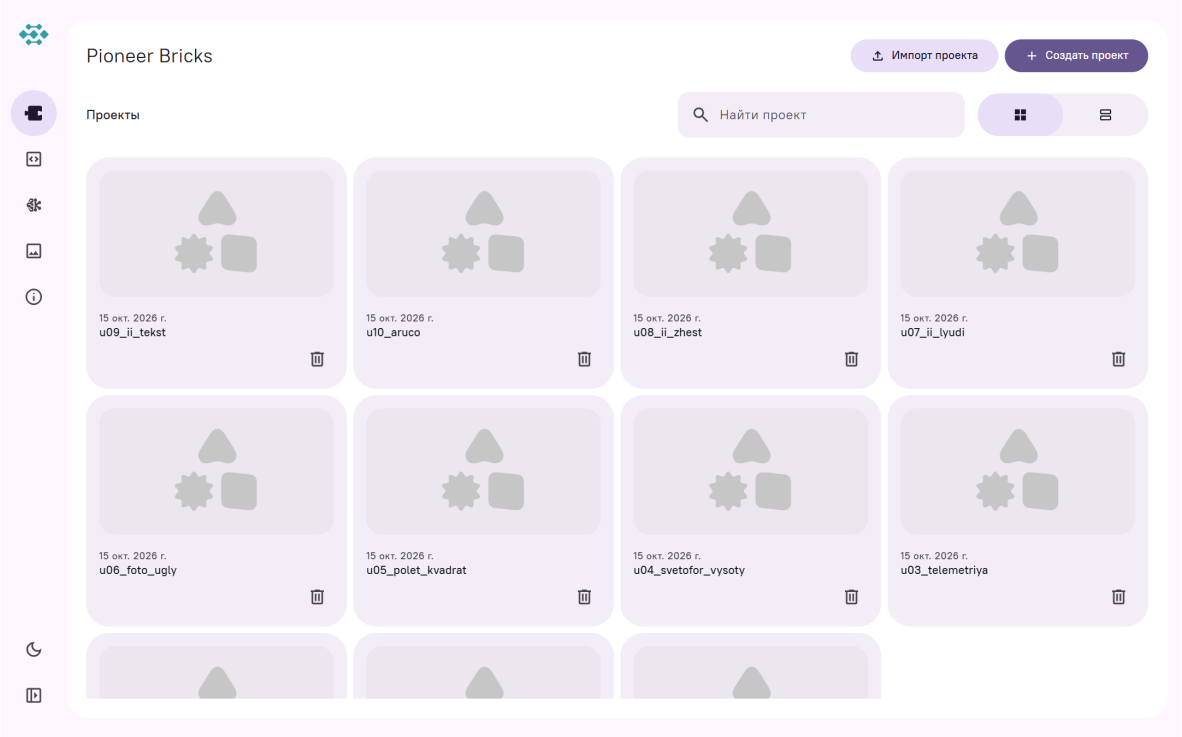


Рис. 33. Стартовая страница Bricks: «Создать проект», «Импорт проекта» и список проектов на дроне

Проекты хранятся на дроне в папке `/usr/local/bin/pioneer-bricks/static/save/<имя>/`: файл `<имя>.xml` — блоки, `<имя>.py` — сгенерированный Python (появляется после сохранения). Кнопки «скачать» и «загрузить» в редакторе экспортируют и импортируют проект целиком, так проекты переносят между дронами. К главе приложены десять готовых проектов — файлы `.xml` в папке `examples/python-onboard/bricks/`: нажмите «Импорт проекта» и выберите файл.

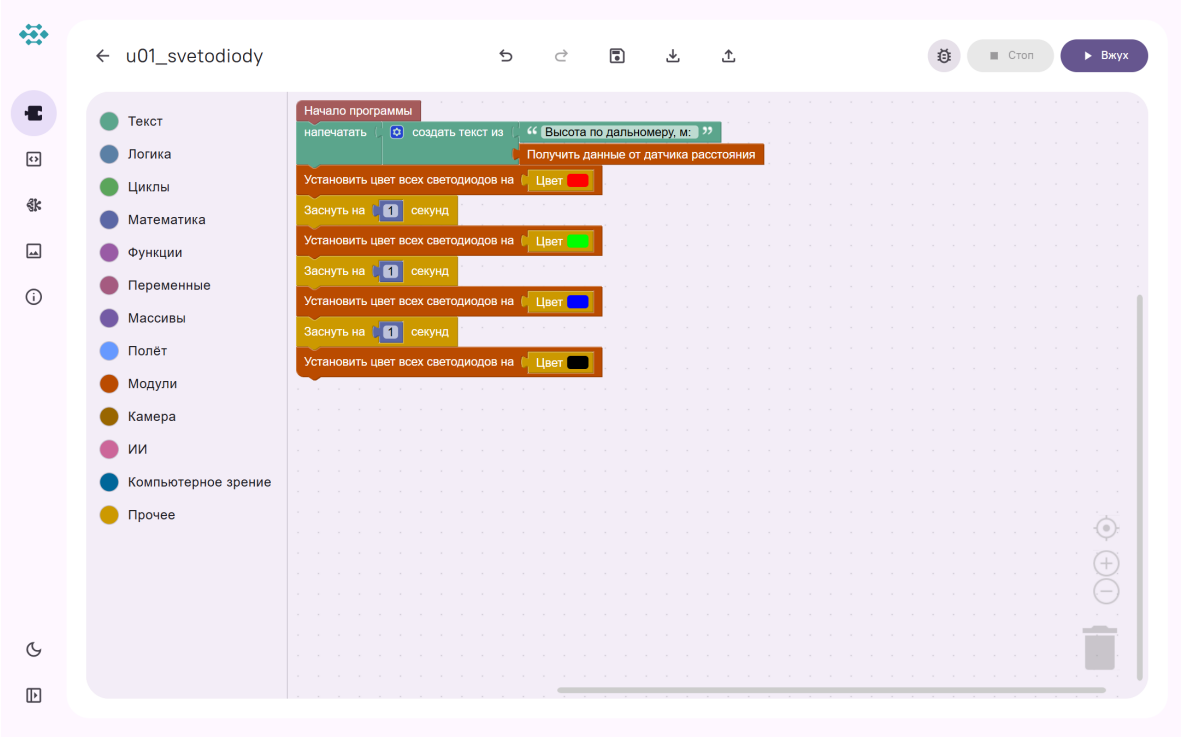


Рис. 34. Редактор Bricks с проектом 1

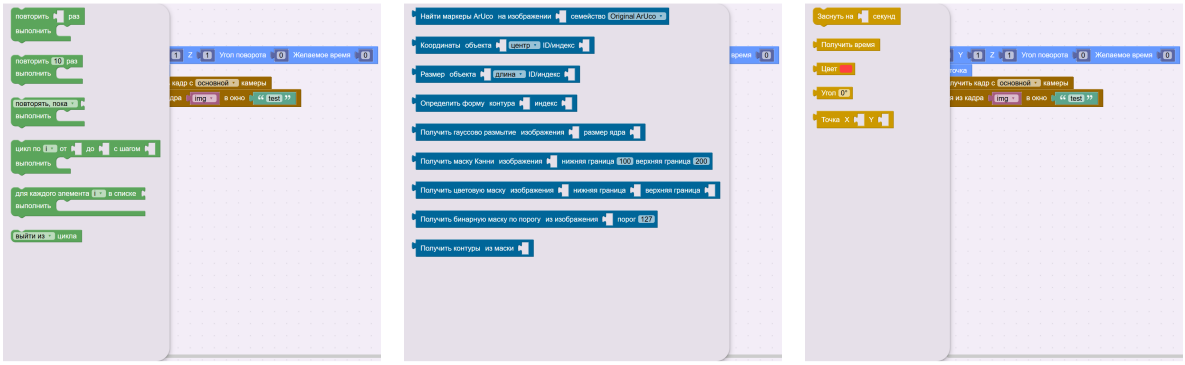
Элемент	Назначение
Палитра слева	категории блоков; щелчок открывает список, блок перетаскивается на рабочее поле
Рабочее поле	программа; выполнение начинается с неудаляемого блока «Начало программы» и идёт сверху вниз
Изогнутые стрелки	отменить / повторить действие
Дискета	«Сохранить» / «Сохранить как»: сохраняет блоки *и* генерирует Python
Стрелки вниз / вверх	экспорт проекта в файл / импорт
Жук	режим отладки
«**Стоп**» / «**Вжух**»	остановить / **запустить программу на дроне**
Справа внизу	центрирование, масштаб, корзина (перетащите блок в неё, чтобы удалить)

При запуске под полем появляются панели «Вывод программы» (то, что печатают блоки «напечатать») и «Просмотр изображений» (окна, в которые программа отправляет кадры блоком «Демонстрация изображения»). Те же окна открываются в браузере по адресу `http://10.42.0.1:8889/<имя окна>/`, как у `ImageViewer` в главе 16.

18.3 Блоки по категориям

Категория	Что внутри
Текст, Логика, Циклы, Математика	стандартные блоки Blockly: строки и печать, «если / иначе», сравнения, повторения, арифметика — то же, что в главах про основы Python
Функции, Переменные, Массивы	свои функции, переменные (*присвоить*, *получить*), списки
Полёт	предполётная подготовка (запуск двигателей), взлёт, посадка, полёт в точку, ожидание точки, скорости, курс
Модули	сервопривод камеры, захват, Ranger, светодиоды, дальномер, ориентация, координаты и скорости
Камера	получить кадр, показать в окне, сохранить фото, размер, поворот, отражение, обрезка, рисование
ИИ	модель (Yolo, YoloPose, PaddleOCR — блока для YoloSeg нет), поиск объектов и поз, сравнение позы, распознавание текста
Компьютерное зрение	маркеры ArUco, координаты и размер объекта, размытие, Кэнни, цветовые и бинарные маски, контуры
Прочее	пауза «Заснуть на», время, выбор цвета, угла, точки

Содержимое палитр, специфичных для дрона, показано на рис. 35 и 36.



Циклы Компьютерное зрение Прочее

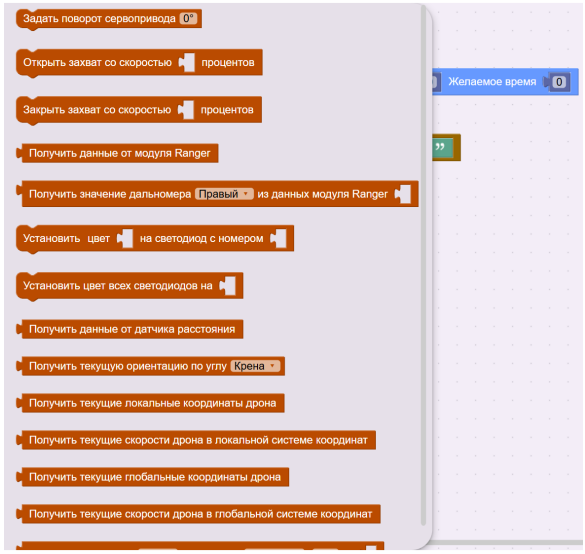
Рис. 36. Ещё три палитры

18.4 Во что превращаются блоки

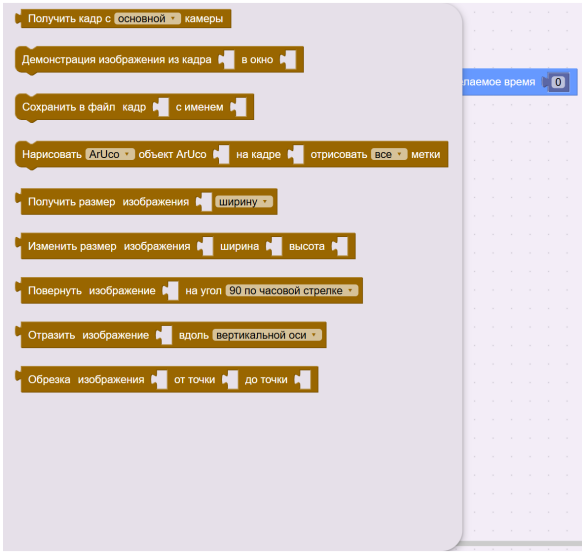
Сгенерированный файл всегда устроен одинаково: импорты и создание объектов (`Pioneer()`, камеры, `ImageViewer`, модели), затем *тело программы* внутри `try`, а в `finally` — безопасное завершение. Вот полный файл самой короткой программы (проект 3 ниже). Это та же схема «подключение → `try` → `finally`», что вы писали руками в главе 16:



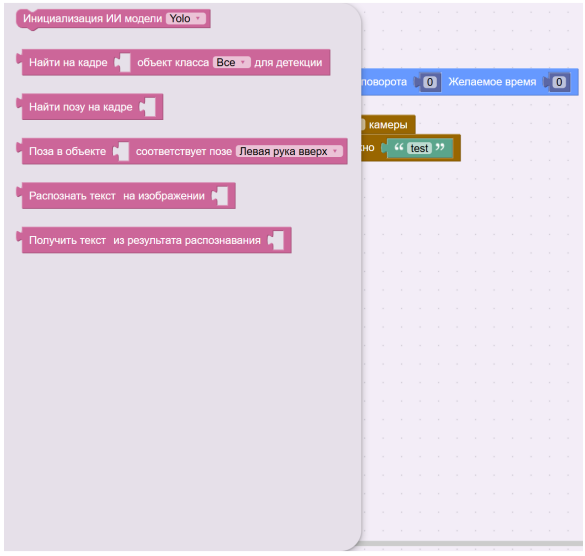
Полёт



Модули



Камера



ИИ

Рис. 35. Палитры блоков для дрона

python-onboard/bricks/python/full_u03_telemetriya.py

```
from pioneer_sdk2 import Pioneer
pioneer = Pioneer()
from time import sleep

print("Начало программы")
try:
    for count in range(10):
      print('Высота по дальномеру, м: ' +
      ↪  str(pioneer.get_dist_sensor_data()))
      sleep(1)
except Exception as e:
    print(e)
finally:
    pio = locals().get("pioneer")
    if pio is not None:
        try:
            pio.wait_callback = False
            pio.land()
            pio.close_connection()
        except Exception:
            pass

    for cam_name in ("cam_main", "cam_opt"):
        cam = locals().get(cam_name)
        if cam is not None:
            try:
                cam.stop()
            except Exception:
                pass

    viewer = locals().get("iv")
    if viewer is not None:
        try:
            viewer.close()
        except Exception:
            pass
        del viewer

    if pio is not None:
        del pio
    print("Конец программы")
```

В остальных листингах главы показано только тело программы; оно вставляется внутрь `try`. Таблица соответствия основных блоков:

Блок	Python (`pioneer_sdk2` / `pioneer_rknn`)
Предполётная подготовка / Взлёт / Посадка	`pioneer.arm()` / `pioneer.takeoff()` / `pioneer.land()`
Лететь в локальные координаты	`pioneer.go_to_local_point(x, y, z, yaw, time)`
Ожидать достижения точки	`while not pioneer.point_reached(): pass`
Установить цвет всех светодиодов	`pioneer.led_control(255, r, g, b)`; цвет из вида `#RRGGBB` переводится в доли 0..1
Установить цвет светодиода с номером	`pioneer.led_control(i, r, g, b)`
Получить данные от датчика расстояния	`pioneer.get_dist_sensor_data()` — дальномер вниз, высота в метрах
Задать поворот сервопривода	`servo.set_angle(angle=...)`
Получить кадр с основной камеры	`cam_main.get_cv_frame()`
Демонстрация изображения в окно	`iv.imshow(name=..., frame=...)`, окно `:8889/<имя>/`
Сохранить в файл кадр	`cv2.imwrite(путь, кадр)`, путь `/mnt/media/photos/<имя>.jpg`
Инициализация ИИ модели Yolo / YoloPose / PaddleOCR	`Yolo(model_name="yolov8n")` / `YoloPose(model_name="yolov8n-pose")` / `PaddleOCR(...)` с моделями `PP-OCRv5_mobile_det` и `eslav_PP-OCRv5_mobile_rec`
Заснуть на	`sleep(секунды)`

Смотрите на сгенерированный код

Если программа из блоков ведёт себя неожиданно, откройте её `.py`-файл (в среде Pioneer Code, папка проекта Bricks): по коду почти всегда видно, в чём дело. А когда блоков станет тесно, переходите к программам на Python из глав 16 и 17: те же задачи решаются там короче и гибче.

18.5 Безопасное завершение

В `finally` каждой программы, которая использует `pioneer`, Bricks вызывает `pioneer.land()`, останавливает камеры и закрывает окна просмотра. Что бы ни случилось внутри (ошибка в блоке, нажатие «**Стоп**», пропавший кадр), дрон в конце садится. Ошибка при этом не «роняет» программу молча: перед `finally` стоит `except Exception as e: print(e)`, и текст ошибки появляется в панели «Вывод программы». Это защита от «зависшего» в воздухе дрона — то самое правило «посадка в `finally`», которое в главе 16 вы соблюдали вручную. Здесь оно встроено, но рассчитывать только на него не нужно: полётную программу всё равно заканчивайте явным блоком «Посадка», а пульт держите в руках.

18.6 Десять проектов

Проекты идут по возрастанию сложности: сначала последовательность команд и циклы, затем условия и датчики, потом полёт и камера, в конце нейросети и компьютерное зрение. У каждого есть снимок блоков, сгенерированный Python (тело программы, без импортов и `finally`), разбор и результат. Готовые проекты лежат в `examples/python-onboard/bricks/`. Если не сказано иное, проект выполняется на земле, при выключенных двигателях. В симуляторе работают все десять, что именно отличается от дрона, сказано в разборе проекта.

№	Проект	Файл	Тема
1	Светодиоды	`u01`	последовательность команд
2	Бегущий огонь	`u02`	циклы, переменная-счётчик
3	Телеметрия	`u03`	датчик и вывод данных
4	Светофор высоты	`u04`	условия, бесконечный цикл
5	Полёт по квадрату	`u05`	полётное задание (**полёт**)
6	Фото под углами	`u06`	сервопривод, кадр, файл
7	ИИ: люди	`u07`	детекция Yolo
8	ИИ: жест	`u08`	поза YoloPose
9	ИИ: текст	`u09`	PaddleOCR
10	ArUco	`u10`	компьютерное зрение

Проект 1. Светодиоды: последовательность команд

Блоки выполняются сверху вниз, один за другим. Это первая программа, цель — увидеть, как цепочка блоков превращается в цепочку строк.



Рис. 37. Проект 1: светодиоды красным, зелёным, синим

python-onboard/bricks/python/u01_svetodiody.py

```
print('Высота по дальномеру, м: ' + str(pioneer.get_dist_sensor_data()))
pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#ff0000")][::-1])
sleep(1)
pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#00ff00")][::-1])
sleep(1)
pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#0000ff")][::-1])
```

```
sleep(1)
pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#000000")][::-1])
```

- Цвет задаётся блоком «Цвет» из категории «Прочее»; Bricks переводит его из вида `#RRGGBB` в доли 0..1 для `led_control`. Чёрный цвет — светодиоды выключены.
- «Заснуть на 1 секунд» — пауза: без неё смена цветов была бы незаметна.
- Первый блок «напечатать высоту» нужен из-за особенности Bricks 2.3.0 (раздел 18.7): без блока, читающего данные дрона, в программу не попадает строка `pioneer = Pioneer()`. В симуляторе эта особенность не повторяется.

Что должно получиться: светодиоды по секунде горят красным, зелёным и синим, затем гаснут.

Проект 2. Бегущий огонь: циклы и переменная-счётчик



Рис. 38. Проект 2: огонь бежит по четырём светодиодам

python-onboard/bricks/python/u02_beguschiy_ogon.py

```
print('Высота по дальномеру, м: ' + str(pioneer.get_dist_sensor_data()))
for count in range(3):
  for i in range(4):
    pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#000000")][::-1])
    pioneer.led_control(i, *[c / 255 for c in hex_to_bgr("#ffaa00")][::-1])
    sleep(0.3)
pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#000000")][::-1])
```

- «повторить 3 раз» — внешний цикл: огонь пробегает три круга.
- «цикл по `i` от 0 до 3 с шагом 1» — внутренний цикл; `i` принимает значения 0, 1, 2, 3 и служит номером светодиода. Bricks превратил его в `range(4)`: граница «до 3» включается.
- Перед каждым включением все светодиоды гасятся, поэтому горит ровно один. В симуляторе у Мини 2 два светодиода (0 — левый, 1 — правый): на номерах 2 и 3 `led_control` ничего не зажигает и возвращает `False`, поэтому огонь там пропадает.

Что должно получиться: на дроне оранжевый огонь проходит по номерам 0, 1, 2, 3 три раза, затем светодиоды гаснут; в симуляторе виден только на номерах 0 и 1.

ЗАДАНИЕ 18.1 · Огонь в обратную сторону

Пустите огонь от 3 до 0 с шагом −1. Затем добавьте второй, «догоняющий» огонь другого цвета на светодиоде с номером `(i + 2) % 4`.

Что должно получиться: по кругу бегут два огонька разных цветов, расстояние между ними не меняется.

Проект 3. Телеметрия: вывод данных

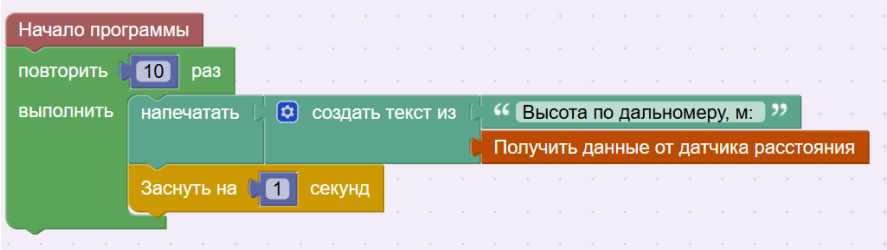


Рис. 39. Проект 3: десять измерений высоты

python-onboard/bricks/python/u03_telemetriya.py

```
for count in range(10):
  print('Высота по дальномеру, м: ' + str(pioneer.get_dist_sensor_data()))
  sleep(1)
```

- «создать текст из» склеивает подпись и число; Bricks сам добавил `str(...)`, чтобы склеить строку с числом.
- 10 повторений с паузой в секунду — десять измерений за десять секунд.

Что должно получиться: в панели «Вывод программы» десять строк «Высота по дальномеру, м: …». Поднимите дрон рукой (при выключенных двигателях!) — значение изменится.

Проект 4. Светофор высоты: условия

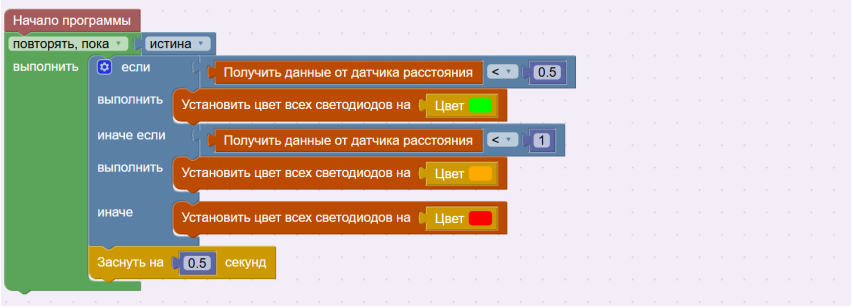


Рис. 40. Проект 4: цвет светодиодов зависит от высоты

python-onboard/bricks/python/u04_svetofor_vysoty.py

```
while True:
  if pioneer.get_dist_sensor_data() < 0.5:
    pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#00ff00")][::-1])
  elif pioneer.get_dist_sensor_data() < 1:
    pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#ffaa00")][::-1])
  else:
    pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#ff0000")][::-1])
  sleep(0.5)
```

- Ветки проверяются по порядку: ниже 0.5 м — зелёный; иначе ниже 1 м — жёлтый; во всех остальных случаях — красный. Проверка повторяется каждые полсекунды.
- Шестерёнка на блоке «если» добавляет ветки «иначе если» и «иначе».
- «повторять, пока истина» — бесконечный цикл; программа работает до нажатия «**Стоп**». Здесь сказывается безопасное завершение: после остановки выполнится `finally`.

ЗАДАНИЕ 18.2 · Свой светофор

Добавьте четвёртый уровень: выше 2 м светодиоды мигают красным. Подсказка: понадобится ещё одна ветка «иначе если» и пауза внутри неё.

Что должно получиться: при подъёме дрона рукой цвет последовательно меняется зелёный, жёлтый, красный, мигающий красный.

Проект 5. Полёт по квадрату

Этот проект поднимает дрон в воздух

Запускайте его только на подготовленной площадке 2×2 м с пультом в руках. При подготовке главы эта программа на дроне **не запускалась**: показаны только блоки и сгенерированный код. Проверьте её сначала в симуляторе, затем запускайте на дроне с одним отрезком пути.

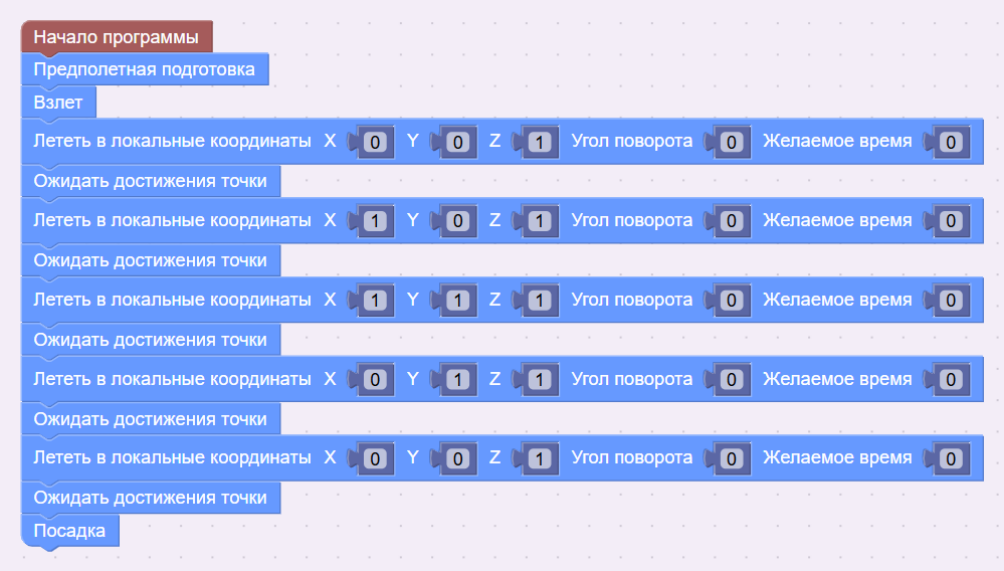


Рис. 41. Проект 5: взлёт, пять точек на высоте 1 м, посадка

python-onboard/bricks/python/u05_polet_kvadrat.py

```
pioneer.arm()
pioneer.takeoff()
pioneer.go_to_local_point(0,0,1,0,0)
while not pioneer.point_reached():
    pass
pioneer.go_to_local_point(1,0,1,0,0)
while not pioneer.point_reached():
    pass
pioneer.go_to_local_point(1,1,1,0,0)
while not pioneer.point_reached():
    pass
pioneer.go_to_local_point(0,1,1,0,0)
while not pioneer.point_reached():
    pass
pioneer.go_to_local_point(0,0,1,0,0)
while not pioneer.point_reached():
    pass
pioneer.land()
```

- «Лететь в локальные координаты» *не ждёт* прилёта, поэтому после каждого полёта стоит «Ожидать достижения точки». Без него дрон сразу получит следующую команду и «срежет» угол квадрата.
- Точек пять: сначала дрон поднимается над стартом до высоты 1 м, затем проходит четыре вершины квадрата со стороной 1 м и возвращается над старт.
- Это та же программа, что полёт по квадрату в главе 16, только собранная из блоков.

Что должно получиться: дрон взлетает, облетает квадрат 1×1 м на высоте 1 м, возвращается над стартом и садится.

Проект 6. Фото под разными углами камеры

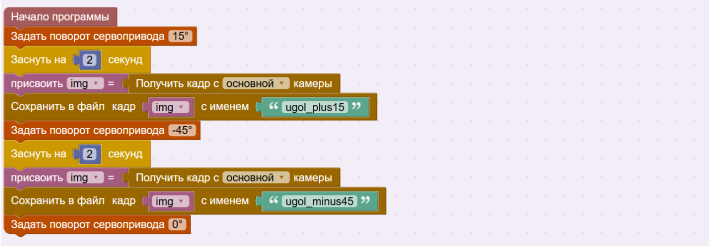


Рис. 42. Проект 6: наклон камеры, кадр, сохранение

python-onboard/bricks/python/u06_foto_ugly.py

```
servo.set_angle(angle=15)
sleep(2)
img = cam_main.get_cv_frame()

cv2.imwrite("/mnt/media/photos/" + 'ugol_plus15' + ".jpg", img.copy())
servo.set_angle(angle=-45)
sleep(2)
img = cam_main.get_cv_frame()

cv2.imwrite("/mnt/media/photos/" + 'ugol_minus45' + ".jpg", img.copy())
servo.set_angle(angle=0)
```

- «Задать поворот сервопривода» — угол выбирается на циферблате с шагом $15°$; сервопривод принимает углы от $-80°$ до $+30°$. Если вписать 20, Bricks округлит до 15, а -80 до -75 (раздел 18.7).
- «Сохранить в файл кадр с именем» пишет снимок в `/mnt/media/photos/<имя>.jpg`: фото сразу видны в «Галерее медиа» Pioneer Code. В симуляторе файлов нет: блок отрабатывает, но снимок нигде не появится.
- Пауза в 2 секунды после поворота нужна, чтобы камера успела повернуться и снимок не смазался.

Что должно получиться: программа печатает высоту, дважды наклоняет камеру ($+15°$ и $-45°$), делает два снимка, возвращает камеру в $0°$.

ЗАДАНИЕ 18.3 · Фото-серия

Снимите четыре кадра с углами $0°$, $-30°$, $-60°$, $-75°$ и именами, по которым видно угол. Используйте цикл с переменной или четыре повтора.

Что должно получиться: в галерее четыре фото с понятными именами; при большем наклоне в кадре больше пола.

Нейросети и компьютерное зрение

Блоки ИИ работают так же, как классы `Yolo`, `YoloPose` и `PaddleOCR` из главы 17: сначала блок «Инициализация ИИ модели» загружает модель на NPU, затем в цикле кадр

камеры передаётся модели. Результат удобно смотреть в окне: блок «Демонстрация изображения из кадра … в окно *имя*» публикует кадр, окно открывается по адресу `http://10.42.0.1:8889/<имя>/`. Проекты 7, 9 и 10 проверены запуском сгенерированного кода на дроне. В симуляторе блоки ИИ вызывают тот же `pioneer_rknn`, что и программы главы 17: находятся люди и машины сцены, а не любые объекты кадра.

Проект 7. Найти людей



Рис. 43. Проект 7: детекция людей и рамки на кадре

python-onboard/bricks/python/u07_ii_lyudi.py

```
print("Выбрана модель:Yolo")
IMG_SIZE = (640, 640)

model_yolo = Yolo(model_name=model_name_picker("Yolo"))
while True:
  img = cam_main.get_cv_frame()

  people = yolo_find_on_image(img, (results_img :=
  ↪  infer_cache_per_frame("results_img", img, lambda fr:
  ↪  model_yolo.run([resize_img_yolo(fr)]))), classNames_yolo,
  ↪  class_filter=["person"])

  draw_yolo_on_image(img, people, hex_to_bgr("#00ff00"))
  iv.imshow(name='people', frame=img)
```

- «Найти на кадре `img` объект класса Человек» — детекция с фильтром по классу (в списке 80 классов COCO по-русски). Результат сохраняется в переменную `people`.
- «Нарисовать Yolo объект `people` на кадре `img`» рисует рамки, «Демонстрация…» отправляет кадр в окно `people`.
- Bricks сам добавил кэш `infer_cache_per_frame`: если на одном кадре нейросеть вызывается несколько раз, она считает его один раз.

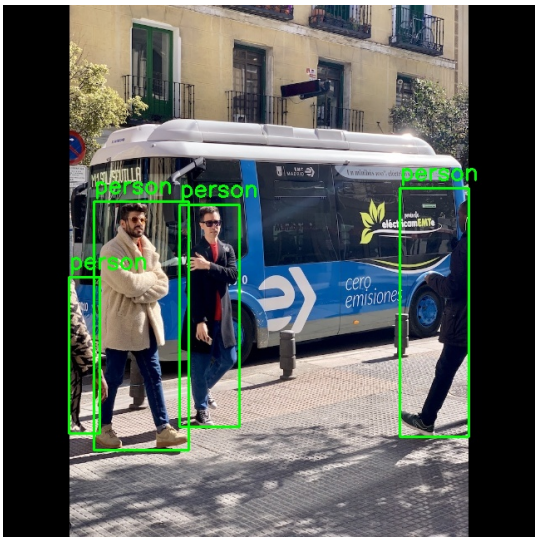


Рис. 44. Результат проекта 7 на дроне (на вход подано тестовое изображение): рамки вокруг людей, автобус не выделен — работает фильтр «Человек»

Что должно получиться: окно `http://10.42.0.1:8889/people/` с рамками вокруг людей; на дроне программа обрабатывала около 7–8 кадров в секунду. В симуляторе рамки появляются у людей, которые попали в поле зрения камеры.

Проект 8. Жест «две руки вверх» включает свет

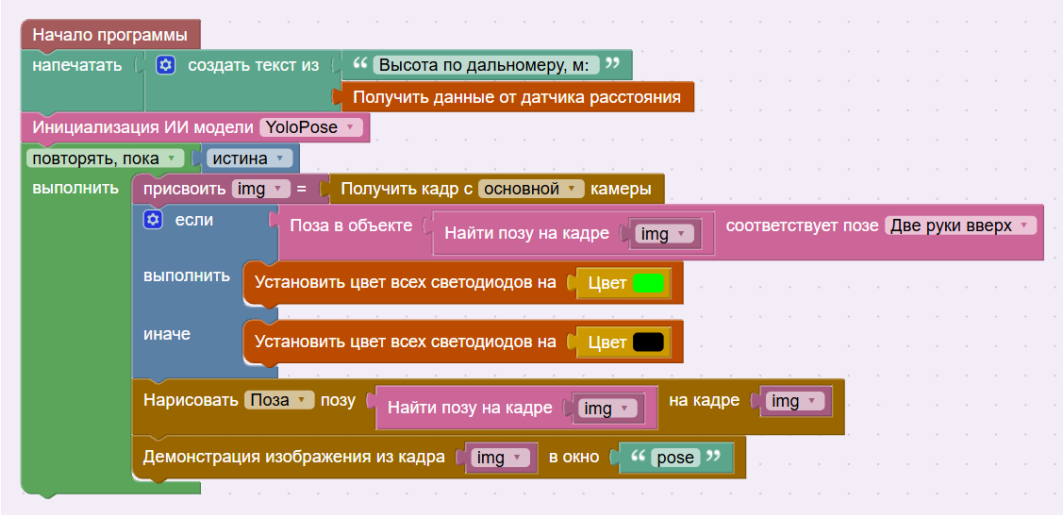


Рис. 45. Проект 8: поза человека управляет светодиодами

python-onboard/bricks/python/u08_ii_zhest.py

```
print('Высота по дальномеру, м: ' + str(pioneer.get_dist_sensor_data()))
print("Выбрана модель:YoloPose")
IMG_SIZE = (640, 640)

model_yolopose = YoloPose(model_name=model_name_picker("YoloPose"))
while True:
```

```
img = cam_main.get_cv_frame()

if pose_compare(find_poses(img, (results_img :=
↪  infer_cache_per_frame("results_img", img, lambda fr:
↪  model_yolopose.run([resize_img_yolopose(fr)])))
, "two_hand_up", last_cmds):
  pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#00ff00")][::-1])
else:
  pioneer.led_control(255, *[c / 255 for c in hex_to_bgr("#000000")][::-1])
draw_poses(img, find_poses(img, (results_img :=
↪  infer_cache_per_frame("results_img", img, lambda fr:
↪  model_yolopose.run([resize_img_yolopose(fr)])))
, tracker_state)
iv.imshow(name='pose', frame=img)
```

- «Поза в объекте … соответствует позе Две руки вверх» возвращает «истину», только если жест держится 8 кадров подряд (параметр `THRESHOLD` в сгенерированном коде) и принадлежит самому крупному человеку в кадре. Это защита от случайных срабатываний: тот же приём устойчивого решения по нескольким кадрам вы встретили в главе 17.
- Доступные позы: левая и правая рука вверх или в сторону, две руки вверх, обе в стороны, кисти рядом в центре.
- «Нарисовать Позу» рисует скелет на кадре, окно называется `pose`.

Что должно получиться: поднимите обе руки — через 8 кадров светодиоды загорятся зелёным; опустите — погаснут.

Проект 9. Распознать текст

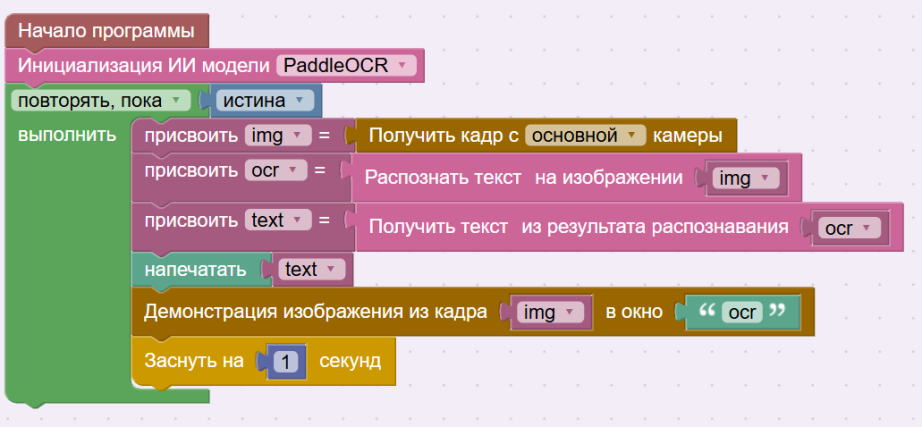


Рис. 46. Проект 9: распознавание текста PaddleOCR

```
print("Выбрана модель:PaddleOCR")
model_ppocr = PaddleOCR(det_model_name=model_name_picker("PaddleOCR")[0],
                        rec_model_name=model_name_picker("PaddleOCR")[1])
while True:
  img = cam_main.get_cv_frame()

  ocr = [results_ppocr_img := infer_cache_per_frame("results_ppocr_img",
  ↪  img, lambda fr: model_ppocr.run(fr))]
  text = "\n".join([text for group in ocr[0][1] for (text, _) in group])
  print(text)
  iv.imshow(name='ocr', frame=img)
  sleep(1)
```

- Результат распознавания сначала сохраняется в переменную `ocr`, и только потом из него извлекается текст. Если вложить блоки друг в друга напрямую, Bricks 2.3.0 сгенерирует код с синтаксической ошибкой (раздел 18.7).
- «Заснуть на 1 секунд»: распознавание тяжёлое, чаще раза в секунду оно обычно и не нужно.

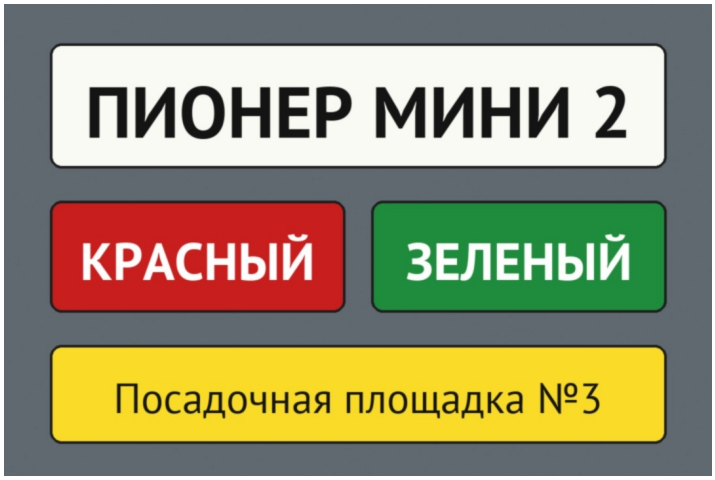


Рис. 47. Окно `ocr` программы на дроне (на вход подана табличка с текстом)

Вывод программы на дроне для таблички с рис. 47:

```
ПИОНЕР МИНИ2
КРАСНЬИ
зеленый
Посадочная площадка
```

В слове «КРАСНЬИ» вторая буква С латинская: модель смешивает похожие буквы двух алфавитов, поэтому результат сравнивают по смыслу, а не посимвольно. В симуляторе читаются только надписи площадок («Н», «ПОСАДКА», «ЗАРЯДКА», «СТАРТ / N»), и камеру нужно направить на площадку сверху.

> **Окно ocr появляется не сразу**
>
> Распознавание медленное, да ещё в цикле стоит пауза в секунду: в окно уходит меньше одного кадра в секунду. Поэтому окно `http://10.42.0.1:8889/ocr/` начинает показывать видео только через 20–30 секунд после запуска — браузер ждёт первого полного (ключевого) кадра. Уберите блок «Заснуть», если нужно более живое изображение.

Проект 10. Маркеры ArUco



Рис. 48. Проект 10: поиск и отрисовка маркеров ArUco

python-onboard/bricks/python/u10_aruco.py

```python
while True:
  img = cam_main.get_cv_frame()

  markers = aruco_find_on_image('img', img, cv2.aruco.DICT_4X4_50)
  draw_aruco_on_image(img, markers, None, color=(0,255,0))
  iv.imshow(name='aruco', frame=img)
```

- «Найти маркеры ArUco на изображении `img` семейство 4x4_50» — семейство должно совпадать с напечатанными маркерами.
- Блоки «Координаты объекта» и «Размер объекта» из «Компьютерного зрения» дают центр и размер маркера с нужным ID — с ними можно, например, поворачивать дрон к маркеру.

✅ **Что должно получиться:** в окне `http://10.42.0.1:8889/aruco/` найденные маркеры обведены и подписаны номерами; на дроне программа работала с частотой около 7 кадров в секунду. В симуляторе маркеры ищутся на метках сцены.

ЗАДАНИЕ 18.4 · Маркер в центре

Выведите в «Вывод программы» координаты центра маркера с ID 0. Включайте зелёный свет, когда маркер находится в центральной трети кадра, и красный — когда нет.

✅ **Что должно получиться:** при смещении маркера в кадре цвет светодиодов меняется на границах центральной трети; координаты в выводе согласованы с положением маркера.

18.7 Особенности и ограничения Bricks 2.3.0

Сгенерированный код каждого проекта проверяли. Вот что обнаружено. Третья колонка показывает, повторяется ли особенность в симуляторе: если нет, проверить обход можно только на дроне.

Ситуация	Что происходит и как обойти	В симуляторе
Программа только из блоков светодиодов	блок светодиодов не добавляет в код `pioneer = Pioneer()`; программа завершится с ошибкой `name 'pioneer' is not defined`. **Обход:** поставьте в начало любой блок, читающий данные дрона (например, «напечатать» и «Получить данные от датчика расстояния») — проекты 1, 2, 8	не повторяется: программа работает и без обхода
«Распознать текст» вставлен прямо в «Получить текст из результата…»	генерируется `:=` внутри генератора списка — `SyntaxError` в Python 3.12, программа не запускается. **Обход:** сохраните результат в переменную (проект 9)	не повторяется: программа работает
Угол сервопривода	поле-циферблат округляет угол до шага $15°$: 20 превращается в 15, -40 в -45	повторяется
Длинные блоки	вложенные блоки быстро становятся очень широкими; удобнее сохранять промежуточные результаты в переменные (проекты 7, 9, 10)	то же
Блоки зрения без нейросети (поворот и отражение кадра, размытие, Кэнни, маски, контуры)	на дроне работают, это обычные функции OpenCV	отключены, пометка «Только на дроне»

> **Версия 2.3.0**
>
> Список относится к Bricks 2.3.0 на Pioneer OS 1.0.0; в других версиях поведение может отличаться. Если сомневаетесь, откройте сгенерированный `.py` и сравните с таблицей соответствия блоков.

18.8 Блоки и Python: что с чем сравнивать

Каждый из десяти проектов можно написать на Python из глав 16 и 17; это хороший способ закрепить и то, и другое.

Проект Bricks	Где то же на Python
1, 2 — светодиоды, циклы	светодиоды в главе 16
3, 4 — телеметрия, условия	дальномер и датчики в главе 16
5 — полёт по квадрату	полёт по точкам в главе 16
6 — сервопривод и фото	камера и сервопривод в главе 16
7 — найти людей	детекция `Yolo` в главе 17
8 — жесты	`YoloPose` в главе 17
9 — текст	`PaddleOCR` в главе 17
10 — ArUco	маркеры ArUco в главе 8, с учётом кадра из `Camera`

ЗАДАНИЕ 18.5 · Из блоков в Python

Выберите любой из проектов 1–4, откройте его сгенерированный `.py` и перепишите тело программы на Python вручную: без `hex_to_bgr`, с обычными числами для цвета. Запустите в Pioneer Code или в редакторе Python симулятора.

Что должно получиться: ваша программа делает то же, что и блочная: те же цвета, паузы и вывод; `finally` с посадкой у вас на месте.

ЗАДАНИЕ 18.6 · Свой проект

Соберите в Bricks программу, которая показывает на светодиодах расстояние до препятствия: дальномер Ranger или датчик высоты, три цвета и пауза в полсекунды. Сохраните проект, экспортируйте его в `.xml` и импортируйте на соседний дрон.

Что должно получиться: цвет светодиодов меняется вместе с расстоянием; проект открывается на другом дроне без правок.

ЗАДАНИЕ 18.7 · Где ошибка

Только на дроне: в симуляторе ошибка не повторяется. Соберите блок «Распознать текст» прямо внутри «Получить текст из результата…» и сохраните проект. Откройте сгенерированный код, найдите строку с `:=` и исправьте проект через переменную `ocr`.

Что должно получиться: без переменной программа не запускается (`SyntaxError`), с переменной — печатает распознанный текст.

Контрольные вопросы

1. Где выполняется программа из блоков Bricks: в браузере, на компьютере или на дроне? Что делает кнопка «**Вжух**»?
2. Что генерирует Bricks при сохранении проекта и где лежат оба файла проекта?
3. Почему после «Лететь в локальные координаты» нужен блок «Ожидать достижения точки»?
4. Что выполняется в `finally` сгенерированной программы и почему всё равно нужен явный блок «Посадка»?
5. Почему результат распознавания текста сохраняют в переменную? Повторяется ли эта ошибка в симуляторе?
6. До какого значения округлится угол сервопривода $-40°$ и в каких пределах он вообще меняется?
7. Какие блоки главы нельзя проверить в симуляторе и почему?

Итоги

- Bricks — блоки на Google Blockly; при сохранении они превращаются в Python на `pioneer_sdk2` и `pioneer_rknn`. Программа выполняется на дроне, кнопка «**Вжух**»

запускает её, включая полёт.

- Файл программы устроен всегда одинаково: создание объектов, тело в `try`, безопасное завершение в `finally` с посадкой. Ошибки печатаются в «Вывод программы».
- «Лететь в точку» не ждёт прилёта: после неё нужен блок ожидания. Цвета переводятся из `#RRGGBB` в доли 0..1, угол сервопривода округляется до $15°$.
- Блоки ИИ вызывают `Yolo`, `YoloPose` и `PaddleOCR`; жест подтверждается 8 кадрами подряд. Блока для `YoloSeg` нет.
- Обходы ошибок Bricks 2.3.0 (блок чтения данных дрона в начале, результат OCR в переменную) нужны на дроне; симулятор этих ошибок не повторяет, а восемь блоков зрения в нём отключены.

Миссии симулятора к главе

Режим «**Обучение**» содержит трек ««**Мини 2 · Bricks**»»: 16 миссий на блоках Pioneer Bricks. Миссия открывает дрон Пионер Мини 2 и редактор блоков Bricks, проверяет состав блоков и полёт в симуляторе. Миссии идут по проектам и логике главы: первые блоки и взлёт (1–2), светодиоды (проект 1, миссии 3–4), циклы и бегущий огонь (проект 2, миссии 5–6), данные и условия (проекты 3 и 4, миссии 7–9), полёт по точкам и квадрату (проект 5, миссии 10–12), камера, сервопривод и фото (проект 6, миссии 13–14), метки ArUco (проект 10, миссия 15) и итоговый световой патруль (16). Проекты 7–9 с нейросетями (люди, жесты, текст) в трек не вошли, но запускаются в редакторе Bricks по готовым проектам `u07`–`u09`.

ЧАСТЬ V

ROS 2 и рой на Пионере

ГЛАВА 19

ROS 2 на Пионере: пакет gs_core

Цели главы

После главы вы сможете:

- объяснить, что такое узел, топик, сервис и параметр ROS 2, и чем программа на `rclpy` отличается от программы на `pioneer_sdk2`;
- провести дрон через полётный цикл вызовами сервисов `gs_core` и задавать точки в системах координат `map` и `base_link`;
- читать телеметрию и события автопилота подпиской и управлять дроном скоростью (`cmd_vel`) и виртуальным пультом;
- читать и писать параметры автопилота, переключать систему навигации, управлять светодиодами и захватом;
- собрать миссию как узел-автомат без блокирующих ожиданий и объяснить, что делает `ros_pioneer_node`;
- отличать отказ сервиса (`success = False`) от ошибки программы и находить причину в журнале.

Нужно знать: Python с классами (часть II) и полёт `pioneer_sdk2` из главы 16.

Миссии симулятора к главе

В треке «**Мини 2 · SDK2**» режима «**Обучение**» к главе относятся миссии 38–56 (главы трека 14–17): сервисы полёта (38–43), топики и телеметрия (44–49), параметры, навигация и нагрузка (50–54), итоговый узел (55–56). Таблица соответствия — в конце главы. Программы запускаются в режиме «**Python · SDK2**» на модели Мини 2 (панель «**Рой**»); ROS 2 — часть этого режима, отдельного языка нет.

На что опирается глава и что не проверено

Основа главы — справочник пакета `gs_core` для ROS 2 Jazzy на docs.geoscan.ru (Пионер Мини 2 и Пионер Базовый с модулем Razero) и десять примеров из `examples/python-onboard/ros2/`. Все программы главы проверены на симуляторе (на его эмуляции `rclpy` и узла `ros_pioneer_node`); *на настоящем дроне авторы их не запускали*.

Справочник Геоскана не описывает часть деталей: имена полей ответов сервисов, частоты топиков, момент возврата блокирующих вызовов. В таких местах глава описывает поведение симулятора и помечает это словами «в симуляторе». Точные поля своих сервисов смотрите командой `ros2 interface show <тип>` на дроне, а первый полёт любой программы проводите на открытой площадке без людей.

19.1 Что такое ROS 2

ROS 2 (Robot Operating System) — не операционная система, а набор правил и библиотек, по которым программы робота общаются друг с другом. Робот делится на небольшие программы, *узлы* (node): один читает датчики, другой считает маршрут, третий управляет моторами. Узлы находят друг друга сами и обмениваются данными по общим правилам, поэтому узел, написанный другим человеком, можно подключить, не меняя остальные. Версия на Пионере — ROS 2 Jazzy, программы пишутся на Python с библиотекой `rclpy`.

Между узлами ходят данные трёх видов:

	Что это	Пример на Пионере
Топик (topic)	непрерывный поток сообщений: кто-то *публикует*, кто-то *подписан*; публикующий не знает, сколько у него читателей	высота борта `/geoscan/sensors/altitude`: несколько сообщений в секунду, каждый подписчик получает каждое
Сервис (service)	запрос и ответ: клиент отправляет запрос, сервер выполняет и возвращает результат	`/geoscan/flight/takeoff` (запрос пустой, ответ — `success`)
Параметр (parameter)	настройка узла по имени: читается и (иногда) меняется во время работы	`topics.gnss` — создавать ли топики GPS

Каждый топик и сервис имеет *имя* (путь вида `/geoscan/flight/arm`) и *тип*: топик передаёт сообщения типа `std_msgs/msg/Float32`, сервис принимает запрос и отдаёт ответ типа `std_srvs/srv/Trigger`. В Python типы лежат в модулях `std_msgs.msg`, `std_srvs.srv`, `geometry_msgs.msg`, `gs_interfaces.srv` и других: поля сообщения — обычные атрибуты, у сервиса есть классы `Сервис.Request` и `Сервис.Response`. Пакет `gs_interfaces` — типы самого Геоскана.

Для знакомых вещей из предыдущих глав сравните:

pioneer_sdk2	ROS 2 (gs_core)
`pioneer.arm()`	вызов сервиса `/geoscan/flight/arm`
`get_local_position_lps()` (опрос)	подписка на `/geoscan/navigation/local/position`: данные приходят сами
`subscribe(f, Event.X)`	подписка на `/geoscan/flight/callback_event`
`set_manual_speed(...)`	публикация в `/geoscan/flight/cmd_vel`
`get_param`, `set_param`	сервисы `/geoscan/board/get_param` и `/geoscan/board/set_param`

19.2 Зачем ROS 2 на Пионере

`pioneer_sdk2` — одна библиотека «всё в одном»: вы вызываете методы, она сама держит соединение. ROS 2 устроен иначе, и у него три причины жить на дроне:

- **Общий язык с остальной робототехникой.** Пакеты навигации, зрения и SLAM из мира ROS 2 ждут именно такие топики и сервисы. Камера Пионера — тоже отдельный узел (`camera_node` из пакета `gs_camera`), и её кадры доступны всем.
- **Много программ одновременно.** Один узел читает телеметрию, другой управляет скоростью, третий пишет журнал; их запускают и останавливают независимо, а не склеивают в один цикл.
- **Готовые инструменты.** Команды `ros2 topic echo`, `ros2 service call` и `ros2 param` позволяют посмотреть на дрон и дёрнуть его из терминала, не написав ни строки кода. Те же правила использует Swarm Kit (глава 20).

Цена — многословность: простое «взлететь» превращается в создание клиента, запрос и ожидание ответа. Поэтому для первых программ и коротких миссий выбирайте `pioneer_sdk2`, а ROS 2 — когда нужны несколько независимых программ, стандартные инструменты или совместимость с чужими пакетами.

19.3 Узел ros_pioneer_node

Связь Python-программы с автопилотом обеспечивает пакет `gs_core`: его узел `ros_pioneer_node` подключается к автопилоту по TCP (`127.0.0.1:20556`), публикует телеметрию в топики, принимает команды и открывает сервисы управления полётом, параметрами автопилота и полезной нагрузкой (рисунок 49).

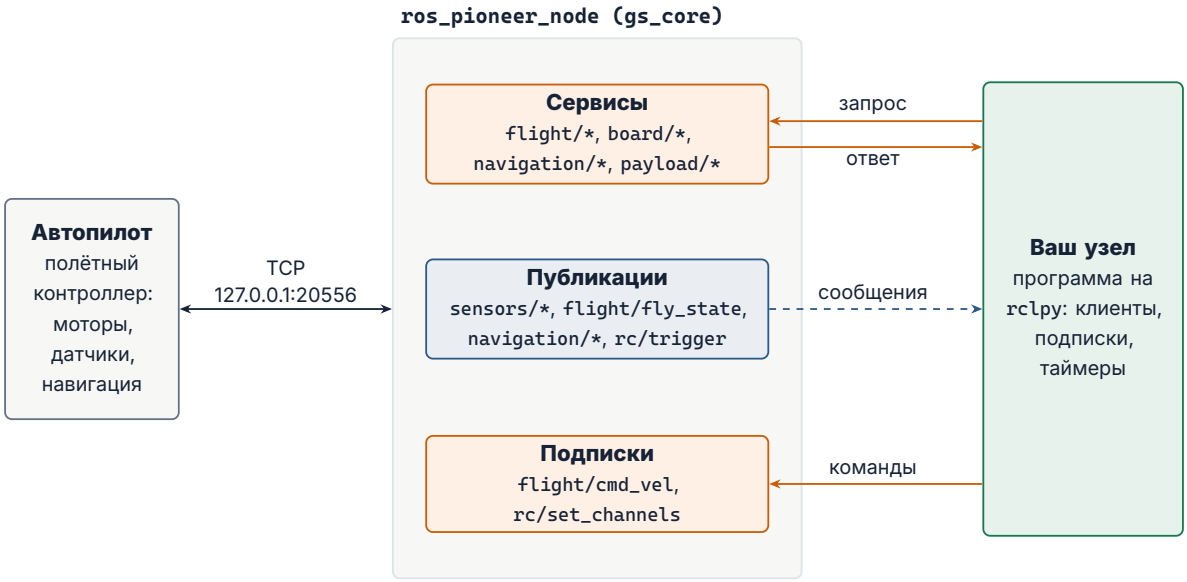


Рис. 49. Узел `ros_pioneer_node` — мост между автопилотом и ROS 2. К автопилоту он подключён по TCP, наружу открывает сервисы (запрос → ответ), публикует телеметрию (штриховая стрелка) и принимает команды скорости и пульта. Ваша программа — ещё один узел сети; все имена начинаются с `/geoscan/`.

Узел запускается launch-файлом, по одному на модель дрона, и читает конфигурацию борта (YAML):

Борт	Launch-файл	Состав
Пионер Мини 2	`pioneer_mini2_full_launch.py`	`ros_pioneer_node` и две камеры (`camera_node`, `camera_servo`)
Пионер Базовый с модулем Razero	`pioneer_razero_full_launch.py`	`ros_pioneer_node` и основная камера

Терминал дрона: запуск узла (по справочнику Геоскана)

```
ros2 launch gs_core pioneer_mini2_full_launch.py
ros2 run gs_core ros_pioneer_node --ros-args --params-file mini2.yaml
```

Пакет входит в стек `ros-jazzy-pioneer-mini2` (для Razero — `ros-jazzy-pioneer-razero`); отдельно его ставят командой `sudo apt install ros-jazzy-gs-core`. Узлу нужен бортовой модуль `proto` версии 2.0.0 или новее: без него узел не запустится. Ваша программа — тоже узел: она подключается к той же сети ROS 2, вызывает сервисы и подписывается на топики `/geoscan/...`.

Узел в симуляторе

В симуляторе `ros_pioneer_node` уже работает, когда вы садитесь за редактор: он запускается сам при первом `rclpy.init()` или при первой команде консоли и берёт конфиг `mini2` для Мини 2 и `razero` для Базового с Radxa. Он переживает вашу про-

грамму: после её остановки исчезают только её узлы. Время (таймеры, ожидания, `spin`) идёт по часам симуляции и ускоряется вместе с ней.

19.4 Консоль ros2 и первый узел

В режиме «**Python · SDK2**» в нижней панели редактора есть вкладка «**ROS 2**»: консоль, в которой работают привычные команды `ros2`. Она работает и без запущенной программы, поэтому с неё удобно начинать:

Консоль ros2 в симуляторе

```
ros2 topic list
ros2 topic echo --once /geoscan/sensors/battery_state
ros2 service list
ros2 interface show gs_interfaces/srv/Arm
ros2 service call /geoscan/flight/arm gs_interfaces/srv/Arm \
  "{timeout: 5, retries: 0}"
```

Консоль понимает `ros2 topic`, `service`, `interface`, `param`, `node`, `launch` и `run` для `gs_core`; она проще настоящего `ros2`: нет `topic hz`, записи bag-файлов и ряда ключей. Вставка нескольких строк выполняет их подряд без пауз, поэтому `go_to_local_point` сразу после `takeoff` будет отклонён (дрон ещё не в воздухе); кнопка «**Выполнить**» у полётного цикла в подсказке консоли ждёт состояние дрона между командами.

Первый узел — проверка связи с автопилотом: сервис `/geoscan/alive` отвечает, жив ли обмен с ним.

Первый узел: на связи ли автопилот

```
import rclpy
from rclpy.node import Node
from gs_interfaces.srv import Live

rclpy.init()
node = Node('hello')
client = node.create_client(Live, '/geoscan/alive')
future = client.call_async(Live.Request())
rclpy.spin_until_future_complete(node, future)
alive = future.result().status
print('Узел', node.get_name(), ': автопилот на связи:', alive)
node.destroy_node()
rclpy.shutdown()
```

Что происходит. `rclpy.init()` готовит библиотеку, `Node('hello')` создаёт узел с именем `hello`. Клиент сервиса создаётся по типу и имени; `call_async` отправляет запрос и сразу возвращает `future` (обещание ответа), а `spin_until_future_complete` крутит обработку, пока ответ не придёт. В конце узел уничтожают, а библиотеку закрывают `rclpy.shutdown()`. Этот каркас — основа любой программы главы.

ЗАДАНИЕ 19.1 · Первое касание

Откройте консоль, выведите список топиков и сервисов, прочитайте одно сообщение `battery_state` (ключ `--once`). Затем запустите программу «Первый узел» и вызовите `ros2 interface show gs_interfaces/srv/Live`: какие поля у запроса и ответа?

Что должно получиться: консоль показывает десятки имён с префиксом `/geoscan/`; программа печатает «автопилот на связи: True»; у `Live` запрос пустой, в ответе — поле `status`.

В симуляторе: миссии Мини 2 SDK2: 38 в панели «**Обучение**».

19.5 Сервисы полёта: полётный цикл

Цель раздела: провести дрон через цикл «моторы → взлёт → точка → посадка → выключение» вызовами сервисов и понять, чем ответ сервиса отличается от окончания действия.

Всё управление полётом — сервисы с префиксом `/geoscan/flight/`:

Сервис	Тип	Что делает
`flight/arm`	`gs_interfaces/Arm`	запускает моторы; запрос: `timeout` (секунд ждать подтверждения), `retries` (повторы)
`flight/takeoff`	`std_srvs/Trigger`	взлёт
`flight/land`	`std_srvs/Trigger`	посадка
`flight/disarm`	`std_srvs/Trigger`	остановка моторов
`flight/rtl`	`std_srvs/Trigger`	возврат домой
`flight/go_to_local_point`	`gs_interfaces/GoToLocalPoint`	полёт в точку: `x`, `y`, `z`, `yaw`, `time`, `frame_id`
`flight/go_to_global_point`	`gs_interfaces/GoToGlobalPoint`	полёт в точку по GPS
`flight/set_yaw`	`gs_interfaces/Yaw`	поворот на месте, `angle` в градусах

Эти сервисы управляют тем же автопилотом, что и методы `pioneer_sdk2`; разница лишь в способе вызова. Полный цикл, как его описывает справочник Геоскана:

python-onboard/ros2/ros2_01_flight_cycle.py

```
# Полётный цикл через сервисы gs_core: arm -> takeoff -> go_to_local_point ->
↪  land -> disarm.
# Пионер Мини 2 и Пионер Базовый с Radxa. Эквивалент команд ros2 service call
↪  из документации.
import rclpy
```

```python
from rclpy.node import Node
from std_msgs.msg import Int8
from std_srvs.srv import Trigger
from gs_interfaces.srv import Arm, GoToLocalPoint

ON_LAND, ARMED, IN_SKY = 0, 1, 2   # значения топика flight/fly_state


class FlightCycle(Node):
    def __init__(self):
        super().__init__('flight_cycle')
        self.fly_state = ON_LAND
        self.create_subscription(Int8, '/geoscan/flight/fly_state',
        ↪  self.on_fly_state, 10)
        self.arm_client = self.create_client(Arm, '/geoscan/flight/arm')
        self.takeoff_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/takeoff')
        self.go_client = self.create_client(GoToLocalPoint,
        ↪  '/geoscan/flight/go_to_local_point')
        self.land_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/land')
        self.disarm_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/disarm')

    def on_fly_state(self, msg):
        self.fly_state = msg.data

    def call_service(self, client, request):
        # Вызов сервиса: отправить запрос и крутить обработку, пока не придёт
        ↪  ответ.
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

    def wait_for_state(self, wanted):
        while rclpy.ok() and self.fly_state != wanted:
            rclpy.spin_once(self, timeout_sec=0.1)


def main():
    rclpy.init()
    node = FlightCycle()
    try:
        if not node.call_service(node.arm_client, Arm.Request(timeout=5.0,
        ↪  retries=0)).success:
            raise SystemExit('Моторы не запустились')
        node.call_service(node.takeoff_client, Trigger.Request())
        node.wait_for_state(IN_SKY)
        node.get_logger().info('Взлетели')

        go = GoToLocalPoint.Request(x=1.0, y=1.0, z=1.0, yaw=0.0, time=0.0,
        ↪  frame_id='map')
        node.call_service(node.go_client, go)
        # Подождать, пока дрон долетит: пауза по времени симуляции.
        for _ in range(60):
            rclpy.spin_once(node, timeout_sec=0.1)
```

```
        node.call_service(node.land_client, Trigger.Request())
        node.wait_for_state(ON_LAND)
        node.get_logger().info('Сели')
    finally:
        node.call_service(node.disarm_client, Trigger.Request())
        node.destroy_node()
        rclpy.shutdown()


main()
```

Что происходит. Узел `FlightCycle` создаёт по клиенту на каждый сервис и подписку на `flight/fly_state` (число 0, 1 или 2: на земле, моторы запущены, в воздухе). `call_service` — общий способ вызвать любой сервис: отправить запрос `call_async` и крутить обработку, пока не придёт ответ. `Arm.Request(timeout=5.0, retries=0)` — запрос со своими полями; ответ `arm` проверяется по `success`, и если моторы не запустились, программа прекращается. Блок `finally` выключает моторы, а узел и `rclpy` закрываются при любом исходе.

> **Ответ сервиса — это «принято», а не «сделано»**
>
> В симуляторе `takeoff`, `land`, `go_to_local_point`, `set_yaw` и `rtl` отвечают, как только команда *принята*, а не когда дрон взлетел или прилетел. Об окончании сообщают топики `flight/fly_state` и `flight/callback_event`, поэтому пример ждёт `fly_state`, а не следующую команду сразу. Исключения: `arm` ждёт подтверждения моторов (по `timeout` на каждую из `retries + 1` попыток), а `grab` с `movement_time > 0` — это время. На настоящем дроне момент возврата сверьте с поведением своих сервисов.

> **Безопасность**
>
> Программа главы — не замена пилота: включите пульт и держите руку на аварийной остановке, первые полёты проводите в сетке или на открытой площадке. В примере `finally` только выключает моторы: если программу прервут в воздухе, это уронит дрон. В своих программах, как в главе 16, при прерывании сначала сажайте дрон (по `fly_state`), а выключайте моторы на земле.

Как узел ждёт: spin, таймеры и future

Узел ROS 2 — не цикл, который вы пишете сами. Ему нужно давать время на работу: пока вы не вызвали `spin`, подписки молчат, а ответы сервисов не приходят.

Вызов	Что делает
`rclpy.spin(node)`	крутит узел, пока работает `rclpy` (до остановки программы)
`spin_once(node, timeout_sec=0.1)`	обрабатывает всё накопленное, но не дольше 0.1 с; удобно в своём цикле `while rclpy.ok()`
`spin_until_future_complete(node, f)`	крутит узел, пока не придёт ответ `f`
`node.create_timer(0.1, f)`	вызывает `f` каждые 0.1 с (во время `spin`)
`create_subscription(тип, имя, f, 10)`	вызывает `f(msg)` на каждое сообщение; `10` — глубина очереди
`client.call_async(request)`	отправляет запрос, возвращает `future`

В примере `wait_for_state` вызывает `spin_once` по кругу, пока `fly_state` не станет нужным: обратный вызов подписки меняет поле, цикл его проверяет. Это основной приём главы: *обратные вызовы коротко запоминают факт, а решения принимает цикл или таймер*. Тот же принцип, что и в правиле об обработчиках событий главы 16.

Как rclpy работает в симуляторе

В браузере нет потоков, поэтому раннер сам дописывает ожидание к `spin`, `spin_once`, `wait_for_service` и `Rate.sleep`: в программе они выглядят как в настоящем `rclpy`. Ответ сервиса приходит в `future` и без `spin` клиента (в настоящем `rclpy` — при обработке экзекьютором). Поля сообщений проверяются строго, как в настоящем: `Point(x=1)` даёт `AssertionError`, нужно `1.0`. Вызов сервиса, которого нет, не повисает навсегда в миссиях симулятора, но в вашей программе проверяйте `client.wait_for_service(timeout_sec=...)`.

ЗАДАНИЕ 19.2 · Полётный цикл руками и кодом

Выполните цикл из консоли (`arm`, `takeoff`, `go_to_local_point`, `land`, `disarm`) с паузами между командами. Затем в программе добавьте вторую точку $(0, 0, 1)$ и дождитесь прибытия в каждую по событию (следующий раздел) или паузой по времени симуляции. Печатайте `fly_state` при каждой смене.

Что должно получиться: дрон проходит обе точки и садится; значения `fly_state` в журнале идут 0, 1, 2, 0 (на земле, моторы запущены, в воздухе, на земле).

В симуляторе: миссии Мини 2 SDK2: 39 в панели «**Обучение**».

19.6 Системы координат map и base_link

Цель раздела: понимать, в каких осях и единицах задаются точки, скорости и курс, — от этого зависит каждая команда движения.

В сообщениях и запросах `gs_core` есть поле `frame_id`: оно говорит, в какой системе координат понимать числа. Допустимы два значения (рисунок 50):

- `'map'` — мировая (локальная) система, неподвижная: начало в точке старта, ось z вверх. Те же координаты показывает `get_local_position_lps()` в `pioneer_sdk2` и топик `navigation/local/position`;
- `'base_link'` — система самого коптера: x вперёд (по носу), y влево, z вверх (так принято в ROS, стандарт REP 103). Для `go_to_local_point` это смещение от текущего положения.

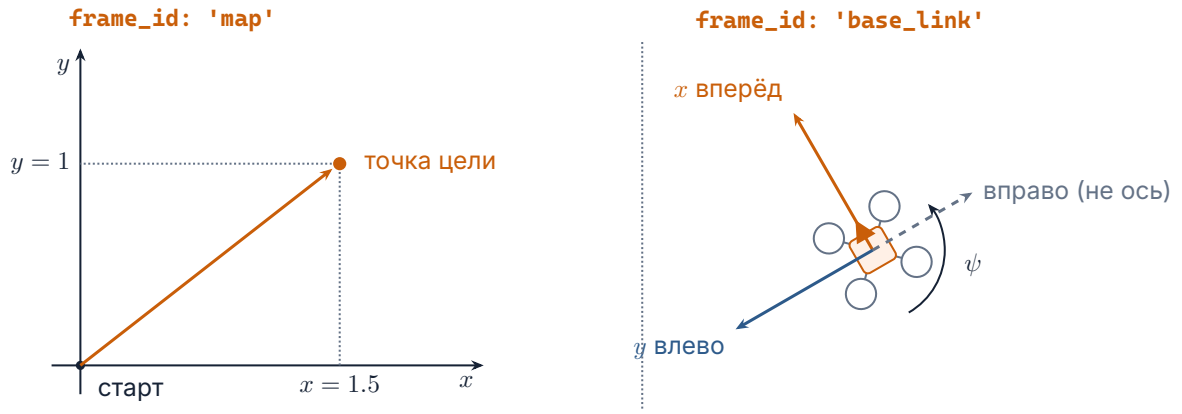


Рис. 50. Две системы для `go_to_local_point` и `cmd_vel`. `map` неподвижна: точка задаётся от старта. `base_link` едет вместе с дроном: x вперёд, y влево (как в стандарте ROS), курс ψ повёрнут против часовой стрелки. У `pioneer_sdk2` в осях корпуса всё иначе: x вправо, y вперёд.

> **⚠ Не путайте оси base_link и оси корпуса pioneer_sdk2**
>
> В `go_to_local_point_body_fixed` и `set_manual_speed_body_fixed` из главы 16 ось x смотрит *вправо*, а y — *вперёд*. В `base_link` наоборот: x — вперёд, y — влево. Поэтому «вперёд на метр» там `y=1`, а здесь `x=1`, и команды переносятся с заменой осей, а не копированием.

Пустой или чужой `frame_id` сервис полёта отвергает: ответ `success = False` и запись в журнале; у `cmd_vel` такое сообщение игнорируется с предупреждением.

Курс (yaw). Запрос `go_to_local_point` содержит `yaw`, а сервис `set_yaw` — `angle`; оба в *градусах* (так сказано в справочнике `gs_core`). Правило справочника: если угол совпадает с предыдущим заданным (в `go_to_*` или `set_yaw`), рыскание не меняется. Поэтому в примере `yaw=0.0` означает не «развернись к нулю», а «курс оставь прежним, если он уже 0». Чтобы повернуться, задайте другой угол: он считается абсолютным курсом, а не приращением. Скорость поворота в `cmd_vel`, наоборот, в рад/с (`angular.z`), а курс из топика `navigation/local/yaw` в симуляторе приходит в радианах.

> **⚠ Единицы курса: проверьте на своём дроне**
>
> Для `pioneer_sdk2` единицы курса в `go_to_*` неоднозначны (врезка в разделе 16.6); справочник `gs_core` говорит о градусах прямо, и симулятор следует ему. Но на-

стоящий дрон авторы не запускали. Проверьте: вызовите `go_to_local_point` с `yaw: 90.0`, затем `yaw: 1.57` и после каждого прочитайте курс из `navigation/local/yaw`. Направление нулевого курса и знак поворота тоже сверьте глазами: в симуляторе при `yaw = 0` нос смотрит по $+Y$ карты, а положительный курс — против часовой стрелки (вид сверху).

Типичные запросы в двух системах:

Консоль ros2: одна и та же команда в разных осях

```
# в map: лететь в точку (1.5, 0.5) на высоте 1 м от старта
ros2 service call /geoscan/flight/go_to_local_point \
  gs_interfaces/srv/GoToLocalPoint \
  "{x: 1.5, y: 0.5, z: 1.0, yaw: 0.0, time: 5, frame_id: 'map'}"
# в base_link: сместиться на метр вперёд от текущего положения
ros2 service call /geoscan/flight/go_to_local_point \
  gs_interfaces/srv/GoToLocalPoint \
  "{x: 1.0, y: 0.0, z: 0.0, yaw: 0.0, time: 3, frame_id: 'base_link'}"
```

Поле `time` — желаемое время перехода в секундах: `0` оставляет выбор скорости автопилоту. Прилёт сервис не ждёт: его фиксируют по событию `POINT_REACHED` (следующий раздел). Для маршрута из нескольких точек действуйте как в миссиях: отправили точку — дождались события — отправили следующую.

ЗАДАНИЕ 19.3 · Квадрат в map и base_link

Напишите программу, которая взлетает на метр и проходит квадрат 1×1 м сначала в `map` (четыре абсолютные точки), затем в `base_link` (четыре раза: шаг вперёд на метр и поворот налево на $90°$ — по правилу курса, абсолютными углами 90, 180, 270 и 0 или 360). После каждой стороны печатайте позицию из `navigation/local/position`.

Что должно получиться: оба квадрата замыкаются, дрон возвращается к старту с ошибкой порядка десятков сантиметров; шагов по `base_link` — четыре строки «Сторона» с растущим или убывающим курсом.

В симуляторе: миссии Мини 2 SDK2: 40, 41 в панели «**Обучение**».

19.7 Отказ сервиса и возврат домой

Сервис полёта может *отказать*: ответ приходит, но `success = False`. Это не ошибка программы, а штатное «нельзя»: `takeoff` до `arm`, `go_to_local_point` на земле, чужой `frame_id`, повторная посадка на земле. Причину узел записывает в журнал (`ros_pioneer_node`); в консоли симулятора причина печатается серым после вызова. Правило: всегда проверяйте `success` и решайте, что делать — прекратить миссию, повторить или перейти в безопасную ветку.

Проверка отказа

```
request = GoToLocalPoint.Request(x=1.0, y=0.0, z=1.0, frame_id='map')
future = client.call_async(request)
rclpy.spin_until_future_complete(node, future)
if not future.result().success:
    node.get_logger().error('Точка отклонена: причина в журнале узла')
```

Возврат домой. Сервис `flight/rtl` (`Trigger`) отправляет дрон в точку старта. Как и остальные команды движения, он не ждёт окончания; закончилось ли возвращение, видно по `navigation/local/position` и по состоянию на земле. В симуляторе `rtl` приводит дрон к старту и зависает над ним на небольшой высоте: автоматическая посадка выключена параметром автопилота `Flight_com_landAtHome`, поэтому после возврата дрон сажают вызовом `land`. На настоящем дроне поведение зависит от параметров автопилота — проверьте его до полёта.

ЗАДАНИЕ 19.4 · Домой по сервису

Взлетите, отлетите в точку $(1.5, 0.5, 1)$, вызовите `rtl` и по позиции дождитесь, пока дрон окажется ближе 0.6 м от старта (печатайте «Далеко от старта», «Возвращаемся», «Дома»), затем посадите. Отдельно вызовите `takeoff` до `arm` и `land` на земле и напечатайте результат каждого.

Что должно получиться: возврат и посадка у старта; на земле `takeoff` и `land` отвечают `success = False`, а программа не падает.

В симуляторе: миссии Мини 2 SDK2: 42, 43 в панели «**Обучение**».

19.8 Топики и телеметрия

Цель раздела: читать данные борта подпиской, а не опросом, и знать, какие топики есть и в каких единицах говорят.

Узел `ros_pioneer_node` публикует данные борта в топики с префиксом `/geoscan/`. Подписчик получает каждое сообщение, как только оно появилось, и не обязан спрашивать. Основные публикации:

Топик	Тип	Что в нём
`sensors/battery_state`	`sensor_msgs/BatteryState`	напряжение (`voltage`, В) и температура АКБ
`sensors/altitude`	`std_msgs/Float32`	высота по барометру, м
`sensors/range`	`sensor_msgs/Range`	дальномер вниз: `range`, м
`sensors/imu`	`sensor_msgs/Imu`	ускорения, угловые скорости, ориентация (кватернион)
`sensors/motors_rpm`	`gs_interfaces/MotorsRpm`	обороты четырёх моторов (у Мини 2)

Топик	Тип	Что в нём
`navigation/local/position`	`geometry_msgs/Point`	локальная позиция, м (карта `map`)
`navigation/local/velocity`	`geometry_msgs/Vector3`	скорость, м/с
`navigation/local/yaw`	`std_msgs/Float32`	курс
`navigation/local/status`	`gs_interfaces/NavStatus`	доверие к позиции: 0 нет данных, 1 не может, 2 низкое, 3 в порядке
`navigation/opt/velocity`	`gs_interfaces/OptVelocity`	скорость по оптическому потоку, `x`, `y`
`flight/fly_state`	`std_msgs/Int8`	состояние: 0 на земле, 1 моторы запущены, 2 в воздухе
`flight/callback_event`	`std_msgs/Int8`	событие автопилота (таблица ниже)
`payload/ranger`	`gs_interfaces/RangerData`	дальномеры Ranger по сторонам, м (у Мини 2)
`rc/trigger`	`std_msgs/Int32`	значение канала пульта

Топики GPS (`navigation/global/*`, `navigation/satellites`) в поставляемых конфигах выключены (подробнее — раздел 19.13). В симуляторе частоты такие: IMU 30 Гц, локальные навигационные топики 20 Гц, высота, дальномер, обороты, `fly_state` и оптический поток 10 Гц, батарея 2 Гц, `callback_event` — по событию. На настоящем дроне частоты смотрите командой `ros2 topic hz` в обычной среде ROS 2 (в консоли симулятора её нет).

python-onboard/ros2/ros2_02_telemetry.py

```python
# Телеметрия: подписка на топики gs_core и вывод раз в секунду.
import rclpy
from rclpy.node import Node
from rclpy.duration import Duration
from std_msgs.msg import Float32
from geometry_msgs.msg import Point
from sensor_msgs.msg import BatteryState


class Telemetry(Node):
    def __init__(self):
        super().__init__('telemetry')
        self.altitude = 0.0
        self.position = Point()
        self.voltage = 0.0
        self.create_subscription(Float32, '/geoscan/sensors/altitude',
        ↪  self.on_altitude, 10)
        self.create_subscription(Point,
        ↪  '/geoscan/navigation/local/position', self.on_position, 10)
        self.create_subscription(BatteryState,
        ↪  '/geoscan/sensors/battery_state', self.on_battery, 10)
```

```
        self.create_timer(1.0, self.report)

    def on_altitude(self, msg):
        self.altitude = msg.data

    def on_position(self, msg):
        self.position = msg

    def on_battery(self, msg):
        self.voltage = msg.voltage

    def report(self):
        p = self.position
        self.get_logger().info(
            f'высота {self.altitude:.2f} м, позиция ({p.x:.2f}, {p.y:.2f},
            ↪  {p.z:.2f}), АКБ {self.voltage:.1f} В')


def main():
    rclpy.init()
    node = Telemetry()
    finish = node.get_clock().now() + Duration(seconds=8)
    while rclpy.ok() and node.get_clock().now() < finish:
        rclpy.spin_once(node, timeout_sec=0.1)
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Узел подписывается на три топика: `create_subscription(тип, имя, обработчик, 10)`. Обработчики `on_altitude`, `on_position`, `on_battery` только запоминают последнее значение в поле объекта — и больше ничего. Печатает отчёт отдельный таймер `create_timer(1.0, self.report)` раз в секунду, поэтому частота печати не зависит от частоты датчиков. `main` крутит узел восемь секунд по часам узла (`get_clock().now()` и `Duration`); значения времени — по часам симуляции. Если подписчик создан, а топик не публикуется (выключен параметром или нет модуля), программа не падает, просто остаётся без данных: поле сохранит значение по умолчанию.

> **Хранить последнее, печатать по таймеру**
>
> Обработчик вызывается на каждое сообщение, а их приходит 10–30 в секунду. Делайте в нём одно: запомнить значение. Расчёты и вывод вынесите в таймер или в основной цикл — иначе очередь обработки растёт, а программа отстаёт от дрона.

Заголовок сообщения. У `MotorsRpm`, `PointGPS`, `SatellitesGPS`, `OptVelocity` и `RangerData` поле `header` заполнено: время `stamp` и `frame_id = 'base_link'`. У `Imu`, `Range` и `BatteryState` заголовок `gs_core` не заполняет, поэтому время сообщения берите из часов узла.

ЗАДАНИЕ 19.5 · Телеметрия в полёте

Взлетите на метр, затем напечатайте из подписок высоту, позицию x, y, z и напряжение батареи, а после посадки — высоту. Сравните высоту из `sensors/altitude` с координатой z из `navigation/local/position`: насколько они расходятся?

Что должно получиться: в воздухе высота около 1 м и $z \approx 1$; барометр и локальная навигация различаются на сантиметры или дециметры; на земле высота около нуля.

В симуляторе: миссии Мини 2 SDK2: 44, 45 в панели «**Обучение**».

19.9 События автопилота

Топик `flight/callback_event` (`std_msgs/Int8`) — то же, что `subscribe` в `pioneer_sdk2`: автопилот сообщает о важных моментах числом. Служба роя на борту публикует в него число `event.value` из `pioneer_sdk2`, поэтому коды те же, что у `Event` (значения сняты с настоящего борта):

Код	Событие	Когда приходит
0	`ALL`	не событие: «все события» при подписке
1	`NULL`	пустое событие
3	`COPTER_LANDED`	дрон сел
4, 5	`LOW_VOLTAGE1, 2`	батарея садится; почти пуста
6	`POINT_REACHED`	точка достигнута
7	`POINT_DECELERATION`	дрон тормозит перед точкой
8	`TAKEOFF_COMPLETE`	взлёт завершён
9	`ENGINES_STARTED`	моторы запущены
10	`SHOCK`	удар
11	`LOW_CHARGE`	заряд слишком мал для запуска
12	`DISARM`	моторы выключены

Эти числа справочник Геоскана не приводит: они сняты с борта через `pioneer_sdk2` 0.16. Проверяйте их на своём дроне, например подпиской с печатью и `ros2 topic echo`. Событие приходит один раз, поэтому его ловят в обработчике и запоминают: `self.event = msg.data`, а решение принимает таймер. Так строится автомат главного проекта этой главы.

Подписка на события

```
EVENTS = {3: 'COPTER_LANDED', 6: 'POINT_REACHED',
          8: 'TAKEOFF_COMPLETE', 9: 'ENGINES_STARTED'}

def on_event(self, msg):
    name = EVENTS.get(msg.data, str(msg.data))
    self.get_logger().info('Событие: ' + name)
```

```
# в __init__:
self.create_subscription(
    Int8, '/geoscan/flight/callback_event', self.on_event, 10)
```

ЗАДАНИЕ 19.6 · Журнал событий

Подпишитесь на `flight/callback_event`, пройдите по двум точкам и печатайте имя каждого события. Покажите порядок: моторы запущены, взлёт завершён, точка достигнута, точка достигнута, дрон сел.

Что должно получиться: в журнале по порядку `ENGINES_STARTED`, `TAKEOFF_COMPLETE`, затем `POINT_REACHED` (по разу на точку) и `COPTER_LANDED`; могут встретиться и события торможения.

В симуляторе: миссии Мини 2 SDK2: 46 в панели «**Обучение**».

19.10 Датчики: IMU, дальномер, Ranger

python-onboard/ros2/ros2_08_sensor_logger.py

```
# Журнал датчиков: IMU (наклон из кватерниона), дальномер вниз и Ranger
↪  (только у Пионера Мини 2).
import math
import rclpy
from rclpy.node import Node
from rclpy.duration import Duration
from sensor_msgs.msg import Imu, Range
from gs_interfaces.msg import RangerData


def tilt_degrees(q):
    # Крен и тангаж из кватерниона ориентации.
    roll = math.atan2(2 * (q.w * q.x + q.y * q.z), 1 - 2 * (q.x * q.x + q.y *
    ↪  q.y))
    pitch = math.asin(max(-1.0, min(1.0, 2 * (q.w * q.y - q.z * q.x))))
    return math.degrees(roll), math.degrees(pitch)


class SensorLogger(Node):
    def __init__(self):
        super().__init__('sensor_logger')
        self.roll = self.pitch = 0.0
        self.below = 0.0
        self.sides = None
        self.create_subscription(Imu, '/geoscan/sensors/imu', self.on_imu,
        ↪  10)
        self.create_subscription(Range, '/geoscan/sensors/range',
        ↪  self.on_range, 10)
        # Топика payload/ranger может не быть (Пионер Базовый): подписка
        ↪  просто останется без данных.
        self.create_subscription(RangerData, '/geoscan/payload/ranger',
        ↪  self.on_ranger, 10)
```

```
        self.create_timer(1.0, self.report)

    def on_imu(self, msg):
        self.roll, self.pitch = tilt_degrees(msg.orientation)

    def on_range(self, msg):
        self.below = msg.range

    def on_ranger(self, msg):
        self.sides = (msg.front, msg.back, msg.left, msg.right)

    def report(self):
        text = f'крен {self.roll:.1f}°, тангаж {self.pitch:.1f}°, до земли
        ↪  {self.below} м'
        if self.sides is not None:
            text += ', Ranger (перед, зад, лево, право): ' + ',
            ↪  '.join(f'{v:.2f}' for v in self.sides)
        self.get_logger().info(text)


def main():
    rclpy.init()
    node = SensorLogger()
    finish = node.get_clock().now() + Duration(seconds=6)
    while rclpy.ok() and node.get_clock().now() < finish:
        rclpy.spin_once(node, timeout_sec=0.1)
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Ориентация дрона в `Imu` задана *кватернионом* (четырьмя числами w, x, y, z), а не углами; функция `tilt_degrees` переводит его в крен и тангаж формулами

$$\varphi = \operatorname{atan2}\big(2(wx + yz),\ 1 - 2(x^2 + y^2)\big), \qquad \theta = \arcsin\big(2(wy - zx)\big).$$

Значение под арксинус зажато в $[-1, 1]$, чтобы ошибка округления не вызвала `ValueError`. Дальномер вниз `sensors/range` даёт `range` в метрах; вне диапазона датчика (параметры `range_sensor.min_range`, `max_range`) в симуляторе значение — `-inf` или `+inf`, как в стандарте ROS (REP 117), поэтому перед расчётами проверяйте его через `math.isfinite`. Ranger (`payload/ranger`) измеряет расстояние до препятствий вокруг дрона: `front`, `back`, `left`, `right`, `above_below`; он есть у Мини 2, а у Базового топика нет, и программа это переживает: подписка просто остаётся без данных, отчёт обходится без строки Ranger.

ЗАДАНИЕ 19.7 · Осторожный датчик

Допишите в `report()` предупреждение, если дальномер вниз показывает меньше 0.15 м или значение не конечное, и печать «крен/тангаж больше $15°$» при заметном наклоне. Запустите программу на земле и в полёте.

Что должно получиться: на земле дальномер показывает малое значение или бесконечность, и предупреждение печатается; в спокойном полёте крен и тангаж малы; ни одна строка не падает при `inf` в дальномере.

19.11 Управление скоростью: cmd_vel

Цель раздела: управлять дроном скоростью, публикуя сообщения в топик, и держать для этого ровный ритм таймером.

Скорость задаётся не сервисом, а *публикацией* в топик `/geoscan/flight/cmd_vel`. Тип сообщения — `geometry_msgs/TwistStamped`: заголовок и *twist* (линейная и угловая скорость).

Поле	Смысл
`header.frame_id`	система осей: `'base_link'` — оси коптера, `'map'` — мировые; пустое или другое значение сообщение отключает (предупреждение в журнале)
`twist.linear.x, y, z`	линейная скорость, м/с
`twist.angular.z`	скорость поворота, рад/с; предел ± 1.82
`twist.angular.x, y`	игнорируются

Топик работает, только если в конфиге включён `topics.cmd_vel` (в поставляемых конфигах включён). Пример квадрата скоростью:

python-onboard/ros2/ros2_03_cmd_vel_square.py

```
# Квадрат скоростью: топик flight/cmd_vel (TwistStamped) и таймер на 10 Гц.
# frame_id 'map' - мировые оси, 'base_link' - оси коптера. Скорость нужно
↪  подтверждать постоянно.
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int8
from std_srvs.srv import Trigger
from geometry_msgs.msg import TwistStamped
from gs_interfaces.srv import Arm

SPEED = 0.4
LEG_TICKS = 25   # 25 тиков по 0.1 с: сторона 1 м
LEGS = [(SPEED, 0.0), (0.0, SPEED), (-SPEED, 0.0), (0.0, -SPEED)]


class SquareByVelocity(Node):
    def __init__(self):
        super().__init__('square_by_velocity')
        self.fly_state = 0
        self.tick = 0
        self.finished = False
        self.create_subscription(Int8, '/geoscan/flight/fly_state',
        ↪  self.on_fly_state, 10)
```

```
        self.pub = self.create_publisher(TwistStamped,
        ↪  '/geoscan/flight/cmd_vel', 10)
        self.timer = self.create_timer(0.1, self.on_timer)
        self.timer.cancel()
        self.arm_client = self.create_client(Arm, '/geoscan/flight/arm')
        self.takeoff_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/takeoff')
        self.land_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/land')

    def on_fly_state(self, msg):
        self.fly_state = msg.data

    def on_timer(self):
        leg = self.tick // LEG_TICKS
        msg = TwistStamped()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'map'
        if leg < len(LEGS):
            msg.twist.linear.x, msg.twist.linear.y = LEGS[leg]
        else:
            self.finished = True
        self.pub.publish(msg)
        self.tick += 1

    def call_service(self, client, request):
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()


def main():
    rclpy.init()
    node = SquareByVelocity()
    node.call_service(node.arm_client, Arm.Request(timeout=5.0, retries=0))
    node.call_service(node.takeoff_client, Trigger.Request())
    while rclpy.ok() and node.fly_state != 2:
        rclpy.spin_once(node, timeout_sec=0.1)

    node.timer.reset()                          # пошёл квадрат
    while rclpy.ok() and not node.finished:
        rclpy.spin_once(node, timeout_sec=0.1)
    node.timer.cancel()

    node.call_service(node.land_client, Trigger.Request())
    while rclpy.ok() and node.fly_state != 0:     # ждём посадки
        rclpy.spin_once(node, timeout_sec=0.1)
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Список `LEGS` — четыре стороны квадрата как пары скоростей (v_x, v_y) в осях `map`: на каждую уходит `LEG_TICKS` $= 25$ тиков таймера по 0.1 с, то есть $25 \times 0.1 \times 0.4 = 1$ м. Таймер публикует скорость текущей стороны и заканчивает работу флагом

`finished`. Сначала таймер отменён (`cancel()`): пока дрон не в воздухе, публиковать нечего; после взлёта его запускают `reset()`. Главный цикл только ждёт флаг. Конец — команда `land` и ожидание `fly_state = 0`.

Скорость нужно подтверждать постоянно

Команда `cmd_vel` не «включает» скорость навсегда. В симуляторе каждое сообщение держит скорость 0.5 с, затем дрон зависает; на земле команда игнорируется с предупреждением. Поэтому публикуйте скорость таймером раз в 0.1 с — и дрон сам остановится, если программа зависнет. (У `pioneer_sdk2` то же делает аргумент `interval`.) На настоящем дроне срок удержания сверьте опытом.

Перед `land` после `cmd_vel` выдержите короткую паузу (`spin_once` несколько раз): запоздавшее сообщение скорости не должно прийти поверх посадки. Квадрат из скоростей замыкается хуже квадрата из точек: ошибка накапливается так же, как у относительных перемещений в главе 16.

Скорость из консоли

```
ros2 topic pub -r 10 /geoscan/flight/cmd_vel \
  geometry_msgs/msg/TwistStamped \
  "{header: {frame_id: 'base_link'},
    twist: {linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {z: 0.0}}}"
```

Ограничение курса и линейной скорости

Предел ± 1.82 рад/с относится к `angular.z`: в симуляторе он понят как *ограничение* (значение обрезается), тогда как в `pioneer_sdk2` команду с большей скоростью поворота отбрасывают целиком. Линейные скорости эмуляция не ограничивает: предел задаёт автопилот.

ЗАДАНИЕ 19.8 · Квадрат скоростью в base_link

Переделайте пример: `frame_id = 'base_link'`, сторона — полёт вперёд ($v_x = 0.4$), затем поворот на месте ω_z около 0.8 рад/с в течение $\pi/2/0.8 \approx 2$ с. Сделайте замкнутый квадрат и сравните его с квадратом в `map`.

Что должно получиться: дрон проходит четыре стороны и поворачивается налево на каждой вершине (положительная скорость поворота — влево); возвращение к старту хуже, чем у квадрата в `map`, за счёт накопления ошибки поворота.

В симуляторе: миссии Мини 2 SDK2: 47, 48 в панели «**Обучение**».

19.12 Виртуальный пульт: rc/set_channels

Топик `/geoscan/rc/set_channels` (`gs_interfaces/RcChannels`) подаёт на автопилот значения каналов пульта — как будто они пришли с настоящего пульта: поля `channel_1` …`channel_8`. Тот же приём есть и в `pioneer_sdk2` (виртуальный пульт). Справочник Гео-

скана требует публиковать каналы *постоянно*, чтобы удерживать связь. Топик создаётся при `topics.rc: true`.

python-onboard/ros2/ros2_09_rc_channels.py

```python
# RC-каналы: топик rc/set_channels (RcChannels) нужно публиковать постоянно,
↪  пока нужна связь.
# rc/trigger сообщает значение канала RC, когда оно меняется.
import rclpy
from rclpy.node import Node
from rclpy.duration import Duration
from std_msgs.msg import Int32
from gs_interfaces.msg import RcChannels


class RcDemo(Node):
    def __init__(self):
        super().__init__('rc_demo')
        self.pub = self.create_publisher(RcChannels,
        ↪  '/geoscan/rc/set_channels', 10)
        self.create_subscription(Int32, '/geoscan/rc/trigger',
        ↪  self.on_trigger, 10)
        self.create_timer(0.1, self.send)

    def on_trigger(self, msg):
        self.get_logger().info(f'rc/trigger: {msg.data}')

    def send(self):
        msg = RcChannels()
        # Ручки по центру, восьмой канал — тумблер: 2000 — включён.
        msg.channel_1 = msg.channel_2 = msg.channel_3 = msg.channel_4 = 1500
        msg.channel_5 = msg.channel_6 = msg.channel_7 = 1500
        msg.channel_8 = 2000
        self.pub.publish(msg)


def main():
    rclpy.init()
    node = RcDemo()
    finish = node.get_clock().now() + Duration(seconds=3)
    while rclpy.ok() and node.get_clock().now() < finish:
        rclpy.spin_once(node, timeout_sec=0.1)
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Таймер `create_timer(0.1, self.send)` раз в 0.1 с публикует все восемь каналов: ручки по центру (1500), восьмой канал — тумблер (2000 — включён). В симуляторе значения — как у `pioneer_send_rc_channels`: от 1000 до 2000, центр 1500. Подписка на `rc/trigger` печатает значение канала, когда оно меняется (в эмуляции — 8-й канал). Через несколько секунд программа завершается: публикация прекращается.

RC в воздухе

Каналы, посланные из программы, приходят на тот же путь ручного управления, что и пульт. Не подавайте их, пока не знаете, что делает каждый канал вашего набора параметров (`Copter_man_rcMode0..2`, раздел 19.13), и не рассчитывайте на них как на замену аварийной остановки с пульта.

ЗАДАНИЕ 19.9 · Тумблер и отклик

Опубликуйте восьмой канал в положении 1000 три секунды, затем 2000 ещё три и после этого прекратите публикацию. Печатайте значения `rc/trigger` и убедитесь, что они меняются вслед за каналом.

Что должно получиться: в журнале после смены канала приходит новое значение (1000, затем 2000); когда публикация прекращена, новых сообщений нет.

В симуляторе: миссии Мини 2 SDK2: 49 в панели «**Обучение**».

19.13 Параметры: автопилота и узла

Цель раздела: различать два вида параметров — настройки автопилота и настройки самого узла — и менять каждый правильным способом.

В ROS 2 слово «параметр» встречается дважды, и путать эти вещи нельзя:

	Параметры автопилота	Параметры узла
Где живут	в полётном контроллере; сохраняются при перезагрузке	в `ros_pioneer_node`; задаются YAML-файлом борта при запуске
Пример	`Flight_com_takeoffAlt` (высота взлёта)	`topics.gnss`, `grab`, `range_sensor.max_range`
Как менять	сервисы `board/get_param`, `board/set_param`, `board/get_parameters`, `board/set_parameters`	команда `ros2 param`, ключ `--ros-args -p` при запуске, YAML

Параметры автопилота

Сервис	Тип	Что делает
`board/get_param`	`gs_interfaces/GetParam`	прочитать параметр: `name`, `update` (перечитать словарь с борта)
`board/set_param`	`gs_interfaces/SetParam`	записать: `name`, `value`
`board/get_parameters`	`gs_interfaces/ ParametersList`	все параметры; при ошибке `status: false` и пустой список
`board/set_parameters`	`gs_interfaces/ SetParametersList`	записать несколько: `params: [{name, value}]`
`board/restart`	`std_srvs/Empty`	перезагрузить автопилот
`alive`	`gs_interfaces/Live`	жив ли обмен с автопилотом

python-onboard/ros2/ros2_06_params.py

```python
# Параметры автопилота: сервисы board/get_param, board/set_param и
↪  board/get_parameters.
import rclpy
from rclpy.node import Node
from gs_interfaces.srv import GetParam, SetParam, ParametersList


class Params(Node):
    def __init__(self):
        super().__init__('params')
        self.get_client = self.create_client(GetParam,
        ↪  '/geoscan/board/get_param')
        self.set_client = self.create_client(SetParam,
        ↪  '/geoscan/board/set_param')
        self.list_client = self.create_client(ParametersList,
        ↪  '/geoscan/board/get_parameters')

    def call_service(self, client, request):
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

    def read(self, name):
        response = self.call_service(self.get_client,
        ↪  GetParam.Request(name=name, update=False))
        return response.value if response.status else None


def main():
    rclpy.init()
    node = Params()

    print('Высота взлёта:', node.read('Flight_com_takeoffAlt'))
    result = node.call_service(node.set_client,
    ↪  SetParam.Request(name='Flight_com_takeoffAlt', value=0.8))
    print('Записали 0.8:', result.status)
    print('Теперь:', node.read('Flight_com_takeoffAlt'))
```

```
    print('Несуществующий параметр:', node.read('No_such_param'))

    everything = node.call_service(node.list_client,
    ↪  ParametersList.Request())
    if everything.status:
        print('Всего параметров:', len(everything.params))
        for p in everything.params[:5]:
            print(f'  {p.name} = {p.value}')

    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Метод `read` вызывает `get_param`; если ответ `status` ложный (параметра нет), возвращает `None`. Программа читает высоту взлёта, записывает 0.8 м, читает снова и проверяет несуществующее имя. Затем `get_parameters` печатает весь список. Значения параметров — числа (`float32`); имена вида `Flight_com_takeoffAlt` — как в автопилоте (у настоящего Мини 2 это 0.5 м; в симуляторе 0.4 м).

> **Параметр автопилота сохраняется**
>
> Записанное значение остаётся в автопилоте и после перезагрузки. Прочитайте и запомните старое значение, а в конце программы верните его (`finally`). Не меняйте параметры, назначения которых не знаете: неверное значение может сломать полёт. Автопилот Мини 2 хранит 232 параметра (имена по группам — в приложении «Протокол Plazlink»); неизвестное имя даёт `status: false`. В симуляторе значения параметров те же, что у борта, кроме `Flight_com_navSystem` (1 вместо 2) и `Flight_com_takeoffAlt` (0.4 вместо 0.5).

ЗАДАНИЕ 19.10 · Высота взлёта по сервису

Прочитайте `Flight_com_takeoffAlt`, запишите вдвое большее значение, взлетите и убедитесь, что дрон поднимается выше; в `finally` верните исходное значение. Отдельно запросите параметр с несуществующим именем и напечатайте результат.

Что должно получиться: после записи высота взлёта изменилась и затем вернулась; несуществующий параметр даёт `status: False` и `None`, а не ошибку программы.

В симуляторе: миссии Мини 2 SDK2: 50 в панели «**Обучение**».

Параметры узла и конфиг борта

Параметры узла определяют, какие топики и модули создаёт `ros_pioneer_node`. Главные у Мини 2 и Razero:

Параметр	Мини 2	Razero	Что включает
`grab`	да	нет	механический захват
`cargo`	нет	да	магнитный захват груза
`nav_system_change_in_flight`	нет	да	смену навигации в полёте
`board_reboot`	нет	да	перезагрузку автопилота при старте узла
`topics.imu`, `altitude`, `range`	да	да	соответствующие топики
`topics.motors_rpm`, `ranger`	да	нет	обороты моторов, дальномеры Ranger
`topics.gnss`	нет	нет	топики GPS (`navigation/global/*`)
`topics.lps`	да	да	локальная навигация (`navigation/local/*`)
`range_sensor.max_range`	4.0	4.0	дальность дальномера, м

Если `topics.X = false`, топика нет вовсе. Если `true`, а модуля на борту нет, топик создан, но данные не публикуются. Включить GPS-топики можно в YAML (`ros_pioneer_node: ros__parameters: topics: gnss: true`) или ключом при запуске:

Включить топики GPS при запуске узла

```
ros2 run gs_core ros_pioneer_node --ros-args \
  --params-file mini2.yaml -p topics.gnss:=true
ros2 param list
ros2 param get /ros_pioneer_node topics.gnss
```

Параметры `topics.*`, `shortname`, `board_*`, `gtlp_version` узел читает только при запуске: менять их `ros2 param set` на ходу нельзя, нужен перезапуск (консоль симулятора подскажет). `grab`, `cargo`, `nav_system_change_in_flight` и `range_sensor.*` действуют сразу. Из программы симулятора узел перезапускают вызовом `gs_core.launch.restart_pioneer_node(parameters={'topics.gnss': True})`: этой функции в настоящем пакете нет, это приём симулятора.

ЗАДАНИЕ 19.11 · GNSS: включить и проверить

Перезапустите узел с `topics.gnss: True` (консолью или функцией симулятора) и сравните список топиков до и после. Подпишитесь на `navigation/global/status` и объясните, почему данных нет, хотя топик есть.

Что должно получиться: после перезапуска в списке появляются `navigation/global/*` и `satellites`; сообщений нет, потому что в симуляторе GPS не моделируется (навигация всегда локальная).

В симуляторе: миссии Мини 2 SDK2: 54 в панели «**Обучение**».

19.14 Система навигации

Автопилот ведёт позицию одной из трёх систем: `GPS`, `LPS` (локальная) или `OPT` (оптический поток). Узнать и сменить систему можно только сервисами навигации, а не через `board/set_parameters`:

Сервис	Тип	Результат
`navigation/get_system`	`gs_interfaces/NavigationSystem`	поле `navigation` (имя системы) и `status`
`navigation/set_system`	`gs_interfaces/SetNavigationSystem`	запрос `navigation: 'OPT'`; ответ `status`

python-onboard/ros2/ros2_07_navigation_system.py

```python
# Система навигации: navigation/get_system и navigation/set_system.
# Менять её нужно именно этим сервисом, а не через board/set_parameters.
# В полёте смена отклоняется, если nav_system_change_in_flight: false (так у
↪  Пионера Мини 2).
import rclpy
from rclpy.node import Node
from gs_interfaces.srv import NavigationSystem, SetNavigationSystem


class Navigation(Node):
    def __init__(self):
        super().__init__('navigation_system')
        self.get_client = self.create_client(NavigationSystem,
        ↪  '/geoscan/navigation/get_system')
        self.set_client = self.create_client(SetNavigationSystem,
        ↪  '/geoscan/navigation/set_system')

    def call_service(self, client, request):
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

    def current(self):
        response = self.call_service(self.get_client,
        ↪  NavigationSystem.Request())
        return response.navigation if response.status else None


def main():
    rclpy.init()
    node = Navigation()
    print('Сейчас:', node.current())
    for name in ['OPT', 'LPS']:
        ok = node.call_service(node.set_client,
        ↪  SetNavigationSystem.Request(navigation=name)).status
        print(f'Переключаем на {name}:', 'готово' if ok else 'отказ', '->',
        ↪  node.current())
    node.destroy_node()
    rclpy.shutdown()
```

```
main()
```

Что происходит. Метод `current` запрашивает текущую систему и возвращает её имя или `None` при ошибке. Цикл переключает систему на `OPT`, затем обратно на `LPS` и печатает результат и новое состояние. На земле это допустимо. В полёте смена *отклоняется*, если `nav_system_change_in_flight: false` — а так настроен Мини 2.

> **GPS и симулятор**
>
> Сервис `flight/go_to_global_point` летит в точку по GPS (`frame_id`: `map` — абсолютные координаты, `base_link` — смещение). В симуляторе физически работает только LPS, поэтому этот сервис всегда отвечает `success: False`, а смена системы лишь меняет записанное значение. Доверие к позиции показывает `navigation/local/status`: 3 — в порядке.

ЗАДАНИЕ 19.12 · Навигация в воздухе

Прочитайте систему на земле и переключите её на `OPT`, затем взлетите и попробуйте переключить на `LPS` в воздухе. Напечатайте «готово» или «отказ» для каждого шага и верните `LPS` после посадки.

Что должно получиться: на земле смена проходит (готово); в воздухе — отказ, потому что у Мини 2 `nav_system_change_in_flight` выключен; в конце снова `LPS`.

В симуляторе: миссии Мини 2 SDK2: 51 в панели «**Обучение**».

19.15 Полезная нагрузка: светодиоды и захват

Нагрузку включают собственные параметры узла; выключенный модуль не падает с ошибкой: сервис создаётся, но вызов вернёт отказ с записью в журнал (`success: False`). Топик `payload/ranger` при выключенном параметре не создаётся вовсе.

Модуль	Сервис, тип	Запрос
Светодиоды (всегда)	`payload/led/set`, `gs_interfaces/Led`	`mode` (`PIXEL`=0, `FILL`=1), `leds` (список цветов), `period`
Механический захват (Мини 2)	`payload/grab`, `gs_interfaces/Grab`	`action` (0 открыть, 1 закрыть, 2 остановить), `velocity` (0–100), `movement_time`
Магнитный захват (Базовый)	`payload/cargo_set`, `std_srvs/SetBool`	`data`: `true` взять, `false` отпустить
Ranger (Мини 2)	топик `payload/ranger`	расстояния `front`, `back`, `left`, `right`, `above_below`

Светодиоды

python-onboard/ros2/ros2_04_led.py

```python
# Светодиоды: сервис payload/led/set. Цвет компонент — 0.0..1.0.
# PIXEL — цвет каждого диода отдельно, FILL — один цвет на все.
import rclpy
from rclpy.node import Node
from std_msgs.msg import ColorRGBA
from gs_interfaces.srv import Led


def color(r, g, b):
    return ColorRGBA(r=r, g=g, b=b, a=1.0)


class LedDemo(Node):
    def __init__(self):
        super().__init__('led_demo')
        self.client = self.create_client(Led, '/geoscan/payload/led/set')
        self.step = 0
        self.timer = self.create_timer(1.0, self.on_timer)

    def on_timer(self):
        if self.step == 0:
            request = Led.Request(mode=Led.Request.PIXEL, leds=[color(1.0,
            ↪  0.0, 0.0), color(0.0, 1.0, 0.0)])
        elif self.step == 1:
            request = Led.Request(mode=Led.Request.PIXEL, leds=[color(0.0,
            ↪  1.0, 0.0), color(1.0, 0.0, 0.0)])
        elif self.step == 2:
            request = Led.Request(mode=Led.Request.FILL, leds=[color(0.0,
            ↪  0.0, 1.0)])
        else:
            request = Led.Request(mode=Led.Request.FILL, leds=[color(0.0,
            ↪  0.0, 0.0)])   # погасить
            self.timer.cancel()
        self.client.call_async(request)
        self.get_logger().info(f'шаг {self.step}')
        self.step += 1


def main():
    rclpy.init()
    node = LedDemo()
    while rclpy.ok() and not node.timer.is_canceled():
        rclpy.spin_once(node, timeout_sec=0.2)
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Цвет — `ColorRGBA` с компонентами от 0.0 до 1.0 (как у `pioneer_sdk2`; не 0–255). Режим `PIXEL` задаёт каждому диоду свой цвет по индексу в списке `leds`, `FILL` — один цвет всей ленте из `leds[0]`. Режимы `BLINK` и `CHASE` справоч-

ник не реализует. Таймер раз в секунду отправляет следующий шаг через `call_async` (ответ не нужен), а в конце гасит светодиоды заливкой чёрным. Параметр `period` для `PIXEL` и `FILL` не используется.

Захват груза

python-onboard/ros2/ros2_05_grab_cargo.py

```python
# Захват груза. У Пионера Мини 2 механический захват (payload/grab), у
↪  Пионера Базового — магнитный (payload/cargo_set).
# Выключенный модуль не падает с ошибкой: сервис отвечает success: false,
↪  причина — в журнале.
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool
from gs_interfaces.srv import Grab

CLOSE, OPEN = 1, 0


class Gripper(Node):
    def __init__(self):
        super().__init__('gripper')
        self.grab_client = self.create_client(Grab, '/geoscan/payload/grab')
        self.cargo_client = self.create_client(SetBool,
        ↪  '/geoscan/payload/cargo_set')

    def call_service(self, client, request):
        future = client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

    def hold(self, close):
        # movement_time больше 0: вызов ждёт столько секунд, и захват
        ↪  останавливается сам.
        request = Grab.Request(movement_time=1.0, velocity=100, action=CLOSE
        ↪  if close else OPEN)
        if self.call_service(self.grab_client, request).success:
            return 'механический захват'
        if self.call_service(self.cargo_client,
        ↪  SetBool.Request(data=close)).success:
            return 'магнитный захват'
        return None


def main():
    rclpy.init()
    node = Gripper()
    print('Захватываем:', node.hold(True))
    print('Отпускаем:', node.hold(False))
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Метод `hold` сначала пробует механический захват: `Grab.Request(movement_time=1.0, velocity=100, action=CLOSE)`. Если `movement_time` больше нуля, вызов блокируется на это время, и захват останавливается сам; при нуле движение идёт до явной остановки (`action = 2`). Скорость вне 0–100 — отказ. Если механического захвата нет (`grab: false`, как у Razero), метод пробует магнитный `cargo_set`; у Мини 2 `cargo = false`, поэтому магнитный там отвечает отказом. Программа не знает модель дрона заранее: она узнаёт её по `success` — это приём «спросить у сервиса», а не «угадать».

ЗАДАНИЕ 19.13 · Светофор из сервисов

Сделайте «светофор»: диод 0 красный, диод 1 зелёный, затем наоборот, затем оба жёлтые (`r=1, g=1`), затем выключены. Отдельно закройте и откройте захват; попробуйте `velocity=150` и магнитный захват и напечатайте результаты.

Что должно получиться: цвета меняются по шагам; закрытие и открытие захвата возвращают `True`; скорость 150 и магнитный захват у Мини 2 — `False`.

В симуляторе: миссии Мини 2 SDK2: 52, 53 в панели «**Обучение**».

19.16 Итоговый проект: миссия-узел

Цель раздела: собрать из приёмов главы законченную миссию, в которой ни один обработчик не ждёт, а ход программы определяет автомат состояний.

Миссия «arm → взлёт → две точки → посадка → выключение» устроена как *узел-автомат*: состояние хранится в поле узла, таймер раз в 0.2 с смотрит на него и решает, что сделать дальше. Сервисы вызываются через `call_async` без ожидания, а об окончании действий сообщают события автопилота.

python-onboard/ros2/ros2_10_mission_state_machine.py

```python
# Миссия узлом-автоматом: arm -> takeoff -> две точки -> land -> disarm.
# Никаких ожиданий внутри обработчиков: таймер раз в 0.2 с смотрит на
↪  состояние, сервисы вызываются через call_async,
# а события автопилота приходят из топика flight/callback_event.
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int8
from std_srvs.srv import Trigger
from gs_interfaces.srv import Arm, GoToLocalPoint

EVENT_COPTER_LANDED = 3
EVENT_POINT_REACHED = 6
EVENT_TAKEOFF_COMPLETE = 8

POINTS = [(1.0, 0.0, 1.0), (1.0, 1.0, 1.0)]


class Mission(Node):
    def __init__(self):
        super().__init__('mission')
```

```
        self.state = 'ARM'
        self.future = None
        self.event = None
        self.point = 0
        self.create_subscription(Int8, '/geoscan/flight/callback_event',
        ↪  self.on_event, 10)
        self.arm_client = self.create_client(Arm, '/geoscan/flight/arm')
        self.takeoff_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/takeoff')
        self.go_client = self.create_client(GoToLocalPoint,
        ↪  '/geoscan/flight/go_to_local_point')
        self.land_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/land')
        self.disarm_client = self.create_client(Trigger,
        ↪  '/geoscan/flight/disarm')
        self.create_timer(0.2, self.step)

    def on_event(self, msg):
        self.event = msg.data

    def request(self, state, client, request):
        self.event = None
        self.future = client.call_async(request)
        self.state = state
        self.get_logger().info('состояние: ' + state)

    def fly_next(self):
        x, y, z = POINTS[self.point]
        self.point += 1
        go = GoToLocalPoint.Request(x=x, y=y, z=z, yaw=0.0, time=0.0,
        ↪  frame_id='map')
        self.request('FLY', self.go_client, go)

    def step(self):
        if self.future is not None and not self.future.done():
            return                                          # ждём ответ
            ↪  сервиса
        if self.future is not None:
            ok = self.future.result().success
            self.future = None
            if not ok:
                self.get_logger().error('сервис отказал в состоянии ' +
                ↪  self.state)
                self.request('DONE', self.disarm_client, Trigger.Request())
                return
        if self.state == 'ARM':
            self.request('TAKEOFF', self.arm_client,
            ↪  Arm.Request(timeout=5.0, retries=0))
        elif self.state == 'TAKEOFF':
            self.request('WAIT_TAKEOFF', self.takeoff_client,
            ↪  Trigger.Request())
        elif self.state == 'WAIT_TAKEOFF' and self.event ==
        ↪  EVENT_TAKEOFF_COMPLETE:
            self.fly_next()
        elif self.state == 'FLY' and self.event == EVENT_POINT_REACHED:
            if self.point < len(POINTS):
                self.fly_next()
```

```
            else:
                self.request('WAIT_LAND', self.land_client,
                ↪  Trigger.Request())
        elif self.state == 'WAIT_LAND' and self.event == EVENT_COPTER_LANDED:
            self.request('DONE', self.disarm_client, Trigger.Request())


def main():
    rclpy.init()
    node = Mission()
    while rclpy.ok() and node.state != 'DONE':
        rclpy.spin_once(node, timeout_sec=0.2)
    node.get_logger().info('миссия закончена')
    node.destroy_node()
    rclpy.shutdown()


main()
```

Что происходит. Состояние `state` проходит цепочку `ARM` → `TAKEOFF` → `WAIT_TAKEOFF` → `FLY` (дважды: две точки из `POINTS`) → `WAIT_LAND` → `DONE`. Метод `request` запоминает состояние, сбрасывает последнее событие и отправляет запрос: `self.future` хранит ответ. Таймерный метод `step` работает так:

- если ответ предыдущего вызова ещё не пришёл (`future.done()` ложно), он ничего не делает;
- если ответ пришёл с `success = False`, миссия прекращается переходом к выключению моторов (`DONE`);
- иначе по состоянию и последнему событию выбирает следующую команду: после `TAKEOFF_COMPLETE` — первая точка, после `POINT_REACHED` — следующая или посадка, после `COPTER_LANDED` — `disarm`.

Коды событий 3, 6 и 8 — из таблицы предыдущих разделов. Ожидание нигде не блокирует: пока дрон летит, узел продолжает принимать данные.

Автомат узла

Состояние — в поле узла; обратные вызовы только фиксируют факты (ответ, событие); решение — в таймере; каждая команда отправляется один раз при смене состояния. Так программа не зависает, при ошибке выбирает безопасный путь, а её ход виден в журнале (`состояние: FLY`).

Чего в этом примере нет

Пример доведён до учебной ясности: у него нет аварийной ветки на прерывание или удар и нет проверки напряжения. На настоящем дроне добавьте такую ветку обязательно: подписку на `sensors/battery_state`, реакцию на события 3, 4 (низкий заряд) и 9 (удар) и посадку перед остановкой узла. Так же, как в проекте-патруле главы 16, программа должна приводить дрон в безопасное состояние при любом

исходе.

ЗАДАНИЕ 19.14 · Патруль узлом

Мини-проект. Расширьте автомат: три точки $(1, 0, 1)$, $(1, 1, 1)$, $(0, 1, 1)$; на каждой вершине диод 0 меняет цвет (жёлтый, зелёный, синий), в конце гаснет; подписка на `sensors/battery_state` запоминает минимальное напряжение, и узел печатает его в конце. Если сервис отвечает `success = False`, миссия переходит в безопасную ветку: садится, если уже в воздухе, и выключает моторы.

Что должно получиться: дрон проходит три точки и садится; диод 0 по очереди жёлтый, зелёный, синий и в конце выключен; в журнале печатается минимальное напряжение; при отказе первого же сервиса дрон остаётся на земле, а узел завершается без ошибки.

В симуляторе: миссии Мини 2 SDK2: 55, 56 в панели «**Обучение**».

ЗАДАНИЕ 19.15 · Аварийная ветка

Добавьте подписку на `flight/callback_event`: при событии 4 (низкий заряд, `LOW_VOLTAGE1`) узел вызывает `rtl` и затем `land`, при событии 9 (удар) выключает моторы только когда `fly_state` уже равен 0. Проверьте ветки, подставляя события вручную (командой `ros2 topic pub --once` в тот же топик с типом `std_msgs/msg/Int8`, если консоль это разрешает) или заменив источник события тестовым флагом.

Что должно получиться: при событии 3 дрон возвращается к старту и садится; при отсутствии событий миссия идёт как раньше; ни одна ветка не вызывает команду, запрещённую в текущем состоянии.

19.17 Справочник gs_core

Все имена — с префиксом `/geoscan/`. Типы с префиксом `gs_interfaces` — из пакета Геоскана.

Сервисы

Сервис	Тип	Назначение
`flight/arm`	`Arm`	моторы: `timeout`, `retries`
`flight/takeoff`, `land`, `disarm`, `rtl`	`Trigger`	взлёт, посадка, остановка моторов, возврат
`flight/go_to_local_point`	`GoToLocalPoint`	точка: `x, y, z, yaw, time, frame_id`
`flight/go_to_global_point`	`GoToGlobalPoint`	точка по GPS
`flight/set_yaw`	`Yaw`	курс, `angle` в градусах

Сервис	Тип	Назначение
`alive`	`Live`	связь с автопилотом
`board/get_param`, `set_param`	`GetParam`, `SetParam`	параметр автопилота
`board/get_parameters`, `set_parameters`	`ParametersList`, `SetParametersList`	списки параметров
`board/restart`	`Empty`	перезагрузка автопилота
`navigation/get_system`, `set_system`	`NavigationSystem`, `SetNavigationSystem`	система навигации
`payload/led/set`	`Led`	светодиоды
`payload/grab`	`Grab`	механический захват
`payload/cargo_set`	`SetBool`	магнитный захват

Подписки узла (его входы)

Топик	Тип	Что делает
`flight/cmd_vel`	`TwistStamped`	скорость: `linear` м/с, `angular.z` рад/с; `frame_id` `base_link` или `map`
`rc/set_channels`	`RcChannels`	каналы 1–8; публиковать постоянно

Публикации узла перечислены в таблице раздела 19.8.

19.18 Проверьте себя

1. Чем топик отличается от сервиса?

   (a) топик — поток, сервис — запрос и ответ

   (b) топик быстрее

   (c) это одно и то же

   Ответ: (a). Топик публикуют и читают без ответа; сервис вызывают и получают результат.

2. Что вернёт сервис `flight/takeoff`, если вызвать его до `flight/arm`?

   (a) ошибку программы

   (b) `success = False`

   (c) взлёт всё равно произойдёт

   Ответ: (b) — это отказ, причина записана в журнале узла; программа не падает, поэтому `success` нужно проверять.

3. Когда в симуляторе сервис `go_to_local_point` возвращает ответ?

   (a) когда дрон прилетел

   (b) когда команда принята

   (c) через время `time`

Ответ: (b). Прилёт ловят по событию `POINT_REACHED` (код 6) в `flight/callback_event`.

4. Как лететь на метр вперёд от текущего положения в `base_link`?

   (a) $y = 1$

   (b) $x = 1$

   (c) $z = 1$

   Ответ: (b) — в `base_link` x направлена вперёд; в осях корпуса `pioneer_sdk2` вперёд — y.

5. Дрон повёрнут на $90°$. Вы вызываете `go_to_local_point` с `yaw = 90.0`. Что будет с курсом?

   (a) дрон повернётся ещё на $90°$

   (b) курс не изменится

   (c) дрон повернётся на $0°$

   Ответ: (b). Курс абсолютный; угол, совпадающий с прошлым заданным, рыскание не меняет.

6. Почему публикацию `cmd_vel` делают таймером, а не один раз?

   (a) сообщение держит скорость недолго, потом дрон зависает

   (b) иначе узел не запустится

   (c) таймер быстрее

   Ответ: (a). Скорость нужно подтверждать постоянно; если программа зависнет, дрон остановится сам.

7. В обработчике подписки выполняется долгий расчёт. Чем это плохо?

   (a) ничем

   (b) следующие сообщения ждут, узел отстаёт от дрона

   (c) обработчик вызовется дважды

   Ответ: (b). Обработчик запоминает значение, расчёты — в таймере или основном цикле.

8. Что общего у параметров автопилота и узла и в чём разница?

   (a) нет разницы

   (b) автопилота — сервисами `board/*`, узла — `ros2 param` и YAML

   (c) узла хранятся в автопилоте

   Ответ: (b). Параметры автопилота сохраняются при перезагрузке; `topics.*` узел читает при запуске.

9. Как правильно сменить навигацию на `OPT`?

   (a) через `board/set_parameters`

   (b) сервисом `navigation/set_system`

   (c) перезагрузкой

   Ответ: (b). Меняют только этим сервисом; в воздухе у Мини 2 он отказывает (`nav_system_change_in_flight: false`).

10. Захват вернул `success = False`. Что это значит?

    (a) модуль выключен параметром узла или нет на борту

(b) программа с ошибкой

(c) захват сломан

Ответ: (a) — например, у Мини 2 магнитный `cargo_set` отказывает, а механический `grab` работает.

19.19 Задания к главе

ЗАДАНИЕ 19.16 · Телеметрия до остановки

Узел подписывается на высоту, позицию и батарею и раз в секунду печатает строку; при низком напряжении (`voltage` меньше порога, который вы задали параметром узла) печатает предупреждение. Остановка по `Ctrl` + `C` закрывает узел без трассировки ошибки.

Что должно получиться: строки идут раз в секунду; порог берётся из параметра узла (`declare_parameter`); после `Ctrl+C` в консоли чисто.

В симуляторе: миссии Мини 2 SDK2: 44, 45 в панели «**Обучение**».

ЗАДАНИЕ 19.17 · Разведчик на двух узлах

Мини-проект. Два узла в двух программах (два файла): первый публикует скорость вперёд по таймеру, второй следит за дальномером Ranger (`payload/ranger`) и при расстоянии впереди меньше 0.7 м вызывает остановку и посадку. Узлы общаются только топиками.

Что должно получиться: дрон летит, пока впереди чисто; при препятствии скорость обнуляется и дрон садится; узлы можно запускать и останавливать независимо.

19.20 Миссии симулятора по разделам главы

Главы 14–17 трека «**Мини 2 · SDK2**» (миссии 38–56) идут в порядке главы:

№	Тема миссий	Раздел главы
38	Первый узел `rclpy`	консоль и первый узел
39	Полётный цикл сервисами	сервисы полёта
40–41	Точка в `map`, шаги в `base_link`	системы координат
42–43	Отказ сервиса, возврат `rtl`	отказ сервиса и возврат домой
44–46	Батарея, высота и позиция, события	топики и телеметрия; события
47–48	`cmd_vel`: вперёд и квадрат по таймеру	управление скоростью
49	Каналы RC	виртуальный пульт

№	Тема миссий	Раздел главы
50–51	Параметры автопилота, система навигации	параметры; навигация
52–53	Светодиоды, захват	полезная нагрузка
54	Конфиг узла, GNSS-топики	параметры узла
55–56	Узел-автомат и патруль	итоговый проект

Итоги

- ROS 2 делит робота на узлы, общающиеся топиками (потоки), сервисами (запрос и ответ) и параметрами (настройки); `ros_pioneer_node` — мост между автопилотом и этой сетью, все имена начинаются с `/geoscan/`.
- Полётный цикл — сервисы `arm`, `takeoff`, `go_to_local_point`, `land`, `disarm`; ответ сервиса означает «принято», окончание видно по `fly_state` и `callback_event`; `success = False` — отказ, а не ошибка.
- `map` — мировая система от старта, `base_link` — система коптера (x вперёд, y влево, не как в `pioneer_sdk2`); курс в сервисах в градусах, абсолютный; равный прошлому не поворачивает.
- Телеметрия приходит подпиской: обработчик запоминает, таймер печатает. События автопилота — числа в `flight/callback_event`.
- Скорость — публикация `TwistStamped` в `cmd_vel` таймером; `angular.z` в рад/с, предел ± 1.82; виртуальный пульт — `rc/set_channels`, публиковать постоянно.
- Параметры автопилота — сервисами `board/*` (прочитать, записать, вернуть), параметры узла — `ros2 param` и YAML; навигацию меняют только `navigation/set_system`.
- Нагрузка: светодиоды 0..1 (`PIXEL`, `FILL`), захват (механический у Мини 2, магнитный у Базового); выключенный модуль даёт отказ.
- Миссия-узел — автомат: состояние в поле, обработчики фиксируют факты, решает таймер.
- Всё проверено на симуляторе; на настоящем дроне сверяйте поля сервисов, единицы курса и срок удержания скорости.

ГЛАВА 20

Рой: Swarm Kit и pioneer_swarm

Цели главы

После главы вы сможете:

- объяснить, как устроен рой Swarm Kit без центрального сервера: юнит, `machine_id`, Heartbeat, почтовый ящик, подписка на телеметрию;
- получить список соседей и отправить сообщение конкретному дрону и всем сразу библиотекой `pioneer_swarm`;
- принимать сообщения без блокировки программы и придумать для них простой текстовый протокол;
- читать телеметрию соседа и использовать её в своём полёте: следовать за ведущим и держать место в строю;
- синхронизировать группу (взлёт по готовности, световое шоу) и собрать итоговый проект «поиск и спасение»;
- назвать ограничения: общая система координат, нет защиты от столкновений, нет гарантий доставки.

Нужно знать: полёт `pioneer_sdk2` (глава 16); главу 19 полезно прочитать до этой (типы телеметрии те же).

Миссии симулятора к главе

К главе относятся миссии 57–64 (главы трека «**Мини 2 · SDK2**» 18–19): общение в рое (57–60) и телеметрия со строем (61–64). Миссия сама добавляет на сцену нужных соседей: Пионер Мини 2 появляются кнопками панели «**Рой**», у некоторых работает своя программа-бот, которую можно прочитать, выбрав дрон в панели. Программы запускаются в режиме «**Python · SDK2**».

На что опирается глава и что не проверено

Основа главы — документация Swarm Kit и `pioneer-swarm` на docs.geoscan.ru (Pioneer OS 1.0.0 и новее) и десять тем в `examples/python-onboard/swarm/` (шестнадцать файлов: примеры из двух частей лежат отдельными файлами). Программы проверены на симуляторе с несколькими дронами и виртуальной сетью; *на настоящих дронах авторы их не запускали*, как и на настоящей Wi-Fi-сети: в симуляторе нет задержек, потерь и перегрузки канала, доставка мгновенная. Числа сети (Heartbeat раз в секунду, исчезновение через 3 с, ящик на 100 сообщений, домены) — допущения симулятора, справочник Геоскана их не называет; они помечены словами «в симуляторе».

20.1 Рой без центра

Рой в Swarm Kit — это несколько юнитов (бортов) в одной сети (Wi-Fi или Ethernet), которые находят друг друга, обмениваются сообщениями и делятся телеметрией *напрямую*. Центрального сервера и наземной станции нет: все равноправны. У каждого юнита есть имя — идентификатор `machine_id` (например, `pioneer1`); по нему строятся адреса данных и почтовый ящик (рисунок 51).

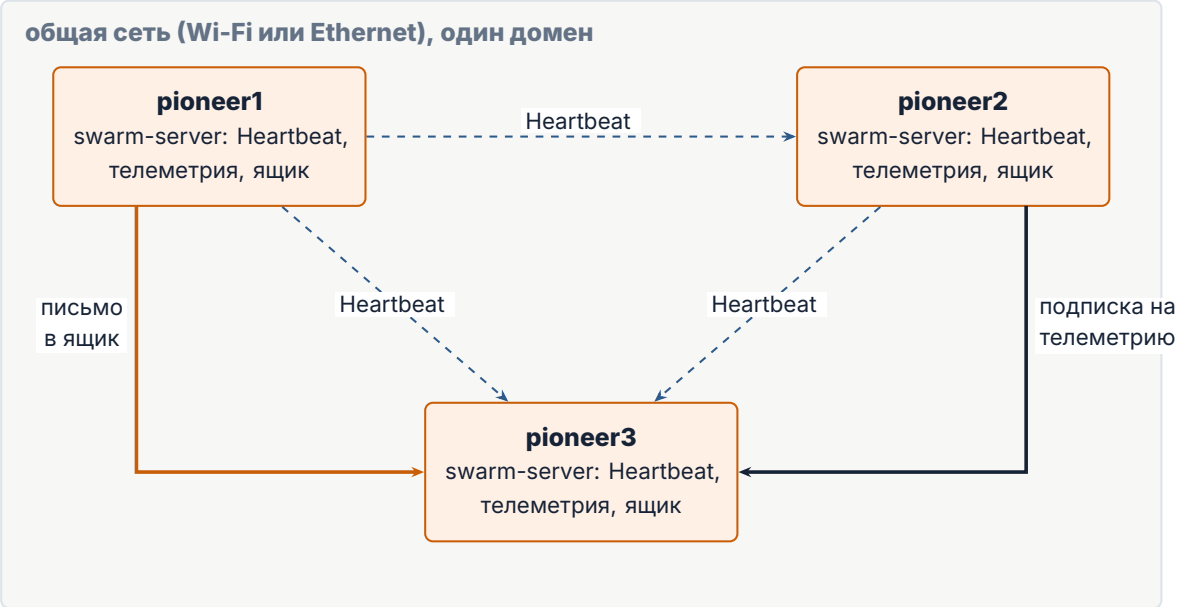


Рис. 51. Рой без центра. Каждый юнит равноправен и сам рассылает Heartbeat («я жив, вот мои данные»; штриховые стрелки); письмо адресуется в ящик конкретного `machine_id` (оранжевая стрелка), телеметрию соседа читают по подписке (тёмная стрелка). Ни сервера, ни наземной станции нет.

На каждом борту с Pioneer OS работает служба `swarm-server`: она объявляет юнит в сети и отдаёт данные. Приложение на борту обращается к ней через библиотеку `pioneer_swarm`. Юнит может быть воздушным (`AERO`), наземным (`ROVER`), водным (`MARINE`), роботом (`ROBOT`) или иным (`OTHER`); типы различаются набором объявляемых данных. У Пионера и Мини 2 в рое обычно тип воздушный: позиция, скорость, IMU, высота, батарея и события полёта.

Участники общаются тремя способами:

Способ	Как устроен	Метод библиотеки
Обнаружение: «кто в сети?»	каждый юнит периодически рассылает Heartbeat («я жив»): свой тип и каталог данных, которые готов отдавать; из него соседи узнают, на что можно подписаться	`get_unit_list()`
Телеметрия: «что у соседа?»	юнит подписывается на потоки соседа (позиция, батарея, …); пока подписчиков нет, поток в сеть не льётся; имя потока начинается с id юнита: `pioneer1/navigation/local/position`	`get_telemetry_from_unit()`
Сообщения: «написать юниту»	у юнита есть почтовый ящик: входящие строки; содержимое не регламентировано (команда, статус, текст)	`send_message_to_unit()`, `get_next_message()`

Если юнит пропал из сети, он исчезает из списка соседей, а подписки на него можно снять. Рой одноранговый: добавить юнит нового типа можно, не меняя остальных: из его Heartbeat понятно, что он умеет.

Рой в симуляторе: сеть и номера

В симуляторе `swarm-server` есть у каждого дрона сцены, работающего на `pioneer_sdk2` (Мини 2 и Базовый с Radxa), и работает всегда — даже без вашей программы. Дроны на `pioneer_sdk` в рое не видны. `machine_id` образуется из конца имени дрона («Пионер Мини 2 · 3» → `pioneer3`), а если номер занят, берётся наименьший свободный; его показывает панель «**Рой**» под названием дрона. Heartbeat идёт раз в секунду, юнит пропадает из списка, если его Heartbeat не слышен 3 секунды (удалённый дрон, выключенный `swarm-server`). Сеть — виртуальная: доставка мгновенная, потерь нет.

Кнопка «Запуск» запускает один дрон

«Запуск» запускает программу только выбранного дрона, но программы остальных дронов продолжают работать. Чтобы запустить рой: выберите дрон, загрузите его программу, нажмите «Запуск»; затем то же на втором дроне. Поэтому примеры из двух частей лежат отдельными файлами, а порядок запуска — *сначала ожидающая сторона* (получатель, ведомый), потом отправитель или ведущий. Каждая шапка файла подсказывает, что запускать на втором дроне.

20.2 Библиотека pioneer_swarm

Минимальная программа

```
from pioneer_swarm import PioneerSwarmUnit, SwarmUnitType

swarm = PioneerSwarmUnit()
# ... работа с роем ...
swarm.close()
```

Объект `PioneerSwarmUnit` — клиент сети в вашей программе. Создание:

Конструктор

```
PioneerSwarmUnit(domain_id=0, machine_id=None,
                 sync_wait_timeout_sec=None)
```

Параметр	Умолчание	Смысл
`domain_id`	0	домен роя: у всех юнитов должен совпадать; соседи видны только внутри домена
`machine_id`	авто	свой идентификатор; определяется с борта сам
`sync_wait_timeout_sec`	авто	сколько ждать появления id, если он ещё неизвестен

Без определённого `machine_id` объект не создаётся (в симуляторе — `RuntimeError` с пояснением). Свойства: `.id` — свой `machine_id`; `.unit_type` — свой тип (`SwarmUnitType` или `None`: значение появляется после первого собственного Heartbeat, в симуляторе — через полсекунды после создания).

Метод	Что делает
`get_unit_list()`	список `machine_id` соседей в сети; себя не включает
`send_message_to_unit(unit_id, message)`	отправляет строку; `True` — адресат найден и сообщение ушло, `False` — юнита в списке нет
`get_next_message()`	следующее входящее сообщение, *без ожидания*: кортеж `(sender_id, text)` или `None`, если ящик пуст
`get_telemetry_from_unit(unit_id, timeout_sec=60.0)`	ждёт телеметрию соседа и возвращает `(тип, словарь потоков)`; `None` — юнит не найден
`close()`	останавливает слежение и отключается от роя

Метод `close()` вызывайте, когда работа закончена, — лучше в `finally`: подписки и ящик объекта освобождаются. Опечатки в типах и значениях ловятся исключениями: неверный тип аргумента — `TypeError`, недопустимое значение (`domain_id` вне 0–232, отрицательный `timeout_sec`) — `ValueError`, вызов после `close()` — `RuntimeError`.

Список соседей в симуляторе отсортирован по номерам (`pioneer2`, `pioneer10`); у настоящей библиотеки порядок не задан, поэтому не опирайтесь на него.

Домены и свой machine_id

Объект с `domain_id = N` переводит юнит своего дрона в домен N на время жизни объекта; соседи видны только внутри домена, письма и телеметрия в другой домен не доходят (`False`, `None`). Своё `machine_id=...` (латинские буквы, цифры, `_`) переименовывает юнит, пока объект открыт; занятое имя — `RuntimeError`. Это поведение симулятора (допущение): на настоящем борту домен и идентификатор задаёт конфигурация, сверьте её по документации Pioneer OS.

ЗАДАНИЕ 20.1 · Рой на панели

Добавьте на панели «**Рой**» ещё два Пионера Мини 2. Выпишите их `machine_id` из панели и ответьте: кого увидит программа на первом дроне в `get_unit_list()` и почему себя в списке не будет? Что изменится, если удалить второй дрон: через сколько секунд он пропадёт из списка?

Что должно получиться: у первого дрона `pioneer1`, соседи `pioneer2` и `pioneer3`; себя в списке нет по определению; удалённый дрон исчезает примерно через три секунды (три периода Heartbeat).

20.3 Кто в сети

Цель раздела: подключиться к рою, узнать свой идентификатор и список соседей и корректно закрыть соединение.

python-onboard/swarm/swarm_01_who_is_here.py

```
# Рой: кто в сети? Запустите на любом дроне (Мини 2 или Базовый с Radxa, язык
↪  «Python · SDK2»).
# Остальные дроны сцены (панель «Рой») отвечать не обязаны: на каждом борту с
↪  pioneer_sdk2 работает swarm-server,
# он сам раз в секунду рассылает Heartbeat («я жив, мой тип и мои данные»).
↪  Добавьте 1–2 дрона и запустите программу.
from pioneer_swarm import PioneerSwarmUnit, SwarmUnitType
import time

swarm = PioneerSwarmUnit()
print('Я:', swarm.id)

# Свой тип известен после первого собственного Heartbeat — до него unit_type
↪  равен None.
while swarm.unit_type is None:
    time.sleep(0.1)
print('Мой тип:', swarm.unit_type, '(воздушный)' if swarm.unit_type ==
↪  SwarmUnitType.AERO else '')

for second in range(5):
```

```
    neighbours = swarm.get_unit_list()  # соседи, себя в списке нет
    print(second, 'соседей:', len(neighbours), neighbours)
    time.sleep(1)

swarm.close()
```

Что происходит. `PioneerSwarmUnit()` подключает программу к рою, `swarm.id` — свой `machine_id`. Свой тип известен только после первого собственного Heartbeat, поэтому цикл `while swarm.unit_type is None` ждёт его: до этого свойство равно `None`. Затем пять раз в секунду печатается список соседей. Он не содержит самого дрона и меняется на лету: новый юнит появляется после своего первого Heartbeat (в симуляторе — примерно через полсекунды), пропавший — через три периода тишины. В конце `swarm.close()` освобождает подписки. Остальные дроны сцены отвечать не обязаны: `swarm-server` каждого борта объявляет его сам.

ЗАДАНИЕ 20.2 · Кто в рое

Добавьте в сцену два дрона Мини 2 и запустите программу на первом. Проверьте, что программа печатает свой `machine_id`, тип и список соседей, и что себя в списке нет. Затем дополните её: для каждого соседа печатайте отдельную строку «Сосед: …».

Что должно получиться: у дрона 1 — «Я: pioneer1» и «Мой тип: SwarmUnitType.AERO (воздушный)», соседи `pioneer2` и `pioneer3` в каждой из пяти строк списка и по строке «Сосед:» на каждого.

В симуляторе: миссии Мини 2 SDK2: 57 в панели «**Обучение**».

20.4 Сообщения: ящик и опрос

Цель раздела: отправлять и принимать текстовые сообщения и строить на них простой протокол.

Сообщение — строка. Её отправляют в ящик конкретного юнита:

Отправка и приём

```
# True или False
ok = swarm.send_message_to_unit('pioneer2', 'ping 1')
# (sender, text) или None
message = swarm.get_next_message()
```

`send_message_to_unit` возвращает `True`, если адресат найден и сообщение ушло, и `False`, если юнита в списке нет — в том числе когда вы пишете самому себе. Содержимое протокол не проверяет, но и доставку не подтверждает: `True` означает «ушло», а не «прочитано». Приём устроен иначе, чем у обычных подписок: `get_next_message()` *не ждёт* и возвращает `None`, когда ящик пуст; ждать сообщение нужно самому — опросом с паузой и тайм-аутом.

Опрос ящика — с паузой и пределом

Цикл ожидания сообщения всегда имеет паузу (`time.sleep(0.1)`), иначе он занимает все ресурсы процессора, и предел (счётчик времени или число попыток), иначе программа зависнет навсегда, если собеседник не ответит. При выходе по пределу программа решает сама: повторить, отказаться, сообщить оператору.

Свойства ящика в симуляторе

Доставка мгновенная, порядок сохраняется. Ящик принадлежит юниту, а не программе: письмо соседу, у которого нет программы, ждёт, пока она появится. Ящик ограничен 100 сообщениями (старые вытесняются), очищается, когда закрывается последний объект юнита (`close()` или конец программы), и пропадает вместе с юнитом. Несколько объектов на одном дроне читают из одного ящика. На настоящей сети эти свойства — допущения: доставку и порядок там никто не гарантирует.

Ping-pong

Классическая проверка связи: один дрон отправляет «ping N» и ждёт «pong N», другой на каждое «ping» отвечает «pong». Два файла — для двух дронов; первым запускают отвечающего.

python-onboard/swarm/swarm_02_ping_pong_pong.py

```
# Ping-pong, часть 2 из 2: этот файл запускают на ВТОРОМ дроне (раньше, чем
↪  swarm_02_ping_pong_ping.py на первом).
# Дрон ждёт сообщения в своём почтовом ящике и на каждое отвечает «pong»
↪  тому, кто написал.
from pioneer_swarm import PioneerSwarmUnit
import time

swarm = PioneerSwarmUnit()
print('Я', swarm.id, '- жду сообщения...')
silence = 0.0
while silence < 10.0:
    message = swarm.get_next_message()
    if message is None:
        time.sleep(0.1)
        silence += 0.1
    else:
        silence = 0.0
        sender, text = message
        print('Получил от', sender, ':', text)
        swarm.send_message_to_unit(sender, 'pong' + text[4:])
print('Десять секунд тишины, закрываю соединение')
swarm.close()
```

Что происходит (pong). Дрон ждёт сообщение в ящике. Пустой ящик — пауза 0.1 с и накопление счётчика тишины `silence`; пришло сообщение — счётчик обнуляется, из адресата и текста выделяется ответ: `'pong' + text[4:]` заменяет первые четыре буквы

«ping» на «pong», оставляя номер. Ответ уходит тому, кто написал: `sender`. Десять секунд тишины — программа закрывает соединение.

python-onboard/swarm/swarm_02_ping_pong_ping.py

```
# Ping-pong, часть 1 из 2: этот файл запускают на ПЕРВОМ дроне, на втором –
↪  swarm_02_ping_pong_pong.py.
# Порядок: на панели «Рой» выберите дрон 2, загрузите pong, нажмите «Запуск»;
↪  затем выберите дрон 1,
# загрузите ping и тоже нажмите «Запуск». Дрон отправляет «ping N» соседу и
↪  ждёт «pong N».
from pioneer_swarm import PioneerSwarmUnit
import time

swarm = PioneerSwarmUnit()
neighbours = swarm.get_unit_list()
if not neighbours:
    print('Соседей нет: добавьте второй дрон на панели «Рой» (язык Python ·
    ↪  SDK2)')
else:
    partner = neighbours[0]
    for number in range(1, 6):
        delivered = swarm.send_message_to_unit(partner, 'ping ' +
        ↪  str(number))
        print('Отправил', partner, ': ping', number, '– доставлено' if
        ↪  delivered else '– юнита нет в сети')
        answer = None
        waited = 0.0
        while answer is None and waited < 3.0:
            answer = swarm.get_next_message()  # без ожидания: пустой ящик –
            ↪  None
            time.sleep(0.1)
            waited += 0.1
        if answer is None:
            print('Ответа нет')
        else:
            sender, text = answer
            print('Ответ от', sender, ':', text)
swarm.close()
```

Что происходит (ping). Первый сосед из списка — собеседник (`partner`). Пять раз: отправка «ping N», затем ожидание ответа до трёх секунд (`waited`) опросом с паузой. Если сосед не отвечает, строка «Ответа нет» — и цикл идёт дальше. Если соседей нет вовсе, программа подсказывает добавить дрон.

ЗАДАНИЕ 20.3 · Ping-pong

Запустите `swarm_02_ping_pong_pong.py` на втором дроне, затем `swarm_02_ping_pong_ping.py` на первом. Измените протокол: в ответ добавляйте время ожидания и свой `machine_id` (`pong 3 от pioneer2`). Что произойдёт, если запустить «ping» раньше «pong»? Объясните по правилу об опросе ящика.

Что должно получиться: первый дрон печатает пять строк «Ответ от pioneer2 : pong N»; второй — пять строк «Получил от pioneer1 : ping N»; если «pong» запущен

позже, письма ждут в его ящике до старта программы (ящик принадлежит юниту).

В симуляторе: миссии Мини 2 SDK2: 58, 59 в панели «**Обучение**».

Рассылка всем соседям

В рое нет «сообщения всем»: письмо уходит каждому юниту отдельно, а общая рассылка — цикл по списку.

python-onboard/swarm/swarm_03_broadcast_receiver.py

```python
# Рассылка всем соседям, часть 2 из 2: получатель. Запустите на втором и на
↪  третьем дроне
# (раньше, чем swarm_03_broadcast_sender.py на первом). Печатает всё, что
↪  пришло в почтовый ящик.
from pioneer_swarm import PioneerSwarmUnit
import time

swarm = PioneerSwarmUnit()
print(swarm.id, 'слушает рой')
received = 0
waited = 0.0
while waited < 8.0:
    message = swarm.get_next_message()
    if message is None:
        time.sleep(0.1)
        waited += 0.1
    else:
        sender, text = message
        received += 1
        print(swarm.id, 'получил от', sender, ':', text)
print(swarm.id, 'получил сообщений:', received)
swarm.close()
```

python-onboard/swarm/swarm_03_broadcast_sender.py

```python
# Рассылка всем соседям, часть 1 из 2: отправитель — запускают на ПЕРВОМ
↪  дроне. Получатели — swarm_03_broadcast_receiver.py
# на втором и третьем дронах (запустите их раньше). В рое нет «сообщения
↪  всем»: письмо уходит каждому юниту отдельно.
from pioneer_swarm import PioneerSwarmUnit
import time

swarm = PioneerSwarmUnit()
time.sleep(2)  # получатели успели запуститься
neighbours = swarm.get_unit_list()
delivered = 0
for unit_id in neighbours:
    if swarm.send_message_to_unit(unit_id, 'Всем привет от ' + swarm.id):
        delivered += 1
print('Соседей:', len(neighbours), ', доставлено:', delivered)

# Юниту, которого нет в сети, письмо не уходит: метод возвращает False.
print('Юниту pioneer99:', swarm.send_message_to_unit('pioneer99', 'эй'))
```

```
swarm.close()
```

Что происходит. Получатель (его запускают на втором и третьем дронах первым) восемь секунд опрашивает ящик и печатает всё пришедшее со счётчиком. Отправитель выжидает две секунды, чтобы получатели успели запуститься, затем перебирает соседей и считает, кому письмо ушло. Вызов для несуществующего юнита (`pioneer99`) возвращает `False`: эту проверку используйте всегда, вместо того чтобы считать, что письмо дошло.

ЗАДАНИЕ 20.4 · Рассылка и подтверждение

Расширьте рассылку: отправитель отправляет «кто здесь?» всем соседям, а каждый получатель отвечает «здесь ⟨id⟩». Отправитель печатает «Ответили: N из M» и строку для каждого ответившего.

Что должно получиться: при двух получателях — «Ответили: 2 из 2» и по строке «pioneerN - здесь pioneerN» на каждого; если один получатель не запущен, число ответивших меньше числа соседей.

В симуляторе: миссии Мини 2 SDK2: 60 в панели «**Обучение**».

Протокол из строк. Сообщение — просто строка, поэтому формат придумываете вы. Для команд подходят короткие слова (`'взлёт'`, `'отбой'`); для данных — слово и значения через пробел (`'НАЙДЕН 2.00 1.50'`), которые разбирают `text.split()` и `float`. Правила хорошего протокола: одно сообщение — одна мысль; команда в начале, данные после; при разборе проверять число частей и допускать мусор (чужое сообщение не должно ронять программу). Примеры дальше (синхронный взлёт, световое шоу, поиск и спасение) — именно такие протоколы.

20.5 Телеметрия соседей

Цель раздела: читать данные соседнего дрона и разбирать ответ, не полагаясь на то, что все потоки уже пришли.

Каждый юнит объявляет в Heartbeat каталог своих данных. У воздушного юнита в симуляторе это девять потоков (имена даны без идентификатора юнита; полное имя в сети, например, `pioneer1/sensors/altitude`). Те же типы и единицы, что у топиков `gs_core` (раздел 19.8), только без префикса `/geoscan/`:

Поток	Тип	Период
`navigation/local/position`	`geometry_msgs/Point`	0.05 с
`navigation/local/velocity`	`geometry_msgs/Vector3`	0.05 с
`navigation/local/yaw`	`std_msgs/Float32` (радианы)	0.05 с

Поток	Тип	Период
navigation/local/status	gs_interfaces/NavStatus	0.05 с
sensors/imu	sensor_msgs/Imu	1/30 с
sensors/altitude	std_msgs/Float32	0.1 с
sensors/battery_state	sensor_msgs/BatteryState	0.5 с
flight/fly_state	std_msgs/Int8	0.1 с
flight/callback_event	std_msgs/Int8	по событию

Датчики `range`, `motors_rpm`, оптический поток, Ranger и GPS в рое не объявляются. Каталог — допущение симулятора (в документации Геоскана указано лишь, что юнит объявляет позицию, скорость, IMU, высоту, батарею и события).

python-onboard/swarm/swarm_04_neighbour_telemetry.py

```
# Телеметрия соседей. Запустите на любом дроне: остальные могут стоять на
↪  земле и не выполнять программ –
# swarm-server каждого борта отдаёт данные тем, кто на них подписался. Первый
↪  запрос подписывает на потоки соседа,
# get_telemetry_from_unit ждёт по одному сэмплу каждого потока (не дольше
↪  timeout_sec) и возвращает словарь потоков.
from pioneer_swarm import PioneerSwarmUnit, SwarmUnitType

swarm = PioneerSwarmUnit()
for unit_id in swarm.get_unit_list():
    result = swarm.get_telemetry_from_unit(unit_id, timeout_sec=5.0)
    if result is None:
        print(unit_id, ': не найден в сети')
        continue
    unit_type, telemetry = result
    if unit_type != SwarmUnitType.AERO:
        print(unit_id, ': не дрон, пропускаю')
        continue
    position = telemetry['navigation/local/position']
    altitude = telemetry['sensors/altitude']
    battery = telemetry['sensors/battery_state']
    flight = telemetry['flight/fly_state']
    print(unit_id, ': позиция (%.2f, %.2f, %.2f) м' % (position.x,
    ↪  position.y, position.z))
    print('   высота %.2f м, батарея %.2f В, состояние полёта %d (0 – на
    ↪  земле, 1 – моторы, 2 – в воздухе)' % (altitude.data,
    ↪  battery.voltage, flight.data))
    print('   потоков:', len(telemetry), '–', ',
    ↪  '.join(sorted(telemetry.keys())))
swarm.close()
```

Что происходит. Для каждого соседа вызов `get_telemetry_from_unit` с `timeout_sec=5.0` возвращает либо `None` (юнита нет в списке), либо пару `(тип, словарь)`. Тип сверяется с `SwarmUnitType.AERO`: не дрон программа пропускает. Ключ словаря — имя потока, значение — *объект сообщения ROS 2*: у позиции

поля `x, y, z`, у высоты `data`, у батареи `voltage`, у состояния полёта `data` (0, 1, 2). Последняя строка печатает число потоков и их имена.

> **Телеметрия по подписке и с тайм-аутом**
>
> Первый вызов для соседа *подписывает* объект на все потоки каталога и ждёт, пока придёт по образцу каждого периодического (самый медленный — батарея, 0.5 с), но не дольше `timeout_sec`; подписка держится до `close()`. Дальше вызов возвращает накопленное. Поэтому при коротком тайм-ауте словарь может быть неполным или пустым — проверяйте наличие ключа, а не разворачивайте словарь вслепую. `timeout_sec=0` — только подписаться и вернуть накопленное; `None` — ждать все потоки без ограничения. Пока подписчиков нет, поток в сеть не льётся. Если сосед пропал во время ожидания, вернётся `None`.

Поток `flight/callback_event` в словаре появляется только после первого события с момента подписки; событий точки (коды 5 и 6) в рое нет: флаг точки принадлежит программе управления дроном. Остальные потоки присутствуют, как только пришёл первый образец.

ЗАДАНИЕ 20.5 · Что у соседа

Запустите `swarm_04_neighbour_telemetry.py` на дроне, стоящем рядом с двумя другими; затем поднимите в воздух второй дрон отдельной программой (полёт и зависание) и повторите. Сравните высоту и состояние полёта соседа до и после.

Что должно получиться: на земле сосед: высота около нуля, состояние полёта 0; в воздухе — высота около заданной (0.7–1.6 м), состояние 2; в каждом ответе одни и те же восемь потоков (и девятый после события).

В симуляторе: миссии Мини 2 SDK2: 61 в панели «**Обучение**».

Монитор роя

python-onboard/swarm/swarm_10_monitor.py

```python
# Монитор роя: кто появился, кто пропал, как дела у остальных. Запустите на
↪  любом дроне и, не останавливая программу,
# добавьте или удалите дрон на панели «Рой»: новый юнит появится через
↪  секунду, удалённый — примерно через три
# (Heartbeat раз в секунду; юнит, чей Heartbeat не слышен три периода,
↪  считается пропавшим).
from pioneer_swarm import PioneerSwarmUnit
import time

swarm = PioneerSwarmUnit()
print('Монитор на', swarm.id)
known = set()
for tick in range(20):
    present = set(swarm.get_unit_list())
    for unit_id in sorted(present - known):
        print('+ в сети:', unit_id)
```

```
    for unit_id in sorted(known - present):
        print('- пропал:', unit_id)
    known = present
    for unit_id in sorted(present):
        result = swarm.get_telemetry_from_unit(unit_id, timeout_sec=2.0)
        if result is not None:
            telemetry = result[1]
            print('   %s: высота %.2f м, батарея %.2f В' % (unit_id,
            ↪  telemetry['sensors/altitude'].data,
            ↪  telemetry['sensors/battery_state'].voltage))
    time.sleep(1)
swarm.close()
```

Что происходит. Раз в секунду программа сравнивает множества: те, кто в сети сейчас (`present`), и те, кто был в прошлый раз (`known`). Разность `present - known` — вновь появившиеся, `known - present` — пропавшие. Для каждого присутствующего печатаются высота и напряжение батареи. Запустите программу и, не останавливая её, добавьте или удалите дрон на панели «**Рой**»: новый юнит появится через секунду, удалённый — примерно через три (три периода Heartbeat).

ЗАДАНИЕ 20.6 · Тревога по батарее

Дополните монитор: если напряжение соседа ниже заданного порога, печатайте «ВНИМАНИЕ: pioneerN, батарея ...», но один раз на юнит, пока напряжение не вернётся выше порога. Для проверки удобно ввести порог чуть выше текущего значения.

Что должно получиться: на каждого соседа ниже порога — одна строка предупреждения, а не по строке на каждый опрос; вернувшийся в норму юнит можно будет предупредить снова.

20.6 Ведущий и ведомый

Цель раздела: использовать телеметрию соседа в собственном полёте — лететь за ведущим на заданном расстоянии.

Идея: ведущий просто выполняет свой маршрут и ни о чём не знает; ведомый раз в полсекунды читает позицию ведущего из его телеметрии и летит в точку со смещением. Ведущему *не нужно ничего отправлять*: данные берутся из публичной телеметрии. Два файла: ведомый запускается первым, ведущий — вторым.

python-onboard/swarm/swarm_05_follow_follower.py

```
# Ведущий и ведомый, часть 2 из 2: ВЕДОМЫЙ - запускают на ВТОРОМ дроне,
↪  раньше ведущего (swarm_05_follow_leader.py).
# Каждые полсекунды ведомый читает локальную позицию ведущего и летит в точку
↪  на метр позади него (по оси x).
from pioneer_sdk2 import Pioneer
```

```
from pioneer_swarm import PioneerSwarmUnit
import time

LEADER = 'pioneer1'  # machine_id ведущего (смотрите его в панели «Рой»)
OFFSET_X = -1.0      # на сколько метров позади (по оси x) держаться
OFFSET_Y = 0.0

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
pioneer.arm()
pioneer.takeoff()
print('Ведомый', swarm.id, 'следует за', LEADER)

for step in range(60):
    result = swarm.get_telemetry_from_unit(LEADER, timeout_sec=2.0)
    if result is None:
        print('Ведущий', LEADER, 'не найден в сети')
        break
    position = result[1]['navigation/local/position']
    pioneer.go_to_local_point(x=position.x + OFFSET_X, y=position.y +
    ↪  OFFSET_Y, z=1.0)
    time.sleep(0.5)

pioneer.land()
swarm.close()
```

Что происходит (ведомый). Идентификатор ведущего `LEADER = 'pioneer1'` задан константой; смещение `OFFSET_X = -1.0` означает «на метр позади по оси x». Шестьдесят раз с шагом 0.5 с ведомый читает позицию ведущего (`get_telemetry_from_unit` с тайм-аутом 2 с) и отправляет `go_to_local_point` в точку «позиция ведущего плюс смещение» на высоте 1 м. Если ведущий пропал из сети (`None`), цикл прекращается; после цикла дрон садится. Обратите внимание: ведомый вызывает команду точки, не дожидаясь прибытия, — следующая точка просто заменяет предыдущую.

python-onboard/swarm/swarm_05_follow_leader.py

```
# Ведущий и ведомый, часть 1 из 2: ВЕДУЩИЙ — запускают на ПЕРВОМ дроне
↪  (pioneer1). Он просто летит по квадрату;
# говорить ведомому ничего не нужно: позицию ведущего ведомый читает из его
↪  телеметрии.
# Ведомый — swarm_05_follow_follower.py на втором дроне; запустите его
↪  раньше, затем ведущего.
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
print('Ведущий', swarm.id, 'взлетает')
pioneer.arm()
pioneer.takeoff()

route = [(2, 0, 1), (2, 2, 1), (0, 2, 1), (0, 0, 1)]
```

```
for x, y, z in route:
    pioneer.go_to_local_point(x=x, y=y, z=z)
    while not pioneer.point_reached():
        time.sleep(0.1)
    print('Ведущий в точке', x, y, z)
    time.sleep(1)

pioneer.land()
swarm.close()
```

Что происходит (ведущий). Ведущий взлетает и облетает квадрат 2×2 м: четыре точки с ожиданием `point_reached()` в каждой и секундной паузой. Затем садится. Работа с `swarm` ограничена созданием и закрытием: ведущий присутствует в сети уже потому, что у его борта работает `swarm-server`.

Что нужно, чтобы следование работало на настоящих дронах

Ведомый подставляет координаты ведущего в *свою* систему координат. Это верно, только если у обоих *общая* система — например, оба летят по одной карте локальной системы навигации или стартуют из одной точки (так в симуляторе после сброса). Если у каждого дрона своё начало координат, следование приведёт ведомого не туда. Кроме того, в примере нет защиты от столкновений, нет контроля курса и запаса по батарее. Проверьте общие координаты на земле: поставьте оба дрона в разные точки и сравните их локальные позиции.

ЗАДАНИЕ 20.7 · Ведомый и ведущий

Запустите ведомого на втором дроне, затем ведущего на первом. Измените смещение ведомого на 1 м слева и 1 м позади и убедитесь по картинке 3D, что он держится ближе к ведущему слева. Добавьте в ведомый печать расстояния до ведущего.

Что должно получиться: ведущий печатает четыре строки «Ведущий в точке …»; ведомый летит следом с отставанием в полсекунды; оба садятся; расстояние до ведущего колеблется около выбранного смещения (порядка 1.4 м при смещении по обеим осям).

В симуляторе: миссии Мини 2 SDK2: 62 в панели «**Обучение**».

20.7 Строй: у каждого своё место

Один ведущий и несколько ведомых образуют строй: каждому ведомому нужно *своё* место — смещение от ведущего. Одну и ту же программу запускают на нескольких дронах, а место выбирают по `machine_id`.

python-onboard/swarm/swarm_08_formation_follower.py

```python
# Строй по относительной позиции. Запускают на втором и третьем дронах;
↪ ведущий — дрон 1 с программой
# swarm_05_follow_leader.py (запустите ведомых раньше). У каждого дрона своё
↪ место в строю, оно выбирается
# по номеру в machine_id: pioneer2 — первое место, pioneer3 — второе. Место —
↪ смещение от ведущего в осях сцены, метры.
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

LEADER = 'pioneer1'
SLOTS = [(-1.0, 1.0), (-1.0, -1.0), (-2.0, 0.0)]

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
number = int(swarm.id.replace('pioneer', ''))
dx, dy = SLOTS[(number - 2) % len(SLOTS)]
print(swarm.id, ': моё место в строю', dx, dy)

pioneer.arm()
pioneer.takeoff()
for step in range(60):
    result = swarm.get_telemetry_from_unit(LEADER, timeout_sec=2.0)
    if result is None:
        print('Ведущий', LEADER, 'не найден в сети')
        break
    leader = result[1]['navigation/local/position']
    pioneer.go_to_local_point(x=leader.x + dx, y=leader.y + dy, z=1.0)
    time.sleep(0.5)
pioneer.land()
swarm.close()
```

Что происходит. Список `SLOTS` — места: $(-1, 1)$, $(-1, -1)$, $(-2, 0)$ (метры в осях сцены). Номер дрона берётся из идентификатора (`int(swarm.id.replace('pioneer', ''))`), и место — `SLOTS[(number - 2) % len(SLOTS)]`: `pioneer2` занимает первое, `pioneer3` — второе. Остальное — как в предыдущем примере. Остаток от деления гарантирует, что при большем числе дронов места начнут повторяться, а не вызовут ошибку; в реальной программе это нужно обрабатывать отдельно: два дрона на одном месте столкнутся.

Ведущего для строя запускают так же, как в прошлом разделе (`swarm_05_follow_leader.py`). Раздача мест по номеру в имени проста, но хрупка: она рассчитана на `pioneer2` и далее. Надёжнее договориться сообщениями: ведущий рассылает каждому его место.

ЗАДАНИЕ 20.8 · Треугольник

Запустите ведомых на втором и третьем дронах, ведущего — на первом. Затем замените раздачу мест по номеру на раздачу сообщением: ведущий перед взлётом отправляет каждому ведомому строку `'место dx dy'`, а ведомый разбирает её и

только потом взлетает.

Что должно получиться: в 3D два ведомых летят справа и слева позади ведущего; при отсутствии сообщения ведомый не взлетает и печатает причину.

20.8 Синхронный взлёт по готовности

Группа не должна взлетать, пока кто-то не готов. Решение — протокол из трёх сообщений: командир посылает «готовность», каждый ведомый запускает моторы и отвечает «готов», командир ждёт всех и посылает «взлёт».

python-onboard/swarm/swarm_06_sync_takeoff_wingman.py

```
# Синхронный взлёт по сообщению, часть 2 из 2: ВЕДОМЫЙ — запускают на втором
↪  (и третьем) дроне, раньше командира
# (swarm_06_sync_takeoff_commander.py). Ждёт команды в почтовом ящике:
↪  «готовность», «взлёт», «посадка».
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

pioneer = Pioneer()
swarm = PioneerSwarmUnit()


def wait_for(command, limit=60.0):
    """Ждёт сообщение с текстом command; возвращает, кто его прислал, или
    ↪  None, если время вышло."""
    waited = 0.0
    while waited < limit:
        message = swarm.get_next_message()
        if message is not None:
            if message[1] == command:
                return message[0]
            if message[1] == 'отбой':
                return None
        time.sleep(0.1)
        waited += 0.1
    return None


print(swarm.id, 'ждёт команду')
commander = wait_for('готовность')
if commander is not None:
    pioneer.arm()
    swarm.send_message_to_unit(commander, 'готов')
    print(swarm.id, 'готов, моторы запущены')
    if wait_for('взлёт') is not None:
        pioneer.takeoff()
        print(swarm.id, 'в воздухе')
        wait_for('посадка')
        pioneer.land()
    else:
        pioneer.disarm()
```

```
else:
    print('Команды не было')
swarm.close()
```

python-onboard/swarm/swarm_06_sync_takeoff_commander.py

```
# Синхронный взлёт по сообщению, часть 1 из 2: КОМАНДИР — запускают на ПЕРВОМ
↪  дроне. Ведомые —
# swarm_06_sync_takeoff_wingman.py на остальных дронах (запустите их раньше).
↪  Команды рою — обычные текстовые сообщения:
# «готовность» -> ведомые запускают моторы и отвечают «готов»; после
↪  обратного отсчёта «взлёт»; в конце «посадка».
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
time.sleep(1)
wingmen = swarm.get_unit_list()
print('Командир', swarm.id, ', в строю:', wingmen)


def tell_all(text):
    for unit_id in wingmen:
        swarm.send_message_to_unit(unit_id, text)


tell_all('готовность')
ready = set()
waited = 0.0
while len(ready) < len(wingmen) and waited < 20.0:
    message = swarm.get_next_message()
    if message is None:
        time.sleep(0.1)
        waited += 0.1
    elif message[1] == 'готов':
        ready.add(message[0])
        print(message[0], 'готов')

if len(ready) == len(wingmen):
    for count in (3, 2, 1):
        print('Взлёт через', count)
        time.sleep(1)
    tell_all('взлёт')
    pioneer.arm()
    pioneer.takeoff()
    print('Все в воздухе')
    time.sleep(3)
    tell_all('посадка')
    pioneer.land()
else:
    print('Не все готовы, взлёт отменён')
    tell_all('отбой')
swarm.close()
```

Что происходит. Ведомый (запускается раньше) ждёт «готовность» функцией `wait_for(command, limit)`: опрос ящика с паузой и пределом, который возвращает отправителя, или `None`, если пришло «отбой» или время вышло. Получив «готовность», ведомый запускает моторы (`arm()`) и отвечает «готов». Затем ждёт «взлёт» и взлетает, а по «посадка» садится; без «взлёта» выключает моторы. Командир собирает ответы в множество `ready` (не дольше 20 секунд), и только когда готовы *все*, делает обратный отсчёт, рассылает «взлёт» и взлетает сам; иначе рассылает «отбой» и ничего не запускает. Моторы запущены до отсчёта, поэтому после «взлёта» дроны поднимаются почти одновременно.

> **Не взлетай, пока все не ответили**
>
> Протокол группы строят вокруг подтверждений: «готовность» → «готов» → «взлёт». Общее действие начинается только после того, как подтвердили все, а отказ любого участника (тишина, «отбой») отменяет всё. Предел ожидания нужен всегда: иначе один неотвечающий дрон держит группу навсегда.

ЗАДАНИЕ 20.9 · Взлёт по готовности

Запустите ведомых на втором и третьем дронах, затем командира. Посмотрите, что произойдёт, если не запускать одного из ведомых: сколько ждёт командир и что напечатает.

Что должно получиться: при двух ведомых командир печатает «pioneer2 готов», «pioneer3 готов», отсчёт 3, 2, 1 и «Все в воздухе»; все три дрона взлетают и садятся; если ведомый не запущен, через 20 с командир печатает «Не все готовы, взлёт отменён».

В симуляторе: миссии Мини 2 SDK2: 63 в панели «**Обучение**».

20.9 Световое шоу

Тот же приём — команды-слова — подходит для синхронной подсветки: дирижёр рассылает название цвета, а каждый игрок включает его.

python-onboard/swarm/swarm_07_led_show_player.py

```
# Световое шоу роя, часть 2 из 2: ИГРОК — запускают на втором (и третьем)
↪  дроне, раньше дирижёра
# (swarm_07_led_show_conductor.py). Включает светодиоды в цвет, который
↪  назвал дирижёр; «off» — конец шоу.
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

COLOURS = {'red': (1, 0, 0), 'green': (0, 1, 0), 'blue': (0, 0, 1), 'white':
↪  (1, 1, 1), 'off': (0, 0, 0)}

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
```

```
print(swarm.id, 'ждёт сигнал дирижёра')
silence = 0.0
finished = False
while silence < 15.0 and not finished:
    message = swarm.get_next_message()
    if message is None:
        time.sleep(0.05)
        silence += 0.05
    else:
        silence = 0.0
        text = message[1]
        if text in COLOURS:
            r, g, b = COLOURS[text]
            pioneer.led_control(255, r, g, b)
            print(swarm.id, ':', text)
        if text == 'off':
            finished = True
swarm.close()
```

python-onboard/swarm/swarm_07_led_show_conductor.py

```
# Световое шоу роя, часть 1 из 2: ДИРИЖЁР — запускают на ПЕРВОМ дроне. Игроки
↪  — swarm_07_led_show_player.py
# на остальных дронах (запустите их раньше). Дирижёр рассылает название цвета
↪  и сам включает такой же цвет.
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

COLOURS = {'red': (1, 0, 0), 'green': (0, 1, 0), 'blue': (0, 0, 1), 'white':
↪  (1, 1, 1)}

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
time.sleep(1)
print('Дирижёр', swarm.id, ', в шоу:', swarm.get_unit_list())

for lap in range(2):
    for name in ('red', 'green', 'blue', 'white'):
        for unit_id in swarm.get_unit_list():
            swarm.send_message_to_unit(unit_id, name)
        r, g, b = COLOURS[name]
        pioneer.led_control(255, r, g, b)  # 255 — все светодиоды
        print('Цвет:', name)
        time.sleep(1)

for unit_id in swarm.get_unit_list():
    swarm.send_message_to_unit(unit_id, 'off')
pioneer.led_control(255, 0, 0, 0)
swarm.close()
```

Что происходит. Игрок хранит словарь `COLOURS` (название → тройка 0..1) и опрашивает ящик с паузой 0.05 с; незнакомое слово игнорируется (мусор не роняет программу), `'off'` завершает шоу. Дирижёр дважды проходит цвета красный, зелё-

ный, синий, белый: рассылает название каждому соседу, включает тот же цвет себе (`led_control(255, r, g, b)` — все светодиоды) и ждёт секунду. В конце рассылает `'off'` и гасит свои светодиоды. Рассылку по списку соседей при каждом цвете делают заново: за время шоу состав мог измениться.

ЗАДАНИЕ 20.10 · Свой номер шоу

Придумайте программу шоу: бегущий цвет по дронам (в каждый момент светится один), затем все вместе, затем гаснут по очереди. Для этого дирижёр рассылает не одно слово, а пару «номер и цвет».

Что должно получиться: в журнале игроков команды появляются в порядке сценария; финальное сообщение гасит светодиоды у всех участников, включая дирижёра.

20.10 Итоговый проект: поиск и спасение

Цель раздела: собрать группу из двух дронов с разными ролями: разведчик ищет, спасатель ждёт сигнал и прилетает в найденную точку.

Протокол одно сообщение: `'НАЙДЕН x y'` — координаты найденного пострадавшего. Спасатель стоит на земле и ждёт его, разведчик летит «змейкой» над площадкой 3×3 м; как только проходит рядом с пострадавшим, рассылает рою координаты и садится. Запускают сначала спасателя, затем разведчика.

python-onboard/swarm/swarm_09_rescue_rescuer.py

```python
# Поиск и спасение, часть 2 из 2: СПАСАТЕЛЬ — запускают на ВТОРОМ дроне,
↪  раньше разведчика (swarm_09_rescue_searcher.py).
# Стоит на земле и ждёт сообщение «НАЙДЕН x y»; получив, взлетает, летит к
↪  точке, зажигает зелёный свет и снижается.
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
print('Спасатель', swarm.id, 'ждёт сигнал')

target = None
waited = 0.0
while target is None and waited < 60.0:
    message = swarm.get_next_message()
    if message is None:
        time.sleep(0.1)
        waited += 0.1
    else:
        sender, text = message
        parts = text.split()
        if parts[0] == 'НАЙДЕН' and len(parts) == 3:
            target = (float(parts[1]), float(parts[2]))
            print('Сигнал от', sender, ': лечу в точку', target)
```

```
if target is not None:
    pioneer.arm()
    pioneer.takeoff()
    pioneer.go_to_local_point(x=target[0], y=target[1], z=1.0)
    while not pioneer.point_reached():
        time.sleep(0.1)
    pioneer.led_control(255, 0, 1, 0)
    print('Спасатель на месте')
    time.sleep(2)
    pioneer.land()
else:
    print('Сигнала не было')
swarm.close()
```

python-onboard/swarm/swarm_09_rescue_searcher.py

```
# Поиск и спасение, часть 1 из 2: РАЗВЕДЧИК — запускают на ПЕРВОМ дроне. Он
↪  летит «змейкой» над площадкой 3 x 3 м;
# когда проходит рядом с пострадавшим (точка VICTIM), рассылает рою
↪  координаты. Спасатель — swarm_09_rescue_rescuer.py
# на втором дроне (запустите его раньше). Камеры в примере нет: «видит»
↪  разведчик по расстоянию до известной точки.
import math
from pioneer_sdk2 import Pioneer
from pioneer_swarm import PioneerSwarmUnit
import time

VICTIM = (2.0, 2.0)   # где лежит пострадавший (разведчик этого не знает
↪  заранее — так проще проверить)
SEEN_RADIUS = 0.8     # с какого расстояния разведчик замечает его, м

pioneer = Pioneer()
swarm = PioneerSwarmUnit()
pioneer.arm()
pioneer.takeoff()

points = [(0, 0), (3, 0), (3, 1), (0, 1), (0, 2), (3, 2), (3, 3), (0, 3)]
found = None
for x, y in points:
    pioneer.go_to_local_point(x=x, y=y, z=1.5)
    while not pioneer.point_reached():
        px, py, pz = pioneer.get_local_position_lps()
        if math.hypot(px - VICTIM[0], py - VICTIM[1]) < SEEN_RADIUS:
            found = (px, py)
            break
        time.sleep(0.1)
    if found is not None:
        break

if found is not None:
    print('Пострадавший найден около', round(found[0], 1), round(found[1],
    ↪  1))
    for unit_id in swarm.get_unit_list():
        swarm.send_message_to_unit(unit_id, 'НАЙДЕН %.2f %.2f' % found)
    time.sleep(8)
```

```
else:
    print('Площадка осмотрена, пострадавшего нет')
pioneer.land()
swarm.close()
```

Что происходит (спасатель). Минуту ждёт письмо (опрос ящика с паузой, как раньше). Разбор строго проверяет формат: `text.split()` должно дать три части, первая — `'НАЙДЕН'`, иначе сообщение игнорируется. Две оставшиеся части преобразуются в `float`: это цель. Дальше — обычный полёт: `arm`, `takeoff`, точка цели с ожиданием `point_reached()`, зелёный светодиод, пауза и посадка. Если сигнала не было, спасатель так и стоит на земле и закрывает соединение.

Что происходит (разведчик). Маршрут «змейка» — список из восьми точек. Пока дрон летит к очередной точке, цикл ожидания прибытия каждые 0.1 с измеряет расстояние до пострадавшего (`math.hypot`) по текущей позиции; если оно меньше `SEEN_RADIUS` (0.8 м), разведчик запоминает свою позицию как место находки и выходит из обоих циклов. В примере нет камеры: «видит» дрон по известной точке `VICTIM`, чтобы проверка была повторяемой. Найдя цель, он рассылает всем соседям `'НАЙДЕН x y'` и ждёт восемь секунд, пока спасатель долетит, затем садится.

> **Безопасность группы**
>
> Два дрона в одном небе — уже риск столкновения: в примере нет защиты от встречного движения, а спасатель летит в точку, которую назвал разведчик. Разводите дроны по высотам (разведчик 1.5 м, спасатель 1 м), не посылайте их через одну точку одновременно и не запускайте группу рядом с людьми. Протокол без подтверждений может потерять сообщение: в реальном проекте спасатель отвечает «принял», а разведчик повторяет сигнал, пока не получит ответ.

ЗАДАНИЕ 20.11 · Найти и вызвать

Мини-проект. Доработайте протокол: спасатель отвечает «принял x y»; разведчик повторяет «НАЙДЕН» раз в 2 секунды до подтверждения (не больше пяти раз) и печатает результат. Добавьте третьего дрона, который слушает рассылку и зажигает синий свет как «наблюдатель».

Что должно получиться: при подтверждении разведчик печатает «Спасатель принял»; без спасателя — «Подтверждения нет» после пяти попыток и посадка; наблюдатель включает синий свет при первом сообщении.

В симуляторе: миссии Мини 2 SDK2: 64 в панели «**Обучение**».

Итоговая миссия трека. Миссия 64 — звено командира и ведомого: «Звено готово: N из N» печатается до `arm`, затем командир летит маршрут $(1, 0, 1) \to (1, 1, 1) \to (0, 1, 1)$,

а ведомый держится позади; диод 0 командира — жёлтый, зелёный, выключен. Основа — примеры о синхронном взлёте и следовании. Вы можете собрать такую связку из программ этой главы.

20.11 Проверьте себя

1. Чем рой Swarm Kit отличается от схемы «все дроны подчиняются наземной станции»?
   (a) ничем
   (b) юниты равноправны, центрального сервера нет
   (c) в рое есть главный юнит
   Ответ: (b). Все юниты подключены к одной сети и общаются напрямую.
2. Что такое Heartbeat?
   (a) периодический сигнал «я жив» с типом юнита и каталогом данных
   (b) письмо в ящик
   (c) поток телеметрии
   Ответ: (a). Из Heartbeat узнают, кто в сети и на что можно подписаться.
3. Что вернёт `get_unit_list()`, если рядом работает один дрон кроме вашего?
   (a) два идентификатора
   (b) список из одного идентификатора соседа
   (c) пустой список
   Ответ: (b). Свой `machine_id` в список не входит.
4. `send_message_to_unit('pioneer9', 'привет')` вернул `False`. Почему?
   (a) юнита `pioneer9` нет в списке соседей
   (b) ящик переполнен
   (c) сообщение слишком длинное
   Ответ: (a). `True` означает «адресат найден, сообщение ушло», но не «прочитано».
5. Что вернёт `get_next_message()`, если ящик пуст?
   (a) подождёт и вернёт сообщение
   (b) `None`
   (c) пустую строку
   Ответ: (b). Метод не ждёт; ждать нужно опросом с паузой и пределом.
6. Почему цикл ожидания сообщения должен иметь паузу и предел?
   (a) пауза не нагружает процессор, предел спасает от вечного ожидания
   (b) иначе сообщения теряются
   (c) так требует `swarm-server`
   Ответ: (a).
7. Что случится при `get_telemetry_from_unit(id, timeout_sec=0)`?
   (a) подписка и возврат накопленного, возможно пустого словаря
   (b) ожидание без предела
   (c) исключение

Ответ: (a). `None` вместо числа — ожидание всех потоков без ограничения.

8. Ведомый использует локальные координаты ведущего. При каком условии это верно?

   (a) всегда

   (b) у дронов общая система координат

   (c) когда дроны одинаковые

   Ответ: (b). Иначе ведомый получит чужие координаты в своей системе.

9. Почему команду «взлёт» посылают только после ответов «готов» от всех?

   (a) иначе часть группы не взлетит или взлетит не вовремя

   (b) так быстрее

   (c) это требует библиотека

   Ответ: (a). Подтверждения превращают рассылку в согласованное действие; отказ любого участника отменяет взлёт.

10. Какие ограничения у письма в симуляторе, о которых нельзя забывать в настоящей сети?

    (a) доставка мгновенная и без потерь только в симуляторе

    (b) письма шифруются

    (c) ящик бесконечен

    Ответ: (a). В симуляторе нет задержек и потерь; ящик ограничен 100 сообщениями; подтверждений доставки нет.

20.12 Задания к главе

ЗАДАНИЕ 20.12 · Перекличка перед полётом

Командир перед взлётом проверяет состав: если число соседей меньше ожидаемого, печатает, кого не хватает, и не взлетает. Затем рассылает «готовность» и ждёт ответы с пределом 15 секунд.

Что должно получиться: при полном составе все отвечают и командир взлетает; при нехватке дрона печатается его идентификатор, и моторы не запускаются.

В симуляторе: миссии Мини 2 SDK2: 57, 63 в панели «**Обучение**».

ЗАДАНИЕ 20.13 · Цепочка ведомых

Мини-проект. Три дрона летят цепочкой: второй следует за первым, третий — за вторым на метр позади. Для этого каждый читает телеметрию своего предшественника, а смещения и номера не зашиты в код, а приходят командой от первого.

Что должно получиться: в 3D три дрона держатся друг за другом без столкновений; при пропаже предшественника ведомый зависает и садится.

20.13 Миссии симулятора по разделам главы

Главы 18–19 трека «**Мини 2 · SDK2**»:

№	Тема миссий	Раздел главы
57	Кто в рое (свой `machine_id`, соседи)	кто в сети
58	Сообщение соседу, доставка и отказ	сообщения
59	Ping-pong с соседом	сообщения: ping-pong
60	Рассылка всем соседям	рассылка
61	Телеметрия соседа (высота, состояние)	телеметрия соседей
62	Лететь за ведущим	ведущий и ведомый
63	Взлёт по готовности	синхронный взлёт
64	Итог: звено командира и ведомого	итоговый проект

Итоги

- Рой Swarm Kit одноранговый: юниты равноправны, у каждого есть `machine_id`; центрального сервера нет.
- Три способа общения: Heartbeat («кто в сети» и каталог данных), телеметрия по подписке, сообщения в почтовый ящик юнита.
- `PioneerSwarmUnit()` подключает программу; `get_unit_list()` — соседи без себя; `close()` — в конце; свойства `.id` и `.unit_type`.
- Сообщение — строка; `send_message_to_unit` возвращает `True` или `False` (юнита нет); `get_next_message()` не ждёт — опрос с паузой и пределом.
- `get_telemetry_from_unit(id, timeout_sec)` возвращает `(тип, словарь)` или `None`; словарь может быть неполным; значения — сообщения ROS 2.
- Следование и строй берут позицию соседа из телеметрии; это верно только при общей системе координат, а защиты от столкновений нет.
- Групповые действия строят на подтверждениях: «готовность» → «готов» → «взлёт»; протокол из строк разбирают с проверкой формата.
- Все программы проверены на симуляторе с мгновенной сетью; на настоящих дронах авторы их не запускали, надёжность доставки и порядок сообщений в настоящей сети не гарантированы.

ЧАСТЬ VI

Приложения

ГЛАВА A

Справочник Lua API Пионера

Краткий справочник функций, которыми пользуется книга, и тех, что пригодятся сверх неё. Полное описание с примерами — в справочнике API симулятора (значок книги в заголовке редактора) и в документации Геоскан [10]. Справочник относится к Пионеру Базовому: у Пионера Мини 2 Lua нет, его методы `pioneer_sdk2` описаны в конце главы 16, соответствие команд — в шпаргалке (раздел B).

A.1 Команды автопилота: `ap`

`ap.push(Ev.MCE_PREFLIGHT)`	предстартовая подготовка и запуск моторов; ответ — `Ev.ENGINES_STARTED`
`ap.push(Ev.MCE_TAKEOFF)`	взлёт на высоту из параметра `Flight_com_takeoffAlt`; ответ — `Ev.TAKEOFF_COMPLETE`
`ap.push(Ev.MCE_LANDING)`	посадка; ответ — `Ev.COPTER_LANDED`
`ap.push(Ev.ENGINES_ARM)`	запуск моторов без взлёта (проверка)
`ap.push(Ev.ENGINES_DISARM)`	немедленное выключение моторов
`ap.goToLocalPoint(x, y, z [, t])`	полёт в точку (метры); необязательное `t` — за сколько секунд долететь, без него — со скоростью `Copter_pos_vMax`; ответ — `Ev.POINT_REACHED`
`ap.goToPoint(lat, lon, alt)`	полёт в точку GPS: широта и долгота, умноженные на 10^7, высота в метрах (для полётов на улице)
`ap.updateYaw(rad)`	курс в радианах; `math.rad(deg)` для перевода

A.2 События: `Ev` в `callback(event)`

`Ev.ENGINES_STARTED`	моторы запущены после предстарта
`Ev.TAKEOFF_COMPLETE`	взлёт завершён
`Ev.POINT_REACHED`	точка `goToLocalPoint` достигнута
`Ev.POINT_DECELERATION`	дрон начал торможение перед точкой
`Ev.COPTER_LANDED`	дрон сел
`Ev.LOW_VOLTAGE1 / LOW_VOLTAGE2`	батарея разряжена: предупреждение / пора садиться
`Ev.SYNC_START`	синхронный старт группы дронов
`Ev.SHOCK`	удар, столкновение
`Ev.CONTROL_FAIL`	потеря управления
`Ev.ENGINE_FAIL`	отказ мотора

Константы `MCE_*` и `ENGINES_ARM/DISARM` — это *команды*: их отправляют автопилоту через `ap.push`. Остальные — *события*: их присылает автопилот, и программа сравнивает с ними `event` в `callback`.

A.3 Время и таймеры

- `Timer.callLater(delay, fn)` — однократный вызов через `delay` секунд.
- `t = Timer.new(period, fn)` — периодический таймер; `t:start()`, `t:stop()`.
- `Timer.callAt(t, fn)`, `Timer.callAtGlobal(t, fn)` — вызов в заданный момент локального или глобального (синхронного для группы) времени.
- `time()` — секунды с момента включения (в симуляторе — модельное время с запуска), для измерений.
- `launchTime()` — секунды с момента взлёта; `deltaTime()` — время с предыдущего шага.
- `sleep(t)` — блокирующая пауза; в полётных программах не используется.
- `boardNumber()` — номер платы (бортовой номер дрона в группе).

A.4 Светодиоды: `Ledbar`

- `leds = Ledbar.new(29)` — линейка: 4 светодиода платы (`0..3`) и 25 модуля матрицы (`4..28`).
- `leds:set(i, r, g, b)` — цвет светодиода `i`, компоненты `0..1`; необязательный пятый аргумент `w` — белый канал.
- `Ledbar.fromHSV(h, s, v)` (или просто `fromHSV`) — перевод оттенка (0–360°), насыщенности и яркости (0–100) в `r, g, b` от 0 до 1.

A.5 Датчики: Sensors

`local x, y, z = Sensors.lpsPosition()`	положение в локальной системе, м
`local vx, vy, vz = Sensors.lpsVelocity()`	скорость, м/с
`local yaw = Sensors.lpsYaw()`	курс по системе позиционирования, рад
`local roll, pitch, yaw = Sensors.orientation()`	углы ориентации, рад
`local ax, ay, az = Sensors.accel()`	ускорения, м/с^2
`local gx, gy, gz = Sensors.gyro()`	угловые скорости, рад/с
`local d = Sensors.range()`	дальномер: расстояние до поверхности под дроном (земля, крыша), м
`local mm = Sensors.tof()`	то же по TOF-датчику, *миллиметры*
`local fx, fy, q = Sensors.opticalFlow()`	оптический поток: скорость «ухода» поверхности под дроном, рад/с (x — вперёд, y — вправо), качество 0–255; `fx * Sensors.range()` — скорость, м/с
`local h = Sensors.altitude()`	высота по барометру, м
`local v = Sensors.battery()`	напряжение батареи, В
`local c1, ..., c8 = Sensors.rc()`	восемь каналов пульта, от -1 до 1 (без пульта — нули); `table.pack(Sensors.rc())` — в таблицу

A.6 Камера: camera

- `camera.requestMakeShot()` — снимок с камеры; управление возвращается сразу. В симуляторе снимок скачивается файлом PNG.
- `camera.checkRequestShot()` — ответ на запрос снимка: -1 — ещё готовится, 0 — готово, 1 — ошибка.
- `camera.requestRecordStart()`, `camera.requestRecordStop()` — начать и остановить запись видео; `camera.checkRequestRecord()` — ответ, коды те же.

Распознавание меток в Lua не выполняется: камера, ArUco и наведение изучаются на Python.

A.7 Прочее

- `Gpio`, `Uart`, `Spi` — выводы и шины платы для своих датчиков и модулей (`Gpio.new`, `Uart.new`, `uart:read`, `uart:write` и др.).
- `mailbox` — обмен сообщениями между дронами группы: `mailbox.connect`, `mailbox.send(to, data)`, `mailbox.receive()`, `mailbox.hasMessages()`.
- `Vehicle.start(name)`, `Vehicle.stop(name)`, `Vehicle.setSpeed(name, v)`, `Vehicle.state(name)` — *только в симуляторе*: управление машиной или поездом сцены для миссий слежения.

A.8 Идиомы курса

- Распаковка: `local unpack = table.unpack`; `leds:set(i, unpack(color))`; `ap.goToLocalPoint(unpack(points[i]))`.
- «Тернарный»: `local v = cond and A or B`.
- Счётчик шагов: `nextPoint()` отправляет `points[curr]` и увеличивает `curr`; вызывается из `TAKEOFF_COMPLETE` и `POINT_REACHED`.
- Конечный автомат: переменная `state` в начале файла (глобальная или `local`) и проверка `event == Ev.X and state == "Y"`; таймеры этапа проверяют `state` первой строкой.

ГЛАВА В

Шпаргалка: Lua, Python и pioneer_sdk2

Одно и то же действие в трёх вариантах. Lua (Пионер Базовый) ждёт событий в `callback`; Python с `pioneer_sdk` (Базовый) опрашивает автопилот в цикле; `pioneer_sdk2` (Пионер Мини 2) в основном *блокирует* вызов до завершения команды. У Мини 2 Lua нет. Подробности по Мини 2 — в справочнике главы 16.

Действие	Lua (Базовый)	Python, `pioneer_sdk` (Базовый)	Python, `pioneer_sdk2` (Мини 2)
Подключение	— (программа на борту)	`Pioneer()`; адрес дрона или сайта в `ip=`	`Pioneer()`; на борту адрес по умолчанию верный
Моторы	`ap.push(Ev.MCE_PREFLIGHT)`	`pioneer.arm()`	`pioneer.arm()`, ждёт `ENGINES_STARTED`, возвращает `bool`
Взлёт	`ap.push(Ev.MCE_TAKEOFF)`	`pioneer.takeoff()`	`pioneer.takeoff()`, ждёт `TAKEOFF_COMPLETE`
Точка	`ap.goToLocalPoint(x, y, z)`	`pioneer.go_to_local_point(x=, y=, z=, yaw=)`	`pioneer.go_to_local_point(x, y, z, yaw=None)`; не ждёт
Ждать точку	`if event == Ev.POINT_REACHED`	`while not pioneer.point_reached(): time.sleep(0.1)`	то же опросом `point_reached()` или подпиской на `Event.POINT_REACHED`
Курс	`ap.updateYaw(рад)`	`yaw=` (рад); текущий — `get_yaw()`	`yaw=` в **градусах**; `set_yaw(град)`; текущий — `get_local_yaw_lps()`
Скорость	— (только точки)	`set_manual_speed(vx, vy, vz, yaw_rate)`, повторять каждые 0,1 с	`set_manual_speed(vx, vy, vz, yaw_rate, interval=1.0)`; `interval` задаёт длительность; в осях дрона — `set_manual_speed_body_fixed`
Посадка	`ap.push(Ev.MCE_LANDING)`	`pioneer.land()`	`pioneer.land()`, ждёт `COPTER_LANDED`; домой — `rtl()`
Выключить моторы	`ap.push(Ev.ENGINES_DISARM)`	`pioneer.disarm()`	`pioneer.disarm()` (после посадки!)
Пауза	`Timer.callLater(t, f)`	`time.sleep(t)`	`time.sleep(t)`
Периодически	`Timer.new(p, f):start()`	цикл `while` с `time.sleep(p)`	то же
События	`event` в `callback`	`get_autopilot_state()`	`subscribe(callback, Event.X)`; `get_fly_state()`
Светодиоды	`leds:set(i, r, g, b)`, 0…1	`led_control(led_id, r, g, b)`, 0…255	`led_control(led_id=255, r, g, b)`, **0…1**; два светодиода
Позиция	`Sensors.lpsPosition()`	`get_local_position_lps()`	`get_local_position_lps()`
Дальномер	`Sensors.range()`	`get_dist_sensor_data()`	`get_dist_sensor_data()`, `get_ranger_data()`
Батарея	`Sensors.battery()`	`get_battery_status()`	`get_battery_status()`: (В, температура)
Груз	—	`cargo_grab()`/ `cargo_release()` (магнит)	`grab_close()`/`grab_open()` (механический захват)
Камера	`camera.requestMakeShot()` (снимок)	`Camera().get_cv_frame()`; нижняя камера симулятора — `Camera(view="bottom")`	`Camera(CameraType.MAIN или OPT).get_cv_frame()`; показ — `ImageViewer().imshow()`
Метки ArUco	—	`cv2.aruco.ArucoDetector(словарь).detectMarkers(frame)`	то же, кадры из `pioneer_sdk2`
Нейросети	—	—	`pioneer_rknn`: `Yolo`, `YoloPose`, `PaddleOCR`
Блоки	Blockly	Blockly	Pioneer Bricks (генерирует Python)
Вывод	`print(...)`	`print(...)`	`print(...)`
Завершение	—	`pioneer.close_connection()`	`pioneer.close_connection()` в `finally`

Три главные мысли

- Дрон — автопилот: вы отдаёте команду и ждёте события, что она выполнена.
- В Lua ждут через `callback(event)`, в `pioneer_sdk` — опросом `while not ...`, в `pioneer_sdk2` `arm`, `takeoff` и `land` ждут сами.
- Координаты — метры от старта, Z вверх; их показывает `Ctrl`+щелчок в симуляторе. Единицы различаются: курс в радианах у Lua и `pioneer_sdk`, в градусах у `pioneer_sdk2`; цвет 0…1 у Lua и `pioneer_sdk2`, 0…255 у `pioneer_sdk`.

Не смешивайте библиотеки

Программа для Мини 2 не запустится на Базовом и наоборот: симулятор остановит импорт чужой библиотеки с подсказкой, а на настоящем дроне получите ошибку. Переносить программу между моделями нужно осознанно: меняются импорт, единицы курса и цвета, порядок ожидания команд.

pioneer_sdk2, ROS 2 (rclpy) и pioneer_swarm

Для Мини 2 (и Базового с Radxa) те же действия можно выполнить через ROS 2 (глава 19); общение с другими дронами — через `pioneer_swarm` (глава 20). Все имена ROS 2 — с префиксом `/geoscan/`.

Действие	pioneer_sdk2	ROS 2 (rclpy) и pioneer_swarm
Узел, соединение	`Pioneer()`	`rclpy.init()`, `Node('имя')`; в конце `destroy_node()`, `rclpy.shutdown()`
Моторы	`arm(timeout, retries)`	сервис `flight/arm`, `Arm.Request(timeout=5.0, retries=0)`, ответ `success`
Взлёт, посадка, выключение	`takeoff()`, `land()`, `disarm()`	сервисы `flight/takeoff`, `land`, `disarm` (`Trigger`); окончание — по `flight/fly_state`
Точка	`go_to_local_point(x, y, z, yaw)`	`flight/go_to_local_point`, `frame_id`: `'map'` или `'base_link'`; x вперёд, y влево
Курс	градусы (единицы `go_to_*` проверить)	`yaw` и `set_yaw`: градусы; `cmd_vel.angular.z`: рад/с
Скорость	`set_manual_speed(vx, vy, vz, yaw_rate, interval)`	публикация `TwistStamped` в `flight/cmd_vel` таймером; пределы ± 1.82 рад/с
Ждать точку	`point_reached()`, `subscribe`	топик `flight/callback_event` (5 — точка, 7 — взлёт, 2 — посадка)
Позиция, батарея	`get_local_position_lps()`, `get_battery_status()`	подписки `navigation/local/position`, `sensors/battery_state`
Параметры автопилота	`get_param`, `set_param`	сервисы `board/get_param`, `set_param`
Светодиоды (0..1)	`led_control(led_id, r, g, b)`	сервис `payload/led/set`: `PIXEL`, `FILL`
Захват	`grab_close()`, `grab_open()`	сервис `payload/grab` (`action` 0, 1, 2)
Соседи по рою	—	`PioneerSwarmUnit().get_unit_list()`
Сообщение соседу	—	`send_message_to_unit(id, text)` → `bool`; чтение — `get_next_message()` (`None`, если пусто)
Телеметрия соседа	—	`get_telemetry_from_unit(id, timeout_sec)` → `(тип, словарь)`; ключи `'navigation/local/position'`
Завершение роя	—	`swarm.close()`

ГЛАВА C

Протокол Plazlink и поля автопилота

Автопилот Пионера Мини 2 общается с бортовым компьютером по протоколу Plazlink: пакеты с номером сообщения, длиной, данными и контрольной суммой (разбор пакета идёт по этапам SYNC, ID, LENGTH, PAYLOAD, CHECKSUM). Данные автопилота устроены как дерево: *компонент* (датчик, подсистема) содержит *поля* (значения), а поле имеет номер, тип, флаги, масштаб и границы. Библиотека `pioneer_sdk2` прячет это за методами: например, `get_battery_status()` читает поля `SensorMonitor.voltage` (или `CBoard.VoltBatt`) и `Battery.temperature`. Ниже — сведения, снятые с работающего борта (11 компонентов, 178 полей, 232 параметра). Значения этих таблиц нужны, если вы читаете поля напрямую или сверяете симулятор с настоящим дроном.

Компоненты автопилота

№	Компонент	Полей	Примеры полей
0	`Battery`	8	`charge`, `current`, `voltage`, `temperature`, `serialBattery`, `serialNumber`
1	`Actuators`	6	`packetErrors`, `rpmMotor0`, `rpmMotor1`, `rpmMotor2`, `rpmMotor3`
2	`SmartBoard`	9	`command`, `rkSwitch`, `rkMaskRom`, `apSwitch`, `rcChan`, `rcServo`
3	`SensorMonitor`	45	`magEnabled`, `magDeclination`, `altitudeOffset`, `magX`, `magY`, `magZ`
4	`EnvMonitor`	4	`temperature`, `humidity`, `windAzimuth`, `windSpeed`
5	`FlightManager`	24	`maxCommands`, `maxIndices`, `maxPoints`, `missionPoint`, `missionTime`, `onMissionHalted`
6	`ManualControl`	22	`mode`, `interval`, `altitude`, `speed`, `heading`, `roll`
7	`UavMonitor`	35	`time`, `uptime`, `roll`, `pitch`, `yaw`, `altitude`
8	`CanMaster`	12	`poolCapacity`, `poolCurrent`, `poolPeak`, `transfersRx`, `transfersTx`, `transfersDropped`
9	`LpsMonitor`	9	`capabilities`, `status`, `x`, `y`, `z`, `velX`
10	`OptCamera`	4	`enabled`, `restart`, `uptime`
	Всего	**178**	

Какое поле читает метод SDK

Таблица связывает группу полей SDK (то, что библиотека обозначает как `messenger['группа']['поле']`) с компонентом автопилота. Если компонентов два, библиотека берёт тот, который есть на борту: так одни и те же методы работают на разных платах.

Группа	Поля и компонент автопилота
`board`	`state`, `time`, `uptime`, `altitude`, `roll`, `pitch`, `yaw` — `UavMonitor`
`accel`	`x`, `y`, `z` — `SensorMonitor`
`gyro`	`x`, `y`, `z` — `SensorMonitor`
`mag`	`x`, `y`, `z` — `SensorMonitor`
`optFlow`	`x`, `y` — `SensorMonitor`
`sensor`	`range`, `status` — `SensorMonitor`
`battery`	`voltage`, `temperature`, `charge` — `SensorMonitor / CBoard`; `Battery`
`rc`	`channel`, `flaps` — `SmartBoard / ManualControl`; `ManualControl`
`gnss`	`latitude`, `longitude`, `altitude`, `velNorth`, `velEast`, `velDown`, `satGps`, `satGlonass`, `status` — `Ublox / Allystar`
`lps`	`x`, `y`, `z`, `velX`, `velY`, `velZ`, `yaw`, `status` — `LpsMonitor / USNav_module`
`grab`	`state`, `velocity`, `time` — `GrabModule`
`flight`	`missionTime`, `event` — `FlightManager`
`manual`	`interval`, `velX`, `velY`, `velZ`, `yawRate`, `mode`, `yaw` — `ManualControl`
`led`	`r`, `g`, `b` — `LedModule`
`ledBar`	`color`, `cargo` — `LedBar`
`ranger`	`right`, `left`, `front`, `back`, `aboveBelow` — `RangerModule`
`actuators`	`rpmMotor0`, `rpmMotor1`, `rpmMotor2`, `rpmMotor3` — `Actuators`

Команды состояния и ручное управление

Набор	Код	Значение
`BoardState`	0	`DISARM` — остановить моторы
	1	`ARM` — запустить моторы
	2	`TAKEOFF` — взлёт
	3	`RTL` — возврат домой
	4	`LAND` — посадка
`ManualSpeedMode`	0	`GLOBAL` — скорость в осях сцены
	1	`BODY` — скорость в осях корпуса

Методы `arm()`, `disarm()`, `takeoff()`, `land()` и `rtl()` записывают эти коды в поле `UavMonitor.mode`, а `set_manual_speed()` и `set_manual_speed_body_fixed()` выбирают режим `GLOBAL` или `BODY`.

Системные команды

Системные команды автопилота запускают калибровки и обслуживание. Из SDK 2 доступна только перезагрузка (`reboot_board()`, код 18, и только если плата её разрешает); остальные команды вызывают станции и сервисные программы. Не запускайте их на дроне, назначение которых вы не знаете.

Код	Команда
1	начать калибровку барометра (`Start baro calibration`)
3	начать калибровку гироскопа (`Start gyro calibration`)
4	начать калибровку магнитометра (`Start mag calibration`)
5	закончить калибровку магнитометра (`Stop mag calibration`)
6	начать калибровку акселерометра (`Start accel calibration`)
7	закончить калибровку акселерометра (`Stop accel calibration`)
8	снять точку калибровки акселерометра (`Make accel sample`)
9	сбросить выравнивание платы (`Reset board alignment`)
10	выровнять плату (`Align board`)
11	начать проверку магнитометра (`Start mag check`)
12	закончить проверку магнитометра (`Stop mag check`)
13	пересчитать магнитное склонение (`Recalc mag declination`)
14	форматировать память автопилота (`Format memory`)
15	начать проверку гироскопа (`Start gyro check`)
18	перезагрузка автопилота (`Restart`)
19	перезагрузка в загрузчик (`Restart to bootloader`)
20	самотест инерциального датчика (`Run IMU self-test`)

Коды ответов протокола

Код	Имя	Значение
-3	`INCORRECT_PARAMETER`	неверный параметр
-2	`CHECKSUM_ERROR`	ошибка контрольной суммы
-1	`GENERIC_TIMEOUT`	общий тайм-аут
0	`SUCCESS`	успех
1	`COMPONENT_NOT_FOUND`	нет такого компонента
2	`FIELD_NOT_FOUND`	нет такого поля
3	`FIELD_TYPE_MISMATCH`	тип значения не подходит полю
4	`FIELD_RANGE_ERROR`	значение вне границ поля
5	`FIELD_ERROR`	ошибка поля
6	`FIELD_READ_ONLY`	поле только для чтения
7	`FILE_NOT_FOUND`	нет такого файла
8	`FILE_ERROR`	ошибка файла
9	`FIELD_UNAVAILABLE`	поле сейчас недоступно
10	`FIELD_TIMEOUT`	поле не ответило вовремя
11	`FILE_ACCESS_ERROR`	нет доступа к файлу
12	`FILE_POSITION_ERROR`	неверная позиция в файле
13	`FILE_TIMEOUT`	файл не ответил вовремя
14	`QUEUE_ERROR`	очередь команд переполнена
15	`COMMAND_UNSUPPORTED`	команда не поддерживается
16	`COMMAND_QUEUED`	команда поставлена в очередь
17	`COMMAND_ERROR`	ошибка выполнения команды
18	`HWID_MISMATCH`	не тот идентификатор платы
19	`LICENSE_EXPIRED`	лицензия истекла
20	`SIGNATURE_ERROR`	неверная подпись

Типы значений, флаги и номера сообщений

Типы полей: 0 — `bool`, 1 — `char`, 2 — `int8`, 3 — `int16`, 4 — `int32`, 5 — `int64`, 6 — `uint8`, 7 — `uint16`, 8 — `uint32`, 9 — `uint64`, 10 — `float`, 11 — `double`. Флаги поля складываются: 1 — читается, 2 — записывается, 4 — важное. Здоровье компонента: 0 — норма, 1 — предупреждение, 2 — ошибка, 3 — критическое состояние. Режим компонента: 0 — рабочий, 1 — инициализация, 2 — обслуживание, 3 — обновление программы, 7 — не в сети.

Номера сообщений протокола: `REQ_PARAM` 0, `PARAM_INFO` 1, `PARAM` 2, `PROTOCOL_REQUEST` 3, `PROTOCOL_INFO` 4, `COMPONENT_COUNT_REQUEST` 5, `COMPONENT_COUNT` 6, `COMPONENT_INFO_REQUEST` 7, `COMPONENT_INFO` 8, `COMPONENT_INFO_ERROR` 9, `COMPONENT_FIELD_REQUEST` 10, `COMPONENT_FIELD` 11, `COMPONENT_FIELD_RESPONSE` 12, `COMPONENT_FIELD_DESCRIBE` 13,

COMPONENT_FIELD_INFO 14, COMPONENT_FIELD_ERROR 15, SYSTEM_COMMAND 16, SYSTEM_COMMAND_RESPONSE 17, COMPONENT_MESSAGE 18, COMPONENT_FILE_DESCRIBE 19, COMPONENT_FILE_INFO 20, COMPONENT_FILE_INFO_ERROR 21, COMPONENT_FILE_WRITE 22, COMPONENT_FILE_WRITE_RESPONSE 23, COMPONENT_FILE_READ 24, COMPONENT_FILE_READ_RESPONSE 25, COMPONENT_FILE_READ_ERROR 26, MEDIA_PASSPORT 27, COMPONENT_RAW_DATA 28, COMPONENT_INFO_EXT 29, COMPONENT_FIELD_INFO_EXT 30, LICENSE_INFO_REQUEST 31, LICENSE_INFO 32, GO_TO_POINT_V2 33, GO_TO_RELATIVE_POINT_V2 34, GO_TO_POINT_RESPONSE 35, GOTO_LOCAL_POINT 36, MANUAL_CONTROL 37, GO_TO_LOCAL_POINT_V2 38.

Типы аппаратов

Автопилот сообщает тип аппарата числом (поле `Board_type`). У Мини 2 оно равно 23, а в списке плат протокола стоят 12 и 23. Названия типов в библиотеке: 0 — `GeoScan 101 3S`, 1 — `GeoScan 300`, 2 — `GeoScan 002`, 4 — `GeoScan 401`, 5 — `GeoScan 102`, 6 — `GeoScan 201 4S`, 7 — `GeoScan 301`, 8 — `GeoScan 425`, 9 — `GeoScan 501`, 10 — `GeoScan 101 4S`, 11 — `GeoScan 201 5S`, 12 — `GeoScan Pioneer`, 13 — `GeoScan 201M`, 14 — `GeoScan 501M`.

Параметры автопилота по группам

Автопилот Мини 2 хранит 232 параметров (`get_param`, `set_param`; значения сохраняются после перезагрузки). Имя состоит из группы и названия. Группы и число параметров в них:

Группа	Число	Группа	Число
Flight_com	30	Serial	4
Copter	24	Offsets_accel	3
Copter_pos	19	Offsets_gyro	3
Imu	17	Board	3
Copter_man	13	Sensors_pressure	3
Copter_xyRate	11	Copter_zRate	3
Copter_alt	10	Flight_mission	3
Copter_optFlow	8	State	3
Sensors_gyro	6	Offsets_board	2
Modules	6	Board_version	2
SensorMux	6	ICM40609_filter	2
Offsets_mag1	5	Sensors_accel	2
Offsets_mag2	5	Sensors_mag	2
Sensors_airspeed	5	Flight_chargeMonitor	2
Sensors_rates	5	SensorMux_pressure	2
BoardPioneerMiniV2_motor	5	Battery	2
Telemetry	5	Sensors_gnss	1
ICM40609	4	Sensors_ns	1
Copter_thrLimit	4	Flight_man	1

Значения часто используемых параметров на настоящем Мини 2: `Flight_com_takeoffAlt` = 0.5, `Flight_com_navSystem` = 2, `Flight_com_maxAltitude` = 4, `Copter_pos_vMax` = 1, `Copter_pos_vTakeoff` = 0.30000001192092896, `Copter_motorCheckTime` = 10, `Copter_flyWithoutRc` = 1, `Copter_man_rcMode0` = 1, `Copter_man_rcMode1` = 2, `Copter_man_rcMode2` = 4, `SensorMux_rc` = 0. Высота взлёта называется именно `Flight_com_takeoffAlt`; параметра `Copter_com_takeoffAlt` на борту нет.

ГЛАВА D

Аппаратура управления

В этой главе мы подробно рассмотрим устройство и настройку аппаратуры радиоуправления FlySky FS-i6S, которая входит в стандартный комплект образовательного квадрокоптера «Геоскан Пионер». Пульт дистанционного управления (аппаратура) — это основной инструмент внешнего пилота, позволяющий контролировать полет, переключать режимы работы и выполнять настройку параметров дрона на расстоянии.

Аппаратура работает в паре с приемником, установленным на борту квадрокоптера, передавая команды по радиоканалу. Для корректной работы системы необходимо правильно настроить пульт и установить связь с приемником.

D.1 Настройка пульта

Перед первым использованием или после сброса настроек необходимо сконфигурировать пульт управления. Это обеспечит правильную интерпретацию команд полетным контроллером.

Для настройки пульта выполните следующие действия шаг за шагом:

1. Вставьте батарейки (4 шт. типа AA) в батарейный отсек и включите пульт дистанционного управления. Для включения необходимо нажать одновременно две кнопки питания (расположены в центре лицевой панели) и удерживать их до включения экрана.
2. После загрузки на дисплее появится главный экран с отображением текущего состояния, напряжения батарей и положения стиков.

> **Подсказка**
>
> **Подсказка:** В целях безопасности пульт проверяет положение переключателей при включении. Если на экране отображается предупреждение «Warning!!! Place all switches in their up position», это означает, что один из тумблеров находится в активном состоянии или стик газа не в нуле. Приведите все переключатели в верхнее положение (от себя), а левый стик (газ) опустите в самое нижнее положение.



Рис. 52. Главный экран пульта управления FS-i6S

3. Изначально сенсорный экран заблокирован от случайных нажатий, о чём сигнализирует иконка замка в нижнем правом углу. Для разблокировки нажмите на иконку и удерживайте её в течении 2-4 секунд. После разблокировки иконка сменится на изображение гаечного ключа.
4. Нажмите на иконку с гаечным ключом, чтобы попасть в меню настроек. Меню разделено на две основные вкладки:
   - **FUNCTION** — здесь производится настройка функций самого пульта (каналы, реверсы и т.д.).
   - **SYSTEM** — отвечает за системные настройки связи и приемника.

D.1.1 Настройки вкладки FUNCTION

В этой вкладке мы настроим назначение каналов и направление их работы. Это критически важно для того, чтобы движения стиков и переключения тумблеров вызывали ожидаемую реакцию дрона.

Для работы с «Пионером» выполните следующие настройки:

1. **Настройка реверса каналов (Reverse):**
   - Перейдите в меню **REVERSE**.
   - Установите каналы **Ch2** (Тангаж) и **Ch4** (Рыскание) в положение **Rev** (Реверс). Остальные каналы оставьте в Norm.
2. **Настройка вспомогательных каналов (Aux. Channels):** Для управления режимами полета и дополнительными функциями необходимо назначить физические тумблеры на логические каналы пульта. Перейдите в меню **AUX. CHANNELS** и настройте каналы следующим образом:
   - **Channel 5:** Нажмите на кнопку выбора канала. В окне **CH TYPE** выберите **SWx** (переключатель). Затем нажмите на надпись SwA (по умолчанию) и в открывшемся списке выберите тумблер **SwC** (трехпозиционный переключатель справа). Этот канал обычно используется для смены режимов полета.
   - **Channel 6:** Аналогично настройте на тумблер **SwD** (крайний правый переключатель).
   - **Channel 7:** Настройте на тумблер **SwB** (трехпозиционный переключатель слева).
   - **Channel 8:** Настройте на тумблер **SwA** (крайний левый переключатель).



Рис. 53. Меню настройки вспомогательных каналов

D.1.2 Настройка вкладки SYSTEM

В этой вкладке задаются параметры протокола передачи данных и режим работы стиков.

1. **Выбор протокола выхода (Output Mode):**
   - Перейдите в меню **OUTPUT MODE**.
   - Убедитесь, что в поле **Output** выбран протокол **PPM**. Полетный контроллер «Пионера» настроен на прием сигналов именно в этом формате. Также должен быть выбран режим **S.BUS** в поле Serial, если используется соответствующее подключение, но для базовой настройки PPM является основным.
2. **Режим стиков (Sticks Mode):**
   - Перейдите в меню **STICKS MODE**.
   - Выберите режим **M2 (Mode 2)**. В этом режиме левый стик отвечает за Газ (вверх/вниз) и Рыскание (влево/вправо), а правый — за Тангаж (вперед/назад) и Крен (влево/вправо). Это наиболее распространенная схема управления для квадрокоптеров.

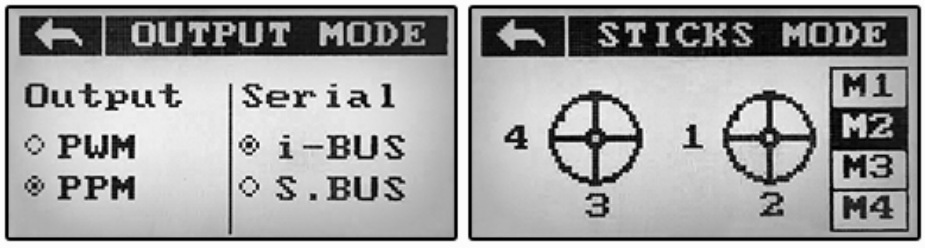


Рис. 54. Системные настройки: выбор режима PPM и Mode 2

D.2 Связь пульта с приемником

Процедура привязки (Binding) необходима для того, чтобы приемник, установленный на квадрокоптере, «узнал» ваш пульт и реагировал только на его команды. Это исключает пересечение сигналов при полетах нескольких дронов одновременно.

> **Внимание**
>
> **Внимание:** Для использования квадрокоптера «Пионер Мини» с пультом радиоуправления, в автопилот должны быть загружены соответствующие параметры! Без этого дрон может не реагировать на команды.

D.2.1 Процедура привязки

Выполните следующие действия для сопряжения устройств:

1. Убедитесь, что приемник (FS-A8S или аналогичный) подключен к плате автопилота.

2. Найдите на корпусе приемника маленькую кнопку с подписью «**BIND**». Нажмите её и **удерживайте**.
3. **Не отпуская** кнопку BIND, подайте питание на квадрокоптер (подключите аккумулятор).
4. После подачи питания светодиод на приемнике должен начать **быстро мигать**. Это означает, что приемник перешел в режим привязки. Теперь кнопку можно отпустить.
5. Включите пульт управления. Зайдите в меню настроек (**SYSTEM**) и выберите пункт **RX Bind**.
6. На экране пульта появится надпись **Binding to RX...**. В этот момент пульт ищет приемник в режиме привязки.
7. Как только связь установится, светодиод на приемнике начнет **мигать медленно**.
8. Для завершения процедуры нажмите иконку «Назад» (←) на пульте, чтобы выйти из меню RX Bind.
9. После выхода из режима привязки светодиод на приемнике должен загореться **постоянным светом** (красным). Это подтверждает успешное сопряжение и наличие стабильной связи.

> **Внимание**
>
> **Важно:** Чтобы избежать конфликтов управления, связывайте только один пульт с одним приемником. Если вы меняете пульт или приемник, процедуру привязки необходимо повторить.

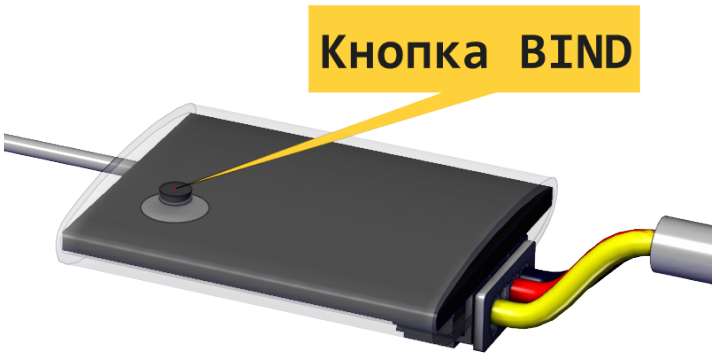


Рис. 55. Приемник FS-A8S: расположение кнопки BIND

D.2.2 Возможные проблемы и их решение

Если после выполнения процедуры квадрокоптер не реагирует на команды:

- **Проверьте протокол:** Зайдите в настройки пульта **OUTPUT MODE** и убедитесь, что выбран **PPM**. Другие протоколы (например, PWM или i-BUS) могут не восприниматься текущей прошивкой автопилота без дополнительной настройки.

- **Повторите привязку:** Возможно, вы отпустили кнопку BIND слишком рано или не успели включить режим привязки на пульте. Попробуйте снова.
- **Проверьте параметры:** Если вы летаете на Pioneer Mini, убедитесь через станцию наземного управления, что загружены корректные параметры для работы с RC-пультом.

В случае, если выполнить привязку не получается, вы всегда можете обратиться в техподдержку: support@geoscan.aero или в telegram-канал @geoscan_edu.

D.3 Стики и тумблеры

Пульт управления предоставляет пилоту гибкие возможности контроля. Основные органы управления — это два джойстика (стика) и набор переключателей (тумблеров).

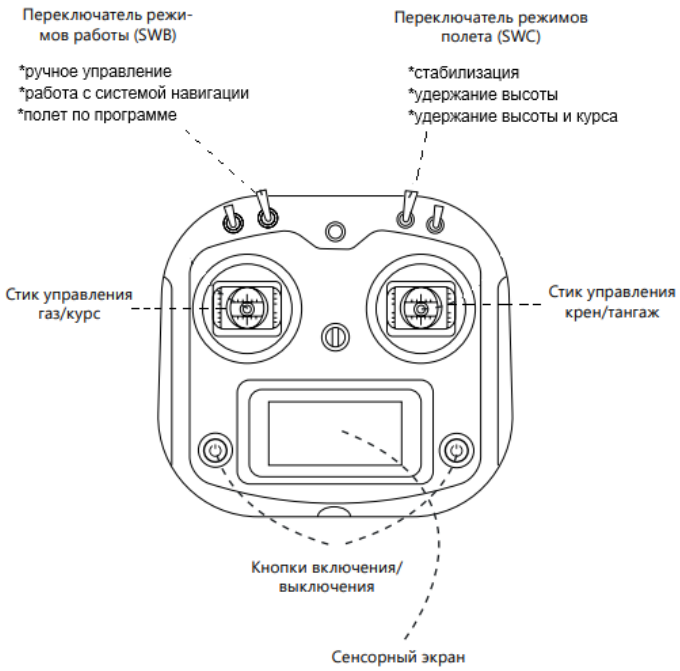


Рис. 56. Обзор органов управления пульта FS-i6S

D.3.1 Режимы управления (Тумблеры)

Тумблеры используются для переключения полетных режимов и активации функций. Ранее мы настроили тумблер **SwC** (трехпозиционный справа) на 5-й канал.

Изменение режима полета (переключатель SwC):

Тумблер выдаёт по 5-му каналу три значения: около 1000, 1500 и 2000 мкс. Им соответствуют три режима автопилота Пионера:

- **Около 1000 (Navigation):** Режим удержания позиции. Квадрокоптер сам держит и высоту, и точку в пространстве по системе позиционирования (LPS, оптический поток или GPS). Отпущенные стики — дрон висит на месте. В этом режиме выполняются и программы.
- **Около 1500 (AltHold):** Режим удержания высоты. Квадрокоптер использует барометр для автоматического поддержания заданной высоты, а по горизонтали его может сносить. Стик газа в центре — высота не меняется. Стик вверх — набор высоты, вниз — снижение. Это рекомендуемый режим для обучения ручному пилотированию.
- **Около 2000 (Stabilize):** Режим стабилизации. Стик газа напрямую управляет мощностью моторов. Это наиболее «ручной» режим, требующий от пилота постоянного контроля высоты. Подходит для опытных пилотов.

Какое положение тумблера даёт какое значение, зависит от настроек пульта (в том числе от реверса канала). Перед первым полётом проверьте это на экране пульта (**Display**) или в мониторе сигнала симулятора (см. ниже), а соответствие режимов — в документации Геоскан для вашей прошивки.

> **Внимание**
>
> **Физика полета:** Для определения высоты полетный контроллер использует барометр, измеряющий атмосферное давление. Помните, что резкие изменения давления (например, открытие окна в помещении или сквозняк) могут быть восприняты дроном как изменение высоты, что приведет к коррекции оборотов моторов.

Режимы работы пульта:

- **Ручное управление:** Вы полностью контролируете дрон с помощью стиков. Режим полета выбирается переключателем SwC.
- **Навигация и GPS:** В режиме Navigation квадрокоптер управляется с пульта, но автоматически удерживает высоту и позицию, если отпустить стики.
- **Полет по программе:** Пионер выполняет загруженную миссию. Вы всегда можете переключить тумблер режимов (например, пощелкать SwC), чтобы перехватить управление и перевести дрон в ручной режим.

D.4 Управление

Пилотирование квадрокоптера требует координации и понимания того, как каждое движение стика влияет на движение аппарата в пространстве. Рассмотрим управление в режиме **Mode 2**.

D.4.1 Управление высотой (Газ / Throttle)

Левый стик, движение вверх-вниз.

- **Стик вверх:** Обороты моторов увеличиваются, подъемная сила растет, квадрокоптер набирает высоту.
- **Стик вниз:** Обороты уменьшаются, квадрокоптер снижается.

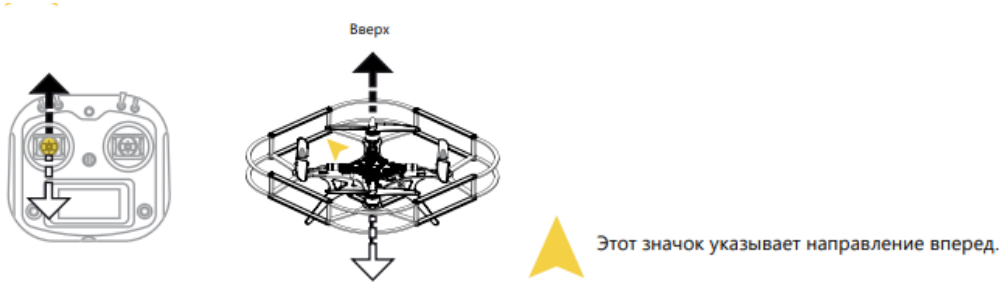


Рис. 57. Управление газом: набор высоты и снижение

> **Внимание**
>
> **Важно:** В режиме AltHold (удержание высоты) центр стика соответствует зависанию. Для взлета необходимо поднять стик выше середины.

D.4.2 Вращение по курсу (Рыскание / Yaw)

Левый стик, движение влево-вправо.

Отвечает за поворот квадрокоптера вокруг своей вертикальной оси. Это позволяет изменить направление «взгляда» дрона.

- **Стик влево:** Нос квадрокоптера поворачивается влево (против часовой стрелки).
- **Стик вправо:** Нос квадрокоптера поворачивается вправо (по часовой стрелке).

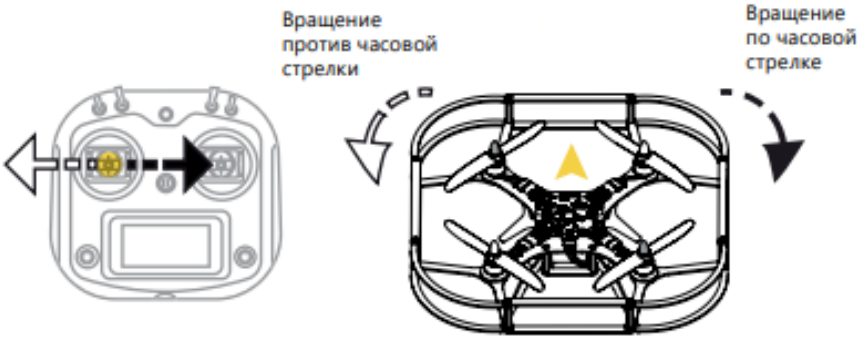


Рис. 58. Рыскание: вращение вокруг оси

D.4.3 Тангаж (Pitch) — Движение вперед/назад

Правый стик, движение вверх-вниз.

Изменяет угол наклона квадрокоптера в продольной плоскости, заставляя его двигаться горизонтально.

- **Стик вверх (от себя):** Нос опускается, дрон летит вперед.
- **Стик вниз (на себя):** Нос поднимается, дрон летит назад.

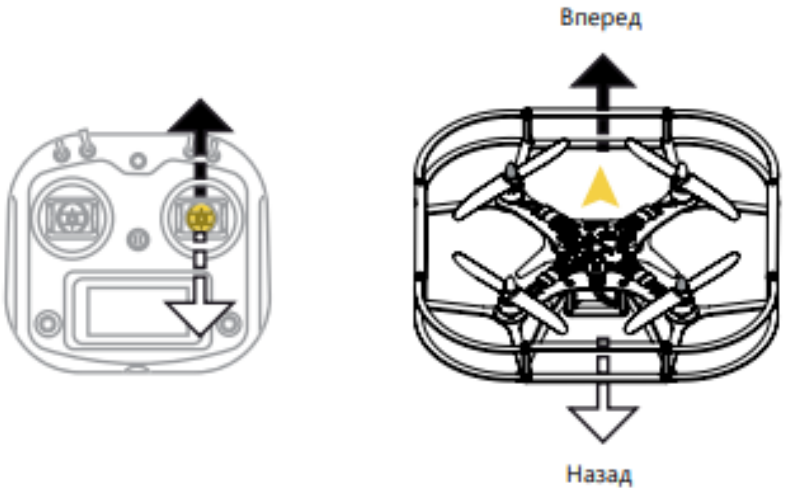


Рис. 59. Тангаж: движение вперед и назад

D.4.4 Крен (Roll) — Движение влево/вправо

Правый стик, движение влево-вправо.

Изменяет угол наклона квадрокоптера в поперечной плоскости, заставляя его смещаться вбок.

- **Стик влево:** Дрон наклоняется влево и летит боком влево.
- **Стик вправо:** Дрон наклоняется вправо и летит боком вправо.

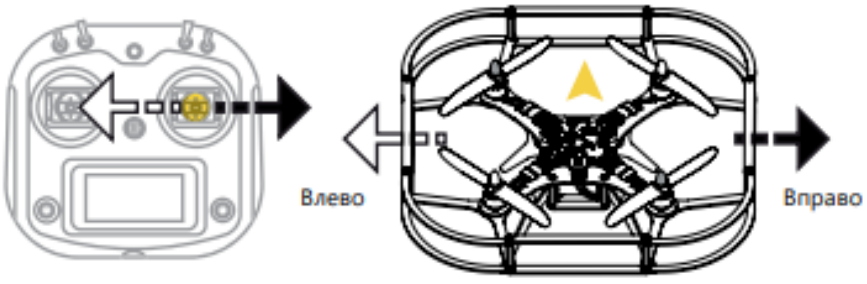


Рис. 60. Крен: движение боком

> **Подсказка**
>
> **Совет:** При обучении полетам держите хвост квадрокоптера (где установлен аккумулятор или светодиод) к себе. Так направления управления будут совпадать: стик вправо — дрон вправо. Если дрон развернется к вам носом, управление станет «зеркальным» (стик вправо — дрон влево с вашей точки зрения).

D.5 Пульт в симуляторе

Летать с пульта можно и в симуляторе: так удобно учиться пилотированию, не рискуя дроном. Пульт подключается к компьютеру по USB и должен определяться системой как джойстик (игровой контроллер); браузер читает его стандартным способом и показывает в панели «**Пульт**».

D.5.1 Подключение и раскладка

1. Подключите пульт по USB и откройте панель «**Пульт**». Пока пульт не найден, в заголовке написано «Не подключён».
2. Нажмите любую кнопку или подвигайте стик — пульт определится автоматически.
3. В поле «**Раскладка**» выберите режим стиков: «**Mode 1**», «**Mode 2**», «**Mode 3**» или «**Mode 4**». Для схемы этой главы (газ и рыскание слева) — «**Mode 2**».

D.5.2 Мастер настройки

Кнопка «**Мастер настройки**» назначает органы управления по очереди. На каждом шаге мастер просит:

1. подвигать стик газа вверх и вниз;
2. подвигать стик рыскания влево и вправо;
3. подвигать стик тангажа вверх и вниз;
4. подвигать стик крена влево и вправо;
5. пощёлкать тумблер режима по всем положениям (нужен трёхпозиционный тумблер, на FS-i6S это SwC);
6. пощёлкать тумблер взвода (Arm) по всем положениям;
7. нажать или пощёлкать канал магнита (захват груза).

Когда вход найден, мастер пишет «найден» и переходит к следующему шагу. Без мастера входы задают вручную во вкладке «**Каналы**»: у каждого канала есть кнопка «**Авто**» (нажмите и подвигайте нужный стик или тумблер) и флажок «**Реверс**», если ось работает наоборот.

D.5.3 Каналы симулятора

Канал	Назначение	Что делает в симуляторе
CH1	Крен (Roll)	наклон и движение вбок, центр 1500
CH2	Тангаж (Pitch)	движение вперёд-назад, центр 1500
CH3	Газ (Throttle)	высота, внизу 1000
CH4	Рыскание (Yaw)	поворот по курсу, центр 1500
CH5	Режим (Mode)	около 1000 — удержание позиции (Navigation), около 1500 — AltHold, около 2000 — Stabilize
CH6	Взвод (Arm)	в диапазоне срабатывания запускает моторы, при выходе из него — выключает
CH7	Магнит (Magnet)	в диапазоне срабатывания включает захват груза

Для тумблеров взвода и магнита задаётся «**Диапазон срабатывания**»; рядом показано текущее значение канала (LIVE), чтобы диапазон можно было подогнать под свой пульт. Тумблер режима при калибровке нужно перевести во все положения — по ним симулятор запомнит границы трёх режимов.

D.5.4 Калибровка и монитор сигнала

- «**Начать калибровку**» — проведите все стики до упора во все стороны и переключите все тумблеры: симулятор запишет полный ход каждого канала. «**Сбросить**» возвращает калибровку к исходной.
- Вкладка «**Монитор сигнала**» показывает в реальном времени первые 16 входов пульта (CH1–CH16). Кнопки «**Заморозить график**», «**Сбросить пики**» и флажок «**Показывать min/max**» помогают найти «дрожащую» ось или тумблер, который не доходит до крайнего значения.

Настройки пульта сохраняются в браузере и при следующем открытии сайта восстанавливаются.

> **Пульт и программы**
>
> Lua-программа читает каналы пульта функцией `Sensors.rc()`: восемь значений от −1 до 1, центр стика — 0; если пульт не подключён, все значения равны нулю. Так можно, например, запускать этап миссии тумблером. Тумблер взвода управляет моторами напрямую: переключение из положения «взведено» выключает моторы даже во время программы.

ЗАДАНИЕ D.1 · Пульт в симуляторе

Подключите пульт (или игровой контроллер) и пройдите мастер настройки. В мониторе сигнала проверьте, что каждый стик проходит от 1000 до 2000, а тумблер режима даёт три разных значения. Взлетите в режиме AltHold, зависните на 1 м и пролетите квадрат со стороной 2 м, затем переключитесь в Navigation и отпустите

стики.

Что должно получиться: все семь каналов найдены мастером; в AltHold высота держится при газе в центре, но дрон немного сносит; в Navigation с отпущенными стиками он висит на месте. Тумблер взвода выключает моторы после посадки.

ГЛАВА E

Методика тестирования и безопасность

E.1 Протокол безопасности полетов

E.1.1 1. Подготовка зоны (Pre-flight)

- Очистить зону 3×3 метра.
- Закрыть окна (ветер).
- Убрать животных и детей.
- Предупредить окружающих о запуске.
- Надеть защитные очки (для оператора).

E.1.2 2. Проверка дрона (Checklist)

- **Пропеллеры:** Целые, затянуты, вращаются свободно.
- **Аккумулятор:** Заряжен > 3.8В на банку, закреплен надежно.
- **Камера:** Линза чистая, шлейф не болтается.
- **Wi-Fi:** Сигнал стабильный (RSS > -60 dBm).
- **Пионер Мини 2:** пульт включён, тумблер `SWB` в нижнем положении (автопилот проверяет наличие пилота), программа содержит блок `finally` с посадкой, прерывание `Ctrl` + `C` проверено.

E.1.3 3. Экстренная ситуация

Если дрон ведет себя неадекватно:

1. Громко крикнуть «ПОСАДКА!».
2. Остановить программу и посадить дрон (кнопка LAND в наземной станции, если есть; у Мини 2 — `Ctrl` + `C` в терминале, он ведёт в `finally` и к посадке, или кнопка «**Стоп**» в Bricks); если он не садится — выключить моторы тумблером взвода на пульте или командой Disarm.
3. Если нет реакции — аккуратно поймать снизу (за батарею).
4. Отключить аккумулятор.

E.2 Тестирование в разных условиях

E.2.1 Погодные условия (Ветер)

Пионер — лёгкий учебный дрон, и ветер сносит его заметно. Ориентир: не летать на улице при ветре сильнее 3 м/с. Для теста в помещении используйте вентилятор:

- Включите вентилятор на минимальную скорость с расстояния 2 метра.
- Попробуйте удержать точку (режим Optical Flow).
- Если дрон сносит > 50 см — условия непригодны для точной навигации.

E.2.2 Освещение (Lux)

Камера Pioneer имеет низкий динамический диапазон.

- **Яркое солнце:** Пересвет (белое пятно). Маркеры не видны.
- **Сумерки:** Сильный шум матрицы. Маркеры «дрожат».
- **Мерцание ламп:** Люминесцентные лампы (50 Гц) могут давать полосы на видео.

E.3 Проверка в симуляторе перед полётом

Прежде чем запускать программу на настоящем дроне, прогоните её в симуляторе — так ловятся ошибки логики, а не железа.

- **Несколько полигонов.** Запустите программу на двух-трёх seed процедурного полигона (миссии Python № 55 и № 62) и после сброса сцены. Алгоритм не должен опираться на координаты, которые совпали случайно (миссия № 107).
- **Неуспешные ветки.** Проверьте исходы «метки нет» (несуществующий ID, как 249 в миссии № 108), «кадр не получен» (`None`) и «точка не достигнута за отведённое время». Каждый должен заканчиваться понятным сообщением и посадкой.
- **Ограничения.** Скорости ограничены, высота — с запасом над препятствиями, зона полёта задана границами.
- **Перенос.** Уберите расширения симулятора (`Camera(view="bottom")`), заново проверьте знаки поправок коротким импульсом, уменьшите скорости для первого полёта.
- **Мини 2.** Выберите модель «**Мини 2**» и проверьте программу на `pioneer_sdk2`. Симулятор упрощает или не поддерживает параметры автопилота, GPS, запись видео и фото, а нейросеть заменяет имитацией (главы 16 и 17): такие места проверяйте только на дроне и в первую очередь без пропеллеров.

ГЛАВА F

Готовые конфигурации (Templates)

F.1 Файл конфигурации camera_params.yaml

Те же параметры камеры, что сохраняют скрипты калибровки в `data.yml`, но в формате для сторонних инструментов: матрица — `camera_matrix`, дисторсия — `distortion_coefficients`. Примеры этой книги читают `data.yml`.

camera_params.yaml

```yaml
%YAML:1.0
image_width: 640
image_height: 480
camera_matrix: !!opencv-matrix
   rows: 3
   cols: 3
   dt: d
   data: [ 600.0, 0., 320.0, 0., 600.0, 240.0, 0., 0., 1. ]
distortion_coefficients: !!opencv-matrix
   rows: 1
   cols: 5
   dt: d
   data: [ 0.1, -0.2, 0., 0., 0. ]
```

F.2 Шаблон скрипта полета (main.py)

Шаблон гарантирует посадку при любой ошибке: `land()` и пауза до касания выполняются в блоке `finally`, и только после них — `disarm()`. Выключить моторы сразу после `land()` значило бы уронить дрон с высоты.

Базовый шаблон

```python
import time
from pioneer_sdk import Pioneer


def wait_point(drone, timeout=30):
    """Ждать прибытия в точку не дольше timeout секунд."""
    deadline = time.time() + timeout
    while not drone.point_reached():
        if time.time() > deadline:
            raise RuntimeError("Точка не достигнута за отведённое время")
        time.sleep(0.1)
```

```
def main():
    drone = Pioneer()
    in_air = False
    try:
        drone.arm()
        time.sleep(1)       # даём автопилоту перейти в режим готовности
        drone.takeoff()
        in_air = True
        time.sleep(3)       # взлёт занимает пару секунд

        # ... Ваш код: например, одна точка маршрута
        drone.go_to_local_point(x=0, y=0, z=1, yaw=0)
        wait_point(drone)

    except Exception as e:
        print(f"Ошибка: {e}")
    finally:
        if in_air:
            drone.land()
            time.sleep(5)   # disarm до касания земли выключил бы моторы в
            ↪  воздухе
        drone.disarm()
        drone.close_connection()


if __name__ == "__main__":
    main()
```

ГЛАВА

Список литературы

1. *Геоскан*. Пионер Мини 2: библиотека Pioneer-SDK2. — URL: https://docs.geoscan.ru/pioneer/uavs/mini/2/pio-sdk2/ (дата обр. 08.11.2026).
2. *Геоскан*. Пионер Мини 2: библиотека Pioneer-RKNN. — URL: https://docs.geoscan.ru/pioneer/uavs/mini/2/pio-rknn/ (дата обр. 08.11.2026).
3. *Геоскан*. Пионер Мини 2: Pioneer Bricks. — URL: https://docs.geoscan.ru/pioneer/uavs/mini/2/bricks/ (дата обр. 08.11.2026).
4. Automatic generation and detection of highly reliable fiducial markers under occlusion / S. Garrido-Jurado [et al.] // Pattern Recognition. — 2014. — Vol. 47, no. 6. — P. 2280–2292. — DOI: 10.1016/j.patcog.2014.01.005.
5. *Olson E.* AprilTag: A robust and flexible visual fiducial system // Proceedings of the IEEE International Conference on Robotics and Automation (ICRA). — 2011. — P. 3400–3407. — DOI: 10.1109/ICRA.2011.5979561.
6. Detection of ArUco Markers / OpenCV. — URL: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html (visited on 11/06/2026).
7. *Zhang Z.* A flexible new technique for camera calibration // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2000. — Vol. 22, no. 11. — P. 1330–1334. — DOI: 10.1109/34.888718.
8. Camera calibration with OpenCV. — 2026. — URL: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html (visited on 03/04/2026).
9. *Kalman R. E.* A New Approach to Linear Filtering and Prediction Problems // Journal of Basic Engineering. — 1960. — Vol. 82, no. 1. — P. 35–45. — DOI: 10.1115/1.3662552.
10. *Геоскан*. Пионер: программирование на Lua. Описание методов API. — URL: https://docs.geoscan.ru/pioneer/programming/lua/lua.html (дата обр. 06.11.2026).
11. MediaPipe Pose. — 2026. — URL: https://developers.google.com/mediapipe/solutions/vision/pose_landmarker (visited on 03/04/2026).
12. *Slatkin B.* Effective Python : 90 Specific Ways to Write Better Python. — 2nd ed. — Addison-Wesley Professional, 2019.
13. *Геоскан*. Пионер: библиотека pioneer_sdk для Python. — URL: https://docs.geoscan.ru/pioneer/programming/python/pio-sdk.html (дата обр. 06.11.2026).